Catapult HLS User View Scheduling Rules

Stuart Swan, Platform Architect, Siemens EDA

31 October 2026

Abstract

This document defines a user-visible scheduling contract and modeling methodology for high-level synthesis (HLS). Its practical objective is to support a proof-backed flow in which much of the system-level functional verification and debug is performed on the pre-HLS SystemC model, for designs and implementations that satisfy the stated applicability and conformance conditions. The contract treats message-passing operations, signal I/O, and explicit synchronization through a small set of uniform rules while still allowing important implementation transformations such as loop pipelining, added FSM states, permitted memory-access reordering, direct-input late sampling, and bounded or more complex channel buffering.

The main functional guarantee is that every covered RTL behavior has a matching behavior in the source model configured with the implementation's relevant buffering and communication structure. It does not require identical cycle timing or internal state. Local correctness proofs for processes and channels, together with integration and environment checks, establish this correspondence. For supported design classes, a generic theorem combines those local proofs into the system result. One reusable proof can cover a declared class of stimuli, but the desired source properties must still hold for all admitted source behaviors; ordinary passing simulations alone do not establish that coverage. This correspondence does not guarantee that RTL reproduces any arbitrarily selected source simulation.

The separate completion guarantee says that the required outputs eventually arrive with the expected values and order at each interface. Start with a finite source run that accepts those outputs, normally using the conservative serial mode or the qualified grouped mode for operations that must complete together. Where the channel structure qualifies, prefer lazy serial simulation of the implementation-bound elaborated model, retaining the qualified grouped mode for atomic operations. Lazy movement itself introduces no per-stimulus RTL run or probe. The run must pass the applicable dependency and resource checks, and the implementation must meet the matching, control and progress requirements. A passing source simulation alone is not enough. These requirements cover the work and decisions needed to reach the outputs, including work that was not taken on a branch. They also keep a proof of request readiness from assuming the very data arrival it is meant to establish. The result does not require unrelated trailing work to finish, promise a cycle bound, or rule out every internal deadlock. Appendix K.CT gives the completion conditions; the Core transfer definitions and Appendix N distinguish an actual handshake from the progress needed to reach it.
The proofs do not by themselves certify a particular HLS invocation or RTL instance. The applicable source well-formedness, scheduling, channel realization and capacity, reset, environment, progress, and implementation-mapping conditions must be established by the tool or verification flow through documentation, checked correspondence records, static checks, targeted RTL/interface checks, formal verification, or equivalent evidence. Subject to those conditions, the document provides a Catapult-compatible, tool-neutral basis for industrial HLS verification and for possible scheduling-contract standardization.

Contents

Abstract 1
Introduction 12
Formal Guarantees at a Glance 13
Document Goals 16
Background 16
An Analogy from RTL Synthesis 18
Catapult HLS Status Concerning Rules in this Document 19
Terms Used in this Document 19
Classes of Operations Involved in Scheduling Rules 20
Basic Conceptual Model 21
Pipelined Loops 22
  Synchronization boundary for pipelined loops 23
Direct Inputs 23
Additional Options for Scheduling Message-passing Interfaces 25
Scheduling of Array Accesses 26
  Definitions: Issue vs. Commit 27
Additional States Added by HLS Synthesis 28
Avoiding Pre-HLS and Post-HLS Simulation Mismatches 29
Returning to the Analogy from RTL Synthesis 33
Summary 34
Appendix A – Factory Analogy 35
Appendix B – Formal Presentation of Pipelined Loops 38
  Modeling convention for pipelined-loop overlap. 38
  B-faithfulness obligation 38
  Multi-input Pop points and coupler policies 39
  Practical note (outer Σ_DUT projection and proof-internal Σ) 39
  Blocked-frontier restrictions and permission to drain 39
    Terminology (pending blocking frontier / Front_P(σ)) 39
    General Core.16 blocked-frontier and drain-only obligation 41
  B.BoundaryBinding — process sites, channel storage and physical movement 41
  B.OF Ordered automatic-flush progress 44
  B.CT Completion progress profiles 46

Appendix C – Issue/Commit Terminology and Linkage 51
Scope / shared definitions 51
Issue/Commit linkage (well-formedness conditions) 52
Appendix D – Modeling Latency-Sensitive Protocols 54
Appendix E – One-Way Handshake Protocols 55
Appendix F - Modeling Guidelines and Design Constraints 55
Design Constraints for Applying the Formal Guarantees 56
Design constraints for the safety and correspondence guarantee 56
Additional requirements for required-workload completion 58
Practical modeling checks for completion 59
Guide to the Formal Appendices 62
Recommended reading paths 62
Appendix map 63
Normative Formal Core for Appendices G–K 65
Prescribed reference model 73
Definition Core.1 (Prescribed source reference model Sys_ref). 73
Definition Core.ElabProj (Elaboration projection/accounting map Π_elab). 73
Remark Core.1 (Interpretation at explicit abstract channels). 74
Remark Core.1a (Operational Sys_ref transition system and issue policies). 74
Observable alphabet and executions 75
Definition Core.2 (Observable alphabet Σ). 75
Definition Core.3 (ε-steps and compact trace notation). 75
Definition Core.ConstructiveOriginCarrier (actual edge origins and designation). 75
Definition Core.RegisteredMessageBatch (registered message whole-edge completion profile) 76
Definition Core.RegisteredAnchorBatch (wait, paired-sync and anchored-signal completion profile). 77
Definition Core.RegisteredFactBatch (complete fact inventory for constructive completion). 78
Definition Core.AnchorConfigObservation (constructive-route configuration observation option). 80
Definition Core.RegisteredResetBatch (edge-sampled reset extension of constructive Batch profiles). 81
Definition Core.RegisteredR2CBatch (edge-observed R2C extension). 82
Definition Core.ExecutionCarrier (legal execution-history carrier and exact-prefix provenance). 82
Definition Core.4 (ε-quiescent cutpoint). 83
Definition Core.Settle (L2 settled boundary-evaluation point). 83
Definition Core.CycleState (model-indexed cycle-entry theorem state) 84

Definition Core.ResetEpochKey and Core.ResetEpochAdvance (canonical reset-epoch provenance). 84
Lemma Core.ResetEpochAdvance-Fun (one-cycle reset-epoch update is functional). 84
Definition Core.QSync (qualified synchronous same-cycle settling model). 85
Theorem Core.SourceL2Settle (finite operational source settling). 86
Lemma Core.SettleRootAgreement (corresponding roots give corresponding settled boundary signals). 87
Theorem Core.QSync-Post (finite operational RTL settling). 87
Definition Core.CycleResult, CompleteCycle, and SilentCycle. 87
Definition Core.SourceCheckpointDomain (independent source property checkpoints). 88
Definition Core.SourceCheckpointProfile (history-preserving checkpoint representation). 89
Definition Core.DeclaredTerminal (modeled terminal-state predicate) 89
Definition Core.EnvStep (time-aware environment evolution). 89
Definition Core.CycleClosure (source full-cycle qualification record). 90
Lemma Core.CycleChoiceExt (phase-legal source-cycle selection extends to a CycleResult). 90
Definition Core.5 (Per-process execution-call universe, frontier-item universe, observable projection, and source order). 91
Definition Core.6 (Done_P, Remain_P, and NextFront_P relative to the fixed witness). 92
Definition Core.7 (Message-operation slice of the next frontier). 92
Definition Core.7a (Blocking message-operation slice of the next frontier). 93
Definition Core.7b (Synchronization-call slice and typed synchronization carrier). 93
Definition Core.7c (Synchronization-carrier well-formedness). 93
Definition Core.SyncProtocol (physical synchronization lifecycle and sampled completion). 94
Definition Core.7d (Source synchronization in-flight completion unit SyncFire). 95
Definition Core.7e (Explicit epoch-gate administrative action). 96
Clarification Core.5a (ordinary source order versus elaborated partial order). 96
Lemma Core.OrdFrontSingleton (ordinary-source frontier uniqueness). 96
Operational interpretation of Sys_ref and source issue policies 97
Definition Core.SourceChoiceBinding (prospective source semantics and choice resolution). 97
Definition Core.PreparationEvidence (validation of independently defined preparation). 98
Definition Core.ReachPrep (overlap-aware source reach/preparation semantics). 98
Shared proof-boundary, poll and implementation issue conventions 99
Definition Core.NBDecisionSnapshot (operational non-blocking decision snapshot). 100
Definition Core.NBDecision (non-blocking success and failure law). 100
Definition Core.DeferredNBHole (carried non-blocking decision obligation). 101
Definition Core.RegSeq (registered sequential-interface semantics) 101

Definition Core.PolicyL (liberal, witness-relative issue). 102
Definition Core.LLegal (Policy-L step legality and fixed-instance binding). 102
Definition Core.SelectLatitude (finite-prefix issue and abstract-action latitude). 102
Definition Core.Phase (Policy-L cycle phases and settled-frontier trigger). 103
Definition Core.PolicyS (conservative serial-blocking issue). 104
Remark Core.PolicyS-Sim (Relation to practical pre-HLS simulation modes). 104
Shared evidence and finite-prefix agreement 105
Definition Core.ExtensionConsumerCoverage (open-class consumer completeness). 106
Definition Core.EvidenceRealization (proof families and executable evidence). 106
Commit, issue, and boundary requests 108
Definition Core.8 (Commit). 108
Definition Core.TransferLaw (mandatory settled message transfer). 108
Definition Core.LogicalPopPayload (causal payload attribution for internal logical Pops) 110
Definition Core.9 (Issue). 111
Definition Core.IssueFair (weak fairness of source issue). 111
Definition Core.10 (Endpoint-owned boundary request). 111
Definition Core.IC (Issue/Commit linkage invariant). 112
Definition Core.SyncFence (synchronization-completion legality). 113
Definition Core.SyncFenceOrder (E5 canonical predecessor and carrier conformance). 113
Remark Core.10a (Non-blocking polls). 114
Channel enablement 114
Definition Core.11 (ChannelEnabled / ChannelDisabled). 114
Definition Core.12 (Buffered channels). 115
Definition Core.13 (Rendezvous channels). 115
Pending blocking frontier 116
Definition Core.14 (Pending blocking operations). 116
Definition Core.15 (Pending blocking frontier Front_P(σ)) 116
Blocked frontier and drain-only discipline 116
Definition Core.16 (Drain-only blocked-frontier discipline). 116
Definition Core.Drain (activated drain-discipline qualification). 118
Definition Core.InFlightStep (already-admitted proof-model progress action). 119
Definition Core.InFlightDrainWF (finite admitted population and well-founded progress). 119
Definition Core.17 (System-level observable compatibility). 119
Shared-memory and reset side conditions 120
Additional Semantic Assumptions (B-rules) 122

Appendix G – Trace Compatibility Rules 126
Formalization of the Trace Compatibility Rules in Appendix G 129
Dynamic source-order convention 131
Conceptual Model for a Single Process 132
R1 (Program order of S). 132
R2 (Location of R/W for sequential processes) 133
R2C (Combinational signal IO at the ε-fixed point). 133
R2C unit-generation convention 133
Definition Core.R2CResetLedger (reset ownership and emission restart). 134
R3 (Isolation around S). 135
R4 (Safe message issue ordering). 136
R5 (Channel capacity semantics; Sys vs. Sys_B). 136
Operational semantics location 137
Trace Compatibility Relation 137
R2C unit matching/index convention 138
E1 (Per-process synchronization-order preservation). 139
E2 (Signal-visibility preservation). 139
E3 (Safe message issue ordering). (Post-HLS preservation of R4). 140
E4 (Per-channel channel semantics: rendezvous and bounded FIFO). 140
E5 (Messages cannot cross their immediate synchronization boundary). 141
Allowed Transformations and Why They Preserve ≈ 142
Proof Structure 147
Causal Dependency Between Processes 147
Formal Definition of Causal Dependency 147
Definition: Causal Dependency 149
System-Level Constructive Correspondence and Safety Transfer 149
Practical Implication (“No Surprises” Guarantee) 150
Latency-sensitive testbench caveat 150
Appendix H – Possible Criticisms 150
1. Designer-Managed Shared-Memory Synchronization 150
2. Latency-Sensitive Signal Alignment (“Snooping”) 151
3. By Default, Matchlib Connections Model a Skidbuffer inside In<> Ports 151
Appendix I – Reserved 152
Appendix J – Constructive System-Level Correspondence 152
J.0 Shared observable units and required order 153

Definition J.RequiredOrderBasis (route-independent unit and generator convention). 153
Definition J.Reach (finite reachability sets Reach_post / Reach_ref). 154
Definition J.GeneratorCompleteness (prospective normative rule inventory). 154
Definition J.SourceGeneratorRealization (reusable source-rule order evidence). 154
Lemma J.SourceOrderWF (source completion rules establish joint strictness). 155
Definition J.RequiredOrderWF (prefix closure and independent strict-order evidence). 155
Lemma J.CycleRankOrderWF (lexicographic evidence for joint acyclicity). 156
Example J.StandardCycleRank (checked message, anchor, R2C and reset classes). 156
Definition J.OuterCanonHist (outer canonical history projection). 157
Lemma J.ProjGroupOrderAcyclic (qualified outer-group contraction is acyclic). 159
Lemma J.RequiredPredCycleOrder (required generators with strict-cycle separation). 159
Lemma J.RequiredPredPrefix (legal execution prefixes are closed under required predecessors). 159
Lemma J.OuterCanon-Ideal (outer projection preserves required-prefix ideals). 159
Definition J.RefProjCover (selected-reference-run observable coverage) 160
Definition J.IdealPrefixClosed (ideal-prefix closure for safety predicates) 161
Lemma J.SourceCheckpointRepresentative (equal-history source checkpoint). 161
Definition J.ReachableHistCov (reachable-source outer-history coverage). 161
Lemma J.SafeTransfer-HistCov (semantic source-side safety transfer). 162
Definition J.ReachableHistCovOn (source-domain history coverage). 162
Definition J.RefProjCoverOn (covering execution for a declared source domain). 162
Theorem J.SafeTransfer-HistCovOn (domain-specific source-property transfer). 163
Definition J.SourceSettleRealization (safety-only operational settling). 163
Lemma J.SourceSettleExists (legal activation snapshot). 164
Definition J.LocalOriginContracts (C0–C5; constructive registered-message correspondence). 164
Lemma J.LocalBoundaryCompose (component certificates construct one source boundary segment). 167
Lemma J.SourceOrderReflection (source strictness precedes canonical equality). 167
Lemma J.LocalOriginCanonExt (generator binding establishes equal extended canonical histories). 167
Theorem J.ConstructiveEdgeInduction (parameterized origin coverage). 168
Theorem J.LG-Origin (local-to-global constructive origin coverage). 168
Lemma J.BatchPrefixCoverage (origin coverage lifts to exact Batch finite prefixes). 169
Corollary J.LG-Batch-Safe (constructive exact-prefix safety for the Batch profile). 169
Constructive source-property discharge and monitor contract (normative). 169
Definition J.StableDirectInputContracts (anchored stable-input lifting; SD0–SD4). 170

Lemma J.StableRead-Reduction (anchored substitution preserves message execution). 172
Theorem J.StableRead-Batch (stable direct-input finite-prefix message coverage). 172
Definition J.LocalAnchorContracts (A0–A6; local correspondence with observable anchors)..... 172
Lemma J.AnchorInventory (causal complete anchor groups). 174
Lemma J.AnchorBoundaryCompose (joint message, signal and synchronization realization)..... 174
Theorem J.LG-AnchorBatch (exact finite-prefix coverage for observable waits and signals). 175
Corollary J.LG-AnchorBatch-Safe (universal source safety transfer). 175
Lemma J.AnchorDirectInputLift (retained direct inputs compose with observable anchors)..... 176
Example J.AnchorBatchPair (symbolic local discharge with changing inputs and message stalls). . 176
Definition J.ConfigValueContracts (CVG0–CVG3; transaction-bound configuration uses). 177
Lemma J.ConfigInventoryAndValues (causal records and local substitution). 178
Theorem J.LG-ConfigBatch (configuration-bearing finite-prefix safety coverage). 178
Lemma J.ConfigWholeGroupProjection (message projection with contracted anchor paths). 179
Corollary J.ConfigBatch-MessageSafe (message safety across configuration intervals). 179
Definition J.LocalResetContracts (RB0–RB5; reset extension of local constructive evidence)..... 180
Lemma J.ResetConfigurationEpoch (configuration rebinding across affected reset epochs). 181
Lemma J.ResetLocalCompose (one reset-containing edge). 181
Theorem J.LG-ResetBatch (finite-prefix constructive coverage with dynamic reset). 182
Corollary J.LG-ResetBatch-Safe (reset-aware source safety)..... 182
Example J.BlockedPipelineReset (symbolic reset-priority instance). 182
Definition J.LocalR2CContracts (RC0–RC6 for edge observation)..... 183
Lemma J.R2CInventoryAndCompose (settlement, emission and edge extension). 184
Theorem J.LG-R2CBatch (finite-prefix constructive coverage with edge-observed R2C)..... 185
Corollary J.LG-R2CBatch-Safe (combined constructive safety transfer)..... 185
Example J.R2CEdgeSample (registered producer and retained consumer). 185
Proposition J.ConstructiveCombinedProfile (application and closeout). 186
J.ProcessEnvClass — Process-environment contracts 186
Definition J.UniformConstructiveEvidence (stimulus-class local constructive evidence). 188
Theorem J.UniformConstructiveCoverage (class-wide semantic history coverage). 189
Corollary J.ConstructiveClassSafe (stimulus-class safety on the constructive route)..... 190
Ordinary constructive locality 190
Definition J.OrdinaryConstructiveProfile (OM/OA structural scope). 191
Definition J.OrdinaryLocalEvidence (restricted vocabulary and LP0–LP5). 192
Definition J.OrdinaryGlue (G1–G7 and integration checks)..... 194
Definition J.OrdinarySettlingEvidence (QF/QSafe alternatives and QM matching). 195

Lemma J.OrdinarySettleComposition (reuse of qualified source settling). 196
Definition J.OrdinaryRelation (assembled component relation). 196
Lemma J.OrdinaryBoundaryCompose (generic ordinary edge realization). 196
Ordinary FIFO residue calculation. 197
Other ordinary component residues. 197
Lemma J.OrdinaryUpdatePrepare (product preservation and legal preparation). 198
Lemma J.OrdinaryCanonicalExtension (ordinary generators and history equality). 198
Theorem J.OrdinaryConstructiveComposition (uniform OM/OA locality and Batch coverage). 199
J.OrdinaryEvidenceOwners (derived premise-discharge map) 200
Scope and status of J.OrdinaryConstructiveComposition 201
Definition J.ElaboratedLocalityProfile (ELM/ELA structural scope). 201
Definition J.ElaboratedTemplateEvidence (primitive and integration evidence). 203
Lemma J.ElaboratedFIFOResidue (owned queue updates). 204
Worked template bindings for elaborated locality. 204
Definition J.ElaboratedNBDecisionBinding (operational outcomes and complete edge inventory). 205
Lemma J.ElaboratedBoundaryCompose (local realization and preparation). 205
Lemma J.ElaboratedCanonicalExtension (closed generator composition). 206
Theorem J.ElaboratedConstructiveComposition (ELM/ELA locality and Batch coverage). 207
Definition J.ConstructionAction and operational footprints 208
Definition J.ProofUnitConstructionSupport (proof-unit to construction-order support) 208
Lemma J.BufSameCycle (same-cycle buffered Push/Pop sequentialization) 209
J.2c Internal-fact correspondence for completion consumers 209
Definition J.ConstructiveFactEvidence (CF0–CF3; local realization of the required fact signature). 210
Theorem J.ConstructiveFactCorr (constructive internal-fact correspondence). 211
Corollary J.RegisteredConstructiveFactCorr (message and anchor specialization). 211
Lemma J.LogicalPayloadRefinement (causal representation and fact attribution) 213
J.3 Causal Dependency and What System-Level Compatibility Guarantees (Informative) 214
Appendix K – Required-Workload Completion 215
K.CT Required-workload completion 215
K.CT.Instance and acceptance contract 215
K.CT.Witness and commit units 217
K.CT.SerialWitness default pre-HLS workflow 218
K.CT.GroupSerialWitness qualified atomic-group issue 219
K.CT.SyncRegion overview of synchronization and epoch capacity 220

K.CT.Exposure and service 220
K.CT.Completion theorem 221
K.CT.Default qualification 222
K.CT Standard discharge lemmas 223
K.CT.LazySerial independent staging completion alternative 230
K.CT Applicability and migration 240
Appendix L – Snooping Introduction, Implementation and Concerns 242
Witness compatibility and non-blocking choices 242
Snooping Introduction 242
Simplified Snooping Approaches 243
Snooping Implementation 243
Wrapper Realization Requirements (Recorded-Observation AND) 245
Snooping Concerns 249
Stress Testing Pre-HLS Models for Latency Robustness 254
Appendix M – Formal Results Overview 256
M.1 Purpose and status 256
M.2 Models and observation boundary 256
M.3 Constructive source-history correspondence 257
M.4 Full facts and required-workload completion 257
M.7 Implementation qualification and evidence 257
M.7a Side-condition discharge record 261
Parameter binding, evidence reuse, non-trigger guards, and invalidation 261
Appendix N – Scheduler and Environment Fairness Assumptions 280
Priority arbiter example: fair vs unfair priority 281
Patterns that typically satisfy the fairness assumptions 283
Patterns that tend to violate the fairness assumptions 283
Appendix O – Support for Multiple Clock Domains (Informative Sketch) 284
Appendix P – AI Discussion of Alternative Formal Frameworks 284
Appendix Q – Multiple Input Pipelines Back-Annotation 285
Lean-oriented clarification / proof-parameter convention 289
Definition Q.1 (Elaboration package for Sys_B^elab). 289
Q.CT Completion qualification of elaborated components 294
Appendix R – Compact Derived Formal Core and Mechanization View 307
R.1 Retained proof structure 307
R.2 Completion dependency chain 307

R.3 Scope and observation 308
R.4 Dependency map and safety branches 308
R.5 Mechanization work packages 309
R.6 Retained-route audit checklist 309
Appendix S – Lean Proof Strategy Document 310
S.1 Mechanization targets 310
S.2 Completion targets and remaining qualification 310
S.3 Proof boundary checks 312
S.4 Declaration and proof order 312
S.5 Carrier, order and completion safeguards 313
S.6 Foundational dependency inventory 313
S.7 Reuse, extraction and extension review 314
Appendix T – Scheduling-Specific Obligations Beyond Standard Simulation Theory 314
T.1 Purpose and Nature of the Formal Result 314
T.2 Scheduling-specific obligations 315
T.2.1 Initial states and reset epochs 315
T.2.2 Dynamic operation identity and legal choices 315
T.2.3 Old-state boundary actions and atomic units 315
T.2.4 Boundary and memory faithfulness 315
T.2.5 Projection and required order 316
T.3 Why local composition needs explicit evidence 316
T.4 Workload progress 316
T.5 Interpretation limits 317
T.6 Reusing generic libraries and checking evidence 317
Appendix U – Revision Record 318

Introduction

HLS loses much of its practical advantage when design teams must return to machine-generated RTL as the primary level for functional verification and debug. RTL verification can be one of the most costly and time-consuming parts of a hardware development flow, and generated RTL may differ substantially in structure and timing from the source model. A useful HLS methodology should therefore let designers and verification engineers reason primarily at the source level while retaining a precise way to qualify the resulting implementation.

This document presents a user-visible scheduling contract intended to provide "no surprises" at process and interface boundaries. It reduces interface scheduling to uniform rules for message-passing operations, signal I/O, and explicit synchronization. A common verification intent can then be reused across the prescribed pre-HLS source reference model and the post-HLS implementation, without requiring the implementation to preserve every source-level cycle or internal state.

The prescribed pre-HLS reference model is the source design together with its modeled testbench or environment, configured to represent the implementation's relevant channel capacity and communication structure. Depending on the design, this may require back-annotated capacities, explicit arbitration or coupling models, and qualified handling of latency-sensitive choices. The original unconfigured SystemC simulation is not automatically this reference model. The compared behavior is the behavior visible at the declared logical interfaces; internal implementation details may differ.

Subject to the conditions stated in the main text and formal appendices, the methodology accommodates sequential and combinational logic, pipelined designs, shared and external memories through the stated mapping conditions, permitted RAM-access reordering, direct-input late sampling, finite channel buffering, and localized latency-sensitive protocols. These capabilities reflect requirements encountered across a broad range of industrial HLS designs and are important for achieving competitive hardware quality of results (QOR).

The methodology builds on established verification practices such as SystemVerilog UVM, constrained-random stimulus, and functional and code coverage. The system under test is modeled and verified in SystemC, while implementation-level verification focuses on establishing that the synthesized or otherwise implemented RTL falls within the scope of the applicable theorem instance. When the selected abstract channel boundaries and back-annotated capacities faithfully represent the RTL storage or credit at those boundaries, the pre-HLS model can also be used to assess capacity and throughput effects rather than deferring those questions entirely to RTL simulation.

The formal architecture separates safety from liveness. Safety is largely compositional: design-specific evidence is supplied for individual processes, explicit channel boundaries, named atomic interactions, the environment, and reset behavior, and a generic theorem composes that evidence into a system-level observable trace-compatibility result. Required-workload completion additionally requires a finite accepting source witness, internal-fact correspondence and value agreement, sufficient resources, persistent requests, implementation progress, arbitration service where needed, and the stated environment premises. Once ready and valid are both high at the sampling edge, the transfer occurs; the progress assumptions concern getting to that point. See Core.TransferLaw, Core.SyncProtocol and Appendix N.

The user-visible scheduling rules are intentionally much simpler than the formal development that supports them. The additional formal machinery is needed not because the basic rules are individually complex, but because the guarantees must remain valid under pipelining, buffering, added implementation states, memory transformations, scheduler variation, reset and environment behavior,

and the composition of many interacting processes. Thus, the complexity of the formal appendices reflects the breadth and rigor of the guarantees rather than complexity that an HW architect or HLS user is expected to manage.

The most important default deadlock-avoidance rule can be stated plainly. HLS may overlap or shift message-passing operations in time, but it may not reverse the dynamic source order of message-passing calls under the default scheduling contract. It therefore cannot introduce a new deadlock merely by reversing the order of source calls. This rule is not a complete deadlock theorem: overlapped loop iterations, eager issue, finite capacities, and environment behavior can still affect deadlock, and those effects are addressed by the separate liveness machinery, whose default practical route checks a finite conservative pre-HLS execution, preferably in lazy mode where eligible, through acceptance of the required outputs and combines it with implementation progress and the applicable service assumptions (Appendix K.CT). The conclusion is acceptance of the required workload, not freedom from all internal wait cycles.

The guarantees remain conditional. The mathematical proofs do not establish that a particular HLS tool invocation or generated RTL instance satisfies the source-side and implementation-side obligations. Those obligations must be discharged by the tool or flow through documentation, checked proof records, lint or static analysis, targeted RTL/interface checks, formal verification, or another appropriate validation method. The same framework can also be applied to hand-written, modified, or otherwise generated RTL when equivalent evidence is supplied.

Because the rules are expressed as a user-visible, tool-neutral scheduling contract rather than as the internal behavior of a particular HLS implementation, they can also serve as a starting point for standardization, for example within the Accellera Synthesis Working Group. The next section summarizes the guarantees without exposing the internal proof API. A separate Guide to the Formal Appendices appears immediately before Appendix G, Appendix F provides user-oriented modeling guidelines, and Appendix M provides a more detailed Formal Results Overview.

Formal Guarantees at a Glance

The statements below provide a user-level orientation. The formal definitions, assumptions, scope conditions, and evidence requirements appear in the appendices.

Implementation safety. Within the stated scheduling, channel, environment and reset conditions, the RTL does not introduce covered observable functional behavior that is absent from the prescribed pre-HLS reference model. This allows most functional verification and debug to remain at the source level. The reference model must represent the implementation's relevant buffering and communication structure; the original SystemC simulation with unadjusted channel depths is not automatically that model. Properties must concern the compared interface behavior, rather than cycle timing or internal details that the comparison deliberately permits to differ.

Process testbenches and stimulus classes. For verification across many stimuli, establish one reusable correspondence proof for the design, its testbench runtime and a declared class of inputs and choices. The constructive route supports this class-wide proof directly; it is not limited to separate proofs for individual stimuli. The testbench is normally included with the design in the compared system, so responses that depend on earlier DUT results are covered by the same joint argument. Once the conditions are established uniformly, each admitted stimulus needs only exact instance binding and any remaining membership or applicability checks. Establishing source safety throughout the class remains a separate obligation; a passing sample campaign or a shared testbench program alone does not establish it. Reuse can reduce total effort across a large, stable campaign, but there is no unconditional cost advantage. (J.ConstructiveClassSafe; Appendix M.7.) Two qualified options can narrow the separate

source-property proof, supplied for each stimulus when a class is claimed. With Core.SourceCheckpointProfile, checking source histories at independently declared initialization or completed-cycle checkpoints is sufficient only after J.SourceCheckpointRepresentative and the stated monitor obligations are proved; cycle-granular simulation alone does not cover all source choices. Alternatively, one selected complete source execution can represent the source domain only with proof that every relevant source history is a finite ideal of its history (J.RefProjCover/J.RefProjCoverOn) and that the property holds on every finite ideal required by the selected transfer clause. Such an ideal retains whole completed units and every required predecessor of each retained unit; independent or later units may be omitted only consistently with that rule. Passing one selected test is not that proof. Neither option replaces implementation-wide constructive correspondence.

Constructive safety for supported observations. The constructive correspondence proof derives the system-level safety guarantee from local evidence about process computations, requests, stored values, channels and their interactions. A generic composition theorem applies to the qualified classes defined later; other classes need their stated extension proofs. Each application requires design-specific local evidence and an independent source-property proof covering every behavior left unresolved in the declared source model. The selected-source-history option retains its separate coverage and property premises. For completion, the same proof architecture must additionally account for all internal operations and control/value facts consumed by the completion argument. Safety at the external interfaces alone does not provide that stronger correspondence. The proof does not automatically cover every transformation or observation boundary.

Supported single-clock designs may combine pipelines, waits and paired synchronization, ordinary signals, configuration inputs, clock-edge reset behavior and combinational signals sampled at the edge. All selected completions on an edge are observed together; intermediate simulator logging states are not separate behaviors covered by this guarantee. This completion contract is defined later as the Batch profile (Core.RegisteredMessageBatch and its extensions). The combined settling network must be acyclic. Asynchronous reset assertion needs a separate conformance proof, partial reset scopes need an explicit protocol for affected communications, and configuration readers outside a paired update need their own ownership and synchronization justification. The guarantee concerns interface histories under the declared observation rules, not raw cycle timing or incidental listing order.

For the ordinary independent-resource profiles, the generic theorem assembles the system-level proof from design-specific process and primitive proofs, prescribed integration checks, settling and environment evidence, and implementation conformance. These profiles and that theorem are defined in J.OrdinaryConstructiveProfile and J.OrdinaryConstructiveComposition. Broader reset, elaborated-storage and combinational-observation extensions retain their own composition obligations; their combined scope is specified in J.ConstructiveCombinedProfile.

Use in the reverse direction. A particular pre-HLS simulation is not automatically guaranteed to be reproduced cycle for cycle, or even as the directly corresponding execution, by the RTL. Direct comparison of selected source and RTL runs requires the additional alignment conditions described later. Latency and throughput claims likewise need their own appropriate timing model and evidence; they do not follow merely from functional safety transfer.

Required-workload completion. The default practical workflow runs a finite, conservative serial pre-HLS execution through acceptance of the required outputs, using the implementation-bound reference model and channel capacities, or a justified smaller-capacity witness. The witness must supply the complete dependency certificate and its applicable acyclicity evidence.

Where K.CT.LazySerial eligibility holds, prefer running the default serial workflow in lazy mode; atomic groups retain the separately qualified group-serial discipline. The restricted construction derives acyclicity from its rank lemmas. K.CT.LazyComposition permits local lazy treatment within mixed designs, with other components retaining their own proofs and acyclicity established for the assembled

certificate. K.CT.Standard remains available when lazy treatment is unavailable or not selected; its FIFO-space dependencies use the guaranteed resource floor even when the witness ran with larger nominal capacity. Ordinary unreserved FIFOs require no additional movement restriction, and within the common restricted profile the resource construction reduces to the ordinary floor argument. Reserved pipeline storage is not automatically freely usable FIFO space. Together with correspondence for the relevant internal operations and facts, control/value agreement, source-aligned implementation control, pipeline/request progress, resource/guard support and the applicable arbitration and internal service, K.CT proves that every covered RTL execution satisfying the service assumptions eventually accepts those outputs with the same values and per-endpoint order. The standard workflow requires the applicable storage, controller, handshake and service conformance in B.CT and Q.CT. Physical pipeline reads are not automatically source-facing Pops: issue order must still preserve the source sequence across iterations. The standard early-staging proof requires placement and persistent request assertion to be established without assuming the target item's terminal arrival. A controller whose only route to that placement/request is travel with the item requires the separately proved Q.CT.Extension pre-request route; a data-travelling valid bit alone does not determine applicability. A passing simulation alone does not establish these conditions. Declared atomic groups use the qualified grouped-serial witness mode; ordinary serial issue cannot issue their same-owner blocking members together. Timing-dependent non-blocking outcomes or observations require a separate agreement qualification, and cases outside the standard profile require their stated extension proofs. The run need not finish unrelated trailing calls or drain the entire design. Failure to obtain a qualified witness is inconclusive, not a deadlock verdict. Completion supplies no cycle bound and does not rule out deadlocks in unrelated work or after the required outputs have been accepted. (Appendix K.CT.) The standard witness certificate also includes K.CT.CtrlSupport: complete entry, branch and required-work membership decisions, including negative decisions, with their historical support or independent invariant discharges.

Lazy mode is a pre-HLS simulation choice. It uses only process-facing Pop requests in that pre-HLS execution to restrict optional abstract movement inside the elaborated staging structure. It introduces no requirement to run or probe RTL simulation for each stimulus, changes no channel capacity and adds no producer-side ready-before-valid rule. Structural back-annotation and implementation-conformance, Facts and Agreement qualifications remain required. Any independently selected RTL-snooping or co-simulation dependency remains separate; lazy mode neither requires nor removes it. K.CT.LazySimulation and its informative example specify the distinction between process-facing demand and early input movement.

RTL coverage and qualification. A clean source execution is only part of the argument: the flow must establish implementation conformance and coverage of every behavior claimed. The retained safety workflow uses constructive correspondence, including its qualified stimulus-class form. Its qualified exact Batch clock-edge profile supplies the required all-finite-prefix coverage; origin correspondence alone covers only its declared origin domain. Checking one matched source/RTL run does not establish safety for other RTL executions. A selected source execution can support the source-property proof only under the explicit source-history coverage and property conditions; it does not replace implementation-wide correspondence. Documentation, static checks, targeted interface checks and formal verification can supply the needed evidence; the theorems do not certify a tool invocation automatically. Asynchronous-reset implementations require their separate argument. Required-workload completion additionally uses complete internal-fact correspondence, conditional agreement, a qualified witness and B.CT/Q.CT conformance with the stated exposure and service premises. Outer-history safety alone does not supply these obligations. (Appendix M.7.)

Document Goals

This document presents HLS tool scheduling rules and a modeling methodology. The goals are:

1. To provide easy-to-understand rules to HLS users.
2. To ensure precise and consistent rules in both SystemC and C++.
3. To offer rules that are effectively compatible with how Catapult currently operates.
4. To enable the best possible *quality of results* (QOR) in Catapult synthesis.
5. To cover all known user requirements and scenarios.
6. To serve as a suitable starting point for a standardization proposal (e.g., in Accellera SWG).

To illustrate the goals of this document more specifically, consider what an engineer writing a testbench for an HLS model needs to understand about how HLS tools operate. This engineer may be using SystemVerilog UVM or may be writing a testbench in C++/SystemC.

They likely are not an expert in any specific HLS tool (and may not want to be), but their testbench should be usable across both the pre-HLS model and the post-HLS model, subject to the latency-insensitivity, observation-boundary, and witness-selection conditions stated in this document. Thus, it is crucial for the DV engineer to have a precise understanding of how the HLS tool will transform the design while still enabling the resulting RTL behavior to be checked against the prescribed source reference model. This document describes what transformations the HLS tool is allowed to perform so that the pre-HLS and post-HLS models can be effectively verified with a common verification intent, with a source execution legally aligned to a selected RTL run when the Appendix L conditions are met. The overarching philosophy of the scheduling rules is to present “no surprises” to such a DV engineer, while still giving the HLS tool ample freedom to optimize the design.

The main body of this document is intended to be sufficiently precise and complete to support the Accellera standardization effort while still being understandable to HW architects. The formal appendices are intended to have a very high degree of precision and completeness sufficient to support AI-driven Lean mechanization of the formal theorems.

Background

HLS tools generate RTL from C++ models. Broadly speaking, this conversion takes a sequential C++ model and turns it into concurrent hardware that maintains the same behavior. HLS tools identify concurrent processes within the C++/SystemC model and then independently synthesize each process. Briefly, some of the techniques that HLS tools use to achieve good HW QOR when synthesizing each process include:

- Optimized scheduling based on the selected silicon target technology.
- Automatic HW pipeline construction according to the user’s specifications.
- Automatic HW resource sharing.
- Automatic scheduling of memory accesses.

The internal behavior of each process is specified by the control and dataflow behavior of the C++ code within the process. However, the external communication that each process has with other processes and HW blocks is specified via IO operations that are coded within the model. To enable a reliable, scalable, and verifiable HLS flow that generates high quality hardware, the scheduling behavior of these

IO operations needs to be precisely handled at all steps of the flow. This document specifies the rules that govern the scheduling behavior for these IO operations within HLS models.

These rules are specified with respect to an individual process, but the intent of the rules is to enable reliable and verifiable behavior of large sets of interacting processes operating as a system in real-world designs. (Appendix G provides the formal trace-compatibility guarantees between the prescribed source reference model and the post-HLS system, through constructive Post → Ref correspondence and its source-property transfer conditions; Appendix L separately supports legal source alignment to a selected RTL run.)

By default, HLS tools can insert additional clock cycles (or latency) anywhere within a process -- for example, when pipelining a loop or to enable HW resource sharing. The overall approach used in this document is to make the entire design and testbench system *latency insensitive* to the maximum extent possible, while still fully enabling key HW optimizations. The approach also supports capacity and throughput modeling when the selected source realization and timing model are qualified for those predictions; functional trace compatibility alone does not establish cycle latency or throughput equality.

In some cases, designs or testbenches may use protocols which are latency-sensitive. These situations can be handled by isolating the latency-sensitive portions to small, self-contained parts of the design or testbench, and then keeping the rest of the design and testbench latency insensitive. See Appendix D for more information.

In many cases the overall design will need to satisfy end-to-end latency requirements. For these designs it is still highly advantageous to use a latency-insensitive modeling approach and verify in the post-HLS model that the overall design latency requirements have been satisfied, since this is typically easy to do.

This document focuses on sequential HW processes. Combinational HW processes are supported under the settled-value observation conditions in Appendix G. The sequential SystemC examples use processes sensitive to a single rising clock edge. The formal guarantees presented here assume a single global clock. Appendix O sketches a possible multiclock extension; it does not establish the corresponding multiclock theorem.

This document distinguishes between the following:

1. The *conceptual model* for the scheduling rules.
2. The simulation behavior of a model using the rules in C++ or SystemC.
3. The synthesis of a C++/SystemC model using the rules in an HLS tool such as Catapult.

The goal is to align each of these three cases as closely as possible, so that the user has easy-to-understand rules, while simulation and synthesis work without surprises. However, as we will see, there are practical considerations which may in certain cases cause small deviations from the conceptual model in either simulation or HLS.

A simple real-world example that illustrates the motivation for the scheduling rules is provided in Appendix A of this document.

The examples referred to in this document are available here:

https://github.com/Stuart-Swan/Matchlib-Examples-Kit-For-Accellera-Synthesis-WG/tree/master

The most up-to-date version of this document is available here:

https://github.com/Stuart-Swan/Matchlib-Examples-Kit-For-Accellera-Synthesis-WG/blob/master/matchlib_examples/doc/catapult_user_view_scheduling_rules.pdf

A summary of the scheduling rules is available here:

https://github.com/Stuart-Swan/Matchlib-Examples-Kit-For-Accellera-Synthesis-WG/blob/master/matchlib_examples/doc/brief_scheduling_rules.txt

Slides which accompany this document are available here:

https://github.com/Stuart-Swan/Matchlib-Examples-Kit-For-Accellera-Synthesis-WG/blob/master/matchlib_examples/doc/HLS_Scheduling_Contract_for_Architects.pdf

A formal architecture guide is available here:

https://github.com/Stuart-Swan/Matchlib-Examples-Kit-For-Accellera-Synthesis-WG/blob/master/matchlib_examples/doc/formal_architecture_guide.pdf

Two white papers describing the motivation for the methodology are available here:

https://hls.academy/making-shift-left-real-why-systemc-matchlib-and-high-level-synthesis-are-the-right-foundation-for-soc-design-and-verification-in-an-age-of-ai/

https://hls.academy/topics/systemc-modeling/verifying-hardware-at-the-source-why-a-user-visible-hls-scheduling-contract-matters-in-an-age-of-ai-generated-design/

An Analogy from RTL Synthesis

To better understand the specific purpose of this document, let's consider how RTL synthesis works. Say you have a sequential block that you are modeling in Verilog RTL, and it has an output port coded like:

```
Out1 <= #some_delay new_val;
```

In Verilog simulation, if some_delay is less than the clock period of the block, then it will probably not affect the overall cycle-level behavior of the system during simulation. However, if some_delay is more than the clock period, it probably will.

During RTL synthesis, all RTL synthesis tools will ignore all delays in the input model, in this case even if some_delay is greater than the clock period. Some RTL synthesis tools might give a warning or error for code like the above such as "Simulation and synthesis results are likely to mismatch because the delay in model is greater than clock period." Some RTL synthesis tools might outright reject a model containing such delays.

One might argue that RTL synthesis tools should always match the Verilog simulation behavior of the input model. But the overall approach works well because RTL is a good and simple *conceptual model* that users and tool vendors can align around. The slight differences between the *simulation model* and the *conceptual model* used by RTL synthesis tools can be easily managed.

We'll return to this example later in this document.

Next, let's clarify some terminology related to signal IO. Say you have a Verilog model like:

```
forever begin
  @(posedge clk);  // wait for 1st rising clock edge
  output1 <= input1 + 10;
  @(posedge clk);  // wait for 2nd rising clock edge
end
```

In this example, input1 is sampled when the process wakes on the first @(posedge clk) and evaluates the RHS. For the purposes of the scheduling rules, we anchor that Read(input1,·) to the most recent synchronization point. The resulting Write(output1,·) is anchored to the next synchronization point (the cycle in which the written value is treated as stable for others to observe).
Cycle-level anchoring convention (used throughout this document). For cycle-level reasoning, each signal IO is assigned an anchor cycle relative to surrounding synchronization calls (e.g., wait() / SyncChannel / @(posedge clk)):

- Read(sig, val) is anchored to the closest preceding synchronization call, i.e. pred_S(Read).
- Write(sig, val) is anchored to the closest succeeding synchronization call, i.e. succ_S(Write).

This convention is only for cycle-level reasoning and does not depend on simulator update regions or delta-cycle ordering.

Catapult HLS Status Concerning Rules in this Document

A separate document provides a list of clarifications related to support within Catapult HLS for the rules described in this document. The items in the list are named *Cat#*, so that each item has a unique number. This document annotates certain rules with *Cat#* to refer to the items in the separate document. These Cat# annotations are not part of the formal proof itself.

Terms Used in this Document

Latency-Insensitive: In digital hardware design*, latency-insensitive* refers to a system that operates correctly despite variable communication delays between components. This is achieved by using mechanisms to decouple computation from communication timing. Such designs improve scalability and reliability in complex systems with unpredictable or variable latencies. ARM's AXI4 and APB are examples of latency-insensitive protocols. ARM's AMBA 5 CHI credit-based NOC protocol is also an example of a latency-insensitive protocol. The named commercial protocols are informative examples, not qualifications of every protocol version, optional feature or integration. A formal application binds and checks the actual protocol contract.

Process: In Verilog, a process is an *always block* and its equivalent constructs. In SystemC, a process is an instance of an SC_THREAD or SC_METHOD.

Message-passing Interface: A *message-passing interface* reliably delivers messages (or transactions) from one process to another. This document uses this term to denote the type of communication found in Kahn Process Networks. See https://en.wikipedia.org/wiki/Kahn_process_networks
Message-passing read interfaces are always separate from message-passing write interfaces – there are no bidirectional message-passing interfaces. In this document, message-passing channels are assumed to be point-to-point: for each channel c, exactly one producer performs all Push_c operations and exactly one consumer performs all Pop_c operations.
Shared resources are represented through explicit arbitration and communication structure. The individual modeled message channels remain point-to-point; the arbiter's choices and any relevant queues, credits, or coupling must be included in the model and its qualification. Shared-memory communication must also preserve the communicated values and dependencies, as described in Scheduling of Array Accesses. Latency-sensitive arbitration may require the choice-alignment techniques described in Appendix L.

Synchronization Interface: A *synchronization interface* synchronizes one process with another and/or with a global clock. For an example of a synchronization interface, see https://en.wikipedia.org/wiki/Barrier_(computer_science)

Signal IO: In digital HW design, signals are the fundamental communication mechanism. Signals enable communication between two HW blocks/processes, but communication with signals in real HW always incurs at least some delay because communication cannot be faster than the speed of light.
In HDLs and in SystemC, signal delays are modeled with the *delayed update* semantics.

blocking / non-blocking: In this document, a blocking message-passing operation is modeled as issuing a persistent request that remains asserted (and with stable payload, if applicable) until the operation commits (the message is sent or received). A non-blocking message-passing operation returns immediately with a status indicating whether the operation committed in that cycle.

One-way / two-way handshake protocols: A one-way handshake protocol only has one signal between a sender and receiver to synchronize communication. A two-way handshake protocol has two signals (one in each direction) between a sender and receiver to synchronize communication.

shall: This term indicates that a compliant tool or flow is required to follow the indicated rule.

may: This term indicates that a compliant tool or flow is allowed to follow the indicated rule but is not required to do so.

Classes of Operations Involved in Scheduling Rules

There are three classes of operations involved in the scheduling rules:

1. Calls to message-passing interfaces (which are all ac_channel methods, all SystemC Push/PushNB/Pop/PopNB calls except calls to SyncChannel)
2. Calls to synchronization interfaces (which are calls to ac_sync and Matchlib SyncChannel, also ac_wait and SystemC wait)
3. Signal IO (which are SystemC signal reads and writes, also C++ model *direct inputs*)

All the operations above are referred to as *IO operations*.

Basic Conceptual Model

The basic conceptual model encompasses processes that have no loop pipelining but may have preserved loops. If a process has a preserved loop, then the user may place a wait statement in the loop. Wait statements explicitly placed in the model are called explicit wait statements in this document and are classified as calls to a synchronization interface. However, for a loop to be treated as an HLS-pipelined loop under the formal guarantees in this document, the loop body shall not contain any wait/ac_wait statement or other explicit synchronization call used as an in-loop wait boundary. Such synchronization boundaries must be placed outside the pipelined loop, or the loop must be treated as non-pipelined/preserved or outside the theorem instance.

Two terms used in these rules distinguish the start of a transfer request from its completion. Issue means beginning the process-owned request at the selected message interface; commit means completing the transfer. Requests on different interfaces may overlap while their issue order is preserved. For example, an independent request on interface B may issue after a request on interface A but before A commits, when the selected issue policy permits that overlap. A request that depends on the result of A must wait for that result and obey the registered-cycle rule described below. Definitions: Issue vs. Commit gives the detailed boundary and timing conventions.
The basic conceptual model rules are:

1. Synchronization interface calls within a process always remain in the source code order.

2. Signal read operations occur at the closest preceding call to a synchronization interface. (Cat1)

3. Signal write operations occur at the closest succeeding call to a synchronization interface.

4. Message-passing operations may be shifted in time, issued in the same cycle, or overlapped by simulation or synthesis, subject to the following constraints:

- All message-passing operations before a call to a synchronization interface shall be issued before or in the same cycle in which the synchronization interface commits. (Here “the synchronization interface commits” refers to the commit of that synchronization call in the calling process’s trace. For blocking message-passing operations (Push/Pop), the process cannot complete the synchronization call until any earlier issued blocking message requests in that interval have committed.) This is a process-facing completion fence: preceding blocking requests may complete on the same edge as the sync. A buffered message already completed at its designated process-facing boundary may transfer externally later. Physical synchronization issue, held requests and sampled commitment obey Core.SyncProtocol.
- All message-passing operations after a call to a synchronization interface shall be issued after the cycle in which the synchronization interface call commits.
- Message-passing operations on the same message-passing interface shall be issued in source order; they shall not be reversed. This same-interface order requirement applies both within a single source interval and across overlapped iterations of a pipelined loop.
- Two message-passing operations on separate interfaces which appear in sequence in the model are source-ordered. They may be issued in the same source sequence, and the later operation may in some permitted modes issue before the earlier operation commits, including in the same clock cycle where allowed by the issue policy. They shall not be issued in the reverse sequence. (Cat2)

Here “parallel” issue means that requests may overlap or be asserted in the same cycle while the calls remain ordered by their dynamic source order. Preserving that order prevents HLS from introducing a

deadlock solely by reversing source-ordered message calls. It does not by itself guarantee deadlock preservation, because loop overlap, request timing, and finite channel capacity can still affect deadlock behavior. The completion route addresses acceptance of a required workload under its additional capacity, correspondence and progress premises; it does not assert general deadlock preservation.

Pipelined Loops

When a loop is pipelined in HLS, the body of the loop is split into pipeline stages. HLS may start the next iteration of the loop before the current iteration has completed, thus increasing throughput as compared to a non-pipelined implementation.

A loop selected for HLS pipelining shall not contain wait/ac_wait statements, SyncChannel/ac_sync calls, or any other explicit synchronization interface call inside the loop body. The pipelined-loop rules in this document therefore apply to wait-free pipelined loop bodies, with ordinary synchronization boundaries only outside the pipelined loop or around the pipelined region. If a design requires a synchronization call inside the loop body, the flow shall either reject that loop for pipelining under this contract, require the source to be rewritten so that the synchronization boundary is outside the pipelined loop, or exclude that pipelined-loop instance from the Appendices G-K formal guarantees unless a separate specialized theorem/certificate is supplied.

For standard required-workload completion, a pipelined controller shall actually meet B.OF and the applicable B.CT/Q.CT contracts, including the selected boundary mapping, source-aligned control and service laws. A flush directive or product-mode name alone does not establish conformance. A controller that may indefinitely freeze completed downstream work solely because upstream work is blocked does not meet B.OF.Upstream. Such a controller can claim constructive safety only if all of that result's scheduling, correspondence and faithful channel/coupler conditions are separately established; safety conformance does not supply completion. See Appendix B for the precise applicability conditions.

When HLS pipelines a loop, multiple iterations of the loop are overlapped and execute at the same time. During loop pipelining, for all IO operations, HLS shall ensure that an access to a specific message-passing interface, signal, or synchronization interface shall not be reordered relative to same-interface accesses from other iterations.

During pipelining, signal reads and writes are physically embedded in specific pipeline stages, but the loop body itself contains no explicit wait/ac_wait or other in-body synchronization call under the contract above. For scheduling and source/RTL comparison, an ordinary signal Read or Write is therefore still associated with the surrounding real synchronization calls, not with an internal pipeline stage. Considering the entire set of signals read or written by the process, if HLS pipelining would cause the order of signal IO to differ between the pre-HLS and post-HLS models, or if any two signal IO operations that occur on the same clock edge in the pre-HLS model would not occur on the same clock edge in the post-HLS model, then the HLS tool shall detect such a situation and require the user to explicitly indicate in the model source (e.g., via a pragma) that HLS pipelining can still be used. (Cat7) User authorization permits the transformation but does not waive the formal signal-observation requirements. Any changed observable behavior must be covered by an applicable modeling contract or a separate proof; otherwise that instance is outside the stated formal guarantee. In other words, if a process communicates with other processes using only latency-insensitive protocols, then HLS pipelining by default shall not break any of the protocols. Appendix G gives the formal signal-anchoring and matching rules.

Synchronization boundary for pipelined loops

When a pipelined region ends at an explicit synchronization call, work already accepted before the boundary may continue to drain. However, the implementation and the back-annotated source model must not expose capacity for work belonging to the next synchronization interval until the synchronization call completes. This prevents upstream transfers from entering the next interval prematurely. See Appendix B for the formal rule and modeling treatment.

Direct Inputs

Ordinary SystemC signal reads occur at the closest preceding synchronization interface call (e.g., wait statement) in both the pre-HLS and post-HLS models. (Cat1). If the HLS tool adds states to the process, or if it pipelines the process, then this implies that the HLS tool must add registers for each such read operation such that the read occurs where specified in the pre-HLS model, and the value is stored until the point where it is consumed in the post-HLS model. The area cost of such registers may be high if there are a lot of signals, and in some cases, it may be unneeded area since the value of the signals may not actually need to be stored internally to the process.

The simplest case is if such external signals are held stable after the process comes out of reset. In this case, HLS may physically sample the signal values as late as possible, with no need for register storage, while for scheduling and source/RTL comparison the Read is still treated as occurring at its prescribed synchronization point. This case is handled with the following pragma on SystemC signals and ports (Cat6):

```
#pragma hls_direct_input
```

For the reset-free registered message safety profile, J.StableRead-Batch below supplies a lifting route for repeated or conditional in-body reads under this same anchored-value contract. It does not require an explicit source initialization copy or a dedicated physical configuration register; its message-only observation and qualification limits are stated there.

To ensure that there are no pre-HLS versus post-HLS simulation mismatches, the environment that drives the signal shall hold it stable within each relevant reset epoch: after all receiving processes that use this signal with this pragma have come out of a reset boundary, the signal value shall remain stable until the next reset boundary that affects those receiving processes, unless the signal is instead governed by an explicit hls_direct_input_sync update contract as described below. A dynamic reset starts a new stability epoch for the affected receiving logic and its associated direct-input signals. Thus it remains allowable to have dynamic resets, i.e. activation of block resets and associated resetting of direct input signals as part of the normal operation of the HW. For workload completion, a reset affecting outstanding required work needs the explicit obligation-preservation or epoch contract required by K.CT; reset-aware safety correspondence alone does not establish completion.

A related but somewhat more complex case is where input signals to the HW block may only be changed at “agreed upon” times, typically while the portion of the HW block that relies on them is temporarily idle. For example, a block may process 2D images. At the start of each new image, it may be desirable for the TB or external environment to update the control signals for how the block will process the next image. Before the DUT offers sync ready, all uses of the old input values must have finished or have retained operands isolated from subsequent pin changes. Fully draining the relevant pipeline is one way to establish that protection, but is not required when remaining old uses are safely isolated. Core.16(7) requires producer-facing availability for capacity belonging only to the next interval to be withheld while

the process is pending at the sync, until it commits. This mandatory capacity withdrawal prevents premature admission; it does not by itself protect old uses. The DUT maintains the required protection while the sync is pending and for any remaining protected uses, as specified by Core.SyncProtocol and the applicable direct-input lifting contract. In this case we can use the SystemC *SyncChannel* or C++ *ac_sync* primitives to precisely synchronize the DUT with the TB/environment to enable the input signals to be updated at the correct time. The `#pragma hls_direct_input_sync` directive shown below associates the sync operation with the direct inputs that it controls. When these old-value protection, sampling and synchronization requirements are met, the scheme shown here ensures that there are no pre-HLS versus post-HLS simulation mismatches even though we are using direct inputs and also changing their values while the design is executing.

When a signal or port covered by #pragma hls_direct_input is also included in the scope of a later #pragma hls_direct_input_sync directive, the sync directive selects the effective update contract for that controlled signal set. Such signals remain direct inputs for purposes of late physical sampling, but they are not required to remain stable for the entire reset epoch; instead, they must satisfy the sync-controlled discipline described here: updates may occur only at the designated synchronization handshake, and the values must remain stable between permitted sync-controlled updates and reset boundaries.

```
  // This is example 61 in Catapult Matchlib examples
  sc_in<bool> SC_NAMED(clk);
  sc_in<bool> SC_NAMED(rst_bar);

  Connections::Out<uint32_t>  SC_NAMED(out1);
  Connections::In<uint32_t>   sample_in[num_samples];
  Connections::SyncIn         SC_NAMED(sync_in);
#pragma hls_direct_input
  sc_in<uint32_t> direct_inputs[num_direct_inputs];

  void main() {
    out1.Reset();
    sync_in.Reset();

#pragma hls_unroll yes
    for (int i=0; i < num_samples; i++) {
      sample_in[i].Reset();
    }

    wait();  // reset state

    while (1) {
#pragma hls_direct_input_sync all
     sync_in.sync_in();

#pragma hls_pipeline_init_interval 1
#pragma hls_stall_mode flush
     for (uint32_t x=0; x < direct_inputs[0]; x++) {
       for (uint32_t y=0; y < direct_inputs[1]; y++) {
        uint32_t sum = 0;
#pragma hls_unroll yes
        for (uint32_t s=0; s < num_samples; s++) {
           sum += sample_in[s].Pop() * direct_inputs[2 + s];
        }
        ac_int<32, false> ac_sum = sum;
        ac_int<32, false> sqrt = 0;
```

```
          ac_math::ac_sqrt(ac_sum, sqrt);  // internal loop unrolled in cat .tcl
file
          if (sqrt > direct_inputs[7])
            out1.Push(sqrt);
        }
      }
    }
  }
};
```

For signals controlled by hls_direct_input_sync, the TB first waits for the DUT's sync rdy. The DUT keeps rdy asserted until the sync completes or an affecting reset occurs. By asserting rdy, the DUT must already have finished using the old input values or protected any remaining uses from changes at the pins. The TB then changes the designated inputs and asserts sync vld, allowing them to settle before the sampling edge. With rdy still high and no affecting reset, the sync completes at that edge, so a one-cycle vld response is sufficient. Do not drop vld merely because a cycle elapsed if the handshake did not occur. Keep the new input values stable until the next permitted update or affecting reset. Core.SyncProtocol gives the formal request, sampling, reset and old-value protection requirements.

It is important to note that the only safe operation to use to synchronize the updating of direct inputs is the sync operation as shown above. Other operations such as Push/Pop or ac_channel operations should not be used for this. For purposes of source/RTL comparison, the logical Read(sig,val) remains associated with its synchronization point even if the implementation samples the stable physical signal later. Such additional internal sampling must not create another logical Read or change the value attributed to the original Read. Appendix G formalizes this trace treatment.

In the example above the DUT block that is being synthesized determines when to call sync, and thus it determines when the direct inputs will be updated. In some cases, it may be necessary for the environment around the DUT to determine when the direct inputs should be updated. In this case the same approach as shown above should be used; however a separate input from the environment to the DUT (either using a signal or a message-passing interface) should request that the DUT call sync as soon as feasible. Before offering sync ready, the DUT must establish that old-value uses have finished or are retained and isolated; receiving a request to call sync does not establish that condition. The separate request to call sync is not itself permission to change the direct inputs. Permission comes from the designated held sync offer under Core.SyncProtocol, including old-user pin isolation.

Additional Options for Scheduling Message-passing Interfaces

The following option may be added during HLS (Cat2):

```
STRICT_IO_SCHEDULING=relaxed
```

When this is specified, the HLS tool is allowed to reorder message-passing interface calls freely on distinct interfaces (still not moving calls across synchronization interface calls). This relaxed mode may violate the default no-reverse discipline and therefore may introduce deadlocks that are not present under the default rules. It is recommended that this relaxed option only be used when the user wants to see what order the HLS tool prefers to schedule message-passing interface calls (e.g., to achieve best QOR).

Once the user knows the preferred order, it is recommended that the user modify the pre-HLS source code to reflect the preferred order, and then return to the use of the default scheduling modes. This methodology ensures that HLS cannot introduce new deadlocks via reversal of source order on distinct message interfaces as described earlier. Deadlock behavior may still depend on bounded-capacity effects and on overlapped execution (e.g., loop pipelining); those cases are addressed separately by the pipelined-loop automatic-flush rules and the required-workload completion conditions in Appendix K. The formal correspondence and required-workload completion results in Appendices G–K assume STRICT_IO_SCHEDULING is not set to relaxed.

Scheduling of Array Accesses

Arrays may appear in HLS models, and they may be preserved through synthesis and mapped to RAMs. Pointers may also appear in HLS models, and pointer dereferences are resolved to array accesses during HLS.

For scheduling and verification, the physical placement of a RAM inside or outside the synthesized block does not by itself change the rules. What matters is the declared logical behavior of the mapped memory and the logical interface at which the pre-HLS and post-HLS designs are compared. The rules below apply to every array access mapped to memory. The distinction drawn after them concerns only whether the memory also carries values from one process to another; the shared rules, including the qualification requirements for transformed and for blocking memory realizations, apply in both cases.

Rules applying to all mapped memory behavior.

- For each mapped memory instance within the scheduling/verification scope, the user model or flow shall declare an *array-access mapping layer*. That declaration identifies the untransformed logical memory behavior, the logical boundary at which source and implementation behavior are compared, and how that boundary is represented using signal IO, message passing, synchronization, or a qualified logical memory interface.
- The user may authorize HLS to cache, merge, split, or reorder array accesses (for example, to improve QOR). The resulting physical RAM command/response sequence may differ below the declared logical boundary, but the transformed implementation shall preserve the declared logical behavior and the ordinary scheduling and functional requirements at that boundary. In particular, a memory mapping or transformation may not create, drop, duplicate, relabel, or illegally reorder an interaction that is supposed to be visible at the selected logical boundary. Preservation at the declared boundary is a per-instance obligation that the qualification flow must discharge, not an assumption.
- If the selected logical boundary exposes a message channel or an anchored signal, that interface remains governed by the same rules as any other message channel or signal in this document. An otherwise visible transfer, signal Read, or signal Write cannot be hidden merely because it arose from an array or memory mapping.
- In all cases, array accesses remain constrained by explicit synchronization calls present in the process source model. All array reads or writes before a synchronization call shall commit before or in the same cycle in which that synchronization call commits, and all array reads or writes after it shall commit in a later cycle. Any cache/merge/split/reorder transformation must preserve this commit-time fence. In particular, one merged storage commit may not represent source accesses lying on opposite sides of the same synchronization boundary; such a merge is forbidden regardless of where the logical comparison boundary is placed.

- If the physical memory system can persistently backpressure or stall a process, the completion qualification shall account for every relevant capacity, request, enabling, resource and service condition that can delay required work. Hidden physical structure may remain below the logical boundary only when the qualification proves that the declared exposure, persistence and service obligations still hold. This requirement applies whether or not the memory carries cross-process value flow. Transaction-changing memory behavior without a suitable logical boundary and Core.MemProfile/Core.MemFaithful qualification is outside the completion guarantee for that instance. Hiding physical memory transactions from the observation alphabet does not remove their progress obligations.

Memory used for communication between processes.

1. No cross-process memory-carried value dependence

A RAM mapping by itself does not make raw physical memory transactions part of the externally compared behavior. If the memory does not carry a value from one process into another process's computation or later behavior, those raw transactions need not be exposed solely because the storage was mapped to RAM. The general mapping, synchronization, visible-interface, and blocking rules above still apply.

2. Cross-process memory-carried value dependence

If one process's memory write can affect another process's control flow, payload or value expressions, or the identity or order of its later operations, every such cross-process dependency shall remain represented in the verification model. The raw physical RAM transactions do not have to be exposed if the dependency, including the communicated value, is represented at a supported logical interface. Synchronization alone may order the accesses, but does not by itself represent the value communicated through memory. If caching, merging, splitting, or reordering changes the physical transaction stream, the standard covered route is to compare behavior at a qualified logical boundary above those transformations whose externally relevant behavior is invariant under the permitted transformation set. A separately supplied proof that directly tracks shared-memory state is an alternative. Without one of those routes, the formal source/RTL safety and required-workload completion results do not cover that shared-memory communication.

When a loop is pipelined, multiple iterations of the loop are overlapped and execute at the same time. During loop pipelining, an access to a memory interface (or array) may be moved over or in parallel with another access to the same memory if the HLS tool can prove that the reordering is conflict-free. For a mapped memory within the scheduling/verification scope, the declared mapping shall explicitly authorize such reordering, and the reordered implementation shall still preserve the logical memory behavior, synchronization fences, visible-interface behavior, and blocking requirements stated above.

Formal treatment. The Normative Formal Core states the exact mapped-memory faithfulness (Core.MemFaithful), synchronization-order (Core.MemSyncOrder), shared-memory lowering (J.Mem), and qualified memory-profile (Core.MemProfile) conditions. Appendix J uses these conditions in the source/RTL safety proof, Appendix K gives the additional required-workload completion conditions, and Appendix M.7a is the definitive per-instance qualification and evidence ledger. The Core also describes the alternative direct memory-state-extension route.

Definitions: Issue vs. Commit

Informally, we can define **commit** and **issue** as follows:

Commit (message channel) — informal operational reading

For the RTL_B implementation reading at a mapped ready/valid boundary, a message commit is observed in the cycle in which an external clocked observer sees the transfer complete:

- The mapped boundary commit is observed when the observer sees both valid and ready high in that cycle.
- The committed payload is the data value seen in that same cycle. This is an implementation-facing observation rule; for the G–K theorem stack, the formal abstract commit and Commit(q)/CommitCycle(q) definitions are in Definition Core.8. The sole internal logical-Pop exception is the explicitly qualified attribution of Core.LogicalPopPayload; physically sampled observation interfaces retain the data value actually seen.

Definition: Issue (message channel) — message request assertion
Issue is the cycle in which the calling process begins requesting the transfer at the selected message-channel boundary. For a Push, its boundary manifestation is assertion of the process-owned vld request; for a Pop, it is assertion of the process-owned rdy request. A raw physical ready/valid level that instead represents channel-side storage, complementary availability, or declared coupler logic is not by itself BoundaryReq or Issue. Appendix C and Core.10/Core.IC define the precise process-owned request and the blocking back-to-back ownership rule.

For RAM reads/writes as presented above, commit denotes the cycle in which the declared mapping-layer storage semantics recognizes the access as committed. The “Scheduling of Array Accesses” constraints are therefore written in commit-time. A fixed-latency memory is one simple implementation for which the scheduler can choose an issue cycle that satisfies the synchronization fence, but fixed latency is not a general requirement; any variable-latency or transformed mapping must still preserve the same commit-time fence. The Normative Formal Core gives the exact audit condition (Core.MemSyncOrder).

Sequential processes use registered cycle semantics: a result returned by a message-passing call at a cycle boundary cannot be used to generate a dependent request in that same cycle. Such a dependent request may first issue in a later cycle. Appendix C and the Normative Formal Core (Definition Core.RegSeq) give the precise rule, including its treatment of failed non-blocking calls and independently active pipeline iterations.

Additional States Added by HLS Synthesis

By default, HLS synthesis tools may add additional states to processes (e.g. add latency to enable resource sharing), which may introduce latency differences in the interface behavior between the pre-HLS and post-HLS models. These additional states are never included in the set of *synchronization interface calls* as described above.

When the directive IMPLICIT_FSM=true is set on a process, the HLS synthesis tool shall ensure that the cycle-level behavior of the interfaces of the pre-HLS and post-HLS models shall be identical. With this option, the internal state machines of the pre-HLS and post-HLS models will be the same. This is a required contract for the selected directive, not evidence that a particular tool version or invocation implements it; bind tool/version/mode conformance separately.

When the directive IO_MODE=FIXED is set on a process, the HLS synthesis tool shall ensure that the cycle-level behavior of the interfaces of the pre-HLS and post-HLS models shall be identical. With this option, it is still possible that the state machine internal to the process is different between the pre-HLS

and post-HLS models (e.g. the post-HLS model may choose to use a pipelined multiplier where the pre-HLS model did not.) This is a required contract for the selected directive, not evidence of conformance by a particular tool version or invocation.

Avoiding Pre-HLS and Post-HLS Simulation Mismatches

The scheduling rules described in this document are designed to be easy to understand, while providing good QOR via HLS and generally avoiding any mismatches between the pre-HLS and post-HLS simulation behaviors.

Non-blocking message-passing operations (PushNB/PopNB in SystemC, C++ ac_channel nb_read/nb_write) are a potential source of mismatches between pre-HLS and post-HLS simulations since their behavior is inherently dependent on the latency within the model, which often changes during HLS. Because of this, non-blocking message-passing interfaces should only be used when no alternative approach is possible. For example, non-blocking message-passing interfaces are required to model time-based arbitration of multiple message streams which access a shared resource. A full discussion of recommended guidelines on the use and verification of non-blocking message-passing interfaces is provided in Appendix L. Note that the scheduling rules described previously in this document fully specify how HLS tools are required to schedule such operations.

Unidirectional message-passing between two processes should not be relied on to achieve synchronization between the two processes, since in general the message latency and storage capacity between the processes may be variable. Also, depending on buffering/pipelining in the realization, the time between a producer's committed Push and the consumer's committed Pop (and the transient in-flight occupancy) may vary. Two unidirectional message-passing channels in opposite directions can be relied upon for synchronization between two processes. (An example of this is the AXI4 ar and r, and aw and b, channels). Note that such synchronization is weaker than explicit synchronization like SyncChannel or signal IO that uses a two-way handshake.

SystemC signal IO operations are a potential source of pre-HLS versus post-HLS simulation mismatches since timing behaviors may change between the two models. The following section provides guidance and rules to help avoid potential mismatches due to signal IO.

When signal IO operations are synchronized with a wait statement, there generally should be a proper two-way handshake associated with the wait statement so that the signal IO is latency insensitive. (Note that this statement does not apply to the signal synchronization approaches described in the Direct Inputs section.)

For example, here's a simple two-way handshake protocol when writing the signal `out_dat`:

```
out_dat = value;
out_vld = 1;
do {
  wait();
} while (out_rdy != 1);
out_vld = 0;
```

And here's a two-way handshake example when reading signal `in_dat`:

```
in_rdy = 1;
do {
  wait();
} while (in_vld != 1);
value = in_dat;
in_rdy = 0;
```

Some signal-level protocols have different two-way handshaking approaches (e.g. ARM APB), but they are still latency insensitive. This named protocol example is informative; the selected variant and operating assumptions require their own qualification.

If signal IO operations are associated with a wait statement and that wait statement does not have a proper two-way handshake, then the signal IO is likely to be latency-sensitive and may result in pre-HLS versus post-HLS simulation mismatches. In some systems a one-way signal handshake is sufficient for reliable system operation. See Appendix E for further discussion.

The scheduling rules state that signal IO operations occur at either SystemC wait statements or SyncChannel calls (sync_in and sync_out). In the remainder of this section, we will use *wait* statements to refer to both.

RULE 1: For the formal guarantees, every observable signal read or write shall have a unique unconditional real synchronization anchor on each dynamic path, and its observed value shall agree with that anchor. The recommended coding style is to group signal writes just before their corresponding wait statement and signal reads just after it. Alternative placement requires an explicit source-level well-formedness justification establishing the same anchoring and value consistency. (Cat3). An example of the recommended placement is below:
RULE 1 retained-Read qualification. For every selected observable Read of a sequential process, the source well-formedness justification shall establish both its retained membership and its value at its real preceding synchronization anchor. Concretely, finite local evaluation from the anchor state and its prescribed signal snapshot determines, for each selected (process, signal, anchor) key, either no retained Read or one retained Read with a fixed value; the result agrees with the qualified source anchor semantics on every admitted continuation. It may not depend on a message result, branch decision or other input first available after that anchor. Once the anchor unit completes, its retained Read membership and value do not change. Conditional source code is permitted when this retained observation is already determined at the anchor; repeated evaluations remain subject to Core.17 canonicalization. Alternative placement and direct-input late physical sampling do not waive this requirement for retained observable Reads. The requirement does not invent an executed read on an untaken branch or turn every physical sample into an observable event. Core.AnchorConfigObservation is a separately declared constructive-route observation option for designated direct-input configuration signals: those signals contribute configuration attributes of real anchors/transactions, not selected process-Read events. Ordinary signals and all Read-based observation routes retain this qualification unchanged; physical use and source execution remain subject to their existing semantic rules.

```
sc_in<int> i1;
sc_in<bool> go;
sc_out<int> o1;
void my_thread() {
  int new_val=0;
  while (1) {
    o1.write(new_val);
    do {
```

```
      wait();
    } while (!go.read());
    new_val = i1.read();
    new_val = some_function(new_val); // function has no internal IO
  }
}
```

By placing the signal IO operations as close as possible to their corresponding wait statement, the HW intent is very clear. And there is no benefit either in terms of simulation performance or HLS QOR if they are placed further away from their corresponding wait statement.

Let's look at another similar example, which now also uses a Matchlib Connections blocking Pop operation:

```
sc_in<int> i1;
sc_in<bool> go;
sc_out<int> o1;
Connections::In<int> pop1;
void my_thread() {
  int new_val=0;
  while (1) {
    o1.write(new_val);
    do {
      wait();
    } while (!go.read());
    int pop_val = pop1.Pop();
    new_val = i1.read();
    new_val = some_function(new_val + pop_val); // function has no internal
IO
  }
}
```

Under the anchoring rules, i1.read() remains associated with the same synchronization point regardless of the intervening Pop. However, in raw pre-HLS simulation, a blocking Pop may stall for one or more cycles, so i1 can change before the i1.read() executes—creating behavior that differs from the anchored interpretation (and potentially from post-HLS scheduling). The fix is simply to place i1.read() immediately after its intended wait() anchor; this removes the mismatch risk without affecting QOR or simulation performance.

For a proof-backed flow, RULE 1 conformance is a source well-formedness obligation, not merely a recommended diagnostic. If a blocking message-passing operation separates a signal read or write operation from its corresponding synchronization interface call, then the flow shall either reject the model for purposes of applying the formal guarantees, require the source to be rewritten so the signal IO is adjacent to its intended anchor, or discharge an explicit source-level well-formedness argument showing that the signal IO has a unique unconditional real synchronization anchor on every dynamic path and cannot observe a value inconsistent with that anchor. HLS tools may implement this requirement using errors, warnings, lint checks, or other diagnostics, but an unresolved warning is not sufficient to claim the RULE 1 / RULE 2-based guarantees. The explicit justification for a retained Read also includes the membership-and-value determination specified by RULE 1 retained-Read qualification. A unique anchor and a stable value alone do not justify a Read whose retained presence is first decided by a later message result.

Another scenario in which RULE 1 applies is shown below:

```
sc_in<bool> go;
sc_out<int> o1;
void my_thread() {

  while (1) {
    wait();              // WAIT 1
    o1.write(some_value);
    if (go.read()) {
      some_value = some_function();
    }
    else {
      wait();            // WAIT 2
    }
    some_other_function();
    wait();              // WAIT 3
  }
}
```

The signal read of `go` is clearly and uniquely associated with WAIT 1. However, the signal write of `o1` associates with WAIT 2 if `go` is false and WAIT 3 if it is not. This is a violation of RULE 1 and should be flagged as an error during HLS. The fix, as before, is to move the signal IO operation as close as possible to its intended wait statement so that the association is unconditional.

Next, let's consider *rolled* (or *preserved*) loops that perform signal IO within the loop body. Consider the following example:

```
sc_in<int> i1;
sc_out<int> o1;
void my_thread() {
    wait(); // reset state
    while (1) {
      wait(); // start of while loop
      #pragma hls_unroll no
      for (int i=0; i < 10; i++) {
        o1.write(i1.read() * i);
      }
  }
}
```

The placement of i1.read() inside the preserved loop may suggest that per-iteration sampling was intended, but the code has no real synchronization anchor inside that loop. If one anchored value is intended for all iterations, moving the read before the loop makes that intent explicit.

An implementation that samples a fresh i1 value or exposes a distinct o1 value on each iteration across added clock cycles can differ from both the raw zero-time pre-HLS simulation and the anchored interpretation. Added implementation states do not create new logical synchronization anchors or authorize these additional observations. In the pre-HLS loop, only the final source write supplies the retained value at the succeeding anchor. If per-iteration signal interaction is intended, place an explicit wait() within this non-pipelined preserved loop and satisfy RULE 1, or use another supported modeling contract. The unrepaired example is not permission to change the anchored behavior.

RULE 2: If you have signal IO operations within *rolled* (or *preserved)* loops, manually place a wait statement within the body of the loop to avoid pre-HLS versus post-HLS simulation mismatches, and while doing so also follow **RULE 1**.
Scope of RULE 2 for pipelining. RULE 2 applies only to rolled or preserved loops that are not selected for pipelining under this contract. A loop body selected for pipelining shall not contain wait/ac_wait statements, SyncChannel/ac_sync calls, or any other explicit synchronization interface call. Therefore, if ordinary per-iteration signal IO inside a loop requires a per-iteration wait anchor, that loop is outside the pipelined-loop contract unless the source is rewritten: move the signal IO outside the pipelined loop, model the interaction through message-passing, use hls_direct_input / hls_direct_input_sync under their stated contracts, or forgo pipelining for that loop. Without an in-body synchronization anchor, repeated per-iteration signal reads or writes inside one surrounding source interval do not acquire separate logical synchronization points merely because HLS later pipelines the loop; they remain associated with the surrounding synchronization calls. Appendix G formalizes the corresponding signal-event treatment.

For a proof-backed flow, a rolled loop with signal IO operations but no wait statement in the loop body shall be treated as a source well-formedness failure unless an explicit source-level argument shows that each observable signal Read/Write still has a unique real synchronization anchor on every dynamic path. HLS tools may implement this requirement using errors, warnings, lint checks, or other diagnostics. However, if a diagnostic is emitted as a warning rather than an error, the warning must be resolved, waived with an explicit well-formedness justification, or treated as excluding that model from the formal guarantees. (Cat4)

If a pre-HLS model has established RULE 1 and RULE 2 conformance, then all signal IO in the post-HLS model shall occur only at clock cycles that correspond to explicit wait statements or explicit synchronization statements. For direct-input signals/ports (e.g., #pragma hls_direct_input and #pragma hls_direct_input_sync), this requirement refers to the logical anchor cycle of the Read/Write: the Read or Write is still treated as occurring at its synchronization point even if HLS physically samples a stable signal later. Such later physical samples are internal implementation activity; they do not create additional observable Read/Write events. User designs that do not adhere to both RULE 1 and RULE 2 are ill-formed and are outside the formal guarantees unless the nonconformance has been eliminated or explicitly discharged by a valid source-level well-formedness argument. For direct inputs included in a retained anchor unit, logical Read membership is qualified at the anchor under RULE 1, while permitted physical samples may occur later. J.AnchorDirectInputLift states the composition with observable synchronization; J.StableRead-Batch retains its separately declared message-only, whole-group-erasure scope. Alternatively, the explicitly selected Core.AnchorConfigObservation option observes designated direct inputs through configuration-bearing anchors under J.LG-ConfigBatch. That option is route-local and does not redefine the general E2/Core.17 Read convention.

Returning to the Analogy from RTL Synthesis

At the beginning of this document, we presented the example of a Verilog sequential block with an output coded like:

```
Out1 <= #some_delay new_val;
```

Recall that in Verilog simulation, if some_delay is less than the clock period of the block, then it will probably not affect the overall cycle-level behavior of the system during simulation. However, if some_delay is more than the clock period, it probably will.

During RTL synthesis, all RTL synthesis tools will ignore all delays in the input model, in this case even if some_delay is greater than the clock period. Some RTL synthesis tools might give a warning for the code above like “Simulation and synthesis results are likely to mismatch because delay in model is greater than clock period.”

HLS tools or associated lint/qualification flows that adhere closely to the conceptual model presented in this document should provide a concrete mechanism for detecting or discharging violations of RULE 1 and RULE 2 as described in the section above. This is analogous to the error or warning that the RTL synthesis tool would provide in the example directly above.

However, the formal guarantees in this document do not depend on any particular diagnostic mechanism. If an HLS synthesis tool does not report a RULE 1 / RULE 2 issue, the design is not thereby made well-formed. The proof-backed flow must still establish that the source model satisfies the RULE 1 / RULE 2 well-formedness obligation before applying the trace-compatibility guarantees.

Summary

At the beginning of this document, we said that the intent was to present “no surprises” to a DV engineer who wants a common verification methodology for the pre-HLS and post-HLS models. The key aspects of the document which support this are:

- Three groups of IO operations are defined (message-passing, signal IO, and synchronization calls) and each is treated uniformly. These IO operations are easy for verification engineers to understand because they are already using them in their testbenches.

- The document specifically avoids complex constructs such as *protocol regions* used in some HLS tools.

- The document preserves the ability to use a library such as Matchlib for pre-HLS throughput modeling, subject to qualification of the selected realization and timing model. Functional safety transfer alone does not establish throughput accuracy.

- Synchronization calls constrain both signal anchoring and message-passing scheduling. Message-passing calls do not move a signal’s logical anchor merely because they appear before or after the signal statement. Likewise, raw lexical placement of signal IO does not add message-order constraints beyond the anchoring and scheduling rules. Signal values may still determine message payloads, control flow, and which message operations execute. In raw pre-HLS simulation, a blocking message operation can delay a signal access and change the value it observes. RULE 1 therefore requires valid anchoring and value consistency; adjacent placement is the recommended way to meet that requirement, with alternative placement allowed only under an explicit well-formedness justification.

- HLS cannot by default reverse the order of message-passing calls, so it cannot introduce new deadlocks via call-order reversal. For pipelined loops (overlapped iterations) and other

transformations that change request/commit timing under bounded capacities, required-workload completion depends on the additional witness, correspondence, capacity, automatic-flush and service conditions described later in the document, not on the no-reverse rule alone.

- HLS pipelining is largely a don't care from the perspective of the verification engineer when the design and testbench are insensitive to latency changes, pipelines flush automatically, and any mapped memory transformations preserve the declared logical memory behavior and the synchronization and blocking rules. When the design or testbench is latency-sensitive, or when memory transactions are rearranged, the affected behavior must instead be handled through the explicit latency-sensitive or mapped-memory contracts described in this document. Appendix G gives the formal source/RTL trace-compatibility guarantee and its scope rather than an unrestricted bidirectional equivalence theorem.

- For sequential processes, signal IO operations in the post-HLS model are logically associated with synchronization points (e.g., wait statements) that are explicit in the pre-HLS source. For direct-input signals/ports, the implementation may physically sample a stable signal later, but that later sample is internal and does not move or duplicate the logical Read/Write. For combinational processes, observable signal behavior is instead taken only after the combinational logic has reached its stable fixed point for the cycle (R2C). Appendix G gives the precise formal treatment.

The principal formal safety and required-workload completion results, their scope limitations, and the evidence required to apply them are summarized in Appendix M. Appendix F provides user-oriented modeling guidelines.

Appendix A – Factory Analogy

The scheduling rules described in this document apply to pre-HLS and post-HLS HW models. Although the rules may seem abstract and perhaps even arbitrary, they are shaped by the need to model systems in the real world that are required to have predictable behavior.

To understand the motivation behind the rules, it might be helpful to draw a simple real-world analogy and its correspondence to the rules described earlier.

Consider a factory that produces various types of wooden furniture:

The factory consists of people (processes) stationed at workbenches with various tools.

Each person is given written instructions about the specific tasks they are to perform (C++ code within a process).

People are instructed to send or receive objects (messages) to or from other people in the factory.

Sending or receiving objects may be blocking or non-blocking from the perspective of a person.

There is a clock with a second hand on the factory wall that everyone can see. (HW clock).

People have colored flags they can raise or lower to communicate with other people (signals).

If someone raises or lowers a flag, this is only seen by others the next time the clock second hand is at the top of the clock. (propagation delay of signals between sequential processes)

A person can choose to pipeline the tasks that they were assigned by hiring subordinates (pipeline stages) and having each one do a subtask. In general, this will improve the throughput for the tasks that the person was assigned.

It is possible for two people to explicitly synchronize their work by communicating via a synchronization protocol such as a barrier (synchronization).

It is possible to restart the work of some or all the people by raising a reset flag (HW resets).

Assume:

1. That the time that people take to complete their various tasks is in general variable, and similarly that the time that objects (messages) take to pass between people is in general variable.
2. That each person in general wants to complete their tasks as quickly and efficiently as possible.
3. That the factory needs to be able to make multiple types of furniture at the same time. For example, it might make chairs that need to be different colors or have different styles.

To reliably produce output, each person in the factory will need to adhere to rules like those described in this document.

Here's a sketch of a specific way the above example relates to the scheduling rules:

Person 1 sends chair seats and chair backs to Person 2. This is done with blocking operations, and there is no storage capacity between the people when the objects are sent. (This means that a Push operation cannot complete until the corresponding Pop operation is performed.) Assume that the fastest an object can be sent between people is 1 minute.

Timing convention for the examples below. The timelines show a Push at the time the calling process executes the Push() call in its source-process context—i.e., for the clocked SystemC analogy, during the process execution immediately following the preceding clock edge. This is **not necessarily the formal Issue cycle** defined elsewhere in this document; formal Issue refers to the cycle in which the corresponding request becomes visible at the selected message-channel boundary (for example, as vld for a Push). A displayed Pop instead denotes the time at which the blocking Pop() completes and returns its value to the calling process. The reverse-order timetable below is explicitly hypothetical and does not show a legal execution. In these zero-capacity examples, the corresponding blocking Push completes when that Pop completes.

The written instructions that person 1 is given are:

```
while (1) {
 // internal processing code for seats and backs ...
 seats.Push(seat_object);
 backs.Push(back_object);
}
```

The written instructions that person 2 is given are:

```
while (1) {
 seat_object = seats.Pop();
 back_object = backs.Pop();
 // internal processing code for seats and backs ...
}
```

If both person 1 and person 2 choose to perform their tasks sequentially as written, the resulting sequence of Push calls and Pop completions is:

```
             Person 1             Person 2
Minute 1:    seats.Push(seat1);
Minute 2:    backs.Push(back1);   seat1 = seats.Pop();
Minute 3:    seats.Push(seat2);   back1 = backs.Pop();
Minute 4:    backs.Push(back2);   seat2 = seats.Pop();
Minute 5:                         back2 = backs.Pop();
```

If both person 1 and person 2 choose to perform their IO operations in parallel, the resulting sequence of Push calls and Pop completions is:

```
            Person 1                                  Person 2
Minute 1:   seats.Push(seat1); backs.Push(back1);
Minute 2:   seats.Push(seat2); backs.Push(back2);     seat1 = seats.Pop(); back1 = backs.Pop();
Minute 3:                                             seat2 = seats.Pop(); back2 = backs.Pop();
```

If person 1 chooses to perform their IO operations in parallel, and person 2 chooses to perform their IO operations sequentially as written, the resulting sequence of Push calls and Pop completions is:

```
            Person 1                                  Person 2
Minute 1:   seats.Push(seat1); backs.Push(back1);
Minute 2:                                             seat1 = seats.Pop();
Minute 3:   seats.Push(seat2); backs.Push(back2);     back1 = backs.Pop();
Minute 4:                                             seat2 = seats.Pop();
Minute 5:                                             back2 = backs.Pop();
```

If person 1 follows the written order but person 2 attempts the reverse order, the following is only a hypothetical timetable of desired calls and completions, not a reachable execution. The minute-2 backs.Push and Pop completion and the minute-3 seats.Pop completion cannot occur; the actual blocked state is explained below.

```
             Person 1             Person 2
Minute 1:    seats.Push(seat1);
Minute 2:    backs.Push(back1);   back1 = backs.Pop();
Minute 3:                         seat1 = seats.Pop();
```

In the actual execution, person 1 remains blocked in seats.Push(seat1) from minute 1, while person 2 waits in backs.Pop(). With zero-capacity channels, neither operation can complete without its matching peer. Person 1 therefore never reaches backs.Push(back1), and person 2 never reaches seats.Pop(). The minute-2 and minute-3 events in the hypothetical timetable are unreachable; the system is deadlocked.

This example directly corresponds to the ordering rules related to message-passing operations in the *basic conceptual model* within this document, and in particular the specific rule that disallows reversing the order of message-passing operations.

Appendix B – Formal Presentation of Pipelined Loops

Modeling convention for pipelined-loop overlap.

Pipelined-loop body no-wait condition. The formal pipelined-loop model in this appendix applies only to HLS-pipelined loop bodies that contain no wait/ac_wait statement, SyncChannel/ac_sync call, or other explicit synchronization interface call inside the loop body. All ordinary synchronization calls selected into S must be outside the pipelined loop body or at explicit boundaries around the pipelined region. This is a well-formedness condition for applying the Appendices G-K theorem stack to a pipelined-loop instance; an in-loop synchronization call requires source rewriting, non-pipelined/preserved-loop treatment, or a separate proof/certificate outside this appendix.
The post-HLS pipelined-loop implementation may overlap iterations and may internally accept/stage up to the finite bound B(c_in) of input values for younger overlapped iterations before older iterations' outputs are produced. This internal accept/stage behavior is part of the back-annotated channel realization counted in B(c_in) between the selected producer and consumer boundary points. Here B(c_in) is an effective abstract-boundary storage/credit bound, not a claim that the RTL contains a single physical FIFO named c_in. In the ordinary Sys_B case, FIFO availability completely determines the complementary ready/valid signal at that boundary. In Sys_B^elab, the same finite storage/credit is represented by explicit bounded channels and any additional mutual-stall behavior is represented by declared stateless combinational ready/valid couplers; those couplers may be located at an internal elaborated boundary or at the process-facing outer boundary selected by E. The proof obligation is that committed transfers remain E4-legal and that the settled ready/valid behavior used by ChannelEnabled/ChannelDisabled is exactly the behavior declared by E. Internal movements that do not cross an explicit abstract boundary remain ε-steps. Σ records only the committed transfers at the selected observable boundary. In particular, the consumer-side boundary Pop_{c_in} is the loop-body Pop_{c_in} operation from the pre-HLS source when that Pop is included in the chosen Σ / Σ_DUT projection. The externally relevant effect of pipelining is therefore captured by the declared bounded storage together with the explicit Sys_B^elab handshake network, without introducing duplicate outer Σ-visible Pops. After Core.16 activation, the implementation obeys its no-new-later-source-work restriction, including preparation and request assertion for younger iterations. Physical storage movement is distinguished from source launch by B.BoundaryBinding below.

B-faithfulness obligation

The use of B(c_in) in this modeling convention assumes that the selected abstract boundary and the reported capacity faithfully summarize the corresponding RTL storage/credit realization: all state that can accept or hold messages is either counted in B(c_in) or represented as a separate explicit Sys_B^elab finite-capacity channel, and movements inside a represented storage element do not create, delete, duplicate, reorder, or relabel the selected boundary transfers. In Sys_B^elab, B-faithfulness additionally requires the declared stateless ready/valid coupler network to match the RTL boundary handshake behavior after same-cycle settling. A coupler adds no capacity and is not a new process; it may gate a complementary ready/valid signal only through logic explicitly declared in E. This is a tool/RTL-flow compliance obligation, not a fact derived by Appendix B. For a qualified Core.LogicalPopPayload endpoint, the no-relabeling obligation is judged on its prospectively defined logical event label, fixed by the matched actual admission; it is not permission to relabel a physically sampled observation or to choose a value during projection.

Multi-input Pop points and coupler policies.

When a pipelined loop consumes from multiple input channels (e.g., c1,c2), back-annotation still assigns finite storage/credit bounds and cycle-boundary occupancies to the explicit storage elements that represent those paths. In the multi-input / coupler cases of Appendix Q the formal source-side proof target is Sys_B^elab of Definition Core.1: a finite network of explicit bounded/rendezvous channels plus stateless combinational ready/valid couplers. ChannelEnabled/ChannelDisabled is evaluated from the settled handshake at the selected boundary. A conjunctive/join dependency is therefore not modeled as extra state or as an unexplained predicate; it is the ordinary consequence of the declared combinational ready/valid equations of E, which may include a coupler at an internal boundary or at the process-facing outer boundary. For the detailed elaboration patterns and obligations, see Appendix Q.

Practical note (outer Σ_DUT projection and proof-internal Σ)

The loop-body Pop_{c_in} in the pre-HLS source (i.e., the consumer-side Pop_{c_in} operation, conceptually after the in-flight capacity B(c_in)) may be exposed in the outer DUT/testbench-visible alphabet Σ_DUT if desired, but is typically omitted since it is latency-insensitive and may be difficult to expose directly in RTL. The completion reasoning still accounts for this blocking point without requiring it to be Σ_DUT-visible: K.CT requires its internal-fact correspondence and the applicable resource, request and persistent-enabling obligations. Actual channel readiness uses the settled ChannelEnabled/ChannelDisabled value. In ordinary Sys_B this reduces to the familiar E4 occupancy/rendezvous test. In Sys_B^elab, the same value is obtained by settling the finite explicit FIFO/rendezvous/coupler network declared by E. Any proof-internal channel used to account for storage remains explicit even if its committed events are omitted from Σ_DUT. In particular, omission of the loop-body Pop_{c_in} from DUT observations does not omit it from the complete outer-channel accounting history: Q.1's π_Σ,E retains both Push_{c_in} and Pop_{c_in}, and the later restriction ρ_DUT selects Σ_DUT. Occupancy is computed before that restriction. This observation choice remains subject to the whole-atomic-group requirements of Q.1(c-order) and J.OuterCanonHist.

Blocked-frontier restrictions and permission to drain

The following Core.16 restrictions describe what admitted work may do after frontier activation. They do not require useful stage movement and can be satisfied by a controller that makes no downstream progress. B.OF below separately requires ordered drainage, legal entry, finite preparation and request assertion for the standard pipelined completion profile.
Here the drain-only discipline does not mean a global pipeline clear or a mandatory drain-to-empty phase. It is the implementation of Core.16 used by constructive correspondence (C1) and K.CT. At the source phase point where the minimal pending blocking frontier is fully channel-disabled after settling, Core.16 activates: no new later source work may launch or issue past that frontier; asserted blocking requests persist; already-admitted work may drain only under Core.16(5) and the actual source phase and request rules. The operational definition governs activation and release. For ordinary non-overlapped control these restrictions may be discharged structurally. A tool's use of the term automatic flush supplies neither this conformance nor the stronger B.OF progress requirements by itself.

Terminology (pending blocking frontier / Front_P(σ))

At a settled boundary-evaluation cutpoint, consider the set of pending source-level blocking message-operation frontier items x ∈ Ω_msg,P in the current interval between adjacent synchronization calls, where each such item has an owning dynamic message-call instance q = MsgOf_P(x) whose Issue(q) has

occurred and whose Commit(q) has not yet occurred. By the Issue/Commit linkage (Appendix C) together with BoundaryReq's definition at the Σ-visible boundary (as given in "Normative Formal Core for Appendices G–K"), each such pending blocking frontier item x has its endpoint-owned boundary request asserted through q (Push: vld, Pop: rdy) until q commits; purely combinational channel-side structure (including couplers) may affect only the complementary mirror signal (Push: rdy, Pop: vld) and thus whether x commits in that cycle after ε-settling. Define Front_P(σ) to be the set of such pending blocking message-operation frontier items that are minimal with respect to the source-induced order ≤_P within that interval. By Lemma Core.OrdFrontSingleton, an ordinary non-elaborated source interval has at most one minimal pending blocking message-operation frontier item. If Front_P(σ) has multiple message-operation members, that multiplicity must come from explicit Sys_B^elab proof-internal structure such as staged/bundle/join/epoch-gate abstract channels; it is not inferred merely from the fact that ordinary source calls use distinct interfaces. We refer to Front_P(σ) as the pending blocking frontier (or "minimal pending blocking frontier"). This Front_P(σ) notion is distinct from NextFront_P (Definition Core.6), which denotes the next source-level frontier-item set over Ω_P and may include synchronization-call items, signal-anchored action items, and message-operation items. A message-operation item in NextFront_P may be pending before it contributes any committed Σ-event.

A pipelined implementation satisfies the following blocked-frontier restrictions if, whenever the pending blocking frontier Front_P(σ) is non-empty and every frontier item x ∈ Front_P(σ) cannot make progress because it is not channel-enabled (evaluated after combinational ready/valid/backpressure has ε-settled for the cycle), then:

1. **Block point.** The process is blocked on message passing at that pending blocking frontier Front_P(σ), as in the unpipelined source.
2. No new later work. While blocked at that frontier (i.e., while Front_P(σ) remains non-empty and all of its members are channel-disabled), the implementation does not issue any new blocking Pop/Push message-operation frontier item y ∈ Ω_msg,P that is later in source order than some x ∈ Front_P(σ), including any younger overlapped iteration, and therefore does not begin asserting BoundaryReq(y,·) for such later work. Here issue is read through the owning message instance q_y = MsgOf_P(y). Here new work means abstract source launch, including the broader preparation restriction of Core.16(2). The blocked-frontier restriction does not require all physical computation or private register updates to stop: abstractly invisible physical computation is permitted subject to Core.16's effect and later-use conditions.
3. **No withdrawal.** While blocked, the implementation does not withdraw any already-asserted blocking Pop/Push request (no "try-and-withdraw"); outstanding blocking requests remain asserted until their owning message instances commit.
4. Clarification (frontier minimality versus other asserted requests). Core.16 activates on the minimal pending blocking frontier Front_P(σ). An already-asserted request outside that minimal set—for example, younger work admitted before activation—does not prevent activation merely because it is enabled. Such requests persist under the no-withdrawal rule and may drain only under Core.16(5) and the actual phase/operation rules. The frontier restriction does not imply a general exclusion of internal wait cycles. Such source-later completion is distinguished from downstream drainage in the following clause.
5. Permission for already-admitted progress. Distinguish downstream drainage from pre-issued younger completion. Downstream drainage advances or completes already-admitted work physically downstream of the blocked frontier; where the boundary binding identifies it with source-earlier work z <_P x, that source/stage correspondence is explicit. Separately, a source-later y with x <_P y may complete if its request was legally issued before Core.16 activation, including in the same L1 pre-settle window preceding the L2 snapshot. That is completion of

pre-issued younger work, not evidence that y is older downstream work. Both cases obey Core.16(5), actual phase/source legality, endpoint ordering and the preservation/frame requirement below. While the frontier remains nonempty and fully disabled, no new source-later evaluation, ReachPrep or Issue may launch work past it; the non-retroactive permission does not reopen L1 or waive that prohibition. Qualified abstractly invisible physical activity retains Core.16's separate effect and later-use conditions.

6. Preservation of stalled work. Apply the authoritative preservation/frame requirement in the Core.16(5) clarification to every pre-issued younger completion. The Appendix B boundary/controller binding must discharge it for the actual storage realization; this clause adds no separate frame law.
7. **Finite explicit boundary storage/credit**. Every selected abstract boundary that can hold or credit already accepted work has a finite declared capacity B(c), or is represented by an explicit finite Sys_B^elab boundary chain whose component capacities are finite and B-faithful. Together with "no new later work," this makes the set of drainable work finite and non-replenishing.
8. **Synchronization-boundary withdrawal of future-epoch acceptance.**
   Let s be an explicit synchronization interface call that separates the current source interval from later work. While the process is pending at s, the implementation shall not present producer-facing availability for any back-annotated capacity intended only for suff_S(s); equivalently, any asserted upstream producer request targeting such capacity shall remain channel-disabled until s commits. This does not prevent drain of work already accepted before reaching s. **Formal elaboration requirement**. For purposes of Appendices J–K, this withdrawal shall be represented in Sys_ref by the elaborated source model Sys_B^elab, not by adding a hidden extra enablement predicate to an otherwise E4-enabled outer channel. Sys_B^elab shall introduce explicit sync-boundary / epoch-gate abstract channels, or equivalent explicit staged realization structure, so that capacity usable only by suff_S(s) is separated from capacity available to the current interval. While s is pending, the gate/boundary for future-epoch acceptance is channel-disabled in the ordinary E4 sense at that explicit abstract boundary. After s commits, the corresponding post-sync capacity may again become producer-facing available according to the ordinary ChannelEnabled rules for the elaborated channels.

General Core.16 blocked-frontier and drain-only obligation

Core.16 applies the drain-only blocked-frontier discipline to every admitted source interval, including pipelined overlap and dependency-independent eager issue. Ordinary source-sequenced calls remain ordered. Already-admitted work includes the explicitly represented storage, requests and atomic action units of the selected Sys_ref elaboration. Appendix B supplies implementation progress contracts; a constructive correspondence still proves actual source legality and preserves the attributed request and admitted-work state at each local extension.

For pipelined or otherwise overlapped control, bind the implementation to the applicable Appendix B laws and prove the C1/C2 activation and C3/C4 source-extension cases. Ordinary non-overlapped control may discharge no-new-launch structurally. Core.IC supplies request persistence; E4 and faithful explicit elaboration supply storage and credit accounting. No inference of eventual completion follows merely from permission for already-admitted work to drain: K.CT separately consumes exposure, persistent enabling and service.

B.BoundaryBinding — process sites, channel storage and physical movement

For each pipelined binding, prospectively identify the process-facing Pop/Push sites, dynamic operation keys, actual process-owned request predicates and retirement events. Identify physical reads, writes

and storage movements separately, including their state ownership and any proof-internal abstract boundaries. Establish Core.10/Core.IC request identification, E3 prefix-closed issue order, Core.16 activation/preparation legality, Core.RegSeq and E4 at those selected boundaries. A physical read need not be a source Pop, but classifying it as channel movement requires a faithful operational elaboration and projection; it cannot hide a source operation or choose an Issue timestamp after observing the execution. Account for every accepted unmatched item once. Distinguish nominal reference storage B, physical queue capacity F, attributed slots, credit C and the separately proved witness floor g. Latency or stage count alone establishes none of these quantities. In particular, the last source-ordered message issue of iteration j cannot follow the first message issue of iteration j+1. For a regular unstalled schedule with fixed per-iteration issue offsets and interval II, this implies last_offset−first_offset <= II. In the single-Pop/data-dependent-Push example, Core.RegSeq also requires at least one cycle between the Pop's return and the Push's use of it, so II=1 places their source-facing issue opportunities one cycle apart when neither stalls. This is an issue-time restriction, not a universal equation between physical stage numbers: stalls, delayed assertion, legal groups and the chosen boundary mapping still require the actual E3 proof.

B.OF stage occurrences and completion/exit refer to this bound process organization, which need not include every physical register in the datapath. If a source-facing Push accepts into an explicit output channel, subsequent transport is governed by that channel's declared resource and service laws, not an unconditional process-stage exit promise. If the physical write itself is the selected Push, its stage is incomplete until that Push retires. B.OF.Drain never removes an uncommitted operation or assumes that an unavailable consumer will accept it.

Worked boundary example (four physical stages, II=1 only under the sustained-flow conditions below). Consider the no-wait source loop that Pops one input and Pushes that same value to an output. Suppose the physical input read is at stage 1 and the output request cannot assert before stage 4. Identifying those physical sites directly with source-facing Pop and Push issue permits Pop(j+1) to issue before Push(j), contrary to E3. Neither an II/latency declaration nor a larger B(c_in) repairs that issue-order failure. The binding below specifies a different, restricted controller organization; it is not a claim that every four-stage pipeline already implements it.

Input-side organization for the example. Take a physical input FIFO of F>0 entries, followed by two one-item staging locations at stages 1 and 2. Keep the unmodified input value and its unique FIFO ordinal throughout this storage. Select the process-facing Pop at stage 3 and the process-facing Push at stage 4. The Pop removes the oldest staged value into process state; the Push uses that returned value no earlier than the following cycle, respecting Core.RegSeq. There is no extra output-side channel in this choice. The controller must implement E3 issue-prefix gating: Pop(j+1) cannot assert before Push(j) has asserted, and Core.16 must permit its preparation and assertion. In an unstalled interval Push(j) at stage 4 and Pop(j+1) at stage 3 may issue in the same cycle, which meets the cross-iteration issue-order condition for II=1. Sustained one-item-per-cycle operation additionally requires F>=2 under the old-state ingress rule, the declared simultaneous-replacement staging option, continuously supplied input and accepted output, and compatible process-stage advance/retirement that selects the full shift each cycle. Use the immediate, non-throttling guard specialization and no additional credit limitation for this example. With old-empty-only staging or F=1, this sustained rate is not asserted; an initial burst is not a sustained-rate proof. This is a specified controller contract, not retrospective attribution of arbitrary RTL signals.

Storage and floor for that organization. All input items in the FIFO and stages 1–2 remain accepted and unmatched until the stage-3 Pop. With no extra credit cap and no other storage, their total storage bound is B(c_in)=F+2; after that Pop, the stage-3/4 value is process state. Use Sys_B^elab with the actual FIFO and staging/handshake equations: the sum F+2 does not imply ordinary single-FIFO enabling. Stage-1 physical read is early capture relative to the selected Pop. This organization uses

Q.CT.InputStaging rather than B.CT.Capture's at-site law. Any integer 1 <= g <= F is a sufficient guaranteed floor here. Use B.CT.EndpointOrder’s zero-based k. Before Push(k), when k>=g, Pop(k−g) and all earlier Pops (k−g+1 items) have retired, leaving at most k−(k−g+1)=g−1 older unmatched items; when k<g, at most k<=g−1 older items exist. Physical FIFO occupancy is no larger than that inventory and is therefore <F; items already staged do not also occupy the FIFO. Q.CT.InputStagingProgress supplies the persistent-space and isolated-item-delivery proof. A smaller-capacity witness still requires K.CT.Capacity's source-legality lift; the resource arithmetic alone does not supply that lift. The one-item staging locations are internal state of this declared channel realization, not automatically separate capacity-one E4 primitives. Q.CT.InputStaging's edge law specifies whether they can be replaced on the same edge. Primitive ingress and outer boundary legality remain unchanged. Where this reference permits the relevant moves as optional abstract actions and all eligibility conditions hold, K.CT.LazySimulation supplies an alternative completion-witness selection on the same structure; it does not change the storage bound or establish the sustained rate described above.

Forwarding under isolation. When the matching Pop is legally pending at its site and is isolated in the sense of B.CT.Guard, its accepted item is the oldest remaining item and the storage positions ahead of it towards that site are free, Q.CT.InputStaging requires each needed internal forward step to have a legal opportunity and receive declared internal service. Forwarding does not wait for new upstream data, for the target Pop to have already retired, or for its item to have already arrived at the Pop site. Each step preserves ordinal, payload and unique ownership and reduces the finite remaining distance. No younger item can overtake or occupy a destination ahead of the target. Q.CT.InputStagingProgress proves arrival and retention from these local laws. Outside isolation the clause imposes no forwarding eventuality: a non-compressing controller may hold this storage while older incomplete process work remains downstream. For the standard staging completion proof, establish target placement, legal preparation and persistent operation-owned BoundaryReq assertion without assuming that target item’s traversal to the terminal location or its retirement. Earlier completed control/value predecessors remain available. These are proof dependencies: earlier forwarding is permitted, but the arrival conclusion cannot be used to establish the placement/request premises of the same delivery lemma. A data-gated transfer pulse is not a substitute for the attributed pending request of B.OF.Request.

Bubble-compression case. Movement into an empty stage-1/2 channel location may continue while an older source frontier is blocked, provided it is lawful channel movement under the fixed elaboration and respects any epoch gate. It does not launch a new process iteration or assert its stage-3 Pop. After Core.16 activation no such new later source preparation or request is permitted. A younger request already legally issued in the same L1 window before activation may persist and complete as pre-issued younger work under Core.16(5) and the stalled-work frame invariant; an unissued one must wait. Bubble movement of already admitted process occurrences likewise preserves their requests and sites and authorizes no new source work. A controller that actually asserts the younger stage-3 Pop after activation fails this binding. B.OF.Drain's permission for bubble compression does not override Core.16.

Profile status and alternative output-side mapping. The restricted early input storage in this example is now a named sufficient component, Q.CT.InputStaging, with resource and delivery proof Q.CT.InputStagingProgress. It requires the component's complete operational and boundary conformance; it does not qualify arbitrary early capture by stage count. B.CT.Capture remains the separate at-site alternative. The direct output Push uses its declared output channel's own capacity and floor. Moving the selected Push earlier instead requires a separate explicit output-channel realization, sufficient available payload under Core.RegSeq, faithful acceptance/storage accounting and the corresponding forwarding/exposure/service proof through Q.CT.Extension unless another stated component applies. Downstream consumer blockage cannot be erased by calling that channel completed process work. E3 and Core.16 are unchanged. For a computed input path that no longer physically retains the original word, Core.LogicalPopPayload and Q.CT.LogicalPayload provide the

singleton representation qualification and the restricted positive two-Pop qualification Q.CT.LogicalPair. The source Pop still returns the matched admitted word; private computation must be accounted for at its legal later use. This is not a transformation of the logical channel item or permission to change the selected source-issue order. The pair profile selects independent paths, equal-length Q.CT.CoupledInputStaging, or fixed-offset Q.CT.StaggeredInputStaging, with Q.CT.InputStagingProgress, K.CT.CoupledDeliveryLemma or K.CT.StaggeredDeliveryLemma respectively. The staggered alternative concerns physical reads; the logical Pops still retire together at one terminal site. Unconditional global stall is not admitted. After a qualified join, Q.CT.PairedRepresentation permits joint private computation without changing the logical items or source-issue order.

B.OF Ordered automatic-flush progress

B.OF.Scope. The ordered automatic-flush profile applies to HLS-pipelined loop bodies satisfying this appendix's no-wait condition and the obligations below. These are additional normative conformance requirements for an implementation claiming this profile; the no-wait condition and the preceding blocked-frontier discipline alone do not establish them. The requirements describe behavior, independently of a tool, directive name, pipeline layout, initiation interval or chosen bubble-compression policy. Under B.BoundaryBinding, a binding shall identify the finite stage structure, iteration and operation identities, entry/exit and retirement events, local computations, guards, capture policy, and declared internal service assumptions. No initiation interval, nested-loop, variable-trip-count or reset behavior is qualified merely by naming this profile.
B.OF.Control. For a completion application, permitted iterations, required stage members and legal reach/operand eligibility refer to the selected source control and operation rules under the fixed fact attribution, not to an implementation-selected subset of operations. B.CT.Control binds their implementation realization. An implementation cannot discharge entry, computation or request obligations by silently omitting an operation that those source rules require. Actual reach still requires the finite legal preparation and placement proved below; agreement of data alone is not physical reach.
B.OF.Order. Under B.OF.Control, iterations enter in source iteration order, without duplication or omission of any permitted iteration. Older live iterations remain ahead of younger live iterations in the stage order; younger work cannot overtake older work. An unretired operation cannot pass its required site. Within an iteration, stage order respects the selected reference's control and operation dependencies. Reordering independently prepared members of one partial-retirement stage requires the separate source-legality qualification in K.CT.Preparation.
B.OF.Complete. A stage occurrence is complete only after all of its source-required I/O members under B.OF.Control have retired and its required finite local computation has finished. Partial retirement retains correctly attributed completion state for each member until stage advance; already-retired members are not reissued. Atomic retirement uses the declared indivisible group. Under the declared internal service assumptions, continuously eligible finite local computation actually finishes. B4/FPD's finite bound alone is not a substitute for this execution requirement.
B.OF.Drain. If a stage occurrence is complete and no incomplete occurrence lies downstream of it, it eventually advances under the declared internal service assumptions; a complete final-stage occurrence eventually leaves the pipeline. This obligation is independent of upstream readiness and of new work being supplied. Blocking may propagate upstream through empty stages. An implementation may move complete work into empty locations upstream of an incomplete occurrence (bubble compression), but is not required to do so. Thus an empty next location alone does not impose an advance obligation. The final-stage exit is the departure of a complete bound process occurrence. Delivery of an item already owned by a separate output channel follows that channel's contract; an outstanding selected output Push still makes its own occurrence incomplete.

B.OF.Upstream. Upstream incompleteness shall not suppress the downstream drainage required by B.OF.Drain. A controller that can indefinitely freeze such completed downstream work solely because upstream work is blocked does not conform to this profile. This excludes that global-stall behavior, regardless of the controller's product or mode name.
B.OF.Entry. If source-aligned loop control under B.OF.Control continuously permits the next iteration, its launch remains legal under Core.16 and the applicable source-preparation rules, and no incomplete occurrence remains in the pipeline, that iteration eventually enters under the declared internal service assumptions. Completed residual work drains under B.OF.Drain so that finite occupied storage cannot indefinitely prevent this entry. This clause authorizes no new work past an activated frontier and no inference that a data-dependent iteration is permitted before its control condition is resolved.
B.OF.Request. When an operation's iteration is at its site, the source-aligned operation is legally reached under B.OF.Control/B.CT.Control and remains eligible for issue, and its required operands are available, the implementation eventually asserts the correctly attributed request under its declared internal service assumptions. Assertion of a blocking request shall not require the peer's complementary request merely to make that assertion possible. Endpoint serialization, Core.RegSeq, Core.ReachPrep and Core.16 remain applicable. Once asserted, the request and its required stable descriptor persist as required by Core.IC; persistence of the request does not by itself prove persistence of the complementary resource or guard. The bound request may be an internal operation-owned condition, qualified by Core.10 and Core.IC. A physical transfer pulse or a raw ready/valid signal gated by channel availability is not by itself that request. Map the process request, declared guard/coupler and actual transfer equation separately; the mapping shall preserve issue timing, persistence and attribution, rather than manufacture an earlier request to satisfy E3. For early-staging uses of K.CT.Standard, the placement/request proof obeys B.BoundaryBinding's arrival-independent proof dependency; a terminal valid bit that can become true only through the delivery being proved cannot itself discharge that dependency. Core.TransferLaw binds every guard and resource predicate to the actual settled transfer signals. A persistent internal operation-owned request may coexist with a withheld physical handshake, but the mapping must expose the cause; no guard can veto an actual overlap after sampling.
B.OF.Service. The eventualities in this profile are evaluated over executions meeting the declared internal service assumptions. A binding must discharge these assumptions by the actual clocked update law, a proved finite local realization, or an explicit service contract maintained until required completion. Pre-grant arbitration fairness in B2 shall not be silently extended to arbitrary internal stage movement or post-handshake selection. The declared service law supplies the relevant internal or arbitration progress; the binding must also establish the opportunities required by the preceding clauses. Clocks, local updates and modeled environment service cannot be stopped prematurely to avoid these obligations. Shared functional units and their arbiters are included when finite computation relies on them: prove opportunity and non-starvation for the required older work under surrounding admitted activity. A nominal modulo schedule alone does not discharge this obligation after stalls. A next-edge assertion or a proved finite latency bound is a sufficient realization where applicable; determinism alone implies neither, and the profile does not impose a uniform cycle bound. Any explicit or equivalent service contract must name the actual action or opportunity and its independently justified request, support, clock/reset and execution-domain conditions. Continuous-enabling fairness, qualified recurrent-opportunity service, bounded response, or a proved finite local realization are admissible when their stated antecedents and operational effects are established. The contract must not assume R, eventual required outputs, or the unit-completion conclusion it is being used to prove. "Until R" bounds the duration of the obligation; it is not an assumption that R occurs. For the K.CT completion discharge, the service-dependent uses of these eventualities follow K.CT.UntilRScope; no service for unrelated trailing work after R is inferred. The operational and conformance invariants retain their stated domains.

B.OF.Target (informative pointer). K.CT.PlacementLemma derives eventual permanent target isolation from the applicable source-aligned control, closure, finite computation, order, entry, drainage and service laws. This pointer is not an additional conformance premise or an independent placement proof. K.CT.Alignment supplies the semantic control/descriptor result and conditional finite preparation before that placement argument; it does not assume PlacementLemma.
B.OF.Binding. B.BoundaryBinding and B-faithfulness bind the selected process sites, requests, boundaries, storage and settled handshake behavior. An accepted unmatched item before the selected Pop is channel storage, while a register after that Pop is process state; neither stage location nor a physical zero-depth link alone determines reference capacity. The standard completion profile additionally establishes B.CT and, for elaborated components, Q.CTQualified. K.CT.Standard then derives exposure from the operational laws. B-faithfulness alone does not establish those progress laws. Existing drain-only restrictions remain in force; B.OF requires useful drainage without an unconditional global drain-to-empty phase.

B.CT Completion progress profiles

B.CT.Scope. This section gives additional implementation requirements for the standard completion discharge in K.CT.Standard. They are behavioral requirements on the bound system, not properties inferred from an implementation's name, a no-wait annotation or B-faithfulness alone. The instance declares operation and iteration identities, endpoint order, the selected process-facing boundaries, explicit storage and credit, retirement groups, capture/advance behavior, synchronization regions and the service assumptions. Ordinary sequential owners satisfy the placement part by source-aligned finite legal control advance and request assertion under B.CT.Control; pipelined owners use B.OF with the same control binding. Requirements apply to the modeled testbench and environment participants as well as the DUT. Every consumed local law and invariant holds over all reachable states and admitted surrounding activity, including activity outside the finite witness.
B.CT.Control. Bind implementation control, iteration permission, required stage membership, operation keys and descriptors to the same prospectively fixed source rules and provenance evaluators used by K.CT.Agreement. This is an implementation-conformance requirement over every admitted reachable state and surrounding activity. When a target’s consumed control/value facts and placement prerequisites are complete, the corresponding operands are legally available after any Core.RegSeq delay, and launch/preparation is permitted by Core.16, the actual controller realizes a finite legal source-control/preparation path to that target’s site/issue eligibility with the descriptor prescribed by those rules. The path requires no new unresolved blocking operation outside the declared predecessors. Its local computation/preparation steps have actual opportunities: required support persists or a well-founded finite local measure advances under the declared internal service. The controller preserves the operation’s identity and required membership until retirement; it may neither skip required work nor replace it by an unrelated occurrence. Pipelined placement and entry remain governed by B.OF, and sequential owners have the corresponding finite control path. This local conformance law does not assume eventual boundary commitment or acceptance of R; it binds control realization and finite preparation. Completed-fact correspondence alone does not prove it. This conformance includes the positive and negative control/membership decisions in K.CT.CtrlSupport; agreement only for operations that complete does not justify omission of other required work. Pipelined owners also satisfy B.OF.Control.
B.CT.State. Each stage occurrence has an owner, iteration identity and ordered site, with retained retirement state for each member. B.OF.Order and B.OF.Complete govern placement, completion and advance. The channel state identifies every accepted unmatched item, its producer key, its intended consumer ordinal and its current explicit storage location; an item has one location, not several accounting copies. At the selected boundary, a Pop retires exactly once and a Push is accepted exactly

once. Initialization, transfer and any permitted epoch transition preserve this accounting and the selected FIFO order. The default proof uses initially empty channels and no affecting reset; initial contents require explicit initial-item keys and the corresponding amended dependencies.
B.CT.FIFO. A standard positive-channel realization has exclusive ordered producer and consumer endpoints, a physical queue of F>0 entries, optional finite slots reserved for attributed Pops, and an optional exclusive outstanding-item credit C>0. Let A be the number of committed Pushes whose matching process-facing Pops have not retired; A equals queue plus reserved-slot inventory. If no additional credit restriction exists, the test A<C is omitted. With the producer's earlier endpoint operations retired, its own resource predicate is queue occupancy <F and, when present, A<C. The next Pop's item is its reserved item or the corresponding queue head; capture and commitment use the declared B.CT.Capture law. No other producer, consumer, hidden shared credit, grant or capacity loss may change these equations. A guaranteed witness floor g is any positive integer with g<=F and, when applicable, g<=C. The largest floor justified by this law is min(F,C) when credit applies and F otherwise; a smaller positive g may also be chosen. Running the source witness at a smaller capacity is optional; the standard K.CT.Closure space-release dependencies in W always use the selected guaranteed floor g, whatever capacity the witness ran at. K.CT.Capacity permits a direct witness in the actual elaborated reference, but an application of the standard resource proof must still satisfy those floor-g dependencies and acyclicity; use of a smaller-capacity witness additionally requires its separate legal source lift. Nominal reference capacity and reserved slots do not by themselves establish freely usable queue space. The selected reference still represents the full actual storage and handshake law. Explicit epoch gates are separate Q.CT.Epoch components: their active support is an additional declared resource conjunct, not a change to queue/credit accounting. Other storage organizations require Q.CT.Extension or a separately stated resource lemma. K.CT.LazySerial supplies a distinct demand-supported resource proof only for its stated independent InputStaging profile. It does not relabel staging as a larger guaranteed floor or qualify arbitrary reserved-slot realizations.
B.CT.EndpointOrder. Retirement at each scalar endpoint follows its fixed dynamic ordinal order, across stages, overlapped iterations and regions. A later endpoint call cannot retire before an earlier call. Each edge contains at most one transfer at a scalar endpoint unless a separately declared multi-lane interface and ordering law is supplied. FIFO reads remove the oldest uncaptured accepted item and attribute it to its unique matching Pop. Capture may precede process-facing retirement, but cannot steal, relabel or overwrite another Pop's item. These are controller and channel invariants, not consequences of the chosen witness schedule. Ordinal convention for the empty-resource laws: dynamic per-endpoint transfer ordinals are k in {0,1,2,...}, beginning at the bound empty-channel initialization or a separately qualified affecting-reset epoch base. Matching Push(k) and Pop(k) use the same channel-epoch and item ordinal; the key retains the epoch, so restarting a local counter never renames historical facts. Ordinary configuration transactions and synchronization intervals do not restart this sequence. Nonempty initial contents, retained cross-reset items or an alternative global numbering require an explicit base/index mapping and amended resource law. This convention does not change other independently defined observation counters.
B.CT.Capture. This at-site capture profile captures only at the selected Pop site. A split capture removes the old FIFO head into that Pop's empty attributed slot; it is not yet the process-facing Pop. The slot and payload remain available until that Pop retires, and capturing does not release its process-facing credit. A coincident read-and-retire removes the same old head and completes the Pop on one edge without persistent intermediate occupancy. A declaration may allow split, coincident, or both forms. At an isolated pending Pop with its item available, at least one permitted form becomes an enabled service opportunity through the declared immediate or finite-response realization of B.CT.Guard; a split capture is followed by a held-item retirement opportunity. Slots cannot be revoked or reassigned while occupied. For capture, own resource means the matching old head and an empty attributed slot, not

readiness of the complete atomic group. Guards for obtaining an item shall not require the Pop's own missing item or unfinished partial-stage siblings. Earlier physical capture is not implied by this at-site law or by a finite-response guard. Q.CT.InputStaging, Q.CT.CoupledInputStaging and Q.CT.StaggeredInputStaging supply separate named sufficient early-input alternatives, each with its own ownership, delivery and compatibility requirements. Early-capture organizations outside the applicable clauses of those three alternatives require Q.CT.Extension. Bind a split capture and the later process-facing retirement as separate declared actions, with separate guard/resource and physical-transfer equations. Each ready/valid action obeys Core.TransferLaw; a genuinely abstract capture/update uses its independently realized movement law. Capture neither retires the Pop nor releases its process-facing credit unless the declared coincident form actually performs both effects.

B.CT.Guard. Define isolation of a target at its site to mean that no incomplete stage occurrence is downstream of it, its source-aligned control and operands are available, and all lower-ordinal operations on the same endpoint are complete. Separate owner-path, stage and older-iteration prerequisites are those of K.CT.Closure. A partial member need not await unfinished siblings. Declare one of two guard realizations. Immediate: while isolation and the member's own resource predicate hold, its commit guard and any required capture guard are true. Finite-response: once those supporting facts persist and the legally prepared request is pending, the concrete local guard realization has a natural-valued progress measure, finite at activation; while the guard is false, actual legal local steps eventually strictly decrease that measure under declared internal service, independently of the target's commitment or any unresolved blocking work outside its predecessors. At zero the guard is true. Surrounding admitted activity cannot replenish the measure, and the guard remains true while the support and request persist, until retirement. Establish this for every reachable activation of the guard, including the held-item phase after split capture. No uniform cycle bound is required. An unsupported assertion of eventual transfer is not a guard realization proof. The guard’s false value withholds the corresponding actual handshake through the declared signal equations. Once the settled handshake holds, the transfer commits by Core.TransferLaw; the guard cannot be checked again as a veto. Finite-response guard service establishes this prerequisite only, not all peer/resource prerequisites.

B.CT.Guard scope and composition. The finite-response option is an additional sufficient progress profile; the immediate implication remains the simpler case. Request persistence and completed captures/transfers still obey Core.IC and retained-state laws. Constant-true and completed-downstream guards have the immediate realization. Additional guard signals require their own realization proof. Atomic and rendezvous units require a realization for the single joint guard under simultaneous persistent support; separate recurring or alternating endpoint readiness is insufficient. A partial member may not acquire an unresolved-sibling condition. Delayed conservative flags additionally preserve E4 safety and the declared source NB decision law: stale permissive flags, hidden credit and a timing-dependent NB outcome are not admitted by this progress clause. The optional K.CT.ClockRefinement theorem retains its stated immediate canonical guard class; finite-response guards need their own state/refinement proof if that representation is used. Any arbitration-dependent support is defined before the arbiter’s own grant and qualified under the selected service contract; current grant denial cannot be used to make the required eligibility antecedent vacuous.

B.CT.Joint. The direct joint profile closes each declared same-stage atomic group under matching process-facing B(0) endpoints. A component is one indivisible commit unit. Each owner contributes members at at most one stage occurrence, or one sequential issue site, and positive channels occur at most once within the component. All strict owner, endpoint, value, synchronization and resource dependencies survive contraction; a strict dependency between component members is inadmissible unless it is explicitly legal internal preparation rather than a commit-before-commit requirement. The entire component's requests can be asserted independently of peer request assertion, from legally prepared descriptors. Each owner realizes any required same-window member batch before frontier

activation; independent eventual assertion of individual members does not replace that batch obligation. With those requests pending, the component's own resource predicate is the conjunction of its positive resources and all matched B(0) requests. Its settled enabling is that conjunction and a guard satisfying the joint form of B.CT.Guard, without an additional mutually recursive readiness equation or hidden grant. Commitment retires all members on one edge. A direct B(0) pair creates no queue or reserved item. A physical zero-depth link that feeds storage before the selected Pop must instead account for that storage under B-faithfulness; it is not a direct process-facing B(0) transfer. When this settled enabling produces the declared joint handshake, Core.TransferLaw commits the complete component on that edge. Coherent equations must prevent a proper subset of its physical legs from transferring; stateless coupling alone does not establish that fact.

B.CT.Compatibility. Partial and atomic retirement may differ by stage, but a member belongs to only one commit unit. Pure positive partial/atomic stages admit at-site split or coincident capture as declared. The standard mixed positive/B(0) joint profile requires positive Pop members to have been captured into retained slots before the joint commit; coincident capture of those mixed members requires Q.CT.Extension. Pure B(0) members never capture into slots. Old-done advance uses retirement/computation state from the old edge state; same-edge advance may additionally use selected member retirements, but may not consume newly returned data before Core.RegSeq permits it. Both preserve unretired operations and all retained slots. For an overlapped partial-member witness, the implementation realizes the same legally prepared pre-activation request batch required by K.CT.Preparation and retains its pending members; staggered assertion after activation is not a substitute. No independent choice of option names qualifies a combination that violates these laws. Positive-input delivery selects one applicable operational alternative: Q.CT.InputStaging for independent paths feeding blocking singleton units or pure positive same-site atomic groups under its composition clause; Q.CT.CoupledInputStaging for its equal-length same-owner, same-site positive pair; or Q.CT.StaggeredInputStaging for its fixed-offset physical-read pair with joint terminal retirement. Each retains its own ownership, topology, placement/request, forwarding, guard and service obligations, including the noncircular qualification of B.BoundaryBinding. Physical representation is qualified separately by Q.CT.LogicalPayload, Q.CT.LogicalPair and, after a qualified join, Q.CT.PairedRepresentation where applicable. These representation clauses add no delivery alternative, readiness permission or storage capacity. Neither the delivery choices nor representation qualifications automatically extend mixed positive/B(0) groups, retry units, epoch-gated staging or the optional canonical ClockRefinement theorem.

B.CT.Clock. At a clock edge all sequential participants read one current registered state and the input samples prescribed by Core.Phase/Core.RegSeq. The complete declared combinational handshake network settles according to its qualified equations before deriving the complete forced transfer set; rendezvous and atomic readiness may therefore be same-cycle combinational facts. All updates for that set and any separately lawful abstract internal actions are installed together; no post-handshake selector may omit a transfer. Each primitive non-fall-through FIFO tests old occupancy/head; a simultaneous read may free space for the next edge, but does not justify a Push that was disabled in the old state. Coincident capture/retirement uses the old head. Per-endpoint conflicts are excluded, queue updates remove the selected old head and append the selected Push, and retained-state updates occur exactly once. Stage movement preserves order and does not generate an extra commitment. Its declared old-done or same-edge convention respects B.OF. Control first enabled by a sync or restoration update cannot issue post-boundary work on that same edge. These laws define a synchronous relation directly; an interleaving representation additionally needs K.CT.ClockRefinement or an equivalent exact relation proof.

B.CT.Service. Every local preparation, capture, retained-item retirement, stage movement, entry, boundary reach and restoration action used in the progress derivation has an actual legal opportunity

and the declared internal service of B.OF.Service or an equivalent explicit execution contract. Every actual settled physical handshake commits under Core.TransferLaw or Core.SyncProtocol. Any service needed to obtain a grant or realize a genuinely abstract unit is bound under K.CT.Service for the entire indivisible unit. An enabled abstract action is not a service opportunity unless the implementation has a corresponding allowed edge or finite locally progressing realization. Infinite stutter at a continuously enabled required abstract opportunity is excluded; finite pre-handshake stalls remain permitted under the selected service law. A sampled physical handshake admits no such stall. These obligations hold until R completes, without requiring unrelated work to terminate. Safety refinement and request persistence alone do not imply this service contract. Finite-response guard updates are internal actions under this clause, distinct from boundary service; their opportunity and decreasing-measure proofs cannot be replaced by B2. Explicit execution contracts obey B.OF.Service's noncircular action/opportunity requirement, including its allowance for independently qualified recurrent-opportunity and bounded-response service.

B.CT.Sync. A unary wait or paired synchronization lies outside the no-wait body. Its participant identities, preceding blocking obligations, finite tail work and any anchored signal members are fixed. Completion of those preceding obligations permits a finite sequence of local computation, completed-stage drainage and boundary preparation to assert the matching pending request under internal service; this sequence requires no new blocking operation not already included among the preceding obligations. Early reach or assertion is permitted only if it preserves their legal drainage and satisfies Core.SyncProtocol: an asserted physical offer cannot later be withdrawn, and any offer authorizing changed input pins already protects all old users. Any additional anchor/readiness condition is determined by the completed declared prerequisites and persistent environment facts included in the instance, and becomes persistently effective by the declared immediate or finite-response joint realization of B.CT.Guard; there is no unrepresented readiness condition. The local guard realization establishes the prerequisites for exposing the irrevocable physical offers. Once the declared settled handshake holds at an eligible edge, Core.SyncProtocol commits the indivisible transaction on that edge. Guard service and request preparation precede that handshake; neither is an additional selection step afterward. Core.SyncFence and Core.SyncFenceOrder govern any same-edge preceding-message/sync combination, and Core.RegSeq governs use of results. A synchronization does not discard in-flight inventory or silently clear required requests.

B.CT.Restore. Future-region availability is represented by explicit epoch/gate state. Its closed state does not disable lawful drainage of already admitted older work. After the controlling synchronization and the declared antecedent local work complete, restoration has a natural-valued progress measure: while closed, local work eventually strictly decreases it under internal service, and value zero opens the gate. The measure is finite at activation and is not replenished by surrounding activity. Its decrease does not require traffic through the closed gate or later work dependent on that gate. There is no requirement for a uniform cycle bound. The open state remains effective for a pending protected transfer until it commits; a later withdrawal requires the properly ordered next boundary. Opening never fabricates FIFO space, changes stored item ownership, or authorizes same-edge post-sync issue.

B.CT.Conformance. Conformance may be established by a reusable proof over a prospectively declared implementation family plus instance bindings, or by a complete proof/check of one specified instance under Core.EvidenceRealization. Both modes establish every consumed law over all admitted reachable states and surrounding activity, not sampled runs. Bind B.BoundaryBinding, actual controller/environment, capacities, groups, clock/reset contract, declared guard realization and internal service. A tool/version/directive name alone is insufficient. A next-edge or bounded-latency RTL assertion is one sufficient discharge when its hypotheses and validity over the admitted domain are proved; it is not mandated for every controller. A successful instance does not prove family-wide conformance. K.CT.Standard remains independent of the implementing tool. Where

Core.LogicalPopPayload is selected, also bind the causal admission record, real retirement event, unique ownership, physical representation and legal-later-use proof of Q.CT.LogicalPayload; J.LogicalPayloadRefinement supplies only its stated correspondence cases. The observer must cover all admitted physical continuations without driving or restricting them.
B.CT.Conformance control evidence. A useful decomposition is: agreed historical control/value inputs and descriptors through the selected J full-fact and agreement interfaces; an actual source-aligned control/identity relation preserving required membership and Core.16 legality; and local opportunity/service or a well-founded realization for each required preparation/computation step. Enumerate every controller wait and bind it to the declared dependency, handshake, resource, guard or internal arbiter. Bounded local control latency can discharge a part of this record; agreement of completed values and a list of waits alone do not prove B.CT.Control's no-omission and actual-reach obligations.
B.CT applicability notes. Output-credit reservation at entry needs a declared law compatible with B.OF.Entry and the resource proof; it is not automatically covered by credit tested only at Push. Such a reservation can obstruct completion, but delaying a source-visible input alone does not establish a safety violation. Bypass/fall-through FIFOs require a replacement for the old-state B.CT.Clock primitive equations. Same-stage atomic I/O uses a legal group-serial or other qualified grouped witness, retaining all external dependencies, and may introduce real deadlock. A wait/ac_wait or paired sync inside the loop body is outside B.OF.Scope, including an otherwise familiar clocked while-loop form. Rewriting must preserve the intended synchronization and E5 behavior; deleting or moving a wait is not a general semantics-preserving transformation. Unsupported organizations use Q.CT.Extension, with all applicable consumers covered under Core.ExtensionConsumerCoverage.

Appendix C – Issue/Commit Terminology and Linkage

Scope / shared definitions

The definitions below are with respect to the Σ-visible endpoints of the chosen source reference model Sys_ref (Definition Core.1)—i.e., the ready/valid interfaces at which Σ records message-transfer events in the raw rendezvous case Sys, the ordinary bounded-FIFO case Sys_B, or the elaborated back-annotated case Sys_B^elab. All shared notions about (i) Σ vs. ε steps, (ii) BoundaryReq / ChannelEnabled / ChannelDisabled, (iii) non-blocking polls as ε on failure, and (iv) the B(c) / occ_c(t) (cycle-boundary, non-fall-through) interpretation of E4 are as defined in “Normative Formal Core for Appendices G–K” and are to be interpreted over the explicit abstract channels of Sys_ref. Appendix C uses Definition Core.8 for the formal Commit(q)/CommitCycle(q) meaning and Definition Core.9 for the formal Issue(q) meaning. It defines ReturnedAtCycle and records the Issue/Commit attribution and linkage conditions used elsewhere in the document.

Commit (message channel) — informal operational reading
Informally, on RTL_B at a mapped ready/valid boundary, a message commit is observed when an external clocked observer sees both valid and ready high in the same cycle, with the committed payload equal to the mapped data value seen in that cycle. This is the RTL-facing realization of the abstract boundary commit; for the G–K theorem stack, Definition Core.8 is authoritative and defines the abstract E4 commit transition, Commit(q) as the source operation’s process-facing commit occurrence, and CommitCycle(q) as its cycle index. Core.LogicalPopPayload separately permits admission-derived payload attribution at a qualified internal logical Pop outside the physically sampled observation interfaces; it does not change the actual retirement event or permit relabeling a visible bus value.

Issue (message channel) — operational restatement of Core.9
Issue(q) is the cycle index of the operational event in which the calling process begins requesting the transfer corresponding to the source-level message operation instance q at the chosen explicit abstract boundary endpoint. Scheduler/witness evidence may record that cycle for rules such as R4, E3, and E5, but the evidence records the operational Issue event rather than selecting an independent timestamp. Definition: ReturnedAtCycle (message result availability). ReturnedAtCycle(q,t) means that the process-visible result of message-call instance q becomes available at the cycle-t boundary. For a successful blocking or non-blocking call, this is the transfer-commit cycle. For a failed PushNB/PopNB call, no message commits, but the success/failure status is nevertheless returned at the cycle-t boundary and is therefore a cycle-t result. ReturnedAtCycle is an operational timestamp used for sequential-result availability. It does not authorize dependent source work in that same cycle.

Same-cycle rendezvous clarification. For a rendezvous channel B(c)=0, if a matched Push_c operation instance and Pop_c operation instance first assert their endpoint-owned requests in the same cycle t, and the ready/valid rendezvous is selected/observed in that cycle, then both operation instances have Issue = t and both committed transfer events occur in cycle t. There is no implicit extra cycle between same-cycle matching endpoint requests and the corresponding rendezvous commit. If one endpoint request was already asserted from an earlier blocking operation, then its Issue may be earlier than t, but the matched Push_c/Pop_c commit is still a same-cycle commit in cycle t.

Issue/Commit linkage (well-formedness conditions)

The list below is intentionally broader than Definition Core.IC: the request-lifecycle, attribution, serialization, and synchronization-boundary clauses restate Core.IC(1)–(5), while “Sequential-result availability” is the separate Core.RegSeq result-availability rule included here for operational completeness. Fix an endpoint direction at an explicit abstract boundary, and let req(t) denote the endpoint-owned boundary request signal in cycle t (valid for Push, ready for Pop), i.e., the process-owned request level captured by BoundaryReq at that boundary (per “Normative Formal Core for Appendices G–K”). A physical ready/valid level that represents complementary channel/storage availability or declared coupler logic rather than the process-owned request is not req(t) for this linkage rule. For each source-level message operation instance q on that endpoint direction:

- Boundary-request soundness (blocking requests are non-withdrawable). req(Issue(q)) = 1. If q is a blocking Push/Pop, then within its reset epoch, once issued it remains the unique outstanding request on that endpoint direction until it commits: req(t)=1 in every cycle t from Issue(q) through CommitCycle(q) (inclusive), with stable payload while asserted for Push. At every cycle-aligned state, a process-owned blocking req has exactly one live issued, not-yet-committed/reset owner; after that ownership ends, a continuing high physical level is not such a req unless the back-to-back rule below transfers ownership. If an affecting RESET occurs first, RW2 ends that dynamic instance rather than withdrawing it within the epoch. (This is the “no deassert-before-commit” discipline used throughout the document.)
- Stable attribution for a single outstanding source-level request. If req(t)=1 and no transfer commits on that endpoint direction in cycle t, then any continued assertion req(t+1)=1 is attributed to the same outstanding source-level message-operation instance q only so long as the requesting source-level operation remains that same outstanding instance. For blocking Push/Pop, this is the mandatory non-withdrawable case above, so attribution cannot change before commit. For non-blocking PushNB/PopNB, continuous assertion of the endpoint request signal, by itself, does not identify later executed non-blocking call instances as the same q; a new executed source-level non-blocking call creates a new dynamic instance $q' \in Q_{exec,P}$ for the owning process P, even if req does not deassert between cycles.

- Back-to-back request attribution under continuously asserted req. Suppose blocking q_o commits in cycle t and req(t+1)=1 on the same endpoint direction. Let q_1 be the next same-direction blocking source-level message operation in the same source interval between adjacent ordinary synchronization calls selected into S, if such an operation exists. If q_1 is that next operation, the continuing process-owned request is owned by q_1 beginning in cycle t+1, and Issue(q_1)=t+1 even though req did not deassert. The process-owned blocking request may not remain asserted but ownerless for one or more later cycles and then be attributed to q_1. Genuine scheduler/HLS delay before q_1 issues remains permitted by Core.SelectLatitude, but during such delay q_1's process-owned BoundaryReq has not yet begun; a high physical ready/valid value caused by channel-side storage, complementary availability, or declared coupler logic is interpreted by the fixed selected boundary/channel/elaboration mapping and is not an ownerless req. Non-blocking calls retain the separate dynamic-instance treatment stated below.
- **Same-endpoint blocking issue serialization.** Let q_o and q_1 be distinct blocking source-level message operations on the same scalar explicit abstract boundary and endpoint direction governed by the Core single-transfer-per-cycle endpoint constraint in one reset epoch, with q_o <_src q_1. If both issue, then Issue(q_o) < Issue(q_1). A scalar endpoint has one operation-attributed process-owned request direction at a time; two distinct blocking dynamic instances may not acquire that same request direction in the same issue cycle. This is the strict same-endpoint fact used by mechanization; R4/E3 retain their general non-strict form because they also cover distinct interfaces and non-blocking cases.
- Sequential-result availability (no same-cycle dependent continuation). Let ReturnedAtCycle(q_o,t) hold in a sequential process P. If a dynamically succeeding operation q_1 is reachable only after q_o returns, or if q_1's control choice, endpoint, address, payload, or other required operand depends on the value or success/failure status returned by q_o, then Issue(q_1) > t. This rule applies to successful commits and failed non-blocking returns, across distinct channels and endpoint directions as well as on the same endpoint. Same-cycle issue or commit remains permitted only when q_1 was independently reached and all required inputs, payload values, and control predecessors were available before the cycle-t boundary under Core.RegSeq; overlap between independently active pipeline iterations is not prohibited, but an unresolved NB result in an earlier iteration places any dependent later-iteration action in a strictly later Horizon.
- Sync-boundary discipline. If $q_o \in$ pref_S(s) and $q_1 \in$ suff_S(s) for an ordinary synchronization call s selected into S, then Issue(q_1) is strictly after s's commit in the same trace (i.e., Issue(q_1) > clk_pre(s) in τ_pre and Issue(q_1) > clk_post(s) in τ_post). If q_o is a blocking Push/Pop, q_o must process-facing commit no later than s; no issued-but-uncommitted blocking operation in pref_S(s) may survive the commit of s into the succeeding source interval. Same-cycle completion is permitted: q_o and s may both commit in cycle t when q_o's process-facing commit is part of the selected cycle-t boundary actions and is accounted for before the interval advance caused by s. Any boundary request assertion that persists while the process is still pending at s is attributed to the outstanding operation(s) in pref_S(s) and does not constitute issuance of any operation in suff_S(s). Under the pipelined-loop body no-wait condition, there are no in-body wait events that create additional sync-boundary cuts inside an HLS-pipelined loop.

For non-blocking message passing (PushNB/PopNB), a failed poll produces no Σ-event and is modeled as an ε-step, while a successful poll contributes the corresponding committed Push_c(v) / Pop_c(v) event; see "Non-blocking message-passing" in "Normative Formal Core for Appendices G–K" and Core.NBPollInterpretation. Each executed non-blocking call is a distinct dynamic source-level message-call instance $q \in Q_{exec,P}$ for the owning process P. Thus, if repeated PopNB/PushNB polls occur in later loop iterations or at later source-level call instances, they are distinct q's even if the endpoint request

remains continuously asserted and no intervening commit occurs. Stable attribution therefore applies to a single outstanding source-level request instance; it does not collapse distinct non-blocking dynamic call instances across iterations into one q. Successful non-blocking calls that commit induce message-operation frontier items $x \in \Omega_{msg,P}$ when their committed transfer is relevant to Ω_P, with owning message instances $q = MsgOf_P(x)$ in $Q_{\Omega,P}$. Failed non-blocking polls remain only in $Q_{exec,P}$ and contribute no Ω_P or $\Omega_{msg,P}$ item.

Appendix D – Modeling Latency-Sensitive Protocols

In some cases, designs or testbenches may use protocols or local control policies which are latency-sensitive. These situations are handled by isolating the latency-sensitive portions into small, self-contained parts, and then keeping the rest of the design and testbench latency insensitive.
A latency-sensitive portion is any local block of logic whose functional or externally visible behavior depends on cycle-level timing that is not represented by the ordinary latency-insensitive Σ interface. This includes ordinary signal-level protocols with fixed cycle timing, global interrupt/request ordering logic, non-blocking arbitration whose winner depends on request arrival timing, and non-blocking retry/timeout logic whose later behavior depends on the number or cadence of failed PushNB/PopNB polls before success.

For such logic, the intended methodology is not to treat the hidden timing dependence as part of the ordinary latency-insensitive source-core proof. Instead, the latency-sensitive portion should be isolated behind an explicit boundary. On one side of the boundary, the transactor or local protocol block may be modeled cycle-accurately, snooped, or otherwise verified with its detailed timing behavior. On the other side of the boundary, the rest of the system should see a latency-insensitive interface, such as message-passing transactions, synchronization events, or a properly handshaken signal protocol.

Consider a case where a signal-level protocol is latency-sensitive. The protocol will have specific timing behavior that it must meet. To handle this, we create a transactor that has two sides: the first side interacts with the signals and handles the detailed timing requirements, and the second side sends and receives messages with the rest of the system. The message-passing side is latency insensitive. The same pattern applies to cadence-sensitive non-blocking polling logic: if retry counters, timeout counters, or arbitration decisions depend on the number or cadence of failed non-blocking polls, that polling loop is part of the latency-sensitive local protocol block. Its detailed failed-poll cadence should be contained inside the isolated block or aligned by the Appendix L snooping mechanism; it should not leak as an unmodeled hidden timing dependence into the ordinary Appendix J latency-insensitive proof.
To properly model the detailed timing behavior, the latency-sensitive transactor or local protocol block is modeled at the cycle-accurate level in SystemC, or is otherwise covered by an explicit snooping/alignment argument. There is an example of such a transactor model in the following document and example:

https://github.com/Stuart-Swan/Matchlib-Examples-Kit-For-Accellera-Synthesis-WG/blob/master/matchlib_examples/examples/53_transactor_modeling/transactor_modeling.pdf

For purposes of the formal appendices, once the latency-sensitive portion has been isolated, the ordinary Appendix J proof is applied at the chosen observable boundary of that isolated block. The failed-poll cadence internal to the isolated block remains local timing behavior; the rest of the system observes only the latency-insensitive or explicitly modeled boundary behavior. A completion claim

additionally requires K.CT.Agreement for every consumed outcome/control/value fact. Timing-sensitive internal polling cannot silently determine the required workload; a Q.CT.Retry use must meet its observation-neutral logical-obligation and service contract. Isolating the block alone establishes neither condition.

Appendix E – One-Way Handshake Protocols

In some systems a one-way signal handshake is sufficient for reliable system operation. When using a one-way handshake, the assumption is that the missing handshake is effectively always true. Establish that assumption over the selected operating, stall and reset conditions, together with the applicable signal or transaction observation contract; simply omitting a handshake signal does not establish it.

For example, in time-domain signal processing hardware designs, signal processing HW blocks may input new samples at a fixed rate. The HW blocks are always ready to receive new samples on each clock, but they need to know if the samples are valid. These types of designs can be modeled with the one-way handshake dat/vld protocol demonstrated in this example:

https://github.com/Stuart-Swan/Matchlib-Examples-Kit-For-Accellera-Synthesis-WG/tree/master/matchlib_examples/examples/32_dat_vld

Appendix F - Modeling Guidelines and Design Constraints

The following are simple modeling guidelines that align with the modeling methodology described in this document.

1) Prefer to use Connections::In/Out + SyncChannel over signal IO and wait().

2) Prefer to use Pop()/Push() over PopNB()/PushNB().

3) When pipelining a loop for standard required-workload completion, require actual conformance to B.OF and the applicable B.CT/Q.CT laws, including B.BoundaryBinding. A flush directive (for example hls_stall_mode flush) is a tool-specific request, not evidence that the generated controller satisfies them. Check the generated realization or its qualified implementation family. Controllers outside this progress profile may still qualify for constructive safety only through that result's complete scheduling, channel/coupler and correspondence conditions. The standard early-staging proof requires placement and persistent request assertion to be established without assuming the target item’s terminal arrival. A controller whose only route to that placement/request is travel with the item requires the separately proved Q.CT.Extension pre-request route; a data-travelling valid bit alone does not determine applicability. The standard witness certificate also includes K.CT.CtrlSupport: complete entry, branch and required-work membership decisions, including negative decisions, with their historical support or independent invariant discharges. Check these applicability and certificate conditions before treating an accepting source run as standard completion evidence.

When using signal IO:

4) If modeling a cycle-accurate process, use `disable_spawn` and follow the example 53_transactor_modeling style and do not use Push/Pop in the process.

5) If modeling a *direct input*, use `hls_direct_input` and possibly `hls_direct_input_sync`, and only change the signal at allowed times.

6) If combining signal IO with Connections::In/Out in the same process, use proper signal IO handshake that does not rely on In/Out ports for process synchronization. Place signal IO operations very close to their wait() statements.

7) Unless you are modeling a cycle-accurate process, you should expect that latency will change between the pre-HLS and post-HLS models.

Design Constraints for Applying the Formal Guarantees

This section consolidates the design choices that most directly determine whether the formal guarantees in this document can be applied. It is written for hardware architects and HLS users rather than for the tool or proof implementer. A design can synthesize and function correctly while still lying outside the stated theorem scope if one of these behaviors is left implicit.

This section is informative. Each constraint restates a normative requirement located in the referenced sections; the referenced text is authoritative.

The architect is responsible for choosing interfaces, reset behavior, and communication structures that admit the required correspondence. The HLS tool and verification flow are responsible for constructing and checking the detailed process, channel, attribution, coverage, and other proof records. The architect does not need to produce those records, but the design must expose enough explicit structure for the flow to do so.

The first list gives constraints for implementation safety and observable correspondence. The completion section adds requirements for eventual acceptance of a declared finite workload. A design outside a stated sufficient profile may need explicit modeling, witness alignment or a separately proved extension; a failed sufficient check is inconclusive.

Design constraints for the safety and correspondence guarantee

These constraints support the Post -> Ref guarantee that the covered RTL behavior introduces no new observable functional behavior absent from the prescribed source reference model.

1. **Give every signal read and write an unambiguous synchronization anchor.** For a sequential process, place signal reads immediately after the synchronization call that defines their observation point and place signal writes immediately before the synchronization call at which they become visible. Branches must not allow one write to be associated with different later waits on different paths. Signal I/O inside a rolled or preserved loop needs an explicit per-iteration wait when each iteration is intended to create a distinct observation. A loop selected for HLS pipelining cannot use an in-body wait under this contract; move the signal interaction outside the loop, use a supported message-passing or direct-input mechanism, or do not pipeline that loop.

2. **Keep direct inputs stable, or update them through the supported synchronization scheme.** A direct input may be physically sampled later than its logical source-level read only when its value remains stable for the applicable reset epoch. If software or the testbench must change control inputs while the design is running, use the documented hls_direct_input_sync handshake and change the values only at the designated synchronization point. An ordinary Push or Pop is not a substitute for that update contract.

3. **Make timing-sensitive environment behavior explicit.** The ordinary safety result assumes that the same external behavior is selected from the same prior observable history, not from hidden differences in absolute cycle count. Testbench branches, watchdogs, stimulus changes, or pass/fail decisions that depend on raw latency must be isolated as latency-sensitive logic, made observable through an explicit protocol, or checked at RTL rather than assumed to transfer automatically from an arbitrary pre-HLS run.

4. **Treat non-blocking polling as an architectural proof-route decision**. NB-CF can simplify qualification when failed polls have no relevant effect on later observable behavior. Typical arbitration, retry counters, timeouts, backoff, and "work on B if A is unavailable" behavior need explicit treatment. For supported constructive profiles, such NB control may instead be handled by qualified local process and primitive evidence, actual operational decision binding, registered continuation, complete settling and source/RTL snapshot matching: see J.OrdinaryConstructiveComposition and, for supported elaborations, J.ElaboratedNBDecisionBinding/J.ElaboratedConstructiveComposition. Observable effects of failure do not alone require an Appendix L snooper under that route. Other supported approaches include a legal witness-alignment mechanism or an isolated and explicitly modeled latency-sensitive protocol block. None of these mechanisms automatically qualifies arbitrary polling code or guarantees reproduction of one chosen source test execution. Prefer blocking interfaces when the architecture does not require poll-dependent control.

5**. Qualify polling under the selected proof profile**. Repeated failed non-blocking polls are executed attempts, not a stable blocking wait. The constructive safety route may cover them when its actual decision/snapshot, registered-continuation, preparation and coverage contracts are proved; safety alone supplies no eventual success. For completion, Q.CT.Retry admits its qualified observation-neutral persistent logical obligation with the stated exposure and service conditions. This is not blanket admission of unbounded busy polling. Where B4/FPD is consumed, its finite internal-step or stable-wait requirement still applies; an infinite local ε-only spin cannot be justified merely by calling it a retry. Otherwise use a blocking operation or an explicitly modeled and qualified protocol.

6. **Expose every J.Mem cross-process shared-memory dependency through a supported proof boundary**. Synchronization alone can order two processes' memory accesses, but the present safety proof does not carry an independent global shared-memory state. Keep the existing J.Mem trigger: whenever one process's memory write can affect another process's control flow, payload/value expressions, or the identity or order of its later operations, every such dependency must be represented through covered proof-visible operations and the flow must establish that no other unlowered cross-process memory-carried path satisfying that trigger exists. Raw physical RAM transactions may remain outside Σ only under Core.MemFaithful. If caching, merging, splitting, or reordering changes the transaction stream of a memory carrying such cross-process value flow, use a qualified Core.MemProfile whose logical boundary is transformation-invariant, use the separately supplied memory-state extension, or treat the design as outside the selected constructive correspondence theorem. The qualified-library route is intended for reusable patterns such as the shared-memory classes discussed in Appendix H, item 1.

7. **Model the channel behavior that the implementation actually provides.** The default buffered-channel abstraction is a bounded FIFO whose storage legality is evaluated from beginning-of-cycle occupancy and which does not permit first-word fall-through at the selected abstract boundary. Internal RTL bypass/fall-through may be hidden inside the realization only when the selected abstraction boundary absorbs that behavior so that the resulting explicit positive-capacity channel still satisfies the Core.12 non-fall-through E4 semantics. If an empty-channel value pushed in cycle t remains observable as a Pop at that same selected positive-capacity boundary in cycle t, the ordinary B(c)>0 E4 model does not cover that boundary; use a separately qualified bypass/rendezvous elaboration template or choose a different abstract boundary that restores the stated semantics. Debug drains, hidden credit paths, side entrances, or other structure that changes enablement or transfer ordering must likewise be represented explicitly. Back-annotated capacity alone is not sufficient when hidden structure changes enablement or transfer ordering. The same boundary-discipline applies to mapped memories: enabling cache/merge/split/reorder beneath a memory boundary used by the proof requires Core.MemFaithful; when a reusable transformation-invariant boundary is needed, use a qualified Core.MemProfile.

8. **Use an elaborated model for coupled multi-input pipelines and other joined structures.** When acceptance on one input depends on another input, a bundle, join, shared stage, coupler, or epoch gate, do not interpret the outer capacity of each input as independent slack. The verification flow must expose the relevant staged, rendezvous, bundle, or join structure in the elaborated source reference model. A declared stateless coupler may also sit directly at a process-facing outer boundary; in that case B(c)/occ_c still describe storage legality while the settled ready/valid handshake determines actual transfer eligibility. The outer interface remains the user-visible boundary, while proof-relevant storage, handshake, and dependency facts may be carried by the selected Sys_B^elab structure.

9. **Give every dynamic reset a channel-consistent disposition.** Reset both endpoints of a live channel together with the channel state, or define an explicit flush, abort, cancellation, or end-of-epoch protocol that determines what happens to outstanding requests and in-flight data. A reset of only one endpoint cannot silently discard, retain, or reattribute a transaction across the reset boundary. The protocol must be represented in both the source model and RTL using ordinary observable actions or another explicitly verified reset-state correspondence.

Additional requirements for required-workload completion

For the default completion claim, specify the required testbench acceptances and use the implementation-bound source model. Record a finite legal accepting serial witness, or the separately qualified group-serial/overlapped mode. For the standard profile, construct the full prerequisite graph using guaranteed-floor space edges even when the source run used nominal capacity. Establish internal correspondence and conditional value agreement, source-aligned control under B.OF.Control/B.CT.Control with K.CT.Alignment, finite legal preparation, ordered pipeline entry/drainage, persistent support and the applicable arbitration/internal service under B.CT/Q.CT. A source pass does not prove implementation conformance, and failure of this sufficient certificate is inconclusive. No affecting reset or cancellation may remove a required obligation before acceptance without a separate preservation proof. These conditions do not assert a latency bound or strict deadlock freedom after completion.

Preferred completion workflow. Assess K.CT.LazySerial eligibility first. Where its conditions hold, prefer running the default serial workflow in lazy mode over constructing a separate standard guaranteed-floor certificate. Atomic groups retain K.CT.GroupSerialWitness; lazy movement does not convert them into ordinary Policy S. In the restricted profile, the witness discipline and rank lemmas derive the resource-

dependency ordering; the complete support inventory and all standing qualifications still require validation. The lazy restriction uses source demands from the pre-HLS run and introduces no per-stimulus RTL simulation requirement.
In a mixed design, apply lazy treatment locally to eligible channels through K.CT.LazyComposition and retain the applicable standard or separately qualified proofs elsewhere. Ordinary Push cases on selected channels still use the standard floor resource argument. A protected singleton witness extension may also return directly to K.CT.Standard. Ordinary unreserved FIFOs need no additional movement restriction; within the shared restricted serial/group-serial profile the resource constructions reduce to the ordinary floor argument.
Retain the standard route for components or witnesses outside lazy eligibility, and whenever a standard certificate is already available or preferable for the selected implementation. This workflow preference changes no theorem premise and does not make exact capacity matching plus an unrestricted Policy S pass sufficient. Failure of a lazy run or certificate is inconclusive; another qualified witness or route may succeed.
Non-blocking completion scope. If issue timing changes a consumed outcome, control decision or value between the witness and a compared prefix, K.CT.Agreement is not discharged. Isolation or snooping alone does not prove that agreement. Q.CT.Retry may represent qualified observation-neutral retries as one logical obligation; its own scope, exposure and service conditions remain mandatory. Standalone non-retry NB attempts are outside K.CT.Standard and require a separate extension proof. Constructive safety qualification of a polling process does not by itself qualify its completion witness.

Practical modeling checks for completion

The following checks help bind the completion workflow to its reference model and implementation. Their normative owners remain Core, B.CT, Q.CT and K.CT. They do not establish a separate strict deadlock guarantee or add progress premises to a safety-only application.

1. **Give each analyzed logical channel one producer and one consumer.** A shared bus, crossbar, multi-client queue, or resource with multiple requesters should be represented by an explicit arbiter process and point-to-point channels. This makes endpoint ownership and the actual handshake participants unambiguous. A custom shared-channel abstraction requires its own ownership and arbitration proof rather than being treated as an ordinary point-to-point channel.

2. **Represent multi-lane, burst, and multi-transfer resources explicitly.** The standard scalar-channel model permits at most one Push and one Pop commit per channel direction in one cycle. A resource that transfers several elements, lanes, or beats at once must expose that behavior in its abstract interface and capacity model. It should not be encoded as several hidden transfers on a channel that the proof treats as scalar.

3. **Keep blocking requests asserted and Push payloads stable until completion**. A blocking Push or Pop is a persistent request, not a speculative try that may be withdrawn when back-pressure lasts longer than expected. Request ownership must remain with the same dynamic operation until commit or an explicit reset. A separate nonterminal cancellation/withdrawal action belongs to a different modeled protocol and is not part of the ordinary persistent-request interpretation; completion across cancellation requires a separate obligation-preservation proof.

4. **Establish the declared automatic-flush progress laws.** After frontier activation, Core.16 forbids new later issue while permitting already-admitted work to drain. For the covered pipeline class, ordered occurrences, finite local preparation, entry and downstream drainage satisfy B.OF/B.CT under the declared service assumptions. No-withdrawal alone is not a drainage proof, and next-epoch capacity must not become usable before the corresponding gate permits it.

5. **Treat serial execution as a conservative completion test.** A protocol that requires a source-later request before an earlier blocking call commits may fail the default serial witness even when an overlapped implementation completes. Such failure is inconclusive. Use the qualified group-serial mode for declared atomic groups, or a separately qualified overlapped witness with legal preparation and issue timing; do not infer completion from favorable overlap alone.

6. **Use the declared capacities and boundary structure.** The source witness runs in the implementation-bound reference model or at a justified smaller capacity. An unbounded-channel run is not evidence for finite hardware. Standard witness space edges always use the guaranteed floor g, including when reserved slots make the nominal reference capacity larger. A passing run must still satisfy closure and every other completion premise. K.CT.LazySerial is the named alternative for eligible independent staging: a protected singleton extension supplies the standard floor certificate, while its demand branch replaces designated floor edges with the separately proved support dependencies. In a mixed LazyComposition instance, record selected-channel eligibility and trace compliance, retain all unselected-component dependencies, and establish acyclicity of the combined graph separately from completeness and exposure.

   Practical interpretation of Policy-S conservatism. A serial witness may fail when a protocol needs a source-later blocking request exposed before an earlier call completes. Use the actual back-annotated reference capacities and grouping, or a justified smaller-capacity witness; unbounded channels are not valid evidence for finite hardware. In K.CT.Standard, space dependencies always use the guaranteed floor g, even when the witness executes at a larger nominal capacity. A passing run still needs the stated closure, correspondence, agreement and progress qualifications. A failed run or cyclic witness graph is inconclusive, not a deadlock verdict. Qualified group-serial or overlapped witnesses are separate alternatives; ordinary Policy S is unchanged. The K.CT.LazySerial restriction applies only to permitted abstract moves and retains exact elaborated capacities; it does not make an arbitrary passing Policy S run a completion proof.

7. **Give resets a completion-preserving disposition.** A reset or cancellation affecting outstanding required work needs a separate proof that the required obligations, identities and acceptance condition are preserved. Matching reset-state behavior for safety does not itself prove workload completion. Without that extension, exclude affecting resets before acceptance in the declared completion instance.

8. **Model barrier, end-of-stream, abort and environment protocols explicitly**. Completion includes every peer and environment obligation needed for the required outputs. A peer that terminates before supplying required data or synchronization does not satisfy that contract. Timed or reactive behavior needs the actual environment correspondence and service assumptions; no hidden rescue or more cooperative testbench may be substituted.

9. **Include mapped-memory backpressure in the completion proof**. Core.MemProfile retains complete logical value/ordering correspondence and requires every hidden resource or service condition that can delay required work to appear in its exposure and service qualification. Hiding raw RAM transactions from the observation alphabet does not hide those obligations.

Formal references. The detailed rules are in RULE 1 and RULE 2, Direct Inputs, Scheduling of Array Accesses, Appendices B, C, D, J, K, L, N and Q, and the shared channel, Core.MemFaithful/Core.MemProfile memory-boundary rules, reset, and registered-interface definitions in the Normative Formal Core. Appendix M.7a remains the definitive per-instance evidence checklist for the tool and verification flow.

Guide to the Formal Appendices

This guide is informative. It explains how the formal appendices fit together and suggests reading paths; it does not introduce new assumptions, definitions, or proof obligations. The appendix letters and theorem identifiers remain authoritative.

How to interpret the qualification detail. The formal appendices deliberately make explicit many assumptions and implementation facts that are often left implicit in hardware verification, whether the RTL is synthesized from HLS source or written manually. The underlying engineering conditions were largely already there: a claimed model-to-implementation result already depends on facts about scheduling, interfaces, buffering, reset, environment behavior, progress, and implementation correspondence, whether or not those dependencies have been individually named. The formalization makes those dependencies explicit and auditable. It should not be read as requiring every design user to understand or manually discharge every formal condition; many conditions are expected to be established automatically, satisfied by simple design restrictions, or discharged once through reusable tool, library, or implementation-flow qualification. The methodology may add explicit evidence and accounting where a theorem-backed guarantee requires it, but that should not be mistaken for creating all of the underlying engineering conditions.

Recommended reading paths

HLS users and hardware architects. Read the main body through Appendix F for the scheduling contract and modeling guidance, Appendices B/C for pipeline and request behavior, and Appendix K for the qualified serial-workload completion workflow. A successful finite reference run is one input to that workflow; implementation conformance and the applicable progress assumptions remain required; Appendix N explains the service conditions. For the preferred eligible completion workflow, start with K.CT.LazySimulation and its examples, then the informative route table at the head of K.CT.LazySerial. Lazy mode uses source-side demand and preserves the bound elaboration.

Safety reviewers. Read the shared Core and Appendix G rules, J.RequiredOrderBasis, J.ReachableHistCov and the constructive local-to-global theorems. Use the selected message, anchor, configuration, reset or R2C profile with its exact carrier and environment premises. J.SafeTransfer-HistCov transfers a separately proved source property; selected-run use also requires its explicit covering-history and ideal-property conditions.

Process-testbench reading path. Use J.ProcessEnvClass and J.UniformConstructiveEvidence for the joint DUT/testbench or external-contract interpretation. Internal testbench processes obey the same local construction and carrier laws; external placement requires C0/A0 step correspondence for every consumed observation. One parameterized proof may cover a declared stimulus class; a test campaign does not establish that quantification.
Completion reviewers. Read J.2c for full internal-fact correspondence and J.InternalFactAgreement, then K.CT for the finite commit-unit induction. The standard exposure proof first binds source-aligned control under B.OF.Control/B.CT.Control and uses K.CT.Alignment; K.CT.Standard then derives placement, resources and persistent/joint enabling from the applicable B.CT/Q.CT contracts. Core.TransferLaw and Core.SyncProtocol govern physical commitment; Appendix N and K.CT.Service cover the separate progress obligations. The conclusion concerns the required workload, not every internal wait cycle or activity after completion. For lazy applications, also read LazyEligibility, RecordedDemand, DemandPlacement/BatchPlacement and DemandPushResource. Use the route table at the head of K.CT.LazySerial to distinguish mixed LazyComposition, rank-derived LazySerialRestricted and the protected all-singleton LazyExtendedStandard branch. Ordinary Push cases retain floor resources; only qualified extra-slack cases use demand replacement.

Mechanization work. Use the Normative Formal Core as the semantic skeleton and Appendix R for its derived dependency map and audit checklist, Appendix S for the named Lean targets and proof-order guidance, and Appendix T as a selective explanation of why some obligations require scheduling-specific machinery beyond standard libraries. The Normative Formal Core and the applicable clauses of Appendices B, G, J, K, L, N and Q are the normative sources for the definitions and theorem interfaces; Appendix M.7a supplies the definitive consolidated side-condition discharge record; Appendix R is derived.

Appendix map

Normative Formal Core for Appendices G–K. Defines the reference and implementation carriers, actual cycle/phase semantics, source preparation and issue policies, request attribution, required order and common evidence interfaces used by constructive safety and workload completion.

Appendix B — Pipelined Loops. B.BoundaryBinding fixes process-facing sites, requests and physical staging; Q.CT.InputStaging supplies the named early-input component. Explains storage attribution and automatic flush; B.OF and B.CT give the tool-independent ordered-progress, source-aligned control, resource, guard, joint and service contracts used by standard completion. Core.LogicalPopPayload and Q.CT.LogicalPayload additionally qualify restricted internal singleton and Q.CT.LogicalPair representations without physical raw retention; they preserve logical item values and require a causal representation and later-use proof. The pair selects independent, equal-length coupled or fixed-offset staggered delivery, with explicit bubble/drainage and joint-service laws for the coupled alternatives. Q.CT.PairedRepresentation permits a joint cache after the qualified join. The standard early-staging proof requires placement and persistent request assertion to be established without assuming the target item's terminal arrival. A controller whose only route to that placement/request is travel with the item requires the separately proved Q.CT.Extension pre-request route; a data-travelling valid bit alone does not determine applicability.

Appendices A and C–F. Appendix A gives the factory analogy; C defines message request/commit and return behavior; D treats latency-sensitive behavior; E discusses one-way handshake protocols; F gives informative design and qualification guidance. Their applicable rules and referenced Core owners remain authoritative.

Appendix G — Trace Compatibility Rules. Defines the observation boundary and R/E scheduling contract, and states the constructive Post → Ref correspondence and source-property safety-transfer use. It does not claim that an arbitrary selected source execution is realizable by RTL.

Appendix H - Possible Criticisms. Collects anticipated objections, limitations, and responses. It is useful for understanding why the formal contract has its present scope and why certain stronger-looking claims are not made.

Appendix I — Reserved. Shared definitions formerly located here are consolidated into Core, Appendix J and Appendix L; the appendix letter is retained.

Process-environment interface in Appendix J. J.ProcessEnvClass fixes the selected system boundary, placement and realization semantics. External environments satisfy J.EnvironmentContract; environments inside the formal system satisfy the same applicable operational and constructive obligations as other processes.

Appendix J – Constructive System-Level Correspondence. Proves legal finite source-prefix coverage from local source/implementation evidence, then transfers source safety properties. Its internal-fact adapter supplies the full signature needed by K.CT; outer safety projection alone is insufficient.

Stimulus-class application. J.UniformConstructiveCoverage quantifies the constructive premises over a prospectively declared class, including DUT/testbench behavior and actual carrier bindings. Witnesses may vary by stimulus and prefix while the proof and its qualifications remain uniform.
Appendix K — Required-Workload Completion. Proves acceptance of a finite workload from qualified internal correspondence, conditional agreement, a legal witness, closure, exposure and service. B.OF.Control/B.CT.Control and K.CT.Alignment connect source-required operations to actual control realization; K.CT.Standard supplies the generic exposure proof for its covered class. Serial, grouped-serial and separately qualified overlapped witnesses retain their scopes. The standard witness certificate also includes K.CT.CtrlSupport: complete entry, branch and required-work membership decisions, including negative decisions, with their historical support or independent invariant discharges. The standard early-staging proof requires placement and persistent request assertion to be established without assuming the target item's terminal arrival. A controller whose only route to that placement/request is travel with the item requires the separately proved Q.CT.Extension pre-request route; a data-travelling valid bit alone does not determine applicability. K.CT.LazySerial supplies the restricted lazy-witness rank and demand-resource construction; K.CT.LazyComposition permits selected eligible channels in mixed designs while retaining other components' own exposure proofs and checking the assembled graph's acyclicity.

Appendix L — Snooping Introduction, Implementation and Concerns. Explains how legal witness selection aligns a source execution to a selected free-running RTL run to discharge L.WC where consumed. It retains source-decision legality, complete choice coverage, recording and RTL-side non-interference; alignment is not an arbitrary-source-to-RTL realizability theorem.

Appendix M — Formal Results Overview. Summarizes constructive correspondence (M.3), full facts and required-workload completion (M.4), and implementation qualification and the definitive discharge ledger (M.7/M.7a).

Appendix N — Scheduler and Environment Fairness Assumptions. Distinguishes mandatory eligible-edge transfer from pre-grant arbitration service, source issue fairness, conditional primitive channel progress and internal service. Each progress antecedent is explicitly discharged; a synchronous implementation alone supplies none of the missing opportunities.

Appendices O and P. Appendix O gives an informative sketch for multiple clock domains. Appendix P provides links to external discussions comparing alternative formal frameworks with the chosen architecture; those comparisons are not reproduced or adopted as normative proofs here.

Appendix Q - Multiple Input Pipelines Back-Annotation. Defines the local elaboration structures and projection/accounting rules used when ordinary bounded-FIFO abstraction is insufficient for staged, bundled, joined, coupled, or sync-gated implementations.

Appendix R — Compact Derived Formal Core and Mechanization View. Maps the retained safety branches and completion dependencies, outlines mechanization work packages and gives a derived audit checklist. It is non-normative and deletion-safe; every consumed definition and premise has another authoritative home.

Appendices S and T. Appendix S records paper-first authority, proof targets and declaration order, carrier/order safeguards, the foundational inventory and evidence/extension review. Appendix T explains the scheduling-specific reset, identity, boundary, composition, coverage and progress obligations that generic formal-methods libraries do not discharge by themselves. Neither appendix supplies a new theorem or a completed Lean implementation.

Appendix U - Revision Record. Records material changes to the formal interfaces and explanatory text so that reviewers can trace the evolution of the document.

Normative Formal Core for Appendices G–K

Scope of comparison. Appendix G first presents the raw rendezvous source model Sys and the R/E scheduling rules. For the Appendices J and K proof stack, and for the Sys_ref-specific uses in Appendix G, the formal comparison is between RTL_B and Sys_ref (Definition Core.1), i.e., the same source processes interpreted under the channel semantics of E4 with capacities B(c) matching the RTL. The E4 channel semantics are case-split: B(c)=0 is interpreted as a capacity-zero rendezvous channel, and B(c)>0 is interpreted as a cycle-boundary, non-fall-through bounded FIFO. The rendezvous case is therefore recovered as the B(c)=0 case of the channel model, not by applying the B(c)>0 FIFO-availability inequalities with B(c)=0.

Authority of the Core. The shared definitions in this Core are normative for the G–K theorem stack. Later appendices may refine these objects and prove properties about them, but they do not redefine them. Material that merely explains the completed J/K architecture or records certificate-discharge choices is placed downstream in Appendix R or Appendix M rather than inside this definitional layer.

Symbol / Rule	Definition — concise but complete
BoundaryReq(x, σ) (shorthand BoundaryReq(q, σ) only in ordinary unique-boundary case)	For any dynamic source-level message-operation frontier item x ∈ Ω_msg,P on an explicit abstract channel c = BoundaryOf_P(x) of Sys_ref, with owning source-level message instance q = MsgOf_P(x) and kind(q) ∈ {Push_c, Pop_c}, BoundaryReq(x,σ) means that q's own endpoint request for item x is asserted at the selected BoundaryEvalPoint σ: producer valid for Push, consumer ready for Pop, with stable Push payload as required. BoundaryReq is operation- and boundary-attributive and is not an already-committed Σ event. In the ordinary unelaborated case, BoundaryReq(q,σ) is shorthand for the unique x-form. In Sys_B^elab, the frontier-item form is authoritative. Stateless elaboration logic may compute the complementary mirror signal at that boundary (Push: ready; Pop: valid), including through a coupler located at the process-facing outer boundary, but it does not withdraw or reattribute the owning source operation's BoundaryReq. When cycle-index notation is needed, BoundaryReq(x,t) is shorthand for BoundaryReq(x,σ_t), where σ_t is the selected BoundaryEvalPoint for cycle t.
ChannelEnabled(x, σ) (shorthand ChannelEnabled(q, σ) only in ordinary unique-boundary case)	ChannelEnabled(x,σ) means BoundaryReq(x,σ) holds and, after same-cycle combinational settling, the complementary handshake signal at BoundaryOf_P(x) is asserted: ready for a Push endpoint and valid for a Pop endpoint. For a primitive uncoupled bounded FIFO, that complementary signal is the current stored-state observation test (Push: occ_c(σ)<B(c); Pop: occ_c(σ)>0). At an L2 SettlePoint it coincides with the E4 beginning-of-cycle selection test. A later observation cannot authorize a further same-cycle transfer; E4 legality of the derived physical transfer set uses occ_c(t). For a primitive rendezvous channel, it is equivalent to the matching complementary endpoint request in the same settled cycle. In Sys_B^elab, E may place stateless deterministic coupler logic between the primitive storage/request roots and the evaluated boundary, so the complementary signal is the value produced by that declared network; E4 still forbids a selected Push when the relevant beginning-of-cycle storage is full and a selected Pop when it is empty, but nonfull/nonempty status alone need not be sufficient for enablement at a coupling-sensitive boundary. If the owning operation represented by x

Symbol / Rule	Definition — concise but complete
	commits, it contributes the corresponding committed Σ event at that boundary. In the ordinary unique-boundary case ChannelEnabled(q,σ) is shorthand; in Sys_B^elab the x-form is authoritative. At the eligible settled sampling edge, this enabling condition is also the attributed mandatory transfer condition under Core.TransferLaw, not eligibility for a later selection. Outside that edge it remains an observation; all guards and grants must already be reflected in the qualified sampled signals.
ChannelDisabled(x, σ) (shorthand ChannelDisabled(q, σ) only in ordinary unique-boundary case)	ChannelDisabled(x,σ) means BoundaryReq(x,σ) holds but the settled complementary handshake signal required by ChannelEnabled is not asserted. At an ordinary primitive FIFO/rendezvous boundary this is exactly the familiar full/empty/missing-peer condition. At a coupling-sensitive Sys_B^elab boundary it may instead be caused by the declared stateless ready/valid network—for example a join withholding ready/valid because another input is absent or downstream storage has no space. Such a cause must be explicit in E and reconstructible at the settled cutpoint; it is not hidden state. In the ordinary unique-boundary case ChannelDisabled(q,σ) is shorthand; in Sys_B^elab the x-form is authoritative.
Σ	Observable event alphabet at the selected proof boundary: committed Push_c/Pop_c transfers, selected synchronization events (including the applicable wait/Sync/START_P/FINISH_P schemas), and Read/Write signal events, as defined by Core.2 and Core.ProofBoundary. Core.2 also distinguishes additional selected proof-internal observable units, such as RESET, from ordinary Σ events; the outer Σ_DUT projection may hide proof-internal events.
START_P	First observable action emitted by process P after reset. Marks the moment P begins executing user code. A new START_P is emitted each time P exits reset.
FINISH_P	Distinct observable synchronization unit emitted by the actual termination transition of process P in the current reset epoch. It is not assigned retrospectively to the last event of an arbitrary finite prefix; a prefix ending while P is blocked or still running has no FINISH_P merely because no later event is shown. If P executes an unbounded loop within that reset epoch, FINISH_P never occurs for that epoch, and the epoch-local trace is treated as infinite until a later reset boundary, if any. If a later reset affecting P occurs and P exits reset again, a new dynamic START_P begins a fresh reset epoch; any later termination is represented by a distinct dynamic FINISH_P occurrence.
ε-step	An *internal*, unobservable RTL state transition (pipeline advance, FSM micro-state, etc.).
B(c)	Back-annotated effective capacity of channel c at the chosen explicit abstract Sys_ref boundary used by E4 and Appendix K. In the ordinary non-elaborated case this is the chosen Σ-observable Sys_B boundary; in elaborated cases it is the corresponding explicit abstract channel boundary of Sys_B^elab, which may be proof-internal and later projected away from the outer Σ_DUT trace. Finite, B(c) ≥ 0. It reflects the total bounded “composite buffer” storage/credit

Symbol / Rule	Definition — concise but complete
	in the RTL realization represented by that explicit abstract channel boundary, including FIFO storage, skid/elastic buffering, HLS-inserted pipeline staging that can hold in-flight messages, and any explicit staged/bundle/join channel capacity introduced by Sys_B^elab. • B(c)=0 ⇒ rendezvous (capacity-zero) channel. • B(c)>0 ⇒ bounded FIFO (of that effective capacity). B-faithfulness. For the formal results to apply to a concrete RTL_B instance, each back-annotated B(c) and, when used, the surrounding Sys_B^elab handshake network must be faithful to the chosen boundary. This means: • Boundary completeness: every RTL storage element, credit, skid/elastic buffer, pipeline stage, or elaborated staged/bundle/join element that can accept or hold messages across the represented boundary is either included in B(c) or represented as a separate explicit finite-capacity Sys_B^elab channel. • E4 legality plus declared handshake behavior: no committed Push/Pop may violate the corresponding E4 capacity/order rule. In ordinary Sys_B, E4 availability also completely determines the complementary ready/valid value. In Sys_B^elab, an additional restriction on that complementary value is permitted only when it is produced by the finite stateless coupler network declared in E; that logic may be at an internal or process-facing outer boundary and must be included in same-cycle settling/reconstruction. • ε-opacity of internal movement: movement of an already accepted message within one represented storage realization is internal ε-state unless E makes it a separate explicit proof boundary; it does not create, delete, duplicate, reorder, or relabel the selected outer transfers. • Evidence obligation: B-faithfulness is a tool/RTL-flow compliance obligation. It may be discharged by a trusted back-annotation algorithm, generated certificate, structural RTL check, interface monitor, formal equivalence/checking flow, or other validation method appropriate to the implementation flow. It is not derived by E4 or by choosing a reference boundary. Point-to-point source/process ownership assumption. Each source-level logical message channel has exactly one producer process that owns its Push operations and exactly one consumer process that owns its Pop operations. Shared source resources must be remodeled through explicit arbitration and point-to-point process-facing channels. In Sys_B^elab, an explicit internal channel endpoint may instead be driven by a stateless coupler; that coupler is not a process and need not be invented as one. Appendix Q declares complete endpoint ownership and actual coupled handshake equations. Completion additionally binds every relevant resource/guard/service dependency. Single-transfer-per-cycle endpoint constraint (scalar channel interface): For every message-passing channel c, in any clock cycle t, at most one Push_c may commit and at most one Pop_c may commit. Equivalently, the set of commits on a fixed channel in a single cycle contains ≤ 1 Push and ≤ 1 Pop (it may contain one of each in the same cycle). For rendezvous channels (B(c)=0), a

Symbol / Rule	Definition — concise but complete
	committed transfer occurs only as a matching same-cycle Push_c(v) and Pop_c(v) pair. For buffered channels (B(c)>0), a same-cycle Push_c and Pop_c do not imply fall-through; the channel semantics are cycle-boundary, non-fall-through as stated in E4. Reset-empty FIFO convention: at the bound empty-channel initialization, every buffered message-passing channel with B(c)>0 has logical FIFO occupancy 0 at its selected abstract boundary. At a dynamic RESET, this emptying applies only to buffered channels in that RESET's affecting channel scope S_c; unaffected channel contents and accounting are preserved under RW5. Thus the initialized channels, and the affected channels immediately after a dynamic RESET, contain no preloaded in-flight messages in their abstract channel realizations; retained contents of unaffected channels are not a preload or a new initialization. If a design requires preloaded channel contents, those contents must be modeled explicitly as post-reset producer actions or by a separate initialized-state extension outside the present empty-FIFO formal model. Modeling note (multi-lane / burst transfers): If an implementation can transfer k>1 messages per cycle on a logical resource, model it as k parallel point-to-point channels (or as a single widened message transferred by one Push_c/Pop_c per cycle) so that the Σ-level event model continues to satisfy the scalar constraint above. Core.TransferLaw additionally requires every actual settled handshake to commit. E4 legality and cardinality are established by the mapped equations and ownership, not by rejecting an otherwise present overlap. Projection may erase an internal event from the outer history but cannot suppress its physical effect.
Core.MemFaithful	Mapped-memory boundary faithfulness. Core.MemFaithful is an admissibility condition on the existing Σ-selection freedom; it is not a new hiding mechanism. Fix (i) the untransformed logical array-access mapping-layer semantics for the selected memory instance as the reference, (ii) the selected abstract memory boundary and its channel/signal alphabet, and (iii) the declared set of permitted cache/merge/split/reorder transformations. The transformed implementation may keep lower-level mapped-memory activity outside Σ only when it induces at the selected boundary the same observable behavior required from the reference by Core.17/E1–E5: it does not create, delete, duplicate, or relabel selected-boundary events, preserves the per-endpoint occurrence order and every required ordering/atomicity constraint, and preserves the values carried by those events. A mapped message channel or anchored signal that is selected at that boundary remains governed by E4 or R2/E2 respectively. For K.CT, Core.MemProfile additionally includes every hidden backpressure, resource and service condition in the completion qualification. Core.MemFaithful may be discharged by a trusted mapping/back-annotation algorithm, generated certificate, structural RTL check, interface monitor, formal refinement/equivalence check, or other qualified mapping-layer validation. It is not derived by E4 or Core.17 alone.
occ_c(t)	Abstract FIFO occupancy at the chosen explicit abstract Sys_ref channel boundary: number of items committed by Push_c but not yet committed by

Symbol / Rule	Definition — concise but complete
	Pop_c at that boundary, measured from the most recent reset boundary affecting c under the reset-empty FIFO convention. In the ordinary non-elaborated case this is the chosen Sys_B channel boundary; in elaborated cases Sys_ref = Sys_B^elab, this is the occupancy of the relevant explicit abstract staged/bundle/join channel boundary. When a clause explicitly identifies c as an outer Sys_B/Chan_outer refinement-target channel, as in Core.ElabProj and Q.1(a)–(c), occ_c, B(c), and the associated E4 legality denote the corresponding outer-channel semantics rather than the explicit elaborated-channel B_E(d)/occ_d state. Equivalently (E4), occ_c(t)=push(π_t)−pop(π_t), where π_t is the Σ-visible boundary-commit prefix for that explicit abstract channel after its most recent affecting RESET (or initialization) and strictly before cycle t (i.e., excluding any Push_c/Pop_c commits that occur in cycle t), so occ_c(t) is the abstract occupancy at the beginning of cycle t. The subtraction push(π_t)−pop(π_t) is the mathematical signed balance. E4 proves that a legal channel history places this balance in [0,B(c)]; non-negativity is therefore a semantic legality property, not the result of truncating natural-number subtraction. For pure-FIFO channels with B(c)>0, the FIFO availability predicates (not-full/not-empty) are evaluated against this begin-of-cycle occupancy; hence a Pop_c cannot commit in a cycle that begins empty solely because a Push_c also commits in that same cycle (no fall-through), and likewise a Push_c cannot commit in a cycle that begins full solely because a Pop_c also commits in that same cycle. Clarification (no “second Push/Pop”): movements of an item within the RTL channel realization (e.g., FIFO→staging/skid buffering, staging/skid buffering→pipeline-stage registers, pipeline-stage→pipeline-stage handoff) are internal ε-steps; they do not constitute additional Σ-visible committed Push_c/Pop_c events beyond the boundary events counted by push(π_t)/pop(π_t). Derived cutpoint notation (Core and Appendix K). At a BoundaryEvalPoint σ, occ_c(σ) denotes the current stored occupancy of the selected explicit channel at that exact occurrence, reconstructed from its committed history through σ in the current channel reset epoch. At an L2 SettlePoint in cycle t it equals the beginning-of-cycle quantity occ_c(t). A later state observation may include subsequent commits/reset and is not shorthand for the beginning-of-cycle value. E4 legality of the derived physical transfer set uses occ_c(t); observation creates no further same-cycle opportunity. Note: occ_c(t) is an abstract analysis quantity derived from Σ-visible boundary commits; in both Sys_B and RTL_B, processes do not directly read occ_c(t) and instead proceed solely based on the per-channel ready/valid outcome of their own Push/Pop attempts. Core.TransferLaw derives all same-edge transfers from these old-state tests and the complete declared network before any update; a proof serialization does not re-evaluate availability to select extra transfers.

Symbol / Rule	Definition — concise but complete
P_push(c)	Primitive-channel finite-progress invariant (producer): this is a persistent-request premise, not a ChannelEnabled premise. In this paragraph, "the producer's request remains asserted continuously" means that the producer presents a persistent, non-withdrawable Push_c endpoint request from the starting cycle through the matching discharge event, with no deassert-before-commit and with stable payload while asserted. • Buffered case (B(c) > 0): If the producer's Push_c request remains asserted continuously from some cycle t_o onward while the channel is full (occ_c = B(c)) —with stable payload while asserted—and the complementary endpoint continuously requests the matching Pop_c (persistent request asserted; Pop_c boundary request remains true), then some cycle $t' \geq t_o$ commits a complementary Pop_c (i.e., the full condition is eventually discharged). • Rendezvous case (B(c) = 0): If the producer's Push_c request remains asserted continuously from some cycle t_o onward (with stable payload while asserted) and the complementary endpoint continuously requests the matching Pop_c (persistent request asserted; Pop_c boundary request remains true), then some cycle $t' \geq t_o$ commits the rendezvous transfer—i.e., a matching Push_c(v) and Pop_c(v) commit in the same cycle. In Sys_B^elab this clause is used when the blocked cause is the primitive full/rendezvous condition of the represented storage element; coupler-mediated disabledness is handled by the Q.1 compositional handshake/progress obligations. For an ordinary uncoupled primitive at an eligible sampled edge with the stated requests and no affecting reset, the enabled complementary transfer is mandatory by Core.TransferLaw. After its full/empty discharge the original persistent request commits at the next eligible settled edge, if that edge occurs without affecting reset. This uses no B2 choice after enablement. Coupled causes, request arrival and continuing clocks remain separately qualified.
P_pop(c)	Primitive-channel finite-progress invariant (consumer): this is a persistent-request premise, not a ChannelEnabled premise. In this paragraph, "the consumer's request remains asserted continuously" means that the consumer presents a persistent, non-withdrawable Pop_c endpoint request from the starting cycle through the matching discharge event, with no deassert-before-commit. In Sys_B^elab this clause is used when the blocked cause is the primitive empty/rendezvous condition of the represented storage element; coupler-mediated disabledness is handled by the Q.1 compositional handshake/progress obligations. For an ordinary uncoupled primitive at an eligible sampled edge with the stated requests and no affecting reset, the enabled complementary transfer is mandatory by Core.TransferLaw. After its full/empty discharge the original persistent request commits at the next eligible settled edge, if that edge occurs without affecting reset. This uses no B2 choice after enablement. Coupled causes, request arrival and continuing clocks remain separately qualified. • Buffered case (B(c) > 0): If the consumer's Pop_c request remains asserted continuously from some cycle t_o onward while the channel is empty (occ_c = 0)—and the complementary endpoint continuously requests the matching

Symbol / Rule	Definition — concise but complete
	Push_c (persistent request asserted with stable payload; Push_c boundary request remains true)—then some cycle $t' \geq t_o$ commits a complementary Push_c (i.e., the empty condition is eventually discharged). At an ordinary uncoupled primitive the complementary Push commits on its eligible settled edge by Core.TransferLaw; the waiting Pop uses a later edge, preserving non-fall-through. • Rendezvous case (B(c) = 0): If the consumer's Pop_c request remains asserted continuously from some cycle t_o onward and the complementary endpoint continuously requests the matching Push_c (persistent request asserted with stable payload; Push_c boundary request remains true), then some cycle $t' \geq t_o$ commits the rendezvous transfer—i.e., a matching Push_c(v) and Pop_c(v) commit in the same cycle. For an ordinary uncoupled primitive the matching requests force this pair on the eligible settled edge by Core.TransferLaw; no additional fair selection is required.
Trace-compatibility rules (E1–E5)	E1 Synchronization-order preservation: For each process P, the sequence of synchronization events executed by P (wait(), SyncChannel, START_P, FINISH_P) is matched by per-process source occurrence and occurs in strictly increasing clock cycles in each trace, under R1 and Appendix G E1. This does not require equal pre-HLS and post-HLS cycle numbers, impose a global order on unrelated processes, or forbid a preceding message from committing on its synchronization edge under E5. E2 Signal visibility: For each sequential process P, each signal Read in P occurs at the closest preceding synchronization event in P, and each signal Write in P occurs at the closest succeeding synchronization event in P, in both the pre-HLS and post-HLS traces (per R2/E2). For each combinational process or explicitly modeled combinational network component C_comb, observable combinational signal Read/Write labels appear only inside emitted R2C fixed-point signal-observation units for the selected R2C observation component, as defined by R2C. Each emitted R2C unit is evaluated at one ε-quiescent combinational fixed point, and all Read/Write labels inside that unit are atomic with respect to that fixed point. Emitted R2C units are matched by component identity and local emitted-occurrence ordinal, not by global observable-cycle bucket number. R2C stutter cycles whose selected observable slice is unchanged do not create separate Σ-visible units unless an explicitly modeled observer/sampler requires a fresh emitted unit. Interpretation (logical timestamping). For sequential processes, E2 defines the logical timestamps of Read/Write Σ-events at their anchors. An implementation may physically sample a Read later than pred_S(r), or physically apply the canonical Write earlier than succ_S(w), provided the anchored value is preserved; if several source Writes share that anchor, R2/Core.17 select the last executed Write value. For combinational processes, the logical observation point is the emitted R2C fixed-point signal-observation unit, not a global observable-cycle bucket number; internal delta-cycle

Symbol / Rule	Definition — concise but complete
	propagation before the fixed point and R2C stutter cycles with unchanged selected observable slices are ε/stutter and are not separately Σ-visible. Direct-input pragmas (e.g., #pragma hls_direct_input and #pragma hls_direct_input_sync) do not change E2 and are not exceptions to E2. For sequential processes, they impose stability/timing contracts that allow later physical sampling, or controlled updates at a synchronization boundary, while the logical Read/Write Σ-events used for ≈ checking remain timestamped at the E2 synchronization anchor points and retain the anchored value labels. For combinational processes, ordinary direct-input pragmas do not introduce a separate anchoring rule; any observable combinational signal Read/Write label remains part of an emitted R2C fixed-point signal-observation unit under the R2C unit-generation and matching conventions. Thus the pragma affects only implementation sampling freedom under its stated contract, not the logical Σ observation used for compatibility. E3 (Safe message issue ordering). (Post-HLS preservation of R4). For any two executed message-call instances q1,q2 ∈ Q_exec,P of the same process P: (i) Same-interface order is always preserved: if q1 and q2 access the same message-passing interface/channel and q1 <_src q2, then issue_post(q1) ≤ issue_post(q2). (ii) Distinct-interface no-reverse: if q1 and q2 access distinct message-passing interfaces and appear in sequence in the source (q1 <_src q2), then issue_post(q1) ≤ issue_post(q2) (i.e., they shall not be issued in the reverse sequence). (iii) Prefix-closed issue obligation: at every cycle-aligned cutpoint σ of the post-HLS execution / witness, if q1 <_src q2 and q2 has issued by σ, then q1 has also issued by σ and Issue(q1) ≤ Issue(q2). This is an implementation-side obligation on issued sets, not merely an inequality over pairs whose timestamps are already defined. (Note: any pipelined-loop internal staging/dequeue discussed in “Pipelined Loops” is ε-microstate within the back-annotated channel realization and is not a Σ-visible boundary issue event associated with any q ∈ Q_exec,P nor a committed-transfer event in M; therefore it does not constitute a counterexample to (ii) or (iii).) E4 FIFO legality: for each channel c, the post-HLS committed (Push_c(v), Pop_c(v)) history is a legal bounded-capacity execution consistent with Sys_B for that channel (capacity B(c)), and reduces to rendezvous consistency when B(c)=0. In particular, on each channel, the committed Pop_c(v) events return exactly the FIFO-ordered sequence of values v previously committed by Push_c(v), with no drops, duplications, or reordering. E5 (Messages cannot cross their immediate synchronization boundary). For every synchronization call s (in the source order used by R3 / pref_S / suff_S): ∀ q ∈ pref_S(s) : issue_post(q) ≤ clk_post(s) ∀ q ∈ suff_S(s) : issue_post(q) > clk_post(s)

Symbol / Rule	Definition — concise but complete
	For every blocking q ∈ pref_S(s): m(q) is defined and clk_post(m(q)) ≤ clk_post(s).

Frontier-item predicate convention. BoundaryReq, ChannelEnabled, and ChannelDisabled are authoritative in their frontier-item form. For a message-operation frontier item x ∈ Ω_msg,P, let q = MsgOf_P(x) and c = BoundaryOf_P(x). BoundaryReq(x,σ), ChannelEnabled(x,σ), and ChannelDisabled(x,σ) are evaluated at that specific explicit abstract boundary and are undefined for non-message frontier items unless the statement explicitly restricts x ∈ Ω_msg,P. In the ordinary unelaborated case, where q has a unique frontier item and a unique evaluated boundary, the q-forms BoundaryReq(q,σ), ChannelEnabled(q,σ), and ChannelDisabled(q,σ) may be used as shorthands for the corresponding x-forms. In Sys_B^elab, one source-level call q may own several frontier items at different explicit boundaries, so the x-form is authoritative.

Declared combinational coupling; no hidden state or predicate. In ordinary Sys_B, the complementary ready/valid value at a channel boundary is determined entirely by the primitive E4 buffered/rendezvous semantics. In Sys_B^elab, a cross-channel or join dependency may additionally affect that complementary ready/valid value only through the finite stateless combinational network explicitly declared by E. Such logic may sit at an internal elaborated boundary or at a process-facing outer boundary. It does not alter the owning source operation's persistent BoundaryReq, add storage, or create a new process. After L2 settling, its effect is visible directly in ChannelEnabled/ChannelDisabled and must be reconstructible from E, the explicit channel state, and the attributed outstanding requests. A grant, gate, coupler, or other condition not represented in that declared network is a hidden enablement predicate and is outside the model.

Prescribed reference model

Definition Core.1 (Prescribed source reference model Sys_ref).

For a fixed design, fixed external stimulus, and selected elaboration package, define the prescribed source reference model Sys_ref as follows: • Sys_ref = Sys_B in the ordinary abstract back-annotated presentation, where B(c)=0 is a primitive rendezvous channel and B(c)>0 is a primitive cycle-boundary non-fall-through bounded FIFO. • Sys_ref = Sys_B^elab in the elaborated/back-annotated case, where the source model is refined by a finite network of explicit finite-capacity/rendezvous channels together with stateless deterministic ready/valid/data couplers needed for staging, joins, bundles, shared-capacity realization, or sync/epoch gating. A coupler may be placed at an internal elaborated boundary or at a process-facing outer boundary; it introduces no storage and is not a modeled process. All source-side semantic notions in Appendices G–K are interpreted with respect to this prescribed source reference model. Appendices B and Q explain practical back-annotation; this definition fixes the reference model used by their qualifications.

Definition Core.ElabProj (Elaboration projection/accounting map Π_elab).

When Sys_ref = Sys_B^elab, fix the coordinated projection/accounting and handshake interface supplied by the selected elaboration package E: the channel map Π_elab, complete outer-accounting trace map π_Σ,E, subsequent DUT-observation restriction ρ_DUT, finite token projection Π_token,E, signed state-accounting map A_E, and the declared handshake/dependency description. Q.1 defines Q.SafetyQualified(E) for the following four operational/history roles. Completion additionally requires Q.CTQualified(E) and the selected correspondence adapter's premises:
(1) Trace projection. π_Σ,E maps proof-internal elaborated histories to the complete outer Sys_B history

over Σ_outer. For each c ∈ Chan_outer, that history retains both committed Push_c and Pop_c events, including endpoints omitted from DUT observations. Introduced staged/bundle/join events with no outer-channel effect may be erased at this accounting level. The subsequent restriction ρ_DUT selects Σ_DUT; write π_DUT,E := ρ_DUT ∘ π_Σ,E for the resulting DUT-observation trace map. The restriction obeys the existing whole-atomic-group requirements and does not change the accounting history. Appendix Q.1 additionally supplies the outer projection-group metadata and required-order-reflection qualification used by Definition J.OuterCanonHist: a complete proof-unit occurrence with several selected outer effects projects to one indivisible group whose complete flattening is exactly the selected π_DUT,E fragment, and every strict outer E1–E5/E4 order between distinct retained groups is backed by a finite chain of already normative proof-unit predecessor rules. Appendix J later collects exactly those admitted rules into its ≺_req relation.

(2) Occupancy projection. For every selected outer abstract channel c, Π_elab identifies the primary explicit storage-channel fiber for c, while AccountSupport_E(c) declares the exact state fields read by A_E, including qualified shared-storage/token fields outside that fiber when needed. Π_elab remains single-valued; overlapping accounting supports are read access to one represented state, not duplicated physical storage or independent capacity. A_E computes the corresponding signed outer accounting value from those declared fields. At an applicable ε-quiescent cycle-boundary state, Appendix Q requires that state-computed value to agree with the E4 occupancy reconstructed from the complete π_Σ,E outer history, before ρ_DUT.

(3) Conservation/accounting. Internal ε-steps and transfers erased by the accounting projection π_Σ,E preserve that complete outer history and A_E accounting. A transfer retained by π_Σ,E but hidden only by ρ_DUT still has its declared accounting effect. When an elaboration token contributes outer effects, Π_token,E may identify a finite set of outer channels; π_Σ,E supplies the corresponding one-or-more outer Push/Pop effects and A_E supplies their accounting. Each projected outer Push_c/Pop_c commit updates the outer accounting exactly as required by E4.

(4) Settled-handshake visibility. E declares the finite stateless combinational ready/valid/data network connecting process-owned requests and explicit storage/rendezvous boundaries. At every BoundaryEvalPoint, the declared network determines the complementary ready/valid value used by ChannelEnabled/ChannelDisabled; no additional hidden gate or coupler state exists.

Appendix Q supplies the trace/token/storage accounting, actual handshake and finite settling obligations. These do not by themselves prove correspondence or progress.

Remark Core.1 (Interpretation at explicit abstract channels).

Whenever Sys_ref = Sys_B^elab, B(·), occ_·(t), and E4 are interpreted on the explicit storage/rendezvous channels declared by E, while BoundaryReq and ChannelEnabled/ChannelDisabled are evaluated at the selected explicit handshake boundary after the complete declared ready/valid network has settled. For a primitive uncoupled boundary, the settled handshake reduces exactly to the ordinary E4 full/empty or rendezvous test. For a coupling-sensitive boundary, E4 remains the storage/transfer-legality rule, but nonfull/nonempty status alone need not assert the complementary ready/valid signal; that signal is computed by the stateless coupler network. The complete outer-channel history is recovered through Π_elab/π_Σ,E; DUT observations apply ρ_DUT afterward. A coupler at a process-facing outer boundary is therefore permitted when it is part of E and satisfies the actual handshake, ownership and projection obligations of Q.SafetyQualified and the additional qualifications of the selected consumer.

Remark Core.1a (Operational Sys_ref transition system and issue policies).

Sys_ref is the selected source-reference transition system defined operationally in this Core. Source reachability, preparation, phase legality, atomic primitives and Core.16 are fixed independently of a

successful witness construction. Core.SourceL2Settle supplies finite same-cycle settling under its stated qualification; no complete/fair continuation is inferred from finite-prefix correspondence.

Observable alphabet and executions

Definition Core.2 (Observable alphabet Σ).

Let Σ be the observable event alphabet consisting of:
(1) committed message-transfer events Push_c(v) and Pop_c(v),
(2) synchronization events selected into S, and
(3) signal events Read(sig, v) and Write(sig, v), interpreted as anchored signal events for sequential processes under R2 and as labels carried by emitted ε-fixed-point observation units for combinational processes under R2C; those R2C units are identified by component identity and local emitted-occurrence ordinal, not by global cycle bucket.
Under the pipelined-loop body no-wait condition, an HLS-pipelined loop body contributes no additional in-body wait/synchronization events to S or Σ.
Observable-unit carrier convention. Core.17 ΣMatch and downstream references to the selected observable-unit projection range over the selected Σ-events together with any additional proof-internal observable units explicitly included by the theorem instance, notably RESET(S_p,S_c). Such proof-internal units need not be members of Σ and may be projected away from the outer Σ_DUT view. Their matching and ordering rules are stated explicitly where introduced; including them in the observable-unit carrier does not convert them into ordinary Σ commits.

Definition Core.3 (ε-steps and compact trace notation).

Let ε denote internal micro-steps that are not observable at the proof-internal alphabet Σ, including internal staging/hand-off within the concrete realization of a single explicit abstract channel, late direct-input sampling between anchors, and failed non-blocking polls. In elaborated cases Sys_ref = Sys_B^elab, transfers across introduced staged/bundle/join channels are not ε merely because Π_elab hides them from the outer DUT/testbench-visible alphabet Σ_DUT; if such a channel is an explicit abstract boundary used by Appendix K, its committed Push/Pop transfers are Σ-events for the proof-internal comparison.
A finite execution prefix τ is understood in the same sense as in Appendix J: it carries the inherited cycle partition / per-cycle observable buckets of the execution, together with any ε-steps between successive Σ-visible cutpoints. For compact notation one may write a linearization over (Σ ∪ {ε}), but only modulo permutation of Σ-events that occur in the same cycle and are not additionally ordered by the document's explicit Σ-visible constraints. Accordingly, τ|Σ denotes the observable projection together with that inherited cycle partition; same-cycle observable events are not assigned any extra order merely by this compact notation. Operational-prefix distinction. Ignoring incidental listing order in this notation does not assert that every ideal of a completed canonical history is an operationally reachable prefix. The selected implementation carrier determines which unit-completion cuts exist. Core.RegisteredMessageBatch below fixes a whole-edge completion interpretation for its explicitly selected registered message profile; other carriers retain their declared legal partial-cycle cuts. Core.RegisteredAnchorBatch supplies a separately qualified whole-edge interpretation for its enlarged synchronization/signal inventory; it does not widen Core.RegisteredMessageBatch by implication.

Definition Core.ConstructiveOriginCarrier (actual edge origins and designation).

Fix an implementation execution carrier, its unit/occurrence declaration and an origin family O_I independently of the source correspondence relation. OriginReach_I(O_I) consists of its designated finite origin-prefix occurrences. Every covered origin has finite ancestry from a prescribed base through actual completed physical cycle edges, with the exact earlier carrier prefix and bound environment/reset

choices preserved. Every actual edge from a covered occurrence and its realized successor is included. This interface does not exclude partial-edge prefixes, assert Batch, or guarantee a future cycle from a terminal state.
Identity designation uses the actual initial/post-update occurrence itself. A separate designation is admissible only with an occurrence-level transport proof: it preserves the full represented state and fixed roots/choices used by the next-edge relation, adds no selected history, and covers every actual continuation from the undesignated occurrence. Either designation occurs on that continuation before the next edge, or a commuting state/step correspondence maps each actual next edge to an edge from the designated view and relates their targets with the same next origin. The inverse use needed to continue the original actual carrier is part of this proof. A merely reachable sibling state with equal history is insufficient. Finite designation requires no future input, observable edge, release or fairness. Terminal occurrences need only their state/history designation, not successor existence.
A constructive relation R is bound to this carrier through its full-state representation and the declared transport; its local fields and preserved invariant must be invariant under the designation view or explicitly transported. Thus an extension constructed from a designated view corresponds to the actual next edge and restores R at the actual successor view. The domain is not defined by successful construction. Origin-only coverage uses this interface; Batch adds its separate all-finite-prefix completion contract.

Definition Core.RegisteredMessageBatch (registered message whole-edge completion profile).

Fix I and the selected message/proof-unit inventory and Σ_DUT. Core.RegisteredMessageBatch_I holds when the implementation carrier is bound to the following single-clock registered profile and the completion requirements BM0–BM4 are established for every reachable prefix in its declared domain. This is an adopted semantics profile with an implementation-conformance obligation, not a consequence of calling a design synchronous or a permission to discard prefixes from an already fixed finer carrier. Source witnesses retain their ordinary phase-bearing Core.Phase semantics; the implementation is not assigned source L0–L4 phases. A completion consumer requiring facts beyond the selected outer message history additionally binds Core.RegisteredFactBatch; ordinary message-history safety uses of BM0–BM4 are unchanged.
BM0 — Scope and origin family. The profile begins from a prescribed reset-established state and has no subsequent dynamic reset, synchronization/SyncChannel/ac_sync action, direct-input or observable signal Read/Write, R2C combinational process, or multiple-clock interaction. Σ_DUT is message-only; request/valid changes and logging callbacks are not selected completed-message observations. Bind Core.ConstructiveOriginCarrier at initialization and after each realized registered update, with exact Core.ExecutionCarrier provenance and the same unit/occurrence declaration used by the comparison. Use its O_I and OriginReach_I(O_I). Binding this origin family does not supply the separate qualifications required by other consumers.
BM1 — Snapshot and edge completion. Ready/valid/data may settle combinationally during the cycle. All selected message-transfer decisions and NB attempt outcomes are determined from the prescribed settled pre-edge snapshot, fixed current registered state, and declared environment/arbitration choices. At the active clock edge, the complete selected message/proof-unit inventory, NB attempt retirements/results and the corresponding simultaneous registered update become complete together. No legal implementation prefix exposes a proper subset of that edge's completed inventory or a retirement/result ahead of the edge. A failed NB attempt emits no message event but still has its edge-bound retirement, status and unchanged-data-on-failure obligations. The return is dated to that cycle; result-dependent source evaluation/Issue remains subject to Core.RegSeq at a strictly later Horizon. Here selected inventory means the full inventory fixed by Core.TransferLaw/Core.SyncProtocol and the separately declared decision laws, not an arbitrarily chosen subset of present handshakes.

BM2 — Coupling and frozen decisions. Decisions at one edge may depend combinationally on other decisions derived from that same snapshot, through the complete declared channel/coupler network. Batch permits only same-edge replacement or forwarding already justified by the selected E4/E boundary semantics. An ordinary primitive FIFO that is full in the prescribed snapshot does not accept a Push merely because a same-edge Pop frees space; any different staged/coupled realization needs its explicit qualified elaboration and boundary mapping. It prohibits using an already-completed transfer's updated registered state as a new input to another decision at the same edge. All current-state/next-state and no-fall-through/no-full-bypass obligations remain those of the selected E4/E network. The frozen decisions include every mandatory physical handshake under Core.TransferLaw and every Core.SyncProtocol completion. Optional abstract movement cannot filter that inventory.
BM3 — Silent intervals and unchanged quotient. Between completion edges, settling, private physical computation and event-listing order introduce no additional completed message/proof unit and do not change CanonHist_Σ_DUT, including order induced through erased units. Events remain distinct canonical groups except where their own channel/transaction semantics already require an atomic group. Completing several groups in one transition does not merge the edge into one ProjGroup or add strict order between them. Mathematical ideals that split the edge's group collection may still exist; they are not thereby operational prefixes of this profile.
BM4 — Origin designation and finite-prefix coverage of completions. The prescribed initial state and every realized registered update, including a terminal update, determine an origin in O_I. If a separate designation segment is needed, Core.ConstructiveOriginCarrier supplies its actual-continuation or commuting transport proof, including preservation/transport of the state used by R. It has a finite legal realization from that same already established state and changes neither the edge inventory nor CanonHist_Σ_DUT; it does not select another observable edge, change already fixed environment choices, or rely on later inputs. Prefixes before the initial designation have its empty selected history. Thus a prefix during silent activity can be associated with the initial state or the last already completed edge without assuming that its current uncompleted cycle will finish. BM4 is an origin-designation fact, not generic successor totality or a fair-extension premise.
Monitor and conformance contract. The qualification shall bind the actual commit/retirement predicates, edge and clock, complete unit inventory, simultaneous update, origins and legal observation cuts to BM0–BM4 on the same implementation histories. A checker may process edge records sequentially internally, but shall evaluate any property transferred through the Batch route only after assembling the complete edge bucket; intermediate log states are outside that guarantee. Its property evaluation must still be extensional on J.OuterCanonHist. The profile does not make a latency-sensitive or cycle-count-sensitive property extensional. Adoption of Batch for a model does not prove that another implementation or a finer monitor semantics conforms to it.

Definition Core.RegisteredAnchorBatch (wait, paired-sync and anchored-signal completion profile).

Fix one single-clock registered instance I with finite process/resource structure, a reset-established base and no later dynamic reset, R2C process or multiple-clock interaction. Permit unary wait/ac_wait anchors and paired two-party synchronization transactions, messages including NB calls, and RULE 1/2-qualified sequential signal Reads/Writes, including qualified direct inputs. Synchronization forms other than unary waits and pairwise SyncChannel/ac_sync transactions, which the current rules do not define, are outside this profile; ordinary multi-process barrier patterns assembled from those supported transactions remain eligible under the complete joint evidence below. Every explicit source synchronization relevant to this comparison, and every selected signal member required to share its anchor, is retained in the declared observation. Pipelined bodies retain the existing no-in-body-sync rule

and surrounding-sync capacity withdrawal. This is a separately qualified extension, not a change to Core.RegisteredMessageBatch BM0.
AB1 — Complete edge inventory. Bind the actual implementation carrier to BM1–BM4's snapshot, simultaneous-update, invariant-between-edges and origin-designation requirements with the inventory enlarged to include complete unary wait, paired synchronization and anchored-signal units. Bind the prescribed initial and every realized successor origin, with any finite designation segment unable to add an observable unit. Every selected message/proof-unit completion, NB retirement/result, selected anchor unit and registered update completes at the edge; no admitted prefix exposes a proper subset of that edge inventory. A paired transaction and its required signal members have their existing atomic structure. Distinct units completing on one edge are not merged and gain no new order. Failed NB outcomes and Core.RegSeq retain their existing rules. Implementing a synchronous circuit alone does not discharge these completion/observation obligations. When consumed by J.ConstructiveFactCorr, also bind Core.RegisteredFactBatch for the entire F inventory, including the applicable anchor members and non-transfer outcomes.
AB2 — Signal and synchronization binding. Each instance names its signal ownership/resolution and same-edge read/write sampling convention, and proves that both models realize it under the same environment choices. Anchoring alone does not select a universal read-during-write convention. Ordinary signals need not be stable between anchors; later use must preserve the designated sample. Retained Read inventory satisfies RULE 1; pending Writes remain private until the real succeeding anchor, where the last executed source Write supplies the value. No future anchor or value is invented for a finite prefix. Wait eligibility uses the declared clock semantics, and paired readiness uses the actual pending transaction participants; a wait is not given a fictitious message peer. Core.SyncFence permits same-edge prior-message completion when its source accounting precedes interval advance, without prescribing an extra drain cycle. Core.SyncProtocol additionally binds physical request issue and retention, transaction/epoch identity, the actual sampling edge, mandatory completion and reset disposition. Anchor typing alone supplies none of those implementation facts.
AB3 — Observation and monitor discipline. Bind the selected complete groups and their occurrence identities to J.OuterCanonHist and, when elaborated, Q.1(c-order). Intra-edge listing order is unobservable. A monitor transferring properties through this profile evaluates complete edge inventories, not sequential logging intermediates. No required signal member may be removed from a retained anchor group; further projections retain or erase complete groups and contract existing required paths. Batch does not make latency-sensitive properties extensional or make every mathematical ideal an operational prefix. This profile exposes no partially completed edge inventory; other carriers require their own prefix qualification.

Definition Core.RegisteredFactBatch (complete fact inventory for constructive completion).

Fix the prospective fact signature F and attribution of J.InternalFactCorr on the same implementation instance and domain as a qualified Core.RegisteredMessageBatch or Core.RegisteredAnchorBatch. This is an additional fact-completion qualification of that actual carrier; it does not change either model's operational semantics, the global observable alphabet, atomic groups, or the selected outer safety projection. A completed F ledger records the actual completed facts under that fixed attribution: typed keys and values, endpoint/epoch ordinals and order, atomic membership and any declared retirement-to-transfer association. It is not a list ordered by logging callbacks or physical sub-cycle serialization. Initial facts are bound separately. Neither an asserted request nor a private physical state change is a completed source fact solely because the ledger names it. The configuration-bearing Core.RegisteredAnchorBatch inventory of CO3/CO4 is included when J.ConfigRecordFactCorr's local fact cases are established. Stable direct inputs instead use J.StableReadFactCorr's full-F reduction/lift on the

original carrier and its qualified reduced message body; this does not remove BM0's direct-input exclusion from the unchanged base profile.
FB0 — Prospective inventory and base. Enumerate every operation/result/observation kind required by F, its operational completion rule, owner/site/epoch/ordinal key, payload/result fields and attribution. Include every consumed internal fact even when erased from the outer observation or accounting projection, and every consumed failed NB result or anchor observation. Bind the actual initial F ledger and initial facts at each prescribed base. The inventory, attribution and claim domain are fixed independently of constructor success or the selected workload run. A physical event without a corresponding source operation is not retyped into one; if it is needed only as a resource-state fact, its conformance invariant remains separate from J.InternalFactCorr. For the configuration specialization, include selected CO1/CO2 records with their real anchor/transaction keys, typed fields, complete group membership and governing-use attribution. Separately bound initial roots are not fabricated historical Read completions. Apply the prospective-signature convention of J.InternalFactCorr: the declared signature and attribution are fixed before and independently of the workload run. A witness or protected extension may consume further facts already covered by that declaration; discovering their occurrence does not enlarge the declared signature. A declaration may specify families of potential occurrence keys without individually enumerating every future occurrence.
FB1 — Edge-complete facts. For every actual edge, the complete F-fact delta is determined by its declared pre-edge snapshot, legally bound operands and selected actions, and completes at that edge under the implementation's attribution. Bind it to BM1, or AB1's enlarged message/anchor inventory, including return status and the selected value fields of failed attempts. No legal implementation prefix exposes a proper subset of this edge delta. An explicitly atomic source unit retains all its required members; distinct units completed at one edge are not merged. Split FIFO capture and process-facing Pop retirement have their respective identities and completion points: capture alone is not the later retirement, and coincident capture/retirement does not count one logical fact twice. Every consumed internal transfer must have its declared source counterpart or be handled as a separately proved state invariant. For configuration-bearing anchors, AB1 is interpreted with CO3/CO4's complete record inventory; a later physical sample emits no additional configuration fact. The stable-direct-input specialization proves this clause for the original carrier through its full-F reduction, rather than assuming outer message preservation suffices.
FB2 — Silent intervals and designation. Prove that settling, private computation, event listing and every origin-designation segment add no F fact and change no prior key, value, atomic membership, attribution or required endpoint/epoch order. This includes prefixes before the initial designation and after a terminal edge. Bind the full state/continuation transport required by Core.ConstructiveOriginCarrier as well as this F preservation. BM3/BM4 or AB1 outer-history preservation alone does not establish this clause for an F fact omitted by that outer projection. A carrier exposing an F completion during a purportedly silent interval does not qualify merely by suppressing that observation.
Lemma Core.RegisteredFactPrefix. Under FB0–FB2 and the selected BM/AB carrier, every finite implementation prefix has the F ledger of its covered initial origin or of the origin for its last already completed edge. Proof. Before any completed edge, FB0 and FB2 identify the base ledger. Otherwise BM4/AB1 provides the origin of the last realized edge; FB1 supplies its complete delta, and FB2 preserves that ledger through the intervening silent/designation segment and the remaining prefix. No next edge, unfinished-cycle completion, fair extension or source witness is assumed. The actual-continuation transport remains the one already required by the carrier. □ A finer carrier with partial F completions requires a separate exact fact-prefix proof; a mathematical ideal or a safety-property lift is not this lemma. The same proof applies to the explicitly qualified configuration inventory and to the original stable-direct-input carrier once the full-inventory FB0–FB2 obligations specified by

J.ConfigRecordFactCorr or J.StableReadFactCorr, respectively, are established; no projection-only argument substitutes for those obligations.

Definition Core.AnchorConfigObservation (constructive-route configuration observation option).

Select this option explicitly for a Core.RegisteredAnchorBatch construction, as extended by J.LG-ConfigBatch. Its observation Σ_cfg and canonical history CanonHist_cfg are local to this constructive safety route. They use J.OuterCanonHist's complete-group, normative-path and prefix conventions with the following configuration-bearing anchor labels. They do not redefine the global Σ, E2/Core.17 matching or other observation profiles. This option changes the representation of designated configuration, not the operational source program, channel semantics, permissible physical sampling or synchronization fences.

CO0 — Fixed designation. Before any execution, declare a typed set D of configuration input signals, their permitted reader processes, and for each signal either reset-stable initialization binding or a designated sync-controlled update protocol. Use one configuration-observation convention for that signal across all its covered readers; it is not selected per read site, branch, or execution. Ordinary signals outside D retain RULE 1/E2/Core.17 process-Read/Write observations. For a signal in D, no process Read is an observable event in Σ_cfg: an executed read expression remains an actual source evaluation with its control, assignments and writer effects, and obtains the value bound to its governing configuration record. Unexecuted branches execute no read. Merely declaring a port in D does not remove any physical sampling or operand-correctness obligation.

CO1 — Reset-stable initialization. For each declared reader P, designate an existing real initial synchronization anchor a_P after the relevant reset release and before any covered use. Attach the typed field vector θ_P to that anchor's group, with key (initial,P,a_P). Distinct process anchors are distinct occurrences/groups carrying equal values for shared reset-stable signals; they are not merged into an invented broadcast transaction. If a declared initial anchor belongs to an already atomic paired unit, its participant-indexed records stay inside that existing group. The initialization contract establishes that each such anchor sees the same fixed value for every shared designated signal and that it remains fixed through all covered uses. No new handshake or physical configuration register is required, and no later dynamic reset is admitted by this extension.

CO2 — Sync-controlled transactions. Declare each configuration transaction kind, its designated field set and covered consumers. At a committed transaction κ, attach exactly one Config(κ,D_κ,θ_κ) record to its existing complete synchronization group, shared by its participants and readers rather than duplicated per process. The record exists because the configuration transaction commits, even if the interval contains no data or a later Pop never completes. θ_κ is the actual designated input vector presented under the same bound environment choice for that committing handshake; it is not selected from later uses. The existing environment protocol presents updated values with sync-valid after the required ready indication. For every covered use, the qualification identifies the latest legally governing transaction for that process and signal. A reader outside the paired participants needs an explicit existing synchronization/ownership protocol establishing that binding and protection across updates; designation alone creates no broadcast or readiness guarantee. Core.SyncProtocol governs the held ready/valid transaction and requires protection of every old user before the ready offer authorizes changed pins, continuing through the pending interval and any later old-value use. This applies to all covered readers, not only the paired participants.

CO3 — Complete groups and order. Construct configuration-bearing groups ab initio. Their selected members comprise the existing sync endpoints, any required ordinary signal observations and the CO1/CO2 configuration records; they contain no process-Read members for D. Every declared field is recorded exactly once per appropriate CO1 record or CO2 transaction, including unused fields. No optional membership is inferred from later source control. Matching requires the same

anchor/transaction identity, participant association, typed field names and values, and internal atomic structure. Configuration attributes add no standalone vertex or new strict generator. Preserve the existing anchor/message/channel and elaboration generators in both directions. Per-process interval attribution may connect outputs to their governing configuration through existing E1/E5 paths; neither the attributes nor their values create an order on an unrelated producer's input acceptance.
CO4 — Route-local interpretation. For designated signals, use configuration-record matching in place of process-Read matching only in this observation option; all other selected unit matching and normative scheduling rules remain. The option retains observable syncs with their complete configuration records. It is not a partial projection of the old retained-Read groups, is not claimed identical to their trace semantics, and does not assert Core.17 compatibility under the ordinary Read alphabet. Core.RegisteredAnchorBatch's Read-inventory clauses apply to ordinary retained Reads; its complete-unit inventory clauses apply also to CO1/CO2 records. The operational structure of the selected Sys_ref is unchanged.

Definition Core.RegisteredResetBatch (edge-sampled reset extension of constructive Batch profiles).

Select one existing constructive registered message, anchor/signal or configuration-observation profile and retain its non-reset local semantics, resource qualification and observation declaration. Replace only its no-later-reset scope restriction by the conditions here and J.LocalResetContracts. This extension is single-clock and edge-sampled; it makes no asynchronous-assertion qualification claim. It is not a change to RW1–RW5, the general execution carrier or the separately stated K.CT reset restrictions. R2C and multiple clocks remain outside this extension.
RBP0 — Named instance fields. Declare ResetScopes, identifying process/channel scopes and owned storage/state under E; ResetPriority, identifying the actual suppressed affected actions or selected ending-epoch actions that precede each RESET; ResetOccurrenceConvention, specifying one RESET on entry followed by hold edges or another explicitly certified occurrence schedule; ResetHold, the affected-state hold and unaffected-progress equations; ResetReleaseInit, the actual release, fresh START and initialization behavior; and ResetObservation, the selected RESET/START groups and reset-order binding. These fields are fixed per instance, matched under the same actual environment choices, and checked on every reachable reset/hold/release edge. A high reset wire alone does not determine how many RESET units the carrier records. ResetReleaseInit also binds phase placement and complete group membership of selected fresh START/FINISH and initial anchors. The optional Core.SourceCheckpointProfile requires the SCP3 phase/representative proof; it does not assume that START is silent preparation.
RBP1 — Scope and edge order. Every declared reset satisfies RW1–RW5 through the actual boundary/ownership mapping. The first scope supported directly by the local proof is a reset island closed over its live message-channel endpoints and storage; partial scopes require an explicit modeled cross-epoch disposition and its local proof, not silent channel loss. Affected pending paired-sync participants are reset jointly in the simple instance; any other disposition requires explicit transaction/peer matching evidence. Signal/configuration sharing is covered by its own ownership and stability proof. A real edge may include unaffected progress and ending-epoch affected actions followed by reset, in the declared RW4 order. It never selects an action from a partially reset state. Shared cells are updated once. Overlapping reset occurrences have a declared order; RW5-disjoint occurrences may commute.
RBP2 — Completion and source-model adoption. The complete selected edge inventory, including atomic RESET units in Core.CycleResult.resetUnits and corresponding registered updates, becomes complete together under the chosen Batch carrier. An affected ending-epoch commit, if any, precedes RESET in the required order even when both complete at the same physical edge. The source model

used for property verification implements the same reset scope, priority, request cancellation, channel clearing, occurrence/hold convention and release/initialization contract. Restarting a process thread alone does not establish those channel/peer effects. Preserve all historical committed units; reset restarts affected channel-epoch occupancy accounting, not the entire history. Retain RESET and relevant fresh START units in the primary observation. Further erasure requires an explicit whole-group projection with contracted reset-order paths and an appropriate property domain.

Definition Core.RegisteredR2CBatch (edge-observed R2C extension).

Select a constructive registered message, anchor or configuration profile, optionally with Core.RegisteredResetBatch. Extend its observation and local composition with R2C signal units under the following conditions; the original narrower profiles retain their own scope. This is the primary constructive R2C profile. It models an observer that samples the cycle's settled pre-edge values at the clock edge. Calling an implementation synchronous alone does not establish this completion contract.
ERB0 — Fixed observation declaration. Declare every component, selected Obs slice, reset ownership, participation predicate, ordinary/cycle-locked mode and explicit sampler trigger. Apply the existing R2C well-formedness and Core.R2CResetLedger rules. R2C components contribute signal observations only; this extension does not authorize combinational processes with process-owned message or synchronization actions. Combinational channel/coupler logic retains its separately qualified operational role.
ERB1 — Sampling and completion. From the cycle-entry registered state and fixed selected exogenous roots, settle the complete network to μ_t. Determine each R2C vector and emission decision from μ_t and the current ledger, before the edge's registered/reset updates. All selected R2C units, messages, anchor groups, NB decisions and RESET units complete together in that edge inventory with the simultaneous registered update. No actual implementation prefix exposes a proper subset. Separate units retain their identities and normative order; sharing an edge neither merges groups nor creates a required-order edge. A sequential consumer uses the declared current-snapshot value, never the fixed point of next state.
ERB2 — Carrier and monitor binding. Bind the actual edge observation to the selected completion carrier and Core.CycleResult inventory. A monitor may serialize records internally, but evaluates transferred properties only on the complete edge inventory. Combinational propagation and delta-cycle ordering are not observations. Source R2C observations are realized after silent L2 settlement, by the legal fixed-point observation rule, before the remaining certified boundary actions using that frozen snapshot. The proof does not make L2 ε-steps visible. A carrier exposing a completed settled observation before the edge requires a separate multi-cut theorem; it is not covered by this profile.

Definition Core.ExecutionCarrier (legal execution-history carrier and exact-prefix provenance).

Fix a proof model M and theorem instance I. Core.ExecutionCarrier_M,I(χ) holds when χ is one particular legal execution history under the fixed initial state and fixed-stimulus/environment interpretation, carrying the exact state and transition occurrences, the canonical causal history used by Core.EnvStep where applicable, the inherited real-cycle partition, complete observable/proof-unit occurrences, dynamic RESET occurrences, and every theorem-instance choice made along that history. Pref(χ) is the set of finite legal prefixes of χ, linearly ordered by exact prefix inclusion. A carrier may be finite or infinite and need not be maximal, fair, or extendible; membership of a finite prefix does not by itself assert any later transition. "Same carrier" means the same historical branch/provenance: whenever two prefixes overlap, their state/transition occurrences, environment/reset history, dynamic identities, and fixed choices are literally identical on the common prefix. A theorem may separately establish a finite exact-history extension $\tau \preceq \tau'$; such an extension may enlarge a finite carrier but may not retcon its earlier history. Appendix J's Reach_post/Reach_ref domains consist of finite reachable prefixes; when a

theorem additionally names a selected execution carrier, Core.ExecutionCarrier supplies the occurrence-level provenance and ordering for those prefixes. No use of this definition turns finite reachability into a maximal-execution, fair-extension, or generic progress premise.

Definition Core.4 (ε-quiescent cutpoint).

For any proof model M in this formal stack with a declared Σ/ε transition relation (including Sys_ref, RTL_B, and explicit elaborated variants), write Quiescent_M(s) when no ε-step of M's full proof transition relation is enabled from state s at its current cycle/phase. For any finite prefix τ of M, Norm_M(τ,ν,s) means that ν is a finite ε-only extension of τ ending at s, s satisfies Quiescent_M, and ν preserves the inherited Σ-visible history and cycle-bucket structure of τ. A selected maximal finite normalization may still be written $\bar{\tau}$. Core.4 quiescence excludes enabled ε-work only; it does not exclude enabled Σ-actions and does not assert permanent inactivity on future cycles. For Sys_ref, Core.Phase is part of the full transition legality used by Quiescent_M, and the meta-level phase-advance markers of Remark Core.1a are not ε-steps. RTL_B has its own cycle/ε relation and is not assigned the source-side L0–L4 phases. Quiescence alone supplies neither source correspondence nor completion.

Definition Core.Settle (L2 settled boundary-evaluation point).

For a Policy-L or Policy-S cycle t, after the selected L1 process-owned pre-settle window closes, let →ε,L2 denote the complete phase-L2 ε subrelation: process-owned L1 source-evaluation and message-issue steps are disabled; process-owned request assertions and Push payloads that were fixed at the L1 endpoint remain fixed; and the cycle-t exogenous/environment-driven values treated as inputs to the selected G–K system remain fixed throughout L2. Every ε-step that remains phase-legal at L2 is a combinational, explicit-boundary-evaluation, or other cycle-local L2 settling step in →ε,L2.

L2 frame (normative). Every →ε,L2 step preserves the cycle index; registered process/FSM/pipeline state; registered explicit-channel storage, occupancy, and credit state; and the selected cycle-t exogenous/environment-driven input values. It emits no Σ-visible commit and performs no L4 or other next-cycle registered update. It may modify only cycle-local ε/settling microstate and derived combinational boundary values such as complementary ready/valid, guard, join/coupler, and ChannelEnabled/ChannelDisabled evaluation results. A same-cycle reactive component whose value is intentionally combinationally dependent on DUT outputs is not a changing exogenous input under this clause: it must be included inside the same qualified settling network or isolated behind an explicitly verified boundary.

Settling fixed-point terminology (normative). A fixed point of a settling relation is an operational endpoint from which no step of that relation is enabled, including a value-preserving step. This convention applies to →ε,L2 and the model-specific RTL settling relation of Core.QSync. An equation solution, by contrast, is a valuation satisfying all represented node equations; identifying such a solution with an operational fixed point requires the applicable realization evidence, including QS3 where used. Quiescent_M continues to mean that no ε-step of the full proof relation is enabled, as defined in Core.4; fixedness of a subrelation supplies quiescence only when every other ε-step is disabled or included in that subrelation. SettlePoint_I(t,σ) holds when σ is the first fixed point of →ε,L2 reached from that cycle's selected L1 endpoint. This definition is conditional on such a fixed point being reached; it does not by itself prove termination or existence. Define BoundaryEvalPoint_I(σ) to mean a state at which the explicit boundary request and ChannelEnabled/ChannelDisabled network has reached the applicable combinational fixed point; every SettlePoint is a BoundaryEvalPoint. SettlePoint is the L2 activation/evaluation notion used by Core.16 before L3 boundary actions. The standard

existence/uniqueness result is supplied by Definition Core.QSync and Theorem Core.SourceL2Settle below.

Definition Core.CycleState (model-indexed cycle-entry theorem state).

For a proof model M and theorem instance I, Core.CycleState_M,I is the full cycle-entry theorem state: registered/internal machine state, time-aware environment state, global cycle index, and every cycle-valued auxiliary or monitor component declared by I. For RTL_B, the cycle origin used by Core.QSync and Appendix J is not a second unrelated state type: it is a same-cycle registered/input/request frame associated with such a Core.CycleState after the cycle's registered/FSM/pipeline state and process-owned registered request outputs have been established. The RTL settling qualification in Core.QSync binds that frame to an actual reachable occurrence with its full cycle-entry state and fixed inputs; equality of bare state values does not identify occurrences on different histories.

Definition Core.ResetEpochKey and Core.ResetEpochAdvance (canonical reset-epoch provenance).

Fix theorem instance I. Each dynamic proof-internal RESET occurrence γ_R = RESET(S_p,S_c) on a selected Core.ExecutionCarrier carries a canonical logical ResetKey_I(γ_R) determined by the theorem instance, its reset scope (S_p,S_c), and its local occurrence ordinal in that scope's reset history on the carrier under the Core.17 canonical occurrence convention. The occurrence object therefore fixes the carrier-relative ordinal; ResetKey_I(γ_R) abbreviates the logical key of that occurrence without adding a model-local event identity. Thus matched Sys_ref/RTL_B RESET units use the same logical key; ResetKey is not a model-local simulator event identifier. Reset keys are not required to be globally unique across unrelated carriers; epoch-key equality is consumed only within one carrier history or across a source/RTL pair whose RESET occurrences are matched under Core.17 and the selected constructive reset contracts. Let e be the theorem-instance reset-epoch valuation over every modeled process/channel/reset coordinate, and let U be the finite resetUnits occurrence family of one completed real cycle with the inherited carrier/reset-boundary order. Core.ResetEpochAdvance_I(e,U,e') holds exactly when, for every modeled coordinate q: (REA1) if no γ_R ∈ U affects q, then e'(q)=e(q); (REA2) if one or more γ_R ∈ U affect q, those affecting occurrences are ordered by the normative reset-boundary order on that carrier and e'(q)=ResetKey_I(γ_last) for the last such occurrence; and (REA3) occurrences that are incomparable only because they are RW5-disjoint update disjoint coordinate sets and therefore commute. If two occurrences both affect q but the selected semantics supplies no required order between them, ResetEpochAdvance is undefined and the reset/epoch qualification fails. The prescribed initial valuation e_0 uses theorem-instance initial epoch keys. This definition supplies the precise meaning of a "fresh epoch": the new coordinate value is the canonical key of the matched RESET occurrence beginning that epoch, not an arbitrary model-local fresh counter.

Lemma Core.ResetEpochAdvance-Fun (one-cycle reset-epoch update is functional).

If U is exactly the resetUnits family of one legal Core.CycleResult occurrence on a fixed execution carrier and the selected reset semantics supplies every required boundary order needed to compare RESET occurrences that affect the same epoch coordinate, with RW1–RW5 holding for those occurrences, then for every incoming epoch valuation e there exists exactly one e' such that Core.ResetEpochAdvance_I(e,U,e'). Proof. Each coordinate unaffected by U is fixed by REA1. For an affected coordinate, the premise guarantees that all occurrences affecting that coordinate are ordered, so REA2 chooses one last affecting occurrence. RW5 makes RESET occurrences that remain incomparable write disjoint coordinates, so their relative linearization cannot change the resulting vector. Hence all coordinates are uniquely determined. If two RESET occurrences affect the same

coordinate without such an order, the premise is not discharged and ResetEpochAdvance is intentionally undefined. For U=∅ the unique result is e′=e. □

Definition Core.QSync (qualified synchronous same-cycle settling model).

Fix a theorem instance I and a proof model M used by the G–K stack. For a phase-bearing source model, the model-specific same-cycle settling relation is →ε,L2 from Definition Core.Settle. For RTL_B, it is the complete same-cycle ε/combinational settling relation beginning at an actual reachable cycle-origin occurrence declared by the fixed operational boundary mapping after the current cycle's registered/FSM/pipeline state and process-owned registered request outputs have been established and ending before any cycle-advancing registered update.

(QS1) Single-clock scope. There is one selected global clock/cycle relation shared by every process and explicit abstract channel inside the G–K theorem instance. The cycle indices used by cycle origins, boundary evaluation, commits, and registered updates are all defined relative to that clock, and no unmodeled cross-clock-domain boundary lies inside the selected instance. The Appendix M.7a ClockScope entry records the concrete evidence for this condition.

(QS2) Registered-phase/frame separation. The same-cycle settling relation satisfies the L2/cycle origin frame: the cycle index, registered process/FSM/pipeline state, registered explicit-channel storage/occupancy/credit state, and selected cycle-t exogenous inputs are invariant during settling; no Σ-visible commit and no next-cycle registered update occurs in the settling relation. Sequential process request/payload drives covered by Core.RegSeq remain fixed after their selected pre-settle value is established. R2C combinational-process outputs, and stateless elaboration logic such as couplers, are instead part of the settling network.

(QS3) Settling-network realization and deterministic local evaluation. The complete same-cycle settling relation is realized by the qualified settling network: every state component that may change during settling is represented by a cycle-local settling node or by a derived value of such nodes; each settling step changes only such represented values by evaluating one or more node functions from their predecessor values; every phase-legal same-cycle settling step is accounted for by this representation; and no additional settling transition exists outside it. A state is a fixed point of the settling relation, in the no-enabled-step sense of Core.Settle, iff all represented node equations are satisfied. In particular, the equation-solution-to-fixed-point direction excludes enabled redundant evaluations at the solution in the selected proof relation; an unchanged valuation alone does not establish this direction. Realization evidence shall additionally establish operational availability of a finite topological evaluation of the declared network from every admitted settling origin, with the QS2 roots and frame fixed. Each required node update, or an equivalent finite implementation segment, shall be legal in the actual settling relation and implement its declared equation while preserving that frame. If the origin is already fixed, the path is empty. A sufficient local availability rule permits evaluation of any node whose value differs from its function of its current predecessor values. Redundant value-preserving evaluations may remain legal during settling, but not at the claimed fixed-point endpoint. Qualification requires existence of a finite legal path to the equation solution and the fixed-point equivalence above, not termination of every evaluation schedule or a fairness premise. Any abstraction of simulator bookkeeping or redundant concrete activity out of the selected proof relation requires realization evidence preserving its represented state, observations and legal paths; such activity is not silently ignored when checking Quiescent_M. Every combinational or other cycle-local settling node has a deterministic value for fixed predecessor values. Any state that can affect a later cycle is represented at an explicit registered/process/channel state boundary rather than being hidden inside the settling relation.

(QS4) Finite acyclic settling dependency. After cutting the network at those explicit registered/storage boundaries, the dependency graph for the complete same-cycle settling network is finite and acyclic. Pure combinational feedback is outside the standard theorem scope; required feedback shall pass through explicit state. An equivalent independently checked structural theorem establishing the same finite acyclic dependency property may be used.

(QS5) Explicit-boundary completeness. Every value used by BoundaryReq, ChannelEnabled/ChannelDisabled, rendezvous pairing, join/coupler/guard/epoch-gate evaluation, at a selected boundary is produced by that qualified settling network from the fixed cycle-t state/inputs. No hidden enablement or storage predicate may bypass E4/Core.ElabProj/B-faithfulness. For Sys_B^elab(E), Appendix Q supplies the elaboration-specific obligations; for RTL_B, the selected boundary mapping and B-faithfulness evidence supply the implementation side.
For a phase-bearing source model M, QualifiedSynchronousModel_M(I) holds when QS1–QS5 hold. For RTL_B, QualifiedSynchronousModel_RTL_B(I) holds when QS1–QS5 hold for its declared settling domain. That domain and its operational boundary mapping are fixed independently of source correspondence: each admitted settling origin is an actual reachable occurrence on a selected Core.ExecutionCarrier with its full Core.CycleState, registered/input/request frame, fixed environment/reset choices and applicable reset-epoch coordinates bound to that same occurrence. QS2–QS5 apply at every origin in that declared domain. A consumer must establish that each occurrence where it invokes Core.QSync-Post belongs to the domain; this qualification does not assert that every arbitrary finite prefix reaches an origin or that an origin starts or completes a real cycle. Core.QSync-Post supplies finite same-cycle settling only. Core.ConstructiveOriginCarrier and the registered Batch profiles retain their independently stated qualifications. R/E, E4, boundary mapping and B-faithfulness remain separately required by their consumers.

These qualification predicates are structural properties and do not assert one unique Sys_ref execution: latency-sensitive/WC choices, Policy-L issue latitude, arbitration/environment choices across cycles, and other explicitly modeled choices may still admit multiple source executions. Nor do they imply B1; B1 remains the separate observable RTL trace-determinism assumption.

Theorem Core.SourceL2Settle (finite operational source settling).

For a phase-bearing source model M and every reachable L1-to-L2 origin under QualifiedSynchronousModel_M(I), there is a finite legal →ε,L2 path in that same cycle to exactly one fixed-point valuation σ_set satisfying Core.Settle/SettlePoint_I(t,σ_set). The path preserves the registered/input/request frame and has no observable commit or next-cycle update. Proof: QS2 fixes that frame. QS3 realizes every cycle-local node and supplies finite legal operational availability of a topological evaluation; QS4 makes the complete dependency graph finite, acyclic and deterministic. Evaluation in that order reaches the unique equation solution. QS3 equates equation satisfaction with an operational fixed point, excluding enabled redundant evaluations there, and QS5 covers every selected boundary value. Thus the endpoint is a legal L2 SettlePoint. In an elaborated source, Q.1(i) supplies only the template contribution: the applicable design contribution and checked combined-network composition remain required. This proves existence of a finite legal settling path and uniqueness of its valuation, not termination under every settling schedule or cross-cycle service. It does not weaken the stated QualifiedSynchronousModel premise.

Lemma Core.SettleRootAgreement (corresponding roots give corresponding settled boundary signals).

Fix two selected boundary evaluations and their declared boundary mapping. Each operational evaluation is bound to one exact reachable occurrence and its phase; a reconstructed evaluation is bound to the operational occurrence it represents. Assume: (SR1) the complete roots relevant to the selected outputs correspond under the mapping and remain fixed during evaluation, including explicit storage/credit, attributed requests, and any payload, process/control, environment or epoch values on which those outputs depend; (SR2) each evaluation uses a complete finite acyclic deterministic network qualified for its actual phase; and (SR3) independently established equation correspondence relates the networks, either through the same declared template or through local refinement proofs to a common finite acyclic template. Such refinement proves each mapped node or subnetwork equation from its predecessor/root correspondence, without assuming the settled-signal conclusion or the desired correspondence conclusion.
For source L2 evaluation, SR2 uses Core.QSync and applicable Appendix Q template qualification. RTL origin-based evaluation uses Core.QSync-Post under QualifiedSynchronousModel_RTL_B. Any other phase-specific evaluation requires its own fixed-root realization evidence. Matching roots does not establish reachability or authorize a phase shift.
Then the settled selected boundary signals correspond, including MirrorAvail_E. For corresponding, correctly attributed message-operation frontier items whose BoundaryReq values agree, ChannelEnabled and ChannelDisabled agree by Core.11. Data equality is asserted only where required by the declared mapping; inactive data values need not agree unless that mapping requires them.
Proof. Evaluate the common template in a topological order. SR1 supplies correspondence at its roots; SR3 and deterministic local evaluation propagate correspondence through each node. SR2 supplies complete phase-appropriate fixed-point evaluation, so the selected operational or reconstructed outputs have those template values. For ordinary uncoupled primitive channels this reduces directly to Core.12's current-state full/empty tests or Core.13's complementary-request test, followed by Core.11. At coupling-sensitive elaborated boundaries, all declared downstream, guard, epoch and other inputs are included by SR1–SR3. □
Scope. This lemma proves settled-signal and enablement correspondence only. It supplies no request attribution, occurrence extraction, inventory coverage, legal extension, persistent enabling or service. Every live NB attempt affecting a selected output belongs in SR1 even if no message transfer results.

Theorem Core.QSync-Post (finite operational RTL settling).

Assume QualifiedSynchronousModel_RTL_B(I). At each actual reachable RTL origin occurrence in the independently declared settling domain of Core.QSync, a finite legal same-cycle settling path reaches the unique equation-solution valuation before any next registered update. Proof. QS2 fixes the frame, QS3 supplies the actual finite topological evaluation and identifies equation solutions with operational fixed points, QS4 supplies finiteness and acyclicity, and QS5 supplies boundary completeness. Deterministic evaluation in topological order gives the unique valuation. □ This proves existence of a finite path and uniqueness of its valuation, not termination of every evaluation schedule, existence of a next clock edge, or fair cross-cycle service.

Definition Core.CycleResult, CompleteCycle, and SilentCycle.

For a proof model M and theorem instance I, CycleResult_M,I(s_t,r) is a typed legal result r = <nextState, committedUnits, resetUnits>. Write CycleResultOn_M,I(χ,t,s_t,r) when Core.ExecutionCarrier_M,I(χ) contains s_t as its cycle-t cycle-entry occurrence and the actual real-clock transition of that occurrence has typed result r. Because Core.EnvStep_I takes the canonical causal history and cycle index explicitly, this occurrence-relative form is normative for any theorem that appends a result to a separately

selected history witness, unless an equivalent same-history binding is proved. The shorter CycleResult_M,I(s_t,r) notation is a context/existential projection and may be used only when the relevant carrier occurrence/history is already fixed or such an equivalent binding is explicit; equality of bare s_t values reached on different histories does not license splicing a CycleResult from one history onto another. The nextState field is a Core.CycleState_M,I at cycle t+1, so the global clock advances exactly once; its environment component is related to the cycle-t environment by EnvStep_I using the applicable canonical Σ-causal history, and its auxiliary/monitor components advance according to their definitions. committedUnits is the finite collection of selected complete observable/proof units produced in cycle t at the boundary of proof model M, including that model's Σ commits and START/FINISH and emitted R2C fixed-point units when those are selected units of the theorem instance. This inventory is model-relative and precedes any later outer-history projection: for Sys_ref = Sys_B^elab(E), complete commits at explicit proof-internal abstract channel boundaries remain in committedUnits even when Π_elab/Π_DUT later erases them from Σ_DUT. Projection changes only the retained outer history, not the typed per-cycle unit inventory or its finiteness. resetUnits is the finite collection of dynamic proof-internal RESET(S_p,S_c) occurrences in that cycle; each occurrence carries its reset scope and corresponds exactly to the state-changing reset transition governed by RW1-RW5. For a phase-bearing source model, a legal result is witnessed by a finite L1 preparation/issue segment, finite L2 settling, the selected legal L3 boundary actions/decisions and reset boundary units, and the L4 registered/result update. For RTL_B it is witnessed by the model-specific real clock transition through its qualified same-cycle settling/boundary behavior and next registered update; RTL_B is not assigned source L0-L4 phases.
CompleteCycle_M,I(s_t,s_{t+1}) holds exactly when there exists r with CycleResult_M,I(s_t,r) and r.nextState = s_{t+1}. Thus CompleteCycle remains the next-state projection used by existing theorem statements; CycleResult is the primary typed object carrying the full cycle evidence.
SilentCycle_M,I(s_t,s_{t+1}) holds when such a result exists with no selected Σ event in r.committedUnits. A SilentCycle is a real cycle, not an ε-step and not an identity transition: registered/internal machine state, environment state, and cycle-valued monitor state may change. A reset-only cycle may be SilentCycle with respect to the selected outer Σ alphabet when RESET is proof-internal/projected away; that does not make the cycle event-free, because the corresponding occurrence remains in r.resetUnits and in the proof-internal history. RESET is therefore not an alternative cycle outcome: a partial reset changes only its named scope while the unaffected state still proceeds to the same result's nextState under RW5. When the fixed semantics permits more than one reset occurrence in one cycle, resetUnits retains each dynamic occurrence and only the order required by RW1-RW5 and ≺_J is imposed; independent disjoint resets are not artificially serialized. The constructive reset contracts consume the emitted RESET unit as part of the enclosing cycle's complete inventory. Persistent blocking requests follow Core.IC: once issued, an outstanding blocking dynamic instance remains asserted and attributed until its process-facing commit or an affecting RESET governed by RW2; a DeclaredTerminal state may instead end the execution while the request remains pending. No other nonterminal transition in the G–K theorem model may deassert, retire, reattribute, cancel, or replace that outstanding blocking instance. Protocols with independent nonterminal cancellation, withdrawal, or retry require a separately defined operational extension and are outside this theorem stack. An activated Core.16 no-new-later-work restriction remains in force while its fully disabled frontier persists.

Definition Core.SourceCheckpointDomain (independent source property checkpoints).

For the prospectively fixed source instance and history function H, let ChkRef(I,H) contain the declared legal initial checkpoints and every exact prefix ending at the L4 endpoint of an already completed source CycleResultOn. Terminal completed-cycle endpoints are included without requiring another cycle to

begin. Initial checkpoints are specified by initialization/reset semantics and a phase/control predicate, not by constructor success; their histories, including any already emitted initialization units, are retained. A profile needing additional initialization checkpoints declares their source primitive boundaries and guards prospectively. Such points may follow complete START/FINISH/initialization-anchor groups, but may not be selected retrospectively to omit a failing property history. This optional extension does not admit arbitrary partial ordinary L3 inventories by default. ChkRef is a subset of Reach_ref, independent of RTL matching or witness generation.

Definition Core.SourceCheckpointProfile (history-preserving checkpoint representation).

SCP1 — Bases. Every source base admitted by the selected constructive local base contract has an exact earlier checkpoint in ChkRef on the same source prefix, with the same H. Prove this from the declared initialization cases, not by adding constructor outputs to the checkpoint definition. SCP2 — Preparation. Every local source extension exports either a completed-cycle checkpoint directly (including termination/hold cases with no subsequent preparation), or that endpoint followed by a finite legal L1 preparation/Issue segment that emits no retained group and leaves H, including contracted order through erased support, unchanged. Any additional initialization checkpoint uses the same silent-tail requirement. SCP3 — START/FINISH and observations. Classify every selected START, FINISH, anchor, configuration and reset/R2C unit by its actual source primitive phase and completion rule. If START/FINISH are L3 synchronization units under the selected convention, prove that placement and complete anchor membership; anchor eligibility alone does not prove phase placement. If another initialization convention emits a selected unit outside L3, bind an independently declared initialization checkpoint after its complete group and prove the remaining tail history-preserving. An unresolved phase or a selected emission inside a tail claimed to be history-preserving fails this profile rather than being silently erased. Source/implementation semantics are not changed merely to obtain the profile.

Definition Core.DeclaredTerminal (modeled terminal-state predicate)

DeclaredTerminal_M,I(s) remains a predicate on the full Core.CycleState, not a field or alternative constructor of CycleResult. It means that the selected machine/test contract explicitly declares the execution terminated, aborted, or at end-of-test at s and that this disposition is absorbing for the ordinary cycle semantics: no ordinary CycleResult_M,I(s,r), CompleteCycle_M,I(s,s’), or other ordinary cycle successor from s is legal. A finite execution may therefore end at s. Only a disposition modeled as part of the selected theorem instance's machine/test semantics counts as DeclaredTerminal. An external verification-harness watchdog that merely stops host simulation or reports a timeout, without being a modeled transition or termination disposition of M, is not DeclaredTerminal and does not truncate Core.CycleResult or Core.CompleteCycle. A modeled cycle-count, watchdog, abort, or end-of-test rule that is part of the selected machine/test semantics does count. Mere absence of a currently enabled Σ-action or ε-step is not DeclaredTerminal when the time-aware environment or registered machine can evolve on a later cycle.

Definition Core.EnvStep (time-aware environment evolution).

Let EnvStep_I(H,t,e,e’) be the fixed theorem-instance environment transition from environment state e at cycle t and canonical Σ-causal history H to the next-cycle environment state e’. Every legal CycleResult uses this relation for the environment component of nextState. The requirement that a compatible next-cycle environment evolution exist at every exact reachable nonterminal source cycle-entry occurrence, relative to that prefix's canonical history, cycle index, and environment state, is folded into Core.CycleClosure below rather than exposed as a separate environment-totality premise. The cycle/environment state is part of the relation: identical Σ history at two different cycles need not imply identical future availability.

Definition Core.CycleClosure (source full-cycle qualification record).

For a selected source theorem instance I, Core.CycleClosure_I is the normative full-cycle qualification record used whenever a theorem constructs a complete/fair source continuation. It packages the following obligations. (C1) Full-state cycle typing: every legal CycleResult has a nextState with cycle index advanced exactly once, environment state related by EnvStep_I, every declared cycle-valued auxiliary/monitor component advanced according to its semantics, and committedUnits/resetUnits consistent with the selected source transition semantics. Each resetUnits occurrence corresponds exactly to one scoped state-changing RESET transition and satisfies RW1-RW5; no reset is represented as an alternative to the enclosing cycle result. (C2) Occurrence-relative nonterminal source-cycle existence, finite phase execution, and partial-cycle extension: let π be any finite legal source prefix on a legal Core.ExecutionCarrier_M,I branch, for the selected source model M, ending exactly at a reachable nonterminal cycle-entry occurrence s_t at real cycle t. The same historical branch, or an exact carrier extension restricting identically through π, admits at least one legal CycleResultOn_M,I(χ+,t,s_t,r_t)/CompleteCycle successor. Its L1 part reaches the selected finite, not necessarily maximal, preparation/issue endpoint under Core.ReachPrep/Core.Phase and its L2 part reaches the required settled endpoint before L3/L4. The selected L1 issue segment may be empty when no operational rule requires an L1 action. In addition, let π_part be any exact finite phase-legal partial prefix of that same cycle extending π on the selected branch or an exact carrier extension, with all steps operationally legal in sequence, respecting Core.Phase/Core.16, the fixed reset/arbitration/witness choices, and the canonical history/environment state already carried by π_part. Then there exists an exact carrier extension χ+ restricting identically through π_part and a CycleResultOn_M,I(χ+,t,s_t,r_t) whose phase projection contains that exact selected segment in the chosen order and completes the remaining required L2/L3/L4 work. No environment, reset, arbitration, witness, dynamic identity, or earlier source choice preceding π_part may be reselected merely because the same bare state value is reachable on another history. Completion to a CycleResult must include the complete mandatory transfer set and sync completions fixed by Core.TransferLaw/Core.SyncProtocol. Core.SelectLatitude retains only legal issue and genuinely abstract internal-action latitude; it cannot omit a present handshake. It does not itself construct a fair selection or prove that independently proposed issue/L3 choices are jointly phase-legal. The standard discharge combines finite source-control/FPD evidence for L1 with Core.SourceL2Settle for L2 and an occurrence-relative operational cycle-completion theorem, or an allowed equivalent same-history source-cycle theorem. (C3) Occurrence-relative environment definedness: at every such exact reachable nonterminal cycle-entry prefix π, with its canonical Σ-causal history H_π, cycle t, and environment state e_t, the fixed time-aware environment supplies at least one EnvStep_I(H_π,t,e_t,e_{t+1}) compatible with a legal CycleResultOn extending that same prefix occurrence. A compatible EnvStep belonging only to another reachability witness for an equal bare state does not discharge C3. The former standalone source-cycle-closure and environment-totality obligations are exactly the source-cycle-existence/finiteness and environment-definedness clauses of this record and are no longer independent premises. Core.CycleClosure is an existence/typing qualification, not a fairness premise: Core.IssueFair and the applicable boundary/internal service assumptions remain separate where a complete/fair continuation is required. No generic RTL cross-cycle totality or fair-extension premise is supplied; finite-prefix correspondence and completion retain their own domain and service assumptions.

Lemma Core.CycleChoiceExt (phase-legal source-cycle selection extends to a CycleResult).

Assume Core.CycleClosure_I. Let π_0 be a finite legal source prefix on a legal Core.ExecutionCarrier_M,I branch χ_0 ending exactly at the reachable nonterminal cycle-entry occurrence s_t at real cycle t. Let π_part=π_0·α be an exact finite phase-legal partial prefix of that same source cycle, on χ_0 or an exact carrier extension restricting identically through π_0, whose steps are operationally legal in sequence,

satisfy Core.Phase/Core.16, respect the selected reset/arbitration/witness contract, and carry the actual canonical history/environment state of that prefix. Then clause C2 supplies an exact carrier extension χ+ restricting identically through π_part and a legal occurrence-relative CycleResultOn_M,I(χ+,t,s_t,r_t) whose phase projection extends α and completes the remaining L2/L3/L4 work required by the source semantics. Let π_1 be the resulting exact prefix through the cycle-t result and next cycle-entry occurrence; then π_part ≤ π_1 and no pre-π_part environment/reset/arbitration/witness choice has been reselected. Consequently, a later proof may extend an already-constructed finite phase-legal selection only from its exact selected prefix occurrence, and dependent choice over repeated cycles shall range over these exact prefix/carrier extensions rather than over bare state-successor pairs. This lemma neither constructs the fair selection nor proves that independently proposed L1 and L3 choices compose legally; Core.IssueFair and B2/B3 remain separate obligations. □ The remaining required L3 work includes every Core.TransferLaw and Core.SyncProtocol completion on the fixed snapshot. The extension premise does not legalize an earlier segment that already omitted or contradicted one of those units.

Determinism distinctions. Core.QSync gives one settled valuation for fixed registered roots and cycle inputs; it does not remove legal source scheduling choices. B1 is observable RTL trace determinism under its fixed stimulus. Core.FinitePrefixProvenance/Core.FinitePrefixAgreement gives conditional agreement of available keyed facts across the admitted source prefixes. A selected serial simulator may be deterministic without establishing any of these broader properties. Bind each actually consumed premise separately in M.7a.

Definition Core.5 (Per-process execution-call universe, frontier-item universe, observable projection, and source order).

For each process P, let Q_exec,P denote its execution-level set of dynamic source-level message-call instances for the particular execution / witness under the fixed external stimulus being analyzed. Q_exec,P includes blocking Push/Pop calls, successful non-blocking PushNB/PopNB calls, and failed non-blocking PushNB/PopNB polls.

Let Ω_P denote P's local dynamic source-level frontier-item universe for that same execution / witness. Thus Ω_P is stimulus-dependent: it contains exactly the local synchronization-call items, signal-action items, and message-operation frontier items that belong to the realized execution being compared, and excludes items on untaken control-flow branches, unentered cases, and unexecuted loop iterations. For sequential processes, signal-action items are positioned by the R2 anchoring rule; for combinational processes, signal-action items are positioned in the observable cycle bucket determined by the R2C ε-fixed-point rule. Message-operation frontier items include reached blocking Push/Pop instances, whether pending or committed, and successful non-blocking PushNB/PopNB instances that contribute committed Push_c(v) / Pop_c(v) events. Failed non-blocking polls are ε-steps: they are members of Q_exec,P when executed, but they are not members of Ω_P merely because they executed.

Because Q_exec,P is a universe of dynamic message-call instances while Ω_P is a universe of source-level frontier items, their message-operation overlap is not a literal set intersection. Let MsgOf_P(x) be the partial projection from a frontier item x ∈ Ω_P to its owning dynamic message-call instance, defined exactly for message-operation frontier items and undefined for pure synchronization or signal-anchor items. Define the message-frontier slice

Ω_msg,P := { x ∈ Ω_P | ∃ q ∈ Q_exec,P : MsgOf_P(x) = q }.

For any message-operation frontier item x ∈ Ω_msg,P, let BoundaryOf_P(x) denote the explicit abstract

channel boundary at which x is evaluated for BoundaryReq, ChannelEnabled, ChannelDisabled, Front_P. In the ordinary unelaborated case, each source-level message-call instance q has a unique corresponding frontier item x and a unique evaluated boundary. In Sys_B^elab, MsgOf_P need not be injective: one owning source-level message-call instance q may project to a chain or bundle of proof-internal frontier items x at several explicit abstract boundaries. The elaboration package must then provide BoundaryOf_P(x), MsgOf_P(x), and the partial order ≤_P for each introduced item.

Define the corresponding message-instance slice induced by Ω_P as

Q_Ω,P := { q ∈ Q_exec,P | ∃ x ∈ Ω_msg,P : MsgOf_P(x) = q }.

Let Σ_P denote the corresponding observable event/label projection: synchronization and signal items contribute their Σ labels when observed, and a message-operation frontier item x ∈ Ω_msg,P with MsgOf_P(x)=q contributes the committed Σ-event m(q) only if q commits. Pending blocking message-operation frontier items may therefore be in Ω_P and Ω_msg,P, and their owning message-call instances may be in Q_Ω,P, without yet contributing any committed Σ-event. Let ≤_P denote the source order on Ω_P, i.e. the order generated by the document's scheduling rules for synchronization calls, signal actions as positioned by R2 or R2C, message-operation frontier items, and any explicit partial-order structure declared by the selected Sys_ref elaboration. The execution-level order constraints E3/E5 use the corresponding source-induced order on Q_exec,P.

Definition Core.6 (Done_P, Remain_P, and NextFront_P relative to the fixed witness).

Fix the particular execution / witness under analysis, hence the associated Q_exec,P, Ω_P, Ω_msg,P, Q_Ω,P, Σ_P, and ≤_P from Definition Core.5. For a state σ occurring as a cutpoint of that compared execution, let
Done_P(σ) ⊆ Ω_P
be the set of local frontier items already realized by σ. For synchronization and signal items, "realized" means that the corresponding Σ-event has occurred. For a message-operation frontier item x ∈ Ω_msg,P with MsgOf_P(x)=q, "realized" means that q has committed and its committed Σ-event m(q) has occurred. A pending issued-but-uncommitted blocking message-operation frontier item x is therefore not in Done_P(σ), even though BoundaryReq(x,σ), ChannelEnabled(x,σ), and ChannelDisabled(x,σ) may be defined at σ, read operation-attributively through q = MsgOf_P(x). A failed non-blocking poll q ∈ Q_exec,P that induces no frontier item in Ω_msg,P is not considered by Done_P, Remain_P, or NextFront_P, because it is an ε-step with no frontier item and no committed Σ-event.
Let
Remain_P(σ) := Ω_P \ Done_P(σ).
Then define the next source-level frontier-item set by
NextFront_P(σ) := min_≤P(Remain_P(σ)).
This next-frontier notion is interpreted relative to the fixed execution universe Ω_P. It may contain synchronization-call items, signal-anchored action items, and message-operation items. It is a frontier of source-level items, not a set of already-committed Σ-event labels.

Definition Core.7 (Message-operation slice of the next frontier).

For the same fixed compared execution / witness, define
MsgNextFront_P(σ) := NextFront_P(σ) ∩ Ω_msg,P.
This includes message frontier items under Core.5, whose owning calls may be blocking calls or successful nonblocking calls contributing committed Push_c/Pop_c events. Failed nonblocking polls do not acquire frontier items. Membership in this execution-relative candidate slice does not assert that a

call is issued or pending at σ; an already realized item is excluded from NextFront by Core.6. Transfer direction Push_c/Pop_c is not used here to classify a call as blocking. This candidate slice is distinct from the pending blocking frontier of Core.14–Core.16.

Definition Core.7a (Blocking message-operation slice of the next frontier).

Define the blocking-call candidate slice BlockingMsgNextFront_P(σ) := { x ∈ MsgNextFront_P(σ) | MsgOf_P(x) is a blocking Push/Pop call }. Blocking refers to the owning source call, not merely to the direction of its observable transfer. This candidate slice does not require Issue to have occurred and is not Pending_P or Front_P. Core.14 independently defines the issued, uncommitted blocking items in the current interval; Core.15 takes their minimal pending frontier, and Core.16 governs activation. Minimal remaining work and minimal pending blocking work must not be identified without a separate justification.

Definition Core.7b (Synchronization-call slice and typed synchronization carrier).

For the same fixed compared execution / witness, let S_exec,P denote P's execution-level set of dynamic source-level synchronization endpoint-call instances selected into the synchronization set S. Let SyncOf_P(x) be the partial projection from a frontier item x ∈ Ω_P to its owning dynamic synchronization endpoint-call instance s ∈ S_exec,P, defined exactly for synchronization-call items and undefined for message-operation and signal-anchor items. Define Ω_sync,P := { x ∈ Ω_P | ∃ s ∈ S_exec,P : SyncOf_P(x)=s }. MsgOf_P remains defined exactly on Ω_msg,P and is not widened by this definition. For s ∈ S_exec,P, let SyncTxn_I(s) be the theorem-instance dynamic synchronization-transaction key. The key records the synchronization interface/site, dynamic occurrence/ordinal, reset epoch, and transaction kind. For a paired two-party SyncChannel/ac_sync transaction, the matching endpoint calls have the same SyncTxn key and commit as the existing atomic two-endpoint synchronization unit; for a unary clock/wait anchor or another declared synchronization form, the key records that distinct kind rather than inventing a message peer. Let SyncParticipants_I(κ) be the finite set of modeled process participants associated with transaction key κ; clock/environment participants, when present, are typed separately and are not modeled-process owners merely because they can release the synchronization. For x ∈ Ω_sync,P with s=SyncOf_P(x), SyncReached_P(x,σ) holds when the operational source/replay control state of P has reached that dynamic synchronization call in its current reset epoch. SyncPending_P(x,σ) holds when SyncReached_P(x,σ) holds and the synchronization endpoint instance has not committed/returned and has not been ended by an affecting reset/terminal disposition. SyncReady_I(x,σ) is the current-state synchronization-completion readiness predicate supplied by the declared synchronization semantics: all required current transaction participants/conditions needed for the synchronization action are present at σ. Define SyncBlocked_I(x,σ) := SyncPending_P(x,σ) ∧ ¬SyncReady_I(x,σ). SyncReady is a state predicate; away from an eligible settled sampling edge it does not itself assert a commit or a future CycleResult. At an eligible edge, Core.SyncProtocol fixes the corresponding completion law and leaves no discretionary post-handshake selection. SyncReached/SyncPending alone do not assert that a physical request has already issued. Neither SyncPending nor SyncReady is a message BoundaryReq/ChannelEnabled predicate. BoundaryReq, ChannelEnabled, ChannelDisabled, MsgOf, Ω_msg,P, Pending_P, Front_P remain message-typed. For a paired physical transaction at a settled eligible edge, the readiness binding includes its actual matched, issued endpoint offers under Core.SyncProtocol; mere source reach or a matching transaction key cannot substitute for an asserted offer.

Definition Core.7c (Synchronization-carrier well-formedness).

Core.SyncCarrierWF_I holds when each synchronization-call item has a unique endpoint identity and reset-consistent transaction key; the fixed pairing and participant schema agrees with the declared

unary or paired synchronization semantics; reached, pending and ready predicates follow from the actual declared process, transaction and environment state; and affecting reset never carries an old transaction into a new epoch. Every environment or guard input to readiness is explicitly represented. Write SyncCarrier_P(x,σ) for the typed record of owner, item, endpoint, transaction key, epoch, participants, pending status and readiness. The record is synchronization-typed and creates no message request. It does not prove eventual readiness or service. Every synchronization instance also satisfies Core.SyncProtocol. The carrier records its issued offer and pending interval separately from source reach; physical protocol evidence is additional to transaction typing and does not follow from a matching key alone.

Definition Core.SyncProtocol (physical synchronization lifecycle and sampled completion).

Fix the synchronization kind and operational interface mapping before correspondence construction. The following clauses are constitutive source legality and implementation-conformance obligations for every synchronization used by the theorem instance. They apply to the existing Core.7b transaction and participant identities; synchronization requests are not message BoundaryReq objects.

(SP1) Identity and issue. Bind the interface/site, endpoint owner, dynamic occurrence, reset epoch, participant set, sampling edge and signal roles for each SyncTxn. A paired SyncChannel/ac_sync uses its declared ready and valid endpoints; an environment endpoint is bound as such and is not invented as a modeled process. SyncIssue(s) is the first assertion of that endpoint's transaction-attributed physical request/offer after legal reach and preparation. Bind the actual level and owner decode prospectively and completely. Source reach may precede issue. No offer is ownerless or retrospectively assigned to a later occurrence. Once a transaction commits, a continuously high level on a later cycle belongs to a newly legally issued transaction or is deasserted; it cannot commit the retired transaction again. Existing E1 and Core.RegSeq constrain subsequent occurrences and post-sync work.

(SP2) Retention. From SyncIssue through the committing sampling edge, the endpoint's asserted ready or valid offer, transaction identity and required stable descriptor are retained. The peer's absence, backpressure, guard delay, scheduler choice or timeout cannot withdraw, replace or reattribute that offer. An affecting declared reset may end the old transaction under SP4. A DeclaredTerminal execution may end with it still pending; terminality does not fabricate completion. Any implementation requiring independent cancellation or retry is outside this protocol unless a separate operational extension and its correspondence obligations are supplied.

(SP3) Settled handshake and fence. At each eligible sampling edge, the complete declared synchronization network is evaluated from its bound state, inputs and fixed choices before updates. For a paired ready/valid transaction, SyncCommit(κ,t) holds if and only if its mapped ready and valid are both asserted in that settled sample under SP4. It commits exactly once as the existing indivisible transaction, including its required anchored effects. Transaction matching, guards, ownership, Core.SyncFence, atomic membership and all other legality conditions must be realized by the qualification of those signals and their supporting state. They cannot veto an already sampled handshake. A sampled overlap with an unmatched participant or an unsatisfied legality condition is a conformance failure, not a legal uncommitted handshake. Preparation or a guard may delay initial assertion; after an offer is asserted, SP2 applies. Qualified early offers must guarantee that any permitted peer response satisfies the fence and all other prerequisites on its commit edge. Same-edge predecessor message completion is permitted under Core.SyncFenceOrder, with independently justified predecessor commitment and no cyclic readiness argument. A later external transfer of an already process-facing-completed buffered message is not a fence prerequisite.

(SP4) Reset and edge binding. Declare clock edge, setup/hold and event-region sampling, reset scope and priority, and the disposition of an offer when reset intervenes. The mapping must distinguish a reset-only edge from an eligible transfer edge and prove its realization; it may not reinterpret an

observed transfer edge after the fact. On a reset-only edge the declared protocol suppresses transfer and ends affected pending identities. On an eligible transfer edge the settled overlap commits by SP3. If the declared semantics permits transfer and reset on the same edge, it fixes their old/new-epoch attribution and ordered effects in advance and retains the committed unit; reset does not erase an already completed handshake. No old pending transaction is silently carried into a new epoch.
(SP5) Sync-controlled pins. Bind the designated signal vector and every covered reader/use to its governing transaction. Before a ready offer authorizes the environment to change pins, every use still requiring old values must be finished or protected by proved retained samples, operands, tokens or payloads. Maintain that protection throughout the pending interval and through any later old-value use. The environment presents the new vector with the matching valid offer, holds it through the committing edge, and preserves the values required by subsequent uses until the next permitted update or affecting reset. Unpaired readers need their own explicit protection/ownership binding; the transaction is not an implicit broadcast. Same-edge logical Read/Write conventions and retained anchor membership remain those of the selected profile. Neither registered outputs, request persistence nor the fence alone proves this pin protection.
(SP6) Unary scope and progress. A unary wait/ac_wait has no fictitious ready/valid peer. Bind its legally issued pending occurrence, target clock/release condition and actual eligible sampling edge under the declared unary semantics; the occurrence completes at the first such edge satisfying that release condition and SP4, with its fence and anchor prerequisites established by the interface qualification. It is not independently postponed by L3 selection. No clause guarantees that the environment supplies a peer, that initial preparation/guard service finishes, or that a future clock edge occurs. Those remain the selected progress consumer's obligations. Once the applicable settled handshake or unary release occurs at an eligible edge, completion is mandatory and requires no additional B2 fairness premise.

Definition Core.7d (Source synchronization in-flight completion unit SyncFire).

Fix a theorem instance I and a currently represented dynamic synchronization transaction key κ. SyncFire_I(κ) is one complete source synchronization boundary-action unit, admitted to the in-flight universe only when its required transaction is already present in the current typed synchronization carrier. Concretely: (SF1) every modeled endpoint whose participation is required for this occurrence is represented by a reached, pending Core.7b carrier with SyncTxn key κ, and SyncReady_I holds for the transaction under the complete declared participant/environment state; a peer that has not yet reached the synchronization still requires legal source reach/preparation and is not already-admitted in-flight work; (SF2) Core.SyncFence holds for every committing endpoint owner, so every applicable pref_S blocking message operation has already process-facing committed or is a distinct earlier same-cycle selected action accounted for before SyncFire; (SF3) the fixed reset/terminal/environment semantics permit the transaction and no affecting reset rekeys or carries it across epochs; and (SF4) selecting SyncFire requires no new source ReachPrep, launch, message Issue, or source-later operation to make the transaction ready. For a paired SyncChannel/ac_sync occurrence, SyncFire is the existing indivisible two-endpoint synchronization transaction; for another declared synchronization kind it is the corresponding already-defined atomic synchronization unit. Any E2/R2 signal actions required to share that synchronization anchor are included in the same declared atomic observable unit rather than split into separately orderable in-flight actions.

The exact SyncFire successor commits that synchronization transaction once, appends the corresponding atomic synchronization/anchored-signal unit to the H-retained history, marks exactly the participating Ω_sync items realized, advances each participating process across that synchronization boundary to its immediate post-sync source-interval/control point, and retires the corresponding pending synchronization carriers. It does not execute post-sync source evaluation, ReachPrep, or

message Issue and does not assert any new suff_S BoundaryReq as part of the action. Existing attributed message requests and explicit message-channel contents are framed except for effects already produced by distinct earlier action units; any synchronization-dependent stateless handshake/epoch-gate roots are then deterministically re-settled as prescribed by the phase/update semantics. Their changed values can change handshake eligibility only in a later cycle's L2 settling; they cannot add, remove or alter a transfer in the already frozen cycle-t inventory (Core.Phase L3/L4 and Core.TransferLaw). Thus completion of an already-pending synchronization is permitted in-flight progress, but launching work from the newly entered source interval is not. SyncFire is synchronization-typed: it is not MsgOf, BoundaryReq, ChannelEnabled. For a physical paired transaction, the complete eligible-edge inventory contains SyncFire exactly when Core.SyncProtocol's settled handshake occurs. An execution-order witness may order its already justified predecessor units before it, but cannot omit this mandatory transaction or add a late readiness veto. Unary release follows SP6.

Definition Core.7e (Explicit epoch-gate administrative action).

For an E-declared explicit epoch gate g associated with synchronization transaction κ, EpochGateAdvance_{I,E}(g,κ) denotes a declared operational action on already-present gate state. Its precondition, exact channel/token/credit update and projection are part of the selected Sys_ref elaboration semantics. It frames unrelated state, emits no additional synchronization commit, and includes no new source evaluation or Issue. It is not admitted merely because κ once occurred. Any required cycle advancement and phase legality follow the actual operational relation, not a reconstructed path. Gate restoration used by completion separately satisfies B.CT.Restore and Q.CT.Epoch; this definition supplies no restoration-progress theorem.

Clarification Core.5a (ordinary source order versus elaborated partial order).

For ordinary user-written sequential source code, dynamic message-call instances that appear in sequence in one process are ordered by dynamic program order, including distinct-interface calls, repeated loop iterations, and unrolled instances whose selected source expansion is ordered. Same-cycle issue, same-cycle commit, or issue before an earlier operation commits is a timing/issue-policy property, not incomparability. A message-operation multi-frontier exists only when an explicit Sys_B^elab elaboration package introduces incomparable proof-internal frontier items and supplies the corresponding projection, boundary, order, and attribution facts. Ordinary distinct-interface source calls are not made incomparable merely because they access different channels.

Lemma Core.OrdFrontSingleton (ordinary-source frontier uniqueness).

Fix a process P, a particular execution/witness, and an ordinary non-elaborated source interval. Assume every message-operation frontier item in that interval is induced by a unique ordinary dynamic source-call instance and no Sys_B^elab partial-order structure is introduced. Then the restriction of ≤_P to the message-operation frontier items in that interval is total. Consequently, for every cutpoint σ in the interval, |Front_P(σ)| ≤ 1.

Proof. By Clarification Core.5a, all ordinary dynamic source-call instances encountered by P in the fixed execution are ordered by dynamic program order, including distinct-interface calls, repeated loop iterations, and ordered unrolled instances. In the ordinary unelaborated case, each such call induces one frontier item and ≤_P is the corresponding reflexive source-induced order. By Definitions Core.14 and Core.15, every member of Front_P(σ) belongs to Pending_P(σ) and hence is a blocking message-operation frontier item in the same interval. Hence any two members of Front_P(σ) are comparable. Two distinct elements therefore cannot both be ≤_P-minimal. Thus Front_P(σ) contains at most one member. □

Operational interpretation of Sys_ref and source issue policies

The R-rules stated in Appendix G are constraints on a selected source-reference transition system, not merely constraints on a finished list of labels. For a fixed design, fixed stimulus, and selected reference-model choice, Sys_ref has the following operational reading.

A Sys_ref state consists of: (1) one local source state per process, including control location, local variables, call-stack/loop state, the current synchronization interval, and the reached/issued/committed status of dynamic message-call instances; (2) one state for each explicit abstract channel of the selected reference model, including B(c), occ_c(t), FIFO order when B(c)>0, rendezvous endpoint requests when B(c)=0, and any explicit Sys_B^elab staged/bundle/join/epoch-gate channel state; (3) signal and synchronization state at the selected observable anchors; and (4) ε-microstate for internal staging, direct-input sampling between anchors, and other proof-hidden activity. In Sys_B^elab, introduced abstract channels are explicit proof-internal boundaries: their commits may be Σ-events for Appendices J-K even when projected away from the outer DUT/testbench-visible alphabet Σ_DUT by Π_elab.

The transition relation contains the following kinds of steps. Pure source-evaluation ε-steps advance ordinary C++/SystemC computation and allocate dynamic operation instances in dynamic source order. For a sequential process, ordinary call-return state and the L1 preparation/reach state are distinguished by Definition Core.ReachPrep below: a later operation may become reached before an earlier blocking operation has process-facing committed only through a legal dependency-independent preparation segment, and this does not make the operations unordered. A message-issue step for q may occur only after q has been reached, the endpoint request can be attributed to q, the R3 synchronization-boundary constraints are respected, and the issued set is prefix-closed under <_src. Issuing a blocking Push or Pop asserts the endpoint-owned persistent request for q and records Issue(q); issuing PushNB/PopNB asserts its one-cycle attributed attempt. A channel-commit step for a bounded FIFO boundary B(c)>0 is mandatory in the phase-legal L3 set exactly when its Core.TransferLaw settled handshake holds. The attributed request and mapped signals come from the applicable L2 snapshot; their qualification establishes E4 legality using beginning-of-cycle occ_c(t), including its same-cycle/non-fall-through constraints. This commit emits the corresponding Push_c(v) or Pop_c(v) event, updates the E4 channel state, and releases the owning call at the process-facing boundary. For a rendezvous boundary B(c)=0, commit is one atomic two-endpoint transfer step with matching producer and consumer endpoint requests in the same settled cycle and one shared payload value. A failed non-blocking boundary decision emits no Σ event but produces a ReturnedAtCycle status and retires its one-cycle attempt. A synchronization boundary action for call s obeys Core.SyncProtocol: its sampled handshake or qualified unary release commits mandatorily. Core.SyncFence is established by the already-completed and same-cycle process-facing message commits before the signals/release may authorize that completion; interval advance occurs only after those predecessors have been accounted for. An unsatisfied fence is not a post-handshake veto. Synchronization and signal-anchor steps then emit wait/Sync and anchored Read/Write events according to R1/R2/R2C. ε-settling and drain steps update only proof-hidden or explicitly elaborated internal state and must respect the declared source semantics, Core.16 and B-faithfulness. For a sequential process, Definition Core.RegSeq below governs which process-owned evaluation, request, and payload actions may share one Horizon.

Definition Core.SourceChoiceBinding (prospective source semantics and choice resolution).

Fix the source transition system, initial/reset semantics, capacities/elaboration, observation and environment placement before any correspondence construction. Distinguish (SCB1) externally resolved data and choices, or a prospectively fixed causal strategy/relation for resolving them from the actual finite history and current registered/environment state; (SCB2) operationally computed outcomes, including NB success/failure under Core.NBDecision and its Core.NBDecisionSnapshot, and data and arbitration results determined by their declared operational laws on the complete applicable snapshot

and selected resolutions; and (SCB3) witness/replay annotations that record those outcomes. An annotation never authorizes an outcome contrary to that law. A fixed transcript, including one keyed by dynamic occurrence, is admissible only with its indexing fixed in advance and compatibility proved for every covered actual successor. Reactive or unresolved choices use the declared causal strategy/relation; no future observation or retrospective choice repair is permitted. Arbitration choices that affect a physical transfer must resolve into the declared settled signal equations before sampling. Neither external choice nor a witness annotation may select away a Core.TransferLaw/Core.SyncProtocol completion after the handshake.
Source reachability is the set of finite paths of that prospectively fixed operational relation, with no fair-extension requirement and independently of source-witness existence, constructor success or package acceptance. Any instance-fixed outcome transcript restricts this relation only as explicitly declared and consistently with its decision laws; choices left unresolved remain in the domain of the source-property premise. The liveness instance of Core.LLegal retains its fixed choices. A safety instance need not acquire a single predetermined transcript merely by citing source step legality; any choices it does fix are respected by all compared paths and incremental extensions.

Definition Core.PreparationEvidence (validation of independently defined preparation).

The operational ReachPrep permission consists of conditions (1)–(4) below together with any additional preparation restriction explicitly declared as a predicate of the source state, candidate finite segment and prospective instance choices. Such a restriction is part of Sys_ref, is fixed independently of the correspondence constructor, and is not defined by acceptance of a certificate or by existence of a matching RTL execution. In the unrestricted eager-preparation instance there is no additional restriction. Evidence validates membership in this declared relation; it does not create its transitions.
For the constructive route, the C0 base and C4 local preparation proof supply Core.PreparationEvidence: each step is derived from the source primitive rules, with dynamic control/operand/writer identity, all ReachPrep guards, owned frames, finite length and every fixed choice checked. A parameterized proof or a sound checked validator with those judgments as its specification is an accepted evidence form. C4 owns successor-word validity; C0 owns initial preparation. A tool assertion with no stated relation or soundness basis is not a discharge.

Definition Core.ReachPrep (overlap-aware source reach/preparation semantics).

Fix a sequential process P, its current source interval, and a cycle t. Sys_ref retains the logical source/replay control position together with reached/issued/committed status for dynamic calls. During the L1 pre-settle phase, an operational preparation cursor may advance through a finite source-control segment and mark a later dynamic operation q as reached without claiming that an earlier blocking operation q0 has committed or returned. Such preparation is legal only when: (1) q remains in the current synchronization interval and all dynamic source-order relations are preserved; (2) the reachability, control choice, endpoint, address, payload, and required operands of every prepared action are independent of any unresolved earlier returned data/status and satisfy Core.RegSeq and the applicable field-sensitive NoDependentUse/footprint restrictions; (3) no synchronization boundary is crossed; (4) no already-activated Core.16 blocked-frontier discipline forbids launching that source-later work. Any additional prospective instance restriction and the separate validation obligation are specified by Core.PreparationEvidence; validation is evidence of legality, not an extra constructor-dependent source transition rule. Reaching q does not itself assert a request. Issue remains a separate transition and remains prefix-closed under <_src. Private physical computation that satisfies the abstract-invisibility clarification of Core.16 is not itself a ReachPrep segment. This distinction does not permit abstract source preparation to bypass condition (4).

This permission applies both to overlapped/pipelined iterations and to ordinary straight-line source sequence q0; q1 when q1 is dependency-independent of the unresolved q0 result and the selected Sys_ref theorem instance admits eager preparation. It does not make q0 and q1 incomparable, and it does not assert that a raw serial SystemC execution has returned from q0 early; it is the source-reference scheduling permission used to represent allowed RTL-matched eager issue. Coverage direction for Post → Ref: for every RTL_B prefix included in a Post → Ref coverage claim, the selected Sys_ref ReachPrep semantics shall admit every dependency-independent preparation/reach segment required by the constructive C0/C4 source extension, for that prefix under the same independently fixed source relation. In the constructive case the local proof establishes this direction for every actual successor; failure cannot be repaired by discarding that successor or redefining Reach_ref. The instance may be more permissive than the observed RTL scheduling, but it shall not exclude preparation needed to realize a covered RTL witness. Once the L2 SettlePoint exposes a nonempty fully disabled pending frontier, Core.16 prohibits every new source-later launch while that frontier persists. Every L1 preparation segment used by a theorem instance must be finite. Local source-rule proofs and the applicable FPD/B4 evidence supply that fact for the selected finite construction segments; Core.CycleClosure's source-cycle-existence/finiteness clause supplies the generic per-cycle finiteness obligation when a complete/fair source continuation is required.

Shared proof-boundary, poll and implementation issue conventions

Core.ProofBoundary. The observable alphabet Σ is as defined in Normative Formal Core for Appendices G–K: it consists of the event schemas {Push_c, Pop_c, Sync, wait, START_P, FINISH_P, Write, Read} instantiated only for channels/signals designated observable at the chosen proof boundary. For constructive correspondence and workload completion, Σ is the proof-internal observable alphabet of the prescribed Sys_ref / RTL_B comparison. Thus all message-passing channels whose facts are consumed by the selected K.CT instance are designated observable at that proof boundary and hence included in Σ. In elaborated cases Sys_ref = Sys_B^elab, this proof-internal Σ may include introduced staged/bundle/join channels even when the projection Π_elab hides those internal events from the outer DUT/testbench-visible alphabet Σ_DUT. Such events are therefore Σ-visible for the proof-internal comparison, but may be projected away from the outer observation boundary.
Core.NBPollInterpretation. In both RTL_B and Sys_ref, a non-blocking call (PushNB / PopNB) is treated as a poll. A successful poll contributes the same observable event as the corresponding committed Push_c / Pop_c on that channel. An unsuccessful non-blocking call (returns “false” / “not completed”) contributes no Σ-event and is modeled as an ε-step. A non-blocking poll may still assert an endpoint request in the cycle of that call. The request level is observable at the selected evaluated boundary and supplies auxiliary Issue/attribution evidence; it is not itself a committed-transfer event in Σ. However, each executed non-blocking call is a distinct dynamic source-level instance $q \in Q_{exec,P}$ for its owning process P. Accordingly, continuous assertion of an endpoint request with no intervening commit does not by itself collapse later executed non-blocking call instances into an earlier q. Only a poll that succeeds contributes the corresponding committed transfer $m(q) \in M$ for that call instance. If a request assertion persists without a new executed source-level call instance being entered, its attribution remains with that same currently outstanding q; but repeated non-blocking polls in later loop iterations or later source-level call instances create additional $q \in Q_{exec,P}$ even if req does not deassert.
Silent step. A transition is written ε only relative to the selected observation or proof alphabet when it contributes no event or complete observable/proof unit in that alphabet. Unsuccessful non-blocking polls have no committed message-transfer event in Σ, but their dynamic call identities, outcomes and consumed control/value facts may still be required as witness or proof evidence. Arbitration bookkeeping and pipeline advances are ε only when they contribute no selected event or unit. An internal transfer retained by the proof alphabet is not ε there merely because an outer accounting or

DUT-observation projection hides it; Core.2/Core.3 and Core.ProofBoundary determine the selected boundary.
Implementation conformance Core.ImplIssueOrder (RTL message-issue discipline). The post-HLS RTL_B execution satisfies the full E3 obligation as an implementation-side scheduling obligation: (i) per-interface issue order is preserved for all message operations; (ii) for distinct interfaces, the no-reverse rule holds; and (iii) issue is prefix-closed under source order, so at every compared cutpoint a source-later issued message operation implies that every required source-earlier message operation has also issued, with Issue timestamps ordered accordingly. This condition is an explicit implementation requirement; it is the RTL/tool-compliance obligation corresponding to the scheduling rules stated in the main text and Appendix G. For hand-written RTL or other non-HLS implementations, including qualified TB runtime realizations when they occupy the implementation side of the selected comparison, Core.ImplIssueOrder is the corresponding proof/qualification obligation on that implementation. An external TB does not acquire this obligation solely because it supplies an environment contract; its selected representation and consumer determine applicability.

Definition Core.NBDecisionSnapshot (operational non-blocking decision snapshot).

For each represented dynamic non-blocking call q at explicit boundary c with Horizon t, ReqSnapshot_c(q) is the complete source-boundary valuation at the actual L2 SettlePoint of that occurrence, after the selected legal L1 preparation and request/attempt writes and before any cycle-t L3 decision return or channel update. Its key κ(q) identifies that exact source occurrence, Horizon and reset epoch. The snapshot contains the E4 beginning-of-cycle channel state and relevant explicit storage/credit; every attributed persistent request and one-cycle attempt affecting c, including q's own attempt and stable descriptor; and every declared gate, join/coupler, arbitration, control and environment field affecting the decision. Any selected grant or resolution is legally bound under Core.SourceChoiceBinding and included in the complete snapshot. The complementary handshake is the value produced by the complete qualified settling network from those roots. A later listed L3 action, FIFO update or favorable handshake does not replace this old-state snapshot. Snapshot identity and contents are operational facts, not chosen by a correspondence constructor or a workload witness.

Definition Core.NBDecision (non-blocking success and failure law).

For a represented, legally issued attempt q, Core.NBDecision(q,ReqSnapshot_c(q)) is the result prescribed by the following boundary law on that snapshot. At an ordinary uncoupled positive-capacity boundary B(c)>0, PopNB succeeds exactly when occ_c(t)>0 and PushNB succeeds exactly when occ_c(t)<B(c). At a coupling-sensitive positive-capacity boundary, the same storage test is necessary and success additionally requires the q-side settled complementary handshake MirrorAvail_E; thus success holds exactly when both conjuncts hold. All relevant explicit credit, gate and grant conditions belong to the complete declared handshake law. At a zero-depth boundary B(c)=0, success holds exactly when the matching complementary attributed request or attempt is present with the required transaction identity and value agreement and, if coupled, the q-side settled MirrorAvail_E condition holds. The successful rendezvous is one joint transfer; the two endpoint outcomes cannot describe separate or inconsistent transfers. In an elaborated model, use the selected explicit boundary's capacity and actual settled handshake, not nominal aggregate outer storage.
A successful attempt returns success and contributes exactly its corresponding committed transfer, with Push data from the stable descriptor and Pop data from the matching old head or rendezvous partner. Declared atomic groups remain indivisible and all participating outcomes must agree with that same selected transaction. A failed attempt returns false in cycle t as an ε decision with ReturnedAtCycle(q,t), produces no channel commit and causes no channel update attributable to that attempt; other independently selected legal actions retain their own effects. The return and local bookkeeping follow

the declared call semantics, and Core.RegSeq prevents dependent use before a later Horizon. Failure does not create a persistent blocking request. An enabled non-blocking attempt cannot be reported as failed merely because a selector omitted its successful transfer, and a completed edge may not omit a represented attempt's prescribed return. Unrelated blocking calls still retain legal issue latitude, but their actual sampled handshakes are mandatory under Core.TransferLaw. Source legality, scalar endpoint cardinality, atomic consistency, complete inventory and snapshot matching remain separate qualification obligations. WC and replay annotations record outcomes and never override this law. An alternative primitive or decision law requires an explicit semantic extension and the selected consumer's proof; this definition does not qualify arbitrary arbitration or widen a constructive or completion profile.

Definition Core.DeferredNBHole (carried non-blocking decision obligation).

A deferred hole for dynamic call q at boundary c is the proof-construction record NBHole(q,c,κ,t,dst,e,χ), also written NBPending(q) while unresolved, where κ is the occurrence-bound snapshot key of Core.NBDecisionSnapshot, t is the selected Horizon, dst identifies the returned status/data destinations, e is the reset epoch, and χ records the expected outcome supplied by legal witness selection or the selected constructive evidence together with NoDependentUse(q); the expected outcome must equal Core.NBDecision(q,ReqSnapshot_c(q)) and cannot override it. NoDependentUse(q) is a locally checkable assertion that no action executed while the hole remains unresolved reads NBResult(q), uses q's outcome to select reachability/control/endpoint/address/payload, or otherwise depends on the returned value/status. The dependence test is semantic and transitively closed: it includes aliases and pointer-derived accesses, loads from result-containing locations, derived dataflow expressions, and control, address, endpoint-selection, and payload dependence. The local evidence shall justify this closure using the normative field-sensitive location/footprint model rather than a purely syntactic scan for NBResult(q). The record also retains the attributed one-cycle attempt and every location needed to recover ReqSnapshot_c(q) under Core.NBDecisionSnapshot. It is ghost construction state, not an implementation state component. All deferred holes for Horizon t must be resolved or retired before the exported post-boundary construction state of t.

Definition Core.RegSeq (registered sequential-interface semantics).

The Core classifies each process for the Appendices J and K proof stack as either combinational under R2C or sequential under Core.RegSeq. For a sequential process P, every process-driven endpoint request and Push-data output assigned to cycle/Horizon t is a function of registered process or pipeline state and operands fixed before the L2 settle point of cycle t. Combinational derivation from that registered state is permitted, but the request and payload must remain stable for the selected boundary action. Direct inputs count as fixed operands only when their applicable E2, hls_direct_input, or hls_direct_input_sync contract makes their value stable at the relevant anchor. If ReturnedAtCycle(q,t) holds, any P-owned evaluation, control selection, endpoint/address selection, payload computation, or message issue that depends on q's returned data or success/failure status has Horizon strictly greater than t. A same-Horizon source-later action is permitted only when its reachability, control, operands, endpoint, address, and payload are independent of every unresolved cycle-t NB outcome and when its writes do not change the ReqSnapshot_c(q) of that unresolved decision, as defined by Core.NBDecisionSnapshot, except through an explicitly ordered dependency. This is a normative theorem-scope restriction, not a consequence of the word "clocked" alone. Tool-side Cat# or equivalent conformance evidence and source-side simulation/lint/witness evidence shall discharge it. Intentional zero-time outcome-dependent NB cascades or retry loops shall be identified by checked source/tool analysis and then rejected or explicitly excluded from the theorem instance, isolated as latency-sensitive logic, remodeled as one explicit atomic multi-way decision, or aligned by Appendix L; silent scope exclusion is not permitted. The fixed request/frame qualification includes each sequential process's

physical synchronization offer under Core.SyncProtocol. It does not by itself prove the inter-cycle old-user protection required for sync-controlled input updates.

Definition Core.PolicyL (liberal, witness-relative issue).

Policy L is the most permissive source-side issue policy used by the safety witness and qualified grouped/overlapped workload witnesses. It may issue a reached source-level message operation whenever the transition-system preconditions above, R3/R4/R5, E4, E5, Core.RegSeq, the applicable Core.SourceChoiceBinding operational decision law and prospectively bound choices, and the source-order prefix-closure requirement are satisfied. A source-later operation may have become reached through Core.ReachPrep even while a source-earlier same-interval blocker remains uncommitted. Policy L may therefore leave the source-later blocking request issued in that state. This is how the reference model represents loop overlap, ordinary dependency-independent eager preparation, RTL-matched eager issue, and other allowed same-cycle/overlapped request timing. Policy L is not a claim that every raw pre-HLS simulator mode can generate every such witness state. Policy L does not override returned-result availability: it cannot issue an operation in cycle t when its Core.ReachPrep reachability, control selection, endpoint or address selection, payload, or any required operand depends on data/status first returned at the cycle-t boundary. Core.LLegal fixes Policy-L step legality, Core.Phase fixes the cycle-settle boundary and the trigger for Core.16. NB outcomes are computed when their operational decision rule applies; SCB3 annotations record those outcomes and do not independently authorize issue or select future results.

Definition Core.LLegal (Policy-L step legality and fixed-instance binding).

Fix a selected liveness instance I consisting of Sys_ref, the initial/reset state, capacities and elaboration/lowering, the fixed stimulus and reactive environment, witness-selected non-blocking and other ε outcomes, arbitration choices, and the selected synchronization, signal-anchor, and ε-normalization conventions. A finite or infinite path ρ is Policy-L-legal for I iff it begins in the selected initial/reset state, every transition is a transition of the operational Sys_ref relation above permitted by Policy L, and every fixed choice in I is respected. R1–R5, E4/E5, dynamic-operation identity, request attribution and persistence, endpoint cardinality, reset-epoch typing, and sequential-result availability are part of this step legality. Core.SelectLatitude is constitutive of this source legality: a legal finite source cycle or prefix need not maximize L1 issue or genuinely abstract internal actions, but its completed edge includes every Core.TransferLaw and Core.SyncProtocol completion on the fixed settled sample. Core.SourceChoiceBinding distinguishes this liveness-instance choice binding from the more general safety-instance convention. Witness-selected outcomes must satisfy the operational decision laws; recording them never overrides those laws.

Definition Core.SelectLatitude (finite-prefix issue and abstract-action latitude).

Fix a phase-bearing source theorem instance I and issue policy $\Pi \in \{\text{Policy L}, \text{Policy S}\}$. Core.SelectLatitude permits a finite legal delay of an otherwise reached L1 message-issue transition, subject to Core.Phase, Π, Core.16, R1–R5/E4/E5, fixed choices and every other issue rule. It also permits only the selection latitude expressly declared for genuinely abstract internal movement or preparation actions. Prospective arbitration choices may affect the sampled physical handshake through the declared complete network. No latitude permits omission of a Core.TransferLaw message transfer or Core.SyncProtocol synchronization completion once its eligible-edge handshake/release holds. Nonselection of issue does not withdraw or reattribute an existing request or alter stable data. A cycle may have an empty observable commit set only when the complete mandatory set projects to empty and all other operational rules permit that cycle; such a SilentCycle still performs its declared full-state updates. Core.IssueFair, pre-grant B2 and applicable internal service are separate complete-execution

obligations within their scopes. Policy S restricts issue legality without requiring earliest issue; it does not weaken either mandatory transfer law.

Definition Core.Phase (Policy-L cycle phases and settled-frontier trigger).

Each cycle t of a Policy-L-legal path is interpreted using the following normative phase boundary.
(L0/L1) Cycle entry and pre-settle issue window. Starting from the cycle-entry state after all prior-cycle boundary updates, sequential processes may take a finite selected sequence of pure source-evaluation ε-steps and message-issue steps for operations that were already reached at cycle entry or become reached during this phase through Core.ReachPrep and evaluation independent of every cycle-t boundary result. Their control choice, endpoint, address, payload, status inputs, direct-input values, and other required operands must be available under Core.RegSeq independently of any cycle-t commit or failed/non-failed NB decision outcome. These issue steps remain subject to R3–R5, source-order prefix closure, stable payload, and the deferred-hole restrictions of Core.DeferredNBHole. Thus a source-later outcome-independent operation, including an ordinary straight-line successor admitted by Core.ReachPrep, may issue during this window while a source-earlier blocker or unresolved NB attempt remains uncommitted/undecided. A value or status first returned at the cycle-t boundary cannot be used in this window. The selected L1 issue sequence is not required to be maximal: under Core.SelectLatitude an otherwise legal issue may remain unselected at this L1 opportunity, subject to Core.16 and every other step-legality condition. Any theorem that constructs a complete/fair source continuation additionally requires the selected L1 preparation/issue segment to terminate finitely as part of Core.CycleClosure.
(L2) Settle point. Close the selected finite L1 window, hold its requests and payloads fixed, and evaluate the complete phase-legal L2 network to SettlePoint_I(t,σ_t^set). Core.SourceL2Settle supplies existence under its stated qualification. This is the settled snapshot at which Core.16 tests the pending frontier and activates the no-new-later-work discipline.
(L3) Freeze and boundary action. Evaluate Front_P and ChannelEnabled/ChannelDisabled at σ_t^set. If Front_P is nonempty and every item is disabled, Core.16 activates immediately: no new source-later operation may issue after that snapshot or while the same fully disabled frontier persists. Already-admitted work may progress only through Core.16-permitted actions. Independently of that per-process frontier test, derive the complete model-wide physical message transfer set from Core.TransferLaw and every synchronization completion from Core.SyncProtocol, with the fixed atomic grouping, reset convention and anchored effects. The qualified equations establish E4, ownership, fence and endpoint-cardinality legality; a violation is not permission to suppress an overlap. Execute that fixed set by the justified simultaneous update or residue representation, together with any separately legal abstract internal actions. For a sync, account for its already justified same-edge process-facing predecessors before interval advance. No updated state, changed grant or re-settled value may add or remove a cycle-t transfer. L4 installs the resulting registered state; old-state FIFO tests still prohibit fall-through and full-FIFO replacement.
(L4) Next-cycle result availability. Boundary commits and failed non-blocking decisions produce their process-visible ReturnedAtCycle results and update registered process/channel state for the next cycle. Process-owned evaluation or issue whose reachability, control, endpoint, address, payload, or required operand depends on a value or success/failure status returned in cycle t may begin only in a strictly later cycle. Every deferred NB hole created for Horizon t is resolved or retired before the exported post-boundary construction state for that Horizon.
The L2/L3 freeze is not retroactive. An issue that was legal in the L1 pre-settle window is not invalidated merely because the subsequent settled snapshot exposes an earlier fully disabled frontier. Conversely, once that settled snapshot exists, an operation not already issued before it cannot use the same-cycle pre-settle window to bypass the activated Core.16 block.

Dependency convention. Core.Phase supplies the settled trigger and legal phase order. Core.16 supplies the activated operational restrictions. Constructive source extensions obey both directly. B.OF/B.CT add positive implementation movement, request preparation and service requirements; those do not follow from the source restrictions alone.

Definition Core.PolicyS (conservative serial-blocking issue).

Policy S uses the same Sys_ref state space, channels, values, witness choices, and transition rules as Policy L, but adds one process-side restriction: within a source interval between adjacent ordinary synchronization calls selected into S, a later source-order blocking Push/Pop operation may not issue until every earlier source-order blocking Push/Pop operation in that same interval has committed in a strictly earlier clock cycle. In Sys_B^elab, the relevant commit for this restriction is the process-facing commit that completes, accepts, or releases the owning source-level blocking operation at its selected boundary; internal staged/bundle/join/epoch-gate transfers do not by themselves satisfy the serial condition unless the elaboration package identifies that transfer as the process-facing commit for the source operation. Policy S serializes only process-side issue of source-level blocking calls. It does not serialize independent peer processes, E4 channel-side drainage, FIFO movement, or ε-steps that are otherwise admissible. Core.SelectLatitude applies under Policy S as well: the serial rule restricts which issue transitions are legal but does not require an earlier legal issue to be selected at its first opportunity. Core.IssueFair remains the complete-execution eventual-issue condition and B2, when invoked, governs pre-grant arbitration eligibility rather than post-handshake selection; a later deterministic Policy-S instantiation may separately fix the concrete choices without changing this legality.

Remark Core.PolicyS-Sim (Relation to practical pre-HLS simulation modes).

A raw serial source simulation mode implements the Policy-S discipline for ordinary unelaborated blocking calls. A Matchlib-style eager or skidbuffered source simulation mode may execute source calls in dynamic program order while issuing requests without consuming a clock cycle until data or space is unavailable; that mode is Policy-L-compatible only to the extent that its request attribution, channel states, and non-blocking outcomes satisfy the transition-system conditions above. Neither mode treats ordinary distinct-interface source calls as unordered. The difference between serial and eager simulation modes is a difference in issue timing and buffering, not a difference in <_src. Lazy movement is a separate witness-selection choice, not a third process issue policy. Core.PolicyS or K.CT.GroupSerialWitness controls source blocking-call issue; K.CT.LazySimulation restricts only permitted abstract staging moves using the matching source Pop or whole-batch demand. Capacities, boundary equations, mandatory physical transfers and the permissive reference used by J are unchanged. For lazy simulation, the source runtime must expose legal process-facing demand independently of item arrival. Lazy completion simulation is distinct from a separately qualified performance/timing model.

Engineering interpretation (informative). Policy S is intentionally conservative about request overlap, not about pipelining as such. Its potential false-negative case is concentrated in protocols whose liveness requires source-later blocking requests to be exposed before earlier blocking calls complete; the simplest example is a crossed two-process/two-rendezvous-channel pattern. In a concrete buffered or elaborated instance, any HLS-created storage or credit that affects acceptance/backpressure is represented by B(c) or explicit Sys_B^elab structure under B-faithfulness. If that stored slack breaks the wait dependency, the selected instance is no longer the rendezvous-only case and Policy-S applicability is assessed on the actual buffered/elaborated architecture. Thus, although the rendezvous counterexample is important for stating the theorem correctly, the corresponding Policy-S conservatism is expected to be much less significant in practice for modern pipelined systems with buffered or credit-

based communication. This observation does not prove completion: finite buffers may fill, and K.CT still requires a legal accepting witness and its closure/resource obligations. Failure of the serial witness is inconclusive; grouped or overlapped witnesses require their own qualification. This stored-slack intuition does not imply that a lazy witness can use all nominal staging capacity. Under lazy selection, an item may leave ingress only after its matching process-facing demand issues. For example, if a producer sends more items than ingress can hold and only then sends a start message, while the consumer waits for start before issuing any data Pop, empty downstream staging does not rescue that lazy run. Its failure remains inconclusive about a particular RTL implementation; a stronger qualified forwarding profile or another qualified witness may succeed.

Shared evidence and finite-prefix agreement

Definition Core.EvidenceHdr (route-neutral evidence binding). A theorem application records one fixed instance identity binding source/reference and implementation artifacts; the selected reference and elaboration, boundaries, capacities, projections and ownership; initial state, stimulus, environment, epoch/reset and clock contracts; fixed roots, witness choices and their legal realization; the observation and logical-key conventions; the claimed conclusion and its coverage domain; the exact consumed interfaces and versions or content hashes; side-condition evidence and its validity keys; generator/checker identities and the declared trusted base. Reusable evidence may be shared only through a checked binding that preserves each consumer's instance, scope and validity conditions. Each consumed normative interface is identified by its semantic revision or definition-level content hash as well as any serialized schema version. Evidence reuse does not relax prospectiveness, fixed-domain, construction-order or other consumer-specific conditions, and different instance identities remain distinct. Unverified tool facts admitted as axioms are named in the trusted base; independently checked facts name their checker and evidence. The header records evidence; it does not discharge any premise. Route-specific records extend this header only with fields whose named consumers require them. A route-neutral header does not require any deadlock observation, issue-policy comparison, response-monitor, waiver or reconstructed-cutpoint field.

Definition Core.EvidencePackage (common package integrity). A certificate-backed or proof-backed application supplies Core.EvidenceHdr and an authoritative consumed-interface manifest, together with the subordinate evidence required by each selected theorem and M.7a trigger. The manifest binds exact artifact and model identities, parameter/observation/coverage domains, semantic revisions, schema versions, checker identities, evidence validity keys and the trusted base. For J construction uses this includes every consumed preparation, choice, action/footprint, PhaseRole_A5 and projection-group/order interface, the applicable source-realization obligations of J.SourceSettleRealization/J.SourceSettleExists and J.LocalBoundaryCompose, the exact-prefix and bound-choice preservation required by J.ConstructiveEdgeInduction, and the old occurrence-map and historical-restriction preservation required by C5 and J.LocalOriginCanonExt; for cycle/carrier uses it identifies the consumed occurrence and epoch interfaces, the applicable Core.QSync QS2 registered/input/request frame and settling obligations, Core.ConstructiveOriginCarrier designation and transport obligations, and the completion and prefix-coverage requirements of the selected registered carrier profile. A field required by an invoked interface cannot be marked inapplicable merely because the final conclusion is safety or completion. Only evidence required by the selected consumers is mandatory; all applicable obligations of those consumers remain required. This common contract adds no correspondence, coverage, progress or semantic assumption beyond the named consumer obligations.

Definition Core.ExtensionConsumerCoverage (open-class consumer completeness).

When a selected normative interface admits an extensible class of actions, complete units, primitive effects, predecessor generators, projection cases or carrier witnesses, the instance shall enumerate its admitted members or provide a prospectively defined typed family. For every admitted member and every applicable consumer, provide the typed realization/proof case required by that consumer, or prove that the consumer is inapplicable to that member within the declared domain. A generic case is permitted only with a proof quantified over the entire class it claims to handle. A prose catch-all, an omitted case or a successful sample shall not stand for that proof. An unsupported member makes the affected application unqualified; it may not be removed from the operational behavior domain merely because construction failed.

The consumed-interface manifest under Core.EvidencePackage records this member-to-consumer mapping, including applicability proofs and semantic versions. For constructive uses it covers the selected local action/frame/settling, source-generator, canonical projection and carrier consumers; full-fact and completion uses additionally cover every consumed fact, dependency, exposure and service case. Q.CT.Extension and other qualified extensions retain all existing owner-specific requirements; this rule does not admit a new operational case by declaration alone. Extension and reuse checks update the mapping and M.7a record whenever an admitted class or consumer changes. J.GeneratorCompleteness remains the particular owner of the full required-order generator inventory.

Definition Core.EvidenceRealization (proof families and executable evidence).

A parameterized mathematical proof of existence over a prospectively declared admitted domain is acceptable without an executable extractor, unless a selected interface explicitly requires executable evidence. Its quantifiers, fixed choices and produced witnesses must meet that interface; an existence assertion without its proof is not evidence. Keep the reusable theorem and its instance data distinct. Classical existence does not by itself supply an executable algorithm.

If an extractor is claimed to realize that proof family, prove soundness of every returned witness against the exact source semantics and consumed interfaces, and termination and totality of witness production on the declared qualified input domain. These obligations do not require an extractor to accept inputs outside that domain. A partial generator may instead supply individually checked evidence for successful instances, but its failures or successful samples establish neither domain-wide existence nor coverage. A checker used to accept generated evidence needs a soundness argument for acceptance under its stated trust base; soundness alone is not completeness or generator totality. Bind these evidence modes, hypotheses, schemas, versions and trust assumptions in Core.EvidenceHdr/Core.EvidencePackage and M.7a.

Definition Core.RequiredAcceptanceObserved. Fix in Core.EvidenceHdr a prospective finite testbench acceptance contract R: logical occurrence identities, expected values or an explicitly qualified acceptance predicate, and required multiplicity and per-endpoint order, together with its interpretation in each compared model. At an exact legal finite prefix π of a bound model, RequiredAcceptanceObserved_I,R(π) holds when the recorded testbench acceptance events satisfy that contract. Assertion of an output request, an arbitrary host stop or absence of further stimulus is not acceptance. This predicate records what happened at π; it asserts neither that another execution reaches such a prefix nor that all work has drained.

Definition Core.DrainedTestObservation (optional diagnostic). In addition to Core.RequiredAcceptanceObserved at an exact legal observation prefix, record a prospectively declared scope and qualified complete FIFO/pipeline/storage inventories that are empty, that every remaining attributed blocking request in that scope is a Pop on an empty channel, and that the declared environment has no future stimulus for this test. Inventory completeness, outstanding return/update treatment and observation/simulator realization must be checked. Empty monitored FIFOs alone do not

suffice. This stronger diagnostic is optional and is not a premise of required-workload completion; it provides no universal eventuality, request cancellation or deadlock-exclusion conclusion.
Definition Core.FinitePrefixProvenance. Fix a common semantic fact-key universe and an independently specified domain of legal finite source prefixes under one Core.EvidenceHdr. The domain shall include the selected workload witness and every source prefix used by the consuming correspondence; it may not be restricted retrospectively to prefixes agreeing with that witness. Keys identify dynamic operations, historical observation occurrences or immutable observation versions, and boundary/epoch transfer ordinals. A later observation at the same reusable site is a different key. Realization maps are partial and correctly attributed; a fact's absence is not an occurrence. Fixed constants, stimulus and legally fixed choices are common roots. Every non-root fact b has a fixed finite predecessor set Pred(b), a deterministic typed evaluator Eval_b, and a common well-founded relation containing every predecessor edge. No edge is introduced merely by simultaneous request enabling or shared clock-cycle membership.
Core.FinitePrefixProvenance availability and adequacy. Avail_π(b) means that b is fixed and legally consumable at the exact prefix π from roots and already realized historical facts. No future issue, transfer, observation or update may be used. If b is available, every evaluator predecessor is available there; at b's evaluation/consumption point its value is Eval_b on those legally available inputs and common roots. The certificate validates these laws against the actual source semantics, Core.RegSeq/Core.ReachPrep and the declared causal observation mechanisms. For an operation fact, Eval determines the optional descriptor none/some(desc), including control and iteration selection, rather than assuming the operation exists. For a transfer fact, it validates producer-key attribution, endpoint ordinal and FIFO/rendezvous semantics. A shared key retains its immutable historical value throughout its allowed lifetime; reset or rebinding cannot silently substitute a different fact under the same key. Epoch-indexed keys and a checked no-affecting-reset scope are one sufficient realization. These conditions establish lawful value provenance, not availability of an unexecuted predecessor or progress of its producer. Occurrence-to-availability requirement: every F fact occurring at any prefix in the admitted provenance domain, including the workload witness and the source prefixes supplied by J.InternalFactCorr, has its keyed semantic value fixed and available as a historical fact at that prefix and throughout its permitted lifetime. This does not authorize process computation to consume a returned value before Core.RegSeq permits it. An event that has not yet fixed the claimed semantic fact must use the later value-defining occurrence, rather than claiming this bridge prematurely.
Core.FixedSupportValidation (guidance for data-selected dependencies). A workload witness may propose a fixed finite Pred(b), but its observed support alone is not a certificate. For a conditional read, skipped iteration or data-dependent source path, identify every earlier control/selection fact needed to determine the consumed predecessor keys. Validate those selectors and their attribution over the independently specified provenance domain before relying on the witness-selected support. Fix the proposed support sets, evaluators and one common well-founded relation for the bound instance, not separately for each compared prefix; include all consumed selector facts and recursively required historical support.
A sufficient validation proceeds by well-founded induction, establishing availability/adequacy and agreement for lower-ranked selector facts before using their values to identify a selected predecessor. Prove from the actual source semantics that, whenever b is available or occurs in any admitted prefix, the selectors have been legally determined, select the same keys as the proposed fixed binding, and the required selected historical facts are available at that prefix with the correct occurrence/epoch attribution. Establish their lower-ranked provenance and evaluator adequacy before applying the agreement step for b. Do not invoke Core.FinitePrefixAgreement for an unvalidated certificate, assume agreement of b, infer availability from value equality, or restrict the domain to witness-agreeing prefixes. Unselected alternatives need not be in Pred(b); inserting every potential alternative is invalid if

it demands availability of unexecuted facts. A negative descriptor is supported by its actual decision facts, not by a fabricated completion of an absent operation.
This is a way to discharge the existing fixed-Pred interface, not a prefix-dependent predecessor definition or an automatic certificate for all data-dependent programs. If a common fixed support and the required domain-wide availability/adequacy cannot be proved, the default provenance route is unavailable for that binding; K.CT.Agreement retains its explicitly permitted independently proved agreement alternative. No availability, reachability or liveness conclusion is obtained from the workload witness alone.
Lemma Core.FinitePrefixAgreement. Under Core.FinitePrefixProvenance, any fact available in two prefixes of the admitted domain has the same typed value in both. Proof: well-founded induction on the fact relation. Common roots agree. For a non-root b available in both prefixes, predecessor closure makes each a in Pred(b) available in both. The induction hypothesis identifies those values; the same Eval_b and common roots identify b. Operation descriptors and transfer payloads are instances. The result is conditional on actual availability/occurrence and proves no reachability or eventual issue. Endpoint order additionally follows only from the separately bound operation/ordinal attribution and channel-order laws. Primitive joint rendezvous enabling is not circular value provenance. This proof uses no source-policy dominance, complete fair execution, frozen component or response monitor.

Commit, issue, and boundary requests

Definition Core.8 (Commit).

A boundary-commit occurrence at an explicit abstract boundary endpoint is the corresponding legal E4 channel-commit transition of the selected proof model: it produces the applicable committed Push_c(v) or Pop_c(v) observable unit and the channel-state update prescribed by the selected rendezvous or bounded-FIFO semantics. For RTL_B, the selected explicit-boundary mapping and B-faithfulness/conformance evidence relate that abstract commit to the implementation clock cycle in which the mapped valid and ready conditions are simultaneously satisfied, with mandatory transfer in both models under Core.TransferLaw; the committed payload is the mapped data value observed at that boundary in that cycle, except for the explicitly qualified internal logical-Pop attribution of Core.LogicalPopPayload. For a source-level message operation q, Commit(q) denotes q's process-facing commit occurrence. In the ordinary unelaborated case, this is the boundary-commit occurrence at q's unique selected explicit abstract boundary. In Sys_B^elab, it is the commit/accept/release occurrence designated by the elaboration package as completing or releasing q; proof-internal boundary commits that do not release q are not Commit(q). Let CommitCycle(q) denote the cycle index of Commit(q). When a rule compares q's process-facing commit with a clock cycle, it compares CommitCycle(q). Per-boundary E4 commit occurrences at explicit abstract channels retain the cycle index of their own boundary-commit occurrence and are not mapped to CommitCycle(q) unless that boundary commit is the designated process-facing commit for q. Process-facing commit identifies completion/release of the source operation; it does not imply that returned data or status is available for dependent use in that same cycle, which remains governed by Core.Phase L4 and Core.RegSeq.

Definition Core.TransferLaw (mandatory settled message transfer).

(TL1) Complete interface scope. Fix before correspondence construction every actual ready/valid transfer interface declared by Core or elaboration E, its signal roles/data, sampling edge, reset convention, endpoint ownership, dynamic occurrence/epoch attribution and atomic/compound membership. The law applies to process-facing, proof-internal and other explicitly declared physical transfer interfaces; internal location or later projection does not exempt an interface. A genuinely abstract internal movement permission is governed by its declared operational law, not automatically by

ready/valid. If an abstraction hides physical transfers, its realization proof must account for every concrete transfer and preserve the abstract updates, observations, owners and retained state. Naming a physical handshake an internal move is not such a proof.

(TL2) Transfer equation. At each eligible sampling edge t, evaluate the complete declared network from the fixed pre-update state, request/payload roots, environment inputs and prospectively bound arbitration choices. Edge eligibility is fixed by the clock/reset protocol; a denied grant, false transfer guard or undesired effect cannot relabel a sampling edge as ineligible. For every declared ready/valid interface b, Fire_b(t) holds if and only if its mapped valid_b(t) and ready_b(t) are both asserted in that settled sample. Fire_b commits exactly the declared transfer occurrence and payload on that edge. For each represented message frontier item x, the fixed interface/owner mapping proves equivalence of that attributed transfer condition with BoundaryReq(x) and MirrorAvail_E(x), hence with ChannelEnabled(x) at the settled sampling point. The persistent operation-owned request and any physical signal gated by a guard are mapped separately; the mapping must account for the gating in the complete settled equation and preserve issue and attribution. An actual handshake is neither optional nor retrospectively suppressible. This law does not turn an arbitrary later observational ChannelEnabled value into another same-cycle transfer. Core.10 gives two sufficient gated-request bindings: GQ1 places the declared E gate in the selected boundary's actual complementary signal while retaining its persistent request; GQ2 makes the gated own-side level the request and permits gating only before issue, with no blocking-request withdrawal afterward except affecting reset. Both require the actual boundary/refinement, identity, lifecycle and transfer-equivalence proofs.

(TL3) Legality before overlap. E4 old-state capacity/order, endpoint cardinality, request ownership, reset priority, guard/resource eligibility and joint/atomic coherence are conformance conditions on the settled signals and their declared effects. All blocking conditions must be represented in those signals or their qualified state; none is a second veto after sampled overlap. Overlap with an illegal effect is a conformance failure, not a legal no-transfer choice. Declare reset-only versus transfer-eligible edges and any permitted transfer-before-reset ordering prospectively, as in Core.SyncProtocol SP4. Reset cannot erase a transfer that already committed. No updated occupancy, newly returned result, changed grant or re-settled signal creates an additional transfer on the same edge. All ordinary FIFO decisions use beginning-of-cycle occupancy and the old head; no same-edge fall-through or full-FIFO replacement is introduced.

(TL4) Complete action set and event identity. Derive the entire physical transfer set from the one settled sample, then apply the fixed event/operation mapping and compound-unit grouping. A paired rendezvous or declared atomic unit commits all its legs coherently; independently true endpoint conditions are not a substitute for the actual joint equations and group inventory. Prove which endpoint views denote the same occurrence and which denote distinct effects. Statelessness alone proves neither event identity nor a one-event transaction. Every complete unit, relevant proof-internal transfer and NB result required by the declared model is accounted for exactly once, even if outer projection later erases it. A proof may serialize this fixed set by a justified residue order, but cannot omit a forced member or add one using an intermediate update. Process-facing Commit(q), physical capture and later external delivery remain distinct unless their mapping explicitly identifies them.

(TL5) Choices and progress. Legal L1 preparation/issue latitude and prospective pre-handshake arbitration choices remain under their declared rules. They resolve before the complete settled sample; they cannot choose whether an existing handshake commits. Core.SyncProtocol separately governs synchronization and its anchored effects. A failed NB attempt must satisfy its operational decision law on the same bound snapshot; nonselection cannot manufacture failure when its transfer condition holds. A persistent request at a primitive uncoupled FIFO whose obstruction has cleared commits at the next eligible settled edge, if that edge occurs without an affecting reset; no B2 fairness is needed for that already enabled transfer. This does not prove the clearing event or a future edge. At coupled

boundaries, other inputs and grants can still withhold the handshake; their exposure, resource and pre-grant service obligations remain. Finite-prefix correspondence supplies none of those liveness premises.

Definition Core.LogicalPopPayload (causal payload attribution for internal logical Pops)

Scope and event. A prospectively declared internal process-facing blocking Pop satisfying the representation and operational clauses of Q.CT.LogicalPayload for a singleton, or of Q.CT.LogicalPair for one member of its declared pair may carry the logical value of its uniquely matched admitted item although that original value is no longer present on its physical datapath. The endpoint is outside Σ_DUT and every other physically sampled observation interface consumed by the theorem instance. Its retirement must still be an actual mapped implementation accept/release event with its actual cycle, dynamic-call identity and request lifecycle. This exception does not create a transfer, change readiness, retime an event or relabel any physically sampled value. Other boundaries retain Core.8's mapped-data observation rule. For the pair, both actual member retirements form one indivisible mapped unit, with two distinct keyed payloads; the exception cannot split that unit.

Admission and ownership. At each actual accepted input Push, a causal observer records that event's actual payload under its fixed channel/epoch ordinal. It follows the implementation's selected old-item transfers and preserves exactly one owner location per accepted, unretired item, in FIFO order. On a logical Pop retirement it selects the old terminal owner and defines that Pop's payload to equal the owner's recorded admission payload. Newly arriving replacement data cannot supply that retirement. Initial storage is empty in this profile, and no affecting reset, cancellation, logical-item merge, duplication, side exit or ownership reassignment is permitted. For Q.CT.LogicalPair, each member uses its own channel ledger and old terminal owner. The pair relation fixes corresponding iteration ordinals; a joint retirement emits both member values together, without merging the logical items or their per-channel accounting. Under Q.CT.PairedRepresentation, one physical allocation may represent both distinctly owned items of (j,j). Each remains counted once on its own channel until the actual joint retirement. Joint physical representation is not a logical-item merge, a shared-capacity pool or a new source event.

Nonrestricting observation. The observer is proof state, not implementation storage or resource credit. It has a total, uniquely determined extension for every admitted physical successor of every related prefix, including unfair, stalled and unsuccessful runs; erasing it recovers exactly the physical execution. It does not drive enabling, action selection, payload computation, request preparation or register updates and cannot discard physical successors. Values are fixed at their actual admissions, not chosen from a workload witness, a future event, a desired output or a retrospectively repaired history. A finite-prefix admission ledger may retain historical values without increasing live channel capacity.

Representation and use. Q.CT.LogicalPayload, with Q.CT.LogicalPair where selected, proves that the physical representation belongs to that same admitted item and that every later source-attributed use agrees with legal source evaluation on its logical returned value. Core.16's effect and legal-later-use conditions, Core.ReachPrep, Core.RegSeq, E3 and all occurrence/footprint obligations remain in force. Inversion of the physical computation is not required; a bare ghost value or FIFO ordinal without the representation/use proof is insufficient. For a qualified joined pair, Q.CT.PairedRepresentation replaces separable per-item physical values by a representation of the same two admitted owners; the actual legal later source expression must still agree.

Facts and projection. The selected logical Pop event, including its value, is defined by this contract before π_Σ,E and ρ_DUT are applied. Those projections retain their existing no-creation, no-duplication and no-relabeling obligations. The full consumed-fact map must explicitly identify this logical interpretation; it shall not assert that the original value appeared on the transformed terminal wire. Physically observed admissions, outputs and other observations retain their actual values. All other J and K premises remain separate obligations.

Definition Core.9 (Issue).

For a source-level message operation instance q at a chosen explicit abstract boundary endpoint, Issue(q) is the first cycle in which the calling process begins requesting the transfer corresponding to q at that endpoint. Issue(q) denotes the cycle index of that operational issue event. Scheduler/witness records may carry this value for rules such as R4, E3, and E5, but they do not define a different event or an independently selectable timestamp.

Definition Core.IssueFair (weak fairness of source issue).

Fix an issue policy Π ∈ {Policy L, Policy S} and a complete phase-bearing source execution ρ. IssueEnabled_Π(q,t) holds at an applicable L1 opportunity when q has been reached, has not yet issued or been retired/reset, and its message-issue transition is legal under the operational Sys_ref relation, Core.Phase, Π, R3–R5/E4/E5, Core.RegSeq, source-order prefix closure, Core.16, and the fixed witness/environment/reset/arbitration choices. IssueFair_Π(ρ) holds iff, whenever from some cycle t0 onward within one reset epoch q remains IssueEnabled_Π at every applicable L1 opportunity until it issues or the execution reaches an affecting RESET for q or an explicit DeclaredTerminal disposition, q is issued after finitely many such opportunities. This is weak fairness: an operation whose issue legality merely recurs intermittently does not satisfy the antecedent. An operation forbidden by Core.16 is not IssueEnabled_Π and imposes no IssueFair obligation. IssueFair governs assertion/issue of a reached source operation; B2 separately governs qualified pre-grant arbitration, B3 supplies its conditional primitive discharge interface, and Core.TransferLaw/Core.SyncProtocol govern actual sampled commitment. A constructive complete-source continuation theorem may use a stronger recurrent-opportunity dovetailing scheduler, but the required property remains Core.IssueFair itself. K.CT consumes Core.IssueFair whenever a source eventual-issue argument invokes this complete-execution property; a supplied finite witness alone does not invoke it.

Definition Core.10 (Endpoint-owned boundary request).

Fix an endpoint direction at an explicit abstract boundary. For any message-operation frontier item x ∈ Ω_msg,P with owning source-level message instance q = MsgOf_P(x) and evaluated boundary c = BoundaryOf_P(x), let req_x(t) denote the endpoint-owned boundary request signal for that endpoint in cycle t, namely valid for a Push endpoint and ready for a Pop endpoint. The predicate BoundaryReq(x, σ) records that this process-owned request for frontier item x is asserted at state σ. BoundaryReq(x,σ) is operation- and boundary-specific: it is true only when the asserted endpoint-owned request at σ is the request of the specific frontier item x, its owning source-level operation q, and its evaluated boundary BoundaryOf_P(x). On RTL_B, identifying that process-owned request is an implementation-side boundary-mapping/conformance fact. For each admitted boundary/direction, the selected mapping is fixed for the theorem instance and shall exhibit the implementation request condition and owner decode; it is bidirectionally complete (BoundaryReq holds exactly when that fixed mapping identifies the applicable x/q as the process-owned request) and may not vary by cycle or dynamic instance to satisfy E3/E5. A raw physical valid/ready level that instead represents complementary channel/storage availability or declared coupler logic is not BoundaryReq merely because it is high. In particular, if Issue(q) has not yet occurred at σ, then BoundaryReq(x,σ) is false. In the ordinary unelaborated case, where q has a unique frontier item and a unique evaluated boundary, BoundaryReq(q,σ) may be used as shorthand for BoundaryReq(x,σ). In Sys_B^elab, the x-form is authoritative. Elsewhere, “operation-attributive” means operation-specific in this Core.10 sense, not witness-selectable attribution.
Two sufficient forms of a gated-request binding are available, subject to the fixed boundary, attribution and realization proofs.
(GQ1) Declared complementary gate. Keep the operation-owned request persistent at the selected evaluated boundary and declare the gate in E’s complete coupler network as part of that boundary’s

actual complementary signal. ChannelEnabled then uses the persistent request and this gated complement. For example, at a declared Push-side boundary, request r and complementary ready g∧ready_peer give Fire=r∧g∧ready_peer. If a concrete downstream interface instead exposes valid=r∧g and ready=ready_peer, the binding must establish the exact transfer/occurrence correspondence between the two declared views, including all effects, payload and storage. The upstream complementary value is the declared network's signal at that boundary, not a fictitious replacement for the downstream raw ready. The dual Pop binding applies the same principle. No storage, event identification or simultaneous multi-leg transaction is inferred from statelessness alone.
(GQ2) Gate before issue. Choose the gated own-side ready/valid level itself as the attributed BoundaryReq. Its first legal assertion is Issue(q); an earlier internal intention or pending-control bit is not that request. The gate may delay this first assertion, but for a blocking instance the asserted request and required stable payload/descriptor must then persist through commit or an affecting reset under Core.IC. A later stall cannot withdraw or reattribute the still-outstanding request. This binding must satisfy the existing source reach/preparation, E3 issue-order, Core.16 and synchronization rules; it cannot relabel an already issued request as not yet issued. In a standard early-staging progress proof, it must also satisfy the independent placement/request obligation: an arrival-dependent gate cannot prove request assertion before the arrival it assumes.
Both forms prove the exact Core.TransferLaw equivalence at the selected boundary. MirrorAvail_E retains its complementary-signal meaning there. Internal pending=1, gated physical own-side level=0 and raw peer complement=1 do not themselves constitute a handshake. A different gate or refinement organization needs its explicit correspondence and lifecycle proof; neither form permits a post-overlap veto.

Definition Core.IC (Issue/Commit linkage invariant).

Fix an endpoint direction at an explicit abstract boundary. For source-level message operation instances on that endpoint direction, the following all hold:
(1) Boundary-request soundness / non-withdrawal.
If q is blocking, then once issued its endpoint-owned request remains asserted, with stable payload while asserted for Push, until q process-facing commits or an affecting RESET governed by RW2 clears that dynamic instance. Ordinary back-pressure, scheduler delay, a silent cycle, or any other nonterminal transition cannot deassert, retire, reattribute, cancel, or replace it. These are the only nonterminal mechanisms in the G–K theorem model by which an outstanding blocking dynamic instance may cease to be outstanding: process-facing commit or an affecting RW2 RESET. At every cycle-aligned state, an asserted process-owned blocking BoundaryReq(x,σ) has exactly one live owner q=MsgOf_P(x): q has issued and has not yet process-facing committed or been cleared by an affecting RW2 RESET. After that ownership ends, any continuing high physical level is not BoundaryReq unless clause (3) transfers ownership. A DeclaredTerminal disposition may instead end the execution with q still pending. Protocols that require independent nonterminal cancellation, withdrawal, retry, or reattribution of a blocking request are outside the G–K theorem model unless an explicit operational extension and its correspondence/progress obligations are separately defined and proved.
(2) Stable attribution while the same source-level request remains outstanding.
If a blocking request remains asserted across a cycle in which no transfer commits on that endpoint direction, the continued assertion remains attributed to the same outstanding q until its process-facing commit or an affecting RW2 RESET. For non-blocking PushNB/PopNB, continuous assertion of the endpoint request signal does not collapse later executed non-blocking call instances into the same q; each later executed source-level poll is a distinct q′ ∈ Q_exec,P unless the witness explicitly identifies the same still-outstanding source-level request instance.
(3) Back-to-back request attribution.

If blocking q_o commits in cycle t and the same process-owned endpoint request remains asserted in cycle t+1, let q_1 be the next same-direction blocking source-level message operation in the same source interval, if one exists. If q_1 is that next operation, the continuing request is owned by q_1 beginning in cycle t+1 and Issue(q_1)=t+1. The process-owned blocking request may not remain asserted but ownerless for one or more later cycles and then be attributed to q_1. Core.SelectLatitude still permits genuine scheduler/HLS delay before q_1 issues, but then q_1's process-owned BoundaryReq has not yet begun; a high physical ready/valid value that is not q_1's process-owned BoundaryReq is interpreted through the fixed selected boundary/channel/elaboration mapping rather than through this clause. Non-blocking calls retain the separate clause (2) treatment.
(4) Same-endpoint blocking issue serialization.
For distinct blocking q_o and q_1 on the same scalar explicit abstract boundary and endpoint direction governed by the Core single-transfer-per-cycle endpoint constraint within one reset epoch, if $q_o <_{src} q_1$ and both issue, then Issue(q_o) < Issue(q_1). Equal issue cycles are not legal for two such scalar blocking instances.
(5) Sync-boundary discipline.
If q_o and q_1 are same-direction source-level message operation instances and $q_o \in$ pref_S(s) and $q_1 \in$ suff_S(s) for a synchronization call s, then Issue(q_1) occurs strictly after the commit of s. Any request assertion that persists while the process is still pending at s is attributed to the outstanding operation(s) in pref_S(s), not to issuance of any operation in suff_S(s). Synchronization-completion legality itself is governed by Core.SyncFence below.

Definition Core.SyncFence (synchronization-completion legality).

Let s be a dynamic synchronization-call instance of process P selected into S, and let BPref_S(s) be the blocking Push/Pop instances in pref_S(s). A boundary action that commits s is legal only if every $q \in$ BPref_S(s) has process-facing committed no later than that same cycle. Equivalently, after s commits and P advances into the succeeding source interval, no $q \in$ BPref_S(s) may remain issued and uncommitted. Same-cycle completion is permitted: q and s may both commit in cycle t provided q's process-facing commit is included in the selected L3 boundary-action set for t and is accounted for before the interval advance caused by s. This "accounted for before" convention is an ordering of the source operational update and of the proof/certificate attribution of the same-cycle boundary batch; on RTL_B it does not require an observable physical sub-cycle ordering between two events that commit on the same active clock edge. In Sys_B^elab, "process-facing commit" means the commit/accept/release event designated by the elaboration package as completing the owning source-level operation q; an internal staged, bundle, join, guard, or epoch-gate transfer that does not release q does not satisfy this fence. Appendix G rules R3 and E5 state the corresponding source-side and implementation-side trace constraints explicitly. For physical synchronization, Core.SyncProtocol enforces this fence before the sampled handshake is authorized. It supplies no veto after ready/valid overlap. Old-value pin users require the separate SP5 protection even where all process-facing predecessor commits have occurred.

Definition Core.SyncFenceOrder (E5 canonical predecessor and carrier conformance).

For every completed synchronization occurrence s and every blocking Push/Pop $q \in$ pref_S(s), E5 requires the strict normative predecessor $m(q) \prec_J s$ between their distinct complete units. Here m(q) is q's actual process-facing commit/release, including its E-declared realization in an elaborated model. This is an ordinary required-order generator, not an instance-selectable edge. It applies when the cycles are equal as well as when the message commits earlier; it does not require clk(m(q))<clk(s), a physical sub-cycle observation, or merger of the two groups. If a separately qualified compound atomic unit contains both effects, realize their order in its internal structure and do not introduce a strict self-edge after

contraction. Failed NB polls have no committed m(q) and acquire no edge through this blocking-only clause. The existing sync-to-suff_S strict-cycle constraints remain unchanged.
Carrier conformance. On the source, Core.SyncFence and the source synchronization primitive require every such process-facing commit before interval advance. On Batch RTL, the complete edge inventory includes each required predecessor with s or after an earlier completed edge. On a finer RTL carrier, every legal prefix containing completed s must already contain each required completed m(q), even when both belong to the current cycle. Atomic-unit integrity and the E5 clock inequality alone do not prove this last condition. A logger permutation cannot repair a carrier admitting a sync-only cut. Bind the actual unit/operation/interval/epoch attribution, and preserve incidence between already existing units under prefix extension. This condition is part of E implementation conformance for every route; it does not assert a physical ordering between same-edge handshakes. Bind Core.SyncProtocol's issue/held-offer lifecycle and sampled commit to these same units and epochs. A finite-prefix record cannot discard a physically completed sync merely to repair the predecessor order.
Projection and correspondence. Include the generator before outer projection. Retain its order through erased synchronization or elaboration support by normative-path contraction; erasure is not deletion of a required path. Translation and rank evidence cover the edge in both source/RTL directions where consumed, including compound-unit internal structure. For selected-source covering histories, prove matching fence incidence, induced order and predecessor closure as required by J.RefProjCover/J.RefProjCoverOn; cycle placement alone supplies none of those facts.
Status and conformance split. Core.IC and Core.SyncFence are constitutive legality/invariant clauses of the selected Sys_ref operational transition system: source executions outside these rules are not executions of the theorem's prescribed source model. They are not global proof axioms to be admitted independently of that semantics. On RTL_B, the corresponding request lifecycle, process-owned BoundaryReq identification, process-facing completion, and synchronization-fence behavior is not obtained by definition. The selected boundary mapping/B-faithfulness and request-lifecycle/attribution evidence establish the applicable Core.IC request facts; E3/E5 then check the resulting Issue/Commit ordering at their stated scope. The named Appendix M.7a Core.IC/Core.SyncFence audit entries record the required implementation-side conformance evidence.

Remark Core.10a (Non-blocking polls).

For non-blocking message passing, a failed poll produces no Σ event and is modeled as an ε-step, while a successful poll contributes the corresponding committed Push_c(v) or Pop_c(v) event. Each executed PushNB/PopNB call is nevertheless a distinct dynamic source-level instance $q \in Q_{exec,P}$ for its owning process P; thus repeated failed polls in later loop iterations are distinct q's even if the endpoint request remains continuously asserted. The ε-modeling concerns observability in Σ, not the dynamic-instance identity of the underlying source-level calls.

Channel enablement

Definition Core.11 (ChannelEnabled / ChannelDisabled).

Let x be a message-operation frontier item at c = BoundaryOf_P(x), with q = MsgOf_P(x). At an actual BoundaryEvalPoint σ, MirrorAvail_E(x,σ) is the complementary handshake value: ready for a Push, valid for a Pop. Define ChannelEnabled(x,σ) iff BoundaryReq(x,σ) and MirrorAvail_E(x,σ); ChannelDisabled uses the same request and the negated mirror. Ordinary primitive boundaries use Core.12/Core.13. Elaborated boundaries use the complete finite stateless network in E from the phase-appropriate fixed roots, including all guards, credits and other participating inputs. A coupler owns no source request and adds no undeclared storage. Determination of the complete L3 physical transfer set uses the L2 snapshot and beginning-of-cycle E4 state; a later changed signal does not authorize another same-cycle

action. At the eligible settled edge, ChannelEnabled commits the corresponding attributed transfer by Core.TransferLaw. All guards and arbitration outcomes are already represented in that snapshot; no subsequent selection may suppress it. The mirror is not a freely chosen availability summary. When the operation-owned pending condition differs from the physical own-side signal, Core.10/Core.TransferLaw require an explicit valid boundary/refinement mapping establishing their attributed transfer equivalence.

Definition Core.12 (Buffered channels).

If B(c)>0, E4 commit legality in cycle t uses beginning-of-cycle occupancy occ_c(t): a Push may commit only if occ_c(t)<B(c), and a Pop only if occ_c(t)>0. Same-cycle Push/Pop legality and non-fall-through remain exactly E4.
At an ordinary uncoupled Sys_B BoundaryEvalPoint σ, the current stored state determines the observational complementary signal:
• Push: ChannelEnabled(x,σ) ⇔ BoundaryReq(x,σ) ∧ occ_c(σ)<B(c);
• Pop: ChannelEnabled(x,σ) ⇔ BoundaryReq(x,σ) ∧ occ_c(σ)>0.
At an L2 SettlePoint these tests use the same beginning-of-cycle state as E4. A later state observation is not permission for an additional same-cycle commit. Disabled-Push⇒full and disabled-Pop⇒empty therefore refer to current occupancy at the supplied cutpoint.
At a coupling-sensitive Sys_B^elab boundary, the derived physical transfer set must satisfy E4 using beginning-of-cycle storage, while observational MirrorAvail_E is evaluated from the current phase-appropriate roots through the declared network. Nonfull/nonempty current state need not suffice for observational availability: another input, downstream space, epoch gate, or other declared condition may withhold the mirror signal. This coupling must be included in E's settling and dependency qualification. At an eligible settled edge at an ordinary uncoupled boundary, the displayed request/occupancy conjunction is sufficient as well as necessary for commitment by Core.TransferLaw. At a coupled boundary use its full mapped settled handshake. Legality is enforced before overlap, not by a late E4 veto.

Definition Core.13 (Rendezvous channels).

If B(c)=0, a committed transfer remains an atomic same-cycle rendezvous Push_c(v)/Pop_c(v) pair. At an ordinary uncoupled rendezvous boundary, MirrorAvail_E(x,σ) is exactly the complementary endpoint request, so ChannelEnabled is BoundaryReq for x conjoined with the matching complementary Push/Pop request. At a coupling-sensitive Sys_B^elab rendezvous boundary, both source-attributed endpoint requests may be present while the declared stateless coupler network still withholds the complementary mirror value because another explicit join/downstream condition is unsatisfied; the rendezvous is then ChannelDisabled at that evaluated boundary. Conversely, a coupler cannot synthesize a committed rendezvous without the required source-attributed requests and E4/rendezvous legality. Thus the ordinary missing-complementary-request characterization is used only for primitive uncoupled rendezvous boundaries; elaborated rendezvous disabledness is characterized by the settled network declared in E. Every eligible settled rendezvous handshake commits this matched pair mandatorily under Core.TransferLaw; a coupled group must prove its complete atomic leg equations and occurrence mapping.

Pending blocking frontier

Definition Core.14 (Pending blocking operations).

Fix a BoundaryEvalPoint_I(σ) for process P within the current interval I_P(σ) between the adjacent synchronization calls that bracket σ in P's current execution. Define
Pending_P(σ) := { x | x ∈ Ω_msg,P is a blocking source-level message-operation frontier item in that same interval I_P(σ), and for q = MsgOf_P(x), q is defined, Issue(q) has occurred, and Commit(q) has not yet occurred in σ }.
By Appendix C's Issue/Commit linkage together with the definition of BoundaryReq, each member x of Pending_P(σ) has its endpoint-owned request asserted at the cutpoint, with ownership read through its dynamic message-call instance q = MsgOf_P(x) under Core.10/Core.IC. By Core.SyncFence, an issued blocking operation from the preceding interval cannot survive the synchronization commit that advances P into I_P(σ); therefore the interval restriction cannot discard an outstanding blocker. Core.16 uses this definition at its L2 activation snapshot.

Definition Core.15 (Pending blocking frontier Front_P(σ)).

Define
Front_P(σ) := min_≤P(Pending_P(σ))
If several ≤_P-minimal pending blocking message-operation frontier items are simultaneously outstanding, they form a frontier. Lemma Core.OrdFrontSingleton proves that this set has at most one member for an ordinary non-elaborated source interval. This is the Appendix B / K notion of the pending blocking frontier (or minimal pending blocking frontier). It is distinct from NextFront_P(σ), which is the broader source-level frontier-item set over Ω_P.

Blocked frontier and drain-only discipline

Definition Core.16 (Drain-only blocked-frontier discipline).

A source interval satisfies the drain-only blocked-frontier discipline if, at the L2 SettlePoint_I(t,σ_t^set) of Definitions Core.Phase/Core.Settle, Front_P(σ_t^set) is non-empty and every frontier item x ∈ Front_P(σ_t^set) is channel-disabled. The discipline activates at that first settled snapshot. It does not retroactively invalidate source-later operations legally issued during the preceding L1 pre-settle window, but it forbids any new such issue after σ_t^set in that cycle and while the same fully disabled frontier persists. Set σ := σ_t^set at activation. This activation state is a BoundaryEvalPoint. Once activated, the following all hold:
(1) Block point. P is blocked on message passing at Front_P(σ).
(2) No new later work. While Front_P(σ) remains non-empty and fully disabled, no new later blocking message-operation frontier item y ∈ Ω_msg,P is issued, where "later" means that, for some x ∈ Front_P(σ), x ≤_P y and x ≠ y, with issue understood through q_y = MsgOf_P(y). More generally, no new source-later dynamic operation is launched past the blocked point. "Launched" is intentionally broader than Issue(q): it includes a source-evaluation or message-issue transition that advances new source-later dynamic work past the blocked point. The no-new-Issue and broader no-new-launch clauses remain distinct. These restrictions govern abstract source progress, not physical datapath activity. They do not prohibit physical computation that can remain abstractly invisible: combinational evaluation or updates of private implementation state need not constitute source evaluation, ReachPrep, or Issue. Such activity is permitted only under the effect and later-use conditions in the clarification below; a physical register write is not by itself a source A1 occurrence.
(3) No withdrawal. Already-asserted blocking requests are not withdrawn before the owning message instances commit.
(4) Frontier-minimality. An already-asserted blocking request whose frontier item is not in Front_P(σ) is

not a minimal pending blocker unless and until that frontier item later becomes frontier-minimal.
(5) Drain-only already-admitted progress. Work admitted before Core.16 activation may continue to advance through complete proof-model action units, including E-declared proof-internal staged/bundle/join transfers and ordinary already-issued boundary commits, provided those actions do not launch or issue new source-later work past the blocked frontier. Under a declared synchronization-qualified profile, an already-reached/pending Core.SyncCarrier whose transaction is currently SyncReady may also complete through Core.7d SyncFire, and a currently represented explicit epoch-gate token/request/credit may advance through Core.7e EpochGateAdvance. SyncFire advances only across the synchronization boundary to the immediate post-sync control/interval entry and does not include post-sync ReachPrep/Issue. “Already admitted” is a current-state property: the work is represented by a current attributed outstanding request, explicit channel/token/storage/gate state, or a current typed synchronization carrier; no historical Issue timestamp is required. A peer that still must execute source control to reach a synchronization is not already-admitted synchronization work. Actions and their legal phase opportunities are taken from the actual source relation.
(6) Finite non-replenishing admitted domain. Every selected explicit abstract boundary that can hold or credit already-admitted work has finite B-faithful storage/credit, directly or through a finite Sys_B^elab chain, and the selected elaboration declares a finite action-unit inventory over that state. Under a declared synchronization profile, the admitted population additionally contains only the finite current set of reached/pending SyncTxn keys, counted once per atomic transaction rather than once per endpoint, plus the finite explicit explicit gate state bound to those keys. Together with clause (2), the currently admitted population is finite and receives no new source work. The applicable drain qualification additionally supplies a well-founded stage/rank or equivalent finite multiset measure strictly decreased by every selected in-flight progress action, including SyncFire/EpochGateAdvance when present. Finite capacity alone is not a termination proof, and the measure is activation-local: no old activation certificate or rank is reused after source progress or reset creates a fresh activation.
Core.16(5) clarification — drainage and pre-issued completion. Already-admitted boundary progress includes both lawful downstream drainage and completion of a source-later request legally issued before activation; these are distinct cases, and source order is not inferred from physical stage location. An L1 issue preceding the activating L2 snapshot is not invalidated retroactively. Neither case licenses a new source-later evaluation, ReachPrep or Issue past the active frontier. Preservation of stalled work. Any completion of pre-issued younger work during a blocked frontier preserves the semantic values and ownership of all live operands, pending request descriptors, attributed storage and pending effects still needed by stalled older work, except for independently legal actions of that older work itself. Prove this frame property for the actual joint transition and subsequent legal use. Physically disjoint attributed stage registers are sufficient, but banked, tagged, versioned or other shared storage is permitted with a proved ownership/noninterference invariant. Permission to commit is not permission to overwrite live state; physical stage position alone establishes neither source order nor separation.
(7) Sync-boundary capacity withdrawal — elaborated-boundary interpretation.
While a process is pending at an explicit synchronization call s, producer-facing availability for any back-annotated capacity belonging only to suff_S(s) is withheld until s commits; work already accepted before s may still drain. For the Sys_ref proof model, this withdrawal is not an additional hidden enablement predicate on an otherwise ordinary E4 channel. It is represented only by choosing Sys_ref = Sys_B^elab with explicit sync-boundary / epoch-gate abstract channels, or equivalent explicit staged realization structure, so that the refusal appears as ordinary ChannelDisabled behavior at an explicit abstract boundary.
Abstractly invisible physical computation. This is a tool-independent permission, independent of pipeline organization or register allocation. Under the selected source-state interpretation and abstract-boundary mapping, the computation component leaves the represented source control,

preparation/reach position and operation lifecycle unchanged. While its result remains unattributed, it does not change selected observations, attributed requests or active payloads, explicit abstract channel/storage/credit state, blocked-frontier enablement or declared resource/readiness dependencies, or the represented already-admitted work. The absence of a visible transfer alone is insufficient. Concurrent legal drain, clock advancement, environment activity and other-process transitions retain their own ordinary semantics and are not erased by this permission.
Legal later use and persisting activations. A privately computed result may be used only when a legal corresponding source computation accounts for that use, with values matching those of the declared operand locations at the corresponding legal source evaluation, preserving the applicable writer/occurrence attribution and field-sensitive ReadSet/WriteSet constraints, and with source order, Core.RegSeq and all applicable dependency, footprint and observation-anchor constraints satisfied. The accounting source computation is itself subject to Core.16(2): precomputation does not authorize source-later evaluation, preparation or Issue while a persisting activation prohibits that action. This restriction does not prohibit further abstractly invisible physical computation or otherwise legal progress of already-admitted work. The field-sensitive location vocabulary of J.ConstructionAction provides the footprint interpretation. An intervening write to an operand location does not alone forbid reuse, but reuse requires proof of the required value equality at the legal source evaluation and all applicable occurrence/provenance bindings; unjustified stale reuse is not permitted. It need not be attributed at the first legal opportunity, and no already-bound source occurrence, outcome or Horizon may be changed retroactively. Computation that does advance abstract preparation or alters a protected boundary effect remains subject to clause (2), even if implemented without a new request. The classification must be justified from the computation and its uses, not merely from a hidden-register label. Such invisible activity neither admits new drain work nor counts as a decreasing Core.InFlightDrainWF action; it does not discharge or weaken any existing progress, fairness, continuation-existence obligation.
Example of channel-derived private computation. An implementation may privately compute from a FIFO head before the corresponding Pop commits, provided that computation remains abstractly invisible. Reuse requires proof that the sampled value is the value returned by that same Pop occurrence; any attributed computation or Issue depending on the returned value must have a Horizon strictly later than the Pop's ReturnedAtCycle, as required by Core.RegSeq, and must satisfy Core.16. If this binding is not established, the private result cannot justify the later use. For a bus value, the applicable sampling, stability or observation-anchor contract must similarly establish the binding to the corresponding legal source observation; physical presence on a wire alone is insufficient.
For a pipelined or otherwise overlapped implementation, Appendix B's automatic-flush rules are the implementation-side construction that supplies Definition Core.16. For ordinary non-overlapped control, the same definition may be discharged by a structural proof that no new source-later dynamic operation can be launched after the blocked point is reached. The discipline applies to every admitted interval that can contain eager issue, not only loop pipelines. In particular, eager issue permitted by Policy L must occur in Core.Phase's L1 pre-settle window; the no-new-launch obligation begins at the L2 settled snapshot, not at the earlier issue of the frontier request. In either case, implementation-side no-new-launch evidence concerns the abstract source effects defined in clause (2); it is not a requirement to suppress physical computation that satisfies the abstract-invisibility clarification.

Definition Core.Drain (activated drain-discipline qualification).

Core.Drain_I(ρ) records that the actual path ρ obeys all seven Core.16 clauses at every activation and throughout each persisting fully disabled frontier. Bind the actual instance, environment, reset, elaboration, attribution, admitted action domain and the finite-work measure required by clause (6).

This packages operational evidence only. It does not imply a future opportunity, fair service, absence of a wait cycle or completion; a safety construction proves its source extensions legal directly, and K.CT separately proves exposure and service.
Core.Drain and completion. B.OF/B.CT supply the positive movement and request laws used by K.CT. Permission for existing work to advance and a decreasing measure do not alone force an action to occur. The proof must establish legal opportunities and the applicable service assumptions on the actual execution.

Definition Core.InFlightStep (already-admitted proof-model progress action).

Core.InFlightStep_{M,E}(s,u,s') is an actual phase-legal source transition completing one declared atomic action on already-admitted work: an issued message commit, explicit staging/join transfer, pending synchronization via SyncFire, or explicit EpochGateAdvance. It has the exact declared state and history effect, respects E4, Core.SyncFence, Core.16 and all bound choices, and launches/issues no new source-later work. Synchronization reaches only the immediate post-sync point. This relation contains operational transitions, not proposed edges of a reconstructed state graph. Any clock advancement needed before selection is governed by the actual cycle semantics.
State-based admission rule. An action is eligible for the in-flight universe only because its required work is currently represented in the declared operational state: an attributed outstanding request, explicit abstract channel/token/storage state, or—under declared synchronization semantics satisfying Core.SyncCarrierWF—a reached/pending Core.SyncCarrier whose transaction is already ready without additional source reach/launch. Historical Issue timestamps are neither required nor reconstructed. A peer that must still execute source control to reach a synchronization has not yet supplied a pending synchronization request and is not converted into an in-flight action. Likewise, an epoch-gate action is admitted only from currently represented E-declared operational gate state. A template that needs hidden state to decide action identity, enablement, or effect shall expose that state in the qualified proof model or prove it irrelevant.

Definition Core.InFlightDrainWF (finite admitted population and well-founded progress).

For a Core.16 activation and selected in-flight action schema, Core.InFlightDrainWF holds when the currently admitted action-bearing population is finite and a declared well-founded rank/multiset measure μ is defined on its represented state such that every Core.InFlightStep selected as drain progress strictly decreases μ. Clause Core.16(2) supplies non-replenishment by forbidding new source launch/Issue into the activation-local domain. Capacity bounds contribute finiteness of the current population but are not by themselves the decrease argument. Under a declared synchronization profile the admitted population and measure also account for current synchronization transaction keys and explicit gate state without double-counting one multi-endpoint SyncTxn per participant. A sufficient simple local ranking for an explicit epoch realization is pending-ready synchronization > residual epoch-gate realization > retired, provided the qualified action/effect theorem proves that SyncFire and EpochGateAdvance strictly move the same admitted obligation downward and never create a fresh source operation. The measure and admission certificate are activation-local and end when the applicable fully-disabled frontier/activation ends, an affecting reset occurs, or later source progress creates a fresh activation.

Definition Core.17 (System-level observable compatibility).

For a witness prefix τ_ref of Sys_ref and a post-HLS prefix τ_post of RTL_B, define ΣMatch(τ_ref,τ_post) to mean that the selected Σ-visible events/observable units match under the canonical occurrence convention of this theorem stack. For each message-passing channel c, the projected sequences of committed Push_c events and of committed Pop_c events have the same length; the k-th Push_c on one

side matches the k-th Push_c on the other, and likewise for Pop_c, with the same endpoint and payload/value label. For each observable signal sig of a sequential process P, the canonicalized sequences of Read(sig,·) and Write(sig,·) occurrences have the same length and the k-th occurrences match with the same process/signal/direction and value label; within each source anchor interval, canonicalization records at most one Σ-visible Read and one Σ-visible Write for each (P,sig) at the E2 anchor; if several source Writes share it, the last executed Write supplies its value and earlier writes are ε-internal. For each R2C combinational process or explicitly modeled combinational network component C_comb, the emitted fixed-point observation-unit sequences have the same length and the k-th units match by component identity, included process/signal/direction labels, and fixed-point values, including any cycle-locked participation requirement. For proof-internal RESET(S_p,S_c) units included in the selected theorem comparison, project each trace to the RESET occurrences of each reset scope (S_p,S_c); the k-th RESET occurrence for a scope on one side matches the k-th RESET occurrence for the same scope on the other, so matched RESET units have identical S_p and S_c. Independent disjoint RESET units may commute except where the reset well-formedness rules or another selected observable-order constraint imposes an order. Synchronization events match by their per-process occurrence index in source order. Independent matched Σ events may commute or occupy different global cycle buckets except where E1–E5 or the selected observable-unit semantics impose an ordering or same-cycle constraint. An unmatched selected Σ event/unit on either side makes ΣMatch false. Define τ_ref ≈ τ_post :⇔ ΣMatch(τ_ref,τ_post) ∧ E1 ∧ E2 ∧ E3 ∧ E4 ∧ E5
This is an ordered compatibility predicate, not a mathematical equivalence relation. The left argument supplies the Sys_ref witness prefix, and the right argument supplies the RTL_B post-HLS prefix. ΣMatch and E1–E5 are normative conjuncts of the Core definition. Appendix G expands the canonical Σ-event matching convention and the E1–E5 rules but does not redefine this relation or add a separate matching conjunct. In particular, the R2C matching statement in E2 below is the Appendix-G expansion of the R2C clause already contained in ΣMatch, not a second R2C matching requirement. For retained sequential Read observations, RULE 1 retained-Read qualification fixes zero-or-one membership and value when the anchor completes; the at-most-one convention is not permission to add a member to an already completed group. Last-write selection is resolved by the actual source-order write record at its succeeding anchor.

Shared-memory and reset side conditions

Array/synchronization commit-order conformance (Core.MemSyncOrder). Core.MemSyncOrder holds when the selected source/HLS/mapping-layer schedules satisfy the commit-time fence stated in “Scheduling of Array Accesses”: for every dynamic array or memory access a and every synchronization call s selected into S that is ordered with a in the process’s dynamic source execution, every mapped storage commit representing a occurs no later than the corresponding commit of s when a is source-before s, and occurs strictly after s when a is source-after s. Any allowed cache/merge/split/reorder transformation shall carry mapping-layer evidence that preserves those synchronization fences. In particular, no one transformed storage commit may jointly represent source accesses lying on opposite sides of the same synchronization boundary; such a merge cannot satisfy both fence inequalities and is forbidden. This is a local scheduling-conformance obligation whether the storage is process-internal or externally mapped. It does not add memory read/write events to Σ and does not extend the selected constructive relation. Accordingly, Core.MemSyncOrder is a separate scheduling-contract/tool-conformance obligation recorded in Appendix M.7a; it is not a premise consumed by a constructive safety theorem. For a constructive application with cross-process shared-memory value flow, J.Mem is separately required; when transaction-changing transformations are hidden below a selected memory boundary, Core.MemFaithful is additionally required.

Shared-memory lowering condition (J.Mem). The channel/signal proof alphabet contains no raw mapped-memory operations. Every cross-process memory dependency that can affect control, payloads or operation identity/order must therefore be represented by value-labeled operations at an explicit logical channel or observable anchored-signal boundary, or by a separately proved memory-state extension of the selected correspondence. The lowering proof covers every such dependency and shows that none remains hidden. Synchronization alone fixes order but does not establish equality of memory values. Transaction-changing transformations additionally require a transformation-invariant Core.MemProfile boundary and Core.MemFaithful. Conditional fact agreement for completion does not substitute for this correspondence obligation.
Qualified memory mapping profile (Core.MemProfile). A reusable qualified memory mapping profile is the standard route for a memory whose physical realization uses transaction-changing transformations while the logical memory interface remains in the J/K proof scope. A profile shall declare: (MP1) the untransformed logical mapping-layer reference semantics, selected logical boundary, boundary alphabet, and permitted transformation set; (MP2) a once-per-profile Core.MemFaithful proof for that boundary and transformation set; (MP3) a J.Mem lowering at that logical boundary, including the completeness argument that no qualifying cross-process memory dependency remains unrepresented; (MP4) Core.MemSyncOrder preservation for all permitted transformations; (MP5) a conclusion-specific hidden-realization disposition. For K.CT, every hidden state, resource or service condition that can delay a required logical obligation is included in the bound exposure/resource/service proof, using B.CT/Q.CT where applicable or an explicit extension; output-only accounting or history equality cannot hide persistent memory backpressure.; (MP6) explicit ownership for shared/multi-client resources, using explicit arbitration and point-to-point process-facing channels rather than treating a raw multi-client port as an ordinary point-to-point channel; when that exposed logical boundary is staged, coupled, joined, or otherwise elaborated as Sys_B^elab, the applicable Appendix Q.1 qualified-template obligations also apply; and (MP7) RW1–RW5/reset behavior for every exposed logical channel, including an explicit flush/abort or other cross-epoch disposition when reset scope would otherwise split a live channel. The profile identity/version and its qualification evidence are bound to the concrete theorem instance through Core.EvidenceHdr/Core.EvidencePackage . If the selected MP5 disposition cannot be satisfied for a persistently backpressuring memory, that conclusion is unavailable for the transformed instance; disable the incompatible transformation or supply another qualified proof. Success of one MP5 case does not discharge the other. Concrete reusable library instances, such as the ping_pong_mem/native_ram_fifo patterns discussed in Appendix H, item 1, remain qualification work until their profile obligations are discharged; the Core.MemProfile interface itself is normative.
Informative memory-state extension. A proof that keeps shared application memory rather than lowering it must extend the constructive local relation with the complete memory state, ownership, read-value and update/frame laws and cover every actual successor. It must separately establish completion exposure and service for any required memory response. No such general extension is supplied by the channel/signal theorems.
Reset-boundary observable unit (RESET) and reset well-formedness (RW1–RW5). When dynamic reset is used, each dynamic reset occurrence is represented in the proof-internal event model as an explicitly atomic observable unit RESET(S_p, S_c): a proof-internal boundary marker whose scope names the set S_p of processes and the set S_c of explicit abstract channels affected by that reset occurrence. RESET units may be projected away from the outer Σ_DUT alphabet, like other proof-internal units. The unit is ordered strictly after every observable unit of the ending epoch owned by the affected scope and strictly before every observable unit of the new epoch of that scope, and the first observable action of each $P \in S_p$ in the new epoch is its fresh dynamic START_P. The state-changing reset transition itself — which occurs before the new epoch's START_P — is exactly what the RESET unit represents. In Core.CycleResult that dynamic occurrence is carried in resetUnits of the enclosing real cycle rather than

replacing the cycle with a reset outcome; it therefore retains its own correspondence obligation (the selected constructive reset extension in Appendix J) rather than being folded into START_P matching. The following reset well-formedness conditions are implementation-side obligations for designs using dynamic reset:

RW1 (channel-consistent scope). For every explicit abstract channel c, interpret its endpoint-process set through the selected source/elaboration mapping: it consists of the modeled source-level process owner(s) associated with c by E's boundary attribution/projection; elaboration-local stateless couplers are not endpoint processes for reset-scope purposes. Either all such modeled endpoint processes are in S_p and c ∈ S_c — in which case c re-initializes to reset-empty occupancy under the Core reset-empty FIFO convention at the boundary and any in-flight (accepted but not yet delivered) contents are discarded consistently on both sides in both models — or none of those modeled endpoint processes are in S_p and c ∉ S_c, in which case c's state and history are unaffected. A reset whose scope intersects but does not contain all modeled endpoint processes of a live channel is permitted only if the design models the cross-epoch disposition explicitly (for example, an explicit flush/abort protocol on that channel); that protocol's events are ordinary Σ events, and the channel is treated per this clause after the protocol completes.

RW2 (request disposition). Every outstanding endpoint-owned boundary request of a process in S_p is cleared at the RESET unit; its owning dynamic instance does not commit, is not re-attributed across the boundary, and any post-reset re-execution is a fresh dynamic instance in the new epoch's universe. The Appendix C non-withdrawal discipline is scoped per reset epoch.

RW3 (corresponding reset transitions). At the RESET unit, RTL_B's post-reset state at the chosen abstract boundary corresponds to Sys_ref's prescribed epoch-initial state: control at epoch-initial locations for every P ∈ S_p, source-visible values at their reset values, reset-empty occupancy on every c ∈ S_c, no stale synchronization state, no unaccounted pending endpoint request, no hidden accepted message absent from the source reference model, and direct-input signals obeying their per-epoch stability or sync-controlled contracts (main text, Direct Inputs).

RW4 (atomic boundary). The reset state transition is not interleaved with same-cycle observable actions of the affected scope. If the implementation asserts reset in a cycle in which affected-scope observable actions also commit, those actions belong to the ending epoch and precede the matching RESET occurrence in Core.CycleResult.resetUnits in the required order; the two reset epochs of the affected scope share no observable cycle bucket ambiguity at the boundary. The enclosing CycleResult may still carry unaffected-scope progress because RW5 frames state outside the reset scope.

RW5 (locality). The RESET unit changes no state of processes outside S_p and no history or occupancy of channels outside S_c. Cross-scope effects, if any, must be expressed by ordinary modeled events.

Additional Semantic Assumptions (B-rules)

The following B rules are assumption interfaces used only where the selected theorem or discharge explicitly consumes them; they are not a blanket premise package for every J or K result. R-rule source legality and the selected operational contracts remain separate. B1 is a separately claimable determinism property. B2 concerns declared pre-grant arbitration. B3 independently states conditional channel progress. B4 concerns finite internal progress where invoked.

ID	Basic-process property (informal statement)	Why it is needed / how it is used
B1 Deterministic Trace Property	For any fixed test stimulus and initial state, the sequence of observable actions emitted by the post-HLS design is unique, including the channel identity and payload/value of each committed Push_c(v)/Pop_c(v) and the value of each Write(sig,val)/Read(sig,val). Internal ε-steps may differ between runs, but the selected Σ-trace (with labels) cannot. Here "Σ-trace" has the Definition Core.3 meaning: the observable projection carries the inherited real-cycle partition / per-cycle observable buckets, while independent same-cycle events are not given an incidental serialization. Finite-prefix clarification. For the observable alphabet Σ selected by the theorem instance, B1 finite-prefix reasoning uses the Definition Core.3 cycle-partitioned observable projection $\tau\|\Sigma$, not a flattened label/value list. ε-steps are absent from $\tau\|\Sigma$, but the inherited real-cycle partition remains. Any two finite reachable RTL_B prefixes under the same fixed initial state and stimulus must agree on every observable cycle bucket that is complete in both, and a canonical observable occurrence may not be assigned to different real cycles in the two prefixes. If a prefix ends inside its current real cycle, its already-complete observable units may form only a legal unsplit partial realization of that same B1 cycle bucket; two such intra-cycle prefixes need not be ordinary list-prefix comparable merely because independent same-cycle units can complete in different micro-orders. Explicitly atomic units remain indivisible. This finite-prefix clause does not assert a fair/maximal	Defines a separately claimable fixed-cycle observable determinism property. Constructive safety and K.CT agreement/progress use their own independent premises; B1 is not a substitute for coverage, correspondence or source-property validity.

ID	Basic-process property (informal statement)	Why it is needed / how it is used
	extension, common internal state or carrier provenance, uniqueness of witness/annotation data, or next-cycle totality. Because ε-steps are absent from τ|Σ, this B1 finite-prefix clause supplies neither finite preparation nor operational settling; those obligations are discharged separately when consumed. Clarification: B1 is assumed of RTL_B (post-HLS) only. The source-level models Sys/Sys_B may admit multiple ε-/latency-different executions with the same Σ-projection. When a single concrete witness execution of Sys_B is required for simulation/debug, Appendix L's snooping wrapper may be used to select one admissible witness whose latency-sensitive events align with the unique RTL Σ-trace. Derived per-process consequence. For every process P, the projection of the unique RTL_B Σ-trace onto the observable units owned by or attributed to P is also unique. This follows from the system-level B1 property; it does not assert that P is deterministic when considered in isolation under arbitrary peer, channel, arbitration, or environment behavior.	
B2 Weak Fairness of Pre-grant Arbitration (WF)	B2 is weak fairness of a declared pre-grant arbitration opportunity, not a license to delay a sampled transfer. For each selected arbiter action a, define ArbEligible_I(a,t) prospectively from the actual pending request, legal owner/phase, peer/resource support and declared guards, excluding that arbiter's own grant or denial. Bind the action and full atomic unit, eligibility equations, grant effect and sampling opportunities independently of the	Consumed where progress depends on a declared pre-grant arbitration opportunity satisfying the B2 antecedent and grant-effect binding. It is not required for every progress argument: Core.TransferLaw/Core.SyncProtocol commit eligible physical handshakes without further selection; other internal and environment service uses its separately bound contract.

ID	Basic-process property (informal statement)	Why it is needed / how it is used
	desired progress conclusion. Grant-dependent or mutually recursive support must be justified by a separate joint contract; it cannot make eligibility false merely because service is withheld. If ArbEligible remains true at every applicable opportunity from some point in an admitted continuing execution until service, B2 requires service/grant after finitely many such opportunities. Intermittent eligibility needs an explicitly qualified recurrent-opportunity or direct finite-service contract if progress is claimed; strong fairness is not imposed globally. The implementation must prove that the granted opportunity yields the required support or complete transfer under the bound equations. Once actual settled ready/valid overlaps, Core.TransferLaw commits on that edge without B2. Synchronization similarly uses Core.SyncProtocol. Request issue, peer arrival, future clocks and internal movement retain their separate obligations.	
B3 Channel Progress Invariants	B3 records the conditional primitive P_push(c)/P_pop(c) discharge properties. At an ordinary uncoupled positive-capacity FIFO, a full channel with a persistent complementary Pop commits that Pop at its eligible settled edge; an empty channel with a persistent complementary Push commits that Push. This follows from old-state E4 and Core.TransferLaw, assuming the edge occurs without an affecting reset. The original still-pending request then commits at the next eligible settled edge, provided that edge occurs without an affecting reset; old-state tests prevent same-edge fall-through or full replacement. An ordinary uncoupled B(c)=0 pair	Required by complete/fair progress arguments, principally Appendix K, to rule out permanent back-pressure loops in which both endpoints continuously request a transfer until the matching discharge event but the matching discharge never occurs; such progress is logically independent of the cycle-by-cycle R-rules. It is not a premise of finite-prefix constructive correspondence.

ID	Basic-process property (informal statement)	Why it is needed / how it is used
	commits at the eligible settled edge with both matching requests. No B2 post-handshake fairness is needed. In Sys_B^elab, nonfull/nonempty state need not enable a coupled boundary. Use Q.1's complete declared request/resource/guard network and its independently proved progress facts. Missing peer support, coupled constraints and pre-grant arbitration remain real obligations; B2 applies only to its separately bound ArbEligible antecedent. A stateless coupler supplies no autonomous service and no guarantee that a closed wait cycle clears. These implications do not guarantee request arrival, obstruction discharge without its stated complementary request, a future edge, or preservation of a required obligation across reset. Complete/fair consumers bind those premises; finite-prefix correspondence does not import them. Core.IC persistence and stable payload remain in force until the appropriate commit or affecting reset.	
B4 Finite Stutter Bound (FPD)	Every process P has a finite constant depth D_P such that whenever P is not externally stalled (i.e., P is advancing internal micro-state toward its next observable Σ-action, or its next Σ-action is locally enabled), P can execute at most D_P consecutive ε-steps before either (i) executing a Σ-action, or (ii) reaching a stable waiting state in which P has no further ε-step available until some external/peer/environment condition changes the enabling predicates.	This guarantees the ε-stutter closure consumed by local finite-preparation arguments when their selected proof invokes B4 and to bound internal micro-latency (pipeline/FSM bookkeeping) by ≤ D_P. Waiting for back-pressure relief, peer stimulus or an arbitration grant uses the separately qualified B3, environment and pre-grant B2 obligations, not B4. An actual eligible settled handshake commits immediately under the applicable transfer law.

Appendix G – Trace Compatibility Rules

This document informally states that a primary goal is to present “no surprises” to a DV engineer using the same testbench to verify the pre-HLS and post-HLS models.

Somewhat more formally, we can show how the scheduling rules within this document establish a set of trace-compatibility rules between the pre-HLS and post-HLS models that provide strong guarantees about their behaviors.

The *basic conceptual model* establishes the base behavior for a single process:

1) Synchronization interface calls always remain in source code order.
2) Signal reads occur at closest preceding synchronization interface call.
3) Signal writes occur at closest succeeding synchronization interface call.
4) All message-passing operations before a call to a synchronization interface shall be issued before or in the same cycle in which the synchronization interface commits. (For blocking message-passing operations (Push/Pop), the separate Core.SyncFence/E5 commit-fence requirement also requires the corresponding commits to occur no later than the synchronization call’s commit cycle.)
5) All message-passing operations after a call to a synchronization interface shall be issued after the cycle in which the synchronization interface call commits.
6) Two message-passing operations on separate interfaces which appear in sequence in the model are source-ordered. They may issue with timing overlap or in the same clock cycle where the selected issue policy allows, but they shall not be issued in the reverse sequence. Operations on the same message-passing interface are issued in source order (they are never reversed).
7) Synchronization calls determine the signal-IO anchors and message-call fences described by R2/R3 and E2/E5. These anchoring rules do not remove real data or control dependencies: a message result may affect a later signal value or control decision, and a signal observation may affect a later message operation. Every transformation must preserve those dependencies and the applicable source, observation, environment and component contracts.
8) The STRICT_IO_SCHEDULING=relaxed option is not used.
9) We distinguish the rendezvous source model Sys (B(c)=0 for all channels) from the buffered source interpretation Sys_B, which uses bounded-FIFO channel capacities B(c) matching the post-HLS RTL (Appendices J and K).

Conceptually, systems are composed of many such processes which follow the basic conceptual model, and which communicate using only synchronization interface calls, latency-insensitive signal-level protocols, and committed message-passing transfers. If a design uses non-blocking message-passing calls (PushNB/PopNB), then unsuccessful polls are treated as internal ε-steps (no Σ-event), and each successful completion is represented in Σ as the corresponding committed Push_c/Pop_c transfer (Σ records transfers, not failed polls). If the resulting latency-sensitive behavior is externally visible at the DUT boundary, the snooping wrapper of Appendix L may be used to align the selected admissible pre-HLS execution with the RTL. Here we call this system of processes the *conceptual system*.

Practically, the modeling approach described above is too restrictive for real-world systems with optimized HW, so the scheduling rules in this document allow for key HW optimizations. But this is done in a carefully controlled manner, such that the covered implementation behavior remains compatible under ≈ with a legal behavior of the implementation-bound source reference Sys_ref, subject to the selected correspondence and source-property premises.

Here is a summary of the HW optimizations that the scheduling rules allow, and a brief description of the conditions for maintaining ≈-compatibility with the implementation-bound Sys_ref:

1. Pipelined loops: In the post-HLS RTL, the pipeline may overlap iterations to improve throughput. See Appendix B for a formal presentation of pipelined loops.

2. Pipelined loops: HLS by default will not break any latency-insensitive signal IO protocols when pipelining loops.

3. Direct inputs: Signals that do not change after a process comes out of reset can be read as late as possible using *hls_direct_input*, saving register area.

4. Direct inputs: *hls_direct_input_sync* permits controlled updates only under the designated synchronization protocol. Before offering ready, the DUT must finish old-value uses or retain and isolate their operands, and maintain that protection through the pending interval and any remaining uses. Closing next-interval capacity alone does not prove isolation. Qualified late sampling can then avoid unnecessary pipeline registers; Core.SyncProtocol and J.AnchorDirectInputLift (or the selected configuration-observation contract) supply the precise conditions.

5. Memory accesses: reordering requires preservation of the selected logical memory behavior and applicable dependencies and synchronization fences. A conflict-free reordering is one case; authorized cache, merge, split and reorder transformations require the corresponding Core.MemFaithful/Core.MemProfile evidence. Selected-boundary events retain their Core.17/E1–E5 obligations.

6. Shared memories: Memories shared between processes are modeled as simple C arrays but always include explicit synchronization between processes in both the pre-HLS and post-HLS models. For the formal results of Appendices J and K, the value-carrying accesses must additionally satisfy the J.Mem lowering condition (Normative Formal Core).

7. Non-blocking message-passing interfaces: successful calls contribute committed transfers; failed polls contribute no committed message event but may still supply required call and control evidence. When latency-sensitive behavior must be aligned in a selected comparison, Appendix L may select a legal source witness under L.WC, W1–W5 and L.Dir. Delaying source arrival alone does not establish matching, source legality or complete choice coverage.

8. Latency-sensitive global signal IO: Appendix L may align selected observations through a legal source witness under its signal, choice-coverage, L.WC and L.Dir contracts. Delaying a signal does not by itself preserve its anchored value, event membership or the full ≈ relation.

9. Latency-sensitive local signal IO: Isolate local latency-sensitive protocols to small, dedicated transactors, and use latency-insensitive signal IO or message-passing to communicate with the rest of the system.

10. One-way signal handshake protocols: use only under an established no-backpressure property over the qualified operating, stall and reset conditions (Appendix E). Runtime assertions monitor exercised execution prefixes; a clean finite run alone does not prove that domain-wide property.

11. Combinational processes: If the pre-HLS model contains combinational processes, their observable signal Read/Write behavior is interpreted at each cycle's ε-quiescent combinational fixed point. Under the well-formedness requirement that the combinational network is deterministic and reaches a unique fixed point, the RTL/tool-flow conformance obligation is to preserve the fixed-point signal relation, so combinational signal IO can be included in Σ without using synchronization anchors.

The methodology verifies source properties on the implementation-bound reference model and proves the tool/controller correspondence and progress conditions separately. Constructive Appendix J provides a legal source prefix matching each implementation prefix in its qualified domain. J.SafeTransfer-HistCov transfers extensional source safety under its universal-source or explicitly qualified covering-run alternative. K.CT separately proves acceptance of required outputs from a qualified finite workload witness, agreement, closure, exposure and service. No unrestricted arbitrary-source-to-RTL trace realization or strict deadlock-preservation theorem is claimed.

Formalization of the Trace Compatibility Rules in Appendix G

The following notation is used in this section.

Symbol	Meaning
P	A process (thread or method) in the design
Σ	The alphabet of observable IO actions
τ_P	**A (finite or infinite) per-cycle trace of observable actions executed by process P:** for each global clock cycle t, τ_P[t] is the (possibly empty) set of Σ-actions committed by P at t, with the inherited cycle partition and complete atomic-unit structure of Core.3. Independent same-cycle actions have no incidental serialization. This representation retains cycle placement; comparison under Core.17/≈ preserves the selected required ordering, grouping, values and matching, not every incidental cross-cycle order between independent actions. Explicit timing requirements, including E1 and E5, remain binding.
S	The subset of Σ that are **synchronization calls** (explicit wait, SyncChannel, START_P/FINISH_P indicating process start/finish)
R	The subset of Σ that are **signal reads**
W	The subset of Σ that are **signal writes**
M	The subset of Σ that are **committed message-passing transfers** (i.e., committed Push_c / Pop_c events on message-passing channels, including successful non-blocking transfers). Unsuccessful non-blocking polls contribute no Σ-event and are modeled as ε-steps, and therefore are not members of M.
Q_exec	The execution-level set of dynamic source-level message-call instances executed by a process at the Σ-visible endpoints (blocking Push/Pop, and non-blocking PushNB/PopNB calls). Elements of Q_exec may or may not commit. In particular, failed non-blocking polls are elements of Q_exec, have no committed Σ-event, and

Symbol	Meaning
	are modeled as ε-steps. The frontier-item universe Ω_P, defined below, is separate from Q_exec.
Ω_P	The local dynamic frontier-item universe for process P in the selected execution / witness. It contains synchronization-call items, signal-action items, and message-operation frontier items; it is distinct from Q_exec,P, the universe of dynamic source-level message-call instances.
Ω_msg,P	The message-operation slice of Ω_P: frontier items x for which MsgOf_P(x) is defined.
MsgOf_P(x)	Partial projection from a message-operation frontier item x ∈ Ω_msg,P to its owning dynamic source-level message-call instance q ∈ Q_exec,P. In Sys_B^elab, MsgOf_P need not be injective: one source-level q may own several elaborated frontier items.
BoundaryOf_P(x)	The explicit abstract channel boundary at which frontier item x is evaluated for BoundaryReq, ChannelEnabled, ChannelDisabled, Front_P. In ordinary unelaborated cases this boundary is unique for the owning source-level operation; in Sys_B^elab it is declared by the elaboration package.

A *system trace* is the tuple
τ = (τ_P)_P, one component per process.

For any action a ∈ Σ, let clk_pre(a) / clk_post(a) be the clock cycle at which a is committed.
For any issued message-call instance q ∈ Q_exec, issue_pre(q) / issue_post(q) denotes the auxiliary Issue timestamp for q in the pre-HLS / post-HLS trace, respectively, as defined in Appendix C and required to satisfy the associated Issue/Commit linkage (well-formedness) conditions with respect to the boundary request signal used as auxiliary Issue evidence on q's endpoint direction.
If q commits, it contributes a unique committed transfer m(q) ∈ M with commit cycle clk_pre(m(q)) / clk_post(m(q)). If q does not commit (e.g., an unsuccessful non-blocking call), then m(q) is undefined / absent and no Σ-event is added to M.
(Q_exec-matching / witness convention.) Because unsuccessful non-blocking polls are ε-steps and produce no Σ-event, Q_exec is not in general reconstructible from the Σ-projection alone. Whenever E3, E5, or any later proof compares executed message-call instances across Sys_ref and RTL_B, the comparison is made relative to the chosen execution witness. That witness supplies a partial matching μ_Q between dynamic source-level message-call instances in the compared traces, preserving process identity, endpoint/interface identity, dynamic source-order position, and source-level call-instance identity where applicable. If q commits, then μ_Q(q) commits the corresponding matched transfer m(μ_Q(q)) with the same Σ-label/value under the event-matching convention. If q is an unsuccessful non-blocking poll, then q has no m(q) and no Σ-label; its presence and its match must be established by the selected evidence: L.WC (with NB-CF only for its stated sufficient case), a legal Appendix L snooping/witness-selection construction, or direct constructive occurrence correspondence under the applicable C1/C2 and boundary contracts. The constructive case identifies the actual dynamic attempts and their matched decision snapshots, proves outcome equality under Core.NBDecision/Core.NBDecisionSnapshot, and respects registered continuation and fixed choices. The decision law alone does not supply occurrence matching; failed calls remain invisible in Σ. Thus E3/E5 quantify over Q_exec only relative to this chosen witness; they do not require failed non-blocking polls to be inferable from Σ alone.

(No deassert-before-commit assumption.) For blocking message requests q ∈ Q_exec, once q is issued, its endpoint-owned boundary request (BoundaryReq at the selected evaluated boundary; Core.IC(1)) is not withdrawable within that reset epoch: it remains asserted in every cycle from issue_pre(q) (resp. issue_post(q)) through q's commit cycle if it commits, and for Push the payload remains stable while the request is outstanding. If an affecting RESET occurs first, RW2 ends the dynamic instance at the epoch boundary; no other nonterminal transition may withdraw or reattribute it.
Notation convention (Issue/Commit functions). The function-like forms clk_pre/clk_post, issue_pre/issue_post, and m(q) abbreviate the unique values supplied when the corresponding Appendix C existence/uniqueness and Issue/Commit-linkage conditions hold. In particular, m(q) remains partial: a rule that requires a committed transfer must also require or entail that m(q) is defined. This notation does not replace the underlying relational Issue/Commit semantics.

Dynamic source-order convention

The order relation used below is over dynamic source-level operation/frontier items in the compared execution / witness, not over all syntactic call sites in the source text.

For a process P, let Q_exec,P denote the execution-level set of dynamic source-level message-call instances of P at the chosen abstract boundary. Q_exec,P includes blocking Push/Pop calls, successful non-blocking PushNB/PopNB calls, and failed non-blocking PushNB/PopNB polls.

Separately, let Ω_P denote the witness-specific dynamic universe of source-level frontier items that are actually reached under the fixed stimulus / selected witness and that are relevant to observable, frontier or completion reasoning. Items in Ω_P include synchronization-call instances, anchored signal Read/Write action instances, and message-operation frontier items at the chosen abstract boundaries. A message-operation frontier item is either (i) a blocking Push/Pop item that has been reached, whether pending or committed, or (ii) a successful non-blocking PushNB/PopNB item that contributes a committed Push_c(v) / Pop_c(v) event. A failed non-blocking poll q is an element of Q_exec,P, has no committed Σ-event m(q), and is not an element of Ω_P merely because it executed.

Because Q_exec,P is a universe of dynamic message-call instances while Ω_P is a universe of source-level frontier items, their message-operation overlap is not a literal set intersection. Let MsgOf_P(x) be the partial projection from a frontier item x ∈ Ω_P to its owning dynamic message-call instance, defined exactly for message-operation frontier items and undefined for pure synchronization or signal-anchor items. Define the message-frontier slice

$$\Omega_msg,P := \{ x \in \Omega_P \mid \exists\, q \in Q_exec,P : MsgOf_P(x) = q \}.$$

Define the corresponding message-instance slice induced by Ω_P as

$$Q_\Omega,P := \{ q \in Q_exec,P \mid \exists\, x \in \Omega_msg,P : MsgOf_P(x) = q \}.$$

Let Σ_P denote the corresponding observable labels/events contributed by items of Ω_P when they are Σ-visible. For a message-call instance q ∈ Q_exec,P, the committed Σ-event m(q) ∈ M exists only if q commits; if q is a failed non-blocking poll, m(q) is absent and q contributes no Σ-event. For a message-operation frontier item x ∈ Ω_msg,P with MsgOf_P(x)=q, item x contributes the committed Σ-event m(q) only if q commits. A pending blocking message-operation frontier item x may therefore be in Ω_P and

Ω_msg,P, and its owning message instance q may be in Q_Ω,P, even though m(q) is not yet present.

Operations on untaken branches and unexecuted loop iterations are not members of Ω_P, Ω_msg,P, Q_Ω,P, or Q_exec,P. Distinct loop iterations, unrolled instances, and repeated calls are represented by distinct dynamic instances.

Default source order for ordinary message calls. For dynamic items x and y of process P, write x <_src y, or equivalently x <_P y when the process must be explicit, to mean that P's source control flow and the R-rules force x to precede y in that dynamic execution / witness. For ordinary user-written source-level message-call instances in a sequential process, this is the dynamic program order along the executed control path. Thus, if two executed message-call instances q1,q2 ∈ Q_exec,P appear in sequence in the same process, exactly one of q1 <_src q2 or q2 <_src q1 holds according to that dynamic program order, including when q1 and q2 use distinct message-passing interfaces, occur in the same synchronization interval, arise from repeated loop iterations, or arise from unrolled instances whose source-level expansion is ordered by the selected source elaboration. Same-cycle issue, same-cycle commit, or issue of a later operation before an earlier operation commits is a timing/issue-policy freedom; it does not make ordinary source message-call instances incomparable. The no-reverse rule R4/E3 is therefore stated over this ordered dynamic source relation.

Limited sources of incomparability. The relation ≤_P used for frontier reasoning is the reflexive closure of the restriction of this source-induced order to Ω_P, extended only by explicitly modeled partial-order structure. Ordinary distinct-interface source calls are not incomparable merely because they are on different interfaces. A message-operation multi-frontier may arise only from explicit Sys_B^elab proof-internal structure such as staged/bundle/join/epoch-gate abstract channels whose elaboration package declares the relevant frontier items, evaluated boundaries, partial order, and projection/attribution facts. Such elaborated frontier items are part of the selected Sys_ref proof model; they are not an implicit weakening of source order among the ordinary user-written message calls.

For sequential-process signal IO, <_src / <_P is interpreted through the R2 anchor semantics rather than through the raw lexical statement location of the signal read or write. Thus a signal Read is ordered, for formal frontier and trace-compatibility purposes, at its pred_S anchor, and a signal Write is ordered, for formal frontier and trace-compatibility purposes, at its succ_S anchor. A lexically earlier signal Write whose succ_S anchor is a later synchronization call does not, merely by lexical placement, precede or block intervening message-operation instances in the same source interval; conversely, a lexically later signal Read whose pred_S anchor is an earlier synchronization call does not, merely by lexical placement, wait behind intervening message-operation instances. Signal IO may constrain message IO only through the explicit synchronization anchors and the stated R/E rules, not by raw lexical adjacency inside an interval.

Conceptual Model for a Single Process

For every process P the pre-HLS trace τ_P must satisfy the following *partial-order constraints* over these dynamic operation instances:

R1 (Program order of S).

∀ s1, s2 ∈ S : s1 <_src s2 ⇒ clk_pre(s1) < clk_pre(s2)

R2 (Location of R/W for sequential processes).

For a sequential process P, ordinary signal Read/Write events are anchored to synchronization events as follows:

$\forall r \in R_P : clk_pre(r) = clk_pre(pred_S(r))$
$\forall w \in W_P : clk_pre(w) = clk_pre(succ_S(w))$

where pred_S(r) (resp. succ_S(w)) is the closest dynamic synchronization-call instance that precedes (resp. follows) the signal Read/Write on the dynamic source path of the current execution / witness, after applying the anchoring convention above. This anchoring determines the signal event's formal position for ≤_P / <_P, Done_P, Remain_P, and NextFront_P reasoning. In particular, a sequential-process Write w is formally placed at succ_S(w), regardless of intervening message operations; when several Writes by P to the same sig share that anchor, the last executed source Write supplies the canonical value and earlier writes are overwritten. Likewise, a signal Read anchored to pred_S(r) is not delayed by intervening message operations merely because the read statement appears lexically after them.

Every observable signal Read/Write of a sequential process must have a real bounding synchronization anchor on the dynamic path on which it occurs. A process-start synchronization event START_P may serve as the initial predecessor anchor when the modeling convention explicitly includes START_P in S. A terminating process event FINISH_P may serve as the terminal successor anchor when the process actually terminates and FINISH_P occurs. If no real predecessor anchor exists for a Read, or no real successor anchor exists for a Write on the dynamic path being modeled, the sequential-process signal IO is ill-formed for this formal model or lies outside the finite observable prefix being compared. No observable Read/Write event is assigned clock time ∞. For an observable retained Read, the RULE 1 qualification establishes its membership and value from the anchor state/snapshot without future-dependent membership. Moving the logical position to pred_S(r) therefore does not require retroactive modification of a completed observable unit.

R2C (Combinational signal IO at the ε-fixed point).

A combinational process P has no process-owned synchronization events and no message-passing events in this formal clause; it may, however, contribute observable signal Read(sig,val) and Write(sig,val) events to Σ through explicitly selected R2C observation components. These events are not anchored by pred_S/succ_S. In every clock cycle t, the global state first reaches the cycle's ε-quiescent combinational fixed point after all sequential clock-boundary updates and all internal combinational propagation for that cycle have settled. Let μ_t denote this unique fixed-point valuation of the signal network.

R2C cycle-entry valuation. The fixed point μ_t is reached from the cycle-t entry valuation established after the applicable sequential clock-boundary updates, with the registered/process-owned request and signal drives and the external/direct-input values selected for cycle t fixed. Internal combinational propagation then settles from that valuation without advancing the architectural cycle.

R2C unit-generation convention

Fix an explicitly modeled combinational process or combinational network component C_comb and its selected observable slice Obs(C_comb), consisting of the process/signal/direction labels whose fixed-point values are included in that component's R2C observation. Define ComponentParticipatesInCycle(C_comb,t) to mean that C_comb is active in the selected reset epoch and Obs(C_comb) is defined at the cycle-t ε-quiescent fixed point μ_t; when cycle-locked observation is

used, the selected observation/elaboration package shall define this participation predicate explicitly. The fixed-point valuation μ_t exists every cycle, but for an ordinary non-cycle-locked component a Σ-visible R2C fixed-point signal-observation unit is emitted only at canonical observation occurrences: (i) the first cycle t for which ComponentParticipatesInCycle(C_comb,t); (ii) any later participating cycle in which the vector of fixed-point values on Obs(C_comb) differs from the vector in the most recent emitted R2C unit; or (iii) any participating cycle in which an explicitly modeled observer/sampler requires a fresh observation unit. Participating cycles in which the observable fixed-point slice is identical to the most recent emitted unit and no explicit observer/sampler requires a fresh unit are R2C stutter cycles for C_comb and emit no new Σ-visible unit. Cycles in which the component does not participate likewise emit no C_comb unit. Delta-cycle re-evaluation, intermediate combinational values, redundant re-drive before the fixed point, and ordinary R2C stutter cycles are ε/stutter for this component and are not separately Σ-visible. The first-participation and last-emitted comparison in this convention use the normative initialization/reset ledger in Core.R2CResetLedger.

Definition Core.R2CResetLedger (reset ownership and emission restart).

Every selected R2C component C has a fixed reset-ownership declaration ResetOwners(C) in the process/reset coordinates of the selected execution carrier and its qualified reset semantics, independently of the constructive route. For a component representing one combinational process P, ResetOwners(C)={P}. For an explicitly modeled network component, declare the nonempty set of process coordinates that owns that observation component and prove the source/implementation ownership binding. This is ownership of the component itself, not the set of producers of signals in Obs(C). A network spanning reset domains requires an explicit joint ownership and participation binding. A genuine RESET(S_p,S_c) affects C exactly when ResetOwners(C) ∩ S_p is nonempty. Both models use the same logical ownership and actual matched RESET occurrence; changes of input values or upstream resets outside ResetOwners(C) do not satisfy this trigger.

For each C, maintain proof bookkeeping seen_C and last_C for emission comparison and a monotonically increasing emitted ordinal k_C. Initially seen_C is false, last_C is undefined and k_C is zero. An emitted unit sets seen_C true, stores its vector in last_C and advances k_C once. An affecting RESET clears seen_C and last_C after any selected ending-epoch unit; it does not clear k_C or remove historical units. Thus the first subsequent participating cycle emits even if its vector equals the last pre-reset vector. In the ordinary generation formula, “first” means seen_C is false, and “most recent emitted” means the last unit since initialization or the most recent affecting RESET. Cycle-locked completeness is unchanged. Temporary inactivity leaves the ledger unchanged; held-reset cycles restart it only when they contain an actual affecting RESET occurrence under the selected carrier’s qualified reset occurrence semantics. No per-instance override of these restart semantics is permitted.

The component carries the initial logical epoch identity, subsequently replaced by ResetKey of the latest affecting RESET. If several RESET occurrences affect C at one boundary, their order must be fixed by the normative reset-boundary order; otherwise this component ownership declaration is unqualified. RW5-disjoint resets may commute only if they do not impose an ambiguous shared observation-epoch update. An unaffected component retains its ledger and epoch. Participation during hold/release and any required START remains explicitly bound to the declared initialization/reset semantics. The selected carrier’s qualified reset-edge participation and ending-action disposition determine whether an old-epoch unit participates on a reset edge; an included unit precedes RESET under RW4. Suppression must follow from the participation contract, not from dropping a mandatory emitted unit.

Component identity and global per-component emitted ordinal remain the R2C matching key. The epoch identity is provenance tied to the matched reset history, not a replacement ordinal or an additional ordinary signal label. Across reset, matching is established by matched reset ownership/occurrences, participation and sampler evidence, and exact emission on both sides. The

normative restart makes the forced first post-reset unit unambiguous; it does not by itself prove those other correspondence facts. Resetting only an upstream producer can cause ordinary change-triggered emission if the selected vector changes, but never forces first emission merely because that producer reset. Route-independent binding. The rule folds actual RESET(S_p,S_c) occurrences and their logical keys on the selected carrier; it does not require Core.RegisteredResetBatch or its completion semantics. For that constructive profile, RBP0's ResetScopes, ResetOccurrenceConvention and ResetPriority instantiate respectively these ownership, occurrence and reset-edge disposition bindings. Other routes supply their existing qualified carrier/reset bindings to the same global ledger rule. This bridge neither introduces a restart override nor changes their completion carriers.

R2C cycle-locked completeness.

If C_comb is declared cycle-locked, then for every cycle t satisfying ComponentParticipatesInCycle(C_comb,t), exactly one R2C fixed-point observation unit for C_comb shall be emitted from μ_t, even when Obs(C_comb) is unchanged and no separate observer/sampler trigger occurs. Conversely, no cycle-locked unit is emitted for a cycle in which ComponentParticipatesInCycle(C_comb,t) is false. This is a positive completeness obligation as well as an emission guard: proving only that every emitted unit occurs in a permitted cycle is insufficient if a required participating-cycle unit is omitted.

If a combinational process P reads sig in an emitted R2C unit of C_comb, the corresponding label is Read(sig, μ_t(sig)) for the cycle t at which that unit is emitted. If P drives sig in an emitted R2C unit of C_comb, the corresponding label is Write(sig, μ_t(sig)), where μ_t(sig) is the final resolved fixed-point value of sig at the chosen observation boundary. All Read/Write labels inside one emitted R2C unit are interpreted at the same ε-quiescent fixed point and may not be split.

Combinational-process signal IO is well formed only when the relevant combinational network is deterministic and reaches a unique ε-fixed point in each cycle being compared, and when each observable R2C component has a well-defined selected observable slice Obs(C_comb). For a multi-input combinational component whose inputs may be updated by independent latency-insensitive sources, the component is Σ-observable under the stutter-canonicalized R2C rule only if the chosen observation boundary provides a single explicit anchoring domain for the slice, or if an additional component-specific well-formedness proof shows that all admissible interleavings induce the same emitted R2C unit sequence. Otherwise the component's internal combinational propagation must remain ε-hidden, or the design must expose the relevant behavior through explicit synchronization, message passing, a cycle-accurate transactor, or a cycle-locked R2C observation boundary. Pure combinational oscillations, ambiguous multiple-driver resolution, hidden stateful behavior, or intentional combinational-loop behavior outside a unique fixed-point interpretation are outside this R2C model unless represented by explicit state or synchronization elsewhere in the formal model.

R2C determinism clarification. Here "deterministic" means that fixed cycle-entry registered state/drives and fixed cycle-t inputs determine one ε-quiescent fixed-point valuation for the selected observation network. It does not require the composed internal combinational microstep relation or evaluation order itself to be functional. Value-preserving re-evaluation, intermediate values, and redundant re-drive may be ε/stutter during settling provided they do not change the unique settled result. This permission does not allow an enabled ε-step at a claimed ε-quiescent endpoint: Core.4 and the operational fixed-point convention of Core.Settle still apply. Any abstraction of concrete bookkeeping out of the proof relation requires the applicable realization evidence.

R3 (Isolation around S).

Let s be a dynamic synchronization-call instance of process P. Let pref_S(s) be the set of executed dynamic message-call instances q ∈ Q_exec,P in the immediately preceding source interval of P whose

source-induced order places q before or at the boundary controlled by s. Let suff_S(s) be the set of executed dynamic message-call instances q ∈ Q_exec,P in the immediately following source interval of P whose source-induced order places q after s. Then
∀ q ∈ pref_S(s) : issue_pre(q) ≤ clk_pre(s) and ∀ q ∈ suff_S(s) : issue_pre(q) > clk_pre(s).
∀ q ∈ pref_S(s) with q a blocking Push/Pop : m(q) is defined and clk_pre(m(q)) ≤ clk_pre(s).
The third clause is the synchronization-completion fence of Core.SyncFence; it permits a blocking transfer and s to commit in the same cycle. In Sys_B^elab, m(q) here is the designated process-facing commit that completes/releases q, not an arbitrary internal elaboration transfer. Failed non-blocking polls in Q_exec,P satisfy the issue-side synchronization-boundary rule, but they contribute no Σ-event and do not become Ω_P frontier items merely because they executed.

R4 (Safe message issue ordering).

For every pair q1,q2 ∈ Q_exec,P within the same process and within the compared dynamic execution / witness,
q1 <_src q2 ⇒ issue_pre(q1) ≤ issue_pre(q2).
(For same-channel operations, this states that dynamic calls on a single interface are issued in source-induced program order. For distinct channels, it forbids issuing dynamic message-call instances in the reverse of their source-induced order. This rule does not impose ordering between syntactic operations on untaken branches or between dynamic instances that are not ordered by the source-induced partial order of the selected witness. Failed non-blocking polls are included in this issue-order universe when they are executed, but remain ε-steps with no Σ-event.) Strict scalar same-endpoint case: when q1 and q2 are distinct blocking instances on the same scalar explicit abstract boundary and endpoint direction governed by the Core single-transfer-per-cycle endpoint constraint in one reset epoch, Core.IC(4) strengthens this to issue_pre(q1) < issue_pre(q2).

Prefix-closed reading of issue order. R4 and E3 are to be read not only as inequalities between already-defined issue timestamps, but also as closure conditions on issue itself. Within a compared dynamic execution / witness, if q1 <_src q2 and q2 has issued by cutpoint σ, then q1 has also issued by σ, and Issue(q1) ≤ Issue(q2) on the same side of the comparison (Issue_pre for R4/source-side prefixes, Issue_post for E3/implementation-side prefixes). Equivalently, in each source interval, the set of issued message-operation instances is downward closed under the source-induced order used by R4/E3. This rule is what excludes the case where a source-later message operation has issued while a required source-earlier message operation in the same interval remains merely reached but unissued.

R5 (Channel capacity semantics; Sys vs. Sys_B).

Appendix G defines the raw rendezvous source model Sys, in which every channel has effective capacity 0. Concretely: for every channel c and any clock cycle t, the number of unmatched committed Push_c actions is zero and the number of unmatched committed Pop_c actions is zero; equivalently, no Push_c shall commit unless a matching Pop_c also commits at t, and no Pop_c shall commit unless a matching Push_c also commits at t.

In Appendices J and K we additionally use the back-annotated source interpretation Sys_B: the same source processes as Sys, but interpreted under the case-split channel semantics of E4 with capacities B(c) matching RTL_B. Thus:
• B(c)=0 is interpreted as the rendezvous case of E4: matching Push_c(v) and Pop_c(v) endpoint events commit in the same cycle, and no unmatched committed endpoint event may remain after any cycle boundary.
• B(c)>0 is interpreted as the cycle-boundary, non-fall-through bounded-FIFO case of E4, using the

occupancy predicate occ_c(t) and the FIFO availability predicates occ_c(t)<B(c) for Push_c and occ_c(t)>0 for Pop_c.

The source conceptual semantics for a process is thus a labeled partial order (Σ, ≤_P) generated by R1–R4 plus the applicable E4 channel semantics: raw rendezvous for Sys, or the case-split back-annotated channel semantics of Sys_B / Sys_ref.

Operational semantics location

The selected Sys_ref transition system, registered sequential-interface semantics, Policy L and Policy S, and the admissible Policy-L execution domain are defined normatively in the preceding "Normative Formal Core for Appendices G–K." Appendix G uses that core while stating the R1–R5 and E1–E5 trace-compatibility rules; it introduces no second operational semantics.

Trace Compatibility Relation

Let τ_pre and τ_post be the Σ-traces (finite or infinite) of the same process before and after HLS, where each Σ-event is labeled exactly as in "Normative Formal Core for Appendices G–K" (e.g., Push_c(v), Pop_c(v), Read(sig,val), Write(sig,val), and synchronization events).

Notation note. We write τ_pre ≈ τ_post as a named, ordered trace-compatibility predicate. The symbol ≈ is local to this document and is not used here to assert a mathematical equivalence relation; in particular, reflexivity, symmetry, and transitivity are not being claimed. Its arguments are ordered: the left trace supplies the pre-HLS/source reference witness, and the right trace supplies the post-HLS implementation trace constrained by E1–E5. In later appendices the same convention applies to τ_ref ≈ τ_post, with τ_ref as the prescribed Sys_ref witness.

Convention (S): all references to S, pred_S/succ_S, pref_S/suff_S, and <_src in E1–E5 use the dynamic source-induced order defined in R1–R4 for the compared execution / witness. Thus τ_pre denotes the Σ-trace of the pre-HLS source over the dynamic operation instances reached under the fixed stimulus / selected witness.

Σ-event matching convention (expansion of Definition Core.17 ΣMatch). We use a canonical per-endpoint occurrence matching to make "the same Σ-events" precise. For each channel c, match committed Push_c and Pop_c events by their ordinal occurrence on that channel (e.g., the k-th committed Push on c in τ_pre matches the k-th committed Push on c in τ_post; likewise for Pop_c). For each such matched pair, the endpoint (c) and the payload/value label must agree. For the ordinal index k to be well-defined under the per-cycle "set/bucket" trace representation used in these appendices, we assume that for any fixed channel endpoint c, at most one committed Push_c and at most one committed Pop_c may occur in any single clock cycle.

For each observable signal sig of a sequential process, match Read(sig,·) and Write(sig,·) by their ordinal occurrence on that signal within the process trace, again requiring the value labels to agree.

For sequential processes, within each anchor interval between consecutive synchronization events in S, record at most one Σ-visible Read(sig,val) and one Σ-visible Write(sig,val) for each (process,sig), at the E2 anchor. If several source Writes share that succeeding anchor, the last executed source Write supplies val and earlier same-interval Writes are overwritten. Additional physical sampling/re-driving that does

not alter the retained anchor value is ε-internal. If an intermediate intra-interval value must be observable, use an explicit synchronized interface rather than ordinary sequential-process signal IO.

For combinational processes, the canonical observable unit is the emitted R2C fixed-point signal-observation unit for the relevant combinational process/network component C_comb: the set of Read(sig,val) and Write(sig,val) labels for the selected observable slice Obs(C_comb), recorded at one ε-quiescent combinational fixed point under the R2C unit-generation convention. Intermediate delta-cycle reads/writes, repeated re-evaluations before the fixed point, and stutter cycles in which Obs(C_comb) is unchanged and no explicit observer/sampler requires a fresh unit are ε/stutter and are not separately Σ-visible.

R2C unit matching/index convention

For each combinational process or explicitly modeled combinational network component C_comb, project each trace to the subsequence of emitted R2C fixed-point signal-observation units for C_comb. The k-th emitted such unit in Sys_ref is matched with the k-th emitted such unit in RTL_B. This local emitted-occurrence ordinal, together with the component identity C_comb, is the correspondence rule for R2C fixed-point units; it is not the global observable-cycle bucket number of the whole-system trace. Thus repeated identical fixed-point observations across stutter cycles do not create additional units, while repeated equal-valued units created by explicit observer/sampler occurrences are distinguished by their local emitted-occurrence ordinal for C_comb. The ordinal continues across resets; Core.R2CResetLedger supplies the reset restart and epoch provenance without renumbering this sequence.

Matched pre-HLS and post-HLS fixed-point units must agree on the included process/signal/direction labels and fixed-point values. The same-cycle agreement required here is local to the matched emitted R2C fixed-point unit: all Read/Write labels inside that unit are interpreted at one ε-quiescent fixed point and may not be split. This local atomicity requirement is one of the explicit same-cycle semantic constraints and does not require unrelated Sys_ref and RTL_B events to share the same global bucket decomposition. If a component is cycle-locked, its local emitted-occurrence sequence is the ordered sequence of cycles t satisfying ComponentParticipatesInCycle(C_comb,t), with one required unit per such cycle. The pre-HLS and post-HLS traces must therefore agree both on the participation-indexed unit values and on the absence of omitted required units. Cycle-locked components fall outside the bucket-stutter freedom claimed for ordinary latency-insensitive R2C components.

R2C cross-model/cycle-lock clarification. Sys_ref and RTL_B are not assumed to share internal combinational state, a common combinational microstep model, or the same global cycle-bucket placement; each side settles its own cycle-entry valuation. For a cycle-locked component, both sides must satisfy the per-side participation/completeness rule above, and the k-th matched units are compared within those respective component-local participation-indexed emitted sequences; equality of absolute Sys_ref and RTL_B cycle numbers is not required. If RTL_B does not expose an explicit combinational-network model from which participation can be derived, the implementation/certificate evidence for the selected annotated prefix shall justify that its emitted sequence obeys the declared cycle-lock/participation rule. The R2C matching statement in E2 is the Appendix-G expansion of this same R2C clause of Core.17 ΣMatch, not an additional independent matching conjunct.

For proof-internal RESET(S_p,S_c) units included in the selected theorem comparison, match by reset scope and local occurrence within that scope's reset-epoch history: the k-th RESET for a given (S_p,S_c) scope in τ_pre matches the k-th RESET for the same scope in τ_post. Matched RESET units therefore have the same scope. Independent disjoint RESET occurrences may commute unless an explicit reset-boundary or other observable-order constraint orders them.

For synchronization events, match by their per-process occurrence index in the source order (R1; preserved on the post-HLS side by E1 below). Independent Σ-events on different endpoints (including distinct message-passing interfaces) may commute and/or be grouped differently within a clock cycle as permitted by the R-rules and E3. Here "independent" is meant in the Σ-equivalence sense: the pair is not additionally ordered by any explicit Σ-visible ordering/timestamp constraint in E1–E5 (notably E3/E5); it is not a statement about (in)comparability under the Appendix G causal-dependency relation (→). Accordingly, τ_pre and τ_post are not required to be identical as a single total order, nor to agree on same-cycle "grouping" of distinct-interface Push/Pop operations, beyond the explicit ordering/timestamp constraints in E1–E5 below.

Consistently with Definition Core.17, we say τ_pre ≈ τ_post iff ΣMatch(τ_pre,τ_post) holds under the convention above and all the following hold:

E1 (Per-process synchronization-order preservation).

For every process P,
$\forall\, s_1, s_2 \in S_P : s_1 <_P s_2 \Rightarrow clk_post(s_1) < clk_post(s_2)$.

Synchronization events are matched by their per-process source occurrence index, with the strict post-HLS cycle inequality above and the strict source timing required by R1; occurrence order alone is not a replacement for those timing constraints. E1 does not preserve incidental global clock ordering between synchronization events of different processes unless that ordering is induced by an explicit inter-process synchronization relation already represented in Σ.

E2 (Signal-visibility preservation).

Partition signal Read/Write events by process kind:
• R_seq and W_seq are the ordinary signal Read/Write events of sequential processes.
• R_comb and W_comb are the signal Read/Write events of combinational processes, interpreted by R2C.

For sequential-process signal IO:
$\forall\, r \in R_seq : clk_post(r) = clk_post(pred_S(r))$
$\forall\, w \in W_seq : clk_post(w) = clk_post(succ_S(w))$

Equivalently, ordinary signal Reads/Writes of sequential processes are logically timestamped at their synchronization anchors, as in R2. The same anchoring convention is used on the pre-HLS side when forming the matched trace.

For combinational-process signal IO, pred_S/succ_S anchoring does not apply. Instead, each R2C fixed-point signal-observation unit is matched by the component-local occurrence ordinal defined above: for each combinational process or explicitly modeled combinational network component C_comb, the k-th R2C fixed-point unit of C_comb in Sys_ref matches the k-th R2C fixed-point unit of C_comb in RTL_B. The matched units must contain the same corresponding combinational Read/Write events and fixed-point values at their respective ε-quiescent combinational fixed points, as defined by R2C. Any delta-cycle re-evaluation, intermediate combinational value, or redundant re-drive before the fixed point is an ε-step and is not separately Σ-visible. This condition does not require the matched R2C units to occur in the same global observable-cycle bucket number, nor to be grouped with the same unrelated events, in the two executions.

E3 (Safe message issue ordering). (Post-HLS preservation of R4).

For every pair of executed message-call instances q1,q2 ∈ Q_exec,P of the same process P:
(i) Same-interface source order: if q1 and q2 access the same message-passing interface and q1 <_src q2, then issue_post(q1) ≤ issue_post(q2). This preserves source order at cycle granularity; any stricter serialization required by the concrete interface is supplied by the channel/interface semantics, not by E3 itself.
(ii) Distinct-interface no-reverse: if q1 and q2 access distinct message-passing interfaces and appear in sequence in the source (q1 <_src q2), then issue_post(q1) ≤ issue_post(q2) (i.e., they shall not be issued in the reverse sequence).
(iii) Prefix-closed issue obligation: for every cycle-aligned cutpoint σ of the post-HLS execution / witness, if q1 <_src q2 and q2 has issued by σ, then q1 has also issued by σ and Issue(q1) ≤ Issue(q2). This applies across same-interface and distinct-interface source order. It is an implementation-side obligation on issued sets, not merely an inequality over pairs whose timestamps are already defined.
(Note: HLS internal pipeline staging, dequeue, and hand-off inside the back-annotated process/channel realization are not, by themselves, Σ-visible boundary issue events in Q_exec,P nor committed-transfer events in M; therefore such internal movement does not constitute a counterexample to (ii) or (iii).)
Strict scalar same-endpoint case: when q1 and q2 are distinct blocking instances on the same scalar explicit abstract boundary and endpoint direction governed by the Core single-transfer-per-cycle endpoint constraint in one reset epoch, Core.IC(4) strengthens clause (i) to issue_post(q1) < issue_post(q2). For this ordinary scalar blocking special case, once the selected boundary mapping has qualified the process-owned BoundaryReq, Core.IC persistence/back-to-back ownership, scalar endpoint serialization, and the dynamic same-endpoint source sequence derive the same-interface Issue order; the implementation audit therefore cites the corresponding Core.IC/BoundaryReq evidence rather than selecting Issue annotations to satisfy clause (i). This does not weaken clause (ii) or the cross-interface content of clause (iii). Core.17/ΣMatch separately checks the matched committed per-channel occurrence/value observations.

E4 (Per-channel channel semantics: rendezvous and bounded FIFO).

For every observable channel c with capacity B(c), the projection of τ_post to events on c is legal for that channel's capacity model. The channel model splits into two cases.

• Rendezvous case, B(c)=0. Channel c is a capacity-zero rendezvous channel. A committed transfer on c consists of a matching same-cycle pair Push_c(v) and Pop_c(v) with the same payload value v. No unmatched committed Push_c or Pop_c may exist after any cycle boundary. Equivalently, the k-th committed Push_c(v) and the k-th committed Pop_c(v) occur in the same clock cycle and carry the same value. A Push_c endpoint can commit in cycle t only when the matching Pop_c endpoint boundary request is also asserted in cycle t, and symmetrically a Pop_c endpoint can commit in cycle t only when the matching Push_c endpoint boundary request is also asserted in cycle t.

• Buffered case, B(c)>0. Channel c is a bounded FIFO with cycle-boundary, non-fall-through semantics. The committed Pop_c(v) events return exactly the FIFO-ordered sequence of values v previously committed by Push_c(v), with no drops, duplications, or reordering. For every cycle-aligned prefix π_t of the c-projection—i.e., π_t contains exactly the events on c after its most recent affecting RESET (or reset-empty initialization) and committed in cycles < t—letting push(π_t) and pop(π_t) be the number of Push_c and Pop_c events in π_t, we have

$$0 \le \mathrm{push}(\pi_t) - \mathrm{pop}(\pi_t) \le B(c).$$

Equivalently, the abstract occupancy after cycle t−1 is occ_c(t) = push(π_t) − pop(π_t). E4 storage legality for commits in cycle t is evaluated against occ_c(t) at the start of cycle t: Pop_c may commit only if

occ_c(t) > 0, Push_c may commit only if occ_c(t) < B(c), and if both Push_c and Pop_c commit in the same cycle then necessarily 0 < occ_c(t) < B(c). At an ordinary uncoupled primitive buffered boundary, these occupancy conditions together with the attributed own-side request also determine the complementary availability used by Core.12. At a coupling-sensitive Sys_B^elab boundary they are necessary storage-legality conditions only; actual ChannelEnabled additionally requires the settled MirrorAvail_E value produced by the declared elaboration network.

Thus the inequality-based FIFO availability predicates are used only for B(c)>0. The B(c)=0 case is not interpreted by substituting B(c)=0 into the bounded-FIFO inequalities; it is interpreted by the rendezvous rule above.

Note: occ_c(t) is the logical occupancy of channel c at the chosen Σ boundary, induced solely by the Σ-visible boundary-commit history (Push_c/Pop_c) and the annotated capacity B(c) in the buffered case B(c)>0. For B(c)=0, the corresponding channel fact is the same-cycle rendezvous request/commit predicate rather than a nonzero FIFO occupancy predicate. The balance is channel-epoch-local: an affecting RESET restarts the empty-FIFO history balance without inventing Pop events for discarded contents. A reset outside c's scope leaves its history and balance unchanged under RW5. At a post-L3 observation use the actual history through that occurrence for occ_c(σ); the formula for occ_c(t) remains the cycle-entry balance. For an elaborated outer-channel balance, the history is the complete π_Σ,E channel history of Q.1, including both endpoints, not the later ρ_DUT-restricted observation history. A hidden DUT endpoint therefore remains included in E4 accounting.

E5 (Messages cannot cross their immediate synchronization boundary).

For every synchronization call s (in the source order used by R3 / pref_S / suff_S over Q_exec,P):
∀ q ∈ pref_S(s) : issue_post(q) ≤ clk_post(s)
∀ q ∈ suff_S(s) : issue_post(q) > clk_post(s)
∀ q ∈ pref_S(s) with q a blocking Push/Pop : m(q) is defined and clk_post(m(q)) ≤ clk_post(s).
The third clause is the implementation-side synchronization-completion fence: no earlier blocking request may survive s into the succeeding source interval, although q and s may commit in the same cycle. In Sys_B^elab, m(q) denotes the designated process-facing commit that completes/releases q. In addition to the non-strict clock inequality, E5 normatively includes the strict canonical generator m(q) ≺_J s of Core.SyncFenceOrder and its legal-prefix conformance. For same-cycle completion this is required semantic precedence, without a separately observable physical sub-cycle ordering on RTL_B. Core.SyncFenceOrder fixes this interpretation for source, constructive, B1 and K.CT uses alike. Under the Issue/Commit linkage above, any boundary request assertion that persists while the process is still pending at s reflects only already-issued outstanding operation(s) in pref_S(s); no operation q ∈ suff_S(s) may have Issue(q) ≤ clk_post(s), and any back-to-back request handoff on that endpoint direction, if present, is attributed in the first cycle strictly after clk_post(s). For non-blocking calls, a later executed source-level PushNB/PopNB in suff_S(s) is a distinct q and cannot be identified with an earlier pref_S(s) call solely because the endpoint request signal remained asserted.

Local source realization for the constructive system theorem is supplied by the contracts and proofs in Appendix J.

For every covered finite RTL_B prefix and actual implementation successor, the selected constructive evidence supplies a legal finite source extension under the prospectively fixed source semantics, choices and environment contract. Core.ReachPrep, Core.PreparationEvidence and the applicable registered-phase rules govern local preparation; the constructive boundary and composition contracts establish

actual shared-boundary decisions and matching completed occurrences. Neither local preparation alone nor a replayed event list proves complementary endpoint presence or a global source execution. Appendix L may supply legally realized witness-selection evidence, but does not replace these constructive obligations or establish an unrestricted reverse correspondence.

Allowed Transformations and Why They Preserve ≈

Transformation permitted by the rules	**Formal justification**
Loop pipelining (overlaps iterations)	Introduces internal pipeline stages (modeled as ε-steps) and overlaps iterations. R1–R5 (and hence E1–E5) are applied to the explicit S set. Loop pipelining may commute independent Σ-actions across iterations (e.g., across distinct channels)—where "independent" here means "not additionally ordered by any explicit Σ-visible constraint in E1–E5 between the pair (notably E3/E5)," rather than "incomparable under Appendix G's causal-dependency relation (→)"—but it must still preserve: (i) per-channel FIFO legality (E4), (ii) the required issue discipline (E3) for source-ordered pairs, and (iii) the sync-boundary constraints (E1/E5) with respect to the actual selected synchronization actions. Pipelining does not introduce implicit in-body S-actions; the surrounding real synchronization boundaries retain their stated scope.
Overlap of internal dequeue/staging vs. later pipeline work (ε)	Permitted because the internal staging/acceptance activity is ε-internal within the back-annotated channel realization (as defined in "Appendix B - Pipelined Loops") and does not constitute a Σ-visible boundary message-issue reorder. Σ-visible boundary message issues must still satisfy E3, and per-channel committed-transfer legality is ensured by E4; Appendix B's automatic-flush contracts and K.CT address required-workload completion under their additional premises.
#pragma hls_direct_input (late sampling of static signals)	This pragma imposes a designer/environment stability contract: the signal's value must remain stable after reset (or after its last permitted update) while the receiving process may consume it. The logical signal Read actions r remain anchored exactly as in E2 (clk_post(r) = clk_post(pred_S(r))). HLS may implement this by sampling the signal later (instead of inserting internal storage), but because the signal is stable the sampled value equals the value at pred_S(r); this establishes the sampled-value part of E2 only under that stability contract. Complete Read membership, source legality, initialization/anchor matching and the remaining correspondence obligations still require the applicable J.StableDirectInputContracts and J.AnchorDirectInputLift evidence; stability alone does not establish ≈.

Transformation permitted by the rules	Formal justification
#pragma hls_direct_input_sync (controlled updates)	This pragma imposes an explicit timing contract on the environment: the controlled direct-input signals may change only on the cycle of the designated synchronization call s* (i.e., during the sync handshake). Under the qualified update/reader binding, the logical anchoring required by E2 is preserved: for the receiving process, the relevant Reads are anchored at the closest preceding synchronization call pred_S(r) = s*, and any environment update Write w_update is aligned with synchronization (succ_S(w_update) = s*). HLS may still implement late sampling internally, but the value observed is the same as the value at the E2 anchor point, provided the environment protocol, pending-interval protection and protection of old-value users meet Core.SyncProtocol and the applicable anchor/configuration contracts. Complete Read membership and actual sampling/update attribution remain obligations of J.AnchorDirectInputLift or the explicitly selected configuration-observation profile; timing alignment alone does not establish ≈ or change E2. **Instrumentation note (logical signal timestamping).** In the Σ-traces used for ≈ checking, sequential-process signal Read/Write events are timestamped at their E2 anchor points (e.g., clk_post(pred_S(r)) for Reads), not at any later physical RTL sample cycle that HLS may introduce. Any internal late-sampling micro-steps are treated as ε-steps. Combinational-process signal Read/Write labels are handled by R2C rather than by pred_S/succ_S anchoring or by global observable-cycle bucket number. A monitor records a combinational fixed-point value only after the simulator/RTL has reached an ε-quiescent combinational fixed point, and that value contributes to Σ only when the relevant R2C observation component emits an R2C fixed-point signal-observation unit under the R2C unit-generation convention. Intermediate delta-cycle reads/writes, redundant re-evaluations, transient combinational values before the fixed point, and R2C stutter cycles whose selected observable slice is unchanged are ε/stutter and are not separately Σ-visible. Therefore, equivalence checking must instrument/record the appropriate logical Read/Write event: the anchored event for sequential-process signal IO, or the emitted R2C fixed-point signal-observation unit for combinational-process signal IO. A wrapper/transactor may be used when needed to expose only those logical events rather than naive physical sample, delta-cycle, or per-cycle stutter events.

Transformation permitted by the rules	Formal justification
Memory-access reordering proven conflict-free	Let addr(a) be the symbolic address of access a. If the tool proves addr(a_1) ≠ addr(a_2) for the overlapped window, then reordering the underlying memory operations may be treated as observationally silent only when that activity lies below a selected boundary satisfying Core.MemFaithful relative to the untransformed logical mapping semantics. If the reordered mapped message/signal events themselves are selected into Σ, the ordinary Core.17/E1–E5 ordering and matching obligations apply; the table entry does not authorize changing that selected event sequence.
Implicit FSM states / added latency	Extra states may be treated as ε-steps only when they introduce no selected observable/proof unit and the implementation preserves source legality, values, complete event membership, required order and atomicity under the selected observation boundary. The selected environment contract must also justify reactions to the added latency. Under these qualifications and the applicable component/correspondence proof, E1–E5 are preserved; inserting states alone does not establish that result.
Shared-memory arrays with explicit synchronization	When a memory is accessed by multiple processes, the designer explicitly coordinates access with synchronization operations modeled as S-actions, and J.Mem applies to every shared-memory dependency through which one process's write can affect another process's control flow, payload/value expressions, or later operation identity/order. Raw physical memory transactions may remain below the proof boundary only under Core.MemFaithful. If caching, merging, splitting, or reordering changes the physical transaction stream, constructive correspondence uses a transformation-invariant Core.MemProfile logical boundary that discharges J.Mem and Core.MemSyncOrder; otherwise the separately supplied memory-state extension (not developed here) is required. E4 applies to the logical message-passing channels actually selected at that boundary; it is not weakened to match variable-cardinality raw memory traffic. For K.CT, the profile also represents every hidden backpressure/resource/service condition needed for completion. The logical interface and primitive/local realization remain separately qualified.
Non-blocking message-passing (PushNB/PopNB) verified by post-HLS snooping	When latency-sensitive non-blocking behavior must be aligned for a selected comparison, an Appendix L wrapper may select a legal pre-HLS witness matching the recorded post-HLS committed events. This requires L.WC, complete coverage of consumed choices, W1–W5 and L.Dir, with source legality under Core.SourceChoiceBinding and the

Transformation permitted by the rules	Formal justification
	actual decision/snapshot rules. Delaying the source or matching committed transfers alone does not prove R1–R5 or E3–E5. Failed polls contribute no committed-transfer event to Σ, but their dynamic Q_exec identities and outcomes still require the chosen witness evidence whenever consumed, as specified by the Q_exec-matching convention above. This is conditional witness alignment, not a transformation that inherently preserves ≈; all remaining theorem premises still apply.
Latency-sensitive *global* signal IO matched by snooping	Latency-sensitive global signal IO may use the same legally realized Appendix L witness-selection mechanism. Qualify the source/environment protocol, complete choice coverage, signal values and anchor/group membership under L.WC, W1–W5 and L.Dir. Inserting synchronization or delaying a signal does not by itself preserve the selected partial order or prove ≈. The wrapper must realize behavior already admitted by the source model, leave the free-running RTL unchanged, and retain every remaining premise of the selected correspondence result.
Latency-sensitive *local* signal IO isolated in cycle-accurate transactors	Local timing-exact protocols may be confined to dedicated transactor processes exposing M or S operations to the surrounding design. The selected observation boundary must be fixed explicitly, and the transactor needs a qualified refinement to that boundary, with complete event/value/order and atomicity preservation and the applicable environment and composition contracts. Hiding the timing-sensitive region or naming a latency-insensitive interface alone does not establish system-level ≈. Any completion claim additionally retains the transactor's resource, exposure and service obligations.
One-way signal-handshake protocols with run-time assertions	For a one-way vld or rdy protocol, refinement to a two-way handshake with the missing complementary input tied to logical 1 requires a proved no-backpressure invariant over every admitted execution of the bound source, implementation and environment. A runtime assertion checks the exercised execution prefix; a clean finite run alone does not establish that universal invariant. With the invariant and the required event, value, ordering and boundary-refinement evidence, the one-way protocol may be compared at the corresponding two-way boundary. E2 governs logical signal Read/Write timestamps and does not itself prove this protocol refinement.
Combinational processes	Combinational processes have no process-owned S actions and no message-passing M actions in this formal clause, but they may have observable signal Read/Write events in Σ

Transformation permitted by the rules	Formal justification
	through explicitly selected R2C observation components. Their signal IO is interpreted at ε-quiescent combinational fixed points as specified by R2C: intermediate delta-cycle propagation is ε, and only emitted fixed-point Read/Write labels in the selected observable slice are Σ-visible. Repeated cycles in which the selected fixed-point slice is unchanged are R2C stutter for that component unless an explicitly modeled observer/sampler requires a fresh emitted unit. Under the well-formedness requirement that the combinational network is deterministic, reaches a unique fixed point in each compared cycle, and has a well-defined R2C unit-generation rule for the selected component, the RTL/tool-flow conformance obligation is to preserve the emitted fixed-point signal-observation unit sequence at the chosen observation boundary. Thus the required correspondence is not equality of simulator delta-cycle behavior, nor equality of global cycle-bucket numbers, nor an automatic consequence of ordinary combinational synthesis; it is equality of matched emitted R2C fixed-point units by component identity and local emitted-occurrence ordinal. Designs with oscillation, ambiguous multiple-driver resolution, hidden stateful behavior, intentional combinational-loop behavior outside a unique fixed-point interpretation, or multi-input observation slices whose interleavings can change the emitted unit sequence are outside R2C unless represented by explicit state, synchronization, a cycle-accurate transactor, a cycle-locked R2C observation boundary, or another formally modeled boundary.

For the purposes of the formal analysis, non-blocking message-passing is modeled at the level of committed transfers: a successful PushNB/PopNB contributes the same committed Push_c/Pop_c event to Σ as a blocking Push/Pop, while unsuccessful polls are ε-steps.

If a design contains latency-sensitive global signals or non-blocking transfers whose timing or ordering is externally visible at the DUT boundary and must be reproduced in the selected comparison, an Appendix L snooping/witness-selection wrapper may be used. When a witness satisfying WC and L.Dir exists, the wrapper selects an admissible pre-HLS execution whose relevant Σ-events align with the observed RTL execution. The wrapper does not enlarge the set of admissible pre-HLS behaviors; it selects one admissible execution. If the wrapper cannot realize such a witness, the compared run is outside the theorem scope and must be handled according to the Appendix L triage rules.

Appendix L defines the witness-existence conditions, monitoring, and triage required to establish this alignment for a concrete compared run.

Proof Structure

The proofs proceed in three stages:

First, source legality, choice binding, finite preparation and the operational channel/synchronization rules are defined independently of correspondence in Core. J.ProcessEnvClass and J.EnvironmentContract bind environment placement and behavior. These definitions alone do not make peer endpoints jointly ready or construct a global source execution; the selected constructive contracts prove those steps.

Second, Appendix J proves local-to-global constructive correspondence. Related local roots and the complete settling graph determine a legal snapshot. Primitive realization and resource/frame laws construct its boundary actions; simultaneous updates and finite legal preparation restore the relation. Actual finite carrier induction and the selected Batch or separately qualified prefix lift yield semantic source-history coverage. The full-fact adapter additionally records every fact consumed by completion.

Finally, K.CT performs a finite induction over prerequisite-ordered required commit units. J.InternalFactCorr and conditional agreement supply identities and values of occurred facts; closure, B.CT/Q.CT exposure laws and service establish the missing unit's eventual commit. The result is workload-relative and does not rule out unrelated or post-completion deadlock. No response timeout, terminal drain-to-empty condition or serial-dominance theorem is needed.
This dependency structure separates source legality, constructive correspondence, source-property evidence and progress. A source pass does not establish the local correspondence or implementation progress laws, and the completion conclusion is not used to justify those premises.

Causal Dependency Between Processes

To prove system-level trace compatibility, we must distinguish between incidental timing and functionally significant ordering.

Formal Definition of Causal Dependency

To formalize the notion of functionally significant dependency and resolve ambiguity, we define a causal-dependency relation, denoted by →, on the set of observable IO actions (Σ) across all processes in the system. The relation is causal/information-flow rather than purely temporal: events related by → may commit in the same clock cycle, and the direction of → indicates which event may be depended on by the other.
Important: → is a causal-dependency graph, not by itself the source order used for the Appendix J induction. In particular, because the methodology permits distinct-interface message operations to be issued with timing overlap or in the same cycle while still preserving dynamic source order, and because rendezvous transfers are same-cycle atomic Push/Pop transactions, the unquotiented dependency graph → need not be acyclic in every legal execution. A cycle in → can represent a legal same-cycle mutually enabling dependency among atomic transactions; it is not, by itself, an observable mismatch. When Appendix J needs a well-founded induction order, it uses the separate required-order relation $\prec_J$ over observable units. That relation is generated only by the explicit ordering, timestamp, FIFO, atomicity, and synchronization constraints that must be preserved by the trace-compatibility predicate,

not by the full causal-dependency graph → and not by raw per-process source order on distinct-interface message commits.

Well-formedness use of causal-dependency. The well-formedness rule below uses → as a design-intent and source-admissibility relation: if correctness depends on the relative timing, value, availability, or cross-process observation of two events, that dependence must be represented by an explicit modeled causal mechanism such as message passing, synchronization, signal handshakes/anchors, declared memory serialization, or an Appendix L snooped/witness-selected boundary. This use of → does not add proof-preserved observable ordering beyond ≺_J, FIFO legality, rendezvous/synchronization atomicity, signal-anchor constraints, synchronization-boundary constraints, and declared memory-serialization constraints.

The causal-dependency relation → is the smallest directed relation satisfying the conditions below:

1. **Intra-Process Order**: If actions a and b occur within the same process P and a precedes b in the program's execution trace, then a -> b. This directly reflects the sequential nature of the code within a single thread. **Clarification (non-temporal).** Here "precedes" refers to the order of IO action *instances* in the process's trace/program order (i.e., the order induced by the process's execution), **not** the relative order of their **commit cycles**; distinct-interface message actions may be **issued** in order while **committing** in either order depending on backpressure.

2. **Inter-Process Communication**: The relation is established by the direct exchange of information or synchronization between processes.

    - **Message-passing (committed transfer):** If a is the commit of a send on channel c in process P1 (a committed Push_c, including a successful PushNB), and b is the commit of the corresponding receive on c in process P2 (a committed Pop_c, including a successful PopNB), for the same logical message instance, then a → b. Here "corresponding receive" is defined by the channel's FIFO semantics: b is the Pop_c commit that consumes the element produced by a (matching is by FIFO position / transfer instance, not by payload-value equality; duplicate payload values do not introduce ambiguity). Within one reset epoch of channel c, enumerate the committed Pushes and Pops from that epoch's reset-empty initialization in commit order. Whenever the k-th Pop occurs, it consumes the k-th Push of that epoch, and that producer occurrence has the causal edge to the Pop. An affecting RESET may discard unmatched stored Pushes without producing Pop events; those discarded messages have no later consuming Pop. This epoch-local producer/consumer pairing does not renumber the canonical per-endpoint occurrence identities used for cross-model matching by Core.17. For buffered channels with B(c)>0, E4 also imposes the required FIFO legality used by Appendix J's required-order relation ≺_J. For rendezvous channels with B(c)=0, the matching Push_c and Pop_c commits occur in the same clock cycle and are treated in Appendix J as one explicitly atomic observable unit. The directed dependency a → b records unidirectional information flow — the Pop observes the value supplied by the Push — but it does not make the two rendezvous endpoints separately orderable for the Appendix J induction, and it does not imply an earlier commit cycle. No reverse edge b → a is implied unless there is an explicit reverse-direction communication, such as another channel or a signal handshake. Unsuccessful non-blocking polls are ε-steps and do not participate in →.

- **Signal Handshake**: If a is a signal write action w(sig) in P1 and b is a signal read action r(sig) in P2 that reads the value written by a as part of an explicit handshake protocol, then a -> b. A complete two-way handshake (e.g., vld/rdy) creates a causal chain, such as w(vld)@P1 -> r(vld)@P2 -> w(rdy)@P2 -> r(rdy)@P1.

- **Explicit Synchronization (two-party SyncChannel / ac_sync**): Let *sout* be the commit of a SyncChannel sync_out (or equivalent ac_sync call) in process P1, and let *sin* be the commit of the matching sync_in in process P2 for the same synchronization instance. Then this synchronization acts as a two-process barrier:
  - Every observable action in P1 that occurs before *sout* (in P1's trace/program order) satisfies a→b for every observable action b in P2 that occurs after *sin*.
  - Symmetrically, every observable action in P2 that occurs before *sin* satisfies a→b for every observable action b in P1 that occurs after *sout*.
  - (The two sync commits *sout* and *sin* are treated as simultaneous; we do not impose *sout*→*sin* or *sin*→*sout*.)

3. **Transitivity**: The relation is transitive. If a -> b and b -> c, then a -> c.

Using this relation, we now provide a formal definition for causal dependency between synchronization events, which are the fundamental ordering anchors in the scheduling model.

Definition: Causal Dependency

A synchronization event s2 in process P2 is **causally dependent** on a synchronization event s1 in process P1 if and only if s1 -> s2.
If neither s1 -> s2 nor s2 -> s1 holds, the events are **causally independent** under this relation. Their observed temporal order may vary. A property intended to tolerate that variation must be established over all orders admitted by the selected source domain; if a particular order is functionally required, it must be modeled or separately justified. This explanatory causal relation does not replace the required-order, observation or agreement premises of a selected theorem.

Design methodology. Express functionally significant cross-process dependencies through modeled mechanisms such as message passing, paired synchronization, signal handshakes/anchors or qualified memory serialization. Appendix L may select a legal witness for a particular comparison, but cannot add a missing source constraint or prove a property for all source behaviors. A property depending on an unmodeled incidental order is not established merely by one favorable simulation. The applicable J source-domain coverage and K agreement premises determine the formal guarantee; this advice adds no independent blanket theorem exclusion and does not convert every causal edge into a strict proof-order edge.

System-Level Constructive Correspondence and Safety Transfer

Constructive system-level guarantee. Fix the same source/reference model, implementation, initial/reset interpretation, capacities and elaboration, stimulus/environment contracts, choices and observation profile. Under the applicable constructive local evidence and actual carrier qualifications of Appendix J, every finite implementation prefix in the declared domain has a reachable finite source prefix with equal selected canonical history. The proof is actual-edge induction over the local source realization, followed by the named prefix lift. It needs neither fairness nor existence of a future edge.

Source safety transfers only through J.SafeTransfer-HistCov or J.SafeTransfer-HistCovOn with their source-property premises. B1, when separately proved, additionally fixes the observable RTL cycle projection; it establishes neither source correspondence nor unique source choices. There is no unrestricted reverse inclusion of all source executions in RTL behavior. K.CT supplies the separate required-workload completion result under its stronger fact, agreement and progress premises.

Practical Implication ("No Surprises" Guarantee)

For a testbench observing the qualified canonical history, constructive correspondence supplies a matching legal source history for each covered RTL prefix. A property of every source history therefore holds on the implementation under J.SafeTransfer-HistCov. An individual pre-HLS pass is insufficient for general safety unless the stated covering-run and all-ideals property conditions hold. Required-workload completion instead uses K.CT's qualified finite accepting witness.
Direction of proof for source-side safety signoff. Constructive Post → Ref correspondence establishes a legal matching source history for every covered RTL prefix. A property valid on every source history in the proved coverage domain therefore transfers through J.SafeTransfer-HistCov or J.SafeTransfer-HistCovOn. A selected source execution can support the source-property proof only with the stated source-history coverage and all-finite-ideals conditions. Appendix L may align a legal source execution to a selected RTL run; neither that alignment nor safety transfer asserts that an arbitrary selected source behavior is realizable by RTL.

Latency-sensitive testbench caveat

A testbench whose stimulus or pass/fail decision depends on raw timing, timeout counters or unmodeled latency is not automatically covered by canonical-history equality. Include its behavior in the actual combined-system local contracts or prove the external environment step correspondence at every consumed observation. Latency-sensitive performance checks remain separate unless represented in the chosen property and proven correspondence. The phrase "no surprises" means no unmatched selected canonical implementation history in the qualified domain, not arbitrary source trace realization or automatic transfer of a single simulation pass.
Appendix G supplies the scheduling/observable contract; Appendix J proves constructive correspondence under explicit local and carrier qualifications. Source-property proofs and K.CT workload witnesses have distinct roles. Their exact semantics, observation boundaries and capacities must agree with the bound reference model.

Appendix H – Possible Criticisms

This section highlights several potential challenges in the verification approach of Appendix G.

1. Designer-Managed Shared-Memory Synchronization

Shared-memory communication requires explicit qualified access coordination and representation of every value-carrying dependency covered by J.Mem. Designer-inserted synchronization can order accesses but does not alone establish matching communicated values; supported message-passing or anchored-signal lowering, a qualified Core.MemProfile boundary, or the separately supplied memory-state extension provides the applicable proof route. The static checks performed by a particular HLS tool

or flow require tool-specific evidence and are not established by Appendix G. In practice, this risk can be mitigated by encapsulating shared memories within parameterized library classes (for example, examples 12_ping_pong_mem and 15_native_ram_fifo) whose memory access coordination must be checked in both the pre-HLS and post-HLS models. The example names are informative pointers, not supplied qualification evidence. Such classes are natural first candidates for reusable Core.MemProfile qualification: the profile places a transformation-invariant logical boundary above any permitted cache/merge/split/reorder behavior, discharges Core.MemFaithful, J.Mem, Core.MemSyncOrder, the MP6 ownership/arbitration structure, reset treatment, and the selected Core.MemProfile MP5 safety or completion disposition once per library pattern, and binds that qualified profile to each concrete theorem instance through Core.EvidenceHdr/Core.EvidencePackage. Until such a profile has been qualified, the class must present value-carrying traffic at boundaries satisfying the existing J.Mem route without transaction-changing mismatch, use the separately supplied memory-state extension, or carry a correctness argument outside the Appendices J and K theorems.

2. Latency-Sensitive Signal Alignment (“Snooping”)

See Appendix L for detailed discussion on snooping.

3. By Default, Matchlib Connections Model a Skidbuffer inside In<> Ports

(Note that this overall document and specifically Appendix G are not intended to be strictly tied to Matchlib. Instead, this document is intended to be applicable to any pre-HLS model written in SystemC using signals, message-passing channels, and synchronization calls.)

Informative library-model description: in the default Matchlib Connections throughput-modeling configuration described here, Pre()/Post() use a one-place buffer for in-flight transactions on Connections::In<> ports, so In<> ports behave like skidbuffers by default. Connections::Out<> ports introduce no additional storage in that configuration. Check this description against the library version and configuration in use. The applicable boundary mapping, storage and handshake conformance remain subject to the qualifications below; the description alone does not establish throughput accuracy.

When source and RTL configurations explicitly represent the corresponding skidbuffer realization, a rendezvous comparison boundary may be selected before that buffer, subject to the applicable boundary mapping and conformance evidence. The default tool configuration described here does not by itself establish trace compatibility. Storage, acceptance/backpressure, transfer identities and ordering must satisfy B-faithfulness or the selected Appendix Q elaboration qualification, as described in Appendix B and Appendix F.

Removing or retaining skidbuffers requires rebinding the prescribed source realization, capacities and handshake structure to the selected RTL configuration. Keeping buffer-internal activity outside Σ does not alone make the change irrelevant: visible acceptance and backpressure may still change. The flow must re-establish B-faithfulness or the relevant Appendix Q qualification at the selected boundaries. The following compile flag is the configuration mechanism described here for removing source port skidbuffers; its support and effect must be confirmed for the selected library/tool configuration:

```
-DFORCE_AUTO_PORT=Connections::SYN_PORT
```

With that flag effective in the selected configuration, source port skidbuffers are removed. Any skidbuffers retained in the RTL must still be represented by faithful Sys_B capacities or explicit Sys_B^elab structure, including any handshake behavior not captured by an ordinary FIFO. A flag or matching total capacity alone does not establish this qualification.

In this case the recommended methodology is to distinguish the following uses:

- Use the default throughput-modeling mode in Matchlib (with timing accuracy qualified for the selected configuration) for most analysis and debug, and for pre-HLS performance verification.

- For final verification, use the supported port configuration and the qualified capacity/elaboration mapping corresponding to the selected RTL; the flag above may be part of that configuration. Rerunning tests checks those selected executions but does not itself establish boundary faithfulness or implementation-wide coverage.

For required-workload completion, use the implementation-bound final-verification capacity and boundary configuration with Core.PolicyS or qualified group-serial issue, and prefer K.CT.LazySimulation on eligible channels. Lazy selection uses the source process-facing demands and only optional abstract staging moves; it is neither a port-buffer-removal flag nor a capacity change. A declared physical skidbuffer handshake remains mandatory. The resulting completion witness supplies no throughput guarantee and does not by itself establish universal source safety. Reusable implementation qualification and any independently selected snooping dependency remain required.

Appendix I – Reserved

Appendix I is reserved to preserve appendix lettering. Its former replay/preparation proof material is retired. The retained proof-boundary, poll and implementation issue-order definitions are in Core.ProofBoundary, Core.NBPollInterpretation and Core.ImplIssueOrder, and the non-blocking preparation definitions are in Core.DeferredNBHole; process-environment contracts are in J.ProcessEnvClass; and witness compatibility and NB-CF are defined in Appendix L. Constructive correspondence is proved in Appendix J, and workload completion in Appendix K.

Appendix J – Constructive System-Level Correspondence

Synchronization-pair well-formedness for ≈.
For SyncChannel/ac_sync operations with paired endpoints, the compared Sys_ref and RTL_B executions use the same dynamic synchronization-transaction pairing. That is, the two endpoint events of a matched sync_out/sync_in transaction are matched by dynamic sync-instance identity, commit as one two-party barrier transaction, and are treated as simultaneous events in the same observable cycle bucket. This synchronization transaction induces the same cross-process before/after ordering as described in the causal-dependency definition.
E1 preserves the per-process source order and strict cycle separation of distinct synchronization endpoint events of each process (with R1 governing source timing), but E1 does not weaken or replace the paired-barrier semantics of SyncChannel/ac_sync. The system-level trace-compatibility predicate $\tau_{ref,system} \approx \tau_{post,system}$ is defined only over synchronization-well-formed traces in this sense: if one endpoint of a dynamic SyncChannel/ac_sync transaction appears in the trace, then the matching

endpoint appears in the same observable synchronization transaction, with the same dynamic pairing, in both Sys_ref and RTL_B.

J.0 Shared observable units and required order

Definition J.RequiredOrderBasis (route-independent unit and generator convention).

Cycle-bucket and required-prefix convention. A system execution may be represented as a sequence of observable cycle buckets $\beta_1 \beta_2 \ldots$, where β_i is the finite set of Σ-events committed in one observable clock cycle of that particular execution. Same-cycle events inside a bucket are unordered except for the explicit constraints imposed by E1–E5, the channel semantics, signal-anchor semantics, and explicit synchronization constructs.

The bucket decomposition records the clock timing of one execution; it is not, by itself, part of the cross-model compatibility obligation. The relation $\tau_{ref,system} \approx \tau_{post,system}$ does not require Sys_ref and RTL_B to use the same observable cycle-bucket decomposition. Events that are not ordered by the explicit required constraints may be placed in different cycles or grouped into different buckets in the two executions, provided the matched executions preserve all ordering, timestamp, FIFO-legality, and atomicity constraints required by E1–E5 and by the explicitly modeled channel, signal, and synchronization semantics.

The required-prefix order is over matched observable units, not RTL_B bucket-prefix order and not the full Appendix G causal-dependency graph →. An observable unit is either:
• a singleton Σ-event; or
• an explicitly atomic same-cycle transaction that the semantics requires not to be split, including a rendezvous Push_c(v)/Pop_c(v) transfer, the two endpoints of a paired SyncChannel/ac_sync transaction, an R2C combinational fixed-point signal-observation unit, a proof-internal RESET boundary unit when dynamic reset is used (Normative Formal Core, reset well-formedness RW1–RW5), and any other same-cycle transaction explicitly identified by the signal/channel semantics.
E2 signal-anchor grouping convention: a synchronization commit together with the sequential-process signal Read/Write events that E2/R2 anchor to it (and therefore require to share its commit cycle) may — and, for mechanization, should — be grouped as one explicitly atomic same-cycle unit. This grouping replaces the same-cycle equality constraint between separately enumerated units by unit-internal atomicity. When anchored signal events are instead enumerated as singleton units, their E2 timestamp equality with the anchor is enforced by the Sys_ref realization semantics (R2) and the within-cycle listing convention, not by the strict order $\prec_J$, which never orders same-cycle-aligned events.
Let $\prec_J$ be the strict required-order relation over observable units generated only by the explicit constraints that must be preserved by $\tau_{ref,system} \approx \tau_{post,system}$:
• E1 synchronization source-order constraints within each process;
• E2 signal-anchor timestamp constraints, interpreted as required alignment with the relevant synchronization or R2C fixed-point unit rather than as raw ordering among all source actions;
• E4 channel/FIFO legality for explicit abstract channels, including same-channel FIFO order for B(c)>0 and atomic rendezvous grouping for B(c)=0;
• E5 immediate synchronization-boundary constraints;
• paired SyncChannel/ac_sync barrier semantics; and
• any other explicitly stated signal/channel/synchronization atomicity constraint.

Per-process source order contributes to $\prec_J$ only where it is reflected in one of these explicit required constraints, such as synchronization order, same-endpoint/same-channel order, signal anchoring, or immediate synchronization-boundary legality. E3 contributes issue-order and request-attribution obligations, but it does not by itself create a strict predecessor edge between distinct-interface committed observable units merely because their source calls are lexically ordered. The Appendix G causal-dependency relation $\rightarrow$ is not a generator of $\prec_J$.

The symbol $\prec_J$ denotes this shared required-order basis in every route that uses it. Its generators come from the stated operational and observation rules, not from a chosen replay order or a proof-construction graph. Required-prefix ideals keep each declared unit indivisible. J.GeneratorCompleteness binds the complete ordinary and extension inventory; J.RequiredOrderWF separately requires prefix closure, restriction stability and joint acyclicity, including any nontrivial contraction. J.OuterCanonHist defines the projected history and contraction of erased normative paths. This definition supplies vocabulary and semantic constraints only; correspondence, source reachability and acyclicity still require their named proofs.

Definition J.Reach (finite reachability sets Reach_post / Reach_ref).

Reach_post and Reach_ref are the sets of all finite legal implementation and source execution prefixes under the fixed instance. Their domains are defined independently of constructor success and have no fair-extension qualifier. A finite prefix ending in a deadlock remains reachable. Core.SourceChoiceBinding and Core.PreparationEvidence fix source legality; a successful proof record does not redefine it. Complete/fair-execution consumers state their service and continuation assumptions separately.

Definition J.GeneratorCompleteness (prospective normative rule inventory).

Before forming the raw required-order graph, bind every selected unit kind, observation/projection and normative generator schema to its semantic owner and applicability guard. Include every applicable ordinary $\prec_J$ rule and every applicable RESET, sampler, elaboration, order-only and other extension rule, including paths through erased support. Classify each schema as applicable with prefix-closure, bidirectional restriction-stability and strictness evidence, inapplicable with a proof of its guard, or part of an optional feature/observation selected prospectively with its complete obligations. An applicable mandatory rule without evidence makes the instance unqualified; it is never silently removed to coarsen the quotient. Ordinary R2C/sampler rules already belonging to $\prec_J$ remain ordinary, regardless of which profile consumes them. E5/Core.SyncFenceOrder owns the mandatory blocking pref_S message-before-sync edge, including its same-cycle case. It is not optional instance evidence; source SyncFence supplies its operational realization and RTL supplies its carrier conformance. The ordinary unit/generator convention is J.RequiredOrderBasis.

Definition J.SourceGeneratorRealization (reusable source-rule order evidence).

For the complete J.GeneratorCompleteness inventory, prove from the source primitive/action rules that each normative edge $u \rightarrow v$ has completion index $s(u) \leq s(v)$ on every legal source completion sequence. Where several units complete in one source primitive, provide a common strict internal rank h for the union of all its applicable cross-unit edges. Atomic members already contracted into one unit are checked internally rather than treated as a strict self-edge. Template evidence is universal over its admitted states, payloads and choices, and composition checks all cross-template edges and any order-only support. Bind declared prefix closure and restriction stability separately. These are semantic/template proofs, not an assumed future canonical equality.

Lemma J.SourceOrderWF (source completion rules establish joint strictness).

Under J.GeneratorCompleteness and J.SourceGeneratorRealization, every finite legal source completion prefix satisfies the joint acyclicity part of J.RequiredOrderWF. Proof. Assign each unit its actual completion-step index s and its declared within-step rank h. Every edge increases s, or preserves s and strictly increases h. A closed walk would keep s constant and strictly increase h around a cycle, a contradiction. Prefix closure and restriction stability are the separately supplied rule facts, so together they establish J.RequiredOrderWF; any additional many-unit contraction retains its separate contraction proof. □

Reusable primitive cases. J.RequiredPredCycleOrder covers the checked strict-cycle generators. Source primitive FIFO availability/order follows its E4 completion sequence; rendezvous and paired-sync endpoints stay in one unit. The mandatory E5/Core.SyncFenceOrder same-edge message-before-sync edge follows the source fence-satisfying completion before interval advance. Its finer-RTL prefix closure is explicit carrier conformance, while Batch uses its complete inventory. RW4 ending-unit → RESET → new-epoch edges follow the legal reset boundary sequence; co-completing resets use the declared joint reset order, and disjoint independent resets add none. E2 anchor membership stays inside the complete anchor group. Sampler/R2C and Q.1(c-order) paths use their declared source realization and common internal rank, including erased/order-only support. Arbitrary sampler, compound-unit or elaboration rules are not automatically qualified by these examples: their semantic/template evidence must close the same interface. Bind proved primitive/template instances per design instead of repeating their semantic proof; new cross-template dependencies still require a composition check.

Definition J.RequiredOrderWF (prefix closure and independent strict-order evidence).

Fix a legal carrier prefix and its finite complete proof-unit inventory, with already indivisible units contracted as specified by the observation. Form the raw normative generator graph before constructing a canonical partial-order history. For each generator u→v, require prefix closure: every legal prefix containing completed v contains completed u, possibly because both complete in one Batch transition. Separately require restriction stability between already existing occurrences in both directions: for exact prefixes $\tau \preceq \tau'$ of one legal carrier and endpoints u,v already present in τ, each admitted generator schema has an edge u→v in τ if and only if it has that edge in τ′. Thus extension neither deletes established edges nor introduces new edges between old occurrences. Preserve the old occurrence identities and their projection/group records under the same bound observation. These two properties do not imply acyclicity on a carrier allowing distinct units to complete together.

Require joint acyclicity evidence for the union of all admitted normative generators on this inventory, independently of any theorem that consumes its canonical-history partial order. A strict rank r into a well-founded ordered set, with r(u)<r(v) for every generator, is sufficient. A finite source completion sequence is another sufficient construction: if step index s never decreases along any generator and the graph induced by the units of each one step has a strict rank h, then the lexicographic rank (s,h) increases along every generator. A step containing one indivisible unit, or distinct units with no generators among them, has an immediate internal discharge. No-internal-generators alone is insufficient without the forward-step check. A unit-separating completion sequence supplies strictness from prefix closure; a coarser carrier needs the internal check or other independent evidence.

A rank is proof evidence, not an added required-order relation, an observable sub-cycle timestamp, or a merger of groups. It must cover the union of rules, including selected reset, sampler and proof-internal edges; checking each rule class separately for cycles is insufficient. Erased proof units are included wherever they participate in the graph or a consumed path. Ordinary projection maps one complete proof unit to its one whole retained group or erases it. Any additional many-unit contraction must separately establish that the quotient has no cycle, for example by a rank on the contracted graph; arbitrary contraction of an acyclic graph is not sufficient.

Scope. These are the strict-order well-formedness facts consumed by J.OuterCanonHist/Q.1(c-order), in addition to their two prefix properties. They may be derived from operational rules or supplied by independently checked evidence; prefix closure is not a substitute. On a construction-covered domain, J.SourceOrderReflection below permits source completion evidence and occurrence/path translation to supply the corresponding RTL group acyclicity before canonical equality is concluded. If a consumer also uses the full proof-unit graph, its erased-only or otherwise unmapped portions still require their own joint acyclicity evidence. Evidence cannot depend on the acyclicity conclusion it is used to prove.

Lemma J.CycleRankOrderWF (lexicographic evidence for joint acyclicity).

Fix a finite raw graph G of all complete proof units consumed by the selected order argument, including retained, erased, order-only and elaboration-internal units. Give every vertex u a total cycle assignment $c(u) \in \mathbb{N}$ and a rank r(u) in one strict well-founded order. For a unit with a defined completion cycle, c(u) is that cycle. For an order-only unit without a natural clock stamp, provide an independently justified placement/index assignment on the same carrier and check every incident generator against it; an unindexed unit is outside this lemma and requires another complete rank proof. The assignment must cover the whole consumed graph before the conclusion is used.

For every normative generator $u \to v$ in the union of all selected rule families, require $c(u)<c(v)$, or $c(u)=c(v)$ and $r(u)<r(v)$. In particular, an increase of r does not excuse a decrease of c. Then the lexicographic pair (c,r) strictly increases along every generator, so G is acyclic. Proof. Along a directed cycle, cycle indices cannot decrease and must return to their initial value; therefore they are constant throughout the cycle. Every edge must then strictly increase r, contradicting its strict well-founded order. Equivalently the lexicographic order gives a strict rank for G. □

This lemma supplies only the joint acyclicity part of J.RequiredOrderWF. Prefix closure, restriction stability and any nontrivial many-unit contraction qualification remain separate. Cross-cycle edges with $c(u)<c(v)$ impose no condition on r. A rank or placement assignment is evidence about existing generators, not a new observable timestamp, required-order edge or operational scheduler. A validated complete-unit enumeration is an alternative sufficient strict rank when it covers every consumed unit and respects every generator; an action-only enumeration does not automatically cover erased or order-only support.

Example J.StandardCycleRank (checked message, anchor, R2C and reset classes).

For an instance whose full generator inventory fits the following checks, use r(u)=(class(u),local(u)) lexicographically, with class order message < synchronization/anchor < R2C < RESET. Assign classes to whole atomic units, not to their members: a paired synchronization with its anchored signals/configuration is one anchor unit; a rendezvous pair or qualified compound message group is one unit. Choose local ranks wherever a class has same-cycle strict edges. The rank assigns arbitrary relative positions to otherwise independent units only as proof data; it does not order them in the canonical history.

Check all same-cycle generators. The E5/Core.SyncFenceOrder message-before-sync generator points forward in the class order even when both complete in one cycle. Its semantic owner is E5, not incidental witness serialization. Same-cycle R2C/sample relations must be checked against the actual selected sampler constraints; if they point backward in this class order, this standard instance is unavailable and another justified rank is needed. RW4 puts affected ordinary units on a reset edge in the ending epoch before RESET, and new-epoch units of that affected scope occur after that reset boundary in accordance with the selected cycle semantics. Check that any RESET-to-new-epoch generator used here strictly advances c; do not assume the class order can accommodate a same-cycle RESET-to-ordinary edge.

For same-cycle RESET-to-RESET generators, local ranks respect every required resetUnits/reset-boundary order. A linearization of independent resets is permitted as proof-only tie-breaking, provided it respects all required edges and does not add a semantic relation. For staging, compound-action, order-only or other elaboration units, explicitly assign c, class and local rank and check every incident generator, or use a different full-graph rank. Any extra generator is checked even when its endpoints belong to the standard classes. Cycle-locked emission completeness remains an independent observation obligation.
Once this finite rule-family check establishes that every generator either advances c or advances r at equal c, J.CycleRankOrderWF discharges joint acyclicity for all instances covered by that uniform check. No assertion is made that every currently permitted observation/elaboration automatically fits these four classes. Failure of this sufficient class assignment is not a proof of a cyclic normative order. Prefix and contraction evidence is still required.

Definition J.OuterCanonHist (outer canonical history projection).

Fix a theorem instance I, elaboration E, and selected DUT-boundary observation alphabet Σ_DUT. For any finite reachable source or RTL prefix τ ∈ Reach_ref(I) ∪ Reach_post(I), let Units_I(τ) be the complete selected observable/proof units already carried by that prefix under the Core.17 canonical occurrence convention and the theorem-instance observation/elaboration boundary.

Outer projected group carrier. For every complete unit u with a non-erased selected outer image, define Group_DUT,E(u) to be one outer projected group G = ⟨gid_G, Eff_G, ≺_G⟩. In an elaborated instance this record is exactly the ProjGroup_E(u) metadata supplied by Q.1(c-order); in the ordinary non-elaborated case it is the corresponding singleton construction. gid_G is the canonical group occurrence identity inherited from the Core.17 occurrence of u together with the fixed projection/token identity. Eff_G is a finite nonempty occurrence-indexed family of constituent Σ_DUT event occurrences with their labels and values, and ≺_G is the declared internal strict order/alignment required among those constituents. Q.1(c-order) requires a multi-effect proof unit/token to contribute all of its selected outer effects to this one group and requires flattening G to reproduce exactly its complete π_DUT,E = (ρ_DUT ∘ π_Σ,E) trace fragment. A group is an indivisible vertex of the ideal-prefix carrier: presence of G means presence of every constituent in Eff_G, and no canonical history or ideal may contain a proper subset of Eff_G. Write Flat_Σ(G) for that complete constituent event fragment, modulo only the independence already permitted by ≺_G.

Define Π_DUT,E unitwise by composing the applicable Π_elab/π_Σ,E complete outer-accounting projection with the subsequent restriction ρ_DUT to Σ_DUT and returning either ⊥ or the unique Group_DUT,E(u). The complete π_Σ,E history remains the input to Q.1(a)–(c), including channel accounting; Π_DUT,E and π_DUT,E do not replace it in a balance equation. Each explicitly atomic proof unit or already contracted proof atomic group is therefore mapped as one complete outer group or erased as a whole; Π_DUT,E may not expose only one endpoint, participant, signal member, transfer leg, token effect, or other proper fragment. Internal staged/bundle/join/epoch-gate units with no selected outer image are erased. A proof-internal RESET(S_p,S_c), START_P, FINISH_P, or other proof-only unit is likewise erased when Σ_DUT does not select it; if RESET is selected at the outer boundary, its scope is preserved exactly inside its retained singleton/group image.

Required-predecessor generator well-formedness. Let ≺_req be the strict required-predecessor relation on Units_I(τ). On ordinary selected observable units, its generators are exactly the generators of ≺_J in J.RequiredOrderBasis, with Core.17/E1–E5 as their semantic owners. A same-cycle alignment or atomicity relation represented inside one contracted proof unit/outer group is not a strict ≺_req edge.

If Units_I(τ) contains a selected proof-internal unit outside the ordinary ≺_J carrier, ≺_req shall additionally contain every applicable explicitly stated normative predecessor/successor edge, such as RESET/reset-epoch constraints, and no undeclared edge. Every admitted strict generator belongs to a declared prefix-well-formed class with two properties. First, predecessor-before-completion: on every legal carrier, every legal finite prefix containing the completed successor also contains the completed predecessor; the two may complete together under a Batch carrier. Second, restriction stability in both directions: whenever $\tau \preceq \tau'$ are exact prefixes of one legal carrier and endpoint occurrences u,v are already present in τ, a given admitted generator schema supplies u→v in τ if and only if it supplies u→v in τ′. The forward direction preserves every established edge; the backward direction prevents a new edge between old occurrences. The generator relation restricted to old occurrences is therefore unchanged. The fixed observation/projection likewise preserves their canonical identities, complete constituent effects and values, and internal group order. Ordinary generators with strict-cycle separation satisfy prefix closure by J.RequiredPredCycleOrder; other admitted generators require the direct prefix-closure certificate of their own semantics. Prove both directions of restriction stability from the occurrence-local rules; RESET edges use RW1–RW5. Each ordinary, sampler, elaboration, order-only and admitted extension generator carries evidence for both directions under its own semantic owner; evidence for only the backward direction is insufficient. In addition, the union of admitted generators and any nontrivial group contraction satisfies J.RequiredOrderWF. Its joint acyclicity evidence is established on the raw graph before this definition treats reachability as a strict partial order; it is not inferred from the two prefix properties. Any other theorem-instance proof-unit or additional ordinary strict generator shall carry a separately checked rule establishing both properties. J.GeneratorCompleteness binds the full applicable inventory. If an applicable normative generator lacks either part of this prefix-well-formedness evidence, the instance is unqualified; the generator is not dropped from ≺_req. An undeclared proposed extra rule is not admitted merely by a logger or witness ordering. Full per-process logging order or lexical source order is not an independently declared semantic order and is not by itself a ≺_req generator; in particular, E3 does not add a strict predecessor edge between distinct-interface committed units merely because their source calls are ordered. Thus the restriction of ≺_req to the ordinary J observable-unit carrier is exactly ≺_J, not a stronger source-order relation.

Define the induced cross-group outer order ≺_Σ_DUT on distinct retained groups A,B by A ≺_Σ_DUT B whenever representatives u,v with Π_DUT,E(u)=A and Π_DUT,E(v)=B are connected by a nonempty ≺_req path u ≺_req^+ v; erased intermediate units are contracted rather than deleting a dependency. In elaborated cases Q.1(c-order) additionally supplies, for every required cross-group outer order, a finite chain of already normative proof-unit predecessor rules. Because ≺_req contains exactly those admitted strict generators, that certificate yields the required ≺_req^+ witness and hence the corresponding cross-group edge; incidental serialization of independent projected effects is not promoted to required order.

CanonHist_Σ_DUT(τ) is the finite quotient consisting of the retained outer groups Π_DUT,E(Units_I(τ)), their canonical group occurrence identities, complete constituent event occurrences/labels/values and internal ≺_G relations, and the induced cross-group ≺_Σ_DUT relation, modulo arbitrary ordering and raw clock-bucket placement among distinct groups unordered by ≺_Σ_DUT. Define Flat_Σ(CanonHist_Σ_DUT(τ)) by replacing every retained group by its complete Flat_Σ(G) fragment while respecting its internal and cross-group required orders. Safety predicates Φ below are predicates on this quotient-valued canonical Σ_DUT history and therefore may inspect all flattened constituent outer events, but they may not distinguish representations that differ only by permitted ordering of independent groups. An incomplete proof atomic unit contributes no group until complete.

This definition is intrinsic to the actual prefix and needs no matching source witness or replay package. It is the authoritative canonical outer history for semantic correspondence and safety transfer. Erased proof units and normative paths remain included where required for the induced outer order; outer projection is not a substitute for J.InternalFactCorr's complete consumed fact signature.

Lemma J.ProjGroupOrderAcyclic (qualified outer-group contraction is acyclic).

For every legal finite prefix τ with J.RequiredOrderWF and the applicable ordinary or Q.1(c-order) whole-group projection, the induced cross-group relation ≺_Σ_DUT is acyclic. Proof. In the one-proof-unit/one-retained-group case, a cycle among distinct retained groups expands through its nonempty normative paths to a closed walk in the raw proof-unit graph, contradicting its independent strict rank/acyclicity evidence. Erasing intermediate vertices and retaining their paths does not change that argument. Where additional many-unit contraction is used, apply its separately checked contracted-graph acyclicity evidence from J.RequiredOrderWF; underlying graph acyclicity alone does not justify arbitrary contraction. Thus no pair of distinct groups can be strictly ordered in both directions, and no contraction may force an external group between its own constituents so as to create a cycle. Prefix closure and restriction stability are separately used for prefix preservation; they do not prove this acyclicity statement. □

Lemma J.RequiredPredCycleOrder (required generators with strict-cycle separation).

Let M ∈ {Sys_ref, RTL_B} and let u→v be an ordinary normative generator whose rule establishes strict-cycle separation. Then clk_M(u)<clk_M(v). The checked cases include E1 successive synchronization order, E5 strict post-sync interval separation, and primitive E4 FIFO availability/order cases whose non-fall-through, non-full-bypass or scalar endpoint rules force an earlier cycle. Apply a channel-template or additional generator case only with its own strict-cycle proof. E2 alignment and indivisible atomic members are not strict cross-unit generators; E3 lexical cross-interface order supplies no such edge. In particular, Core.SyncFence permits an earlier blocking call and its succeeding sync to complete in the same cycle: accounting that call before interval advance does not establish a strict clock inequality. Nor do RESET order, a finite within-cycle source serialization, or generic prefix closure establish strict-cycle separation. This lemma makes no such claim for those cases; use their direct prefix-closure and J.RequiredOrderWF evidence instead. □ The E5 blocking pref_S edge of Core.SyncFenceOrder is strict canonically but need not be strict in cycle index; use its direct carrier-prefix proof for the same-cycle case, not this lemma.

Lemma J.RequiredPredPrefix (legal execution prefixes are closed under required predecessors).

Let τ ⪯ τ′ be finite prefixes of one legal source or RTL carrier, with v complete in τ. If u ≺_req^+ v in τ′, then u is complete in τ. Proof. For each generator on the path, apply its prefix-closure certificate: every prefix containing the successor contains the predecessor. For the strict-cycle ordinary cases this follows from J.RequiredPredCycleOrder. The E5 blocking pref_S case uses Core.SyncFenceOrder's direct source/Batch/finer-carrier prefix condition, including same-cycle completion. Other ordinary or proof-internal generators use their direct rule certificate; RW1–RW5 handle reset-epoch boundaries, including distinct units completing together on an edge. Internal atomic members stay in one unit and are not a residual strict-edge case. Compose the one-edge implications along the finite path. Restriction stability separately ensures old-old rules do not appear retrospectively. Joint acyclicity is supplied by J.RequiredOrderWF and is not derived by this prefix lemma. □

Lemma J.OuterCanon-Ideal (outer projection preserves required-prefix ideals).

Let τ ⪯ τ′ be reachable finite prefixes on the same legal source or RTL execution carrier. Then CanonHist_Σ_DUT(τ) is a finite ideal of CanonHist_Σ_DUT(τ′) under the group-indivisible carrier and

cross-group ≺_Σ_DUT order of Definition J.OuterCanonHist. More generally, if H_1 is a finite required-prefix ideal of a larger complete-unit history H_2 under the same theorem-instance required order, then their Π_DUT,E outer-group projections have the same ideal relation. Proof. Take a retained outer group B already present in the smaller prefix/projection and any retained outer predecessor group A of B in the larger outer history. By Definition J.OuterCanonHist, A ≺_Σ_DUT B is witnessed by a nonempty ≺_req path between retained proof-unit representatives, possibly through units erased by Π_DUT,E. Definition J.OuterCanonHist builds ≺_req only from admitted prefix-well-formed required-predecessor generators—ordinary ≺_J plus the separately certified proof-unit edges—and not from full replay/lexical source order. In the execution-prefix case, Lemma J.RequiredPredPrefix implies that presence of the representative of B in τ entails presence of every required predecessor on that path, including the representative of A. In the abstract H_1/H_2 case, the same fact is exactly the assumed required-prefix ideal property. Hence A is already in the smaller outer history. For equality of the induced order on old retained groups, backward restriction stability maps each larger-history path ending at an old group into the smaller prefix, since predecessor closure puts all its intermediate units there; forward restriction stability preserves each smaller-history path in the larger prefix. Fixed occurrence/projection metadata preserves group identities, labels, constituent effects and internal order. In the abstract ideal case these restrictions are inherited from the same larger complete-unit history and required order. Group-indivisible projection prevents any proper subset of an atomic/multi-effect outer group from being manufactured by erasure. In elaborated cases Q.1(c-order) ensures that the cross-group order used here reflects every required order of the flattened outer trace, including per-channel FIFO order through multiple elaborated fibers. □

If the extension from τ to τ′ adds no retained outer group—only ε/non-unit activity and/or complete units erased by Π_DUT,E—the two outer histories are equal, including their order components. The retained group vertex set and every group's constituent/internal-order record are unchanged. Suppose the larger prefix had a cross-group order edge between two already-retained groups that was absent from the smaller prefix, and take a witnessing ≺_req path in τ′. Because the path ends at an already-present retained representative, Lemma J.RequiredPredPrefix forces every proof-unit occurrence on that path to have already been present in τ. The restriction-stability field of Definition J.OuterCanonHist's generator well-formedness condition then forces every adjacent admitted generator edge on that old-occurrence path to have already been admitted in τ as well. Hence the same ≺_req path, and therefore the same contracted cross-group edge, already existed in the smaller history, a contradiction. Thus an erased-only extension cannot create a new order edge between old retained groups. Conversely, every path witnessing an old cross-group edge persists in τ′ by forward restriction stability of each generator and the unchanged projection records. No old order edge disappears. Together these two directions, the unchanged retained vertices and fixed group records establish equality of the complete outer histories.

Definition J.RefProjCover (selected-reference-run observable coverage)

Fix a theorem instance I, a DUT-boundary observable projection Σ_DUT, and a selected complete source-reference execution ρ_ref^0. Define RefProjCover_I,Σ_DUT(ρ_ref^0) to hold iff, for every τ_ref ∈ Reach_ref of I, the finite outer canonical history CanonHist_Σ_DUT(τ_ref) of Definition J.OuterCanonHist is a finite ideal (a down-closed canonical prefix) of CanonHist_Σ_DUT(ρ_ref^0) under the induced outer required-order relation ≺_Σ_DUT. Lemma J.OuterCanon-Ideal supplies the projection-of-ideals rule. The definition covers observable ideals/down-closed prefixes; it does not require identical cycle buckets or a fixed linear order among independent observable units. An E5-aware discharge must preserve the applicable blocking-call/synchronization incidence under the source-history occurrence embedding and establish induced outer-order agreement and predecessor closure, including any erased fence-support

paths. Direct proof of this source-history ideal inclusion supplies the required coverage; RTL determinism is not a substitute.

Definition J.IdealPrefixClosed (ideal-prefix closure for safety predicates)

Fix theorem instance I and outer observation projection Σ_DUT. For a predicate Φ on the quotient-valued finite canonical Σ_DUT histories of Definition J.OuterCanonHist, J.IdealPrefixClosed_I,Σ_DUT(Φ) holds iff, for all finite outer canonical histories H and H′ in the selected theorem instance, whenever H′ is a finite ideal of H under cross-group ≺_Σ_DUT—with each retained outer group indivisible—Φ(H) implies Φ(H′). Φ is evaluated extensionally on the group-respecting flattened Σ_DUT content Flat_Σ(H): every constituent event of a retained multi-effect group is visible to Φ, but an ideal cannot expose only a proper subset of that group. In the safety results below, 'prefix-closed' means this ideal-prefix notion, not merely closure under deleting a suffix from one chosen linearization of otherwise independent events. The property depends only on this intrinsic quotient, not on a chosen replay or listing representative.

Practical scope. Typical qualifying properties include assertion-style "no bad observable event has occurred" invariants, per-channel FIFO/order/value-legality invariants, and protocol-legality predicates explicitly formulated to be downward closed over the retained outer required-order history. A cross-interface transaction checker need not satisfy J.IdealPrefixClosed when it requires an explanatory event that the induced outer relation ≺_Σ_DUT permits an ideal to omit—for example, an output on one channel checked against an otherwise independent input on another. Conversely, source/replay order that is not an explicit ≺_req generator does not by itself force that explanatory event into every outer ideal. Conversely, projecting away a proof-internal RESET does not erase the applicable epoch boundary: contraction through the erased RESET preserves the required predecessor relation between retained affected-scope units on opposite sides of that reset. RW5-framed unaffected units and independent disjoint reset scopes are not ordered merely because such a RESET occurred. When this ideal-closure condition is unavailable, prove the selected-source-execution method's all-ideals property premise directly, or use universal source-property validity and exact semantic source-history coverage on the relevant prefix domain. Any refinement of the selected observation/order must independently justify the retained dependencies.

Lemma J.SourceCheckpointRepresentative (equal-history source checkpoint).

Under Core.SourceCheckpointProfile, every source witness exported by the qualified constructive base/edge induction has a prefix λ_o in ChkRef(I,H) with H(λ_o)=H(λ). Proof. At a base use SCP1. At an edge, SCP2 identifies the already realized completed-cycle endpoint or independently declared initialization checkpoint and its history-preserving tail. Truncate that tail; source reachability is prefix-closed, and its certified absence of new groups or old-old order changes preserves H. SCP3 covers selected START/FINISH and initialization histories, so no visible unit is lost by truncation. A terminal witness already at its completed endpoint needs no continuation. □ This is a representative theorem, not a claim that λ itself belongs to ChkRef. Keep λ, not its truncation, as the inductive state for the next source extension. The profile is explicit evidence; the general constructive coverage theorem and its universal-Reach_ref safety alternative remain available when it has not been discharged.

Definition J.ReachableHistCov (reachable-source outer-history coverage).

Fix the same theorem instance I and selected domain D ⊆ Reach_post(I) used by the safety claim: the prescribed Sys_ref, initial/reset interpretation, elaboration E and capacities, fixed stimulus and environment/testbench placement and contracts, fixed implementation choices, and the DUT observation projection and quotient convention of J.OuterCanonHist. ReachableHistCov_I,E,Σ_DUT(D) holds iff, for every τ_post ∈ D, there exists τ_ref ∈ Reach_ref(I) such that CanonHist_Σ_DUT(τ_ref) =

CanonHist_Σ_DUT(τ_post). Equality uses the same E, whole-group projection, occurrence convention and required-order quotient on both sides; it is not equality of one arbitrary linearization.
Witness scope. The outer history is intrinsic to the actual prefix. Instance-bound choices remain fixed, and any incremental construction preserves its existing source prefix and occurrence choices. RefProjCover still quantifies over all legal source prefixes, not only witnesses emitted by a constructor. ReachableHistCov alone supplies no current-state correspondence, coherent extension across independently chosen witnesses, or progress. For elaborated models, bind Q.SafetyQualified and every additional operational, settling, environment and observation premise consumed by the proof. Full internal-fact correspondence requires the separate J.ConstructiveFactCorr qualification.

Lemma J.SafeTransfer-HistCov (semantic source-side safety transfer).

Assume ReachableHistCov_I,E,Σ_DUT(D), with D containing every RTL prefix relevant to the claimed conclusion, and let Φ be an extensional predicate on finite canonical outer histories under J.OuterCanonHist. Then: (1) if Φ holds on the outer history of every τ_ref ∈ Reach_ref(I), it holds on every τ_post ∈ D; (2) if a selected complete source execution ρ_ref^0 satisfies Φ at every finite canonical ideal and RefProjCover_I,Σ_DUT(ρ_ref^0) holds, it holds on every τ_post ∈ D. Neither clause requires J.IdealPrefixClosed.
Proof. Fix τ_post ∈ D and choose the reachable source witness supplied by ReachableHistCov. For (1), apply the universal source premise and substitute equal outer histories. For (2), RefProjCover places that witness's outer history among the finite ideals of the selected source execution; apply the explicit all-ideals premise and substitute equality. No construction-package field is used. □ Origin coverage alone yields a conclusion only on that domain. This lemma supplies property transfer only, not an additional construction or trace-compatibility theorem. For a separately qualified Core.RegisteredMessageBatch domain, J.BatchPrefixCoverage is an exact-history lift from origin coverage to all finite prefixes of that profile and requires no ideal-closure premise. Its completion/observation contract must be bound to the implementation. Coverage of a finer or different domain requires a separate exact semantic coverage proof.

Definition J.ReachableHistCovOn (source-domain history coverage).

Fix an independently defined source domain A⊆Reach_ref(I), an implementation domain D and the same intrinsic H/projection on both sides. ReachableHistCovOn_I,H(D,A) means that every κ∈D has λ∈A with H(κ)=H(λ). Ordinary ReachableHistCov is the specialization A=Reach_ref. For a qualified constructive checkpoint profile, J.SourceCheckpointRepresentative after the selected origin/Batch induction yields A=ChkRef(I,H). The domain is never the image of a successful constructor or a favorable subset chosen after checking Φ.

Definition J.RefProjCoverOn (covering execution for a declared source domain).

For A⊆Reach_ref fixed independently as above and a selected complete source execution ρ^o under the same instance/history convention, RefProjCoverOn_I,H(A,ρ^o) means that every H(λ), λ∈A, is a finite canonical ideal of H(ρ^o). Original J.RefProjCover remains the stronger A=Reach_ref specialization. It implies the restricted predicate by domain inclusion; restricting the source domain does not assert that every mathematical ideal is operationally realizable. The same E5 incidence, induced-order and predecessor-closure checks apply to each λ∈A against ρ^o, including erased support, within exactly this restricted source domain. They may be supplied by a direct ideal-inclusion proof or appropriately source-typed reusable evidence. This is source-domain coverage, not a conclusion from RTL determinism.

Theorem J.SafeTransfer-HistCovOn (domain-specific source-property transfer).

Assume ReachableHistCovOn_I,H(D,A) and an extensional finite-history predicate Φ. Clause (1): if Φ(H(λ)) holds for every λ∈A, then Φ holds on every κ∈D. Clause (2): the same conclusion follows from RefProjCoverOn(A,ρ^o) and Φ on every finite canonical ideal of H(ρ^o). Proof. Coverage supplies λ∈A with equal history. For (1), apply the universal source-domain property. For (2), RefProjCoverOn places H(λ) among the finite ideals of the selected covering history; apply the all-ideals property premise. Substitute equal histories in either case. □ The original two clauses of J.SafeTransfer-HistCov remain sufficient specializations with A=Reach_ref.

Evidence distinction. Restricting RefProjCover to A weakens the covering-execution obligation, but does not weaken clause (2)'s all-ideals property premise. For a finite covering history with n mutually independent groups there can be 2^n ideals; an infinite covering execution can have infinitely many finite ideals. A single finite covering-history evaluation implies the all-ideals premise only with the separate J.IdealPrefixClosed proof. Ordinary prefixes of one linearization are not all ideals. Neither clause (1) nor a directly proved all-ideals premise requires ideal closure.

Evidence-organization remark. Under RefProjCoverOn(A,ρ^o), the finite ideals of H(ρ^o) realized by some λ∈A are exactly {H(λ) : λ∈A}. Proving Φ on that set is therefore equivalent to clause (1)'s source-domain premise. It may organize an existing source proof, but is not a third transfer clause, a weaker proof obligation, or an automatically cheaper covering-run method. Determining realizability is itself a source-semantic question. No independently checkable superset or abstract checker is supplied by this remark.

Definition J.SourceSettleRealization (safety-only operational settling).

Guarded phase-entry obligation. This interface consumes an exact source L1 endpoint; R may store another legal phase checkpoint. Whenever an actual successor case invokes J.SourceSettleExists from such a checkpoint, that local case must first derive a finite, legal exact-prefix extension to the required L1 endpoint, under the actual phase/reset/initialization rules, retaining fixed choices and the prior source history and establishing the related roots there. The extension may be empty. Any selected units it emits belong explicitly to that case's matched inventory; it is called history-preserving only when proved so. A reset-hold or initialization case may not manufacture an ordinary L1 phase, assume eventual release, or skip required initialization units. A terminal export with no actual successor needs no phase-entry extension. If the case does not consume settling, it retains its own stated operational premises. C0/C4 or the applicable A/RB/RC successor contract owns this proof; it is not generic source-cycle totality.

For every exact source L1 endpoint λ related to an actual implementation origin κ, bind the complete phase-legal source →ε,L2 relation to the declared combined evaluation graph. Every mutable L2 component is represented by a node or derived value; legal evaluation steps implement its equations, no unrepresented L2 transition remains, and satisfying all node equations is equivalent to being an operational L2 fixed point. Supply a finite legal evaluation path from λ under the frozen Core.Settle frame; if its initial state is already fixed the path is empty, and otherwise stop at the first operational fixed point. Pointwise equation/refinement evidence relates that endpoint to the implementation snapshot used by the actual edge. The graph includes every boundary/frontier root and predicate relevant to Core.16, E4, source primitive guards, result decisions and selected observations, including absent requests, nonminimal pending calls, live NB attempts, environment logic and elaboration gates, not only selected descriptors.

It is quantified only over the related source origins used by the construction, while every actual implementation successor there remains covered. Finite deterministic acyclic graph evaluation is the retained C2/RC1 sufficient evidence: realization must justify the actual topological evaluations as legal operational steps and account for all L2 activity. Separate component ranks must compose over the whole graph. A fixed point of a partial graph alone is insufficient. Core.SourceL2Settle may discharge the

path when all its own premises hold; otherwise prove the complete actual operational settling and matching required by this interface directly.

Lemma J.SourceSettleExists (legal activation snapshot).

Under J.SourceSettleRealization, the exact λ admits a finite history-preserving legal L2 extension to SettlePoint_I with the matched values of every consumed boundary/frontier predicate. Proof. Execute the supplied legal evaluation path while the L2 frame holds. Completeness excludes hidden enabled settling activity at its endpoint; the first-fixed-point condition makes that endpoint Core.Settle's SettlePoint. L2 emits no selected unit. Evaluate the complete frontier there, so Core.16 activation is the source operational activation, including for requests absent from the selected action set. □ Only existence of this suitable endpoint is consumed by the construction; no uniqueness theorem is required. This does not remove C2/RC1's deterministic acyclic profile restriction or admit combinational feedback.

Definition J.LocalOriginContracts (C0–C5; constructive registered-message correspondence).

Fix one theorem instance I, Core.ConstructiveOriginCarrier with origin family O_I and finite actual-edge ancestry from a covered base, and one exact source-prefix construction relation R(κ,λ,g). Here κ is an implementation origin prefix, λ a reachable source prefix at the phase checkpoint declared by the local contract, and g causally accumulated finite proof data. An active ordinary edge uses λ after its selected L1 window; a terminal export may end at its already completed L4 endpoint, and reset-hold/initialization cases use their explicitly bound source phase. No next L1 window is demanded merely to export a terminal history. If an actual later edge exists, its local case must still construct a legal matching extension from that exact phase checkpoint; no successor is discarded by tagging a state terminal. The profile has finitely many registered processes and explicit point-to-point boundaries, one clock, reset-established initialization with no later reset, and message-only observations; it excludes the synchronization, signal/direct-input, R2C combinational-process and multiple-clock extensions excluded by Core.RegisteredMessageBatch. Eager issue, NB calls, private computation and overlapped iterations are permitted when the following contracts hold. The same source model, capacities, environment placement/contracts, fixed choices and occurrence/projection convention are used throughout. Require Q.SafetyQualified(E) when elaborated, ordinary channel qualification otherwise, and implementation boundary/capacity faithfulness. This is a direct semantic coverage route under Q.1's consumer rule; every separately invoked theorem retains its own premises. Source choices and preparation use Core.SourceChoiceBinding/Core.PreparationEvidence. The relation is transported across any nonidentity designation by Core.ConstructiveOriginCarrier; equal histories alone do not discharge that transport. R consists of process-local control/preparation and value/operand-writer relations, complete attributed request/result records, explicit resource/token/order relations, environment binding and occurrence/activation ledgers. It may use relational ghost values tied to actual earlier events rather than a functional decoder from current registers. Old source prefixes, dynamic identities and bound choices are retained literally. The contracts below hold for every construction-reachable related origin and every actual implementation successor there; evidence restricted to one favorable schedule, NB outcome or arbitration choice does not discharge this quantifier. Shared invariant facts must be established at the base and preserved by the component postconditions; they may not assume existence of the desired global source continuation. For the structurally declared OM/OA profiles, J.OrdinaryConstructiveComposition constructs this relation from J.OrdinaryLocalEvidence and J.OrdinaryGlue and derives the global preservation obligations. Its local proofs have the prescribed vocabulary and do not hide arbitrary global preconditions. The broader C0–C5 interface retains its full stated scope and premises. For the restricted elaborated ELM/ELA profiles, J.ElaboratedConstructiveComposition supplies the corresponding local-to-global discharge through its

explicit enlarged completion vocabulary and closed generator inventory. It does not weaken the broader contracts or automatically qualify arbitrary elaborations.
C0 — Base and environment. Construct a finite legal source base and proof data related to every prescribed implementation base, with equal completed canonical history. Bind current environment roots. For each actual implementation environment step, provide a legal source counterpart preserving the exact past, choices and environment relation. This can be obtained from modeled peers/testbench processes satisfying these local contracts. Otherwise it is explicit evidence covering every observation on which a reactive environment depends; equal DUT histories alone do not establish it. No future stimulus or more cooperative environment may be substituted.
C1 — Process roots, operand identity and activation. Each local relation supplies equality of all relevant own-side request roots, including absences, owner/site/epoch/ordinal, endpoint, direction and stable Push value, every nonminimal pending blocking call, and each live NB attempt. Operands are bound to their actual source writers/field states at attribution. Private candidates meet Core.16's abstract-invisibility, reuse and payload-isolation requirements. From the settled boundary predicates and the declared frontier record, establish the correct L2 activation and preserve its owning occurrence across later windows. Distinguish pre-activation admission, already-admitted completion and invisible physical work from new source-later evaluation/Issue; persisting activation inhibits both forms of new launch. Cover the attributed roots of every declared transfer boundary, including proof-internal boundaries and nonminimal pending calls; a selected subset of roots cannot justify the complete forced set.
C2 — Whole-network settling and coherent selection. Declare ownership of all registered process/resource state and every relevant same-cycle root/dependency; shared accounting views read one represented state. Prove J.SourceSettleRealization from a finite deterministic acyclic evaluation order for the complete combined graph and the L2 frame, including all source-operational L2 transitions and cross-template dependencies. Combinational feedback is outside this sufficient profile. Invoke J.SourceSettleExists at the exact paired λ and actual κ; equal related roots and local equations yield the consumed settled boundary values and the complete frontier activation. Derive the complete finite descriptor set T and NB outcomes from that snapshot and the fixed choices, including every forced Core.TransferLaw and Core.SyncProtocol unit and every separately selected legal abstract action. Each descriptor names its issued call(s), participants, closed read/write footprints, snapshot guard, atomic legs, payload/result, accounting effects and retired owners; silent failed NB decisions are included. The complete physical transfer set satisfies E4, endpoint cardinality and coherent shared-resource admission by qualification of its equations and attribution. All sampled handshakes are mandatory; no maximality claim is needed for genuinely abstract movement or L1 issue. An NB outcome is computed by Core.NBDecision from Core.NBDecisionSnapshot and the legally fixed resolutions represented there, not chosen from replay latitude. Core.SourceChoiceBinding fixes the indexing and choice strategy/relation prospectively; any already-fixed witness must agree on every covered successor. All same-snapshot E4/declared resource updates consumed by an NB decision are checked explicitly.
C3 — Local boundary realization. For T provide a finite acyclic execution-dependency graph L and component residue functions on predecessor-closed subsets U of T, retaining the original snapshot. Components comprise process completion records and explicitly owned resource blocks; shared allocation is one jointly owned resource block, not independently framed views. Residues agree on shared reads, start at the current related states and are computed from the declared action updates. For every L-minimal remaining descriptor a, each participant proves the premises of its source primitive/declared compound-action rule from its residue and the fixed snapshot, its exact update to the residue for UU{a}, and the frame for all other components. Every guard read and write is covered; overlapping writes require a combined block or proved joint update. Compound legs require the actual atomic rule and compatible participants, not merely separate legality. Process completion effects use the existing call identities and do not execute result-dependent continuation. Final residues match every

implementation boundary effect exactly once. L is a witness execution order and supplies no canonical generator. For replacement-capable early staging, Q.CT.InputStagingResidue gives the fixed-selection residue obligation to instantiate with the component's actual source transitions, frozen owners/payloads and combined dependency order. It does not expand ELM/ELA coverage or replace the remaining C0–C5/A0–A6 contracts. C3 realizes the whole fixed forced-action inventory established by C2. Residue order cannot suppress a handshake, invent an additional same-edge transfer or exchange capture, source retirement and external delivery identities. Conflicting forced effects indicate failed qualification, not permission to choose a convenient subset.

C4 — Registered update and finite next preparation. For the local outcome packet established by C3, prove each process's legal L4 update and finite next-L1 evaluation/Issue word, restoring its local relation to the actual implementation successor. Uncompleted requests and values persist. Registered next states are computed separately from the common current state using only specified, proved forwarding, and published simultaneously. Returns at cycle t support dependent source work only at a strictly later Horizon; a private physical write is not backdated into source attribution. The word obeys the independently defined ReachPrep relation, Issue prefix closure, result availability and persisting Core.16 activation. Its local source-rule derivation supplies Core.PreparationEvidence, including any declared instance restriction; a successful constructor is not itself the legality test. Its writes are confined to owned locals/requests and its legality and operands are independent of other processes' newly prepared requests. Resource and environment updates have compatible frames. Consequently the finite local preparation words can be concatenated without interference; a profile requiring interprocess preparation dependencies needs an additional proof and is outside this sufficient interface. If the actual successor is a terminal export with no next preparation, retain its completed L4 source endpoint and record the phase in R. For a hold/initialization successor use the selected extension's explicitly qualified phase case. Every later actual implementation edge, if any, still requires a local source extension from that retained checkpoint; this convention asserts no future source-cycle existence. Before a subsequent actual case consumes J.SourceSettleExists, discharge J.SourceSettleRealization's guarded phase-entry obligation from the exact stored phase. No unproved L1 window, reset release or continuation is supplied by the relation. The simultaneous next state includes every mandatory physical effect of C2/C3. Revalidate any preparation or state-restoration proof that previously depended on leaving a present handshake uncommitted.

C5 — Canonical units and generator translation. Supply a causal occurrence map for completed proof units relevant to projection/order, including erased units. Matched retained groups have identical canonical identities, complete constituent effects/values and internal atomic structure. Translate each occurrence-local normative generator to a matching generator or finite normative path on the other carrier, in both directions; unmatched erased units are order-neutral or covered by those translations. Cover new successors against both old and new predecessors. Every generator has the predecessor-before-completion and bidirectional restriction-stability certificates of J.OuterCanonHist/Q.1(c-order), and the occurrence map preserves its old restriction. No extra edge is inferred from L, a logger, physical placement or full lexical process order. Q.1 outer-order reflection does not alone prove the reverse no-extra-order direction. C5 is a generator-schema binding obligation, not an assumption of equal future canonical histories. Make the retained complete-group occurrence map a bijection and every translated strict cross-group edge/path nonempty between distinct images. Bind J.GeneratorCompleteness and derive source joint order well-formedness through J.SourceOrderWF from reusable primitive/template realization proofs and their common-rank composition, or equivalent independent J.RequiredOrderWF evidence. Any additional generator lacking a realized forward-step/internal-order case is a qualification failure, not an optional omitted edge. J.SourceOrderReflection derives retained RTL group acyclicity on covered prefixes before canonical equality; separately cover any full proof-unit graph consumed by the instance. This makes the strictness justification explicit without adding a hardware scheduling or

observation rule. For a completion application using J.ConstructiveFactCorr, also establish its CF0–CF2 local fact laws and exact F-prefix coverage. C5 outer-history equality alone does not cover an erased internal fact or non-transfer result.

Lemma J.LocalBoundaryCompose (component certificates construct one source boundary segment).

Under C1–C3, the finite selected descriptor set has one jointly executable source L3 realization with exactly its stated effects. Proof. Apply J.SourceSettleExists, whose operational realization C2 supplies, from the exact related L1 endpoint. Its legal first fixed point supplies equal boundary values and the complete frontier activation before any descriptor is executed. Starting with U empty, choose an L-minimal remaining descriptor. Its participant certificates provide every source-rule premise from the current residues and frozen snapshot. Execute the compatible atomic rule and frame the other components; the successor-residue equations re-establish the induction invariant. Finiteness and acyclicity exhaust T and yield the implementation-matched final effects. No later state is fed back into a snapshot decision. This constructs joint executability rather than assuming it. □

Lemma J.SourceOrderReflection (source strictness precedes canonical equality).

Consider a finite legal source prefix already constructed by the operational local certificates, and the raw completed-unit/group graph of the corresponding implementation prefix. Establish the source graph's J.RequiredOrderWF evidence before invoking canonical-history equality, using J.GeneratorCompleteness and J.SourceOrderWF with reusable source primitive/template proofs, or equivalent independent full-graph evidence. A sufficient discharge indexes the actual source completion steps, proves every normative generator points forward between steps, and checks the internal generator graph of each co-completing step. One atomic group per action has no cross-group internal edge. Multiple R2C components may co-complete with no internal edges only after checking all selected sampler/other normative rules; absence of combinational-dependency or serialization edges alone is not the full check. Otherwise supply the internal rank. Compound pool handoffs and paired syncs retain their actual atomic groups. This evidence comes from source primitives and normative rules, not from assuming that every edge in the witness DAG L is canonical.
Assume the complete retained-group occurrence map is a bijection, hence injective, preserving whole groups and identities. Before treating either group graph as a partial order, translate every strict implementation group edge, including paths through erased support, to a nonempty normative source path between its distinct image groups. A hypothetical implementation cycle then maps to a nonempty closed source walk, contradicting the established source acyclicity. Thus implementation group acyclicity follows using only the implementation-to-source direction. No edge may be discharged by a zero-length path or by collapsing distinct retained groups. Reverse translation is needed subsequently for equality, not for this acyclicity argument. The result does not certify an erased-only implementation cycle or other unmapped proof support; where that larger graph is consumed its independent J.RequiredOrderWF evidence remains required. □

Lemma J.LocalOriginCanonExt (generator binding establishes equal extended canonical histories).

Under C5 and equal canonical pasts, a completed operationally constructed edge preserves the old history restrictions and establishes equal extended J.OuterCanonHist histories. Proof. First retain the actual legal source prefix supplied by the local boundary/update construction. Second establish source-side joint strictness by the completion-step/internal-graph check or independent J.RequiredOrderWF evidence; do not infer it from the desired equality. Third extend the complete-group bijection and apply J.SourceOrderReflection to its nonempty implementation-to-source path translations, obtaining

implementation group acyclicity. Finally translate generators/paths in both directions: complete group carriers, labels, internal atomic structure and required reachability agree after contracting erased support. Prefix closure prevents missing old predecessors; restriction stability and the fixed old map preserve the prior canonical history. These acyclic matched graphs therefore define equal canonical histories. The witness serialization itself adds no normative order. □

Theorem J.ConstructiveEdgeInduction (parameterized origin coverage).

Fix Core.ConstructiveOriginCarrier, an independently defined legal source relation, one intrinsic history function H on each side with the same observation/quotient, and a relation R on exact prefixes and finite causal proof data. Assume, for every prescribed implementation base κ0, a legal source prefix λ0 and finite causal proof data g0 with R(κ0,λ0,g0) and H_ref(λ0) = H_post(κ0); for every construction-reachable R-state and every actual next carrier edge, assume local rules derive a finite legal source extension retaining the exact old prefix and bound choices, restoring R at that actual successor (including designation transport), and establishing equality of the two extended H histories. Then every finite reachable implementation origin has a related legal source prefix with equal H. Proof. Induct on the actual finite edge ancestry. Use the supplied base; apply the universally quantified local extension to the actual next edge, then transport its target through the qualified designation. Retain the resulting exact source prefix for the next step. Induction covers every actual branch and produces no fairness or future-edge premise. □

The extension premise of this induction is discharged by C0–C5 through J.SourceSettleExists, J.LocalBoundaryCompose, legal simultaneous update/preparation and J.LocalOriginCanonExt. It is not a replacement axiom for those local proofs. The anchor, configuration, reset and R2C theorems instantiate the same schema with their explicitly enlarged unit inventory, primitive cases and history function. Joint state/root compatibility of combined contracts remains required; separate relations are not simply conjoined.

Theorem J.LG-Origin (local-to-global constructive origin coverage).

Assume J.LocalOriginContracts C0–C5 with its fixed instance, qualification, origin-family and finite-carrier premises. Then every actual implementation edge from a construction-reachable R-state has a finite legal source extension preserving the exact prior source prefix and bound choices, restoring the structural relation, and establishing equal extended canonical histories. Hence J.ReachableHistCov_I,E,Σ_DUT(OriginReach_I(O_I)) holds. Proof. C0 supplies common legal environment roots and the base. C1/C2 supply attributed roots and the legal operational settling of J.SourceSettleExists. J.LocalBoundaryCompose constructs the joint L3 segment after the correct frontier trigger. Close that boundary selection without adding actions from recomputed post-transfer signals. C4 installs results at L4 and concatenates the finite next-L1 words using their frames and independence; C0 supplies the compatible environment step. Their postconditions assemble the new structural relation, and J.LocalOriginCanonExt supplies canonical equality. All extensions retain the actual old source history and choices. Apply J.ConstructiveEdgeInduction over Core.ConstructiveOriginCarrier, including actual-successor transport across any designation. No future complete execution, fairness, progress or B1 is used. □

Scope of J.LG-Origin. This is a sufficient, design-independent theorem for the stated registered message profile, not a claim that every permitted HLS transformation automatically supplies C0–C5. It produces semantic reachable-history coverage and exact constructed prefix extension, not implementation progress or the full internal-fact interface without its additional qualification. Failure of a chosen residue/order construction is not alone a semantic noncoverage theorem. A source property must hold on the selected, possibly coupled and back-annotated, Sys_ref; it is not inherited from the original FIFO depths merely by this theorem. Accounting for a complete edge does not assert that every

implementation prefix is whole-edge: origin coverage uses Core.ConstructiveOriginCarrier alone; the exclusion of partial inventories and the lift to arbitrary finite prefixes remain the separate Batch hypotheses.

Lemma J.BatchPrefixCoverage (origin coverage lifts to exact Batch finite prefixes).

Assume Core.RegisteredMessageBatch_I with origin family O_I and J.ReachableHistCov_I,E,Σ_DUT(OriginReach_I(O_I)), under the same observation, occurrence and elaboration binding. Then J.ReachableHistCov_I,E,Σ_DUT(Reach_post(I)) holds for the declared Batch profile domain. Proof. Take any finite legal implementation prefix. If no edge has completed, BM3/BM4 give the covered initial history. Otherwise use its last already completed edge. BM1 places the whole inventory and registered update there; BM4 provides that realized state's origin, with at most finite history-preserving designation, and BM3 preserves its canonical history through the remaining silent prefix. Apply origin coverage and substitute equal histories. No proper partial inventory exists, no extra observable edge is selected and no unfinished cycle must be completed to obtain this witness. The lemma does not assert operational realization of every mathematical ideal. □
Assembly independence. Origin coverage may come from any valid proof of J.ReachableHistCov under these bindings. The Batch lift needs its actual carrier premises regardless of how origin correspondence was established.

Corollary J.LG-Batch-Safe (constructive exact-prefix safety for the Batch profile).

Bind J.LocalOriginContracts and Core.RegisteredMessageBatch to the same I and O_I. J.LG-Origin followed by J.BatchPrefixCoverage establishes J.ReachableHistCov on all finite reachable prefixes of that profile. J.SafeTransfer-HistCov therefore transfers any extensional Φ holding on every reachable source canonical history to that domain, without J.IdealPrefixClosed. Its selected-source-run alternative still requires the separately stated RefProjCover and explicit all-ideals property premise. Monitoring obeys the Batch complete-bucket contract. K.CT retains its separately stated domain and premises. Checkpoint specialization: if Core.SourceCheckpointProfile is also established, apply J.SourceCheckpointRepresentative to the constructed source witnesses and obtain ReachableHistCovOn(Reach_post,ChkRef). J.SafeTransfer-HistCovOn then permits universal safety only on ChkRef, or the selected-execution clause (2), with RefProjCoverOn and its all-finite-ideals property premise. This specialization neither assumes a future source cycle nor weakens the implementation Batch completion contract.

Constructive source-property discharge and monitor contract (normative).

Before claiming a constructive safety corollary, bind H, Φ and A=Reach_ref or the qualified ChkRef domain. Prove correctness of the actual configured source model: its capacities, explicit couplers/gates, choices, environment/testbench, reset and observation semantics must be the ones used by correspondence. No executable runtime is identified as automatically realizing this model. A flow records its executable model or formal representation and the proof connecting its explored/verified histories to all of A. A model that merely generates some legal Policy-L histories is insufficient for universal coverage. A sound overapproximation may be used if its property proof covers all H histories of A and respects the same observation interpretation.
Universal-proof path. An inductive invariant, exhaustive finite-state exploration or another sound source verification method proves Φ throughout the independently selected domain A. With ChkRef, bind monitor evaluation to declared initial/initialization or completed-cycle checkpoints and prove J.SourceCheckpointRepresentative; ordinary intra-L3 snapshots are not additional property obligations in this specialization. A passing finite sample campaign does not supply universal state/choice or stimulus-class coverage.

Selected-execution path. Bind a complete source execution and independently prove RefProjCoverOn(A,ρ^{o}). Blocking deterministic dataflow or NB-CF may supply determinacy ingredients, but none is an automatic covering-run theorem: prove occurrence/value, normative-order, environment/arbitration, capacity/coupling and completeness facts on this exact model. Establish the selected J.SafeTransfer-HistCovOn property clause. Direct all-ideals enumeration may be exponential for a finite history and is not a finite method for an arbitrary infinite history. A symbolic proof may establish the all-ideals premise without enumeration. A proof restricted to independently source-realizable checkpoint histories instead discharges universal-source clause (1), as explained by the evidence-organization remark; it is not a third covering-run alternative. Checking only ordinary prefixes of one execution is insufficient.

Profile binding. Message/pipeline evidence models legal preparation/issue latitude, pre-handshake arbitration, mandatory complete transfer sets, all explicit storage and resource admission, and operational NB decisions. StableRead evidence additionally proves the in-place reduction/lift at original checkpoints. Anchor/signal evidence realizes retained Read membership, complete last-Write groups, sample/resolution and START/FINISH phases. Configuration evidence uses the selected CanonHist_cfg records, governing transactions and every consumer's update isolation. Reset evidence implements the complete RBP/RB scope, priority, cancellation, storage, hold/release and initialization effects, with a defined cross-epoch protocol for partial scopes. R2C evidence implements participation, samplers, Core.R2CResetLedger and complete emission. Combined rows require one compatible model and invariant, not unrelated passing artifacts. A row without an executable/checkable realization is not claimed available to a simulation flow; a purely mathematical source proof remains a distinct evidence path.

Monitor example and extensionality. A transaction scoreboard may collect every completed group on one edge, merge the bucket into an occurrence-keyed request/response ledger, and evaluate once after the merge. For the history predicate "each retained response has one matching retained request and the specified value," define matching by identities and values reconstructible from H; processing an independent request and response in opposite logger orders must give the same result. Prove invariance under every ordering/timing representation erased by H, not only intra-edge permutations, and retain indivisible multi-effect groups. A requirement that the request occurred in an earlier physical cycle is not this predicate and needs a timing-sensitive theorem. If a needed request/response relation is not reconstructible from H, enrich the prospectively selected observation or use another property theorem; a logger cannot create a canonical edge. For a ChkRef source monitor, evaluate at the certified source checkpoints. These monitor conditions establish faithful property evaluation, not coverage of untested executions.

Definition J.StableDirectInputContracts (anchored stable-input lifting; SD0–SD4).

Fix an original source/implementation instance I_{θ} and its reduced registered-message body instance $\bar{I}_{\theta}$, with the same message boundaries, capacities/elaboration, environment placement, fixed choices and canonical message observation. θ is the typed configuration vector at the real preceding initialization synchronization anchor(s), fixed for the execution. The source may read the ports repeatedly or conditionally inside the body. The reduced body replaces each direct-input read expression by θ at the same expression position; no source initialization copy or physical configuration register is required. The following contracts permit use of the existing message theorem through a reduction and lifting proof; they do not make the original programme satisfy unchanged Core.RegisteredMessageBatch BM0 by declaration.

SD0 — Scope, initialization and stability. Supply legal matched finite initialization prefixes with genuine source synchronization anchors, before any retained message commit, and the corresponding body bases. The fixed Σ_{DUT} is message-only and erases the initial anchor/direct-input Read units as

complete groups, without erasing part of a group containing a retained message. There are no later synchronization boundaries, signal Writes, sync-controlled configuration updates, affecting resets, R2C processes or multiple-clock interactions. The actual environment keeps each configured input equal to θ from its relevant logical anchor through every permitted physical use, and source and implementation share that value. Any theorem for a set Θ of configurations quantifies over every θ∈Θ and every admitted execution; it may not reselect θ to match a later prefix. Initialization/environment behavior and the real anchors remain proof obligations, not invented program operations.

SD1 — Anchored samples and occurrence interpretation. Bind every relevant physical sample to its configured signal and θ value, including late or repeated samples affecting control, endpoint selection, payloads or registered updates. The Direct Inputs/E2 contract leaves the logical Read at its preceding synchronization anchor. Core.17 ΣMatch records at most one retained Read per process/signal/anchor interval; repeated expression evaluations do not create one retained Read per iteration, and physical samples create no additional logical Read. Availability of θ at the anchor is distinct from execution of a conditional source read: a branch not taken executes no read. Each actual local assignment or arithmetic use retains its own dynamic occurrence and writer.

SD2 — In-place reduction and source lifting. Replace only the value obtained by a direct-input read expression, preserving its control context, source preparation position, assignments and message operations. Do not hoist conditional evaluation, substitute later reads of mutable locals, or remove the source-evaluation/launch legality checks merely because an operand becomes constant. Prove that every reduced source tail and construction extension lifts from the same original source base by executing the original reached expressions with value θ; all original source step guards, Core.RegSeq and persisting Core.16 restrictions remain satisfied. The implementation reduction shall preserve its actual settled decisions, message edge inventories and simultaneous updates by substitution at every relevant sample. Stable configuration does not excuse stale token operands, unbound shadow registers or unstable pending payloads. Checkpoint specialization additionally proves that this lift preserves the declared original-source checkpoint predicate and its initialization histories; a reduced-body checkpoint cannot be identified with an original checkpoint without that proof.

SD3 — Uniform message-body evidence. Discharge J.LocalOriginContracts C0–C5 and Core.RegisteredMessageBatch for the reduced body family, with the same origin binding and actual environment choices, for every declared θ. Configuration-dependent branches and endpoints require their corresponding local proofs; one tested configuration is insufficient. Bind Q.SafetyQualified(E_θ) when elaborated and actual boundary/capacity faithfulness. Any parameter-dependent E_θ is fixed within its instance, not changed during the proof. The original body’s physical private computations remain subject to the existing invisibility, operand binding and isolation conditions.

SD4 — Canonical message projection and carrier binding. Bind the same message groups, occurrence keys and primitive/elaboration generator schemas on the original and reduced carriers. Retain the legal initial anchors and direct-input Read attribution in the original proof record. Check that their complete erasure supplies no required path from one body message to another: E2 gives read/anchor alignment, the genuine initial anchors precede body work, and neither arithmetic dependence nor full lexical process order adds a cross-interface required generator. There are no later signal/synchronization units in this profile to supply such a path. Any additional normative generator must be checked explicitly rather than assumed harmless because the Read is hidden. Cover zero, first and repeated executed reads. Original implementation prefixes are either in the message-empty initialization or map to the reduced Batch tail with equal canonical message history. No theorem about full signal-observing prefixes is inferred.

Lemma J.StableRead-Reduction (anchored substitution preserves message execution).

Under SD0–SD4, every permitted source read or implementation sample of configured signal s returns θ(s). Replacing its value in place preserves the containing expression and guard; induction over the finite settled graph preserves boundary decisions, and substitution into separately formed next states preserves registered updates. For a reduced source step, execute the original reached expression in its unchanged control/preparation position. SD2 retains every source launch/result-availability guard. The value equality preserves the step and local state, so induction gives a legal exact source-tail extension with the same message occurrences and values. A conditional read is not executed on an untaken path, and the logical anchor is not moved to a later physical sample. SD4 preserves message atomic groups and both directions of required-order reachability after whole-unit erasure; thus the original and reduced message canonical histories agree at their compared finite prefixes. This is a message-projection result, not a claim that an enriched signal-observation record is unchanged at every intermediate point. □

Theorem J.StableRead-Batch (stable direct-input finite-prefix message coverage).

Assume J.StableDirectInputContracts for the original instance I_θ and reduced body instance Ī_θ. Then J.ReachableHistCov_I_θ,E,Σ_DUT holds on all finite reachable implementation prefixes in this declared stable-direct-input profile. Proof. Fix θ and an actual prefix. A prefix confined to initialization has empty selected message history and a reachable initial source witness. Otherwise J.StableRead-Reduction maps its actual implementation body history to the reduced Batch instance. SD3 and J.LG-Origin/J.BatchPrefixCoverage supply a reachable reduced source witness. Lift that tail from the legally established original source base using J.StableRead-Reduction. SD4 equates its canonical message history with the original implementation prefix, while preserving the environment and previously bound choices. This covers every admitted prefix and, when claimed uniformly, every θ∈Θ. No future branch, cycle completion or fair execution is selected. □

Safety and integration scope. J.SafeTransfer-HistCov therefore transfers extensional properties valid on every reachable original source message history, without J.IdealPrefixClosed. The selected-source-run alternative retains RefProjCover and its explicit all-ideals property premise. The theorem is a stable-input lifting route around the existing registered-message result; BM0's base profile remains unchanged, and this safety extension does not discharge K.CT's additional completion obligations. Repeated or conditional in-body reads require no explicit source initialization copy. Later anchors, sync-controlled updates, observable signal histories, dynamic reset, R2C or multiple clocks require a further extension, not merely deletion of these scope conditions. With Core.SourceCheckpointProfile and the corresponding representative/projection proof, the sharper alternative uses Φ only on ChkRef and J.SafeTransfer-HistCovOn. Its selected-run form uses RefProjCoverOn and clause (2)'s all-finite-ideals property premise. Proving Φ directly on ChkRef instead uses clause (1); ordinary cycle-by-cycle simulation alone establishes neither universal source safety nor covering-execution evidence. For completion-facing fact correspondence, use J.StableReadFactCorr with its strengthened full-F reduction, matched initialization and exact-prefix obligations; the present message-only result is unchanged.

Definition J.LocalAnchorContracts (A0–A6; local correspondence with observable anchors).

Fix Core.RegisteredAnchorBatch, the chosen Sys_ref and one exact source-prefix relation R(κ,λ,g) over every construction-reachable paired origin and every actual implementation successor. Use the same environment, bound choices, occurrence/projection declaration and ordinary boundary qualification or Q.SafetyQualified(E). Extend the contents and local obligations C0–C5 of J.LocalOriginContracts to the following cases; their message equations, finite residue construction, source guard requirements and two-way canonical binding remain. This definition replaces that schema's message-only scope for this separately named theorem; it does not invoke the old theorem outside its domain. No global matching

continuation is assumed. The inherited contracts include Core.SourceChoiceBinding/Core.PreparationEvidence, J.SourceSettleRealization and Core.ConstructiveOriginCarrier transport. Any checkpoint-domain safety use additionally discharges Core.SourceCheckpointProfile for this enlarged observation, including initialization and selected START/FINISH; it is not inherited merely from the message-only case. The OA specialization of J.OrdinaryConstructiveComposition supplies these local construction obligations only within its explicit pre-update signal, primitive, initialization and ownership restrictions. It does not qualify every Core.RegisteredAnchorBatch sampling convention or START/FINISH convention.

A0 — Base and environment. Construct related legal bases with equal completed history, initialized logical signal state and genuine initial anchors. Bind all current signal and synchronization environment roots and provide legal source counterparts for actual environment steps, as in C0. A reactive environment is covered on every selected observation it consumes. Do not substitute future input values or a more cooperative environment. Ordinary signals may change between anchors; direct inputs use their specified stability/update contract.

A1 — Process and anchor descriptors. In addition to C1, relate reached/pending synchronization kind, owner/site/ordinal, transaction key, participants and readiness roots, plus actual pending source-write records and source operands. A selected anchor descriptor includes its complete RULE 1-qualified retained Read inventory, resolved final Writes, samples, effects and absence facts. Logical membership is distinct from later physical sampling. Every physical operand use is bound to the proper sample, source writer and token/interval identity. Preparation, private computation and activation obey Core.16 and Core.RegSeq. No prospective source statement is executed merely to guess a future branch. Relate the actual Core.SyncProtocol issue/held-offer state and SP4 epoch disposition, including any SP5 old-user pin protection; reached/pending typing alone does not supply these facts.

A2 — Settling and resource ownership. Extend C2's complete graph and ownership declaration to clock/wait eligibility, paired synchronization matching, signal drives/resolution and synchronization-dependent capacity withdrawal. One owned resource block represents each shared signal state and its joint update; readers share that state. Multiple drivers require the actual legal deterministic resolution rule and its joint certificate, or the instance excludes them. Every instance states the signal sampling/update convention. Related roots and local equations determine matching settled samples and coherent selected descriptors; simulator process order is never an implicit resolver. J.SourceSettleRealization covers all actual source L2 transitions of this enlarged network and every activation-relevant frontier predicate; selected-sample agreement alone is insufficient. The complete snapshot determines every mandatory synchronization completion under Core.SyncProtocol. Prove both that each such completion has its descriptor and that no sync descriptor is invented without its actual handshake/release. Protocol identity, fence support and reset disposition are qualification obligations on that snapshot, not filters applied after the handshake.

A3 — Local boundary realization. Extend C3's finite descriptor DAG and resource residues with unary wait and paired-sync primitive cases. For each minimal descriptor prove pending participants, readiness, source eligibility, the exact anchor effects and the Core.SyncFence completion premises. Earlier same-edge process-facing message completions may discharge the fence through the witness order; that order adds no canonical edge of its own. Anchor values and final Writes must already be legally available: a returned message value cannot supply a same-edge dependent source computation merely because its completion satisfies the fence. Signal blocks evaluate from the declared frozen snapshot and collect the prescribed joint update. The sync action retires the transaction once and crosses to the immediate post-sync control point without new body evaluation or Issue. Frame every other resource by its actual footprints. Apply the Core.SyncProtocol primitive: the descriptor realizes the entire mandatory settled transaction, not a chosen subset. The residue order only accounts for the same-edge

predecessors already justified by the complete snapshot; it cannot postpone the transaction or change its pins.
A4 — Simultaneous update and legal preparation. Extend C4 with the declared joint signal update, retained samples and anchor/interval records. Form next states separately from the common current state, with only proved forwarding, and publish simultaneously. Finite next-L1 source words use the bound samples, preserve true local writers and pending values, and obey strict post-sync Issue timing and result-availability rules. They introduce no new member into a completed anchor group. Their shared signal reads use established samples and their pending writes have owned frames, so local words compose without unproved cross-process preparation dependencies. Pending pipeline capacity follows the existing source synchronization rule and the declared elaboration; persistent FIFO contents and their accounting are not reset by sync.
A5 — Canonical translation. Extend C5 to every applicable E1 synchronization-order, E2 anchor/alignment and E5 interval-fence generator, and preserve complete group membership and internal structure. Match unary/pair identities and signal occurrences, in addition to message/FIFO/elaboration identities. Translate normative generators or certified paths in both directions, covering old and new predecessors and keeping the old occurrence map literally fixed. Preserve predecessor-before-completion and restriction stability. Contract hidden paths rather than erasing their order. Physical cycle coincidence, arithmetic provenance, source lexical order and witness serialization do not introduce new strict generators. Include the mandatory E5/Core.SyncFenceOrder blocking pref_S commit → sync edge at equal cycles, its internal form inside any qualified compound group, and paths through erased anchors. This is distinct from the strict-cycle sync → suff_S edge.
A6 — Inventory closure and carrier realization. Instantiate Lemma J.AnchorInventory at each selected anchor using the RULE 1 qualification and A1/A3 write records, and bind its descriptors to the actual source and implementation completed units. Establish AB1–AB3 and the completeness of the selected edge inventory. Read membership closure is derived from RULE 1, not an independent permission to choose a favorable inventory. Correct implementation realization, finite local evaluation and carrier binding remain proof obligations; source well-formedness alone does not prove transformation correctness. A completion application additionally binds the consumed anchor/message facts through J.ConstructiveFactEvidence and Core.RegisteredFactBatch; RULE 1 inventory closure does not by itself cover unrelated internal facts.

Lemma J.AnchorInventory (causal complete anchor groups).

Under RULE 1 retained-Read qualification and the applicable finite source preparation, each selected (process, signal, anchor) Read key has an absence/presence-and-value descriptor determined at the anchor. By qualification this descriptor agrees with the source semantics on every admitted continuation, so its membership and value never change. For Writes, fold the actually executed source Writes sharing the succeeding anchor in source order, retaining the last writer/value; untaken Writes do not participate. Before that real anchor completes, the fold is private and contributes no observable Write. At a legally selected anchor the fold is complete, and no later source Write can join that previous interval. Combining these descriptors with the already determined sync participants yields a finite complete immutable anchor unit. Later physical samples and re-drives are internal and cannot add members. This proves inventory closure without predicting a future message outcome or adding a virtual anchor. A source whose retained Read presence is first determined by a later message-dependent branch fails the clarified RULE 1 qualification for that observation. □

Lemma J.AnchorBoundaryCompose (joint message, signal and synchronization realization).

Under C1–C3 as extended by A1–A3, execute the finite residue induction of J.LocalBoundaryCompose over the enlarged descriptor DAG. Message cases use their existing primitive certificates. A unary wait

uses its actual clock eligibility; a paired sync uses its matched issued and held endpoint offers and mandatory sampled handshake under Core.SyncProtocol. The A2 inventory includes every such transaction, and unary completion obeys SP6. The fence predecessors have already completed in the residue when needed; the anchor rule appends exactly the complete unit from J.AnchorInventory and advances no next-interval work. Shared signal readers consume the declared snapshot, while writer effects belong to the single certified joint update. Their frames therefore do not depend on process visitation order. Minimal-descriptor induction preserves all component residues and constructs one legal source boundary segment with exactly the selected effects. No K release-family or progress premise is consumed. □

Theorem J.LG-AnchorBatch (exact finite-prefix coverage for observable waits and signals).

Assume J.LocalAnchorContracts A0–A6 with Core.RegisteredAnchorBatch and the fixed source/boundary qualification. Then every actual edge from a related reachable origin has a finite legal source extension preserving the exact past and environment choices, restoring the structural relation and establishing equal extended canonical histories; J.ReachableHistCov_I,E,Σ_DUT holds on all finite reachable implementation prefixes in this profile. Proof. A0 supplies the base and common current environment. A1/A2 supply matched roots, samples, settling and selected descriptors. J.AnchorInventory fixes complete anchor groups, and J.AnchorBoundaryCompose realizes the joint boundary. A4 supplies legal simultaneous updates and finitely composable next preparation, restoring R. A5 extends the occurrence map; translate all generators in both directions and contract erased intermediates as in J.LocalOriginCanonExt. Fixed group membership and restriction stability preserve the old canonical history and establish equality of the extension. Induct over the actual finite edge sequence. By AB1–AB3 an arbitrary finite implementation prefix has the completed history of its covered base or last completed origin, so that already constructed source witness supplies coverage. No future edge, fairness or ideal-prefix-closure premise is used. □ Explicit induction interface: instantiate J.ConstructiveEdgeInduction with A0 bases, J.AnchorInventory and J.AnchorBoundaryCompose boundary cases, A4 preparation, and A5 complete generator translation. Bind actual-successor designation transport and the enlarged whole-edge prefix contract separately. If Core.SourceCheckpointProfile holds for this history, J.SourceCheckpointRepresentative also gives ReachableHistCovOn with source domain ChkRef.

Corollary J.LG-AnchorBatch-Safe (universal source safety transfer).

Under J.LG-AnchorBatch, every extensional predicate Φ valid on all reachable source canonical histories at the same selected observation holds on every finite implementation prefix, by J.SafeTransfer-HistCov. The source is the selected Sys_ref, including its back-annotated storage, couplers and existing synchronization-dependent capacity withdrawal; validity on another ungated or differently sized model is not assumed. No particular controller, extra drain edge or globally empty output FIFO is required. The theorem is sufficient local evidence, not automatic for all HLS transformations, and does not establish completion conformance. Dynamic reset is covered only by the separately qualified J.LG-ResetBatch extension below; R2C is covered only by the separately qualified J.LG-R2CBatch extension; multiple clocks remain outside these profiles. Synchronization forms other than unary waits and pairwise SyncChannel/ac_sync transactions, which the current rules do not define, are outside this profile. Ordinary multi-process patterns built from supported transactions still require their joint local evidence. Raw timing assertions remain outside the canonical property domain unless represented by the selected observation/order contract. With Core.SourceCheckpointProfile and the corresponding representative/projection proof, the sharper alternative uses Φ only on ChkRef and J.SafeTransfer-HistCovOn. Its selected-run form uses RefProjCoverOn and clause (2)’s all-finite-ideals property premise.

Proving Φ directly on ChkRef instead uses clause (1); ordinary cycle-by-cycle simulation alone establishes neither universal source safety nor covering-execution evidence.

Lemma J.AnchorDirectInputLift (retained direct inputs compose with observable anchors).

Suppose A0–A6 hold except that process sample/value equations are discharged by direct-input reduction. At each real anchor a bind the permitted configuration θ_a from the actual environment and bind the complete retained logical Read descriptor by RULE 1. Retain that descriptor inside the complete synchronization/anchor unit; do not erase its Read members. Prove that every physical sample used for the corresponding source interval equals θ_a, under either reset-epoch stability or the designated sync-controlled update protocol, and that operand/token/payload isolation preserves earlier work when configuration changes. The environment may present new values with sync-valid before the commit edge after the required readiness; therefore old users must either be finished or provably isolated from the changed pins. Existing Core.SyncFence and capacity-withdrawal requirements remain in force, without mandating a particular ready-state controller. Core.SyncProtocol SP5 makes this protection a prerequisite before the ready offer authorizes changing pins, and maintains it throughout the pending interval and any later old-value use; SP2–SP4 bind the held offer, mandatory handshake and reset case. Replace direct-input expression values in place by θ_a within each interval, preserving source control, actual executed statements, source launch legality and writer identities; retain the same anchor descriptor as part of the reduced comparison rather than deleting it. At a sync-controlled update, bind θ only from that actual handshake, retain residual channels/history, and permit next-interval Issue only after commit. Value equality proves each local expression/guard and separately formed next-state equation unchanged; induction on the local settling graph preserves decisions. Anchor descriptors are identical by qualification, not inferred from value equality. Therefore the reduced local evidence supplies the corresponding A1–A4 facts of the original programme, while A5/A6 preserve all retained groups and generators, and J.LG-AnchorBatch applies to the original full observation. No source initialization copy or dedicated physical configuration register is generally required. □

Direct-input observation boundary. This lemma permits conditional or repeated physical use, and conditional source evaluations whose zero-or-one retained Read descriptor is already anchor-determined. It does not admit a newly observable Read whose presence is first decided by a future message. Nor does it replace an absent logical Read by a speculative always-present port observation. J.StableRead-Batch remains a different, message-only conclusion with whole erased anchor groups and no later sync; its substitution proof alone does not license selective deletion of a Read from a retained sync unit. If the selected observation erases a complete anchor group, the existing projection/order obligations apply. Observing a sync does not alone order an unrelated upstream producer’s input Push after it; frame-separation properties require the relevant declared causal observation/order relationship. This paragraph describes the retained-Read option of J.AnchorDirectInputLift. Core.AnchorConfigObservation and J.LG-ConfigBatch below provide the distinct transaction-observation option, including body reads after a blocking Pop; that option constructs its groups from the outset rather than partially erasing these Read-bearing groups.

Example J.AnchorBatchPair (symbolic local discharge with changing inputs and message stalls).

Consider two sequential processes P and Q, a rendezvous message channel c from P to Q, a paired sync g, an ordinary single-writer signal link initialized to L, an ordinary external input v, and a sync-controlled direct input cfg. Declare current-value signal sampling: every edge reader samples σ, selected writes are published together into σ’. External inputs are fixed for each selected snapshot; v may change afterward. At initial real unary waits, P reads a=v and k=cfg, and Q reads link=L. P computes y=(a+k) modulo 256, executes link.write(y+1 modulo 256) followed by link.write(y), and issues c.Push(y), then reaches g. Both Writes have g as their successor anchor. Q may take a fixed finite sequence of real waits

before issuing c.Pop(), then reaches g. Immediately after g, P reads the newly permitted cfg value and Q reads link; Q then takes a real wait h and reads link again. No process besides P writes link. Configured values remain stable as required, and old work uses the captured k or its isolated computed payload before the environment may update cfg. All Read presence in this example is unconditional at its stated anchor.

The relation carries σ(link)=L until g, P's stored a,k,y and pending last-write record (P,g,link,y), the stable pending c payload, both dynamic sync keys, and each bound sample. Source and implementation use the same snapshot equations and independent local records. A0/A1 follow from equal initialized resources and actual anchor samples, regardless of later v changes. A2 settles the single rendezvous, pair readiness and snapshot reads without feedback. Until Q requests c, the Push persists with the same owner/payload; no Write is yet observable. When both requests are present, the unique c transfer returns y. Select g only after both source endpoints have legally reached it and the fence is satisfied. A3 orders that process-facing transfer before g if both are selected on one edge; otherwise its earlier retirement already supplies the fence. Returned y supplies no dependent same-edge anchored computation in Q. This covers every permitted message-stall duration and legal selection, not merely one schedule.

At g, the two pending Writes fold to one retained Write(link,y); Q's same-edge Read(link,L) uses the old σ, and P's direct-input Read binds the newly presented θ_g. The simultaneous update installs σ′(link)=y and the post-sync states. Q's later h Read is y. A4 follows because both processes prepare from their bound samples, pending writes are owned, and future requests are later than g. Reversing the process evaluation order cannot change L to y in Q's g sample. A5 maps the same wait/pair occurrences, E2 members, E5 message fence and rendezvous group in both directions; A6 follows from J.AnchorInventory and the declared edge carrier. Thus the local contracts discharge J.LG-AnchorBatch for this symbolic family. The example is informative paper evidence, not a claim of mechanized or exhaustive RTL checking. It requires no new pipeline scheduling restriction, and analogous pipeline instances retain their separately proved resource certificates.

Definition J.ConfigValueContracts (CVG0–CVG3; transaction-bound configuration uses).

CVG0 — Initialization and transaction binding. Supply CO0–CO4 and legal related bases under the same actual environment choices. Establish each CO1 initial anchor/value, CO2 transaction and governing-record function from current control/history. Transactions reset only their configuration/interval bookkeeping; pending channel contents, global channel ordinals and completed histories persist. Qualify ordinary resources or Q.SafetyQualified(E), including the existing synchronization-dependent withdrawal of next-interval pipeline capacity.

CVG1 — Every use. Every executed source expression reading s∈D and every implementation sample that can influence covered behavior obtains θ_g(s) for its legally governing record g. Cover control, payload, endpoint, count/loop-bound, shadow-register and private-candidate uses. Source expressions may occur conditionally, repeatedly, or after blocking operations; their occurrence need not be decided at g because it creates no observable process Read in this option. Preserve the actual local evaluation and writer identity, Core.RegSeq and Core.16 launch legality. Value availability never authorizes an otherwise forbidden evaluation or Issue.

CVG2 — Stability and update isolation. Discharge the existing reset-stable or sync-controlled environment contract through every relevant physical use. When the environment may present θ_new before the update edge, prove that old work no longer consumes changed pins, or that its operands and payloads are retained and isolated so every old use still equals its old bound value. Core.SyncFence, admitted-work drain and next-interval capacity withdrawal remain applicable. No readiness schedule, additional drain edge or globally empty output FIFO is prescribed. A retained old output payload may

drain later. An unbound live sample, reused payload slot or stale shadow value fails qualification even when the configuration labels match.
CVG3 — Uniform local evidence and record realization. Establish the C0–C5/A0–A6 local equations and residues for the actual reachable residual states and all admitted configuration/environment choices, using the observation-specific inventory of CO3/CO4. In local value proofs, substitute θ_g in place for each designated input use; preserve code control and source attribution and retain the same transaction labels. A reduced proof does not erase the config records or restart at empty channels. Prove that the physical commit and initialization mapping emit exactly the corresponding records and that no late physical sample emits an additional unit. This supplies implementation realization, not merely a favorable chosen trace.

Lemma J.ConfigInventoryAndValues (causal records and local substitution).

Under CO0–CO4 and CVG0–CVG3, the configuration portion of each completed anchor inventory is determined causally: CO1 fixes its reader/real-anchor key and initialized vector, while CO2 fixes the field set and vector from the actual committed transaction. Record presence is independent of later message completion or source read occurrence. Thus each record and group is complete and immutable at its edge; ordinary Read membership and final Writes use J.AnchorInventory. CVG1 gives equal values at every actual source use and implementation sample. Substitution in unchanged expressions preserves guards and results; induction over the finite local settling equations and separately formed next states preserves selected decisions and updates. CVG2 preserves binding through old-work drain and future configuration changes. Lift local source words by executing their original expressions with these values, retaining every source guard and writer. No input read is invented on an unexecuted branch, and no future transaction is used. □

Theorem J.LG-ConfigBatch (configuration-bearing finite-prefix safety coverage).

Assume the selected Core.RegisteredAnchorBatch completion carrier with the CO4 observation-specific inventory, J.ConfigValueContracts and J.LocalAnchorContracts for ordinary signal/message/synchronization cases. Then, for every finite reachable implementation prefix κ, there is a reachable source prefix λ under the same instance/environment choices with CanonHist_cfg(κ)=CanonHist_cfg(λ). Equivalently the semantic J.ReachableHistCov condition holds for Σ_cfg. Proof. J.ConfigInventoryAndValues supplies the configuration descriptor and local value cases in the induction of J.LG-AnchorBatch. A0 gives related legal bases; each actual edge has matching complete records and roots, the joint residue proof constructs its legal source boundary actions, and A4 composes its simultaneous update and finite preparation. A5 translates existing normative generators in both directions; configuration attributes match and add no order. Ordinary anchors retain J.AnchorInventory, while CO1/CO2 replace only the designated-configuration membership case. Induction gives one prefix-preserving source construction. The selected whole-edge carrier reduces every finite prefix to a covered initial or last-completed origin. This is the AnchorBatch proof instantiated with an explicitly extended observation interface, not an invocation of the unchanged Read-based theorem on incompatible labels. Universal extensional source properties on CanonHist_cfg transfer by the semantic history-coverage argument of J.SafeTransfer-HistCov. □ Explicit induction interface: instantiate J.ConstructiveEdgeInduction with J.ConfigInventoryAndValues plus the A0–A6 cases, with CanonHist_cfg as H. Bind actual-successor designation transport and the enlarged whole-edge prefix contract separately. If Core.SourceCheckpointProfile holds for this history, J.SourceCheckpointRepresentative also gives ReachableHistCovOn with source domain ChkRef.
Scope and property use. The theorem covers both reset-stable inputs in designs with later observed waits/syncs and repeated sync-controlled updates, including pipelined body reads after a blocking Pop and zero-length intervals. A completed configuration record does not assert that any body read or

message eventually occurs. Dynamic reset remains excluded from this theorem itself and is handled by the separately qualified J.LG-ResetBatch extension. R2C, multiple clocks, and synchronization forms other than unary waits and pairwise SyncChannel/ac_sync transactions, which the current rules do not define, remain outside this profile. Configuration-to-output properties are expressible when the governing record and relevant input/output identities are determined by the selected history and existing E1/E5 order; their truth remains a source-property proof, not a consequence of observing θ. Hidden operands may prevent a particular functional relation being expressible. No frame-separation order is inferred for independent upstream producers. J.ConfigRecordFactCorr separately specializes the full-fact adapter for this observation, including governing-record attribution. It adds no completion conclusion to the present safety theorem.

Lemma J.ConfigWholeGroupProjection (message projection with contracted anchor paths).

Fix one whole-group-respecting projection P_msg on CanonHist_cfg that retains exactly the declared complete message groups and erases the remaining complete groups, including every erased sync's configuration attributes and ordinary signal members together. Require that no indivisible retained message group also has a non-message constituent that would need removal. Define the projected order by reachability through the original required-order graph, contracting erased intermediate units; retain any erased proof support needed to compute that order. Use the same projection and operational Sys_ref on both sides. If full configuration histories agree, their projected group carriers, identities, labels and path-induced orders agree. This follows by applying the full-history occurrence isomorphism to each retained group and each finite path in both directions. E5 paths through erased syncs therefore survive, including paths whose endpoints lie in different intervals. Prefix stability follows from the inherited predecessor-before-completion and restriction-stability conditions; erasure cannot retrospectively add or remove an order between old retained groups, with the two directions supplied by bidirectional restriction stability and predecessor closure. The result is a projection of the same reachable execution, not a new ungated FIFO system or an assertion that ordinary lexical order becomes a generator. □

Corollary J.ConfigBatch-MessageSafe (message safety across configuration intervals).

Under J.LG-ConfigBatch and J.ConfigWholeGroupProjection, every finite implementation message history has an equal projected reachable source message history. Hence every extensional property valid on all such source histories transfers without ideal-prefix closure. Proof. Select the full source witness supplied by J.LG-ConfigBatch for the actual prefix and apply the projection lemma; source reachability and the exact environment are unchanged. □ With Core.SourceCheckpointProfile and the corresponding representative/projection proof, the sharper alternative uses Φ only on ChkRef and J.SafeTransfer-HistCovOn. Its selected-run form uses RefProjCoverOn and clause (2)'s all-finite-ideals property premise. Proving Φ directly on ChkRef instead uses clause (1); ordinary cycle-by-cycle simulation alone establishes neither universal source safety nor covering-execution evidence.
Relationship of message-only results. Where both sets of qualifications and the same projected message observation are established, J.ConfigBatch-MessageSafe supplies the reset-stable message-safety conclusion of J.StableRead-Batch and additionally supports later configuration synchronizations. J.StableRead-Batch remains available under its narrower message-only reduction interface; its premises do not automatically establish the configuration-observation contracts.
Configuration examples. In sync; for (k=0; k<count.read(); ++k) { x=in.Pop(); y=f(x,mask.read()); out.Push(y); }, designate both count and mask in D as configuration signals governed by transaction κ of that sync, with a finite nonnegative count N=θ_κ(count) and mask value θ_κ(mask). CVG1 covers each executed loop-bound use of count as well as every executed body use of mask. The group records both fields at commit whether N=0, a Pop blocks forever, or later data selects a branch that never reads

mask. No observable process Read(count) or Read(mask) is generated in this option; actual expression evaluations retain their local writers and source legality. In particular, the zero-length interval still has a complete configuration record even though no loop-body read executes. If mask is reset-stable and P and Q have distinct real initial anchors a_P and a_Q, those groups each record their designated mask field; initialization proves equal values. Subsequent waits do not create new reset-stable configuration records, and each use in either process binds to that process's initial record. A sync-controlled paired update instead contributes one transaction record even when both endpoints read the designated fields. A change of pins during an old unisolated computation remains a CVG2 failure. These are symbolic applications, not executable or tool-specific certification.

Definition J.LocalResetContracts (RB0–RB5; reset extension of local constructive evidence).

Fix Core.RegisteredResetBatch, the same environment/source/elaboration and one exact prefix relation over all reachable origins, including reset-hold and partial-initialization states. Ordinary active components satisfy the applicable C0–C5/A0–A6/configuration local cases. The fields below add the reset cases, with no assumption of a globally matching continuation. Earlier reset-free theorem statements are not invoked on an unqualified reset-containing execution. The inherited contracts include Core.SourceChoiceBinding/Core.PreparationEvidence, J.SourceSettleRealization and Core.ConstructiveOriginCarrier transport. Any checkpoint-domain safety use additionally discharges Core.SourceCheckpointProfile for this enlarged observation, including initialization and selected START/FINISH; it is not inherited merely from the message-only case.

RB0 — Actual reset choices and holds. Match the selected edge-sampled reset inputs, scope, occurrence key and prescribed priority/hold/release behavior under the same environment choices. Bind every emitted reset to its actual occurrence; no future reset is anticipated and no pulse or repeated occurrence is silently omitted. On a declared hold edge, affected processes/storage stay at their prescribed reset state with no affected ordinary work; unaffected components use their ordinary local certificates. A finite prefix may stop during reset or initialization, so neither eventual release nor eventual START is assumed.

RB1 — Scope and footprints. Supply RW1–RW5 evidence for every owned affected process, channel, allocation pool, request, activation, synchronization record and signal/configuration state. Include every physical valid/credit field that could expose discarded work. Resetting shared accounting views operates on one represented storage block. Prove frames for unaffected owned state and channel history. Any cross-scope effect has its explicit modeled action and local certificate. Settled complementary signals may change when reset changes their roots; the proof uses the complete declared network and the appropriate phase rather than claiming that all outside ready/valid levels remain unchanged.

RB2 — Ending actions and cancellation. Certify actual selected ending-epoch descriptors from their prescribed snapshot and realize them before the affected reset. Then clear still-outstanding affected requests by RW2 without a fabricated transfer or return and without reattributing the call across reset. A completed pre-reset NB decision retains its actual outcome/retirement; a canceled unperformed attempt is not relabeled as a failed poll. Discard private pending Writes whose real succeeding anchor did not commit, without emitting ordinary Write events. Canceled sync endpoints obey the declared transaction disposition. Previously completed units remain historical observations. Core.TransferLaw/Core.SyncProtocol determine the full transfer-eligible-edge inventory before any declared reset effect. A reset-only edge follows its prospectively bound suppression convention; the certificate cannot erase an already sampled committed unit.

RB3 — Reset state and epoch update. Prove the local reset equations establish the prescribed source reset state and its implementation relation: affected control/visible values, reset-empty storage, cleared requests and stale sync state, and invalid old private candidates/activations. Unaffected state is framed. Advance occurrence coordinates by Core.ResetEpochAdvance using matched logical ResetKeys, not

arbitrary independent counters. New operations and writers use fresh epoch-qualified identities; old records remain only as historical facts and cannot later complete under a new key. Private physical data bits need not be cleared if proved invalid and isolated; source-visible reset values must correspond.
RB4 — Fresh START, preparation and configuration. Cover the actual post-release initialization steps as they occur. The first ordinary observable action of each affected process in its new epoch is the required fresh START_P; RESET itself is the earlier state-changing boundary unit. A reset that interrupts initialization is handled by its own reset case without assuming the unfinished initialization completes. Subsequent local preparation is finite and legal when invoked, obeys Core.RegSeq/Core.16 and does not reuse canceled operands. Re-establish ordinary samples and every governing configuration binding before use according to J.ResetConfigurationEpoch. Hold/release and finite-prefix coverage do not require fairness or an eventual new transaction. Retain the actual phase placement specified by ResetReleaseInit. Selected START and any associated complete anchor group cannot be hidden inside a tail claimed to be history-preserving. A checkpoint specialization invokes SCP3 explicitly; without it retain the universal-Reach_ref property premise.
RB5 — Canonical and carrier binding. Match complete RESET scopes, keys, START units and new epoch-qualified occurrences, along with unaffected old identities. Extend the applicable canonical generator translation with the existing strict ending-epoch → RESET → new-epoch generators, including selected same-edge pre-reset commits. Preserve old group membership, predecessor-before-completion and restriction stability. Unaffected groups gain no order merely by sharing a reset edge. Bind this inventory to the actual implementation whole-edge completion carrier, source primitive effects, origin designation and Core.CycleResult.resetUnits. A source witness's serialization realizes the required atomic/order rules without inventing physical sub-cycle observations.

Lemma J.ResetConfigurationEpoch (configuration rebinding across affected reset epochs).

For the retained-Read observation, re-establish the actual new-epoch anchors and their RULE 1 membership/value bindings. For Core.AnchorConfigObservation, interpret CO1's reset-stable initial anchors per affected reset epoch: each covered reader has a fresh real post-release initial anchor before its first use, and shared designated signals have the same fixed value at every such anchor. Add the canonical epoch identity to its initial-record key. Distinct reader anchors remain distinct groups except for their existing atomic transaction grouping. A pin shared with an unreset reader cannot change in violation of that reader's continuing stability contract; resetting one consumer does not authorize such a change.
For CO2 sync-controlled configuration, obtain a new legal governing transaction before new-epoch use unless an explicit retained-configuration state mapping proves continued legal binding, stability and isolation across this reset. Clear stale pending transactions according to RW3 and the paired-endpoint disposition; a new endpoint cannot silently match an old peer instance. Historical config records are never edited. Under RB3/RB4 and the ordinary CVG value/isolation conditions, each subsequent actual use is bound to the proper epoch/transaction and value. Proof. Reset invalidates precisely the affected bindings and candidates named by RB3. Legal initialization or the explicitly justified retained-state mapping supplies the new governing record before use, and the same CVG1/CVG2 local substitution argument preserves every executed value and writer thereafter. Nothing asserts that initialization or a blocked use must eventually execute. □

Lemma J.ResetLocalCompose (one reset-containing edge).

Under the applicable local evidence and RB0–RB5, a related origin and an actual reset-containing edge admit a legal source extension with exactly the same completed inventory and reset effects. Proof. From the declared snapshot, execute certified ending-epoch descriptors in their finite residue order. Apply each selected RESET as its RW1–RW5 atomic transition; RB1/RB3 establish owned equations and frames,

while RB2 distinguishes committed work from canceled outstanding instances. Disjoint ordinary/reset actions commute only by their proved footprints; overlapping reset occurrences follow the prescribed order. Core.ResetEpochAdvance assigns the same fresh logical coordinates. Publish the combined next state simultaneously and apply only the legal initialization/preparation actually covered by RB4. RB5 extends the occurrence map and both directions of required-order paths. The old prefix is retained, completed groups agree and the successor relation holds. Reset-hold edges use their declared hold/frame equations and do not manufacture new RESET occurrences. □

Theorem J.LG-ResetBatch (finite-prefix constructive coverage with dynamic reset).

Assume Core.RegisteredResetBatch, the applicable ordinary local contracts, J.LocalResetContracts and ordinary boundary qualification or Q.SafetyQualified(E), for every construction-reachable origin and actual successor under the fixed environment choices. Then every finite reachable implementation prefix has a reachable source prefix with the same selected canonical history, including the retained reset/initialization/configuration observations. Proof. Start from the qualified base. Use the corresponding message/anchor/configuration boundary-composition proof on ordinary edges; use J.ResetLocalCompose on reset edges and the declared hold/frame case while reset is held. RB4 covers actual initialization prefixes and J.ResetConfigurationEpoch supplies new sample bindings. Induction on completed edges preserves the exact constructed source past, restores the relation and establishes equal extended canonical histories. The whole-edge carrier maps any remaining finite prefix to the covered initial or last-completed origin. No future release, completion, fairness or K reset disposition is consumed. For the configuration observation, this is the same route-local matching extension as J.LG-ConfigBatch, with additional reset/epoch cases; this configuration observation is route-local and does not redefine other observation profiles. □ Explicit induction interface: instantiate J.ConstructiveEdgeInduction with ordinary composition, J.ResetLocalCompose, RB hold/initialization cases and reset-aware generator translation. Bind actual-successor designation transport and the enlarged whole-edge prefix contract separately. If Core.SourceCheckpointProfile holds for this history, J.SourceCheckpointRepresentative also gives ReachableHistCovOn with source domain ChkRef.

Corollary J.LG-ResetBatch-Safe (reset-aware source safety).

Under J.LG-ResetBatch, every extensional property valid on all reachable canonical histories of the same reset-aware source model holds on all finite implementation prefixes, by semantic history coverage. Validity proved only for reset-free source executions is insufficient. A reset-aware occupancy property uses the appropriate channel-epoch balance; global Push-minus-Pop counts need not equal current occupancy after permitted discards. This theorem supplies no completion result or preservation of outstanding workload obligations across reset. It leaves R2C and multiple clocks outside scope and makes no assertion about asynchronous reset implementation qualification. With Core.SourceCheckpointProfile and the corresponding representative/projection proof, the sharper alternative uses Φ only on ChkRef and J.SafeTransfer-HistCovOn. Its selected-run form uses RefProjCoverOn and clause (2)'s all-finite-ideals property premise. Proving Φ directly on ChkRef instead uses clause (1); ordinary cycle-by-cycle simulation alone establishes neither universal source safety nor covering-execution evidence.

Example J.BlockedPipelineReset (symbolic reset-priority instance).

Use the previously qualified two-cell Q/S pool, output FIFO F of capacity C>0, producer U, processing process P and receiver V. Reset the closed island {U,P,V} with all corresponding explicit channels and shared storage under the actual E ownership map; a P-only reset across live U/V channels instead needs the RW1 cross-epoch protocol. At a reachable origin q=o=1 and n=C, with V not requesting, P's old output is blocked and d=m=a=0. For C=1 this origin is reached from empty by A_o; then M_o,A_1; then

D_o,M_1,A_2, leaving the FIFO with token 0, S with token 1 and Q with token 2. Any newly issued U request is still outstanding and untransferred.
Declare one RESET on sampled entry, affected transfer suppression on that edge, and reset-state hold until sampled release. The simultaneous reset equations are q′=o′=0, F′=[], prescribed reset control/visible values, cleared affected requests/syncs/activation and invalid private candidate state. Reset the one shared allocation, not separate Q/S copies. No Pop explains discarded storage and no Push/return is fabricated for a canceled request. Global history retains its three accepted inputs and one old output send in the C=1 case and appends RESET; fresh channel-epoch balances are zero. RW3 supplies the same source state, RB2 cancels exactly the uncommitted calls, and ResetEpochAdvance gives matching fresh identities. These equations apply to every related blocked state and payload for any C>0, independent of whether f is injective.
An independent unaffected island may transfer on that edge using its disjoint owned resource certificates. Subsequent actual START and initialization follow RB4, rebind configuration and cannot reuse canceled tokens/writers. This symbolically discharges the reset and locality cases; the ordinary pipeline cases retain their existing proofs. It is not an executable campaign or universal reset-scope certification. A commits-before-reset implementation must instead certify and retain its actual ending commits; it cannot be modeled by this reset-priority example merely for convenience.

Definition J.LocalR2CContracts (RC0–RC6 for edge observation).

Use one common construction-reachable relation, environment choice and source/elaboration binding for all selected local contracts. Extend that relation with the component observation ledgers and epoch provenance. The following obligations range over every related reachable origin and actual successor, not only sampled executions. They extend the applicable C0–C5, A0–A6, CVG0–CVG3 and RB0–RB5 cases; independently proved relations cannot simply be conjoined without proving their shared roots, state and effects agree. The inherited contracts include Core.SourceChoiceBinding/Core.PreparationEvidence, J.SourceSettleRealization and Core.ConstructiveOriginCarrier transport. Any checkpoint-domain safety use additionally discharges Core.SourceCheckpointProfile for this enlarged observation, including initialization and selected START/FINISH; it is not inherited merely from the message-only case.
RC0 — Components and roots. Bind the ERB0 declaration and initial ledgers on both sides, including actual participation and sampler predicates, reset ownership and every transitive input root/value/provenance. Supply the R2C multi-input well-formedness evidence: one explicit anchoring domain for the slice or the required interleaving-invariance proof, or the separately qualified cycle-locked boundary. Equality of combinational functions alone supplies none of this observation qualification.
RC1 — Joint settlement. Give finite deterministic evaluation equations and a finite acyclic dependency graph for the whole combined network after cutting at explicit registered/storage state. Include sequential drives, R2C logic, signal resolution and channel/elaboration gates, and prove root agreement and selected fixed-point value agreement by local equations along the graph. Per-component acyclicity does not establish joint acyclicity. This sufficient construction excludes combinational feedback even where another unique-fixed-point proof might exist under the general R2C rules. Supply J.SourceSettleRealization for this graph and its operational source L2 relation; J.SourceSettleExists provides the legal first fixed point and full frontier activation.
RC2 — Exact inventory. For an ordinary component with current vector v, derive emit iff participating ∧ (¬seen_C ∨ v ≠ last_C ∨ fresh_sampler). For a cycle-locked component derive emit iff participating. Prove both mandatory emission and absence of forbidden extra units. Each emitted unit contains exactly its declared labels and values from μ_t; all its members are indivisible. Advance its ledger exactly

once. Apply Core.R2CResetLedger after any ending-epoch emission when an affecting reset occurs. A sampler cannot produce a unit while participation is false.
RC3 — Legal source observation and frames. Exhibit the legal fixed-point observation primitive at the settled snapshot and its effects on the observation history/ledger. It does not advance process control, alter channel storage, change registered roots, initiate another boundary selection or complete a process message. Prove its frames against the remaining owned resource actions. A stateful observer has separately modeled state transitions and qualified triggers; it cannot hide state in an allegedly pure R2C node. The resulting ordinary/anchor/reset boundary sequence remains jointly executable from the same frozen snapshot under the local residue certificates.
RC4 — Consumers and configuration. Establish every actual sequential operand/sample from its declared current snapshot/anchor and retain it through subsequent stalls; all registered next-state values publish simultaneously and cannot feed back into μ_t. Under Core.AnchorConfigObservation, a designated configuration signal contributes no observable process-Read label, including in a selected R2C slice; fix a compatible slice from the outset, without deleting members of completed groups. Ordinary labels and selected computed outputs remain eligible. Every actual use still satisfies CVG value and isolation obligations. In particular, continuously observed logic that sees changed pins before a configuration transaction needs its own legal governing-value binding; the transaction label does not conceal that behavior.
RC5 — Occurrences, generators and reset. Extend the existing bidirectional normative generator/path translation and prefix-stable occurrence map to R2C component sequences, actual sampler/atomic constraints and reset-epoch constraints. No combinational dependence or chosen source serialization is automatically a new required-order generator. Bind the same actual ResetKeys and ownership, emission comparison restart and uninterrupted ordinal. Certify included ending-epoch R2C units before RESET and absent units through participation. Bind fresh activity to required START/initialization and frame unaffected ledgers. Old group membership and history never change.
RC6 — Complete edge carrier. Bind ERB1/ERB2 to all actual implementation prefixes and successors, including finite prefixes ending during settlement, reset hold or initialization. The edge inventory and simultaneous successor state are exactly those realized by the combined local certificates. No assumed future edge, completion or release is needed to cover a prefix before that edge; it has the last completed origin's history. A repeated numerical value does not excuse an omitted required unit.

Lemma J.R2CInventoryAndCompose (settlement, emission and edge extension).

Under the applicable existing local contracts and RC0–RC6, related origins and one actual edge admit a jointly executable source extension with matching complete inventory, successor relation and canonical history. Proof. Induct over RC1's dependency graph from RC0's related roots to obtain the equal selected fixed-point vectors. Equal participation, sampler and related current ledgers make RC2 select exactly equal units and labels; this derives inventory equality rather than assuming emitted trace equality. After silent L2 and the established Core.16 trigger, RC3 realizes these observations legally and frames all remaining resource actions. Use the applicable local boundary/anchor composition and, on reset edges, J.ResetLocalCompose for the same selected snapshot. RC4 preserves operand attribution and excludes next-state feedback. Apply the selected ending-unit/reset order; the normative ledger fold gives matching successor seen/last, epochs and ordinals, including forced first emission after a later participating restart. Publish registered updates simultaneously and perform only the legal preparation/initialization covered by the ordinary/reset contracts. RC5 extends the occurrence map and both directions of required-order paths without changing the old prefix. □

Theorem J.LG-R2CBatch (finite-prefix constructive coverage with edge-observed R2C).

Assume Core.RegisteredR2CBatch, J.LocalR2CContracts, the applicable message/anchor/configuration/reset local contracts on their common relation, and ordinary boundary qualification or Q.SafetyQualified(E). These premises hold at the qualified base and every construction-reachable origin and actual successor under the fixed environment choices. Then each finite reachable implementation prefix has a reachable source prefix with equal selected canonical history. Proof. Apply J.R2CInventoryAndCompose inductively on completed edges, preserving the constructed source past. ERB1/RC6 give the same two prefix cases as the Batch argument: a prefix before the next completed edge has the last covered origin history; a prefix after that edge has its covered successor history. No intermediate R2C-only completion prefix exists in this declared carrier. The proof reuses the Batch prefix argument with the enlarged inventory, not a narrower theorem applied outside its hypotheses. No ideal-prefix closure, fairness, future edge or K conformance is consumed. □ Explicit induction interface: instantiate J.ConstructiveEdgeInduction with J.R2CInventoryAndCompose and the jointly qualified ordinary/reset cases. Bind actual-successor designation transport and the enlarged whole-edge prefix contract separately. If Core.SourceCheckpointProfile holds for this history, J.SourceCheckpointRepresentative also gives ReachableHistCovOn with source domain ChkRef.

Corollary J.LG-R2CBatch-Safe (combined constructive safety transfer).

Under J.LG-R2CBatch, an extensional property valid on every reachable canonical history of the selected source model holds on every finite implementation prefix. This is semantic ReachableHistCov and SafeTransfer-HistCov, with the existing route-local configuration observation extension where selected. The verified source model must implement the selected back-annotated/coupled storage, signals, configuration, reset and R2C observation semantics. A property proved only on different capacities, gates, reset behavior or observation emission conventions is not supplied by this corollary. Existing no-R2C profiles and K.CT completion results retain their own scopes and premises. With Core.SourceCheckpointProfile and the corresponding representative/projection proof, the sharper alternative uses Φ only on ChkRef and J.SafeTransfer-HistCovOn. Its selected-run form uses RefProjCoverOn and clause (2)'s all-finite-ideals property premise. Proving Φ directly on ChkRef instead uses clause (1); ordinary cycle-by-cycle simulation alone establishes neither universal source safety nor covering-execution evidence.

Example J.R2CEdgeSample (registered producer and retained consumer).

Let one sequential owner publish a,b at a common real anchor and let a combinational component compute z=(a+b) mod 256 and p=z mod 2. Select one complete R2C slice Read(a), Read(b), Write(z), Write(p), with the common anchoring domain and declared current-value resolution. A sequential consumer samples z at its anchor and retains that operand for a later Push. At a=1,b=2 the vector is (1,2,3,1). If this edge publishes next-state a=4,b=5, its R2C unit and current consumer sample still use z=3; the next cycle settles to (4,5,9,1). Intermediate propagation such as z=6 emits nothing. An unchanged ordinary cycle emits nothing, an explicit fresh sampler emits the equal vector at the next ordinal, and a cycle-locked component emits every participating cycle.

If an affecting RESET follows the old-epoch unit on that edge, clear seen/last after recording it and retain its ordinal/history. The first later participating cycle emits even for the same vector. If only the upstream producer resets and C is not affected, keep C's ledger and emit only on the ordinary changed-vector/sampler rule (or cycle-locked participation). This is a symbolic instance of the equations and ordering cases, not an executable campaign or a proof that arbitrary implementations meet the qualification.

Proposition J.ConstructiveCombinedProfile (application and closeout).

A single-clock constructive instance may combine registered message/pipeline behavior, unary waits and pairwise SyncChannel/ac_sync transactions, anchored ordinary signals, the fixed configuration-transaction observation option, qualified edge-sampled dynamic reset, and edge-observed R2C. Select only applicable rows of the matrix below and establish them on one common reachable-state relation and whole-edge carrier. J.LG-R2CBatch-Safe applies when R2C is selected; otherwise use the corresponding existing Batch safety corollary. This proposition packages the established sufficient local composition and prefix proofs; it does not certify an arbitrary design, replace local HLS correctness evidence. Uniform application over a declared stimulus class is supplied by J.UniformConstructiveEvidence/J.UniformConstructiveCoverage/J.ConstructiveClassSafe, with the same combined-profile and source-property qualifications.

Combined-profile closeout matrix. “Established” means a conditional paper theorem under every named qualification, not completed mechanization or concrete flow qualification; the final column remains instance evidence.

Case	Observation and order	Reset case	Instance evidence
Messages and pipelines	C0–C5; FIFO/rendezvous and E order	RB cancellation and epoch balance	Payloads, persistence, Core.16, actual E storage/gates
Waits and paired sync	A0–A6; whole anchor groups, E2/E5	Pending-group disposition; fresh START	Read inventory, write fold, sampling/resolution
Designated configuration	CO/CVG; one transaction record, contracted paths	New or justified retained governing binding	Actual θ uses, old-work isolation and source admission gates
Dynamic reset	RB0–RB5; ending units → RESET → new epoch	Declared scope, occurrence, hold and release	RW1–RW5 source/implementation effects. Partial reset scopes require an explicit modeled cross-epoch disposition and its local proof.
R2C	RC0–RC6; exact complete units and normative generators	Own-scope restart; global ordinal retained	Joint acyclic settlement, participation, sampler, ownership
Finite prefixes and safety	Whole-edge carrier; extensional canonical properties	Includes finite hold/init prefixes	Actual monitor/carrier and source property validity

The constructive J route has conditional paper-level coverage theorems for the declared combined profile, with the explicit semantic, settling, generator, carrier and local obligations stated here. This is not a report of completed mechanization or a qualified executable flow. Checkpoint-domain specialization additionally requires Core.SourceCheckpointProfile; START/FINISH phase placement and initialization representation are not assumed universally. Multiple clocks, unsupported synchronization forms beyond unary waits and pairwise SyncChannel/ac_sync transactions, unqualified feedback/observation domains and unproved implementation mappings remain outside it. Ordinary multi-process patterns assembled from supported transactions require joint evidence but are not excluded by process count. This constructive correspondence result supplies no K.CT completion guarantee; that guarantee requires its separately stated premises. No asynchronous-reset audit is supplied. Asynchronous-assertion implementations are not qualified by this result; applying it to such an implementation requires a separate conformance or reduction proof to the declared edge-sampled reset model. No such proof is supplied here.

J.ProcessEnvClass — Process-environment contracts

J.EnvironmentContract. Fix the declared environment semantics, placement, initial-state and externally fixed choices for the comparison. Reactive responses are governed by that contract and actual interactions; they may not be reselected to manufacture a match. External environments supply the legal source-step and consumed-observation correspondence required by the selected constructive theorem. Internal environment processes satisfy the same applicable operational, implementation and constructive contracts as the DUT.

Fixed-stimulus interpretation (reactive environments). When the same stimulus includes a reactive testbench or peer processes, it means the same declared global causal behavior and externally fixed choices, not a function of one process's local history alone. The selected constructive relation must justify the environment's responses at matched states and observations; neither identical code nor an assumed future DUT trace match supplies this fact.
Process-based environment classes. An environment may be specified as a network of sequential and combinational processes with declared initial states, communication/signal contracts, and external or nondeterministic choices. Its admitted behaviors define a stimulus/environment class; a uniform proof may quantify over all those behaviors. If environment processes are included within the selected formal system boundary, their applicable process, channel, clock, reset and settling obligations shall also be discharged. If they remain external, their behavior is governed by the bound environment contract. Using identical testbench/environment code in the pre-HLS and post-HLS comparisons does not, by itself, establish the required correspondence between their interactions. The application shall preserve the externally fixed choices and justify the environment's interaction with the compared histories, including any sensitivity to cycle timing, reordered events, non-blocking outcomes or reset. Responses generated through interaction are execution behavior, not an independently chosen input trace that may be substituted to obtain a match.
Process-environment discharge. J.ProcessEnvClass and J.UniformConstructiveEvidence support internal T processes obeying the same local contracts as the DUT, or an external T contract with legal source counterparts for every actual environment step and consumed observation. Reactive behavior is tied to matched state and actual return values. No assumption of future DUT trace agreement supplies that correspondence.
Definition J.ProcessEnvClass (process-environment stimulus class)
Fix a declared process testbench/environment T, a DUT and a semantic profile: source and implementation artifacts or their proved parameterized families; process/channel semantics, capacities and elaboration; clock/reset contracts; choice attribution; the proof-internal comparison boundary Σ; and the existing qualified DUT observation mapping to Σ_DUT. T may contain sequential and combinational processes. Declare whether T is internal or external to the selected comparison. This placement is fixed throughout each family proof. The primary profile includes T in the compared system; the secondary external-contract profile below remains normative.
A process-environment class C is a declared family of admitted initial-data and stimulus/environment resolutions s. Each s fixes externally supplied inputs, initial-state parameters and the choices or choice policies declared fixed. Both comparisons use the same declared T semantics, corresponding initial state and the same fixed choices. Choice attribution uses logical occurrences or a declared causal policy, not incidental simulator visitation order. Unresolved choices remain quantified in the reachable behavior sets; their presence is not silently replaced by a selected test run.
For each s in C, I_s is the induced fixed-stimulus comparison instance under J.EnvironmentContract. In the internal profile, its source system consists of the selected DUT source reference together with T_ref, and its implementation system consists of the DUT RTL together with T_post. These combined systems instantiate the existing Sys_ref/RTL_B comparison interfaces; calling the implementation system RTL_B does not require T_post to be synthesized. In the external profile, the compared systems interact with the bound external T contract. Responses generated by T are execution behavior, not an independently reselectable input trace. A witness may depend on s and the selected implementation prefix under L.WC where the selected evidence consumes it, and under the exact-instance restrictions; it may not change a choice fixed by s.
Internal T realization contract. T_post executes the declared T program through a bound, qualified TB runtime, including the scheduler, libraries, wrappers and transactors. Qualification identifies supported versions, configurations, semantics and interface usage, and supplies the applicable implementation-

conformance evidence. T_ref uses the same program semantics and fixed choices, with only Core.SelectLatitude finite-prefix selection latitude under Core.LLegal and the selected Policy L or Policy S semantics. All applicable issue, synchronization, channel, reset, atomicity and fixed-choice constraints remain authoritative; this reference introduces no new latitude and does not import complete-execution fairness into finite-prefix correspondence. Unsynthesized or unchanged program text does not mean identical execution schedules. A fixed concrete TB schedule may be imposed only if the selected construction remains realizable under that restriction. The M.7a tool/simulator-semantic qualification and T source/realization conformance entries bind reusable evidence and discharge the instance-specific applicability conditions. Revalidate the TB runtime and wrapper against Core.TransferLaw and Core.SyncProtocol. Any previous source realization that skipped a present handshake is outside the revised source relation; unchanged TB code or equal histories do not supply the new qualification.

Internal T proof scope. Testbench processes obey the same applicable source, implementation, primitive, environment and carrier contracts as DUT processes. Their returned data and state determine reactive behavior. External T instead supplies the C0/A0 step correspondence and actual observation contract. A uniform class claim requires these facts throughout the declared class, not only on tested instances.

External T profile. T remains outside the selected formal system and satisfies J.EnvironmentContract over the construction domain. The selected constructive theorem must prove preservation of that contract under its source realization and fixed choices. Untimed or TLM implementations are not automatically Core.RegSeq-conforming; this profile supplies no additional TLM abstraction theorem.

External-profile selection guidance. Use the external profile when the chosen theorem boundary leaves T external and suitable representation/response evidence is available. Examples include reusing an established external contract with qualified representation maps, or intentionally retaining an existing DUT-only comparison boundary. T need not be impossible to model internally for this choice to be useful. External placement is not a default reduced-effort route: the applicable C0/A0 legal source-step and consumed-observation correspondence under J.EnvironmentContract still requires semantic evidence, and its cost depends on existing qualification and contract proofs. Internal T remains the primary process-testbench profile. This guidance adds no untimed/TLM abstraction theorem.

Definition J.UniformConstructiveEvidence (stimulus-class local constructive evidence).

Fix a declared class C of admitted stimulus/environment resolutions and the induced instances I_s, with the placement and binding discipline of J.ProcessEnvClass. The following UCL0–UCL3 clauses are proved uniformly throughout C. Replay, source-choice, operational settling and other evidence is required exactly where the selected local proof consumes it; class membership and coverage are independent of construction success.

UCL0 — Family and observation binding. Fix the design or a qualified parameterized finite design family, source/implementation systems, tool/runtime and integration choices, initial/reset semantics, boundary capacities/elaboration, environment placement and contracts, and one named constructive completion/observation profile. For every $s \in C$ bind these choices prospectively in I_s. A parameter-dependent E_s or configuration is fixed by its declared instance binding, not selected retrospectively to fit an execution. Bind the actual completed-unit carrier, projection, atomic groups and normative generators. Let H_s denote its selected canonical history function: ordinary CanonHist_Σ_DUT or the explicitly selected route-local configuration history and any qualified whole-group projection. Both sides use exactly the same H_s convention. Class membership and reachable-prefix domains are defined independently of source-witness existence, constructor success or package acceptance. Bind the source-property domain A_s prospectively: Reach_ref(I_s), or ChkRef(I_s,H_s) with Core.SourceCheckpointProfile proved uniformly, including any declared initialization checkpoint cases.

Domain choice does not depend on constructor or checker success. Bind semantic revisions or definition-level content hashes for preparation, choices, generator admission, reset emission and anchor inventory under Core.EvidenceHdr and M.7a's reuse-key discipline, with the consumed dependencies identified for each owner.

UCL1 — Reusable local construction. Give a family of causal exact-prefix relations R_s with legal matched bases and prove all applicable C0–C5, A0–A6, CVG0–CVG3, RB0–RB5 and RC0–RC6 clauses required by the selected constructive theorem, uniformly over s, every construction-reachable related origin and every actual implementation successor. The finite actual carrier induction must cover every reachable prefix of the selected profile; one cannot discard a failing successor or an unsuccessful extraction. When J.StableRead-Batch is selected through its reduction interface, supply SD0–SD4 and uniform reduced-body evidence instead. Bind ordinary resource qualification or Q.SafetyQualified(E_s), complete joint settlement, source realization, simultaneous updates and finite legal preparation, actual environment choices, occurrence/operand/activation identity and J.RequiredOrderWF as applicable. Prove shared invariants at the base and preserve them under the local updates. Reuse of a rank template still checks its complete unit/generator domain. For OM/OA, J.OrdinaryConstructiveComposition derives this family of relations and construction obligations from uniform restricted-vocabulary local evidence and the named integration/settling inputs; UCL0/UCL2/UCL3 and optional checkpoint qualification remain explicit.

UCL2 — Environment and testbench composition. In the primary internal-T placement, apply the same local contracts to DUT_ref ‖ T_ref and DUT_RTL ‖ T_post, with explicit shared resources and the complete joint settling graph. T obeys the selected profile too; its runtime integration and the environment still outside the combined system are qualified. Reactive responses follow from the related current T state and actual returned values, not from assuming future DUT trace agreement. In the external-T placement, C0/A0 and the selected extensions supply a legal source counterpart for every actual environment step and every observation on which the environment depends, under the fixed contract and choices. An existing response lemma may be reused only after proving it entails these exact local obligations and retaining its own premises. Neither J.ProcessEnvClass membership nor equal DUT histories alone supplies that step correspondence.

UCL3 — Completion and projection. Uniformly bind the chosen BM/AB/CO/RBP/ERB carrier and observation clauses, as applicable, so each finite implementation prefix has the history of its covered initial or last completed origin. Include finite reset-hold/initialization prefixes and exact R2C/configuration inventories when selected. This is the actual qualified Batch domain, not all mathematical ideals or an unqualified finer monitor. For a further DUT/message projection, prove the applicable whole-group and normative-path contraction obligations on the same executions; J.ConfigWholeGroupProjection is available in its stated scope, not automatically for a new mixed reset/R2C erasure. A source witness may vary with s and the implementation prefix, but previously constructed history, choices and occurrence identities remain fixed along each construction.

Theorem J.UniformConstructiveCoverage (class-wide semantic history coverage).

Under J.UniformConstructiveEvidence(C), for every $s \in C$ and every finite reachable implementation prefix κ of the declared profile there exists $\lambda \in \mathrm{Reach_ref}(I_s)$ with $H_s(\lambda)=H_s(\kappa)$. Equivalently, the corresponding semantic J.ReachableHistCov holds throughout the family under its ordinary or explicitly extended route-local observation. Proof. Fix arbitrary s and κ. UCL0 binds the exact instance. UCL1/UCL2 discharge the selected local constructive theorem on the combined or externally driven system, without substituting stimulus choices. UCL3 supplies its finite-prefix carrier and any qualified projection. Apply J.LG-Origin plus J.BatchPrefixCoverage, J.StableRead-Batch, J.LG-AnchorBatch, J.LG-ConfigBatch, J.LG-ResetBatch or J.LG-R2CBatch as appropriate; use the coverage portion, without assuming source safety. The applicable local operational induction and canonical extension produce λ, and projection preserves

its equality and reachability. Generalize over s and κ. This produces semantic coverage and exact constructed source extension under the selected observation; a full internal-fact consumer additionally uses J.ConstructiveFactCorr. □ If UCL0 selects ChkRef with uniform Core.SourceCheckpointProfile, the same witnesses followed by J.SourceCheckpointRepresentative establish ReachableHistCovOn(Reach_post(I_s),A_s). The original post-L1 witnesses remain the inductive states; their checkpoint representatives are used only for the property conclusion.

Corollary J.ConstructiveClassSafe (stimulus-class safety on the constructive route).

Assume J.UniformConstructiveEvidence(C). For each s∈C let Φ_s be extensional on H_s, and independently prove ∀λ∈A_s, Φ_s(H_s(λ)), where A_s is the independently bound UCL0 source domain, uniformly throughout C. A common Φ may be used when the history type/property is independent of s. Then ∀s∈C, ∀κ∈Reach_post(I_s) in the selected profile, ∃λ∈A_s: H_s(λ)=H_s(κ) and Φ_s(H_s(κ)). Proof. J.UniformConstructiveCoverage supplies domain-specific coverage, using J.SourceCheckpointRepresentative when A_s=ChkRef; apply J.SafeTransfer-HistCovOn clause (1) and substitute equal histories. The source-property proof concerns the actual selected back-annotated/coupled, signal/configuration/reset/R2C model, including internal T where selected. It is not inferred from correctness of the transformation. □

Optional source alternative. For every s, supply a selected complete source execution ρ^{o}_s, prove RefProjCoverOn(A_s,ρ^{o}_s) in exactly H_s, and prove Φ_s on every finite canonical ideal of H_s(ρ^{o}_s), as in J.SafeTransfer-HistCovOn clause (2). The original RefProjCover over all Reach_ref remains sufficient but is not required for a qualified checkpoint specialization. Restricting A_s weakens coverage, not the all-ideals property premise. A proof only on A_s-realizable histories is the universal-source clause (1), even if organized using a covering execution. Bind route-local observation extensions explicitly. Neither passing samples nor a shared TB proves these family premises. The universal-source form requires neither ideal closure, B1 nor fairness; neither form supplies K.CT completion, progress or performance conclusions.

Application and reuse. The uniform proof may use per-design templates and class invariants; instantiated states, finite records, witnesses and their construction computations may depend on s and κ. Applying it to an admitted instance still requires exact binding and any residual membership/qualification checks, but not a fresh mathematical correspondence proof. A finite test campaign, one reusable structural artifact or one testbench program does not prove UCL0–UCL3 or the source property for all C. Class quantification does not widen profile scope. If only LG-Origin or another restricted-domain result is supplied, the claim remains on that explicit domain; it cannot be reported as J.UniformConstructiveCoverage's all-finite-prefix conclusion. The established all-prefix result can of course be restricted to any explicitly declared D_s.

Class scope. Uniform local construction and carrier evidence establish the semantic coverage theorem for every admitted stimulus instance. Source-property evidence remains separate. A completion class additionally binds the workload, fact signature, agreement and progress contracts for each instance, with a uniform discharge where claimed.

Ordinary constructive locality

For the explicitly defined OM/OA profiles below, local correspondence evidence over a prescribed interface vocabulary, ordinary primitive laws, generic integration checks, qualified settling, and the applicable environment/carrier evidence suffice for global constructive correspondence. The combined relation is assembled from these inputs; its global preservation is theorem-derived. The local evidence includes design-specific process correspondence proofs. This result does not assert that every R/E-correct implementation admits such local evidence, and it does not replace the broader constructive

profiles. O1–O7, LP0–LP5, G1–G7 and QF/QSafe/QM below are clauses of these named ordinary interfaces, not additions to similarly numbered clauses elsewhere.

Definition J.OrdinaryConstructiveProfile (OM/OA structural scope).

OM denotes the ordinary message-only specialization of Core.RegisteredMessageBatch, with no selected synchronization/signal extension. OA denotes the ordinary specialization of Core.RegisteredAnchorBatch that adds only the O4 anchor/signal primitives and conventions. O1–O3 and O5–O7 apply to both; O4 applies to OA. The restrictions below are additional sufficient-profile conditions, not a redefinition of all R/E-conforming designs.

Fix a stimulus class C and instances I_s before constructing any witness. For every s in C:

O1. There are finitely many sequential processes, including any internalized testbench processes, one clock, a prescribed reset-established base, and no later dynamic reset. Finite process structures may depend on s through a declared family parameter. Eager issue, NB operations, private computation, pipelining and overlapped iterations remain allowed under Core.RegSeq, Core.ReachPrep and Core.16.

O2. Each explicit message channel is an ordinary uncoupled scalar E4 FIFO of fixed capacity B>0 or a B=0 rendezvous, with one declared producer endpoint and one declared consumer endpoint. Each endpoint direction has the existing single-transfer-per-cycle and unique dynamic-owner discipline. A process may own many distinct endpoints. Blocking and NB uses of an endpoint obey the same ownership and attribution rules. A self-connected channel is allowed when the two endpoint directions and call records remain distinct and the source-local completion rules apply.

O3. Channel storage is independently owned. The profile excludes shared capacity pools, multiple owners of one endpoint, hidden multi-channel allocation, shared mutable application memory, extra compound multi-channel operations, and additional elaboration/c-order or sampler generators. Physical implementations may have complex private representations, but the supplied component refinement and footprint laws must realize this independent abstract ownership. This proof does not qualify an implementation that needs overlapping ownership or an omitted cross-resource admission predicate. An arbiter may be implemented as an ordinary sequential process with private registered state and distinct point-to-point input/output channels. Its decisions then belong to that process's local proof. An unmodeled arbiter shared by multiple clients of the same endpoint is excluded. A separately proved shared-resource primitive would be an extension of this profile, not an implicit ordinary case.

O4. OA permits unary wait/ac_wait and paired SyncChannel/ac_sync transactions under Core.RegisteredAnchorBatch. Pairing is explicit. Synchronization completion on each process obeys E1, so at most one sync occurrence involving that process completes in one real cycle. Retained sequential Reads/Writes satisfy RULE 1/2, including anchor-time membership/value, writer provenance and complete anchor groups. Signals have one declared writer or one external driver and any number of readers. OA selects pre-update edge sampling: each selected anchor reads the common logical signal state fixed before this edge’s anchor updates, and writes committed on the edge affect subsequent sampling. Source and implementation must realize this exact convention. This is a selected OA restriction, not a convention inferred from AB2 or from QSync. Logical signal state is owned by a signal component; per-process pending writes and captured samples are private. Same-edge forwarding, multiple-driver resolution, general configuration-observation extensions and R2C are outside this OA theorem. A visitation-independent joint sampling/update extension would need its own operational and canonical-order primitive proof; it is not established by this theorem. Direct inputs are allowed only with the applicable ordinary Read-based stability/operand qualification. The profile adds no exemption from retained-Read membership. This first OA theorem uses only unary wait/ac_wait and paired SyncChannel/ac_sync anchor primitives. It does not use START_P or FINISH_P as completed observable/proof units, as anchors for retained signal IO, or as erased support contributing required order. A convention using them needs explicit primitive cases and is outside this theorem as stated.

Initial Reads must therefore have an admitted real wait/paired-sync predecessor anchor; retained final Writes must have an admitted real successor anchor. Termination alone does not invent FINISH or flush unanchored writes. All admitted synchronization instances additionally satisfy Core.SyncProtocol. This physical lifecycle does not alter OA's pre-update logical signal sampling convention or authorize an otherwise excluded configuration-observation extension.
O5. The complete normative generator inventory is exactly the applicable ordinary message/FIFO/rendezvous generators and, in OA, E1/E2/E5. Every sync relevant to the full OA comparison is included before any declared whole-group outer projection. No extra order is introduced by logging, witness serialization, physical layout or unrelated lexical source order. Any further observation erases complete matched groups and contracts the same required paths on both sides.
O6. Implementation transitions have the BM1–BM4 whole-edge carrier interpretation, or AB1–AB3 in OA. The base is the prescribed reset-established, pre-completion state with empty selected history; any observable initialization actions are included among the subsequently covered edges and have supported primitive phases. An origin chosen only after an unaccounted observable initialization segment is not this theorem's base. In the basic statement the origin is that actual prescribed base or an actual post-edge occurrence: designation is identity. A nonidentity designation can be accommodated only with the separately supplied Core.ConstructiveOriginCarrier transport premise; it is not derived from local process proofs. Terminal prefixes do not assume another edge exists.
O7. Either the environment is represented by ordinary modeled processes with the same local contracts, or a separately qualified external environment relation supplies C0/A0 step matching for its actual input/output observation interface. The external relation r_env is restricted to source/implementation environment-owned state, prospectively declared choices, matched interface objects from G2, and their causal observation histories. It cannot inspect DUT-private registers, encode an unexposed DUT invariant, or assume a matching future DUT continuation. It is universally qualified over admitted interface observations. An external environment that observes physical timing or additional implementation signals needs an explicitly enlarged matched interface and evidence for those observations; otherwise it is outside this theorem. Canonical DUT-history equality alone is insufficient.
The profile is defined by these declarations, not by whether a global proof later succeeds. Failure to establish a required local law does not redefine profile membership or authorize an arbitrary extra global premise.

Definition J.OrdinaryLocalEvidence (restricted vocabulary and LP0–LP5).

For process P the permitted relational vocabulary consists of:
• P's source and implementation control, registers, private pipeline/staging state and local variables;
• P-owned pending call, operand/writer, result, activation, interval, anchor and captured-sample records;
• finite causal ghost data derived only from P's past interface actions, private state and actual inputs;
• P's endpoint requests, stable push payloads, return packets, and the declared settled availability/frontier observations at those endpoints;
• the ordinary channel/sync/signal facts exposed by incident primitive contracts, and the glue facts below.
The vocabulary cannot contain an opaque Boolean for an unrelated global invariant, a ghost copy of a peer's internal state, knowledge of a future response, or an assumption that the desired global source extension exists. Fixed parameters and external inputs must be declared; admissibility may not be defined by successful correspondence construction.
Let r_P relate the permitted source/implementation local states and causal records. The process proof is universal over every locally admissible interface valuation/packet satisfying the enumerated incident-

primitive facts. It is not restricted to inputs whose admissibility presupposes global RTL/source correspondence.
LP0 — Initialization. For the prescribed local reset state and declared initial inputs, construct related source-local state and a finite legal owned initialization/L1 preparation that completes no selected unit and changes no required order. The common source initial state is the prescribed one; local proofs may not choose inconsistent values for shared signals or channels. A selected initialization completion is realized on a covered edge by its actual supported primitive phase, not hidden in this preparation. A convention requiring a different base needs its own explicit base lemma and is outside O6 as stated.
LP1 — Roots and attribution. r_P entails equality of P's complete request roots, including absences, nonminimal pending blocking calls and live NB attempts, with matching owner/site/ordinal/direction and stable payload. Operand values refer to their actual local source writers and captured inputs. The local implementation has no unrecorded abstract endpoint or signal effects.
LP2 — Activation and choices. From the incident settled frontier observations, derive the actual Core.16 activation and its persistence. Prove inhibition of new source-later evaluation/Issue where required, while retaining permitted earlier-admitted completions and abstractly invisible private computation. NB outcomes obey Core.NBDecision on the complete Core.NBDecisionSnapshot; annotations do not choose them. Any external unresolved choice is supplied through the prospectively fixed interface relation. At an eligible edge, no local choice can omit a Core.TransferLaw/Core.SyncProtocol completion. Separate legal issue or abstract movement choices from the complete forced physical packet.
LP3 — Completion bookkeeping. Already issued calls admit their selected source completion/return records under the primitive guards. Distinct simultaneous results are stored by dynamic call identity; consuming returned values in ordinary evaluation, control, requests or payloads waits until a later Horizon. The local source bookkeeping for independent completed calls commutes, or has an equivalent canonical record representation with the same available primitive steps and final local state. Any last-writer/control reconciliation belongs to the checked local L4 rule, not to an invented cross-process completion dependency. For an anchor, the source pending sync, complete Read membership, final pending Writes and samples are already determined; an edge-returned message value cannot supply a new same-edge dependent anchor computation. Prove this from the actual source control representation, not from a merely abstract packet model. The incident packet is complete for every mandatory handshake/release involving P on that snapshot, not merely a collection of individually legal returns. Retain all issued and held synchronization offers required by Core.SyncProtocol.
LP4 — Update and preparation. For every admissible incident completion packet, prove the actual registered implementation update is matched by legal source L4 and a finite source-local preparation/Issue word, restoring r_P. Pending requests/results and causal records persist correctly. The word accesses only owned state and already established samples/results; it neither waits for nor reads another process's newly prepared requests. Prove ReachPrep, issue prefix closure, strict post-sync timing and result availability. An actual later successor from a stored non-L1 checkpoint includes the guarded legal phase-entry proof. No source release/continuation is assumed for a terminal export.
LP5 — Primitive effects and order keys. Newly completed observable units have the exact effects/values, call and sync occurrence identities, endpoint/channel ordinals, interval attribution, and complete anchor-group members declared by the supported primitive rules. Prove local issue/order/fence conformance on those identities. No premise asserts equality of entire global histories.
These are design-specific local proofs. They can be supplied by qualified transformation templates but are not made automatic by R/E conformance. LP3 is substantive: the theorem does not assume that arbitrary per-process action orders are globally compatible. Instead, ordinary source completion bookkeeping and the explicit message-before-anchor order establish the particular compatible

construction below. An implementation/source representation that cannot supply LP3 is not silently repaired by a global invariant.
Each channel has a local realization relation r_c between its source abstract FIFO/request state and its implementation state, universally qualified for the E4 input cases. Each OA signal has r_z for its owned logical value and declared sampling/joint-update law. Their certificates cover initialization, concrete footprint soundness, request/selection/outcome laws, effects, framing and update preservation. Pair-sync primitive qualification binds both participants atomically. These are primitive-local qualifications, not a global execution certificate.

Definition J.OrdinaryGlue (G1–G7 and integration checks).

Use exactly the following shared facts G:
G1 — Identity and ownership. Every component and endpoint has its declared unique identity/owner; private write footprints are disjoint; channel/signal storage has one owner. Atomic rendezvous and paired sync are single shared descriptors, not duplicated effects.
G2 — Wiring agreement. Duplicated endpoint request, payload, call key, return destination, sync participant, signal/sample and clock/epoch fields are projections of the same interface objects. Actual implementation connections and abstract interface projections agree.
G3 — Channel accounting. Each FIFO represents one ordered token sequence Q_c with 0 <= |Q_c| <= B_c. Its source and implementation projections agree under r_c. Push/pop occurrence counters and token provenance follow this sequence; rendezvous uses matched endpoint ordinals and values with no storage token. These are per-channel invariants; no arithmetic relation between unrelated queues is assumed.
G4 — Request lifecycle. An active endpoint has its one permitted dynamic owner, stable request/payload, current interval and issued status. Completion retires that owner once. NB attempts and failures are accounted for once. Back-to-back owners and incomplete requests obey Core.IC and local issue rules.
G5 — Phase and snapshot agreement. Components share the same clock/phase and the one actual frozen edge snapshot. Return values are keyed to their actual return cycle. Every settled frontier observation has a unique source in the qualified network; no post-transfer recomputation creates a new selection. Physical transfer and synchronization commitment use Core.TransferLaw and Core.SyncProtocol on this same snapshot; neither later residue state nor a local choice changes their complete inventory.
G6 — Anchor and signal agreement in OA. Pairing and participant keys agree; every selected anchor has its complete fixed members. Shared signal samples are supplied by the declared frozen sampling rule, and pending writes are attributed to their unique owners. Source process intervals agree with the incident message/sync occurrence keys.
G7 — Inventory and normative-schema closure. The actual edge inventory contains all and only the supported completed units, silent NB retirements and updates. Every applicable ordinary generator is included and has the reusable source-rule realization and prefix/restriction facts. No extra hidden generator or split atomic unit is admitted. For every declared physical interface, establish both directions: each sampled handshake maps to its complete declared unit/effects, and each listed physical unit/effect is backed by its sampled handshake or qualified unary release. Include proof-internal transfers and reconcile shared endpoint views by the fixed occurrence mapping. Legality of only the listed units is insufficient.
The integration evidence checks the structural declarations, footprint/interface projections, common parameter/choice binding, primitive selection, complete emitted-unit instrumentation, and generator schemas. G is not an independent assumption that some corresponding source state exists: its base and

preservation are proved below from the local laws. Dynamic values of G are maintained by those laws, not inferred from static wiring alone.

Definition J.OrdinarySettlingEvidence (QF/QSafe alternatives and QM matching).

Use one of the following two named discharge paths, uniformly over s and the related origins. Both have the same consumed safety conclusion. They are alternatives, not cumulative obligations.

QF — Existing source QSync qualification. Assume QualifiedSynchronousModel_Sys_ref(I_s) over the complete combined model. Identify the exact constructed source L1 endpoint with its source cycle origin under Core.Phase/Core.Settle; a non-L1 checkpoint first uses its actual guarded phase-entry proof. Invoke Core.SourceL2Settle once: QS2 supplies the frozen registered/input/request frame; QS3 supplies complete operational node realization; QS4 supplies finite acyclic deterministic evaluation; QS5 supplies complete represented boundary values. The theorem gives the legal finite path to the first source SettlePoint (indeed the unique fixed-point endpoint). It does not require a second independent proof of source settling.

QSafe — Safety-only operational qualification. When QF is not already available or is unnecessarily strong, establish J.SourceSettleRealization from the same QS1 single-clock, QS2 frozen-frame, QS3 complete operational realization and QS4 finite acyclic evaluation facts, together with safety-boundary completeness: every value consumed by the source primitive guards, NB decision laws, pairing, signal samples and the entire Core.16 frontier, including absent/nonminimal requests, is represented. Apply J.SourceSettleExists. It retains the acyclic profile restriction and makes no claim that merely dropping arbitrary QS5 fields from full QSync preserves other QSync conclusions.

QM — Source/implementation snapshot matching, required on either path. LP1 and G2 provide matching registered/request/input roots. Each ordinary primitive and process interface supplies functional equation/refinement evidence mapping the actual RTL snapshot computation to the same source boundary quantities. The complete composed network/refinement map must account for every consumed root and dependency; a checked topological evaluation or equivalent compositional equation proof gives equality of the consumed settled values. These matching facts are not implied by qualifying the source and RTL separately as QSync. Full RTL QualifiedSynchronousModel is not a new premise here: actual RTL frozen-snapshot realization, the applicable local refinement evidence and the complete-network check are sufficient for this interface.

QM equation-satisfaction alternative. Instead of an RTL topological-evaluation proof, give a complete valuation of the represented source nodes induced by the actual RTL decision snapshot under the declared refinement map. Check matching fixed registered/input roots and resolved choices, and prove that this valuation satisfies every source node equation. The qualified finite acyclic functional source equation system has a unique solution, so that induced valuation equals the source SettlePoint valuation on every consumed value. Stability of an RTL snapshot alone is insufficient: complete equation satisfaction, the refinement map, matching roots and actual edge use must be established. This argument does not itself require full RTL QSync or a separate RTL acyclicity proof; all independently applicable carrier, network and profile restrictions remain. Hidden or omitted source nodes cannot be skipped merely because only interface outputs are observed.

QF or QSafe plus QM discharges the entire J.SourceSettleRealization interface used here. Source-path existence is supplied by the selected path; matched endpoint values and actual edge binding are supplied by QM. Every applicable frontier predicate is then evaluated at that endpoint, so Core.16 activation is the source operational activation.

Per-process registered separation and primitive-local equations are reusable evidence, but do not alone prove the combined model qualifies. Integration still checks complete representation and cross-component acyclicity. The source-node legality/functional facts must justify the actual evaluation path,

not merely a netlist with no syntactic cycle. In the QF path those checks are part of the already supplied QSync qualification; in QSafe they are its safety-only structural counterpart.

Lemma J.OrdinarySettleComposition (reuse of qualified source settling).

Assume the prescribed source L1 endpoint and related actual implementation origin. On QF, Core.SourceL2Settle produces the finite legal settling path and fixed point. On QSafe, J.SourceSettleExists produces them. In either case the source frame stays fixed and no selected unit completes during the path. For the QM refinement/evaluation route, induct along its order: corresponding input roots agree by LP1/G2; local equation/refinement laws therefore establish each represented downstream boundary value. For the QM equation-satisfaction alternative, the complete valuation induced by the actual RTL decision snapshot satisfies every source node equation under the same fixed roots and resolved choices; the finite acyclic functional source equations have a unique solution, so this valuation equals the source SettlePoint valuation. Completeness covers all consumed observations, including the complete activation frontier. Thus the path ends at the actual matched decision snapshot required by C2/A2. This establishes the settling premise used in Lemma J.OrdinaryBoundaryCompose without re-proving QSync-Ref or assuming source/RTL equivalence from QSync alone.

QSync is not a boundary-update theorem. QS2 excludes commits/registered updates during L2; it does not prescribe the read-during-write convention in L3, imply LP3 completion commutation, establish LP4 process refinement, prove Core.16 Issue compliance, or supply BM/AB prefix semantics. Those obligations retain their owners. In particular, QSync does not remove O4's pre-update sampling restriction.

Definition J.OrdinaryRelation (assembled component relation).

For s fixed, use the prescribed state decomposition and let

R_s(kappa, lambda, g) := [AND_P r_P] AND [AND_c r_c] AND [AND_z r_z] AND r_env AND G.

The z conjuncts are absent in OM. g contains only the causally accumulated local/interface records, counters and the already-built occurrence map. Local ghost witnesses can be chosen independently because shared choices and interface values are fixed inputs, not privately existentially chosen outputs. Duplicated exported keys refer to one shared object under G2. This explicit gluing rule prevents individually satisfiable but mutually inconsistent local witnesses.

R does not contain a free-form global invariant, the desired next source execution, or equality of future canonical histories. Source-prefix legality and current canonical-history equality are proved alongside R by the construction. Mathematical whole-prefix data in the theorem do not become additional local preconditions.

Lemma J.OrdinaryBoundaryCompose (generic ordinary edge realization).

Fix an R-related state and any actual implementation edge, including its actual choice resolution. Apply the settling composition lemma: QF reuses Core.SourceL2Settle, or QSafe uses J.SourceSettleExists, while QM establishes equality with the actual implementation snapshot. The resulting operational SettlePoint supplies the complete frontier evaluation and correct Core.16 activation. From the actual edge, Core.TransferLaw/Core.SyncProtocol and the local primitive laws construct the complete finite descriptor set T. Matching roots, complete equations and the same reset/edge binding establish equality of the full forced source and implementation inventories; G7/LP3 supply exact incidence and bookkeeping.

Partition T into M, the message commits and silent NB returns, and A, the OA anchor transactions. T contains exactly the actual inventory by LP1/LP5 and G7. Every mandatory message handshake and synchronization release is represented. Discretionary issue or genuinely abstract-action latitude cannot

remove one of these units. The declared NB law, rather than a discretionary choice of success, fixes NB results.
Construct L with precisely the same-edge blocking-message predecessor -> anchor edges required by each selected anchor's SyncFence. All other primitive prerequisites are already established by the pre-edge snapshot, a prior completed cycle, or atomic participants. Thus every edge of L goes from M to A and no edge leaves A. Every directed path has length at most one; acyclicity follows from that direction, not from bipartiteness alone. No design-specific graph acyclicity certificate is required.
Why no omitted reverse or same-class prerequisite is needed:
1. Message-result-dependent computation cannot produce another selected same-edge message or anchor: Core.RegSeq/LP2–LP4 require a later Horizon. Requests and anchor values used here already exist.
2. No two transfers use the same endpoint direction on the edge. Different-channel completion records are keyed separately; LP3 supplies the source-local bookkeeping/frame fact even when a process participates in several transactions.
3. A FIFO with both directions selected has interior initial occupancy. The two updates are independently legal in either order, as shown below.
4. A rendezvous is one two-endpoint action, not a pair requiring a circular ordering.
5. E1 strictly separates a process's successive syncs by cycle. Consequently distinct selected anchor transactions have disjoint process participant sets. A cycle of same-edge paired-sync dependencies through shared participants cannot occur.
6. E5 places source-later interval issue strictly after the sync cycle. It does not generate a same-edge anchor -> later-message edge.
7. O4 fixes pre-update sampling. Shared signal reads therefore use the same pre-edge logical values even after other anchors have been visited. Unique-writer effects are collected for the prescribed joint update. No selected Read consumes a same-edge anchor Write, so no writer-anchor-before-reader-anchor operational dependency is needed.
These arguments depend on the explicit ordinary vocabulary and primitive laws. They would not follow for arbitrary graphs formed by taking a union of separately acyclic local precedence relations.

Ordinary FIFO residue calculation.

For B>0, let the pre-edge contents be Q and n=|Q|. Let p,d in {0,1} denote the selected successful push and pop. If p=1 then n<B; if d=1 then n>0; if both, 0<n<B and the returned value is head(Q). For a subset U of performed descriptors let p_U,d_U be their indicators. Define
Q(U) = drop(d_U,Q) followed by ([v] if p_U=1 else []).
The pop always removes an old head: simultaneous pop requires n>0, so no same-edge pushed item supplies it. Both update orders finish at tail(Q) followed by [v]. Intermediate occupancy is within [0,B]: the extreme push-first occupancy n+1 is <=B, and pop-first occupancy n-1 is >=0. For either single-action case the same formula applies. With no committed transfer the queue is unchanged.
A failed NB return changes no queue. Its legality/result is determined by the frozen snapshot; it is not recomputed after a successful opposite-end action. The ordinary source L3 rules permit exactly this frozen-decision use. Each actual successful/failed endpoint action retires its attributed record once.

Other ordinary component residues.

Rendezvous contributes one matched transfer and its two call results; private state and other channels are framed. Process residues are the source states obtained by recording the subset's keyed completion packets under LP3. Different return records compose; normal result-dependent continuation is deferred. OA anchor residues contain the already fixed complete group, the retired transaction and interval bookkeeping. An L-minimal anchor sees all selected same-edge fence predecessors already

recorded and all older fence predecessors in the exact past, so its source SyncFence guard holds. Its participant readiness and value/anchor guards were established locally from the snapshot.
Signal residues retain the O4 pre-update samples and collect the uniquely owned prescribed update, independently of anchor visitation order. Primitive qualification proves the actual source and RTL implement this convention. The values may originate in earlier updates, initialization or the declared environment input; the proof does not require every sample to have an observable earlier-cycle writer. This is not permission to freeze an otherwise order-sensitive signal model.
For every L-downset U and minimal a outside U these formulas/local rules supply its source guard, exact update, and frame. Shared reads agree because their interface objects/snapshot are common. This is C3 (and OA's anchor case), derived parametrically; it is not enumeration of global subsets.
J.LocalBoundaryCompose or J.AnchorBoundaryCompose produces the legal boundary segment. The terminal residues equal the actual implementation packet/effects. Because T was proved complete for the fixed forced inventory, closing the boundary segment after its last descriptor discharges every required physical action of that edge. This completeness step, not individual guard legality alone, supplies C2’s mandatory-action requirement.

Lemma J.OrdinaryUpdatePrepare (product preservation and legal preparation).

Apply each LP4 to its incident final packet and each channel/signal local update law to its owned state. Every registered next state is formed from the same current state and designated packet, then published with the actual simultaneous-update semantics. Disjoint ownership and G2 make the resulting product state well defined.
Each source preparation word is finite and writes only process-owned state/requests; its inputs are already established local results, samples or fixed environment interface values. It does not depend on a peer's newly prepared requests. Executing one word therefore leaves every other word's preconditions and operands unchanged. Induction over the finite process set concatenates the words into a legal source L1 segment. Its preparation portions satisfy Core.ReachPrep and Core.PreparationEvidence, including any additional prospectively declared instance restrictions; Issue transitions retain their separate source-legality obligations. Cross-interface source issue prefix closure remains local to each process and follows LP4; global execution order adds no canonical order.
This restores every r_P, r_c and r_z. G1 persists by framing; G2 by the shared interface objects; G3 by the FIFO/rendezvous update equations; G4 by keyed retirement/persistence and the local issue laws; G5 by common cycle advance and guarded phase entry; G6 by matched anchor retirement/sample storage and interval advance; G7 by exhaustive exact-once effect accounting and the fixed normative inventory. Environment roots are updated by the internal process contracts or the explicitly supplied C0/A0 external step relation. No desired future trace is used to establish any of these facts.
Thus G's preservation is a generic theorem of the interface laws. In particular it does not require equality between unrelated occupancies, correlated data in different queues, an application protocol invariant, or an arbitrary global scheduling invariant. If a proposed local proof needs one of those as an assumption not derivable from its declared interface, this theorem does not hide or discharge it.

Lemma J.OrdinaryCanonicalExtension (ordinary generators and history equality).

Extend the past occurrence map by the descriptors just realized. Namespace separation, endpoint cardinality and atomic pair construction give a bijection of complete full-comparison groups. LP5 and primitive payload/sample laws give equal labels and internal group structure. Failed NB returns extend only causal proof records, not the selected group carrier.
Check the exhaustive ordinary generator list:
• Per-channel FIFO/endpoint generators use the same channel and occurrence counters and the same old-token matching. Rendezvous endpoints belong to one unit.

• E1 uses each process's sync occurrence index. Distinct same-process syncs are in different cycles; paired participants share the one mapped unit.
• E2 keeps every required signal member inside its actual complete anchor group. Writer provenance remains subject to the actual semantic attribution rules, but is not by itself a new strict canonical generator: the Appendix G causal relation → is expressly not a generator of ≺_J. O4 pre-update sampling avoids same-edge forwarding entirely. A future joint-update extension must independently audit every applicable normative order generator; neither attribute recording nor merging anchor groups may erase a required strict edge.
• E5 predecessor-message -> sync uses the same source interval/call identities. Its current-edge predecessors precede the anchor in the source realization constructed above. Sync -> suff_S uses the strict later-cycle issue/completion argument and cannot be reversed by preparation.
• E3 supplies the local issue obligations but creates no additional cross-interface committed-event generator merely from lexical order.
These schema checks establish both directions of generator translation, not merely implementation-to-source reflection. Ordinary whole-group erasure preserves the resulting equality by contracting identical paths on the two full graphs; hidden anchors are not ignored.
For the source graph, primitive FIFO/rendezvous realization, E1's forward completion, E2 internal grouping and the explicit E5 fence sequence provide J.SourceGeneratorRealization. All consumed ordinary generators point forward in the actual source completion sequence; each action completes one ordinary group with its checked internal atomic structure. Invoke J.SourceOrderWF with the exhaustive generator inventory and its prefix/restriction facts. Only then use J.SourceOrderReflection and J.LocalOriginCanonExt to establish well-formed implementation group order and equal extended canonical histories.
No new design-specific joint acyclicity premise is inserted. The finite execution order L is not itself treated as canonical order. Extra c-order, sampler or other generators are excluded by O3/O5, rather than dropped when this check encounters them.

Theorem J.OrdinaryConstructiveComposition (uniform OM/OA locality and Batch coverage).

Assume J.OrdinaryConstructiveProfile uniformly over the prospectively declared class C, J.OrdinaryLocalEvidence for every process and primitive, the J.OrdinaryGlue integration facts, one J.OrdinarySettlingEvidence path with QM, and O6/O7 actual carrier/environment conformance. Then J.OrdinaryRelation supplies the related bases and universally quantified actual-edge extensions consumed by J.ConstructiveEdgeInduction; the selected OM/OA Batch domain satisfies J.ReachableHistCov for every s∈C. With uniform Core.SourceCheckpointProfile it also satisfies ReachableHistCovOn with ChkRef witnesses. The construction discharges UCL1; the explicitly retained class, environment and carrier/projection bindings discharge UCL0/UCL2/UCL3. No independent design-specific global correspondence invariant is a premise. Proof follows.
Construct the base from the prescribed global reset state, whose source component states and owned channel/signal states are fixed by the same declarations. Local initialization laws establish r_P/r_c/r_z and G on those states. Owned finite source preparations concatenate by the preparation lemma. O6 binds an actual pre-completion base with empty selected history, and LP0 proves its preparation is silent; there is no unproved equality between independently chosen nonempty local initialization histories. Observable initialization on later edges is limited to the actually supported message/wait/paired-sync primitives and their complete anchor groups. O4 excludes START/FINISH primitive conventions from this first theorem; they are not treated as ordinary waits by naming alone. The environment relation supplies the corresponding base roots. Hence the base is a legal source prefix with the prescribed equal initial history. No independent existential global base relation is assumed.

Fix arbitrary s in C. Define R_s by Definition J.OrdinaryRelation. J.OrdinaryBoundaryCompose, J.OrdinaryUpdatePrepare and J.OrdinaryCanonicalExtension derive a finite exact-prefix source extension for every R-related origin and every actual next implementation edge, preserving all fixed past choices, restoring R_s and equal history. With identity designation the target is the actual post-edge state. This discharges the base/successor premises of J.ConstructiveEdgeInduction, through C0–C5 in OM and the corresponding A0–A6 cases in OA. Induction over finite actual ancestry yields origin coverage. This is not an assumption of R invariance over reachable global states: the preceding product and residue arguments prove it.

BM1–BM4, or AB1–AB3, supply prefix coverage. Every finite implementation prefix has the completed history of the prescribed base or last realized edge, including prefixes in silent intervals and terminal prefixes. J.BatchPrefixCoverage or J.LG-AnchorBatch gives

for every s in C and kappa in BatchReach_post(I_s), there exists lambda in Reach_ref(I_s) with H_s(lambda)=H_s(kappa).

This is J.ReachableHistCov on the declared Batch domain. The locally generated family R_s supplies the construction portion of UCL1. The prospective bindings, environment alternative and carrier/projection qualifications explicitly supply UCL0, UCL2 and UCL3; the theorem does not infer them from UCL1 alone. Uniformity follows because every local/primitive/interface/settling/carrier hypothesis is universally quantified over s and its admitted interface valuations. Fixing s was arbitrary; no sampled execution or successful constructor defines membership in C.

If Core.SourceCheckpointProfile is additionally proved uniformly, J.SourceCheckpointRepresentative replaces the exported source witness by its equal-history representative in ChkRef for the property conclusion. The inductive state still retains the exact constructed source prefix. SCP1–SCP3, including initialization phase and tail-history facts, are not inferred from ordinary channel topology. Supplying SCP does not lift O4's START/FINISH exclusion; a profile using those units first needs the missing primitive cases.

If the extensional source property is proved throughout the independently selected source domain for every s, J.SafeTransfer-HistCovOn/J.ConstructiveClassSafe transfers it. All admissible unresolved choices remain in that property domain. Complete-bucket monitor evaluation and extensionality are required when a monitor supplies property evidence; an executable monitor is not a premise of the pure correspondence theorem.

This establishes the conditional theorem J.OrdinaryConstructiveComposition for OM and OA.

J.OrdinaryEvidenceOwners (derived premise-discharge map).

C0/A0 — Evidence: LP0, owned primitive initialization, explicit environment relation. Derived: A common legal base and actual environment counterpart.

C1/A1 — Evidence: LP1/LP2/LP5, incident primitive fields. Derived: Matched complete roots and correct activation.

C2/A2 — Evidence: QF: existing Core.SourceL2Settle, or QSafe: safety-only J.SourceSettleRealization; QM matching and actual primitive selection on either path. Derived: Equal settled values, complete frontier evaluation and one coherent descriptor inventory.

C3/A3 — Evidence: FIFO/rendezvous equations, LP3, disjoint sync participants, signal primitive. Derived: Generic acyclic M-to-A order and jointly executable boundary.

C4/A4 — Evidence: LP4 and owned update/frame laws. Derived: Finite legal global preparation and product-relation preservation.

C5/A5 — Evidence: Ordinary occurrence keys and exhaustive reusable source-generator cases. Derived: Joint order well-formedness and equal extended canonical histories.

A6/BM/AB — Evidence: Actual instrumentation, complete groups, carrier conformance. Derived: Every finite Batch prefix is covered.

UCL1 — Evidence: All above local laws uniform over s. Derived: Uniformly assembled relation and local construction.
UCL0/UCL2/UCL3 — Evidence: Prospective class binding, environment and carrier/projection evidence. Derived: Uniform all-prefix reachable-history coverage.
ChkRef specialization — Evidence: Separate uniform SCP1–SCP3. Derived: Equal-history source checkpoint representatives.
Safety transfer — Evidence: Separate universal source-property proof. Derived: Implementation safety.
The checks are not all mere syntactic graph checks. Primitive realization, process refinement, observation instrumentation and environmental behavior require actual semantic evidence. The locality conclusion is that their global composition is proved generically, not that local evidence is cheap or automatically extracted. Remaining practical costs include the selected QF/QSafe qualification and QM matching, restricted external-environment correspondence when used, actual carrier/inventory conformance, and the separate source-property proof. They do not disappear into LP0–LP5 of J.OrdinaryLocalEvidence merely because composition is generic.

Scope and status of J.OrdinaryConstructiveComposition.

The proof permits arbitrary computation and communication topology inside OM/OA, but it has explicitly excluded shared endpoint arbitration, shared capacity, extra elaboration/order generators, shared application memory, dynamic reset, configuration-observation and R2C extensions, multiple clocks, same-edge forwarding, unsupported signal resolution, selected/anchor-bearing START/FINISH conventions and bases after unaccounted observable initialization. These are not newly declared illegal R/E designs; they are outside this theorem's stated ordinary independent-resource scope. No conclusion for the whole combined constructive profile follows solely from this proof.
For an admitted ordinary design, the result needs no additional application-specific cross-component invariant beyond G. It still needs LP0–LP5 of J.OrdinaryLocalEvidence, primitive refinements, the external-environment alternative when selected, operational settling and actual Batch conformance. There is no proof here that every R/E-correct implementation admits LP0–LP5 of J.OrdinaryLocalEvidence: an optimization relying on a whole-application input correlation may require a stronger local interface or a different proof route. That evidence-completeness question is separate from this composition theorem.
Framework status. These definitions and proofs establish a conditional paper-level composition theorem. Ordinary source completion uses Core.Phase, Core.RegSeq, Core.ReachPrep, Core.SyncFence and E4; anchor inventory and signal realization use RULE 1/2, Core.RegisteredAnchorBatch and J.AnchorInventory/J.AnchorBoundaryCompose; generator realization uses J.GeneratorCompleteness/J.SourceOrderWF. Per-process and concrete primitive implementation qualifications remain evidence inputs. No worked implementation, automatic generator, mechanization, measured cost advantage or complete combined-profile locality result is asserted here.

Definition J.ElaboratedLocalityProfile (ELM/ELA structural scope).

ELM and ELA are sufficient elaborated specializations of the constructive semantic safety route. ELM retains the message-only OM profile; ELA retains OA's supported anchor and pre-update signal regime. They replace only O2/O3's uncoupled boundary restriction and O5's ordinary-only generator inventory by EL1–EL8 below. All other ordinary restrictions, local-evidence disciplines, base/environment bindings and actual carrier/observation obligations remain. OM/OA are unchanged. Require Q.SafetyQualified(E), plus every qualification separately triggered by an invoked consumer; this definition does not assert that every Sys_B^elab(E) instance qualifies.
EL1 — Owned resources and supported primitives. Storage consists of separately owned explicit positive-capacity FIFOs and existing ordinary two-endpoint rendezvous primitives. Admit the qualified

MoveAB, request-fed JoinFire and FIFO-fed ForkABC schemas below in addition to ordinary endpoint operations. Shared capacity, hidden allocation, mutable shared application state, arbitrary rendezvous compounds, additional atomic templates and undeclared storage are outside this sufficient profile. A coupler is not a new modeled process. Every selected compound is already one declared source primitive with its complete atomic grouping; actions may not be merged to remove an order cycle or manufacture commutation. Overlap between AccountSupport_E sets qualified under Q.1's Accounting-support qualification does not by itself violate the shared-capacity exclusion. A fixed multi-effect representation using separately owned FIFO entries remains subject to the declared physical capacities, the admitted template laws and all applicable Q.SafetyQualified obligations; overlapping accounting views create neither additional physical storage nor an additional admission permission. This clarification does not admit capacity pools or dynamic allocation.
EL2 — Complete typed inventory. Each successful transaction declares its Pop/Push legs, attributed request participants, designated process-facing completions, fixed operand/payload functions, closed read/write footprint, projection/accounting effects and complete group identity. Boundary effects and process completions are separate inventories. Every actual boundary effect and returned NB result is represented exactly once. An internal transfer is not Commit(q) unless E designates it as q's process-facing release under Core.8. Failed NB returns emit no message unit. Template refinement must establish the exact actual source primitive and implementation effect, rather than treating a desired effect list as a source rule. The inventory is bidirectionally complete under Core.TransferLaw/Core.SyncProtocol on the matched snapshot: no forced physical effect is omitted and no physical effect is invented. Explicitly identify shared views of one occurrence without collapsing distinct effects merely because their coupler is stateless.
EL3 — Selection and storage legality. At the matched snapshot, at most one selected transaction uses each FIFO Push direction and at most one uses each Pop direction; no dynamic call is consumed twice. All legs satisfy beginning-of-cycle E4, with no empty fall-through or full-FIFO replacement. Every consumed head is an original stored token. Each selected transaction's full guard is true. The supported successful guard is a conjunction of a fixed gate, positive participating-request factors, input-nonempty tests and output-nonfull tests. Every request factor required by such a transaction belongs to its declared participants and is protected from unrelated retirement. Any additional mutable guard dependency requires a different qualified template and is outside this first result. Selection here is the complete set determined by the declared physical equations at the eligible edge, together with any separately legal abstract actions. Every template whose declared handshake holds is included. Prove that this entire forced set satisfies the cardinality and storage conditions above; a conflict is failed qualification and cannot be repaired by dropping one of the overlaps.
EL4 — Gates and operands. ELM admits constant parameters and already-qualified registered/process/environment values with locally proved frames fixing them throughout the message segment. Such values must already be available under the retained process, operand and environment contracts; this clause introduces no new direct-input or configuration protocol. ELA additionally admits gate values from its existing OA anchor/pre-update regime. Messages consume the bound pre-update value before any anchor update that changes it. A same-edge gate opening cannot justify new source-later issue or dependent computation. Any next-state gate update belongs to the owned LP4/anchor update proof. Stable operands and payload functions must implement the declared source semantics. Neither proof-only freezing nor combinational implementation authorizes an otherwise illegal computation. Dynamic reset and further configuration/R2C observation extensions are not supplied by this profile.
EL5 — Complete settling and activation. Establish J.SourceSettleRealization and actual snapshot matching for the complete combined graph, including absent/nonminimal requests, live NB attempts, all template and environment dependencies and cross-partition wiring. QF may reuse Core.SourceL2Settle

with its own premises; QSafe uses J.SourceSettleExists; QM matching remains required. Template acyclicity alone does not establish whole-network operational settling. Local evidence establishes the actual Core.16 activation and no-new-launch discipline; internal transactions consume only already represented admitted work. A stable value or stored token does not authorize new source-later evaluation or Issue behind an activated frontier.

EL6 — Enlarged local-evidence domain. Supply LP0–LP5 of J.OrdinaryLocalEvidence with the same restricted local vocabulary, extended explicitly to the incident primitives and outcome packets of EL1/EL2. In particular LP3 must prove actual source bookkeeping for empty internal-transfer packets, keyed compound returns, and mixed successful/failed NB packets; LP4 must cover every such admissible packet and actual owned register update, including empty packets, and construct finite legal preparation restoring the local relation. Disjoint call keys do not alone prove source bookkeeping commutation. An ordinary-only evidence artifact whose domain omits these packets is insufficient. Retain LP1 root/operand identity, LP2 activation/choices, LP5 primitive order keys, Core.RegSeq and Core.PreparationEvidence; no local precondition may assume the desired global source continuation. An empty or partial incident packet is admissible only when consistent with the full forced inventory for that snapshot. Local evidence cannot define away a sampled transfer by declaring its packet inadmissible.

EL7 — Closed required-order inventory. Admit the existing normative FIFO/endpoint occurrence generators; token-supply and capacity-release edges only where they are already generators of the selected FIFO/template semantics; the declared internal order of each indivisible atom; and, in ELA, the existing E1/E2/E5 anchor/interval generators, including message-before-anchor fences and strict later-interval issue. E3 retains its issue obligations and creates no extra cross-interface committed-event edge. Token provenance, resource arithmetic, physical serialization and a witness execution order do not independently create canonical generators. Every consumed group is operational/action-backed; additional order-only units, additional c-order rules, and nontrivial many-unit contractions are outside this sufficient profile. Bind the complete inventory on both carriers, including erased operational units and paths, and the same qualified whole-group E projection. No extra generator may be silently discarded. J.ElaboratedCanonicalExtension derives two-way translation and order well-formedness for this inventory.

EL8 — Instance and domain binding. Fix E, source operational primitives, decision laws, choices, environment, capacities, occurrence conventions and carrier prospectively. All local/interface evidence holds at every construction-reachable related origin and every actual successor there. Retain the existing base, phase, initialization, ordinary rendezvous, OA anchor and source-property restrictions. A sampled schedule or constructor acceptance cannot define the covered domain. NB cases additionally satisfy J.ElaboratedNBDecisionBinding; source legality, not witness annotation, determines outcomes.

Definition J.ElaboratedTemplateEvidence (primitive and integration evidence).

Supply reusable source-rule and implementation-refinement facts for each admitted primitive: exact guard/read/write footprint, atomic effects and frame, request/operand identities, designated completion packet, projection/accounting updates and EL7 generator schema. The integration glue checks ownership, wiring, endpoint cardinality, complete inventory and the common snapshot, extending the ordinary G facts to the declared legs. Q.1(a)–(c) state accounting and outer projection, complete-group metadata and all remaining Q.SafetyQualified obligations remain independently established inputs. They are not inferred from topology. The local evidence vocabulary cannot hide arbitrary cross-component invariants. The lemmas below derive simultaneous execution and assembled relation preservation from these facts; evidence must not assume C3 or equality of future histories globally.

Lemma J.ElaboratedFIFOResidue (owned queue updates).

Let Q_c be FIFO c's beginning-of-cycle contents. For any subset U of completed selected transactions, let d_c(U), p_c(U) ∈ {0,1} indicate its selected Pop and Push, and v_c the selected Push value if present. Define Q_c(U) = drop(d_c(U),Q_c) followed by ([v_c] if p_c(U)=1, else []). Initial selection requires |Q_c|>0 for a Pop and |Q_c|<B_c for a Push. When both are selected, 0<|Q_c|<B_c. Thus a same-FIFO Pop/Push atom cannot fire at capacity one.

Proof. No other selected transaction can consume an unexecuted transaction's input head or occupy its output slot in the same direction. An opposite-direction Push preserves the original input head; an opposite-direction Pop preserves or increases output space. Therefore its head value and FIFO premises persist after U. Applying its qualified atomic updates gives exactly the residue for U∪{T} and frames other queues. If one atom has both directions of a FIFO, its actual compound rule supplies that combined update. The bounds hold for every residue; no new token supplies a same-edge Pop. Final queue contents are independent of transaction visitation order. This proves the resource part only; legal atomicity, process bookkeeping and non-storage guards come from EL4/EL6 and template evidence. □

Worked template bindings for elaborated locality.

MoveAB. For distinct FIFOs a,b, the guard is n_a>0 and n_b<B_b, optionally conjoined with an EL4 gate. The declared atom removes the original a head occurrence u with value v and appends its transported occurrence/value to b. Its process completion packet is empty when both boundaries are internal. A concurrent Push_a preserves u and a concurrent Pop_b increases space; other selected Pop_a/Push_b actions are excluded by EL3. The residue lemma proves legality and the exact frame. A producer already completed at admission to a is not retired again. The empty packet still invokes LP4 for actual pipeline/register progress. An E binding that instead designates a process-facing release at this move needs the corresponding local completion evidence; it is not the empty-packet instance.

For serial staging of outer channel c into a then b, one illustrative accounting binding is B(c)=B_a+B_b and A_E(c)=|Q_a|+|Q_b|, with admission into a projecting to Push_c and delivery from b to Pop_c. MoveAB preserves the sum and transports the original token identity; admission/delivery change it by +1/−1. Empty initialization and these updates prove state-versus-history balance and FIFO order. Admission to a implies the total is below B(c); delivery from b implies it is positive. The network may block more often than an uncoupled capacity-sum FIFO, so this example does not establish every Q qualification or a liveness result. It does not introduce shared storage.

JoinFire. Two distinct attributed producer requests q1,q2 carry fixed values v1,v2 and feed one sole-producer output FIFO d. For fixed gate g and common declared epoch, the simple guard is G=r1∧r2∧g∧(n_d<B_d). The qualified source atom consumes those requests, appends F(v1,v2) once, and produces exactly E's complete atomic effects. This worked binding releases both q1 and q2 at the join, giving one keyed completion each and true status for an NB participant. It produces no consumer Pop_d completion. F is the declared source function, not new same-edge process computation. No competing Push_d is selected; a concurrent Pop_d only increases space. Participating requests, operands and gate persist, so the atom remains legal. If both requests belong to one process, EL6 still proves legal reach/issue and compatible bookkeeping; distinct endpoints alone do not do so. This primitive shape alone does not qualify a concrete source/RTL template; its declared source atom, implementation refinement and complete bookkeeping laws remain proof obligations.

ForkABC. For distinct FIFOs a,b,c, the guard is n_a>0, n_b<B_b and n_c<B_c, with an optional EL4 gate. The qualified source atom removes old head u with value v and appends f_b(v), f_c(v) to b,c as one complete group. Its fully internal completion packet is empty. A concurrent Push_a preserves u; Pop_b/Pop_c preserve output space. The residue lemma proves the atomic effect and frame. The source primitive must justify the functions and grouping. Partial or staggered branch completion needs explicit state and a separately qualified template. Two branch occurrences do not retire one producer

twice. E must justify the outer projection: two specified outer outputs may have separate constituent effects in one declared group, whereas duplicating a single outer event would violate Q.1. There is no universal fork projection.
Rendezvous scope. Only the existing ordinary two-endpoint rendezvous primitive is retained. Its matched requests, single atomic transfer, exact return packet and source bookkeeping use the existing ordinary local laws. The FIFO residue lemma supplies no proof for arbitrary zero-storage compounds; those are excluded here. FIFO admission/delivery around the internal move or fork may use ordinary NB endpoints under Core.NBDecision.

Definition J.ElaboratedNBDecisionBinding (operational outcomes and complete edge inventory).

For every represented NB attempt, derive the actual decision by Core.NBDecision from Core.NBDecisionSnapshot and the fixed resolutions bound by Core.SourceChoiceBinding, and match its call identity and result to the implementation. Success is paired with exactly one corresponding committed transaction; failure is one silent retirement and no transfer. Every attempt in the completed edge's return inventory is accounted for. An annotation, omitted selection, or later favorable handshake cannot supply a different result. These are local source-rule and implementation-refinement facts; snapshot matching and complete outcome inventory must be proved.
Ordinary FIFO endpoints retain the frozen-snapshot rule used by J.OrdinaryBoundaryCompose: a failed attempt is not recomputed after an opposite-end action. Thus an initially empty output PopNB can fail while an internal fork appends to that FIFO in the same edge; an initially full output prevents a fork Push even when its consumer Pops on the edge. Source primitive conformance, including the retained snapshot, is mandatory.
For the simple request-fed JoinFire NB case, establish as a template prerequisite that the actual q-side settled handshake and Core.NBDecision give success iff G, with every grant/resolution input explicitly included and fixed. In a completed edge containing these attempts, G true requires their successful atomic join and returns; G false retires each participating NB attempt with false and leaves blocking participants pending. A successful and a failed participant cannot describe the same join firing. Core.SelectLatitude remains subject to this law: discretionary nonselection cannot manufacture a false result or omit a required successful transfer from a completed return inventory. For blocking joins as well, the actual eligible settled handshake commits the full declared group under Core.TransferLaw; it cannot be skipped by a selector. This does not establish that missing joint support or a grant eventually arrives, and it adds no fairness premise to finite-prefix safety. A different coupled decision law lies outside this NB schema until separately proved.
Lemma J.JoinFailureFirst (failure-first realization for the simple join law). Before storage or gate updates, retire the failed NB attempts. A failed retirement removes only its own request and records its keyed false result. Removing request roots cannot turn the false positive conjunction G into true; remaining attempts retain their own request, operands and failure condition. No selected success consumes a failed request, by decision/transaction consistency, and EL3's complete positive footprint excludes hidden reads of a removed request by a successful transaction. Hence all selected-success guards persist. Ordinary frozen FIFO failures may be retired in the same initial segment by their existing primitive rule. EL6 supplies the actual compatible process bookkeeping. The following successful transactions remain legal by J.ElaboratedFIFOResidue and the guard frames. This proves the NB boundary ordering for the stated laws; absence-sensitive arbitration, mutable priority or additional unframed reads are not covered. □

Lemma J.ElaboratedBoundaryCompose (local realization and preparation).

For an arbitrary related origin and actual implementation successor, EL5 supplies the legal matched source SettlePoint and activation. Extract the complete finite message/return inventory using EL2/EL3

and the fixed decision laws. Matched roots and complete template equations give equality of every forced source/implementation handshake; EL2 supplies its exact effect/occurrence map. Execute the legal failed-return segment just proved, then selected successful transactions, then the ELA anchors after all applicable message predecessors. The message guards persist by the residue lemma, disjoint request consumption, fixed gates/operands and primitive frames. Ordinary rendezvous is one qualified action. EL6 supplies keyed completion bookkeeping. No new source-later work is launched. In ELA the existing OA pre-update samples and SyncFence facts justify the anchor segment; gate-changing anchor updates cannot back-enable a message in this segment.
For each transaction T let C_T be its finite partial map from dynamic call keys to return records; an internal move/fork contributes the empty map. Failed attempts contribute their singleton false-result maps. EL2/EL3 make the domains disjoint. The process residue for U records exactly the union of maps restricted to that process, using the actual LP3 steps or its equivalent canonical representation. This establishes exact-once attribution, not merely queue equality. The graph ordering failures before successful messages and messages before anchors is finite and acyclic and introduces no canonical generator. The queue, gate and packet residues provide C3/A3 for its downsets.
Finally apply the EL6 LP4 proofs to the exact final packets and actual owned updates. Their finite L4/preparation words concatenate by the existing owned-state and established-operand frames, as in J.OrdinaryUpdatePrepare. Internal token/projection records update through their declared local rules and the qualified accounting invariants. This restores the assembled product relation without an arbitrary application-specific cross-component invariant. Retain the existing guarded phase-entry and terminal-export rules; no future cycle or preparation is invented. □ The final segment contains the entire forced set, so source edge completion does not depend on discretionary omission. EL3 rules out conflicting forced effects as an interface-qualification failure before this construction is invoked.

Lemma J.ElaboratedCanonicalExtension (closed generator composition).

Extend the exact old occurrence map by the realized ordinary/template groups, using fixed E identities, participating call keys, channel counters and old-token occurrences. Complete inventory and atomic grouping give a bijection of full matched operational groups with equal constituent values/internal structure; failures extend only causal return records. EL7 fixes the exhaustive generator schemas on both carriers. Matching channel occurrences, token links, interval keys and primitive group structure translates each normative generator in both directions. Q.1(c-order)'s outer reflection alone is not that two-way proof. Preserve every old identity and old-old edge, and include paths through erased operational actions.
For joint acyclicity instantiate J.CycleRankOrderWF and the message/anchor portion of J.StandardCycleRank on every consumed whole group. Same-direction distinct FIFO occurrences have distinct cycles by endpoint cardinality. Old-token suppliers are strictly earlier than their consumers. Any normative capacity-release predecessor needed for beginning-of-cycle space lies in the past: a same-edge Pop cannot justify a Push into an initially full FIFO. Thus these admitted cross-group FIFO/template generators advance the cycle coordinate. In ELA, the only same-edge cross-group generators are existing message-before-anchor fences, which advance the class coordinate; E1 and later-interval E5 edges advance cycles. Declared constituent order is checked inside its atom and does not split that atom into extra units. Ordinary rendezvous is one unit. Local tie-breaking is proof data only. This uniform incident-generator check covers the union, including erased groups, rather than inferring acyclicity from separately acyclic components.
Occurrence and primitive-order rules also establish predecessor completeness and restriction stability: new successors may cite old predecessors, but extension neither creates nor deletes edges between old occurrences. Both directions are proved for every applicable generator schema, with the old projection/group records unchanged. Apply the same qualified whole-group E projection to both

matched full graphs. Nonempty strict paths between distinct retained images correspond, including erased intermediates. No additional many-unit contraction is admitted. J.SourceOrderWF/J.SourceOrderReflection and J.LocalOriginCanonExt then yield well-formed retained implementation order and equal extended canonical histories. The rank lemma supplies only acyclicity; the just-stated prefix, translation and projection facts remain essential. Additional c-order or order-only support requires a different proof and is not an omitted case of this lemma. □

Theorem J.ElaboratedConstructiveComposition (ELM/ELA locality and Batch coverage).

Assume EL1–EL8, J.ElaboratedTemplateEvidence, J.ElaboratedNBDecisionBinding for admitted NB cases, and the explicitly enlarged LP0–LP5 domain, uniformly over the declared class C. Retain the corresponding ordinary ownership/integration facts adapted to the template legs, complete operational settling with QM matching, O6/O7 base/environment/carrier bindings, and the applicable message or anchor Batch conformance. Then the assembled local relation gives the bases and every actual-edge extension required by J.ConstructiveEdgeInduction, deriving C0–C5 for ELM and the applicable A0–A6 for ELA. Consequently J.ReachableHistCov holds on the declared Batch domain for every $s \in C$. No independent design-specific global correspondence invariant or certificate package is a premise.
Proof. Initialize the product of local process, owned resource, environment and occurrence relations from the common prescribed base, using LP0 and the qualified E accounting/projection initialization. The inherited O6 restrictions prevent unaccounted observable initialization. Thus the initialized relation holds at every prescribed implementation base, and LP0's silent preparation together with O6 and the qualified E accounting/projection initialization establishes equal initial canonical histories. EL6 gives matched complete roots and operands; EL5 gives the operational snapshot and activation. J.ElaboratedBoundaryCompose derives the exact finite source boundary extension and owned preparation restoring that product, with all old choices/prefixes retained. J.ElaboratedCanonicalExtension supplies independently justified canonical equality. These discharge the constructive base/successor contracts for every actual successor. Induction over actual finite ancestry gives origin coverage; the retained BM/AB complete-inventory conformance and existing Batch lift cover every finite prefix by its last completed edge or base. Uniform quantification supplies UCL1 with UCL0/UCL2/UCL3's independent bindings. A uniformly proved Core.SourceCheckpointProfile permits the existing ChkRef representative specialization; it is not inferred from elaboration topology. Independent source-property evidence then uses the existing safety-transfer theorem. □
Scope and evidence status. This is a conditional paper-level locality theorem for the stated schemas, not a qualification of every Q elaboration, a concrete HLS/RTL flow, or an automatic evidence generator. Primitive refinement, enlarged local process evidence, whole-network settling/matching, Q.SafetyQualified projection/accounting, environment, carrier and source-property proofs remain substantive inputs. Ordinary and elaborated local evidence may be reused uniformly when proved over the admitted class. No shared-capacity theorem, general CS2 insertion or certificate-package coverage is supplied. A K.CT use may obtain internal-fact correspondence through J.RegisteredConstructiveFactCorr only after separately discharging its full-signature local fact and carrier obligations; this composition theorem alone supplies neither those obligations nor completion. The broader constructive contracts remain available outside ELM/ELA with their full premises. In particular, atomic process-facing paired-Pop retirements, internal joint moves consuming one old head from each of two ingress FIFOs, and simultaneous-replacement staging shifts are not added to ELM/ELA by the Q.CT profiles. Their use follows the general constructive route with complete C0–C5/A0–A6 evidence and all remaining global premises. J.ElaboratedFIFOResidue remains scoped to its ordinary old-state E4 FIFO primitives.

Definition J.ConstructionAction and operational footprints

Fix a finite RTL_B prefix τ_post and declared local source-realization evidence L. Act_L(τ_post) is the finite set of primitive source-construction actions named by the actual source primitive instances, finite preparation segments and complete observation/return inventories. An action is one of: (A1) a process-owned ε evaluation/control transition, including outcome-independent continuation past a carried NB hole; (A2) assertion or retention of an endpoint-owned blocking request, or assertion of a one-cycle attributed NB attempt; (A3) a shared-boundary NB decision/return that reads the declared settled snapshot, writes NBResult(q), retires NBPending(q)/the attempt, and on success is contracted with the corresponding transfer commit; (A4) any other complete explicit-channel or observable atomic-unit commit; or (A5) a selected proof-significant environment, reset, or explicit Sys_B^elab operational transition not already classified as A1–A4. Every A5 occurrence carries a checked PhaseRole_A5 in exactly one of three construction classes: PreL1, for a cycle-entry/environment/elaboration prerequisite that must execute before the Horizon's process-owned L1 actions; L3Boundary, for a phase-L3 environment/elaboration boundary action whose guards/inputs are compatible with the already frozen Horizon snapshot; or ResetBoundary, for the atomic RESET transition represented by that cycle's resetUnits occurrence, ordered by RW1–RW5 after every affected-scope ending-epoch predecessor and before L4. ResetBoundary is an L3-owned boundary-tail class for Appendix-J construction; it does not reopen L1/L2 and does not permit fresh affected-scope new-epoch work in the same Horizon. The ordinary next-cycle EnvStep component of CycleResult.nextState is cycle-completion state evolution rather than an A5 construction action, and ε/combinational work belonging to →ε,L2 is realized by the unique L2 settle rather than inserted into Act_L as an A5 vertex. A complete commit at an explicit Sys_B^elab abstract channel is A4 unless a separately declared atomic contraction gives it another already typed action identity. Thus a selected external/elaboration occurrence is A5 only when its operational semantics supplies a distinct proof-significant occurrence with one of the three PhaseRole_A5 classes. A3 does not write the ordinary control/operand locations used by certified same-Horizon outcome-independent actions; dependent continuation is an A1 action at a strictly later Horizon. Every action has a typed identity, owner set, reset epoch, Horizon, associated unit, semantic stage, local ordinal, and finite ReadSet/WriteSet over a normative location vocabulary. That vocabulary distinguishes at least Ctrl(P), Local(P,x)/Operand(P,e), NBResult(q), NBPending(q), endpoint request and Push-payload locations, each explicit channel/elaboration location, environment locations, and reset locations. Certificates may realize these locations as components, fields, or lenses, but shall map them to this common vocabulary and prove the corresponding disjointness/frame laws. The vocabulary is intentionally finer than a monolithic “process-local state” component. Custom Sys_B^elab components require local footprint/commutation and well-founded ordinal certificates. Construction-action occurrence convention for persistent blocking requests: the fact that an already-issued blocking BoundaryReq level remains asserted in a later cycle does not by itself manufacture a fresh A2 action in every later Horizon. Act_L contains an A2 occurrence when the selected Sys_ref operational construction actually asserts or performs a modeled retention/update occurrence for that request; otherwise persistence across a later cycle is carried as request state plus Core.IC and the local persistence/frame evidence. If a concrete source semantics does model a per-cycle retention transition, that transition is a distinct Horizon-indexed A2 action and must be inventoried and replayed normally. Back-to-back transfer to a new dynamic owner is always a new A2 assertion occurrence. Thus action generation follows operational occurrences, while Core.IC separately constrains the persistent request level.

Definition J.ProofUnitConstructionSupport (proof-unit to construction-order support)

Fix the complete unit inventory and declared source-action realization. Each unit has action-backed support (CS1): one actual action or one already-declared atomic group realizes it; or explicitly qualified

order-only support (CS2): it has no operational transition but carries its actual identity and every incident normative predecessor rule. An order-only unit is never assigned a fictitious source action. Any path contraction preserves all normative edges, atomic structure and required order; any nontrivial grouping separately proves J.RequiredOrderWF. The ordinary OM/OA and elaborated ELM/ELA profiles admit only their stated action-backed inventories. A broader proof with order-only units must establish their occurrence, prefix closure, strictness and complete two-way canonical translation independently. This support vocabulary introduces no certificate-construction order or new scheduling permission.

Lemma J.BufSameCycle (same-cycle buffered Push/Pop sequentialization)

Let c have B(c)>0, let the pre-cycle logical FIFO contents be Q with n=|Q|, and suppose one Push_c(v) and one Pop_c(w) both commit in the same RTL cycle under the non-fall-through E4 semantics. Then 0<n<B(c) and w=head(Q). For an ordinary uncoupled primitive buffered boundary, the common L2 snapshot also establishes both endpoint enablements, and the two FIFO state updates may be proof-sequentialized in either order: Push_c(v);Pop_c(w) and Pop_c(w);Push_c(v) terminate with logical contents tail(Q) followed by v, occupancy n, the same producer and consumer ordinals, and the same unordered same-cycle observable bucket. For a coupling-sensitive boundary, occupancy alone is not an enablement proof: the selected primitive realization shall either provide a same-snapshot commutation certificate showing that the two already-selected L3 actions may be sequentialized without re-evaluating the intervening MirrorAvail_E value, or use an atomic boundary-action unit already required by the declared source primitive. If n=0, E4 legality forbids the Pop from being created by the same-cycle Push; if n=B(c), E4 legality forbids the Push from being created by the same-cycle Pop. For B(c)=0 the endpoints remain one indivisible rendezvous action and this lemma does not apply. Proof: E4 gives the interior-occupancy necessity; the ordinary case uses the FIFO update equations, while the coupling-sensitive case uses the stated same-snapshot commutation/atomicity certificate. ■

J.2c Internal-fact correspondence for completion consumers

Definition J.InternalFactCorr. Fix Core.EvidenceHdr and an explicit required fact signature F. F names every internal and external committed operation/result/observation fact consumed by the workload proof, its logical keys, values, endpoint/epoch ordinals, atomic-unit membership and any retirement-to-transfer attribution. For every finite implementation prefix in the independently declared claim domain, provide a legal finite prefix of the same Sys_ref whose F projection matches it under the fixed attribution map, with no omission of a required internal fact. Preserve the selected units' atomicity and required endpoint order. Failed NB outcomes or other non-transfer observations belong to F whenever their values/control effects are consumed and must be covered by the chosen correspondence. Initial facts are bound separately. No output-only projection may stand in for this complete signature.
Prospective signature and consumed facts. In J.InternalFactCorr and its completion consumers, F denotes the fact signature declared for the bound theorem instance and Core.EvidenceHdr: its key families and membership rules, value/result schema, atomic grouping, retirement-to-transfer association and attribution convention are fixed before and independently of the selected workload run and correspondence-constructor success. A declaration may cover families of potential dynamic occurrences without individually enumerating all future occurrence keys. Distinguish this fixed declaration from both the actual F ledger at a prefix and the finite inventory of facts consumed by a workload certificate. Those inventories may grow when further actual completions or historical support are consumed, but every such fact must already belong to the declared F under the same attribution. Predeclaring a possible completion does not assert that it occurs, place it in a prefix ledger, or make it a W member.
Changing the declaration. A genuinely different signature or attribution is a new theorem-instance binding, not an in-place extension of the old F. Bind a new Core.EvidenceHdr and instance and establish

the same prospectivity requirement for the workload application; choosing a new header after inspecting the selected run does not by itself establish that requirement. Establish J.InternalFactCorr over the new full signature and unchanged operationally admitted implementation-prefix domain for an extension of the same completion claim, retaining all applicable adapter and carrier premises. Re-establish K.CT.Facts, K.CT.Agreement, provenance/availability and selector-support obligations, initial roots, occurrence/group/order/retirement attribution, and the legal workload witness and its consumed support under that binding. Prior evidence may be reused only where its proved validity domain and recorded dependencies cover the new binding, or through an explicit preservation/transport proof; otherwise revalidate the affected obligations under Core.EvidencePackage and M.7a. Rebinding neither narrows the implementation domain nor supplies missing facts or agreement.
The relation supplies history/fact correspondence, not equality of uncommitted request signals, current storage microstate, enabledness or fairness. A resource or exposure argument consuming any such state fact must supply its explicit conformance invariant. Independently selected source prefixes suffice for conditional value agreement; a consumer needing nested prefix extension or one coherent infinite source execution must establish that stronger property separately.
J.InternalFactCorr semantic adapter. A qualified J.ReachableHistCov construction may supply the interface only if its covered history and annotations actually include F and the required atomicity/attribution. Outer observable safety or J.SafeTransfer-HistCov alone is not evidence for an omitted internal fact. The existing Q.SafetyQualified consumer rule and every invoked theorem's premises remain in force. J.ConstructiveFactCorr supplies the explicit local-construction adapter below; its full-signature inventory and exact prefix proof are additional to an outer-history conclusion.

Definition J.ConstructiveFactEvidence (CF0–CF3; local realization of the required fact signature).

Fix F, the independently declared implementation domain, attribution and initial facts of J.InternalFactCorr, and a qualified constructive exact-prefix construction over Core.ConstructiveOriginCarrier. Bind the same Sys_ref, capacities/elaboration, environment/testbench contracts, choices, reset interpretation and legal source relation throughout. Use the applicable C0–C5 and, for an anchor construction, A0–A6 contracts; another constructive extension must establish its own corresponding local cases. The F ledger of a prefix is intrinsic to its actual completed operations/results/observations under the fixed attribution, not a record that a constructor may fill independently of execution. Source and implementation ledger equality preserves the F fields and required order; it does not equate unrelated physical clock indices or order independent units by their listing order.
CF0 — Bound bases and stable identities. At every prescribed implementation base construct the legal source base already required by the selected local contracts, and prove their F ledgers and separately bound initial facts agree. Fix the typed occurrence correspondence, endpoint/epoch ordinals, atomic membership and retirement association prospectively. Preserve the old correspondence literally along each constructed extension. Its keys are tied to actual operations and source rules, not renamed to fit an observed value or a chosen workload witness.
CF1 — Complete local deltas. At every construction-reachable related origin and for every actual successor edge, enumerate all newly completed F facts from the actual implementation transition, including selected internal/erased operations and consumed failed NB outcomes. Bind each to the C1/C2 roots, descriptor, outcome packet or applicable anchor/extension record that actually produces it. Show the inventory is exhaustive and has no duplicate occurrence. Classify all actual completion cases by their operational rules, not by equality with the eventual constructed history; every other case must be proved unable to emit an F fact. Primitive evidence fixes the legal source counterpart and attribution of each selected fact. Capturing a token, transferring an internal token and retiring a source call are distinct unless the declared operational rule identifies their completion.

CF2 — Local source realization and frame. For each primitive/action and process-bookkeeping case in CF1, prove from its source rule, the related local state and fixed snapshot that the C3 boundary segment or C4 update realizes exactly the attributed fact fields and values, with the required whole atomic unit and endpoint/epoch order. In the anchor case use A1–A4/A6 for the actual samples, sync participants and complete anchor membership. Prove the local frames preserve every previously realized F fact and attribution. Cover all steps of the legal source extension, including environment steps, designation and finite preparation: any step outside the enumerated cases emits no F fact. These local emission/frame laws and complete inventory derive equality of the appended F deltas; equality of future full histories is not a substitute for them. No outcome-dependent continuation may use a return before Core.RegSeq permits it, and no grouping or serialization chosen by the construction creates a new source action or required order.
CF3 — Exact finite-prefix lift. Establish that every finite implementation prefix in the claimed domain has the F ledger of a covered base or actual origin, preserving all of F rather than only its outer projection. Core.RegisteredFactPrefix supplies this clause under Core.RegisteredFactBatch on the same carrier and signature. Alternatively, a finer carrier may supply a separate exact lift that constructs a legal source prefix with the matching F ledger for every such partial prefix from the qualified local evidence. Neither ideal-prefix-closed safety nor a completion of the unfinished implementation cycle supplies this exact match. Any source-prefix truncation or checkpoint substitution must separately preserve F; the constructed legal prefix itself needs no such substitution.

Theorem J.ConstructiveFactCorr (constructive internal-fact correspondence).

Under J.ConstructiveFactEvidence and all premises of its selected constructive local construction, J.InternalFactCorr holds on the declared implementation domain for F. Proof. At each base, CF0 gives a legal source prefix and equal fact ledgers. Induct on the actual finite edge ancestry. For the actual next edge, the local construction supplies a finite legal extension of the very same source prefix, retaining its old choices and occurrence identities. CF1 inventories the actual new implementation facts. CF2's primitive and bookkeeping laws match each to the corresponding source occurrence, value and complete atomic unit; exhaustiveness and the no-extra-emission cases give the reverse inclusion. The frame laws retain the old ledger and attribution. The local endpoint/epoch-order and group laws supply the structure required by F without inventing a total order among independent facts. Hence the extended ledgers match at the next actual origin. Apply CF3 to an arbitrary finite implementation prefix: substitute its covered base/origin ledger, or use the separately proved exact partial-prefix lift, to obtain a legal source prefix with that F ledger. This is precisely J.InternalFactCorr. □

Corollary J.RegisteredConstructiveFactCorr (message and anchor specialization).

For a qualified J.LocalOriginContracts message construction or J.LocalAnchorContracts anchor construction, bind Core.RegisteredFactBatch to the same actual carrier, instance and F, and establish CF0–CF2 by the complete local primitive, result and anchor cases. Then J.ConstructiveFactCorr applies: Core.RegisteredFactPrefix discharges CF3, and the respective local constructive theorem supplies the legal exact-prefix base/edge construction. BM1/AB1 already place their declared completions at the edge; FB0/FB1 check that this inventory contains all consumed F facts, while FB2 supplies the additional F-preserving silent/designation claim. C1/C2 supply keys, values and selected outcomes; C3/C4, or A1–A4/A6, provide the source realization and frames required by CF2. These facts do not follow from outer-history equality alone. The OM/OA and ELM/ELA composition theorems may supply the local construction within their stated profiles, with the same additional CF0–CF2 and FB obligations; this corollary neither broadens their primitive inventory nor qualifies a new implementation by its name. A parameterized proof of these obligations yields the same conclusion uniformly over a prospectively bound stimulus class. No finite set of executions establishes that uniformity. J.StableReadFactCorr and

J.ConfigRecordFactCorr below give the separate direct-input and configuration specializations, with their additional full-signature premises.
Corollary J.StableReadFactCorr (reset-stable direct-input full-fact lift). Fix an original instance I_θ and reduced body instance $\bar{I}_\theta$ satisfying SD0–SD4, the same prospective consumed signature F and original legal source relation. Bind θ at the genuine SD0 initial anchors as a common initial root, with legally matched initialization, fixed environment choices and no later configuration update or affecting reset. When used by K.CT, this root belongs to the common Core.FinitePrefixProvenance domain for both the workload witness and constructed prefixes, including every dependent CtrlSupport decision. Root binding neither asserts execution of a conditional Read nor exempts actual source preparation from Core.RegSeq/Core.16.
StableRead full-F premises. Establish J.ConstructiveFactEvidence for the reduced body's full consumed body signature, using J.RegisteredConstructiveFactCorr when its carrier qualifies. Strengthen the SD2/SD4 reduction and source lift to a prefix-preserving correspondence for that entire signature: each actual original edge and silent/designation segment maps to the reduced carrier with the same completed internal/external facts, keys, values, order, groups and retirement attribution; every constructed reduced source extension lifts from the same original source prefix with exactly those facts and no extra emission. Include erased internal Push/Pop retirements and every consumed non-transfer NB result, using the actual decision law. Prove these properties by the local substitution and frame cases, not by equality of the outer message projection. For initialization prefixes, establish CF0 and an exact legal source-prefix match for all selected initialization facts; retain that immutable ledger through the body lift. A selected ordinary Read fact must have its actual legal occurrence and exact-prefix proof; θ alone does not create one. Establish FB0–FB2 on the original carrier, including full-F silence and designation transport, or supply CF3's separate exact partial-prefix lift.
StableRead conclusion and proof. Under these premises, J.InternalFactCorr holds for F on every admitted finite prefix of I_θ. Initialization uses the qualified base-prefix case. For a body prefix, reduce its full ledger, apply the reduced constructive fact construction, then lift its legal source prefix and ledger back to the original source while retaining the matched initialization facts and choices. The full-F frame preserves earlier facts and the exact carrier lift covers intervening prefixes. This instantiates CF0–CF3 and J.ConstructiveFactCorr. Uniform premises give a uniform result over a prospectively declared θ family. SD0–SD4's message-safety theorem alone does not supply the strengthened premises, and neither this result nor root equality supplies request exposure, service or timing-independent agreement of arbitrary NB decisions.
Corollary J.ConfigRecordFactCorr (configuration-bearing anchor fact correspondence). Select CO0–CO4, CVG0–CVG3, the J.LG-ConfigBatch local construction and its Core.RegisteredAnchorBatch carrier with CO3/CO4 inventory. Fix F before execution, including each consumed committed $\mathrm{Config}(\kappa, D_\kappa, \theta_\kappa)$ record, any selected CO1 initialization records, and all other consumed message/anchor/internal/result facts. Preserve the distinction between separately bound initial roots and completed records. Discharge FB0–FB2 for this full inventory, not merely CanonHist_cfg. The local fact proof binds actual initial records and roots (CF0); inventories each actual message, result, ordinary anchor and CO1/CO2 record exactly once, including zero-length intervals and unused designated fields (CF1); and realizes those records and every other consumed fact through the same legal source extension with complete atomic membership, fixed identities, frames and governing-use attribution (CF2).
ConfigRecord proof and consumer binding. J.ConfigInventoryAndValues and CVG0–CVG3 supply causal record values, original-expression substitution and update isolation; A0–A6 and the separately established internal/result cases supply the joint source realization and frame. CO3/CO4 preserve the complete synchronization group; late samples and untaken source reads create no new records. FB0–FB2 give CF3 through Core.RegisteredFactPrefix. Thus J.ConstructiveFactCorr yields J.InternalFactCorr for this configuration-bearing signature on every admitted finite prefix. For K.CT, a consumed CO2 record is

produced by its actual synchronization commit unit in W, strictly preceding each dependent target or CtrlSupport decision's target under Closure, with recursive provenance support included; a record already present at the bound base is instead discharged by the matched base facts. CVG0–CVG3 bind each use to its legally governing record across updates, without clearing channels or renumbering transfers. K.CT.Agreement additionally establishes common provenance/availability over the workload witness and constructed prefixes; record correspondence alone does not identify timing-dependent configuration choices across schedules. All other closure, preparation, progress and service premises remain. This corollary does not qualify dynamic reset, R2C, new synchronization forms, or a different retained-Read alphabet.

Completion-consumer scope. K.CT.Facts may use J.ConstructiveFactCorr and J.InternalFactAgreement. Its workload witness must still lie in the common Core.FinitePrefixProvenance domain, and all K.CT closure, preparation, exposure, capacity, B.CT/Q.CT conformance and service obligations remain. The adapter proves correspondence of completed facts, not that a missing fact becomes exposed or commits, and not agreement of timing-dependent NB or signal choices across source schedules. Dynamic reset, configuration, R2C or other extensions require complete local fact and carrier cases for their own inventory; safety qualification of those features does not supply completion preservation of outstanding obligations. A physical resource fact not represented by a legal source operation remains an explicitly proved conformance invariant. The named stable-direct-input and configuration-record adapters discharge only their stated fact cases; extensions outside those scopes still require their own complete local and carrier evidence.

Lemma J.InternalFactAgreement. Suppose J.InternalFactCorr holds, its source witnesses lie in the Core.FinitePrefixProvenance domain, and a fixed legal workload witness lies in that same domain. Any implementation F fact also realized in the workload witness has the same typed value and attributed identity as that witness fact. Proof: use InternalFactCorr to obtain the corresponding legal source fact, apply Core.FinitePrefixAgreement to it and the workload fact, then transport through the fixed attribution. Endpoint order follows from the bound channel/order laws. This lemma does not establish that the missing fact ever occurs; the completion consumer must prove exposure and service.

Lemma J.LogicalPayloadRefinement (causal representation and fact attribution)

Fix the same instance, reference and complete boundary/order inventory as the selected constructive route. Assume Core.LogicalPopPayload and the representation/operational clauses of Q.CT.LogicalPayload, including Q.CT.LogicalPair for a declared pair, and provide the remaining local source-preparation, resource, settling, environment and frame cases for that reference. Then causal admission/ownership supplies the logical Pop payload and legal-later-use cases of that construction without requiring a decoder of current physical registers. With the independently complete CF0–CF3 and applicable FB0–FB2 carrier cases, the same interpretation is available to J.ConstructiveFactCorr. This lemma is a local discharge, not a replacement for C0–C5 or a qualification of an arbitrary controller.

Proof of causal coverage. Extend the related state by the admission ledger and location-owner map. At empty initialization both are empty. For any actual successor, its selected admissions and old-owner movements uniquely update this observer; the ordered one-owner invariant supplies every required old item and prevents duplication or substitution. A terminal retirement selects its existing owner, even on simultaneous replacement. No observer value participates in choosing the physical successor. Thus every admitted physical execution lifts from the base, including all stalled and unfair prefixes, and erasure returns that exact execution. Previously bound admissions, identities and source prefixes are retained literally; the observer cannot use a later event to choose an earlier value. In the paired case apply the induction to each disjoint input ledger. A joint retirement selects the two old owners of the same iteration; the Q.CT.LogicalPair invariant prohibits a split or mismatched pair. For coupled paths, Q.CT.CoupledInputStaging preserves aligned occupancy and ordinals by matched old-owner moves. For

Q.CT.StaggeredInputStaging, retain A-only prefix ownership until the late read and then paired suffix ownership; equal full-path occupancy is not assumed. A joined physical cache retains both distinct logical owners even when neither value is recoverable from current registers.
Proof of payload and source use. The Q.CT.LogicalPayload induction gives the physical representation of the old terminal owner j. Core.LogicalPopPayload assigns its actual retirement the logical value x_j fixed at admission. This is the head value in the selected reference's FIFO order. The legal source boundary segment therefore returns x_j at the declared call; its already-qualified later preparation computes the source expression and, by the representation/use law, matches the physical descriptor. Private computation does not discharge preparation legality or advance source evaluation through a persisting frontier. Explicit storage movements retain their own source action realization. For a pair, the legal boundary segment returns (x_j,y_j) at the two declared keys as one atomic unit. Each is the head of its own reference channel. The separate terminal equations and H, or the joined-cache induction of Q.CT.PairedRepresentation, prove that the actual held result equals the declared source expression S(x_j,y_j); source attribution and any dependent Issue still occur at a legal later Horizon. Same-edge private combination is not same-cycle source use of a return.
Proof of order and full facts. Ownership and event cycles preserve the selected endpoint, token-supply and capacity-release facts; the remaining two-way generator translation still proves that no required generator is omitted or invented. Projection is applied to events already carrying their declared logical values and cannot relabel them. At each edge, the message fact delta includes the actual admission values, the logical values of selected internal Pops and the physically committed output values, each once at its attributed key. Observer bookkeeping and still-unattributed private arithmetic create no completed source facts. If the consumed signature additionally exports a legal later source-evaluation or other non-message fact, its complete local emission and carrier case remains required; it cannot be erased by this representation argument. Complete CF emission/framing cases and the selected exact-prefix carrier then give the stated full-fact conclusion. Neither outer-history equality nor the mere availability of a ghost ledger establishes those carrier cases. For the pair the fact delta is one indivisible packet containing both distinct Pop member facts, each once. Neither source carrier nor observation projection may expose an unauthorized half. Group contraction, complete generator translation and the exact full-F carrier cases remain required; same-cycle placement alone does not establish atomic-history equality.
Safety and completion scope. The observer/representation proof uses no service or workload witness. It therefore supports the safety correspondence independently of K. For K, the logical fact values are fixed before comparison with the witness; agreement cannot choose or repair them. Actual request exposure, legal source witness construction/capacity lifting and B.CT/Q.CT service remain separate. A reference with different downstream readiness requires its own local cases even when the same admission-to-Pop matching is used. The paired safety construction uses the same declared source atom. A property established on an ungrouped source requires a separate group-erasure/inclusion proof for its observations, order and admissible cuts; that reuse is not inferred from two Pops being selectable in one cycle.

J.3 Causal Dependency and What System-Level Compatibility Guarantees (Informative)

The system-level result of Appendix J establishes compatibility of the aggregate observable behavior, i.e. $\tau_{ref,system} \approx \tau_{post,system}$ over the alphabet Σ of channel operations, synchronization events, and signal Read/Write actions.
This compatibility is intentionally *observational*: it constrains what an external environment can distinguish at Σ, not internal RTL micro-timing.

Appendix J preserves the selected canonical unit identities, values, required order and atomicity. The explanatory causal-dependency graph is not automatically an extra canonical generator or the induction order. The proof induction is over actual finite edges; J.RequiredOrderBasis and the independently qualified extension rules determine the history quotient.

Accordingly, the system-level theorem implies the following required-order preservation principle:
• If two observable units γ_1 and γ_2 are ordered by the required-order relation $\gamma_1 \prec_J \gamma_2$, then the matched Sys_ref and RTL_B executions preserve that required order. This follows because each elementary $\prec_J$ generator is preserved by the corresponding rule: synchronization source order by E1; signal-anchor timestamping by E2; issue-side and request-attribution obligations by E3; channel FIFO legality and rendezvous atomicity by E4; immediate synchronization-boundary constraints by E5; and paired-barrier semantics by the explicit synchronization well-formedness conditions. In particular, for each explicit abstract channel c, the matched executions preserve the committed Push_c/Pop_c history: for B(c)=0, a committed transfer is one explicitly atomic rendezvous Push_c(v)/Pop_c(v) unit; for B(c)>0, within each reset epoch of c, the k-th committed Pop_c consumes the k-th committed Push_c of that epoch, under reset-empty, FIFO-order, no-underflow/no-overflow and non-fall-through E4 semantics. An affecting RESET may discard unmatched stored Pushes without producing Pops; those discarded messages have no later consuming Pop. This epoch-local consumption pairing does not renumber Core.17's canonical cross-model occurrence identities.
• If two observable units are not ordered by $\prec_J$ and are not members of the same explicitly atomic unit, then their relative cycle placement or interleaving is not constrained by the methodology, and differences are treated as incidental latency variation. A designer who needs a particular cross-process ordering to be preserved must express it through an explicit Σ-visible mechanism such as message-passing FIFO order, a rendezvous/handshake atomic unit, explicit synchronization, or signal-anchor semantics.

The "single testbench / no surprises" interpretation requires actual testbench correspondence and a property extensional on the selected history. Under those premises, every covered RTL history has a legal matching source history. A universal source property, or the explicitly qualified covering-run alternative, therefore transfers.
This is source-history coverage of implementation behavior. It is not an assertion that every source behavior is realized in RTL or that a particular finite source pass alone proves safety. K.CT separately derives eventual acceptance of its required workload under the fact and progress contracts.

Appendix K – Required-Workload Completion

K.CT Required-workload completion

Scope. K.CT is the workload-relative completion theorem. It consumes only the common operational rules and the correspondence, agreement, closure, exposure and service premises stated below. It does not claim absence of every internal wait cycle or progress after the required workload has been accepted.

K.CT.Instance and acceptance contract

Bind Core.EvidenceHdr and fix a source reference Sys_ref, its implementation, initial state, stimulus, modeled testbench, epoch and exact operation/endpoint boundary map. Use the same Sys_ref as the

applicable Appendix J correspondence. Let R be a finite, prospectively specified collection of required testbench acceptance occurrences, with their logical keys, values and per-endpoint order. R denotes acceptance, not merely assertion or a DUT-side write. The testbench and required environment behavior are part of the bound system; their actual behavior and continued service must satisfy the same declared instance contract. No affecting reset, cancellation or premature termination may remove a required obligation before R completes, unless a separate epoch qualification proves the corresponding preservation.

K.CT.Facts. Supply J.InternalFactCorr for every finite prefix of every operationally admitted implementation execution of the bound instance, with its domain fixed independently of service satisfaction, successful completion or correspondence-constructor success. The explicit signature F includes every R acceptance fact, every W member's completion fact and every consumed historical control/value predecessor fact used to align those members, and any legally recorded demands consumed by K.CT.LazySerial, with their source descriptors. Initial facts and roots are bound separately. Include the complete recursively consumed provenance support, not just the output projection. Bind exact occurrence identity, endpoint order, matching, payload and retirement attribution for every consumed fact. An output-only comparison is insufficient when a progress lemma consumes internal facts. Source/RTL boundary and storage/coupler back-annotation shall be faithful. The selected J adapter retains its own premises; history matching alone does not transport request presence, enabling or fairness, which require their separate bindings. The constructive option is J.ConstructiveFactCorr, with J.RegisteredConstructiveFactCorr for its qualified message/anchor specialization; record its local fact inventory and exact F-prefix discharge rather than citing only J.SafeTransfer-HistCov. For any consumed internal Pop using Core.LogicalPopPayload, record that logical value interpretation prospectively and discharge Q.CT.LogicalPayload, Q.CT.LogicalPair when selected, and the applicable J.LogicalPayloadRefinement cases. The fact is attached to the actual retirement and its matched prior admission, not to a presumed original value on a transformed wire; the workload witness cannot choose that value. The consumed signature also includes every historical fact recursively supporting the selected control-support argument, including K.CT.CtrlSupport decisions when that standard discharge is used, including decisions that a conditional operation is absent. A negative decision is not itself a fabricated I/O retirement fact. J.StableReadFactCorr may bind reset-stable θ as a common initial root and lift the full body fact signature. J.ConfigRecordFactCorr covers selected CO1/CO2 records and their governing-use attribution. Consumed post-base configuration records are historical facts from actual preceding synchronization units, including when they support CtrlSupport; common roots and matched base records retain their separate base treatment. Auxiliary demand records add no completion fact for the demand and assert no implementation request occurrence. Include every historical fact consumed by their alignment, preparation and resource proof. When a source extension adds actual W completions, the certificate's consumed fact inventory may grow only within the prospectively declared F and attribution, as specified by J.InternalFactCorr. Facts coverage includes those added completions and every newly consumed historical support fact over every finite prefix of every operationally admitted implementation execution, independently of the selected witness and service. A genuinely different signature or attribution requires the new instance/header binding and revalidation stated there; it is not a same-instance extension.

K.CT.Agreement. Establish Core.FinitePrefixProvenance over the workload witness and all source prefixes used by J.InternalFactCorr, or supply an independently proved conditional finite-prefix agreement result with the same consumed conclusion and coverage. Core.FinitePrefixAgreement and J.InternalFactAgreement then identify values and logical identities of facts that actually occur. Endpoint order and attribution remain explicit premises. None of these lemmas establishes occurrence. Outcome-dependent nonblocking control, arbitration, signal observations and reset-sensitive facts must meet the declared provenance/availability and domain obligations; fixed roots may not be chosen contrary to the

operational decision law. No serial comparison-availability or frozen-continuation premise is imported. Default-workflow scope: values and control consumed across the serial witness and RTL-matched source prefixes must have common timing-independent provenance, or a separately proved observation/outcome alignment. An issue-timing-dependent NB outcome or signal read that changes a consumed fact fails this premise; a passing serial run alone cannot cover it. For K.CT.LazySerial, Agreement covers the witness actually used, including every added step of a protected source extension. Determined values alone do not establish either this coverage or coverage of all newly consumed facts within the declared signature by the selected J adapter. Any genuinely changed signature or attribution follows the new-binding requirements of J.InternalFactCorr.

K.CT.Witness and commit units

Supply a finite legal source execution accepting R and a finite set W of commit units containing those acceptances and the prerequisites needed to expose them. Every member of W occurs with its declared completion in the supplied finite legal source witness; W may not add a hypothetical operation absent from that witness. Each unit has fixed logical members, a declared completion predicate and a fixed finite predecessor set; the resulting prerequisite graph is acyclic. These choices precede, and do not depend on, the implementation execution being proved. Completion is monotone in the bound epoch: a retired member cannot be cancelled, counted twice or replaced by another occurrence. Every predecessor and every resource/control dependency consumed in the proof must be represented or separately discharged as an invariant over the admitted surrounding activity. For an application using K.CT.Standard or its Alignment/PlacementLemma discharge, bind the finite decision set K.CT.CtrlSupport(q) below for each target q in W and for each legally recorded demand whose alignment or placement is consumed under K.CT.LazySerial. A general K.CT.Completion or extension exposure proof need not use that particular record schema, but must explicitly discharge every control, entry and membership dependency it consumes, including negative decisions and absence of additional obstructing required work, with the corresponding historical facts, common roots or independently proved invariants. State the replacement control-support argument and its noncircular connection to the claimed exposure conclusion; omitting the standard schema does not waive those semantic obligations. An extension reusing the standard Alignment/PlacementLemma retains their CtrlSupport premises. Its records do not enlarge W: untaken conditional operations remain absent, and no fictitious completion unit is introduced.

K.CT.CtrlSupport (control and membership support). For the standard Alignment/PlacementLemma route, for each q in W, and each legally recorded K.CT.LazySerial demand using this machinery, prospectively bind a finite set CtrlSupport(q) of provenance-keyed decisions needed by placement and finite preparation. Include decisions establishing the complete required blocking-I/O membership of older iterations, permission for the target iteration to enter under B.OF.Entry, and the complete membership of preceding stages of its iteration; sequential owners include the corresponding earlier control path. Record the selected source control path and its reach/branch, entry and membership decisions, including Eval_r = none for conditional sites whose absence is used. A branch exclusion may justify absence of its subordinate sites without evaluating unreachable statements. Record each decision’s deterministic evaluator, witness result and complete recursively consumed provenance support Pred(d); pure local expressions may be inlined. The inventory must account for all required work in each region used by the proof, rather than only enumerating the operations already in W. These are control records, not new source events, commit units or completion facts. They are fixed by the bound source rules and legal witness, independently of the implementation continuation being proved. If no finite, complete, acyclic support certificate can be supplied, this standard route is inconclusive. For data-selected support keys, apply Core.FixedSupportValidation under Core.FinitePrefixProvenance; witness evaluation alone does not prove selector agreement or availability throughout the admitted source-

prefix domain. For a demand, bind the occurrence, owner or complete atomic batch, source issue record, matching admission where consumed, and CtrlSupport before the implementation execution being proved. Its support is supplied by actual completed W units or the permitted independently proved invariants. Recording source issue establishes no implementation issue or completion; K.CT.DemandPlacement or K.CT.BatchPlacement derives implementation placement and assertion.
A singleton blocking call is one possible unit. A primitive zero-depth rendezvous contains both matching endpoints and commits jointly. A coupled atomic group is indivisible and uses its elaborated source handshake law. The general theorem permits partial-retirement singleton members or a separately proved residual-unit interface covering every already-retired subset and the actual member dependencies. K.CT.Standard below uses singleton units for partial-stage members; it does not infer whole-residual simultaneous enabling from per-member progress. A source trace that serializes a genuinely atomic group is not an adequate witness. For a joint unit, its prerequisites include the legally required preparation of every owner; one owner's availability cannot stand in for the other's. For a multi-member residual unit, that separately proved interface must establish that its residual enabling and declared service imply completion of the whole unit from every reachable already-retired subset, with monotone retirement and all member dependencies preserved. Service of one member alone is insufficient. A finite member-by-member argument is admissible when it proves eventual completion of each successive residual obligation; simultaneous enabling of all remaining members is not silently assumed.
K.CT.Capacity. A conservative positive guaranteed capacity may be used to construct the witness only after a legal lift into the fixed Sys_ref is proved. Both the source-side enabling law and the implementation-side resource proof must support that floor. Preserve the actual staged storage and coupler law in Sys_ref and the safety binding. Nominal storage, equal capacities or unused pipeline registers alone do not establish available space. A direct witness in the actual elaborated reference may be used instead. Failure to find a witness at a conservative capacity is inconclusive. B=0 uses joint rendezvous semantics, not a positive-capacity lift.

K.CT.SerialWitness default pre-HLS workflow

The default witness is a finite legal execution of the selected elaborated Sys_ref under the conservative serial blocking issue discipline of Core.PolicyS: within a source interval between adjacent ordinary synchronization calls selected into S, an owner issues no later source-order blocking Push/Pop until every earlier source-order blocking Push/Pop in that same interval has committed in a strictly earlier clock cycle. Independent owners may proceed concurrently. The witness shall use the implementation-bound reference capacities and actual declared enabling laws, or a proved smaller-capacity lift under K.CT.Capacity; unbounded or deeper-than-bound channels are not a valid optimistic substitute. Primitive rendezvous endpoints commit together. A multi-member atomic stage with source-ordered blocking calls on the same owner is not supported by ordinary Policy S: it uses K.CT.GroupSerialWitness below or another explicitly qualified overlapped witness. Atomic units may not be split to force a serial pass. The source run is recorded through a prefix satisfying Core.RequiredAcceptanceObserved for R; it need not be a maximal terminal run, drain all inventories, respond to unrelated trailing calls or satisfy a complete-run bounded-response monitor. Its finite preparation, issue, phase and transfer steps must be legal. A deterministic simulator is useful for reproducibility; the argument consumes the checked finite witness and its stated qualifications, not a complete-run reduction. The witness includes every Core.TransferLaw and Core.SyncProtocol completion on each sampled edge. A prior witness that used discretionary nonselection of a present handshake must be reconstructed or rejected; serial issue and source/RTL agreement alone do not repair it. Declining a permitted genuinely abstract internal move under K.CT.LazySerialWitness is not discretionary nonselection of a physical handshake. The lazy restriction selects a legal witness in the same reference; it changes neither Core.TransferLaw nor the source

relation used by J. Where LazySerial eligibility holds, prefer running this default serial workflow in lazy mode. Lazy mode adds the permitted abstract-movement restriction of K.CT.LazySerialWitness, not a new source issue discipline. Atomic groups retain the separate qualified group-serial mode. Plain unreserved FIFOs need no additional movement restriction.
Construct W and its acyclic prerequisite relation from that finite source witness and the selected qualified resource/controller rules. Retain the needed source-earlier owner operations, value/control preparation, matching items and space-release prerequisites; preserve each joint unit and reject an unsatisfied strict prerequisite internal to a unit rather than silently discarding it. A finite execution order can orient witnessed dependencies, but cannot justify omitting a dependency required for exposure in other executions. For initially empty positive resources using K.CT.Standard, matching Push(k) precedes Pop(k), and Pop(k−g) supplies space before Push(k) when k>=g for the zero-based ordinals of B.CT.EndpointOrder, where g is the selected B.CT.FIFO guaranteed floor (for the ordinary plain-FIFO case at the same selected boundary, where the modeled storage consists exactly of the physical queue and no tighter credit restriction applies, g=F (=B) may be chosen). These space edges are required even when the witness ran at the full reference capacity B; check K.CT.Closure and acyclicity after adding them. A graph that is acyclic only with weaker nominal-capacity edges does not certify under this standard resource proof; this is inconclusive, not a deadlock verdict. Initial contents, gates, shared credits and alternate storage need their own explicitly qualified resource rules. K.CT.Capacity governs any conservative lift. The serial passing run is therefore conservative source evidence, not a standalone RTL progress proof. Failure of the serial run is inconclusive for workload completion; a separately qualified overlapped witness may succeed. K.CT.LazySerial is a separately stated sufficient alternative for its qualified independent-input profile. Its singleton extension satisfies the standard floor closure; its demand branch replaces specified floor edges by proved demand support and does not claim those standard edges have been satisfied.

K.CT.GroupSerialWitness qualified atomic-group issue

This is a conservative group-serial scheduling restriction of legal Core.PolicyL execution, not a change to Core.PolicyS. Partition selected source-interval blocking operations into declared atomic groups and singleton groups. Groups do not cross synchronization or reset boundaries; the binding fixes members, owners, keys, scalar endpoint conflicts and the actual elaborated joint handshake. For each participating owner, all earlier groups required by its source order have committed in strictly earlier cycles before the current group is issued. Every member of the group is legally reached/prepared and issued in one L1 window before any L2 frontier activation; no member depends on an unresolved return from a sibling. Core.RegSeq, Core.ReachPrep, Core.16 and every endpoint/source-order legality condition remain in force. Grouping alone is not permission to issue an otherwise illegal source call.
The whole group commits indivisibly under the declared source and implementation semantics. A rendezvous-connected group includes all matching endpoints and the prerequisites of every owner. Member issue order may respect source order within L1 without requiring earlier sibling commitment; only this separately qualified witness mode relaxes the serial-before-commit restriction inside a group. Partial-retirement stages use their existing residual qualification rather than acquiring atomic semantics. Contract the qualified members into one witness unit, preserve all external prerequisites, and check acyclicity. A strict dependency internal to the group may be discharged only by the explicit legal preparation/atomic structure, never by dropping an unresolved value, endpoint or synchronization prerequisite. The mode reduces to ordinary serial blocking issue when all groups are singletons.
Group-serial source legality follows by issuing the complete independently prepared member batch before settle, performing one declared joint transfer, and making its results available at the prescribed update; the next group waits for a later cycle. The binding must show this batch is legal in the same Sys_ref used by J. This supplies a finite witness only. Group resource support, joint readiness, continuing

clocks and any required pre-grant or abstract/internal service still require K.CT.Exposure/Service. An eligible physical joint handshake commits indivisibly under Core.TransferLaw; there is no later fair selection. In particular, a directive that removes elasticity can create a real deadlock that this mode does not cure.

K.CT.SyncRegion overview of synchronization and epoch capacity

Informative overview. Unary waits and paired synchronization connect completion-qualified regions outside no-wait pipeline bodies. The authoritative implementation requirements are Core.SyncProtocol, B.CT.Sync and B.CT.Restore; elaborated gate, inventory and dependency requirements are Q.CT.Epoch; K.CT.SyncLemma proves their progress consequence. This overview introduces no additional obligation or alternative discharge. The general unit induction remains K.CT.Completion, with K.CT.Standard supplying the standard exposure proof.

The synchronization remains synchronization-typed, including its matching participants and any anchored signal members required by E2/R2. Core.SyncFence, Core.SyncFenceOrder and Core.RegSeq govern preceding-message order and result availability. B.CT.Sync permits qualified early reach while preserving prior drainage; B.CT.Clock prevents newly enabled post-sync work from issuing on the synchronization edge. Core.SyncProtocol commits the indivisible transaction at its eligible settled handshake or unary release. Service needed to reach and expose that opportunity applies to the complete transaction; separate endpoint fairness does not establish it.

Q.CT.Epoch accounts for gate withdrawal, resident items and global matching order; B.CT.Restore requires noncircular eventual reopening and protection of a pending target. A fully drained balanced boundary is one sufficient specialization, not a general requirement. A cycle requiring traffic through its own closed gate is rejected by the dependency conditions. Wider barriers, affecting resets and other unsupported protocols require Q.CT.Extension. Exact clock serialization, when used, retains the narrower scope of K.CT.ClockRefinement.

K.CT.Exposure and service

K.CT.UntilRScope (service-dependent progress scope). Fix the prospectively bound acceptance contract R of K.CT.Instance. An implementation execution accepts R when some finite prefix satisfies Core.RequiredAcceptanceObserved for that contract. In K.CT.Exposure and K.CT.Service, quantify over operationally admitted executions satisfying the declared service assumptions that never accept R; all prefixes of each such execution remain in scope. These are the until-R forms, also denoted K.CT.ExposureUntilR and K.CT.ServiceUntilR. The nonacceptance condition is a hypothetical case used in the completion proof, not an assumption that R occurs, a definition of qualified implementation execution, or a restriction of the operationally admitted domain. The stronger unrestricted forms omit this condition and imply the until-R forms, but are not required by the completion theorem. No exposure or service for unfinished W units after R is accepted follows from the until-R forms.

K.CT.Exposure. For every operationally admitted implementation execution satisfying the declared service assumptions that never accepts R, and every prefix of that execution, consider any incomplete u in W whose predecessors are all complete. After finitely many further steps, either u completes, or the execution reaches a point from which u's actual enabling remains continuously true for as long as u remains incomplete. For an atomic or rendezvous unit this is simultaneous enabling of the whole unit. For a partial unit the qualified residual predicate applies to the remaining members. This obligation must be proved from preparation, controller movement, resources and guards over all continuations in the K.CT.UntilRScope domain, including permitted work outside W. It shall not assume the conclusion of K.CT.Completion, eventual required outputs, or an unproved source-to-RTL liveness transfer.

K.CT.Service. On every execution in K.CT.UntilRScope, every required unit with continuous actual enabling receives its declared service. For physical transfers, Core.TransferLaw and Core.SyncProtocol

supply commitment on the eligible settled edge; clocks and supporting conditions must persist to that edge. Where obtaining a grant is needed, use B2 only after establishing its independent pre-grant ArbEligible antecedent and the grant-to-support/transfer binding. A finite eventual grant may instead discharge K.CT.Exposure's finite-completion branch directly; continuous handshake enablement before that grant is not required. Genuinely abstract units, local preparation, finite computation, request assertion, internal movement and restoration use their explicitly bound service laws, including B.OF.Service where applicable. A coupled unit requires a proof for the whole indivisible unit; separate endpoint fairness is insufficient. Qualified NB retry uses its persistent logical obligation and attempt/service contract, not a claim that each failed attempt is a completed required unit. Required clocks, updates and modeled environment service continue until R is accepted. None of these service conditions follows from safety correspondence or from a successful finite witness alone. Shared scope for supporting progress results. The service-dependent eventualities in K.CT.PlacementLemma, K.CT.ResidualLemma, K.CT.JointLemma, K.CT.SyncLemma, the delivery half of Q.CT.InputStagingProgress, K.CT.CoupledDeliveryLemma, K.CT.StaggeredDeliveryLemma, K.CT.DemandPlacement, K.CT.BatchPlacement and K.CT.DemandPushResource use K.CT.UntilRScope for the same bound instance and R. Their service-dependent uses of B.OF, B.CT and Q.CT progress laws are evaluated on that domain, where service required until R remains in force throughout. This convention also governs the per-unit progress discharges assembled by K.CT.Standard and the LazySerial completion theorems, including a selected retry or extension discharge. It does not restrict K.CT.Facts, conditional Agreement, K.CT.Alignment's semantic and conditional-preparation conclusion, K.CT.ResourceLemma, the space-invariant half of Q.CT.InputStagingProgress, or operational legality, accounting and conformance invariants. Source-witness extension keeps its separately stated protected-continuation premises; this convention supplies no source continuation after acceptance. A use claiming progress on accepting executions after R requires the stronger service assumptions for that use explicitly.

K.CT.Completion theorem

Under K.CT.Instance, K.CT.Facts, K.CT.Agreement, K.CT.Witness, K.CT.Exposure and K.CT.Service, every qualified implementation execution eventually accepts R, with the witness's values and per-endpoint order. Here a qualified implementation execution is an operationally admitted execution of the prospectively bound K.CT.Instance satisfying its declared service conditions and other execution-level contracts. Qualification does not require successful acceptance of R or success of a correspondence constructor. K.CT.Facts is independently required for every finite prefix of every operationally admitted execution, including those not satisfying service. K.CT.Exposure and K.CT.Service are the until-R forms of K.CT.UntilRScope: their progress obligations cover every prefix of every service-qualified admitted execution that never accepts R. The theorem's conclusion still covers every qualified implementation execution, including accepting ones; the stronger unrestricted forms are sufficient but unnecessary. K.CT.Standard is a sufficient discharge of exposure under its additional profile premises, not a restriction of the Facts domain or a filter selecting only successful executions.

Proof. Fix any qualified implementation execution. If a finite prefix already accepts R under Core.RequiredAcceptanceObserved, Facts and Agreement supply the stated witness values and endpoint order. Otherwise suppose the execution never accepts R. K.CT.UntilRScope then makes Exposure and Service available at every prefix, because service required until R continues throughout this putative execution. Choose a topological ordering of the finite W and induct over that order. Once a unit's predecessors have completed, an already complete unit requires no action. Otherwise Exposure gives finite completion directly or eventual continuous actual enabling while the unit remains incomplete; Service gives finite completion in the latter case. Completion remains recorded, so the induction completes every unit of W. Since W contains the finitely many required acceptances, they have all completed by a common finite prefix. K.CT.Facts and K.CT.Agreement identify the committed

occurrences and values and preserve the required endpoint order, so that prefix satisfies Core.RequiredAcceptanceObserved for R, contradicting nonacceptance. Thus every qualified execution accepts R. No completion or exposure of trailing W work after acceptance of R is required.
The conclusion gives no cycle bound, arbitrary cross-endpoint total order, other-stimulus completion, or exclusion of a deadlock after R or in unrelated work. Extra-output exclusion and other safety properties remain obligations of the applicable safety result. Full-witness completion may be requested by requiring service through completion of all W. A finite W does not imply that the whole system has only finitely many actions; any finite-whole-workload restriction of an executable qualification remains a restriction of that qualification.

K.CT.Default qualification

The default sufficient pipeline profile combines B.OF with B.CT and the applicable Q.CT components. Partial retirement is the ordinary multi-member case; declared atomic and direct rendezvous groups use the joint clauses. K.CT.Standard proves the exposure discharge for this profile from the lemmas below. A deployment selects compatible operational laws and supplies its source witness, preparation, agreement, instance binding and service evidence. The theorem is independent of the implementation tool. Naming a no-wait body or supplying only B-faithfulness does not establish the additional progress conformance. The at-site and Q.CT.InputStaging resource/delivery alternatives have distinct laws. Neither removes E3 issue-prefix order or permits later-iteration source launch during Core.16 activation. A physically transformed singleton uses Q.CT.LogicalPayload; a transformed positive two-Pop pair uses Q.CT.LogicalPair with independent paths, Q.CT.CoupledInputStaging or Q.CT.StaggeredInputStaging. Each retains its separate J, fact, source-witness and delivery obligations. Other transformed atomic groups remain outside these named qualifications.
K.CT.Resources. Required items, space and matching endpoint opportunities must become jointly available and remain sufficient for the missing unit until retirement, under completed predecessor closure. K.CT.ResourceLemma discharges this for B.CT exclusive FIFO/reservation/credit resources; K.CT.JointLemma combines those persistent resources with matching direct rendezvous requests. Q.CT.InputStagingProgress supplies persistent space and independent isolated delivery; K.CT.CoupledDeliveryLemma supplies the corresponding joint delivery for Q.CT.CoupledInputStaging. Q.CT.Epoch handles capacity availability across declared boundaries. Alternate storage or shared credit uses an explicit Q.CT.Extension resource proof over all admitted surrounding activity. A nominal aggregate storage capacity is not a resource law. Q.CT.StaggeredInputStaging uses K.CT.StaggeredDeliveryLemma for its A-only prefix, matching late read and paired suffix. The three delivery laws are alternatives with distinct premises; representation does not supply any of them. For eligible lazy extra-slack Pushes, K.CT.DemandPushResource supplies the separately proved persistent ingress-space/credit alternative from recorded demand support. Ordinary selected-channel Pushes retain the standard floor proof. The target's other resource conjuncts, placement, guards and service remain obligations; see the route table at the head of K.CT.LazySerial.
K.CT.Guard (GC1). The standard guard obligation is the declared immediate or finite-response realization in B.CT.Guard at the bound commit and capture sites. Persistent isolation/resource support, retained legally prepared requests and the exact settled handshake law yield eventual persistent enabling in K.CT.ResidualLemma and K.CT.JointLemma. A finite-response realization additionally establishes its non-replenishable finite local measure and actual internally serviced opportunities; joint units need the joint realization. Additional guard inputs are permitted when these obligations are proved, without a syntactic restriction to particular input signals. B-faithfulness matches equations but does not itself prove either realization. Only guards outside both declared options require Q.CT.Extension. Cover all admitted continuations, not a frozen comparison state.

K.CT.Preparation. Finite source preparation and its implementation counterpart remain explicit obligations. Descriptors may be stable functions of literals and registered returns from completed earlier stage occurrences, available before the current L2 snapshot; they need not be literals. They shall not depend on unresolved same-Horizon returns, unavailable control decisions or speculative status, and must preserve the bound identities, footprints and operation inventory. Core.ReachPrep and Core.RegSeq govern finite source legality. Core.IssueFair applies whenever a source-side eventual-issue argument is invoked, under the independent trigger in M.7a; a recorded finite legal witness alone needs no complete-run fairness certificate. B.OF.Request or an equivalent qualification supplies implementation assertion. Value correctness from J and determinacy does not establish operand availability.

For the narrowly qualified unordered members of a partial stage, all members must be legally prepared and issued in the same L1 window, before the L2 settle point can activate the minimal pending frontier. The binding must establish that complete batch preparation is possible without unresolved sibling returns and does not already violate an activated frontier. Core.16(2) then forbids new later work, while Core.16(5) permits those already-issued siblings to retire and their done state to persist. The witness may require previous stage occurrences rather than earlier siblings to complete only with this source derivation and the residual-member proof. Cross-stage overlapped witnesses are not qualified by this clause.

The standard exposure discharge is K.CT.Standard. Its placement, resource, residual, joint and synchronization lemmas establish finite completion or the required persistent support from local operational laws and declared service. K.CT.Completion then supplies the finite unit induction. Finite-response guards add a finite internally serviced readiness step. For a physical interface, the ensuing eligible settled handshake commits mandatorily under Core.TransferLaw/Core.SyncProtocol, assuming the required clock and support continue. Pre-grant arbitration and genuinely abstract service retain their independently established contracts. This distinguishes the generic implication from proof that an actual implementation and source witness satisfy its premises; no later fair selector is inserted after the handshake.

K.CT Standard discharge lemmas

K.CT.Closure. For the standard profile, W and its acyclic predecessor relation satisfy the following explicit closure. Each target contains every strictly earlier endpoint operation, meaning every lower-ordinal operation at that same scalar endpoint, together with its matching item and guaranteed-floor space release. Separately, for every target, include every source-earlier required blocking operation on its owner control path in the current synchronization interval, on any endpoint, subject only to the qualified batch exceptions stated below. This includes prologue work, preceding loops and their required iterations, and loop work before an epilogue target; it applies to mixed sequential/pipelined owners as well as purely sequential ones. Same-endpoint order alone is insufficient. A pipelined target includes every required I/O member of preceding stages in its iteration and every required I/O member of every older iteration of that owner in the pipeline, including partial-stage siblings and members that would otherwise lie outside W. Thus the unlaunched-target case can reach B.OF.Entry's empty-of-incomplete-work condition, and launched targets can await exit of all older work. These requirements order the sufficient certificate, not the actual implementation schedule. All such members occur in the finite source witness; inability to form this finite acyclic closure is inconclusive, not a deadlock verdict. All required sync/gate predecessors and source control/value predecessors are included. For alignment, each non-root historical fact in Pred(q) is in F and is supplied by a strictly preceding W unit, recursively; bound initial facts are established at the base. Pure deterministic local expressions may be inlined into Eval_q rather than inventing unsupported commit units for computation. Required remaining local computation uses legally available operands and B.CT.Control/B.OF.Complete service, without new

unresolved I/O. Atomic contraction preserves every external predecessor; a strict commit-before-commit dependency internal to a unit is inadmissible. Qualified partial-stage members may omit earlier same-stage sibling commits only under K.CT.Preparation's complete legal pre-activation batch; they may not omit earlier stages or older-iteration obligations. Independently prepared atomic members use their declared legal batch. No graph edge is treated as an implementation scheduling command. For every d in CtrlSupport(q), each non-root historical fact in Pred(d), recursively, belongs to F and is supplied by a W unit strictly preceding q; initial roots are bound at the base. Alternatively, a separately proved state/control invariant may discharge the decision or its dependency over all admitted surrounding activity, with legal operand availability explicit. Such an invariant may not assume the placement, absence of obstructing work, or completion it is intended to establish. This closure covers negative membership and iteration-entry decisions as well as Eval_q = some(desc). The complete control inventory establishes that the earlier-work lists above omit no additional required blocking operation along the relevant actual source paths. Decision evaluation uses B.CT.Control and, for pipelined owners, B.OF.Control; it introduces no unrepresented unresolved I/O. K.CT.LazySerial may use this closure unchanged after a protected singleton extension, or its separately proved demand-supported replacement for specified Push floor edges. The latter is an alternative exposure discharge and does not weaken the definition of standard closure.

Lemma K.CT.Alignment (source-required target, control membership and finite preparation). Assume K.CT.Facts/Agreement, B.CT.Control, B.OF.Control for pipelined owners, and completed K.CT.Closure predecessors for target q, or the completed alignment-support predecessors specified by K.CT.DemandPlacement/K.CT.BatchPlacement for a legally recorded demand. Alignment applies both to Eval_q = some(desc) for q in W or such a source-required recorded demand and to every d in CtrlSupport(q), including negative membership Eval_r = none and entry/branch decisions. For each decision, its consumed non-root historical facts have occurred in the implementation or are discharged by the independently proved invariant permitted by Closure; common initial roots agree. J.InternalFactCorr supplies the corresponding historical facts, occurrence-to-availability covers both that prefix and the workload witness, and J.InternalFactAgreement identifies their witness values and keys. The common deterministic evaluator therefore yields the same positive descriptor or control/membership decision whenever its operands and phase conditions are legally available. Branch exclusion does not require execution of unreachable statements. Together with the complete CtrlSupport inventory, this establishes that no additional unwitnessed required blocking operation obstructs the older iterations, earlier stages or sequential path used in PlacementLemma. This is semantic alignment, not an assertion that the target is already physically reached.

B.CT.Control/B.OF.Control bind actual control and finite preparation to these decisions. At a control point where the stated placement and phase conditions hold, B.CT.Control supplies finite legal local preparation with Core.ReachPrep/Core.RegSeq and internal service. This conditional result assumes no placement theorem. PlacementLemma uses the aligned decisions and B.OF entry/order/computation/drain laws to establish placement; B.OF.Request then supplies assertion when legally eligible. Any persisting Core.16 activation must be discharged or permit the batch under the stated predecessor and preparation obligations. Agreement cannot bypass it. This lemma supplies neither channel resources, the required clock/environment or pre-grant/internal service, nor R completion. Actual physical commitment is governed by Core.TransferLaw/Core.SyncProtocol once the qualified handshake/release occurs.

Lemma K.CT.ResourceLemma. Consider an incomplete target endpoint whose K.CT.Closure predecessors have completed in a B.CT.FIFO/EndpointOrder realization. For Push(k), no later Push has retired. For zero-based k, if k>=g, Pop(k-g) and all earlier Pops have retired, removing k-g+1 of the k earlier items and leaving at most g-1 unmatched; if k<g, at most k<=g-1 earlier items remain. Queue occupancy is at most unmatched inventory, hence <F, and A<C when credit applies. Before the target retires, no later

producer call can take this space; consumer activity can only preserve or increase it. For Pop(k), Push(k) and all earlier Pops have retired. Exact FIFO matching places its item either at the oldest uncaptured queue position or in its own retained slot. No other consumer can remove that item. At the Pop site, the item and empty attributed slot (or already held item) give persistent own-resource support. Once the legally prepared request is pending, the declared immediate or finite-response capture guard supplies a permitted capture/read opportunity; local service and retention supply the held item or retire the Pop directly. These arguments remain valid with unrelated later work because it cannot overtake either endpoint or steal an attributed item. Every conjunct for distinct positive channels is simultaneously persistent. Initial items, shared credits and alternate storage need the explicitly amended law rather than this empty exclusive-FIFO calculation. For Q.CT.InputStaging resources, use Q.CT.InputStagingProgress in place of this primitive queue-to-at-site-capture argument. That lemma first proves persistent Push space and, after placement/isolation and legal request assertion, finite delivery into the target's retained terminal location; it does not assume an item is already at the process site. For Q.CT.CoupledInputStaging, the same per-channel inventory calculation proves Push space, but joint item delivery is supplied by K.CT.CoupledDeliveryLemma; do not apply independent forwarding to a coupled path. For Q.CT.StaggeredInputStaging, the same per-channel inventory bound supplies producer space, while K.CT.StaggeredDeliveryLemma supplies delivery. A joint physical cache keeps two logical item counts and cannot release either channel early.

Lemma K.CT.PlacementLemma. With completed standard closure, K.CT.Alignment fixes the source-required target, descriptor and finite legal preparation under B.CT.Control/B.OF.Control. Alignment of CtrlSupport(q), including negative membership decisions, establishes that the closure inventory contains all required blocking I/O of the relevant older iterations and preceding stages, and fixes the target iteration’s source entry permission. Every older iteration’s required I/O is therefore complete by K.CT.Closure; this conclusion does not follow merely from completion of the listed positive W members. Its remaining eligible finite local computation finishes under B.OF.Complete/Service; ordered movement and B.OF.Drain advance the oldest occurrence through its finitely many remaining stages and out. Repeating over the finite older population establishes their eventual exit, even though an occurrence may become incomplete again on entry to a later computation stage. No younger work blocks this downstream drainage. If the target iteration is unlaunched, no younger iteration has entered, so entry follows after older work exits and the aligned control and Core.16 conditions permit it. Its preceding stages then finish their serviced local work and advance it to its site. If already launched, the same argument advances it through its remaining earlier stages. Unless it retires first, the unfinished target eventually remains at its site with no older occurrence downstream. Younger work cannot pass it and the target cannot advance past its site unretired, so this eventual isolation persists. A sequential owner uses B.CT.Control’s finite source-aligned path after every source-earlier blocking operation on that path has completed. This is finite target/older-work progress, not termination of unrelated activity or a global drain requirement. When used before a Q.CT staging delivery lemma, this placement argument and subsequent B.OF.Request discharge must not assume the target’s terminal data arrival. If their actual controller proof needs that arrival, use the separately qualified pre-request extension instead. The recorded-demand target-domain extension is K.CT.DemandPlacement. Resource-independent preparation of a complete atomic input batch is K.CT.BatchPlacement. These lemmas retain the complete control and older-work inventory and derive implementation request assertion without treating a source issue record as an implementation occurrence.

Lemma K.CT.ResidualLemma. For a singleton partial-stage target unit in K.CT.Standard, K.CT.Alignment and PlacementLemma require the complete preceding-stage and older-iteration closure and establish eventual permanent isolation; only qualified same-stage sibling commits may be omitted. B.CT.State retains every already retired sibling and excludes reissue. K.CT.ResourceLemma supplies the target's own persistent resource for the primitive profile; for Q.CT.InputStaging, Q.CT.InputStagingProgress

supplies persistent Push space or finite delivery and retention of the Pop item after legal request assertion, and B.CT.Guard has its declared immediate or finite-response realization under isolation without an unfinished-sibling condition. B.OF.Request first supplies the legally prepared request and Core.IC retains it. With persistent own support, finite-response guard steps decrease their finite measure until the capture/commit guard is persistently true; the immediate option needs no such delay. A required split capture receives local service and retains its item; its held-item phase has the same guard qualification, or a permitted coincident form retires directly. The target therefore completes or reaches persistent support for the actual boundary opportunity. A physical handshake commits on its eligible edge by Core.TransferLaw, with continuing clocks and support as required; a genuinely abstract action uses its declared service. The argument applies to every possible retired subset and is unchanged when unrelated siblings retire earlier. This establishes K.CT.Exposure for that singleton member for every retired sibling subset. K.CT.Service must be supplied per member; it does not assert simultaneous enabling of a multi-member residual unit. It does not authorize source preparation outside the same-window conditions of K.CT.Preparation.

Lemma K.CT.CoupledDeliveryLemma (flushable equal-length paired input delivery). Assume Q.CT.CoupledInputStaging, K.CT.Facts/Agreement and completed K.CT.Closure predecessors for a missing positive Pop pair. Closure includes both matching input admissions, all earlier retirements on both input endpoints, and the owner/control/value and older-work prerequisites used for placement and legal batch preparation. Use K.CT.Alignment, PlacementLemma and lawful request assertion to establish the correct pending pair and eventual permanent isolation unless it has already retired. This step consumes no item-delivery conclusion. Both producer admissions must have been discharged earlier in the W induction by their own resource, preparation and service cases; consumer demand and local forwarding acyclicity do not prove producer progress. This lemma retains its equal-length scope; the staggered alternative uses K.CT.StaggeredDeliveryLemma. Its placement/request premises are independently discharged under B.BoundaryBinding's noncircular qualification.

Proof of paired delivery. Let j be the target pair ordinal. Exact per-channel FIFO order and the paired-ownership invariant put its two admitted members either at the two ingress heads or together in one paired location. Earlier Pops have retired, so there is no older unmatched item ahead; younger work cannot pass j or steal an empty destination ahead of it. Older process work may temporarily hold forwarding, but its finite local computation and drainage eventually finish under the placement/service premises. After permanent isolation, the coupled movement law enables the joint ingress or next paired move without waiting for a younger read, new upstream traffic, or retirement of the target itself. The support persists by order and retention. Joint internal service performs the move and decreases the finite number of boundaries remaining to the terminal. Induction on this distance gives paired terminal arrival; both members are retained. The proof uses old-empty destinations and thus also applies when simultaneous replacement is permitted. It does not require bubble compression behind incomplete older work.

At the terminal, legal persistent batch requests and the declared conjunctive retirement law give continuous joint enabling, immediately or through the separately qualified retirement-guard realization. The Q.CT.LogicalPair reserved-empty-position invariant, when used, supplies no extra retirement-readiness gate; this claim is confined to that profile, not a simultaneous-replacement result-slot extension. At a physical joint interface, Core.TransferLaw commits both members on the eligible settled handshake; required continuing clocks and support remain premises. Any genuinely abstract retirement uses its separately qualified service. Per-channel Push support still follows the floor arithmetic of Q.CT.InputStagingProgress: g_l<=F_l and, with exclusive credit, g_l<=C_l. This proves the resource/delivery alternative consumed by K.CT.JointLemma and Standard, not source-witness lifting, arbitrary coupled-controller conformance or completion of a cyclic W. No RTL completion result is used

in this argument. The delivery argument uses ownership and movement, not separate physical values; Q.CT.PairedRepresentation is compatible only when it preserves these operational laws.
Lemma K.CT.StaggeredDeliveryLemma (fixed-offset physical input delivery). Assume Q.CT.StaggeredInputStaging, K.CT.Facts/Agreement and completed K.CT.Closure predecessors for the missing terminal Pop pair j. These include both actual input admissions, all earlier input Pop retirements and the owner/control/value and older-work prerequisites needed for legal preparation and permanent isolation. Alignment, PlacementLemma and lawful request assertion establish the correct pending terminal pair unless it has already retired; they do not assume its input delivery. Each admission is an earlier producer obligation in W, not a consequence of consumer demand. In particular, terminal placement and pending-pair assertion are independently discharged under B.BoundaryBinding's noncircular qualification.
Proof of late-read approach. If j is already in the paired suffix, use the suffix argument below. Otherwise its A item is the oldest remaining A owner, in its ingress or A-only prefix, and its B item is the B ingress head. Earlier Pop retirements and the staggered matching invariant rule out an older unmatched item ahead. Younger items cannot pass j or occupy an empty destination ahead. After completed older process work finishes its local computation and exits under service and drainage, permanent isolation makes the declared forwarding guard continuously true. Internal service moves A through finitely many early positions. At the late-read site, the actual old A owner and matching old B head coexist with an available paired destination; joint internal service performs that read. The step requires neither new younger traffic nor prior retirement of the target.
Proof of suffix delivery and resources. The joined pair now advances under the same empty-destination retention argument, with joint service for every suffix move. Each early or joined move decreases the finite number of A-path boundaries to the terminal; holding cannot increase it. The movement law prohibits a missing younger read from freezing older downstream work. At the terminal, both retained owners and the legally pending batch give the declared persistent joint readiness, including the separately qualified retirement guard. At a physical joint interface, Core.TransferLaw commits both on the eligible settled handshake under the continuing-clock/support premises; a genuinely abstract retirement uses its separately qualified service. Q.CT.LogicalPair's reserved-empty U invariant is not an extra retirement gate in that profile; it does not cover a result-slot extension whose readiness depends on downstream departure. Per-channel Push space follows the usual floor argument: at most g_l-1 older unmatched items remain and ingress occupancy is no larger, so it is $<F_l$. The staggered family here has no additional credit cap. These proofs use logical ownership and actual operational laws and remain valid for Q.CT.PairedRepresentation; they do not require separable physical values, supply a source-witness lift, or establish completion of cyclic producer dependencies. The producer calculation uses zero-based endpoint ordinals: $Push_l(k)$ requires $Pop_l(k-g_l)$ and its predecessors when $k\ge g_l$; for $k<g_l$ there are at most k older items. The suffix retirement count and owner ordinals use that same base.
Lemma K.CT.JointLemma. Let a finite atomic/direct-rendezvous component satisfy Q.CT.Groups and have completed closure. Apply K.CT.Alignment and PlacementLemma to every participating owner. The latter establishes placement after the owner's finite older work has exited, hence permanent isolation while unfinished, rather than relying on transient completion of downstream stages. Because each owner contributes only one site and an unfinished member cannot leave it, all finitely many persistent placements coexist after a finite time unless the component has already committed. Legal group preparation and B.OF.Request establish all requests without waiting for peers merely to assert. Each positive input is retained or supplied by a permitted old-head coincident read on the committing edge, as required by B.CT.Compatibility, and K.CT.ResourceLemma preserves every primitive positive resource. For independent positive input paths, apply Q.CT.InputStagingProgress to each after owner placement and legal batch assertion; its forwarding cannot wait for the other inputs. Finitely many retained arrivals

coexist. For the equal-length paired alternative Q.CT.CoupledInputStaging, use K.CT.CoupledDeliveryLemma instead: both admissions are completed predecessors, aligned paired movement receives joint internal service, and the pair reaches and remains at the terminal despite absent younger reads. For Q.CT.StaggeredInputStaging, K.CT.StaggeredDeliveryLemma instead carries the target through its A-only prefix, matched late read and paired suffix after both admissions and placement are established. Each of these three qualified alternatives establishes persistent positive support. A computed pair additionally satisfies Q.CT.LogicalPair; representation alone supplies no delivery. Direct B(0) endpoints remain pending without storing or consuming the item separately. With these persistent requests and resources, B.CT.Guard makes the joint guard true immediately or after its finite decreasing-measure realization; it then remains true. In a mixed group, any required positive capture has its separately qualified local guard realization before the joint phase. The exact conjunctive settled law therefore enables the whole component continuously. Indivisible joint service commits it. This proves exposure without solving a recursive readiness fixed point or deriving joint fairness from separate endpoint fairness. A strict internal dependency or two sites of one owner invalidates the profile rather than this argument repairing it. For a physical joint handshake, that boundary service is the mandatory eligible-edge commitment of Core.TransferLaw; it adds no fairness choice after overlap. All preceding delivery, placement, guard and continuing-clock premises remain necessary.

Lemma K.CT.SyncLemma. Suppose the preceding blocking/control obligations of a unary/paired boundary are complete. B.CT.Sync reduces boundary reach to finite serviced local tail work and retained request assertion; the ordinary placement/drain argument supplies any completed pipeline tail. All matching requests therefore coexist, unless the synchronization already committed. Its fence and persistent anchor support permit the declared B.CT.Sync/Guard realization to expose legal held offers. With the required continuing clock and matching environment/participant support, their eligible settled handshake, or the qualified unary release, commits mandatorily under Core.SyncProtocol. The finite-response guard proof supplies readiness before that handshake; no B2 choice remains afterward. For a required next-region target, B.CT.Restore's finite natural measure decreases to zero under internal service, independently of traffic through the closed gate. Q.CT.Epoch requires on every admitted execution that no later closing boundary or equivalent withdrawal precede retirement of the protected target, including before its request becomes pending. This operational conformance requirement, together with B.CT.Restore’s persistence for a pending transfer, keeps the restored availability effective until retirement. Witness ordering alone does not establish that implementation fact. Q.CT.Epoch preserves any resident items and credits, so K.CT.ResourceLemma or the explicitly supplied regional resource law applies without assuming empty storage. Core.SyncFenceOrder accounts for preceding same-edge message commits before sync; post-sync issue remains in a later cycle. Cyclic gate dependencies fail closure. This proves the claimed boundary and gated-target exposure under the stated operational laws, not for arbitrary synchronization/reset protocols.

Definition K.CT.HoldingRelation. For the optional interleaving presentation of the canonical B.CT profile, the abstract state records stage locations, launch counts, retired members, queue contents, held items, pending syncs and gate/restoration state. Its steps are one permitted capture, one singleton or whole joint commitment, one completed-suffix movement (including a proper held suffix), one legal entry, or one boundary/restoration update, each with the same declared state guard and exact effect as B.CT. Optional suffix holding does not remove the maximal legal drainage opportunity required by B.OF. Coincident positive capture/retirement is represented by capture followed by the same singleton or whole positive-group commit; joint B(0) is always one step. No split projection of a rendezvous or atomic unit is admitted. The identity map on these declared fields is the abstraction; additional implementation registers need a separate invariant proving that projection.

Lemma K.CT.ClockRefinement. For the canonical clock relation with split-capable at-site capture, guards either true or completed-downstream, B.CT.FIFO/Joint/Compatibility, and old-done or same-edge

completed-suffix advancement, every synchronous edge has a finite K.CT.HoldingRelation path with exactly the same committed keys, payloads, endpoint order and final abstract state. Proof: perform its selected captures first, including the conceptual capture of a selected coincident positive Pop, then its selected singleton/whole-group commits, then selected movements/entries and eligible control updates. A selected read uses the old head; no new Push supplies that read. Removing an old head cannot disable another selected Push, and appending a Push cannot invalidate another selected read. Endpoint order and the single-transfer restriction exclude conflicting writes/reads. Captures preserve completed downstream work; commitments only increase retirement state, so the canonical guards remain true before movement. Joint positive supports are distinct and retained, and B(0) groups are not split. Old-done movement may leave newly completed work held; the holding relation permits exactly that suffix. Same-edge movement sees the selected retirements but no prematurely consumed return. Pending/fence control precedences place any qualified same-edge sync after its prerequisite messages; no newly opened gate contributes a post-sync action on that edge. Disjoint frame updates commute, yielding the same final state. For other guards or movement laws, this serialization conclusion requires their own frame/preservation proof; their direct synchronous progress may still follow K.CT.Standard without an interleaving presentation. Here selected captures and commits mean the complete forced physical inventory plus independently lawful abstract actions, fixed before serialization. The holding path only represents that edge; an arbitrary path omitting a forced transfer is not thereby a legal source or implementation clock edge.

K.CT.ClockRefinement service qualification. Matching finite edges is a safety statement. Transporting an abstract progress proof additionally requires an actual synchronous realization of each required opportunity, persistent support or a well-founded local progress measure to that opportunity, and the relevant clock/environment/service premises. For physical transfers, an eligible sampled handshake commits mandatorily; even one such edge with an omitted transfer violates the transfer law. Finite delay may instead occur before the handshake through a qualified guard, grant or preparation step. Genuinely abstract/internal opportunities use their separately declared service, where one idle edge can be legal but infinite continuously enabled stuttering is excluded. B.CT.Service and the placement/resource/guard lemmas supply these distinctions for the standard direct proof. No finite global rank of unrelated activity is assumed. An implementation that matches safety edges but never restores a needed gate fails B.CT.Restore/Service.

Theorem K.CT.Standard. Fix K.CT.Instance, Facts, Agreement and a finite legal K.CT.Witness satisfying K.CT.Closure. Suppose every owner and component conforms to the applicable B.OF, B.CT and Q.CT clauses, including B.CT.Control’s source-aligned control realization and finite legal preparation, exact boundary/resource attribution, compatible composition and the declared service assumptions. Then K.CT.Exposure holds in its K.CT.UntilRScope form. Proof: fix an admitted implementation execution meeting those service assumptions that never accepts R, and select any incomplete unit with completed predecessors at any prefix. Service required until R remains in force throughout this execution. Closure and K.CT.Alignment first identify the source-required members and descriptors and supply their finite legal preparation route; Facts and Agreement are consumed here, not only after commitment. For a primitive or singleton partial positive unit use PlacementLemma, ResourceLemma and ResidualLemma, substituting Q.CT.InputStagingProgress for the resource/delivery step when that named component is used; for an atomic or direct rendezvous component use JointLemma, using K.CT.CoupledDeliveryLemma or K.CT.StaggeredDeliveryLemma for the named positive coupled alternatives and Q.CT.LogicalPair for its computed representation when applicable; for a boundary or gated unit use SyncLemma followed by its regional unit lemma. A qualified logical retry uses Q.CT.Retry. Partial-stage members are singleton units with per-member exposure and service; genuine atomic and rendezvous components remain indivisible. Standalone non-retry NB attempts and multi-member non-atomic residual units are outside this standard case list and require an explicit extension proof. Each

progress proof uses the shared K.CT.UntilRScope convention and covers arbitrary admitted surrounding activity on that domain by the universal operational invariants and retained endpoint order. The invariants themselves retain their full stated domains. Thus the unit completes or becomes persistently enabled. Applying K.CT.Completion proves acceptance of R with the witness's values and endpoint order. No implementation-specific example or successful implementation run is an additional theorem premise. A nonstandard component remains conditional on its explicit Q.CT.Extension proof.

K.CT.LazySerial independent staging completion alternative

Completion-route comparison (informative). The following table summarizes the existing theorem branches; their controlling statements and premises remain authoritative. K.CT.LazySerial names the section and family of constructions, not an additional theorem.

Route	Resource / witness treatment	Acyclicity and conclusion
K.CT.Standard	Actual completed floor releases under standard Closure; nominal witness capacity does not waive them.	Supply standard acyclic closure and conformance; Standard discharges Exposure for Completion.
K.CT.LazyExtendedStandard	Restricted all-singleton construction plus a protected source extension adds actual missing Pop completions.	LazyRank retains its original issue/admission ranks; the resulting standard closure permits Standard.
K.CT.LazySerialRestricted	Complete restricted serial/group-serial certificate. Only qualified extra-slack releases use demand support; ordinary cases retain floor resources.	LazyRank or LazyMixedRank derives acyclicity; invoke LazyComposition and Completion.
K.CT.LazyComposition	Selected eligible channels use demand support only for qualified extra-slack releases; other units and resource conjuncts retain their own proofs.	Prove or soundly check the complete assembled graph. Local consumer discipline does not impose the restricted global issue discipline.

K.CT.LazySerialScope. The restricted rank-derived construction is a sufficient alternative to K.CT.Standard for initially empty exclusive positive channels realized as independent Q.CT.InputStaging paths, together with ordinary unreserved FIFOs. It uses the same implementation-bound Sys_ref and actual storage/handshake equations. Its singleton mode uses Core.PolicyS; its atomic mode uses K.CT.GroupSerialWitness for pure positive, independently prepared, same-site atomic input groups, with each positive channel appearing at most once in a group. The complete older-work, preparation and CtrlSupport inventory remains required. The automatic rank result concerns these blocking message units in the bound source interval. K.CT.LazyComposition separately permits selected eligible lazy channels in mixed designs containing rendezvous, synchronization, gates, coupled, staggered or reserved-slot components; unselected components retain their own qualified dependencies and exposure proofs, and acyclicity is established for the combined graph. Such components do not themselves acquire the demand-resource proof by composition. Physical representation may reuse Q.CT.LogicalPayload/LogicalPair only while preserving the selected independent operational law and every logical-fact obligation. Naming: K.CT.LazySerial denotes this route and section; K.CT.LazySerialRestricted is its rank-derived corollary, K.CT.LazyComposition its general mixed-component theorem, and K.CT.LazyExtendedStandard its separate direct return to Standard.
K.CT.LazyEligibility (selected channels and consumers). A channel selected for lazy treatment has initially empty exclusive independent InputStaging storage, ordered endpoints, the declared ingress/credit equations, and no additional producer epoch gate or gated internal movement. Its consumer follows

serial or qualified group-serial issue throughout the relevant witness, giving at most one outstanding Pop or group per owner and completed owner-path predecessors before demand issue. Channels consumed by an owner using an incompatible overlapped mode are not eligible. The consumer satisfies DemandPlacement or BatchPlacement, including B.OF/B.CT.Control, complete control/preparation support and arrival-independent request assertion. A pre-request, data-driven placement extension does not inherit these lemmas. The producer need not use serial issue for the local space/credit proof, but must satisfy endpoint order and its own source-legality, placement, preparation and exposure obligations in the assembled certificate. The restricted automatic rank construction retains its stronger global issue-discipline premises.

Atomic eligibility and plain FIFO definition. If a selected demand is an atomic member, the whole group must satisfy BatchPlacement: pure positive, compatible same-site placement for each owner, complete independent legal batch preparation, and every input independently staged or a compatible plain FIFO. Here plain FIFO means B.CT.FIFO with no attributed capture slots or other reserved storage, retiring by coincident capture on the group commit edge under B.CT.Compatibility. A split-capture member is a reserved-slot realization and is excluded from this batch profile. A B(0) leg, coupled or staggered pair in the same group likewise makes its members ineligible for this demand treatment. Those component kinds may occur elsewhere with their own qualified joint/delivery proofs. Space/credit alone does not establish a gated Push's resources; selecting a gated channel requires a separately proved combination of demand-space support with gate restoration and persistence under the same completed predecessors, which is not supplied by this first composition profile.

K.CT.LazySerialWitness (component-bound witness restriction). Bind each channel c to ingress $F_c>0$, finitely many ordered one-item staging locations and optional exclusive outstanding credit $C_c>0$. Set $g_c=\min(F_c,C_c)$, or $g_c=F_c$ if credit is absent. Write $P_c(k)$ and $D_c(j)$ for zero-based matched Push and Pop occurrences. Select an internal abstract move of item j only while $D_c(j)$ has legally issued and is unretired. For a group member this means the complete legal batch has issued and its atomic retirement has not occurred. Apply this restriction only where Core.SelectLatitude and the declared reference law permit declining that move. Every mandatory physical handshake and its effects must still be recorded. A physical staging interface cannot be treated as optional abstract movement. The permissive reference used by J remains unchanged; the restriction selects one legal witness and does not constrain implementation scheduling. The selected channel set and certificate are fixed before, and independently of, the implementation execution being proved. Selection may follow source simulation, but the full recorded trace, including any extension, must satisfy the lazy restriction on every selected channel; an eager move that violates it cannot be repaired by later selection. Check and record this trace property in M.7a. A convenient workflow generates lazily on every eligible channel and selects the final subset afterwards. Nonselection is legal only for abstract moves that the declared reference law makes optional; abstract classification alone waives no mandatory movement or applicable progress obligation. A more conservative witness may fail inconclusively on an ultimately unselected channel.

K.CT.LazySimulation (operational meaning, demand visibility and selection guidance). Lazy mode changes the selection of optional abstract staging moves in the pre-HLS elaborated channel model. It does not change the source program's Push/Pop operations, the selected serial/group-serial issue discipline, channel capacities or mandatory handshake semantics.

For each item j, an internal staging move is eligible in lazy mode only when both its ordinary movement conditions hold and its matching process-facing Pop(j) has issued and remains unretired in the same pre-HLS execution. Apply this additional condition to every optional staging step, including the ingress-to-staging move (early capture in the B.BoundaryBinding sense). It disables prefetch before the source consumer reaches that Pop; it still permits movement that releases ingress space after Pop issue and before Pop retirement. At-site split capture into attributed slots under B.CT.Capture is a separate reserved-slot realization and is not covered by this lazy profile.

The process-facing Pop is the source operation that ultimately returns the matched item to the consumer. An earlier read at a pipeline's outer interface may instead move an item into elaborated channel storage. The fixed boundary mapping distinguishes these actions; an outer ready signal alone is not evidence that the process-facing Pop has issued. The channel model or simulator scheduler uses its own source-demand record to select internal moves; the producer does not need that record.
K.CT.LazySimulation.DemandVisibility (data-independent demand requirement). A simulator implementing the recommended lazy mode shall expose the process-facing blocking Pop request independently of matching-item availability. Once the Pop is legally reached and prepared and its issue is permitted by the selected issue discipline and phase rules, the simulator shall assert and record that request without waiting for the item to reach the Pop site. For an atomic group, this applies to the complete legally prepared batch. Item arrival controls transfer eligibility, not whether the consumer's demand is represented. All Core.RegSeq, Core.16 and other issue-legality conditions remain in force. A demand need not strictly precede arrival when its item is already available; the requirement is independence from arrival. A read implementation that exposes demand only after data arrives can otherwise block the very movement needed to deliver that data.
The producer's behavior is unchanged: it asserts its legally prepared Push request/valid without introducing a wait for ready, and retains the request and payload while blocked. Lazy movement can change ingress occupancy and therefore readiness, but it adds no producer-side ready-before-valid rule.
The restriction applies only to abstract moves whose nonselection is permitted by the declared reference law. It never suppresses a mandatory physical handshake or overrides a required movement/progress obligation. The reference transition relation used by J remains unchanged.
K.CT.LazySimulation.Selection (recommended movement selection). Deterministic simulators should select lazily permitted moves promptly, using a fixed selection rule and the declared old-state, simultaneous-update and conflict constraints. Where moves are jointly legal, select them at their first legal opportunity. This avoids unnecessary internal stuttering and supports reproducibility. In particular, one item cannot cross multiple staging locations on one edge contrary to the component law. Later selection remains permitted where the reference law allows it; no general claim that prompt selection dominates every other legal witness schedule is made. This is generator guidance, not a new hardware progress assumption.
K.CT.LazySimulation.Example (informative: one ingress entry and two staging locations). Assume independent staging with F=1, m=2 and no tighter outstanding-credit cap. Item a occupies the ingress, both staging locations are empty, and the consumer has not issued its process-facing Pop(a). In normal mode, if the declared movement conditions permit it, the channel model may move a into staging and free the ingress. In lazy mode, that move is not selected yet. A producer offering the next item b remains pending normally; it does not withhold valid while waiting for ready.
When the pre-HLS consumer legally issues Pop(a), the lazy restriction permits a to advance under the ordinary movement conditions. Moving a out of ingress permits b to be admitted on a subsequent eligible edge, even though Pop(a) has not yet retired. Item b remains in ingress until its own Pop(b) issues. Under singleton Policy S, Pop(b) cannot issue before Pop(a) retires. Thus the lazy witness can hold at most F+1=2 unmatched items here: one in ingress and one in staging. The elaborated storage capacity remains F+m=3; laziness restricts its use rather than changing the back-annotated capacity. Exploiting all three entries requires a separately qualified movement/progress profile, such as an elastic-staging extension; this example supplies no such theorem.
For a qualified atomic input group, the corresponding demand condition is legal issue of the complete batch. Each independently staged member's admitted item may then advance without waiting for the other members' items. Whole-batch eligibility and the declared joint retirement law remain required.
Atomic-batch case of K.CT.LazySimulation.Example (informative). Let c have F_c=1, two independent optional abstract staging locations and no credit cap, or C_c>1. Let e be an unreserved positive FIFO with

space for x. The producer executes c.Push(a); c.Push(b); e.Push(x). The consumer legally prepares and issues one same-site atomic input batch {c.Pop(), e.Pop()}, independently of item arrival. The whole batch satisfies LazyEligibility and BatchPlacement; the FIFO member uses coincident capture/retirement without an attributed slot.

Before batch issue, a remains in c's ingress and the producer may hold b pending. Once the batch issues, its c-member demand permits a to advance independently of the missing x. After the ingress release has taken effect, old-state admission permits b. The producer can then reach e.Push(x); after both items are present and the joint guard/service conditions hold, the batch retires with a and x. Item b remains in ingress until its own later demand issues. Movements and dependent issues obey their actual edge laws and Core.RegSeq; this is not instantaneous traversal or a same-edge full-FIFO replacement.

With only a depth-one ingress FIFO on c and the same atomic consumer, a cannot retire before x arrives, b cannot enter before a retires, and the producer cannot reach the send of x. The standard floor-only certificate for the staged case likewise cycles from batch retirement to c.Push(b), then e.Push(x), then batch retirement. Qualified member-demand support releases c's ingress without assuming batch retirement and removes that particular cycle. The complete support graph and all standing conformance, Facts/Agreement, exposure and service premises remain required. Separate sequential Pops on c and e would change the example and can complete with the depth-one FIFO. Thus lazy staging is not generally equivalent to deleting the elaborated structure; any simplifying replacement needs its own qualification.

K.CT.LazySimulation.RTLIndependence (no per-stimulus RTL execution introduced). The demand state used above comes entirely from the pre-HLS execution. The simulator does not consult RTL request signals, RTL progress or RTL completion to decide lazy movement. The implementation supplies the structurally bound capacities, topology and boundary mapping, together with the applicable reusable conformance evidence and instance checks. The completion proof derives the corresponding RTL placement, request and resource progress from the witness support; it does not observe those occurrences in an RTL run.

If another selected qualification independently uses RTL snooping or co-simulation to obtain witness-significant outcomes, that application retains that dependency. LazySerial neither requires such a dependency nor removes one introduced elsewhere. Successful source simulation alone does not establish implementation conformance or the required Facts/Agreement coverage.

Lemma K.CT.LazyInventory. Along such a witness at most one unretired item per channel is outside its ingress, and it belongs to the issued Pop or issued group member. Proof. Initially staging is empty. Serial issue permits at most one outstanding blocking operation per owner; group-serial issue permits one outstanding group per owner, with at most one member per positive channel. Only the item of that demand may move into or through staging. Movement preserves unique ownership and ordinal; retirement removes that item. These facts inductively preserve the invariant. Group members remain indivisible at retirement. Ordinary FIFOs have no staged item and need no movement restriction.

Lemma K.CT.LazySlack (classification at a witnessed Push). Immediately before the old-state admission test for $P_c(k)$, let r count retired matching Pops. There are k earlier committed Pushes and $A=k-r$ unmatched items. The ingress test $Q_c<F_c$ and LazyInventory imply $A<=F_c$. If $A<=F_c-1$, the standard floor predecessor $D_c(k-g_c)$, when $k>=g_c$, has already retired, using $A<C_c$ as well when credit is tighter. If $A=F_c$, credit is absent or $C_c>F_c$; exactly one item is staged, namely $r=k-F_c$, and its demand $D_c(r)$ has issued but is unretired. Its matching admission has committed. Call this the extra-slack case. When $C_c<=F_c$ every admitted Push uses an ordinary retired floor predecessor; the extra-slack case is impossible. An ordinary unreserved FIFO likewise has only the ordinary case.

K.CT.LazyClosure (singleton finite inventory). For a finite lazy Policy S prefix σ accepting R, let $U_σ$ comprise its completed units and its issued, unretired singleton Pops whose matching Push has committed. Close R under the standard endpoint, owner-path, older-stage/iteration, matching-item,

floor and historical control/preparation predecessors, retaining initial roots and independently discharged invariants separately. Pending nodes are auxiliary demands, not W members. Every reached pending node enters through an extra-slack floor edge: Policy S prevents later owner operations from issuing while it is pending, an unretired Pop supplies no completed return fact, and a Pop is never a matching-item admission. This argument uses the restricted singleton profile; mixed composition separately requires complete finite support rather than assuming all other components satisfy this closure construction. Its earlier owner/preparation operations, lower-ordinal Pops and matching Push are completed units. A completed supporting Push can introduce another pending floor demand, so closure is recursive, not depth one. LazySlack shows that each such node remains in U_σ; the complete source-order/control inventory supplies the other predecessors. Thus closure is finite and fixed before any extension. Its pending set D has at most one Pop per owner. Inability to supply the complete inventory or its required source-order evidence is failure of this sufficient route, not permission to omit predecessors. Fixing D before the protected extension is distinct from fixing F before and independently of the workload run. Every consumed completion or support fact introduced by extending D must already be covered by that prospective declaration, or the application must use the new-binding procedure of J.InternalFactCorr.

Lemma K.CT.LazyRank (singleton acyclicity). Define ρ(P_c(k))=(commitσ(P_c(k)),0) and ρ(D_c(j))=(max(issueσ(D_c(j)),commitσ(P_c(j))),1), ordered lexicographically. A completed Pop has rank no later than its completion. Owner-path and control/preparation predecessors commit before target issue by the serial and registered-use qualifications. Required older-stage and older-iteration predecessors use E3 prefix closure, B.OF.Order and the fixed B.BoundaryBinding site/request map to establish their source-earlier membership; Policy S then supplies the same ordering. Pop endpoint strictness comes from serial issue after previous retirement, not merely one transfer per endpoint per edge. Push endpoint edges are strict; matching-item edges increase rank with Push before Pop on a tie. Ordinary floor predecessors retired before the target admission. In the extra-slack case the demand issued no later than the move that released ingress, and that move took effect before the target old-state test; its matching Push also precedes the target by endpoint order. Hence its modified Pop rank is strictly smaller than the target Push rank. Every edge increases rank and the closed graph is acyclic. This lemma supplies ordering, not missing W completions or omitted control/support evidence.

Lemma K.CT.LazySourceExtension (singletons only). Fix a finite set D of issued, unretired singleton demands on selected channels satisfying LazyEligibility. For each demand, its matching item has been admitted, all lower-ordinal Pops have retired, and its required preparation support is complete. The recorded witness satisfies LazySerialWitness on those channels. Assume a legal source continuation window after R: required clocks and reference guard-resolution opportunities continue; the bound testbench permits continuation; and no affecting reset, cancellation or termination removes any demand, item or consumed support until every demand in D has retired. The continuation remains within the same prospectively bound instance and fixed choices. Then a finite legal lazy extension completes D. Eligible singleton demands may also use this local construction in a mixed design under LazyComposition, with the protected continuation window applying to the whole bound design. No group completion or missing sibling admission is supplied by that use. LazyClosure supplies these local premises in the restricted construction. A mixed LazyComposition application instead establishes them directly for its fixed selected extension set; it need not construct restricted global LazyClosure. Structural eligibility alone does not establish these occurrence-specific admission, retirement and support facts. Permitted abstract moves, the reference terminal-transfer or finite guard-resolution law, and the protected whole-design continuation remain required.

Proof of LazySourceExtension. Each unfinished D_c(j) has its admitted oldest unmatched item, with every location ahead empty: lower-ordinal Pops have retired and younger items cannot overtake. Its issued, unretired status permits each selected abstract move under LazySerialWitness. Core.16 permits

this progress of already-admitted work; it introduces no new source issue. Select legal moves through the finite path, each reducing remaining distance. At the terminal apply the declared reference transfer rule, or the reference's qualified finite guard-resolution steps followed by retirement. No new matching admission or sibling input is needed. Preserve the complete mandatory transfer inventory on each edge and decline new optional issues only where legal. Exclusive ownership prevents other activity from stealing the target or its forward destinations. Repeat for the fixed finite D, skipping demands already retired. Mandatory incidental transfers need not enter W unless closure consumes them. This constructs one legal finite source continuation; it does not assume RTL isolation or transfer source fairness to RTL.
Corollary K.CT.LazyExtendedStandard (restricted all-singleton protected extension). Assume the all-singleton LazySerialScope, a finite accepting lazy Policy-S witness with the complete LazyClosure inventory and LazyRank hypotheses, and LazySourceExtension for the fixed pending singleton set D, together with the standing Facts, Agreement, component conformance and service premises. Add the actual completions of D to W and their completion facts to the certificate's consumed inventory within the already declared F and attribution. This same-instance corollary requires that declaration to cover those facts and their consumed support before and independently of the workload run; the pending set being fixed before extension alone is insufficient. The predecessor inventory was fixed before extension; completing D introduces no new required Push targets. LazyRank uses the original issue/admission times and remains valid. Subject to Facts for the declared signature covering the entire extended certificate and Agreement over the actual extended witness, the resulting singleton certificate satisfies K.CT.Closure and K.CT.Standard applies. The protected extension window is a source-witness condition, not an obligation for RTL service after R. In the putative implementation execution that never accepts R, K.CT.Service remains in force and the ordinary completion induction applies. An instance that terminates or resets affected state at R cannot use this extension. A pending atomic group is not completed by this lemma. LazySourceExtension may also be used locally in a mixed design, but that use does not establish these restricted global rank hypotheses. Such an application feeds LazyComposition with its own complete assembled-graph acyclicity evidence; local extension eligibility alone does not justify this corollary.
K.CT.RecordedDemand (fixed support record). A demand is a source-required, legally recorded Pop occurrence, or an individually identified Pop member of a complete legally issued atomic batch. Bind its owner/site, issue evidence, endpoint ordinal, matching admission where consumed, descriptor, finite CtrlSupport and every recursive preparation, phase, earlier-owner, preceding-stage and older-iteration support before the implementation execution being proved. Supporting historical facts must be in F and be supplied by actual W completions ordered before the resource target, or by the independently proved invariants allowed by Closure. Initial roots remain separate. A demand is not a commit unit merely because it has been recorded; its source issue asserts neither an RTL request nor an RTL completion. An actual completed singleton or group may also have a separate demand record when its completion is not used for the selected resource release.
Lemma K.CT.DemandPlacement (singleton target-domain extension). Assume Facts/Agreement, B.CT.Control, B.OF for pipelined owners, applicable preparation/service laws and a RecordedDemand q whose full singleton predecessor support is complete. This includes the same matching admission, lower-ordinal endpoint operations, owner-path, older-work and control-support inventory consumed by the ordinary Pop placement proof, but does not require q itself to belong to W or to have completed. Then either q retires or it becomes permanently isolated at its selected site and its legally prepared operation-owned request becomes and remains asserted until retirement.
Proof of DemandPlacement. Alignment applies to the source-required demand and its prospectively bound CtrlSupport. Historical facts/common roots and the same evaluators fix its descriptor, negative membership and all required earlier work; source issue alone supplies no implementation occurrence. Completed earlier I/O leaves only finite eligible computation and ordered drainage. The

PlacementLemma argument advances older occurrences out, permits legal entry when needed and advances the target through its earlier stages. B.OF.Order prevents younger work passing it or the unretired target passing its site. B.OF.Request and Core.IC give legal assertion and persistence after preparation. Sequential owners use B.CT.Control's finite path. No step uses the target's own completion, W membership, terminal item arrival or an unresolved operation outside the support. This is the existing proof with an explicitly enlarged target domain, not an implementation-conformance assumption inferred from the witness.

Lemma K.CT.BatchPlacement (resource-independent atomic input preparation). For a pure positive input group satisfying LazyEligibility, with independent InputStaging or unreserved coincident-retirement FIFO members, bind the complete legal batch and CtrlSupport for every member/participating owner. Suppose all owner-path, preceding-stage, older-iteration, control, phase and batch-preparation predecessors are complete. Exclude the group's matching admissions only as target resource prerequisites; an admission genuinely required to compute a descriptor or decide control remains a preparation predecessor. Assume the existing arrival-independent placement and request binding, B.CT.Joint and the compatible same-site independent-input clauses. Then either the group completes or its legally prepared member requests become pending at permanently isolated sites, without requiring every sibling matching admission.

Proof of BatchPlacement. Alignment fixes the complete positive and negative membership inventory and legal operands. As in DemandPlacement, completed earlier work leaves finite local computation, ordered drainage and entry/advance to each target site. Neither that argument nor B.OF.Request requires the current group's terminal item arrivals. The qualified complete batch is prepared and asserted without unresolved sibling results and without violating Core.16 activation. Requests persist; no younger work can pass. B.CT.Joint supplies request independence, while the independent InputStaging composition supplies per-input forwarding without waiting for other inputs. Thus an admitted oldest member item can forward even while another member lacks an admission. This lemma supplies placement/preparation, not joint resource availability or group completion. Data-arrival-driven placement, coupled movement and incompatible batches remain excluded.

Lemma K.CT.DemandPushResource. Let missing P_c(k) be an extra-slack witnessed Push, r=k−F_c, and q=D_c(r). Assume the qualified independent operational law and completed supporting W units: every earlier Push on c; Pops 0,...,r−1; matching P_c(r); and q's complete DemandPlacement support, or its BatchPlacement support for an atomic member. Credit is absent or C_c>F_c. These dependencies replace completion of q as the target's floor release. For every service-qualified admitted continuation, either P_c(k) completes or its own ingress and credit resources become persistently available until completion. The statement covers all surrounding activity, including implementation forwarding earlier than in the witness.

Proof of DemandPushResource. While P_c(k) is incomplete, endpoint order excludes later Push completions, so at most k items have been admitted. If q has retired, at least r+1 Pops have retired and A<=k−(r+1)=F_c−1, giving ingress space and credit. If q is unretired but item r is outside the ingress, at least r Pops have retired, A<=F_c, and ingress occupancy Q_c<=A−1<=F_c−1; credit follows from C_c>F_c. Otherwise item r remains in the ingress. DemandPlacement or BatchPlacement establishes permanent isolation and the pending request unless retirement occurs first. Item r is oldest and its forward destinations are empty. The existing isolation-scoped forwarding law and internal service move it out after finitely many steps, independently of missing sibling inputs, reducing to a preceding case. Once obtained, space persists: no later Push can overtake P_c(k), forwarding never returns an item to ingress, and retirement cannot increase inventory. Credit also persists. No bubble compression before isolation or unconditional forwarding under older incomplete work is assumed.

K.CT.LazyDemandCertificate (singleton branch without extension). Use LazyClosure and LazyRank, retain completed units as W, and keep pending nodes only as auxiliary demands. Eliminate each demand node

by replacing every predecessor-to-demand-to-resource-target path by its supporting dependencies. The full recursive support includes any completed Pushes with their own floor/demand dependencies. Path replacement preserves acyclicity. For each replaced floor edge apply DemandPushResource; combine its persistent resources with the target Push's ordinary placement, request and immediate/finite-response guard proof to establish Exposure. Other unit cases retain their applicable exposure proofs. No completion of the auxiliary demand is asserted. This branch is an alternative exposure discharge under K.CT.Witness and Completion, not satisfaction of the unchanged standard floor-edge requirement.
K.CT.LazyGroupCertificate (independent atomic input extension). Use the legally group-serial witness and LazyInventory/Slack per channel. For every extra-slack Push whose demand is a group member, use member-demand support rather than the group's completion floor edge, including when that group actually completes later in the witness. Bind the complete batch-preparation support and the member's matching admission and lower-ordinal Pops. Keep all genuine completion dependencies, including every sibling admission, attached to the actual group unit whenever that group is in W. A pending group is not inserted into W to fabricate a completion. Close supports recursively over the finite completed occurrence inventory and the finite set of legally issued member demands with admitted matching items. All non-resource supports precede the issued group; only the demanded member admission is needed for its ingress release. Distinct positive channels occur at most once in a group. A group record can support multiple member-demand nodes; the singleton one-demand-per-owner bound is not applied to their cardinality.
Lemma K.CT.LazyMixedRank (restricted positive-channel singleton/group certificates). Rank Pushes and actual completed atomic groups by (commit,0). Rank each singleton Pop, completed or pending, by (max(issue, matching_Push_commit),1). Rank each member demand by (max(batch_issue, matching_Push_commit),1). Retain singleton floor edges only when their Pop is an actual W completion, including a completion obtained by protected extension; otherwise use singleton demand support. Replace every extra-slack floor edge from an atomic group by its member-demand support, even when the group later completes. The one-staged-item invariant and old-state release argument place each extra-slack demand or modified singleton Pop rank below its target Push. Matching admissions rank below their Pop or demand by the tie convention. Owner/preparation predecessors commit before the next singleton or group issues; a singleton's modified rank is no later than its actual completion. Lower-ordinal endpoint operations obey serial/group-serial order. These facts cover singleton-to-group and group-to-singleton owner, endpoint and floor edges as well as the cases in LazyRank. For actual group completion, its matching admissions and earlier preparation units strictly precede commit under the declared old-state/registered-use rules. Every edge in this restricted expanded graph increases rank; eliminating demand nodes by path replacement preserves acyclicity. DemandPushResource uses DemandPlacement or BatchPlacement, and actual group units retain complete joint resource/guard/service proofs. This rank theorem does not cover arbitrary additional component kinds; LazyComposition instead requires acyclicity of their complete assembled graph.
Mixed-certificate branch discipline. Singleton demands may use protected extension or demand support; group-member demands use member-demand support regardless of continuation permission. An extension-completed singleton floor edge may coexist with member-demand edges even when that singleton's actual completion follows the target Push: LazyMixedRank orders it by issue and matching admission, not retirement. A certificate with any replaced floor edge is an alternative exposure discharge, not unchanged Standard for the whole graph. Record either the applicable LazyRank/LazyMixedRank proof for the restricted positive profile or a proof/sound acyclicity check of the complete assembled certificate under LazyComposition. Do not combine rank assignments without a common proof. All W members actually complete in the chosen witness, and all auxiliary demands are eliminated before the completion induction.

K.CT.LazyFactsAgreement. In either branch, F and Agreement cover every fact consumed by the resulting certificate, including recorded-demand control/preparation support. F is covered over every finite prefix of every operationally admitted implementation execution, independently of service and success, as K.CT.Facts already requires. An extension enlarges the certificate's consumed fact inventory by newly consumed actual completions already covered by the prospectively declared F and attribution; it changes neither that declaration nor this implementation domain. Agreement covers the witness actually used, including the steps added after R. The selected J adapter must cover every newly consumed fact within the declared signature and all consumed correspondence obligations; determined values/control alone do not establish coverage. If a genuinely different signature or attribution is needed, use the new-instance/header and revalidation procedure of J.InternalFactCorr rather than modifying F in place. Auxiliary demand nodes assert no RTL completion or request occurrence. Implementation requests are derived by placement and preparation. The demand branch may consume further historical support already covered by F but adds no completion fact merely for a demand.
Theorem K.CT.LazyComposition (mixed-component completion). Fix K.CT.Instance, Facts, Agreement and Service, one legal source witness accepting R (including any protected singleton extension actually used), and a finite set W of its actual completed units. Select a set L of channels satisfying LazyEligibility and verify LazySerialWitness on every selected channel. Serial or group-serial issue is required of the consumers of selected channels as specified by LazyEligibility; other owners may use their own qualified witness modes and retain their source-legality, placement, preparation and exposure obligations. Bind all channel, owner, atomic-unit, boundary and fact identities in the same instance. Classify selected-channel Pushes by LazySlack. Ordinary Push cases retain the standard guaranteed-floor resource proof, using Q.CT.InputStagingProgress and $g_c=\min(F_c,C_c)$, or F_c if credit is absent; for $k\geq g_c$ the required floor predecessor is a completed retirement, while $k<g_c$ uses the initial inventory bound without a floor predecessor. For extra-slack Pushes on selected channels use DemandPushResource with complete RecordedDemand support and DemandPlacement/BatchPlacement, or retain the standard floor proof where a protected singleton extension supplies the required actual completion. For every other resource conjunct and unit, retain its applicable standard or separately qualified closure, preparation, resource, exposure and service proof, including JointLemma, SyncLemma and coupled/staggered delivery where applicable. Preserve atomic units intact. Include every cross-component dependency and recursively consumed control/provenance fact. Auxiliary demands may replace only selected eligible extra-slack floor releases; an absent completion required by any other component remains an unsatisfied obligation. After demand-node elimination, supply a complete finite predecessor graph on W and a proof or sound check that it is acyclic. Each local exposure proof must hold under its predecessors in this same graph over all admitted surrounding activity, not only in an isolated component execution. These records shall satisfy K.CT.Witness, including prospective binding and actual completion membership. Then every qualified implementation execution eventually accepts R with the witness values and per-endpoint order.
Proof of LazyComposition. All W members have genuine witness completions; demand elimination adds only justified support dependencies. An ordinary Push on a selected channel obtains persistent space/credit from the standard floor proof, including its initial-inventory case when $k<g_c$. An extra-slack Push obtains persistent space/credit from DemandPushResource, or from the standard proof when the required singleton floor completion has been supplied by protected extension. Each selected Push's remaining placement/request/guard proof then gives Exposure. Other units and resource conjuncts obtain Exposure from their retained qualified proofs under the same completed predecessor graph. The acyclicity evidence supplies a topological order. K.CT.Completion applies with the common Facts/Agreement, Witness and Service premises; its noncompletion argument requires RTL service only until R. A topological sort checks acyclicity only: dependency completeness, actual witness membership, compatibility, fact coverage and exposure validity are separate premises. Cyclic or incomplete

certificates are inconclusive. This theorem permits heterogeneous components elsewhere but supplies no new local conformance, gating, coupled-delivery or reserved-slot theorem.
Corollary K.CT.LazySerialRestricted (restricted rank-derived completion). Assume K.CT.Instance, Facts, Agreement and Service, and every K.CT.Witness requirement other than acyclicity of the constructed prerequisite graph, which is discharged below by the applicable rank lemma and auxiliary-node elimination; the complete B.OF/B.CT/Q.CT conformance, source-control, preparation, endpoint/storage, guard and service premises of the restricted positive-channel LazySerialScope; and a finite legal accepting witness in which every relevant owner uses the applicable singleton Policy S or qualified group-serial discipline. Every staged channel used by this restricted construction satisfies LazyEligibility and the component-bound abstract-movement restriction; ordinary unreserved FIFOs retain their standard resource law. Supply the complete finite support/closure and prescribed certificate construction of LazyClosure/LazyDemandCertificate or LazyGroupCertificate. For every demand replacement bind RecordedDemand, its prospective CtrlSupport and the applicable DemandPlacement/BatchPlacement support. For every source extension retain LazySourceExtension's protected continuation window and Facts/Agreement coverage of the witness actually used. Then LazyRank or LazyMixedRank derives acyclicity of the constructed graph, and every qualified implementation execution eventually accepts R with the witness values and per-endpoint order. No separate acyclicity proof is needed once the applicable rank lemma's hypotheses and prescribed graph construction are established; witness legality, eligibility, dependency completeness, fact coverage and exposure obligations still require validation. This corollary does not impose its global issue discipline on mixed designs covered directly by LazyComposition.
Proof of LazySerialRestricted. LazySlack classifies each selected-channel Push. Ordinary cases retain the standard floor proof; extra-slack cases use demand support or an actual singleton floor completion supplied by protected extension. LazyRank establishes ordering for the singleton construction, and LazyMixedRank establishes ordering for the restricted mixed singleton/group construction. Auxiliary-node elimination preserves acyclicity, leaving only actual completed W units. This establishes acyclicity of the constructed prerequisite graph on W; together with the independently established remaining Witness requirements, it supplies the complete K.CT.Witness obligation. DemandPushResource, the retained standard resource proofs and each target's placement/request/guard proof establish the selected exposure cases; independent delivery and joint proofs cover the remaining restricted units. All premises of LazyComposition are therefore satisfied, which gives completion. Separately, the all-singleton protected-extension result LazyExtendedStandard returns directly to K.CT.Standard and remains an independent sufficient route; it is not needed as a premise of LazyComposition. Neither result requires implementation service after R or claims global deadlock freedom.
Synchronization and extension scope. Ordinary synchronization does not reset global channel ordinals within the accounting epoch. The consumer's preceding blocking operations retire before its synchronization completes, so its pending demand does not cross that completion; A=k−r and the lazy inventory continue across such boundaries. Other owners may synchronize while the demand remains pending subject to the existing operational contracts. Affecting resets, cancellation and capacity withdrawal retain their separate qualifications. A selected ungated independent path needs no new peer admission to finish an eligible singleton demand, but extending its witness still requires a legal whole-design continuation with preserved support and no affecting reset/cancellation/testbench termination through completion of the selected extension set. This does not guarantee completion of pending groups or discharge other components' missing witness predecessors.
Applicability and failure disposition. A failed lazy run, incomplete support inventory, unavailable extension window or rejected certificate is inconclusive, not a deadlock verdict. Exact capacity matching and a passing unrestricted Policy S run do not supply the lazy invariant or implementation conformance. The extra-slack proof admits one staged item beyond ingress only when its demand is supported; it does

not establish unrestricted use of all F_c+m_c capacity. A permissive independent reference with coupled implementation movement can pass this per-item restriction and still deadlock; such a pairing needs its own component-specific restriction and progress theorem. Reserved-slot, coupled, staggered, elastic or other new profiles are not admitted by renaming storage or by changing the floor. K.CT.Standard and K.CT.Completion remain unchanged as the standard discharge and generic theorem respectively. The exclusions concern use of the local lazy resource/batch proof, not the mere presence of these component kinds elsewhere in a LazyComposition design. At-site split-capture reserved slots remain outside the selected-channel and batch profile; their apparent analogy to isolated forwarding is not a proved extension here.

K.CT Applicability and migration

Standard applicability. K.CT.Standard covers the declared finite-stage ordered profile with initially empty exclusive positive resources, source-aligned, legally prepared blocking singleton partial-stage members or indivisible atomic same-stage groups, and the separately qualified direct B(0) components. Positive input storage may use B.CT.FIFO/Capture's at-site law or Q.CT.InputStaging with Q.CT.InputStagingProgress and its compatibility restrictions. A transformed internal singleton may use Q.CT.LogicalPayload with its actual retirement, source/value and selected delivery qualifications. Pure positive groups may combine independently qualified input paths. The named two-input alternative Q.CT.CoupledInputStaging permits equal-length flushable paired movement and consumes K.CT.CoupledDeliveryLemma; Q.CT.StaggeredInputStaging is the separate fixed-offset late-read alternative and consumes K.CT.StaggeredDeliveryLemma. Q.CT.LogicalPair admits their computed representations, including Q.CT.PairedRepresentation after a qualified join, or those of independent paths. Logical retirement remains one terminal pair. Unconditional global stall, staggered process-facing retirements, variable offsets and arbitrary unequal coupled paths are not admitted by these additions. Mixed B(0), retry and epoch-staging combinations do not follow from them. Unary or paired boundaries use Q.CT.Epoch, and observation-neutral logical retries retain Q.CT.Retry's own admitted resource profile. These are conditional mathematical profiles, not claims that every implementation or feature combination conforms. E3 still orders issues across iterations. For fixed-offset, unstalled schedules, the last-to-first issue-offset span cannot exceed II; a data-dependent Pop/Push at II=1 must have issue opportunities one cycle apart. Physical reads farther upstream require a qualified storage/boundary realization such as Q.CT.InputStaging, not a silent exception to E3. K.CT.LazySerial is a separate sufficient discharge for its narrower independent-staging witness profile; its demand branch invokes K.CT.Completion directly rather than asserting unchanged standard floor closure.
Explicit exclusions and extensions. Standalone non-retry NB attempts as W units, multi-member non-atomic residual units, early capture outside the applicable Q.CT.InputStaging, Q.CT.CoupledInputStaging or Q.CT.StaggeredInputStaging clauses (whether single- or multi-member), arrival-dependent terminal placement/request assertion without the separately proved Q.CT.Extension pre-request route, recursive readiness networks, shared credit pools, arbitration outside the explicitly qualified pre-grant B2, B.OF.Service internal-arbiter or B.CT.Guard realization contracts, timing-dependent NB/signal control, resets crossing outstanding obligations, and synchronization among more than two participants require additional stated laws and proofs. These standard-unit exclusions do not exclude otherwise qualified NB observations from safety correspondence; a consumed NB predecessor still needs its own completion discharge before it can support standard alignment. Global-stall behavior excluded by B.OF is not qualified by changing its label. Initiation interval, nested-loop structure and data-dependent trip counts must satisfy the actual iteration/control/finite-closure contracts; this document makes no unconditional claim for all such designs. Q.CT.Extension is the interface for proving an additional class without silently widening the standard one. LogicalPair issue against an occupied result slot, or pair retirement depending on old_empty(U) or depart(U) outside the reserved-empty profile, likewise requires the

explicitly qualified matching handshake, downstream support and Q.CT.Extension evidence stated in LogicalPair.

Witness and implementation responsibilities. The designer fixes R, stimulus/testbench behavior, the selected reference/elaboration and a legal finite witness with complete prerequisites and fact agreement. Reusable implementation-family conformance establishes B.CT.Control's source-aligned control realization and the other B.CT/Q.CT operational laws, while each instance binds its generated boundaries, groups, storage, environment and remaining side conditions. K.CT.Standard and Completion are proved once from those interfaces. A concrete implementation may provide useful conformance evidence, but executing one example is neither a premise nor a proof of the generic theorem. For K.CT.LazySerial, the witness producer also binds the abstract-movement restriction, completed/pending occurrence inventory and demand supports, or the protected source-extension record. This supplies no implementation conformance by simulation alone.

Obligation ledger. O1 commitment and required service: Core.TransferLaw/Core.SyncProtocol for mandatory physical commitment; K.CT.Service/B.CT.Service for the separately required clock/environment, pre-grant and abstract/internal service. O2 request assertion: B.OF.Request and finite legal preparation, including DemandPlacement/BatchPlacement when recorded lazy demands are used, with arrival-independent placement/request evidence for standard staging or a separately qualified pre-request replacement. O3 blocking persistence: Core.IC/B.CT.State. O4 activation-scoped no-new-later-work: Core.16. O5 drain permission versus required movement: B.OF.Drain/Service. O6 ordered target approach: B.OF.Order/Complete/Drain/Entry/Service, discharged by K.CT.PlacementLemma or its qualified DemandPlacement/BatchPlacement target-domain extension after control alignment without assuming the target staging arrival it will support; an arrival-driven alternative supplies its separate delivery/placement proof. O7 local computation: B.OF.Complete/Service. O8 faithful settled enabling: B-faithfulness/B.CT.Clock/Q.CT.Inventory. O9 persistent resources and guards: B.CT.FIFO/Guard and K.CT.ResourceLemma, or Q.CT.InputStaging/InputStagingProgress for independent early inputs, or Q.CT.CoupledInputStaging with K.CT.CoupledDeliveryLemma for flushable equal-length paired inputs, or Q.CT.StaggeredInputStaging with K.CT.StaggeredDeliveryLemma for fixed-offset physical reads, including the declared immediate or finite-response guard realization. For qualified extra-slack lazy Pushes, O9's ingress-space/credit component instead consumes DemandPushResource with its complete support; guards and other resource conjuncts remain. Ordinary Push cases retain floor resources. O10 source witness and any capacity lift: K.CT.Witness/Capacity/Closure, including CtrlSupport for the standard alignment/placement route or the explicitly proved alternative control-support argument. For lazy use, O10 also binds eligibility, trace compliance, demand/extension records and the applicable restricted rank or mixed-graph acyclicity evidence. O11 rendezvous: B.CT.Joint and K.CT.JointLemma. O12 atomic/residual stages: Q.CT.Groups and K.CT.JointLemma/ResidualLemma. O13 source-aligned control and resolved preparation: B.OF.Control/B.CT.Control, K.CT.Alignment/Preparation with standard CtrlSupport, including the supported recorded-demand target domain, or the selected replacement exposure/control-support proof, and Core.ReachPrep/Core.RegSeq. O14 concrete conformance, testbench and service binding: B.BoundaryBinding/B.CT.Conformance/Q.CT.Composition, with LazyComposition for a mixed demand-supported certificate. The lemmas discharge the generic progress implications; O10/O13 and the actual O14 instance evidence remain application obligations, not open steps in the proof of K.CT.Standard. This O1–O14 ledger is a progress-navigation checklist, not an exhaustive list of theorem premises. Every completion application also binds K.CT.Instance, K.CT.Facts and K.CT.Agreement. Synchronization/epoch applications additionally record B.CT.Sync, B.CT.Restore and Q.CT.Epoch, with K.CT.SyncLemma supplying the derived progress result. Logical-retry applications record Q.CT.Retry. M.7a and the selected theorem's full premise list govern the complete discharge record.

Appendix L – Snooping Introduction, Implementation and Concerns

Witness compatibility and non-blocking choices

Definition L.WC (WC: witness-compatible non-blocking outcomes). Because failed polls are modeled as ε (unobservable), any selected witness-alignment or constructive evidence that consumes WC requires witness-compatible comparisons: any ε-modeled non-blocking poll outcome (and any other ε-modeled internal choice that can affect the next Σ-visible control flow / next Σ-visible blocking frontier) is fixed/selected to be consistent with the compared RTL_B execution prefix τ_post (e.g., via the Appendix L witness-selection/snooping wrapper). WC also supplies the μ_Q matching for ε-modeled failed non-blocking call instances when E3/E5 or the proof construction refers to q ∈ Q_exec,P; those failed q's are not inferred from Σ alone.
WC scope and operational legality. Core.SourceChoiceBinding applies: externally resolved choices are distinguished from operationally computed NB outcomes and from annotations recording them. No witness selection or snooping permits an outcome contrary to the declared decision law; the local construction must prove equality of the implementation outcome and the actual source decision at the matched snapshot. Applicable witness-significant choices may be absent, uniquely fixed, snooped or directly constructed; NB-CF is a sufficient method for its stated failed-poll case, not the only admission method. The direct constructive semantic route instead establishes the corresponding outcome and call-attribution facts through its local contracts and matched snapshot; it does not acquire a separate Appendix-L package requirement merely from this paragraph. Evidence may use justified internal predicates, refinement mappings or instrumentation; proof availability does not require a new externally observable interface.
Definition L.NBCF (NB-CF as a sufficient condition). If a design satisfies the following non-blocking control-flow property—failed PopNB/PushNB polls do not influence the next Σ-visible control flow/frontier—then those failed-poll choices need no nontrivial witness selection. This is a sufficient discharge of that part of WC only. Complete WC still requires every other witness-significant ε choice to be absent, uniquely fixed, snooped, or directly constructed under the same selected theorem instance. A reusable NB-CF site classification does not by itself discharge the selected constructive theorem's per-resolution completeness requirement. For those failed-poll choices, the condition means: between two consecutive Σ-events of a process, inserting/removing any finite sequence of failed PopNB/PushNB polls may change only internal ε-behavior/timing, not which Σ-visible action (Push/Pop/synchronization anchor) is next in program order.

Snooping Introduction

If designs are completely latency insensitive, verification of the pre-HLS versus post-HLS models is straightforward. However, if the design contains some components such as arbiters which have latency-sensitive behavior, and if that latency-sensitive behavior is in some cases externally visible to the DUT, then verification becomes somewhat more complex. Techniques can be applied to simplify verification.

Consider a design which has an arbiter like Matchlib toolkit example 09*. The arbiter uses non-blocking PopNB() on its inputs, and blocking Push() on its output to emit the winner of the arbitration. Since the latency between the pre-HLS and post-HLS designs will differ, the order of transactions presented to the arbiter in the two scenarios will differ, and thus the order of the winners will differ. This may result in verification mismatches between the two models if the order of the winners is externally visible to the DUT.

To force the pre-HLS and post-HLS simulations to match, we can run the two designs side-by-side. We can snoop the inputs to the arbiter in the post-HLS model, and only allow the inputs to the pre-HLS arbiter to proceed when the corresponding inputs are seen in the post-HLS model. This will force the order of the inputs to be equivalent between the pre-HLS and post-HLS models, and thus both arbiters will pick the same winners. When this technique is used, the pre-HLS simulation will be throttled by the post-HLS simulation, but the overall verification will still work properly.

Another related example involves interrupt request signals feeding into an interrupt controller within a CPU model. Typically, each interrupt request signal is a single bit sc_signal<>, indicating that an interrupt request is pending. If the requests originate from accelerator blocks that are being synthesized through HLS, then because of the latency differences between the pre-HLS and post-HLS models, the order of interrupt requests arriving at the interrupt controller will differ between the two models, and this will likely result in verification mismatches if the differences are externally visible to the DUT. To force the order of the requests to match, we snoop the requests arriving at the controller in the post-HLS model, and only then allow the requests to be seen in the pre-HLS model.

Simplified Snooping Approaches

In the snooping example directly above in which the arbiter is snooped, the entire post-HLS RTL DUT is simulated alongside the pre-HLS model to force the arbiter requests to be aligned in time. This is the most general case and assumes the input delays to the arbiter are difficult to determine via analysis.

Sometimes there are simpler cases where the input delays to the arbiter in the RTL DUT are easier to determine. For example, each input to the arbiter might arrive a fixed number of clock cycles after one of the primary inputs to the RTL DUT is pushed by the testbench. In this case, a much simpler model can be used to align the pre-HLS model with the post-HLS model. We can simply monitor the primary inputs to the pre-HLS model and then apply the fixed cycle delays to the pre-HLS arbiter inputs.

The two cases above illustrate two ends of a spectrum of possible approaches for extracting the delays from the RTL DUT. Between these two points there exist other possible approaches.

Snooping Implementation

For a selected pre-HLS/post-HLS comparison, Appendix L can align latency-sensitive global signals and non-blocking operations only when a legal source witness exists and L.WC, W1–W5 and L.Dir are discharged. The mechanism below selects such a witness; synchronization alone does not guarantee matching or cover every consumed choice.

As explained earlier in this appendix, in the general case the entire post-HLS DUT RTL must be run alongside the pre-HLS system to achieve alignment. Examples of signals that need to be synchronized include global interrupt request signals (as discussed earlier in this appendix) and rdy/vld signal pairs for non-blocking operations on message-passing channels. All such synchronization signals and their corresponding handshake signals must be level-stable:

- A latency-sensitive signal s is **level-stable** if, once s becomes 1 in cycle t, it must remain 1 until the corresponding handshake signal h is 1 in some cycle t′ ≥ t.

Once the set of signals that need to be aligned between the two simulations is identified, the general rule to implement snooping is simple:

- For each polarity-normalized Boolean commit-enabling snoop signal (for example, the vld/rdy handshake signals of snooped non-blocking operations, or level-stable request lines), make all readers in the pre-HLS simulation see the logical AND of the value driven in the pre-HLS simulation and the recorded post-HLS observation of the same dynamic instance ordinal — captured at the post-HLS-side event and retired at the pre-HLS-side handshake of that ordinal, per Wrapper Realization Requirements W1–W4 below. Payload/data values are never ANDed or substituted; they are checked for equality at the matched ordinal (Requirement W4). The post-HLS simulation runs free: its readers see only its own driven values, and the wrapper observes but never gates, delays, or modifies any post-HLS-side signal. This recorded-observation scheme is applicable only when it realizes a legal execution of the prospectively bound source/protocol model. Core.SourceChoiceBinding, Core.NBDecision and its complete Core.NBDecisionSnapshot, phase and fixed-choice constraints, and any declared timing/isolation contract remain authoritative. The recorder supplies observations, not new scheduling permissions; it may not add a gate to the source semantics merely to force a match.

Often the post-HLS side will not run ahead of the pre-HLS side at the snoop points, because HLS typically only adds (rather than subtracts) latency within each process. This tendency is a heuristic, not an invariant: HLS can also subtract latency, for example by removing conservative waits present in the source, by improving loop initiation intervals, or by realizing channels with fall-through timing. The one-sided rule above is therefore normative, and a symmetric variant — driving the logical AND of the two simulations' nets into the readers on both sides, so that the pre-HLS side can gate the post-HLS side — must not be used, even as a fallback when the post-HLS side runs ahead. Gating a post-HLS-side reader changes the compared RTL transition system; the wrapper is required to observe the selected free-running RTL_B execution without such interference, whether or not B1 is separately assumed. No result about a modified RTL_B′ is thereby a result about the selected RTL_B. Pre-HLS-side throttling is permitted only when its realization remains a legal execution of the bound source/protocol model under the conditions stated above; under-specification alone does not prove this legality.

Two situations must be distinguished when the post-HLS side does run ahead at a snoop point. (a) Benign cycle skew: the RTL asserts a snooped level-stable signal some cycles before the pre-HLS side does, but the order of Σ-visible events at the snoop points is unchanged. Unchanged snoop order alone does not prove existence of a matching source witness. A lag is benign only when the selected source execution satisfies the applicable ordering, value, grouping, phase, fixed-choice and timing/protocol obligations; separately triggered finite-preparation and operational-settling premises retain their own scope. The wrapper handles this case by recording the RTL assertion (level-stability makes the capture clean; the recorded assertion is held until the pre-HLS-side handshake of the same ordinal, per Requirement W1 below) and letting the pre-HLS side catch up, arbitrarily behind in host time. Host-time lag of an otherwise legal recorded replay is not itself an error. Modeled-cycle delay is distinct and must satisfy the selected contract. A per-cycle comparison is diagnostic where cycle equality is not required; it does not waive an explicitly selected timing, anchor or cycle-bucket obligation. (b) Divergence: no admissible pre-HLS execution can consume or produce the RTL's event sequence — its ordinal order, kinds, and payloads — at the chosen boundary; the witness required by WC does not exist. This is the genuine failure mode; it violates Condition L.Dir below, must cause the wrapper to abort loudly, and is

triaged as described under Snooping Concerns. Comparison of the pre-HLS-side AND against the value driven on the post-HLS side remains a mandatory monitor (Requirement W4 below), but its role is diagnostic: a per-cycle mismatch reports skew, while the L.Dir violation itself is defined at the event level — ordinal order, kind, and payload — and is detected by an order/kind/payload check together with a watchdog/timeout that converts witness non-existence, or an inconclusive failure to realize the witness within an unproved bound (Requirement W4), into a reported failure rather than a silent gate or an indefinite hang.

Wrapper Realization Requirements (Recorded-Observation AND)

The AND rule above is stated over a recorded observation, not over the live post-HLS net. A naive realization in which the pre-HLS reader sees the AND of the two currently driven values is a cycle-window mechanism: it delivers a snooped event to the pre-HLS side only if the post-HLS assertion is still asserted when the pre-HLS side is ready to commit. Because the post-HLS side runs free, its assertion window ends at its own handshake, which is unrelated to whether the pre-HLS side has caught up; level-stability guarantees that the signal remains asserted until the post-HLS-side handshake, not until the pre-HLS-side handshake. The live-AND realization is therefore faithful exactly when, for every dynamic instance ordinal k at every snoop point, the pre-HLS-side commit of instance k falls inside the post-HLS-side assertion window of instance k. This per-ordinal window-overlap condition is strictly stronger than Condition L.Dir, and it is an empirical per-run property: it shall be checked, not assumed.

Where the window-overlap condition fails — including on runs that are L.Dir-clean, i.e., runs exhibiting only benign cycle skew — the live AND fails in three ways. First, it drops events: the post-HLS assertion retires before the pre-HLS side arrives, the AND never rises for the pre-HLS side, and the event is silently lost (for example, a lost interrupt downstream, or a pre-HLS process waiting forever on a request that never becomes visible), converting an L.Dir-clean run into an apparent failure or value divergence — precisely the misclassification this appendix forbids. Second, it aliases ordinals: if the post-HLS side runs two or more transactions ahead on the same signal, the pre-HLS side's k-th commit can be enabled by the post-HLS side's (k+1)-th assertion, and pulse-type signals held across two post-HLS instances can merge into one pre-HLS observation, or drop. Dropped and aliased events violate the no-add/remove/reorder premise of Lemma L.2 and break the per-instance μ_Q matching supplied by WC: the harness is then sampling a wire, not replaying the witness. Third, it detects nothing: genuine order divergence (a real L.Dir violation) manifests as a hang indistinguishable from a dropped-event hang, so the triage of Snooping Concerns cannot begin; moreover, the harness-induced infinite wait can prevent required acceptance. It is a wrapper failure rather than evidence of a design deadlock, and is outside any separately qualified finite-delay wrapper service on L.Dir-clean runs. B2 is not a permission to gate away an already sampled physical handshake.

W1 (Recorded per-ordinal observation). For each snoop point, the wrapper shall capture each post-HLS snooped event — the committed transfer for message-passing snoop points, the assertion edge for level-stable global signals such as interrupt request lines — indexed by its dynamic instance ordinal, and shall hold the captured observation for the pre-HLS side until the pre-HLS-side handshake of the same ordinal, at which point the observation is retired. For a single-outstanding interface this reduces to one sticky flop, set by the post-HLS event and cleared by the pre-HLS-side handshake; in the general case it is a small per-snoop-point event FIFO. The AND shall consume the captured observation, never the raw net; level-stability is what makes the capture clean, and pulse-like signals shall be queued as distinct edge events so that dynamic instances are neither merged nor dropped.

W2 (Gate only the commit-enabling mirror signal). Per operation kind, the wrapper shall gate only the mirror signal that enables the pre-HLS-side commit: the observed vld for a pre-HLS-side Pop, and the observed rdy for a pre-HLS-side Push. The endpoint-owned request driven by the pre-HLS side itself is never modified, consistent with the Appendix C non-withdrawal discipline.

W3 (Registered sampling point). In a shared-clock co-simulation, the post-HLS observation shall be registered by at least one cycle before it enters the AND, so that delta-cycle ordering races between the two simulations cannot affect the observed value. This sampling discipline prevents the stated recorder race; it is not by itself a source-legality proof. Any resulting modeled-cycle delay must realize behavior permitted by the bound source/protocol model, its fixed choices, phase rules and actual decision law. Host-time buffering alone does not authorize changing modeled timing or outcomes.

W4 (Monitor and watchdog; Lemma L.5(iv)). At each pre-HLS-side commit at a snoop point, the wrapper shall compare the committed event against the recorded post-HLS event of the same ordinal, in order, kind, and payload. Payloads must match under the selected correspondence and primitive evidence; the wrapper shall never substitute a post-HLS payload into the pre-HLS side, since substitution relabels Σ-events and masks value divergence. A pre-HLS-side snoop commit with no recorded post-HLS counterpart, or an order/kind/payload mismatch at a matched ordinal, shall abort loudly as an L.Dir violation. A recorded event outstanding beyond a configured watchdog bound, or a pre-HLS-side request that remains unmatched beyond that bound, shall likewise abort loudly, but as an L.Dir/WC discharge failure: if the bound has been proved sufficient for the selected instance, the abort may be classified as an L.Dir violation; otherwise it shall be triaged as an inconclusive witness-realization failure — the compared run remains outside the discharged theorem scope, but the abort shall not be converted into a claimed design deadlock or a claimed proven violation. In all cases the abort converts witness non-existence, or failure to realize the witness within the bound, into a reported failure rather than a silent gate or an indefinite hang. Per-cycle skew shall be logged separately as diagnostics and shall not be treated as a failure.

W5 (Coverage remains a separate obligation). Requirements W1–W4 concern the fidelity of the replay mechanism at each snoop point; they do not discharge the first premise of Lemma L.2. If the snooped set misses an ε-modeled choice that can affect Σ-visible control flow, frontier membership, request attribution, guard values, or payload values, the replayed witness is underdetermined regardless of the recording discipline, and the resulting failure shall be triaged as harness incompleteness under Snooping Concerns.

Remark (relation to the naive live AND). The live-AND and recorded-observation realizations coincide on single-outstanding snoop points at which the pre-HLS consumer commits within every post-HLS assertion window — for example, a non-blocking polling consumer that polls every cycle while the pre-HLS side remains at-or-ahead in cycle time at that snoop point throughout the run. Because W4 requires the W1 recording in any case as its comparison reference, gating with the live net offers no economy; the recorded observation is therefore the normative gating term, and the live net may be used only as an additional diagnostic tap.

Shared or replicated reactive testbenches. A shared reactive testbench driven by one model can supply different effective stimulus to the other when their responses have different latencies. Replicating the testbench, even with identical code and random seed, does not by itself solve this: clocked reactions, timeouts or non-blocking outcomes may change later inputs. Bind the stimulus, externally fixed choices,

runtime and environment semantics prospectively, and justify the required interaction correspondence under J.EnvironmentContract (or the selected route's environment contract). Possible implementations include prerecorded stimulus, legally recorded/replayed reactions under W1–W5, or separately qualified replicated or timing-sensitive environments. A reaction defined from the selected DUT-visible trace still requires legal replay, complete consumed-choice coverage and the chosen correspondence proof; trace-functionality alone supplies none of these. Preserve all L.WC and L.Dir obligations when using the wrapper route.
Process-testbench binding. Use J.ProcessEnvClass for joint DUT/testbench or external-contract placement. Snooped observations may supply evidence for actual choices and values, but the selected constructive local theorem still proves legal source decisions, preparation, environment steps and complete prefix coverage. Recording a run alone does not prove universal correspondence.

Definition L.1 (Witness selector / Snooping).

Fix an initial state, fixed external stimulus, and an RTL_B execution prefix τ_post. A witness selector W for a source process P is a partial strategy that, at each ε-quiescent source cutpoint σ_ref and each source-level choice point whose outcome is ε-modeled but may affect later Σ-visible behavior, returns the outcome selected by the matched RTL_B execution.
Such choice points include failed/successful outcomes of non-blocking PushNB/PopNB calls when the outcome affects later control flow, frontier membership, request attribution, or payload/guard values; and any other ε-only implementation choice that can affect the next Σ-visible frontier.
Cadence-sensitive non-blocking polling. If a source process computes externally visible behavior from the number or cadence of failed PushNB/PopNB polls — for example, retry counters, timeout counters, or arbitration policies whose winner depends on how long an input remained empty/full — then that polling logic is latency-sensitive in the sense of Appendix D. Such a process is not covered by the ordinary Appendix J latency-insensitive source-core argument merely by treating the failed polls as hidden ε-steps. To bring such a design into scope, the cadence-sensitive polling logic shall be isolated as a latency-sensitive local protocol block, or shall be aligned by an Appendix L snooping/witness construction that makes the selected source execution an admissible execution of the same isolated protocol behavior observed in RTL_B. The observable interface to the rest of the system shall then be the chosen boundary of the isolated block, not the internal failed-poll cadence itself.
The selected outcome must be causally available from the matched RTL_B prefix and the fixed global Σ-causal history. W is not allowed to depend on future RTL_B events beyond the matched prefix needed to identify the current source choice. For cadence-sensitive polling logic, the witness must select an admissible execution of the isolated latency-sensitive block; it may not manufacture a retry/timeout outcome that is inconsistent with the RTL_B behavior being used as the witness, the fixed stimulus, or the chosen observable boundary of that block.

Lemma L.2 (Snooping establishes WC).
Let τ_post be an RTL_B execution under fixed stimulus, and let W be the witness selector induced by the Appendix L snooping wrapper for the corresponding source execution. The selected source execution and every recorded outcome must satisfy Core.SourceChoiceBinding and the applicable operational decision, snapshot, phase and fixed-choice rules; snooping supplies no exception to WC's operational-legality clause. Assume:

1. the snooped signals / outcomes include every ε-modeled source choice whose result can affect the next Σ-visible control flow, frontier, request-attribution, guard value, or payload value;
2. the snooping wrapper does not add, remove, reorder, or relabel any Σ-visible event except by selecting ε-only source outcomes at the chosen observable boundary;

3. each selected ε-only outcome is consistent with the matched RTL_B prefix and the fixed global Σ-causal history;
4. failed non-blocking polls remain ε-events and successful non-blocking transfers produce exactly the corresponding committed Push_c / Pop_c Σ-event; and
5. if the source contains cadence-sensitive non-blocking polling whose externally visible behavior depends on the number or cadence of failed polls, that polling logic has been isolated or snoop-aligned as a latency-sensitive local protocol block as described in Appendix D and Definition L.1. The WC witness is then applied to the chosen observable boundary of that block, not to an unconstrained hidden failed-poll cadence inside the ordinary latency-insensitive source core.

Then the source execution selected by W satisfies L.WC for the selected witness-alignment use; constructive consumers retain their separate local legality and coverage obligations.

Corollary L.3 (NB-CF identity witness).
If NB-CF holds for process P, then failed non-blocking poll outcomes do not affect the next Σ-visible control flow, frontier membership, request attribution, guard values, or payload values. Therefore no additional snooping is needed for those failed-poll choices when selecting among legal source executions satisfying the actual decision law and the applicable protocol/phase conditions. “Identity / arbitrary selector” does not permit inventing a failure at a snapshot requiring success, omitting a committed successful transfer, or changing a fixed choice. Other witness-significant choices retain their WC obligations.
Corollary L.4 (Cadence-sensitive polling is handled by isolation).
If NB-CF does not hold because the source observes the number or cadence of failed PushNB/PopNB polls — for example, retry counters, timeout counters, or arbitration policies whose later Σ-visible behavior depends on how long a channel remained empty/full — then the design is latency-sensitive at that polling site. Such a site must be isolated or snoop-aligned under Appendix D / Appendix L before applying the ordinary Appendix J source-core proof. If no such isolation, snooping, or equivalent explicit timing model is supplied, the process is outside the scope of the applicable constructive Appendix J correspondence theorem for the selected source model and observable boundary.

Condition L.Dir (One-directional alignment / RTL-side non-interference).
Fix one selected reachable free-running RTL_B execution prefix τ_post under the bound theorem instance, stimulus and choices, and the Appendix L wrapper for a chosen set of snoop points. B1 is not required to define or discharge this selected-execution alignment; a separately selected B1 consumer retains its own obligation. L.Dir holds for a compared run iff the source-side snoop sequence selected by the witness selector W of Definition L.1 is exactly an ordinal-preserving consumption of the free-running RTL_B snoop sequence at the chosen boundary: (i) each pre-HLS-side snoop commit consumes the next recorded RTL_B event of the same snoop point, with matching kind and payload; (ii) no pre-HLS-side snoop commit occurs without such a next recorded RTL_B event; and (iii) every recorded RTL_B snoop event within the compared run is eventually either consumed by the source witness or reported outstanding according to the proved or declared comparison bound of Requirement W4 above. Throughout, the alignment never gates, delays, or modifies any RTL_B-side signal or event (RTL-side non-interference): the wrapper never drives a post-HLS-side reader with a value different from the value the free-running RTL_B drives. Cycle-level lag of the pre-HLS side (benign skew) is permitted under L.Dir provided the ordinal-preserving consumption above is maintained via recorded/buffered replay of the level-stable assertions; order, kind or payload divergence, unmatched source commits, failure of the required recorded-event accounting, or RTL-side interference violates the corresponding L.Dir field. A failure of this selected realization does not by itself classify its root cause or prove that no other legal witness exists; W4 governs bound-exceeded failures. L.Dir is a named side condition with its own entry

in the side-condition discharge record (Normative Formal Core). A run that violates L.Dir provides no theorem-backed guarantee. The harness shall abort loudly on any detected divergence and on any exceeded Requirement W4 watchdog bound, with the classification of bound-exceeded aborts governed by Requirement W4, and every abort shall be triaged as described under Snooping Concerns below.

Lemma L.5 (Online realization of W / harness soundness).
Fix the selected reachable free-running RTL_B execution prefix τ_post under the bound instance, stimulus and choices, together with the source-side witness selector W of Definition L.1. B1, Core.FinitePrefixProvenance and complete-run fairness are not added merely by this wrapper lemma; each separately invoked theorem retains its own premises. This statement is one-directional by construction: no premise of those results requires RTL_B to be stallable by the source, and τ_post is a fixed object rather than a participant that can be gated. Separately, suppose the co-simulation wrapper (i) observes but never gates, delays, or modifies any post-HLS-side signal or event; (ii) records snooped level-stable assertions and replays them to the pre-HLS side in RTL order, permitting arbitrary finite host-time skew (Wrapper Realization Requirements W1–W3); (iii) satisfies the premises of Lemma L.2 (see Requirement W5); and (iv) monitors Condition L.Dir and aborts loudly upon violation (Requirement W4). Then on every run on which the wrapper does not abort, the wrapper computes W online, the selected source execution satisfies L.WC for the selected witness-alignment use (by Lemma L.2), and any result proved over free-running RTL_B + W applies only after its remaining premises are discharged on the declared compared prefix/run. A clean finite monitor log does not establish future consumption, package coverage, source-property validity or liveness; outstanding records retain their W4 disposition. Bidirectional stalling therefore never enters any theorem whose conclusion is transferred to the shipping RTL: legal post-stalls-pre realization selects behavior already permitted by the bound source/protocol model, and the pre-stalls-post direction is excluded by the monitored Condition L.Dir rather than assumed, endorsed, or repaired by gating the RTL.

Snooping Concerns

The snooping technique is used to align pre-HLS and post-HLS latency-sensitive global signals and latency-sensitive non-blocking message-passing operations in cases where their latency differences are externally visible at the DUT boundary. This includes cadence-sensitive polling blocks, such as retry/timeout or polling-arbiter logic, when their externally visible behavior depends on the number or cadence of failed non-blocking polls. When such a wrapper is applied, the pre-HLS model is throttled to follow the latency choices made by the post-HLS RTL, and the resulting traces at the observable interfaces are forced to agree. Snooping does not make failed polls into committed message-transfer events; failed polls remain ε-steps. Rather, snooping selects an admissible execution of the latency-sensitive local protocol block, while the rest of the system is compared at the block's chosen observable boundary.

Readers with a formal background may be concerned by this technique, because it appears to use the post-HLS RTL implementation to modify the behavior of the pre-HLS specification. From a formal point of view, the important observation is that the pre-HLS model is intentionally under-specified with respect to micro-timing/latency: subject to source legality, fixed choices, the operational rules, and the explicit Σ-visible ordering, FIFO, signal-anchor, synchronization, and atomicity constraints used by ≈, it can admit multiple ε-/latency-different executions that respect the same required-order constraints over the chosen observable units. This is compatible with B1, which asserts that the post-HLS Σ-projected trace is unique under a fixed stimulus; the pre-HLS side may still have multiple admissible

realizations that (when projected to Σ) match that unique RTL Σ-trace. In the revised model this timing latitude acts through legal issue, preparation and prospectively bound pre-handshake choices. It does not permit omitting a sampled physical handshake; any wrapper realization must preserve Core.TransferLaw and Core.SyncProtocol. A B-rule or fairness premise applies only when the selected consumer invokes it; finite-prefix witness selection alone imports no complete-run fairness assumption.

Snooping does not change what executions are allowed by the pre-HLS specification. Instead, it selects—purely for verification purposes—one admissible pre-HLS execution whose latency-sensitive events align with those observed in the RTL. In other words, the implementation is used to pick a witness execution of the specification, not to redefine the specification.

Triage of L.Dir violations. A violation of Condition L.Dir (order, kind, or payload divergence at a snoop point, including a pre-HLS-side snoop commit with no recorded RTL_B counterpart) is a relational failure: the compared run is outside the discharged theorem scope, and no theorem-backed claim attaches to it. It shall not be defined a priori as a bug in the pre-HLS model, because witness non-existence has three possible root causes, and defining the event as a model bug by fiat forecloses detection of the third: (1) an ill-formed pre-HLS model — for example, reliance on the ordering of causally independent events, cadence-sensitive polling that was not isolated per Corollary L.4, or a snooped signal that is not level-stable — in which case the failure is of the same character as a standalone pre-HLS test failure and the model is outside the well-formedness envelope of the stress-testing section below; (2) an incomplete snooping harness — the snooped set misses an ε-modeled choice covered by the first premise of Lemma L.2, so the replayed witness is underdetermined; or (3) a non-compliant tool/RTL — the RTL exhibits behavior outside the admissible envelope (for example, an R/B-rule or B-faithfulness violation), in which case the Post → Ref result itself fails and the divergence is evidence of a synthesis defect. In a flow in which the harness has been certified against Lemma L.2 and the tool is trusted, the pre-HLS model is the default suspect and the failure should be filed against it first; the formal status of the run, however, is "L.Dir/WC undischarged — outside theorem scope," not "pre-HLS model incorrect," until triage assigns the root cause. A watchdog abort under a bound that has not been proved sufficient for the selected instance is triaged in the same way but is recorded as an inconclusive witness-realization failure per Requirement W4, not as a proven L.Dir violation. Benign cycle skew (unchanged Σ-event order with the RTL merely earlier in cycle time) is not an L.Dir violation and shall not be classified as a model defect. Experienced SoC architects tend to view this slightly differently, in terms of design tradeoffs and verification cost.

First, it is usually possible at the architectural level to avoid latency-sensitive behaviors entirely, for example by insisting that all communication be latency-insensitive and fully synchronized. However, the quality-of-results (QOR) costs of such designs may be unacceptable. Introducing latency-sensitive behavior—non-blocking arbiters, latency-sensitive global interrupt signals, and so on—is a deliberate choice made by the designer to meet performance, power, or area goals.

Second, in many practical systems, latency-sensitive behavior is not functionally observable at the DUT boundary under the trace-compatibility predicate ≈ with Σ instantiated to include only the DUT-boundary observables (Appendix G; Normative Formal Core above). As an example, consider a DUT that contains a single-port RAM used to exchange data between two internal processes. An arbiter is required to control access to that RAM port, and the arbiter uses latency-sensitive non-blocking Pop operations on its request channels. During HLS, internal latencies will change, so the order in which the arbiter sees pending requests and chooses winners may differ between the pre-HLS and post-HLS models.

In this example:

- The individual RAM read/write operations and the internal arbitration decisions are not directly visible at the DUT interface.
- Only the higher-level IO of the two processes (for example, their message-passing interfaces to the rest of the system) is externally visible.

One might initially conclude that it is necessary to snoop the arbiter's inputs to make the pre-HLS and post-HLS traces match. However, the modeling rules impose two additional constraints:

1. Arbiter service, when required by the selected liveness consumer, uses B2's prospectively defined pre-grant ArbEligible predicate. Prove persistent request/resource/peer/guard support independently of that arbiter's grant, and prove the eventual grant's operational effect. Pending status alone establishes neither eligibility nor eventual service. Recurrently eligible actions require a separately justified recurrent-opportunity or direct finite-service contract when weak fairness is insufficient. Every actual eligible settled handshake commits under Core.TransferLaw; arbitration cannot delay it afterward. These liveness premises are not added to finite-prefix constructive correspondence.
2. **Causal-dependency discipline on shared memory.** The system must make any functionally significant shared-memory dependence explicit, either through synchronization that orders the relevant accesses or through declared array-access mapping / memory-serialization semantics. This discipline ensures that sufficient synchronization or declared serialization exists between the processes so that there are no read/write races through the RAM in either model.

To conclude that different internal arbitration orders permit snoop omission, establish that the allowed joint variations preserve the selected canonical DUT history, including event presence, identity, values, required order and atomic-group structure. Fairness and serialized memory access alone do not prove this boundary noninterference. When those preservation facts and the required legal matching witness/cut availability are established, the differences are incidental for that selected observation and the designated snooping may be omitted. Note 1 and L.JointOptionality below specify the safety-only evidence and retain the complete correspondence and property obligations.

(More precisely: the trace-compatibility predicate ≈ is parameterized by the chosen observable alphabet Σ. If internal functionality (such as the above RAM arbiter) uses channels/signals that are not designated observable in the chosen instantiation of Σ (see Normative Formal Core), then mismatches on those internal actions are outside ≈ by construction: they are not in Σ. If those internal channels/signals are designated observable (e.g., by snooping for debug, or by expanding Σ for K.CT internal-fact correspondence), then they are Σ-events and must match under ≈. In addition, even when B1 holds (unique post-HLS Σ-trace under fixed stimulus), the pre-HLS model may still admit multiple ε-/latency-different executions consistent with the same required Σ-ordering, FIFO, signal-anchor, synchronization, and atomicity constraints; the selected constructive correspondence theorem captures the intended quantification ("existence of a matching witness execution prefix") rather than requiring a single pre-chosen trace.)

Experienced SoC architects therefore try to avoid designs in which latency-sensitive internal behaviors leak directly into the observable behavior of the system under test, since this both complicates verification and makes RTL behavior less predictable. When they do introduce such behaviors—examples include non-blocking arbiters whose winners affect externally visible ordering, global interrupt

request signals at the DUT boundary—they typically know exactly where those latency-sensitive interfaces are and can isolate them.

A useful analogy is a subsystem whose clock frequency is adjusted dynamically based on on-die temperature. As temperature changes, the externally visible behavior of the SoC can change (for example, in terms of throughput or timing of events), and designers may choose such a temperature-dependent scheme to meet overall system requirements. To verify such a system, we may wish to compare a concrete RTL trace captured at a specific temperature profile with a higher-level reference model. To do so, the reference model must be driven with the **same** temperature evolution as the RTL saw. If that temperature profile is not otherwise under direct control, the simplest way to obtain it is to "snoop" the temperature as observed in the RTL trace and replay it into the reference model.
In this analogy, temperature is an external parameter of the environment rather than a fundamental part of the functional specification, but it still influences observable behavior. Snooping simply provides a mechanism to recover that parameter from the implementation when it cannot be easily prescribed a priori. Similarly, snooping of latency-sensitive IO provides a mechanism to supply implementation-specific latency choices — subject to legal source realization, fixed-choice/environment binding, and the explicit required Σ-ordering, FIFO, signal-anchor, synchronization, and atomicity constraints — back to the pre-HLS specification so that the two models can be compared under a common environment. Fairness and other service conditions remain separate obligations only for consumers that require them.

Note 1 (unobservable NB timing). A silent failed poll may still affect later control or values. The local construction must prove its actual source legality and preserve every consumed result; K.CT additionally requires conditional agreement across the workload witness and compared source prefixes. Hiding a poll from Σ does not establish either obligation. NB-CF is one sufficient restricted strategy under its stated conditions, not a permission to choose an otherwise illegal outcome.
Definition L.JointOptionality (per-stimulus and class-wide omitted-set preservation). Fix I_s, its qualified DUT observation mapping, a designated set S of internal choice/snoop sites, retained bindings b, and the bound logical occurrence/epoch identification. A legal resolution r of S specifies only choices allowed by the unchanged source/protocol semantics and b; it cannot change any component fixed by s. For each omitted subset O of S, quantify over all legal resolutions r and r' agreeing on S outside O and on b. Compare finite legal executions at matched logical interaction cuts defined from the declared occurrence/epoch identification and corresponding externally supplied causal inputs, independently of any equality being proved. Equality of responses generated by the varied component is not a premise of the cut matching. Write L.JointOptionality_s(S,b) when every such admitted paired cut has equal CanonHist_Σ_DUT, including event presence, labels/values, identities, required order, synchronization/anchor and atomic-group structure. The cut correspondence must cover the histories consumed by the omission claim in both executions and preserve restriction to their relevant finite ideals; it cannot omit a divergent boundary event, choose cuts retrospectively just to obtain equality, or compare only successful terminating runs. Legal continuation/cut availability is discharged separately; preservation is not an existence theorem. Define class-wide L.JointOptionality_C(S,b) by this condition for every s in C with its parameterized bindings. Optionality of each point with all others held snooped is insufficient.
Consequence for safety-only snoop omission. For a fixed s, start from a legal fully snooped source execution whose matching RTL history is established by the selected correspondence route. For each omitted subset O and each consumed partially snooped source execution/cut, separately establish source legality and the declared cut/occurrence correspondence to that base. L.JointOptionality_s then gives equality of canonical DUT histories; combining that equality with the base's canonical DUT-history equality preserves the safety observation. This uses equality, not transitivity of directional ≈. A class

claim requires the matching base, availability/cut coverage and joint optionality uniformly for every s. Omission removes the designated measurement/replay data, not the existential obligation that a legal matching witness exists, and does not establish property or implementation-prefix coverage. Full proof-internal package obligations remain: Σ_DUT-only optionality cannot erase T-internal matches required by the internal-T profile. No Appendix K obligation is added or waived.

Evidence separation. Record the jointly optional set, retained bindings, logical cut correspondence, domain and boundary-preservation proof under the M.7a L.JointOptionality owner. Per-site NB-CF/WC classifications remain in the WC record. A mixed application names both records and their compatibility; it does not infer a joint-set proof from independent site labels.

Example L.JointOptionality-Positive (commuting internal service choices). Fix one admitted input pair (x,y) and a boundary with two tagged outputs A(x) and B(y) on one FIFO channel. Two private service blocks each have a binary service-order choice. Each choice only permutes two finite deterministic operations on disjoint private locations, with no dependency between the permuted operations. All four operations finish before an explicit modeled join permits the emitter to read their results; the emitter then produces A(x) followed by B(y). All reads feeding those outputs see the same final private values for every joint resolution. The two choice bits are permitted internal variations, not components fixed by s; all other bindings and the input identities remain fixed.

Positive joint proof. Let S contain both service-order sites. Every legal permutation executes each operation once, preserves the disjoint-state result, and reaches the same modeled join; finite operation/handshake completion or the appropriate legal execution evidence is supplied separately. Define logical cuts by the fixed transaction and output ordinals: before the first output, after A, and after B, with internal intermediate prefixes mapped according to their already committed output ordinal. At each consumed cut, all joint resolutions have respectively the empty boundary history, A(x), or A(x) followed by B(y), with the same identities and FIFO-required order. This proves canonical-history equality for simultaneous variations and therefore every omitted subset, not merely for one changed site at a time. Bind the legal fully snooped matching base and source/cut availability as required by the omission consequence; the commutation calculation alone does not supply DUT/RTL correspondence.

Example L.JointOptionality-Negative (single-site checks miss a joint effect). Let u and v be two legally variable internal choice bits, with fully snooped base u=v=0. After both choices are explicitly collected, a legal emitter sends A then B on one FIFO channel unless u=v=1, when it sends B then A. With v held at its snooped value 0, varying u preserves the boundary history; with u held at its snooped value 0, varying v also preserves it. Varying both to 1 changes the history: the first boundary event and the ordered two-event sequence differ. FIFO events are ordered in the canonical boundary history, so this difference cannot be erased as a reordering of independent events. Both program branches are legal; this example does not reverse a fixed mandated A-before-B constraint. It shows why single-site preservation at the fully snooped base does not prove L.JointOptionality for S={u,v}.

Example scope. These examples use the stated boundary abstraction and compare logical cuts, not elapsed-cycle cuts. The positive argument generalizes over any admitted x,y for which its premises hold. In the negative case the failure already occurs at one fixed stimulus. Neither example authorizes omission of T-internal matching required by the primary full-Σ internal-T profile or of separately selected Appendix-K evidence.

For completion, any latency-sensitive interface that affects request preparation, persistent enabling, resource availability or progress must be represented in the declared completion contract. An omitted interface requires a proof that it cannot invalidate those obligations. Its safety effects remain covered by the selected observation and constructive correspondence.

Stress Testing Pre-HLS Models for Latency Robustness

The finite-prefix trace-compatibility results apply under their selected scheduling, implementation-conformance, witness and construction premises at the declared observable boundaries. Fairness and complete-run obligations remain separate premises only where the selected theorem consumes them. Stress testing is a diagnostic method and does not by itself discharge those proof obligations.
These results implicitly assume that the pre-HLS specification is *well-formed*: it must not rely on incidental, implementation-specific timing alignments for functional correctness. Designs that do rely on such incidental alignments fall outside the formal guarantees.

Designs that use non-blocking message-passing operations (e.g., PushNB/PopNB, ac_channel nb_read/nb_write) or latency-sensitive global signals are particularly vulnerable to this kind of fragility. For such designs, it is strongly recommended to subject the pre-HLS model to aggressive *latency stress tests* in which message and handshake latencies are varied across executions. The intent is to validate that the design behaves correctly across the range of weakly fair schedules permitted by the modeling rules, not just under one convenient execution order.

More concretely, verification should check that:

- **Causal dependencies are explicit.** All functionally significant causal-dependency relationships are enforced via message-passing, SyncChannel/ac_sync operations, explicit signal handshakes/anchors, or declared memory-serialization constraints, rather than by relying on the incidental execution order of concurrent processes. In the terminology of Appendix G, the design shall not depend on the ordering of causally independent events for correctness.
- **Weak fairness does not hide deadlocks or starvation.** B2 permits finite delay only at a declared continuously eligible pre-grant arbitration opportunity; it does not allow delaying a sampled ready/valid transfer. Required-workload progress remains conditional on the full K.CT exposure, resource, clock, environment and service premises. Intermittent eligibility is outside B2's weak-fairness guarantee unless a separate service contract covers it.

A failure under randomized latency requires triage. First establish that the perturbation and comparison respect the selected source, environment, capacity, timing and observation contracts. A reproducible property failure on a legal source execution refutes the claimed source-safety premise, but does not by itself prove reliance on causally independent ordering or source ill-formedness. Possible causes include a functional error, an unmodeled timing dependence, an invalid perturbation or an incomplete harness. Use the Snooping Concerns triage when alignment fails. Passing a finite stress campaign does not establish universal source safety, joint optionality or witness coverage.

Informative, release-dependent Matchlib mechanisms described for automating these stress tests include the following. Their availability and semantics are not verified here; qualify the actual library version, mode and configuration before use:

- **Random stall injection on message-passing channels**, to simulate variable communication delays while preserving rendezvous semantics.
- **Latency and capacity back-annotation**, to model specific per-channel buffer depths and delays, including the capacities B(c) chosen during HLS.

Worked examples of this methodology are provided in Catapult Matchlib examples 60* and 72*, which illustrate how to configure randomized stalls and back-annotated latencies when validating that a design

remains well-formed and latency-robust. These are unverified external example pointers, not conformance evidence for the current tool/library release or an arbitrary design.

Appendix M – Formal Results Overview

M.1 Purpose and status

This appendix gives a readable overview of the formal results and the relationship among their principal components. Its overview portions are informative; the explicitly normative discharge and packaging clauses in M.7a retain their stated authority. Those clauses do not replace or weaken the controlling definitions, premises, theorem statements, proof obligations, applicability restrictions, and coverage requirements in the Normative Formal Core and the applicable appendices, including Appendices B, G, J, K, L, N and Q. The overview intentionally uses formal names where they help a reviewer locate the controlling result, but it does not reproduce the complete proof API.

The document distinguishes finite-history safety from required-workload completion. Safety establishes ordered observable compatibility with the selected source reference. K.CT additionally establishes eventual acceptance of a finite required workload under its witness, correspondence, agreement, exposure and service premises. It does not assert strict internal wait-cycle freedom. Neither result by itself certifies that a concrete source, HLS invocation, RTL instance, environment or boundary mapping satisfies its premises.

M.2 Models and observation boundary

The implementation-side model is RTL_B. The source-side theorem target is the selected Sys_ref under the fixed stimulus and the recorded environment, reset, arbitration, witness, and abstraction choices. In the ordinary bounded-channel case, Sys_ref is Sys_B with capacities B(c) faithful to the selected explicit abstract channel boundaries. The rendezvous case is the special case B(c)=0 at the relevant ordinary boundaries. When the implementation contains staged, bundled, joined, coupled, or synchronization-gated realization structure that cannot be represented faithfully by one ordinary bounded FIFO, Sys_ref is Sys_B^elab(E), with E satisfying Q.SafetyQualified. A completion application selects Q.CTQualified in addition to all requirements of its actual J adapter. Every invoked theorem retains its own qualification requirements.

The safety relation is defined over a selected observable alphabet and an ordered trace-compatibility relation rather than over equality of internal states or equality of cycle-by-cycle execution. Observable units include the committed channel transfers, synchronization events, and anchored signal reads and writes selected by the formal model. Independent units may commute or be grouped into cycles differently when the required ordering, value, atomicity, and observation-boundary conditions are preserved. Proof-internal elaboration events and epsilon steps are hidden or projected as specified by the applicable abstraction map.

Shared-memory communication between processes is covered by the proved safety result only under the J.Mem mapping-layer lowering condition stated in the Normative Formal Core. That condition makes memory-carried dependencies visible through operations already represented in the proof alphabet. A direct global memory-state extension is discussed as a possible future extension but is not part of the present theorem.

Current scope and flow availability. ClockScope requires one global clock relation within each selected theorem instance. A multi-clock design may be assessed island by island, but CDC/environment assumptions require separate justification and Appendix O supplies no normative system-level composition theorem. Separately, an otherwise valid Sys_B^elab instance may lack an available executable or independently checkable source artifact. Such a category is mathematically admitted only under its stated premises but is currently unavailable to that evidence-producing flow; no cost or availability claim follows merely from mathematical admission.

M.3 Constructive source-history correspondence

J.ConstructiveEdgeInduction and its qualified local realizations produce legal source prefixes on the actual implementation carrier. The exact Batch lift establishes J.ReachableHistCov over its declared all-finite-prefix domain. Origin correspondence alone covers only the declared origin domain; all-finite-prefix coverage requires the qualified exact Batch lift or a separate exact semantic coverage proof. Exact Batch coverage does not require ideal closure. Source safety uses J.SafeTransfer-HistCov or J.SafeTransfer-HistCovOn, retaining independent source-property validity and, only for the selected-source-history method, its source coverage and all-finite-ideals premises.

M.4 Full facts and required-workload completion

J.ConstructiveFactCorr supplies J.InternalFactCorr for the complete consumed signature under its CF and carrier obligations. K.CT.Completion combines that correspondence with conditional agreement, a legal finite workload witness, closure, exposure and service. The standard qualification binds source-aligned control under B.OF.Control/B.CT.Control and consumes K.CT.Alignment before deriving placement and persistent enabling. Applicable B.CT/Q.CT resource, gate, request, joint and service laws supply the remaining operational discharge. The conclusion is eventual acceptance of required outputs with values and endpoint order, not freedom from every internal wait cycle. The standard witness certificate also includes K.CT.CtrlSupport: complete entry, branch and required-work membership decisions, including negative decisions, with their historical support or independent invariant discharges. The standard early-staging proof requires placement and persistent request assertion to be established without assuming the target item's terminal arrival. A controller whose only route to that placement/request is travel with the item requires the separately proved Q.CT.Extension pre-request route; a data-travelling valid bit alone does not determine applicability. General or extension exposure proofs retain the route-specific control-support obligations stated in K.CT.Witness. For their respective scopes, J.StableReadFactCorr and J.ConfigRecordFactCorr provide the specialized full-fact discharges, retaining their exact-prefix and composition obligations. The informative route table at the head of K.CT.LazySerial distinguishes Standard, LazyExtendedStandard, LazySerialRestricted and LazyComposition. For eligible extra-slack Pushes, recorded demand support and DemandPlacement/BatchPlacement establish persistent space through DemandPushResource without assuming the supporting Pop or batch has retired. Ordinary selected-channel Pushes retain floor resources. Restricted certificates obtain acyclicity from their rank lemmas; mixed certificates supply assembled-graph evidence. Auxiliary demands are not W completions, and Facts/Agreement cover the actual witness and every consumed support fact, including any protected extension. The service-dependent exposure chain uses K.CT.UntilRScope; its nonacceptance case proves completion for every qualified execution without requiring trailing W work to progress after R.

Numbering note. M.5, M.6 and the former M.7b, M.7c and M.8 sections are retired. Retained section numbers are unchanged to preserve references.

M.7 Implementation qualification and evidence

The formal proofs specify a contract between source modeling, implementation behavior, and validation evidence. They do not prove that ordinary synthesis automatically discharges the named conditions. A concrete theorem application must identify the selected Sys_ref and abstraction boundaries; establish source well-formedness and witness conditions; provide the local process, channel, atomic-interaction, environment, reset, footprint, request-snapshot, agreement, and coverage evidence required by the safety construction; and establish faithful realization of every selected channel capacity or elaborated boundary.

Recommended stimulus-class safety application. For safety within J.ConstructiveCombinedProfile, first assess a uniform constructive proof: UCL0–UCL3 of J.UniformConstructiveEvidence supply J.UniformConstructiveCoverage, and independent class-wide source safety supplies J.ConstructiveClassSafe. This covers all finite reachable implementation prefixes for every admitted stimulus under the selected actual Batch carrier, environment, observation and source-model binding. It supports the qualified message/pipeline, wait/signal, configuration, edge-sampled reset and edge-observed R2C combinations; family quantification does not widen those profiles. Exact semantic transfer requires no ideal-prefix-closure premise. The source property must hold on the selected back-annotated/coupled model and its observation/reset semantics. For the prospectively qualified ordinary OM/OA subset, J.OrdinaryConstructiveComposition discharges local-to-global assembly uniformly. Do not infer that result for arbitrary elaboration, reset, R2C, forwarding-signal or START/FINISH profiles. Once uniform local evidence and source safety have been proved, applying the result to an admitted stimulus requires exact parameter binding and residual class-membership/applicability checks, not a fresh substantive mathematical correspondence proof. Instantiated records and witnesses may vary with stimulus and prefix. A finite campaign, one testbench program or reusable structural qualification alone does not prove universal coverage. This preference reflects the direct semantic safety interface; it is not a measured or unconditional cost ranking.

For workload completion, establish K.CT.Instance/Facts/Agreement/Witness, the complete prerequisite closure for the chosen profile, source-aligned control and preparation, and implementation exposure and service. Bind B.OF.Control/B.CT.Control and use K.CT.Alignment to connect witness reach decisions and descriptors to the implementation. B.CT/Q.CT conformance may be proved once for a controller or tool family and bound to each instance; it is not implied by one passing run. Prefer the lazy workflow where eligible. K.CT.LazySerial names the family: LazyComposition is the general mixed-component theorem, LazySerialRestricted its rank-derived specialization, and LazyExtendedStandard the restricted all-singleton return to Standard. See the route table at the head of that section and the route-specific evidence in M.7a.

The proof separates operational legality, constructive finite-prefix correspondence, source-property evidence and completion exposure/service. No one of those premises stands in for another. Reusable controller/runtime qualifications can discharge their named obligations only within the proved validity domain.

Division of verification responsibilities. The following table separates the principal dynamic design-user activity from per-instance flow evidence, reusable tool qualification, and framework work that remains to be completed. Ownership may be shared in a particular implementation; the table identifies the expected primary source of evidence.

Primary source of evidence	Principal items	Current interpretation
Design user	Specify required testbench acceptances R and the actual source/environment contract. Record a finite legal accepting conservative source run, using the qualified grouped mode when needed; check the complete prerequisite graph using guaranteed-floor space edges for the standard resource proof or justified demand replacements for selected eligible channels. Record rank evidence for the restricted LazySerial profile or	The run supplies the finite completion witness, not implementation conformance. Failed sufficient certification is inconclusive.

Primary source of evidence	Principal items	Current interpretation
	checked acyclicity under LazyComposition, separately from dependency completeness and exposure validity. For the standard route, record CtrlSupport entry/branch/membership decisions, including untaken required-work sites, and their support for the closure checks; the flow may generate these records from the bound source witness.	
Per-instance tool / verification flow	Bind source/RTL, actual storage and groups, stimulus, observation and internal fact signature, reset/environment/service assumptions, witness and closure, conditional agreement, and the selected J correspondence. Bind source-aligned control and B.CT/Q.CT conformance or an explicit extension. Use Core.EvidenceHdr/Core.EvidencePackage with all triggered M.7a obligations. Validate CtrlSupport completeness or the selected extension's replacement argument. Check that standard early-staging placement/request evidence does not assume target arrival; otherwise bind the pre-request Q.CT.Extension proof. For lazy applications, validate selected-channel and whole-batch eligibility, actual trace compliance, complete demand support, ordinary/extra-slack classification, and restricted rank evidence or assembled-graph acyclicity. Bind any protected source extension, its added completions and consumed support within the prospectively declared F and attribution separately; a changed declaration requires the new-instance/header and revalidation procedure of J.InternalFactCorr.	A clean source run does not establish these facts. Reusable family proofs may supply them after exact instance binding to the constructive and completion interfaces.
Reusable tool qualification	Prove the selected controller/channel/TB runtime family satisfies operational semantics, boundary faithfulness, source-aligned control under	Rebind or requalify when a validity-affecting tool, model, grouping, capacity, clock/reset, environment, observation or

Primary source of evidence	Principal items	Current interpretation
	B.OF.Control/B.CT.Control, finite preparation, request/movement/guard/joint/service laws and correspondence interfaces. Validate reuse domains and source simulation semantics. A separately claimed B1 determinism property requires its own evidence. Prove the controller family's placement/request-to-delivery dependency is noncircular, or qualify its separate pre-request route. Supply replacement-staging residue evidence where the constructive realization consumes it. For a lazy simulator mode, qualify source-demand visibility independently of item arrival, lawful issue/phase behavior and optional-movement selection with exact old-state and mandatory-transfer semantics. Qualify the selected component and consumer laws consumed by demand placement and resource progress; source-witness serial discipline does not require serial RTL scheduling.	semantic choice changes. A family proof establishes conformance once over its declared class; testing one example is not that proof.
Framework / mechanization work not yet completed	Mechanize shared operational/history/settling interfaces, constructive fact coverage, B.CT/Q.CT and K.CT. Preserve exact domains and independently qualify concrete memory/controller families. Keep standard CtrlSupport and alternative exposure/control-support proofs distinct, and mechanize the component residue obligation under its M.7a trigger. Include LazyInventory/Slack, demand and batch placement, DemandPushResource, auxiliary-node elimination, ranks, protected singleton extension and the named route assemblies listed in the K.CT.LazySerial route table.	The rules provide conditional paper proofs, not completed mechanization or concrete tool qualification. Framework proof work and implementation-family conformance remain distinct; neither is supplied by a passing simulation.

Interpretation of the discharge ledger. M.7a is an explicit accounting of the conditions required for the selected formal claim; it is not a list of separate manual tasks imposed on the design user. Many of the design- and implementation-conformance entries formalize engineering conditions or verification

assumptions that were largely already present in either an HLS-generated or manually written RTL flow but might otherwise remain unnamed or implicit. The ledger makes those dependencies visible, assigns responsibility for establishing them, and permits them to be checked, reused, packaged, or qualified systematically. Depending on the selected theorem and instance, an entry may be inapplicable, satisfied by a simple design restriction, theorem-derived, discharged through reusable tool or library qualification, or established automatically by the per-instance flow. For the functional-safety claim, many of the design- and implementation-conformance entries formalize engineering conditions that an HLS-generated or manually written RTL flow already relies on, such as clock and reset discipline, registered-interface semantics, request persistence, explicit channel behavior, and faithful implementation-boundary mapping. Making those conditions explicit primarily adds checkable evidence and qualification interfaces rather than new design requirements. The safety proof also requires proof-construction, provenance, and coverage artifacts that are not ordinarily produced by a conventional RTL flow. A fully discharged uniform class proof can supply the required mathematical artifacts through theorem instantiation without per-run generation or validation. For the constructive route, the artifacts include C0–C5 process/boundary evidence and, where applicable, A0–A6 anchor evidence, together with the actual BM/AB Batch carrier and complete integration/environment qualification. These are local and carrier proofs over the declared domain, not a certificate-route witness-extension stack. Reusable tool/library qualification, translation validation or independently checked proof records can supply them; the full mechanization and concrete flow qualification remain separate work. A small number of Appendix F conditions are genuine theorem-scope design restrictions rather than merely explicit versions of existing verification assumptions. Required-workload completion additionally needs exact internal-fact provenance and agreement, witness closure, guaranteed resources, source-aligned control under B.OF.Control/B.CT.Control with K.CT.Alignment, legal preparation, persistent enabling, joint exposure, restoration and service. Reusable B.CT/Q.CT controller or component conformance can discharge its stated laws; each instance still binds the evidence and all remaining premises.

M.7a Side-condition discharge record

Before a constructive safety or K.CT claim, maintain an explicit side-condition record bound through Core.EvidenceHdr/Core.EvidencePackage to the exact instance, source and implementation artifacts, local contracts, carrier, environment, choices, capacities, observation/fact signature and property or workload. For each consumed premise record its evidence and validity domain, or prove its applicability guard false. Derived lemmas are not extra assumptions. Unverified tool claims belong to the declared trusted base. This record does not itself prove any missing semantic obligation.

Parameter binding, evidence reuse, non-trigger guards, and invalidation

Reusable evidence/guard registry. M.7a remains the only side-condition discharge ledger. This registry is a logical organization of that existing discharge record; it need not be a separate physical file, package, or second ledger. For each reusable proposition or separately named qualification owner, record once for the validity domain of that evidence: the existing owner/proposition identity; the evidence, theorem, certificate, generator, or checker identity and version/content hash; the minimal validity/reuse key containing only parameters on which that evidence actually depends; the exact trigger/non-trigger/vacuity guard and its binding level (artifact/tool, class/environment, per-resolution/selected-run, or theorem/profile/route); the invalidation/requalification rule; and the consumed-interface/schema status. These owner/evidence fields need not be duplicated in every fixed-stimulus package while they remain unchanged and current. Reuse of one underlying artifact never removes, merges, or strengthens distinct M.7a owners merely because that artifact contributes evidence to more than one owner.
Per-instance binding record. For each selected fixed-stimulus theorem instance I_s, record the current binding/reference from that instance to each reused evidence object, the result of every guard whose

binding level is per-resolution/selected-run, the exact-resolution provenance described below, and all run-specific residual evidence. A stable artifact- or class-bound non-trigger may therefore be amortized inside its declared validity domain, but a per-resolution guard shall be re-evaluated for every I_s even when the guarded obligation remains absent, unless its condition is discharged by an applicable class-wide proof as described below.

Class-wide discharge. A binding or discharge record may be represented by a parameterized family with a proof that its bindings, applicability guards and required evidence are valid for every s in a declared class C. Such a family need not be physically enumerated into one record per stimulus. The record shall identify C, the fixed semantic parameters, the uniformly proved obligations, and any remaining obligations requiring instance-specific discharge. A per-resolution check is replaced only when its condition is itself proved uniformly over the class; evidence reuse alone does not remove it.

Symbolic header binding. A parameterized family denotes the checked mapping s ↦ Hdr(I_s) under Core.EvidenceHdr's existing shared-evidence and distinct-instance rules. Its checker or proof kernel validates the parameterized bindings, applicability conditions and referenced evidence for all s in the declared domain; accepting symbolic fields or sampled headers is insufficient. No new header identity kind is introduced.

J.ProcessEnvClass / J.EnvironmentContract / J.UniformConstructiveEvidence — stimulus-class and environment binding. Trigger: a class-wide or process-testbench correspondence claim. Bind the class and induced instances prospectively, the T placement, runtime and boundaries, and UCL0–UCL3 throughout the admitted class.

Discharge: for internal T prove the same complete local contracts, settling graph and actual carrier as for the DUT. For external T prove C0/A0 correspondence for each actual step and consumed observation under the fixed J.EnvironmentContract; equal DUT histories or a more cooperative environment do not supply that proof. Source-property evidence remains separate.

Evidence reuse follows Core.EvidenceHdr and the registry validity keys. Changes to T/runtime, choices, reset, attribution, capacity, elaboration, observation/fact signature or coverage domain require revalidation of every affected fact. Sampled runs do not establish a universal environment contract.

T source/realization conformance — coordinating qualification owner. Trigger: T is included in a claimed J.ProcessEnvClass internal comparison, or a declared external representation relies on corresponding T realization facts. Bind the program, runtime/scheduler, libraries, wrappers/transactors, supported versions/modes and configuration, and the selected formal boundaries. For primary internal T, consume reusable runtime tool qualification and prove the design's applicability to its supported usage domain. Custom or unsupported interface behavior requires additional qualified or design-specific evidence; unchanged source text alone is insufficient.

T conformance discharge and evidence. Reference each triggered normative owner for source/control semantics, Core.ImplIssueOrder/R/E order, Core.IC request attribution/persistence and stable payload, Core.RegSeq/direct-input/signal anchors, channel capacity/faithfulness and elaboration, clock/reset, memory/footprints/noninterference, witness choices, and the selected route's phase/settling and preparation facts. Appendix F provides informative guidance, not additional normative preferences or blanket Appendix-K premises. Tool qualification may supply reusable semantic evidence; the instance binding proves its preconditions. Lint/static restrictions, formal semantics/refinement proofs, qualified library evidence and interface checks are alternative or complementary evidence forms. A separate lint artifact is not mandatory when another accepted proof discharges the condition. Existing owners remain authoritative and are referenced rather than duplicated. The direct route retains its stated finite-preparation/B4 and global-QSync non-trigger cases when applicable.

Uniform-route semantic evidence. The T realization binding supplies a formal model or precise semantic contract and refinement evidence for its actual runtime and DUT integration. It must support the roots, primitive actions, values, local frames, environment steps and complete carrier inventory consumed by

UCL0–UCL3 on the full declared domain. A simulator name or unchanged testbench source alone is insufficient.
Combined implementation qualification binding. Whenever a selected consumer requires an implementation-side carrier or settling predicate, or a completion predicate of the selected carrier profile, its subject is the complete DUT RTL ‖ T_post implementation under the bound integration semantics. Bind the runtime/RTL integration evidence, including initialization/time-zero, event/delta-region ordering, TB signal updates versus RTL registered updates, clock/reset handling and the selected boundary/commit mapping. Separate qualification of the RTL simulator and TB runtime does not by itself qualify their composition. Reusable tool qualification may establish the combined predicates for a declared integration class; each application proves membership and any remaining instance-specific conditions. Existing ClockScope, Core.QSync and the selected constructive/Batch carrier owners remain authoritative and are referenced here, not duplicated.
L.JointOptionality — safety-only omitted-set evidence. Trigger: a claim omits designated snoop/replay data using L.JointOptionality. Bind I_s or C, S and retained choices b, the existing qualified outer mapping, all admitted legal variations/subsets, and the independently declared logical cut/occurrence relation with its coverage. Prove canonical DUT-history equality jointly, and separately establish the legal fully snooped matching base and availability/legality of the source histories consumed after omission. Record exactly which data may be omitted and which witness/construction obligations remain. Reject use as a substitute for wider proof-internal correspondence or Appendix-K evidence. Per-site NB-CF evidence belongs to WC and cannot by itself discharge this joint-set owner.
Exact-resolution provenance. Before accepting run-bound evidence, verify that the selected RTL run, source/environment realization, reset realization, witness/WC choices, and every arbitration or other nondeterministic choice declared to be resolved by s agree with the s recorded in I_s. Tool-semantic choices not represented as components of s—including any relied-on initialization/X treatment, event-region/delta scheduling, race semantics, clock/reset interpretation, and other history-affecting simulator/tool settings—shall instead agree with the semantic profile of the bound qualification evidence. Two executions with different such bound choices shall not be treated as the same fixed-stimulus instance merely because their user-visible test data are equal.
Reuse-key and invalidation discipline. A reuse key shall contain only truth- or evidence-validity-affecting parameters for that owner; parameters shall not be added solely because they occur elsewhere in the theorem instance. Changes in artifact/tool semantics, capacities/elaboration, lowering, boundary/projection, arbitration, reset/environment contract, normalization, Σ, Φ/checker configuration, initialization/X behavior, delta/race behavior, route selection, or consumed-interface schema/version invalidate only the owners, guards, or stored evidence whose declared keys depend on those changes. Each consumed normative definition is identified by its semantic revision or definition-level content hash, independently of serialized schema versions; a change in meaning can invalidate dependent evidence even when its serialized representation is unchanged. Bind the consumed definitions through the Core.EvidenceHdr interface manifest and the owner's declared dependency key, including referenced definitions on which the evidence depends. A whole-document revision alone does not provide definition-level dependency identification; unchanged definitions need not receive new semantic revisions solely because unrelated text changes.
R/E scheduling-contract conformance — R1–R5/E1–E5, including R2C where applicable.
Trigger: Required for every theorem or flow-qualification scope that invokes the Appendix G–K scheduling contract. Apply the applicable R1–R5 source rules to the selected Sys_ref behavior and the applicable E1–E5 implementation/compatibility rules to the compared RTL_B behavior at the selected abstraction and observation boundaries. The R2C fixed-point semantics and matching obligations are required whenever combinational processes or explicitly modeled combinational networks are in scope.
Discharge: Bind the concrete source/tool/RTL instance to the applicable R/E rules. Establish the required

source dynamic-operation/source-order and synchronization/signal-anchor legality; post-HLS synchronization-order preservation; sequential signal visibility/anchoring or R2C fixed-point observation and matching; prefix-closed message issue order under Core.ImplIssueOrder as the implementation-side E3 owner; E4 explicit-boundary/channel-capacity legality; and E5 synchronization-boundary issue/completion ordering. For ordinary scalar blocking operations on one endpoint, E3(i) and the same-interface portion of E3(iii) may cite the separately discharged Core.IC/BoundaryReq qualification from which that Issue order is derived; E3(ii) and the cross-interface prefix-closure content remain independent implementation scheduling checks. The record may cite constitutive Sys_ref semantics where a source rule is built into the normative operational model, but corresponding RTL_B uses shall be backed by tool/RTL/checker evidence or a proved refinement/conformance result. Evidence-reuse rule: when the underlying E4 boundary legality or Core.IC request-lifecycle/attribution obligation has already been authoritatively discharged for the selected boundary and instance, downstream B-faithfulness, the channel primitive contracts, correspondence or completion consumers may cite that same checked evidence rather than re-prove the underlying legality merely because it appears again in their package interface; each consumer still must discharge its distinct layer-specific mapping, abstraction, or correspondence obligations. This roll-up does not replace separately named entries such as Core.RegSeq, Core.SelectLatitude, Core.IC, Core.SyncFence, B-faithfulness, Core.MemFaithful, Core.MemSyncOrder, J.Mem/Core.MemProfile, or RW; those remain separately discharged when triggered.

Evidence examples: source well-formedness/lint and simulator-semantics qualification; HLS schedule and issue-order certificates; RTL/interface assertions or formal checks for synchronization and signal anchoring; an R2C combinational fixed-point/unique-settling certificate; and explicit-boundary E4/capacity conformance checks.

Core.RegSeq — registered sequential-interface scope and conformance.

Trigger: Required for every Appendix J or K theorem instance containing a sequential process whose message requests, Push payloads, or continuation can depend on a message result.

Discharge: Classify each process as R2C combinational or Core.RegSeq sequential. For each sequential process, prove or validate that process-driven request and Push-data outputs for Horizon t are determined from registered state and operands stable before the L2 settle point; that direct inputs satisfy their E2/direct-input stability contract; and that every use of a cycle-t returned data/status result has strictly later Horizon. Tool-side Cat# or equivalent schedule/RTL evidence and source-side simulation, lint, wrapper, or Appendix L evidence shall identify and reject, isolate, or explicitly remodel zero-time NB-result cascades and retry loops. Evidence examples include schedule reports, control-step allocation, generated RTL cone/register checks, source lint, and cycle-accurate transactor or snooping artifacts.

> Core.ReachPrep — legal overlap-aware preparation. Trigger: any constructive extension or grouped/overlapped workload witness that reaches later work before an earlier blocking call returns. Prove each finite preparation segment under the independently defined source relation, fixed operands/choices, Core.RegSeq, Core.16 and the transitively closed NoDependentUse/footprint conditions of Core.DeferredNBHole. All admitted RTL successors must be covered; failed construction cannot narrow the domain. Issue remains a separate legal transition.
>
> Core.SelectLatitude — issue and abstract-action latitude. Trigger: a proof defers a legal L1 issue or a genuinely abstract action with a declared permissive law. Bind the delay to the actual source relation, phase, fixed choices and exact prefix. Pre-handshake arbitration must be represented in the complete network. Neither this entry nor witness annotations can omit any Core.TransferLaw/Core.SyncProtocol completion. Revalidate older L3-nonselection examples

and constructors; their absence of commitment is not legal evidence for a present handshake in the revised model.
ClockScope — single-global-clock theorem scope.
Trigger: Required for every theorem instance using the formal results of Appendices G–K.
Discharge: Confirm that the selected theorem instance is interpreted under one global clock/cycle relation for every process and explicit abstract channel to which the G–K rules are applied. A larger multi-clock design may be partitioned into synchronous islands, but any clock-domain crossing used by the selected theorem instance must be excluded from the single-clock comparison or represented by a separately specified and verified CDC/refinement boundary under an explicit multi-clock semantic layer. In the absence of that layer, the G–K theorems shall not be applied across clock domains. Evidence examples include a clock-domain/elaboration manifest, structural CDC partition report, or an equivalent checked instance-scope declaration.
Island-composition boundary. Separate applications to synchronous islands do not, in this document, compose into a system-level result across CDC boundaries. Each island's external/CDC behavior is part of its bound environment assumptions, which must be independently justified. Appendix O is informative and supplies no general theorem discharging that composition.
Core.QSync / actual carrier qualification. Trigger: the selected proof consumes Core.SourceL2Settle or Core.QSync-Post; discharge the corresponding source or implementation qualification below. For Core.SourceL2Settle, prove the complete source QS1–QS5 network, legal finite topological evaluation and fixed-point realization; Q.1(i) covers only the template portion. For Core.QSync-Post, prove QualifiedSynchronousModel_RTL_B (QS1–QS5), bind each consumed reachable origin occurrence and its full fixed frame to the independently declared settling domain, and establish finite legal operational settling. This does not supply arbitrary-prefix coverage or real-cycle completion. Registered constructive carriers retain their separately named BM/AB and, for consumed facts, FB obligations. Qualify the combined DUT/testbench integration, not isolated components only.
Core.CycleClosure — occurrence-relative source-cycle qualification. Trigger: a proof invokes Core.CycleChoiceExt or constructs a source continuation requiring this qualification. Establish full-state CycleResult typing, finite phase execution, exact-prefix partial-cycle extension and compatible environment definedness. Do not splice a successor from another history. This premise supplies neither fair selection nor implementation progress.
Core.IssueFair — source issue fairness. Trigger: a K.CT or other complete-source-execution argument invokes eventual issue. Prove eventual selection of every operation continuously IssueEnabled at the applicable L1 opportunities, under the actual fixed choices and Core.16 restrictions. A finite supplied workload witness does not require a complete-run fairness proof merely to be legal.
Core.EvidenceHdr / J.InternalFactCorr / Core.FinitePrefixProvenance — completion binding. Trigger: K.CT.Facts or K.CT.Agreement. Bind the exact R acceptance contract, legal finite workload witness, prospectively fixed implementation and source-prefix domains, F signature, attribution, roots and provenance proof, and selected J adapter with all its premises. J.InternalFactCorr covers every finite prefix of every operationally admitted implementation execution quantified over by completion, independently of service, success or construction success. F includes R acceptance facts, every W member's completion facts, and all consumed historical control/value predecessor facts and their recursive support used by K.CT.Alignment. Every W member occurs with its declared completion in the supplied legal witness; hypothetical additions do not qualify. Bind occurrence-to-availability in every admitted source prefix, including the workload witness, while preserving Core.RegSeq's later-use restriction. Record

state/exposure invariants separately from history correspondence. Use Core.EvidencePackage. For the constructive adapter bind CF0–CF3 of J.ConstructiveFactEvidence, including local primitive/return/anchor emission, frame proofs and the complete implementation F-prefix domain. If Core.RegisteredFactBatch is used, bind FB0–FB2 and Core.RegisteredFactPrefix to that same actual carrier. List split capture/retirement and consumed non-transfer outcomes explicitly. Record the prospective declaration separately from the certificate's finite consumed fact inventory. Same-instance protected extension may enlarge only the latter, within the declared F and attribution. A changed F or attribution follows J.InternalFactCorr's new-instance/header procedure and requires revalidation or explicit evidence transport for the affected fact, agreement, provenance, witness and attribution obligations. Changes to F, attribution, the carrier or completion convention invalidate reuse unless covered by the prior proof's validity domain or a proved transport. For direct inputs/configuration, select J.StableReadFactCorr or J.ConfigRecordFactCorr and record the full-F lift or configuration inventory, exact-prefix carrier evidence, governing-record attribution and common-root/historical-fact distinction. Outer safety coverage is insufficient. For data-selected predecessors, record the fixed support and selector validation, lower-ranked source/availability argument and unchanged provenance-domain coverage described in Core.FixedSupportValidation, or bind the independently proved K.CT.Agreement alternative. B.CT / Q.CTQualified / K.CT.Standard — standard completion conformance. Trigger: a completion application claiming the standard operational discharge. Bind actual controller/environment conformance to B.OF where used, explicitly including B.OF.Control and B.CT.Control source-aligned control realization and finite preparation. Record these as family-wide or complete per-instance conformance obligations under Core.EvidenceRealization and consume K.CT.Alignment before PlacementLemma and K.CT.Standard exposure. Bind B.CT.State, resource/endpoint-order, guard, joint, clock and service laws; compatible Q.CT components; and K.CT.Closure/Preparation with all owner-path, earlier-stage and older-iteration prerequisites. Preserve all J.InternalFactCorr adapter premises. Bind Q.CT.Epoch's execution-level protection against later gate closure/withdrawal before the protected target retires, including before its request is pending, and B.CT.Restore for pending transfers. The standard partial-stage profile uses singleton units and per-member service; standalone non-retry NB attempts and multi-member non-atomic residual units require their separate extension proofs. Bind the actual witness capacity, guaranteed floor and mandatory floor-based space dependencies. A smaller-capacity witness additionally requires the source lift of K.CT.Capacity; a larger nominal witness capacity does not weaken Closure. A family proof may discharge operational laws, but each instance still binds topology, boundaries, capacities, groups, witness and service validity. Any interleaving use additionally binds the precise K.CT.ClockRefinement profile or its proved replacement. The B.BoundaryBinding record distinguishes physical capture/movement, process-facing sites and requests, and source launch under Core.16. For early input storage, bind Q.CT.InputStaging and consume Q.CT.InputStagingProgress for independent paths, or bind Q.CT.CoupledInputStaging and consume K.CT.CoupledDeliveryLemma for equal-length paired paths, or Q.CT.StaggeredInputStaging and K.CT.StaggeredDeliveryLemma for fixed-offset late reads. Record topology, per-channel invariant/floor, source model, isolation-scoped forwarding, bubble drainage and actual joint internal service where required. Other early capture or output transport beyond the stated profiles binds its Q.CT.Extension rather than inheriting qualification from stage count. Record the immediate/finite-response guard choice, its preserved support and local decreasing measure where used, including joint readiness and shared-resource service; keep K.CT.ClockRefinement's narrower assumptions explicit. For Q.CT.InputStaging, additionally

record old-empty-only versus simultaneous-replacement internal acceptance, exact action selection and old-payload updates, and the distinction between internal locations and explicit E4 primitive channels. Throughput claims require separate supply, sink, ingress-depth, guard/credit and advance conditions; completion service alone supplies no rate. Record CtrlSupport(q), including negative membership and entry decisions, their complete historical support in F, preceding W producers or independently proved invariant discharges, and coverage of all older-work/earlier-stage membership used by PlacementLemma. Check the zero-based floor edge at k=g, the frame for pre-issued younger completions, and independence of standard placement/request proofs from terminal arrival. Token-driven arrival/placement uses the separately proved Q.CT.Extension route. Logical-retry trigger: whenever K.CT.Standard uses Q.CT.Retry, record the persistent observation-neutral logical obligation and its concrete attempt mapping; prove that failed attempts consume no required item/credit and do not count as completion, that guard progress is retained across retries, and that request/opportunity/service and resource behavior meet Q.CT.Retry's admitted profile. Bind the successful retirement and complete source/fact/choice evidence. A retry label alone does not qualify arbitrary NB polling or additional resource combinations. Record the K.CT.UntilRScope binding to the same prospective R for all service-dependent supporting lemmas and assembled exposure proofs. Do not replace it with unbounded trailing-work service or restrict Facts, Alignment, resource invariants or the operationally admitted domain. A stronger unrestricted progress claim requires its additional service evidence explicitly.

Q.CT.InputStagingResidue — constructive realization of replacement-capable staging. Trigger: a J local construction uses simultaneous-replacement internal staging transitions, directly or through a qualified coupled, staggered or representation profile. Owner: the component/local-proof provider supplies the residue and source-transition proof; the integration flow binds the exact topology, selection, ownership and dependencies. Discharge: fix T and old owners/payloads; prove an acyclic combined order, predecessor-closed occupancy/value residues, legal C3 premises and updates, and complete frames. Bind actual joint groups and representation maps where used; retain every explicit E4 boundary law. An equivalent C3 proof is permitted only with an explicit mapping to these obligations. Evidence is reusable only within its proved component domain. This supplies local realization evidence, not ELM/ELA primitive expansion, source-witness lifting or delivery/service progress.

Core.LogicalPopPayload / Q.CT.LogicalPayload — internal logical payload representation. Trigger: a consumed internal process-facing Pop is assigned its matched admission value although that raw value is not physically retained. Bind the endpoint's exclusion from physically sampled observations, actual retirement/call attribution, causal admission record, complete old-owner movement/occupancy relation and total nonrestricting observer extension. Prove the physical representation and legal later use with exact arithmetic, source control, Core.16/Core.RegSeq and occurrence/footprint constraints; preserve actual readiness and all physical observations. Consume J.LogicalPayloadRefinement with complete remaining local, generator and full-fact/carrier cases as applicable. For a positive computed pair, additionally bind Q.CT.LogicalPair: corresponding old owners, actual private combination, complete same-window issue, the selected reserved-empty result-slot invariant (single shared U or an explicitly qualified per-occurrence slot assignment), the chosen independent, equal-length coupled or staggered forwarding law, two-return atomic packet and joint service when completion is consumed. This record supplies no source-witness lift or service by representation alone and cannot be discharged by the workload witness itself. A combined cache additionally binds Q.CT.PairedRepresentation: two retained logical owners, actual join/stage/retirement transition equations, exact machine arithmetic and equality to the legal source expression, without new readiness or capacity. The staggered case records the early owner, late FIFO-head match and

older-work drainage past a missing younger read. Bind the actual result-storage realization. For a single shared U, record reservation and safe release with the stated rate limitation. For a multi-slot assignment such as U[j mod 2], prove target-slot emptiness at issue, exclusive ownership through retirement and safe consumption/advance, legal reuse, output order, backpressure, and source/control/resource correspondence. For occupied-slot issue or departure-dependent retirement, bind the separate matching handshake, B.CT.Guard and K.CT.Closure downstream support and complete Q.CT.Extension evidence; the reserved-empty proof cannot discharge that gate. Two slots alone establish neither a sustained II=1 rate nor the required service timing.

B2/B3 — pre-grant service and primitive channel progress. Trigger: K.CT.Service or another explicit progress consumer. Bind actual requests, complete joint units, clock/environment/reset scope and resources. For B2 exhibit ArbEligible before the arbiter's own grant and prove its continuous-support antecedent independently, then establish eventual grant and its actual effect. Do not use ChannelEnabled as a post-handshake selection opportunity. Recurrent eligibility or a direct finite-response promise requires its separately named and proved contract. Primitive uncoupled B3 discharge and subsequent eligible-edge commitment follow from E4/Core.TransferLaw once the stated complementary requests and continuing edges exist. Coupled boundaries retain their complete Q-level exposure and service obligations. Source legality, finite-prefix correspondence and a passing witness do not establish these progress premises.

B1 — observable RTL determinism. Trigger: the flow separately claims this fixed-cycle observable determinism property. Qualify the selected alphabet, initial/reset and environment contracts and actual RTL/runtime semantics. B1 alone supplies neither implementation-wide constructive correspondence nor source-property validity, conditional source agreement or completion. No single-RTL-run safety shortcut is provided by this document.

Tool/simulator-semantic qualification — reusable evidence for RTL and testbench realization. Trigger: a selected constructive or completion conformance obligation relies on tool/runtime semantic evidence, or the J.ProcessEnvClass internal-T realization uses a qualified runtime. Bind the family/version, RTL/TB execution modes, libraries/wrappers/transactors, initialization/X behavior, event-region/delta scheduling, race semantics or exclusion proof, clock/reset and every behavior-affecting option. Identify the proved propositions and supported artifact/configuration/interface domain, and establish the actual semantics consumed by each use. T runtime evidence references the T-conformance owners and UCL0–UCL3; omitted runtime details require an adequate abstraction argument. A separately claimed B1 property needs its own determinism evidence; runtime qualification alone does not imply it. Record unverified assurances in the trusted base. Reuse requires a valid evidence key and proof of instance membership; changed options, versions or unsupported behavior require requalification. Qualification does not replace L.WC where consumed, source legality, complete prefix/fact coverage or any constructive/completion premise.

B4/FPD — finite internal source progress. Trigger: a finite preparation, normalization or source-cycle existence proof consumes this bound. For each relevant process, provide a finite bound or well-founded measure showing that consecutive internal work reaches the next observable action or a stable external/peer wait. Core.PreparationEvidence binds each selected segment to actual source legality; Core.CycleClosure additionally needs the uncapped per-cycle finiteness premise it states. This bound alone neither makes blocked work ready nor guarantees fair implementation movement. Record each consumed use separately in the instance ledger.

Endpoint, reset and synchronization well-formedness. Trigger: the selected instance uses scalar endpoints, initialized/reset buffered channels or paired synchronization. Discharge the Core

single-transfer-per-cycle endpoint constraint, and B.CT.EndpointOrder for completion; use an explicit qualified multi-lane model for higher cardinality. Discharge the Core reset-empty FIFO convention and B.CT.State's empty-start condition where applicable, or explicitly model and qualify any preload. Bind paired-sync participation and Core.SyncFence; completion additionally requires B.CT.Sync and the applicable gate/restoration laws. These checks do not infer eventual peer arrival from syntax: the completion closure/exposure proof supplies that progress. Evidence includes interface cardinality checks, channel initialization/reset realization and synchronization/protocol conformance.

Core.IC — request lifecycle / Issue–Commit conformance.

Trigger: Required on RTL_B whenever the selected theorem or certificate uses implementation-side Issue/BoundaryReq/Commit attribution or blocking-request persistence, including the applicable E3/E5 trace obligations, constructive root/fact attribution or K.CT request persistence. On Sys_ref, Core.IC is constitutive operational legality and is not a separate premise to be assumed or independently discharged.

Discharge: For every admitted explicit boundary and endpoint direction in the selected scope, establish the applicable Core.IC lifecycle and request-identification rules. First, the selected boundary mapping shall satisfy Core.10's fixed, bidirectionally complete request-identification rule and exhibit the implementation request condition/owner decode; an arbitrary physical ready/valid level caused by complementary channel/storage availability or declared coupler logic is not a source request. For blocking operations, also discharge Core.IC(1)'s general live-owner invariant; once issued that process-owned request remains asserted with stable Push payload until its process-facing commit or an affecting RW2 RESET; no hidden nonterminal cancellation, withdrawal, retry, replacement, or reattribution is permitted. After blocking q_o commits, if the same process-owned request remains asserted in the next cycle and q_1 is the next same-direction blocking source operation in that source interval, q_1 owns the continuing request and Issue(q_1) is that next cycle; ownerless blocking-request latency is not an attribution option. The scalar same-endpoint serialization rule and synchronization-boundary attribution/Core.SyncFence rules must also hold. For ordinary scalar blocking E3(i), this Core.IC/BoundaryReq discharge is the implementation evidence from which the same-interface Issue order is derived; it does not discharge the distinct-interface no-reverse or cross-interface prefix-closure checks. A commit must also satisfy Core.8/Core.11/E4; Core.17/ΣMatch additionally catches any resulting committed ordinal/value mismatch, while ungranted cycles rely on the fixed request-identification evidence itself. Evidence examples include an explicit-boundary/interface mapping, RTL/interface assertion or monitor, structural analysis, a formal refinement check, or a checked request-attribution/ordinal certificate tied to the selected boundary and dynamic per-endpoint source sequence. A protocol requiring an additional nonterminal request disposition is outside the present Core.IC model unless that operational extension and its correspondence obligations are separately defined and proved.

Core.SyncProtocol / Core.SyncFence — synchronization lifecycle and completion conformance.

Trigger: Core.SyncProtocol is required for every admitted synchronization instance, including one with no blocking predecessor. The additional Core.SyncFence predecessor obligation is vacuous only when there is no blocking q in pref_S(s).

Discharge: Establish SP1–SP6 for the actual source and implementation interfaces: prospective transaction/endpoint/epoch identity, distinction between reach and physical issue, held ready/valid offers through commit or affecting reset, sampled mandatory completion, unary scope, and exact reset/edge binding. For sync-controlled pins, prove all-reader old-value protection before the ready offer authorizes changes, throughout the pending interval and through later old-value use, plus new-value retention under the designated transaction.

Establish R3/E5/Core.SyncFence so every earlier blocking q process-facing commits no later than s and no outstanding q survives interval advance. Same-edge predecessor commits are permitted when the complete batch accounts for them before interval advance; this is not a physical sub-cycle ordering. Later external delivery of an already process-facing-completed buffered message is not a fence prerequisite. Neither a logger permutation nor a late guard may suppress an observed handshake. Bind the same identities and units in the Batch or finer carrier and in A1–A4/AB2/CO2 evidence. Evidence examples include protocol and reset assertions, a held-offer monitor, pin-isolation and sample-lifetime proofs, the complete edge inventory and an independently checked refinement of the declared sampling/event-region law. Typing, RegSeq, finite tests or a valid conditional theorem alone do not discharge these implementation obligations.

Core.EvidencePackage — identity, schema and trust boundary. Trigger: proof-backed or generated evidence discharging a constructive or completion claim. Bind the complete consumed-interface manifest, artifact revisions, semantic definitions, hypotheses, validation method and trusted base under Core.EvidenceHdr. Reuse only within the proved validity domain. A checker accepting recorded conclusions without validating their stated premises is not evidence for those conclusions.

Core.ExtensionConsumerCoverage / Core.EvidenceRealization — extension completeness and evidence mode. Trigger: an application admits an extensible interface class or uses parameterized/generated evidence. Bind each admitted typed member/family to every applicable consumer case or proved inapplicability; a quantified generic proof must cover its entire declared family. Record missing cases as qualification failures, not silent behavior exclusions. State whether evidence is a mathematical proof family, a total sound extractor on its qualified domain, or individually checked outputs from a partial generator. For an extractor claim supply soundness and domain-relative termination/totality; for checker acceptance supply its soundness and trusted-base binding. Sampling is not family coverage. Preserve the mapping under schema, semantic, feature and consumer changes.

- L.WC — witness-compatible ε choices / non-blocking outcomes.

  Trigger: Required when failed non-blocking polls or other ε-modeled choices can affect later Σ-visible control flow, frontier membership, request attribution, guard values, payload values, or other later Σ-visible choices.

  Discharge: Provide evidence that proves L.WC to the extent consumed by the selected evidence. The audit entry shall identify the provenance route by which WC was established:

  – NB-CF route: show that failed PushNB/PopNB outcomes do not affect later Σ-visible control flow, frontier membership, request attribution, guard values, payload values, or other later Σ-visible choices. Under this route, identity or arbitrary selection ranges only over legal source executions/outcomes under Core.SourceChoiceBinding and the actual decision law, with the applicable latency-sensitive local-protocol conditions. It does not authorize suppressing a required success or omitting a committed transfer.

  – Appendix L snooping route: provide a witness selector W satisfying Lemma L.2. Show that the snooped outcomes cover every ε-modeled source choice that can affect later Σ-visible behavior, that the wrapper changes only ε-choice selection, and that it does not add, remove, reorder, or relabel Σ-visible events.

  – Direct witness-construction route: otherwise construct and prove the μ_Q / WC witness consumed by the selected constructive evidence.

  – Mixed per-site route: partition the relevant non-blocking or ε-choice sites among the applicable NB-CF, snooping, or direct-construction routes, and show that the partition covers every relevant site.

NB-CF and Appendix L snooping are therefore provenance routes for proving WC, not separate theorem premises that replace WC. When useful for review or automation, the WC audit entry may include optional per-site metadata identifying each relevant call site or ε-choice site, its selected provenance route, and the corresponding source restriction, tool/RTL evidence, wrapper evidence, or witness construction.

Mixed-evidence assembly rule. Per-site NB-CF and witness-selection evidence may be assembled only within one bound instance and the selected joint construction. Declare a complete partition of relevant choice sites/occurrences, compatible WC/μ_Q bindings and fixed-choice provenance. Validate every shared-state selected outcome through the common settled snapshot/the actual source NB decision law, and retain common source-choice agreement, footprint/dependency, atomicity and global realizability checks. Independent local outcome proofs do not establish their simultaneous legality. Joint optionality is a separate set-level proof under L.JointOptionality; record its declared set, retained bindings and applicability alongside the partition. A boundary-only optionality result cannot replace wider full-Σ obligations. This assembly rule organizes existing evidence and is not an independent local-to-global correspondence theorem.

Evidence examples: Source inspection or a static control/dataflow check establishing NB-CF; a generated WC witness or certificate; a wrapper proof or monitor specification satisfying Lemma L.2; a co-simulation/snooping harness, subject to the L.Dir entry below; or a complete per-site coverage report for a mixed discharge.

L.Dir — one-directional alignment / RTL-side non-interference.

Statement: L.Dir is the scope restriction stated normatively in Condition L.Dir (Appendix L), which is its authoritative wording: the source-side snoop sequence selected by W is exactly an ordinal-preserving consumption of the free-running RTL_B snoop sequence at the chosen boundary — each pre-HLS-side snoop commit consumes the next recorded RTL_B event of the same snoop point with matching kind and payload, no pre-HLS-side snoop commit occurs without such a next recorded RTL_B event, every recorded RTL_B snoop event within the compared run is eventually consumed or reported outstanding per the Requirement W4 bound, and no RTL_B-side signal or event is gated, delayed, or modified. Benign cycle skew is permitted; every alignment/accounting and RTL non-interference field remains required, with W4 governing the classification of bound-exceeded failures. A run violating L.Dir is outside the Appendices J and K theorem scope for that comparison and carries no theorem-backed guarantee.

Trigger: Required whenever WC is discharged through an Appendix L snooping / witness-selector co-simulation harness (Lemma L.2 / Lemma L.5).

Discharge: Every evidence route shall establish all L.Dir fields for the declared compared-run/prefix domain: the bound canonical ordinal/occurrence matching with kind and payload agreement; no unmatched source-side snoop commit; complete capture/retirement and consumed-or-reported-outstanding accounting under W4; and RTL-side non-interference. The monitor route supplies a qualified wrapper satisfying W1–W5 and L.2, its checked monitor behavior, the clean result for the declared domain, and the outstanding-record disposition and watchdog configuration. A finite clean log does not prove future consumption. A static/structural or model-checking route proves the same complete contract; latency/order analysis may discharge its relevant component only when separately checked evidence supplies all remaining fields. A proof that the RTL cannot run ahead in event order, by itself, is not a complete L.Dir discharge. Each route states whether the W4 bound is proved sufficient; expiry under an unproved bound remains an inconclusive witness-realization failure, not a proved L.Dir violation or design deadlock. WC choice coverage and source legality, the selected constructive correspondence and full-fact coverage, source-property validity, and any independently

triggered K.CT premises remain separate.
Evidence examples: a qualified capture/retirement and comparison monitor with a run-bound result and complete disposition record; static latency analysis combined with checked identity/value, event-accounting and non-interference evidence; or a model-checking certificate for all L.Dir fields over its declared domain. L.Dir violations are triaged per the Snooping Concerns section of Appendix L: the pre-HLS model is the default suspect in a certified-harness, trusted-tool flow, but harness incompleteness and tool non-compliance must not be excluded a priori.

- Sys_B^elab / Q.1 — elaborated reference qualification. Trigger: the selected reference uses explicit elaborated storage, couplers, gates or compound units. Discharge every Q.SafetyQualified accounting, token/group/order, ownership, attribution, actual handshake and template-settling clause. Prove whole-system settling/matching separately where consumed. Completion additionally requires Q.CTQualified and its component progress laws. A primitive FIFO abstraction cannot omit an actual coupled enabling condition; no shared-pool or arbitrary compound locality theorem is implied.
- B-faithfulness — faithfulness of each back-annotated capacity B(c) and chosen explicit abstract channel boundary.
  Trigger: Required for every back-annotated explicit storage/handshake boundary whose B(c), occupancy, BoundaryReq, settled ChannelEnabled/ChannelDisabled is used by the selected theorem instance. For Sys_B^elab this includes the declared combinational boundary mapping that determines the complementary ready/valid value; the stateless coupler itself carries no capacity.
  Discharge: Show all of the following for the chosen abstract boundary of c:
  Boundary completeness — every Σ-visible transfer of c in RTL_B crosses the chosen abstract boundary as the corresponding committed Push_c or Pop_c, with no hidden side entrance or exit.
  Exact boundary behavior — committed transfers obey the corresponding E4 capacity/order/rendezvous legality, and the mapped RTL ready/valid/data signals at the selected BoundaryEvalPoint agree with the ChannelEnabled/ChannelDisabled behavior of the selected Sys_ref. For ordinary Sys_B this means the primitive E4 full/empty/rendezvous equivalence. For Sys_B^elab it means the same storage legality plus the E-declared settled coupler network; nonfull/nonempty status alone is not required to assert a complementary signal at a coupling-sensitive boundary. Realized stored occupancy remains bounded by the declared explicit capacities. For boundary comparisons, apply the selected boundary correspondence's Exact observation binding: mapped state, surviving requests, committed history and settled signals must refer to one consistent RTL occurrence and to the same pre- or post-transfer view as the source observation. Current post-L3 occupancy must not be paired with pre-transfer RTL ready/valid. Abstract decoding or a changed observation phase requires the independent refinement/correspondence proof specified there; it is not supplied by history/fact reconstruction itself. Core.TransferLaw requires equality of the full sampled transfer inventory in both models: mapped valid/ready overlap is sufficient as well as necessary for commitment at an eligible edge. Include every declared physical boundary, even if projected away; check reset, atomic attribution and the old-state snapshot without a post-handshake veto.
  No hidden enablement, disablement, or storage — every grant, throttle, mutual-stall, join, epoch-gate, or backpressure condition that changes the selected boundary handshake is represented in the declared complete signal network and its explicitly modeled state/roots, including prospectively fixed scheduler/arbiter resolutions; every stateful capacity/credit effect

is represented by B(c) or an explicit finite-capacity Sys_B^elab channel. No unmodeled condition may change the reconstructed settled handshake. A scheduler/arbiter resolution must affect the signals before sampling and cannot suppress an overlap afterward. An internal persistent request may be distinct from a gated physical level only under the fixed complete attribution/transfer mapping.

Internal-movement accounting. Movement below the selected boundary that changes no declared explicit state or action remains internal behavior and creates no additional observable Push/Pop. Movement that does change the declared carrier must be represented by its actual operational action and correspondence law. Its progress requirements are supplied separately by B.CT/Q.CT.

Evidence examples: A trusted back-annotation algorithm; a per-channel capacity and boundary certificate; a structural RTL check; an RTL/interface monitor; a formal refinement check comparing the realized boundary behavior with E4; or equivalent tool/RTL-flow evidence establishing the complete B-faithfulness obligation.

J.LocalOriginContracts / J.LG-Origin / J.LG-Batch-Safe — direct constructive semantic safety route. Trigger: required when semantic coverage is produced by J.LG-Origin, with Core.RegisteredMessageBatch additionally required for its exact all-finite-prefix Batch consequence. Discharge: fix I, Σ_DUT, origin family O_I and exact-prefix relation R; prove C0–C5 on every construction-reachable related origin and every actual successor, including all peer/environment choices, finite whole-network settling, coherent resource/action residues, operand/occurrence/activation binding, independent finite next preparation, and bidirectional canonical-generator translation. Bind ordinary channel qualification or Q.SafetyQualified(E) and the actual B-faithful implementation boundary. For the Batch consequence, establish BM0–BM4 against the implementation's commit/retirement predicates, clock, inventory, simultaneous update and legal cuts, with history-preserving initial/successor designation and complete-bucket monitor evaluation. Supply finite actual origin-carrier decomposition; a simulation sample or favorable scheduler is insufficient. Source-settling and K.CT consumers retain their own separately triggered evidence requirements; semantic outer-history coverage alone does not discharge them. Property discharge uses J.SafeTransfer-HistCov on all Reach_ref, or J.SafeTransfer-HistCovOn on the independently bound source domain A after its coverage proof. The selected-run clause retains RefProjCover or RefProjCoverOn respectively and the explicit all-finite-ideals property premise. Evidence examples: a component refinement proof and checked joint-resource/action and generator templates, with a bound whole-edge completion semantics for the implementation. No future edge or fair continuation is supplied by Batch. Canonical discharge additionally binds J.RequiredOrderWF and J.SourceOrderReflection: source strictness is established before the complete-group bijection and nonempty path translation derive retained RTL acyclicity and equality. Any consumed unmapped proof-unit support retains its own independent order evidence. J.CycleRankOrderWF and the checked J.StandardCycleRank instance provide optional reusable joint-acyclicity evidence; they do not replace the source-realization, complete-group/path-translation or prefix obligations. For stimulus-class use record UCL0–UCL3 under J.UniformConstructiveEvidence and the separate source-property discharge of J.ConstructiveClassSafe; bind the combined DUT/T evidence or exact external-environment step relation and all-prefix carrier uniformly. Sampled local checks do not discharge this family quantification. Explicit owners added by this revision: C0/C4 supply Core.PreparationEvidence for the prospectively fixed source relation and Core.SourceChoiceBinding; C2 supplies J.SourceSettleRealization/SourceSettleExists including the entire frontier, matching dynamic NB attempts and their complete snapshots, and equality of actual outcomes under Core.NBDecision/Core.NBDecisionSnapshot with Core.RegSeq continuation; carrier qualification supplies Core.ConstructiveOriginCarrier and R transport; C5 supplies J.GeneratorCompleteness and reusable SourceGeneratorRealization/SourceOrderWF or equivalent full-graph evidence. Missing mandatory generator evidence is a qualification failure. The optional

checkpoint owner supplies Core.SourceCheckpointProfile/SourceCheckpointRepresentative and binds the exact A/H/Φ and selected SafeTransfer-HistCovOn clause. None of these is discharged by package acceptance alone. C5 includes the mandatory E5/Core.SyncFenceOrder generator and its source/RTL realization on the covered carrier. For ordinary OM/OA, use the J.OrdinaryConstructiveComposition owner below instead of supplying an unexplained global invariant: it derives the assembled relation and preservation from its named local and integration evidence. This specialization does not cover every elaborated/extension profile. When an internal Pop uses admission-derived logical payloads, the Core.LogicalPopPayload / Q.CT.LogicalPayload entry in this registry is also triggered for safety-only use. Discharge its observer, representation and legal-later-use obligations with J.LogicalPayloadRefinement; completion service is not a premise of that safety discharge.

J.OrdinaryConstructiveComposition — ordinary locality qualification. Trigger: a claim that OM/OA correspondence needs no separate design-specific global composition proof. Discharge: bind O1–O7 of J.OrdinaryConstructiveProfile prospectively; provide LP0–LP5 of J.OrdinaryLocalEvidence and the incident primitive refinements over the restricted vocabulary; check G1–G7 of J.OrdinaryGlue; supply QF or QSafe and QM of J.OrdinarySettlingEvidence; bind the actual BM/AB completion inventory and identity origins (or separately qualified designation transport); and qualify the internal/external environment under O7. Use the generic residue, preparation and source-generator lemmas to derive R preservation and coverage. Source-property validity remains separate, including all admissible unresolved choices; complete-bucket monitor evaluation/extensionality is required when a monitor supplies property evidence. For checkpoint use additionally prove SCP1–SCP3. Local assumptions about unexposed DUT/peer state or application-wide correlations are not admitted through LP or r_env. The stronger/general profiles keep their existing owners and are not silently classified as ordinary when this theorem fails.

J.ElaboratedConstructiveComposition — elaborated locality qualification. Trigger: a claim that an ELM/ELA design needs no separate design-specific global correspondence invariant. Discharge: bind EL1–EL8 of J.ElaboratedLocalityProfile prospectively over the declared class; provide J.ElaboratedTemplateEvidence, J.ElaboratedNBDecisionBinding for admitted NB cases, and LP0–LP5 over the explicitly enlarged completion-packet vocabulary. Retain the ordinary ownership/integration facts adapted to the template legs, including the applicable G1–G7 obligations, QF or QSafe with QM for complete operational settling/matching, and O6/O7 base/environment/carrier bindings. Supply Q.SafetyQualified(E), including qualified accounting/projection initialization, and the actual message or anchor Batch completion inventory and designation transport. The local boundary, preparation and canonical-extension lemmas derive the assembled relation, equal-history bases, preservation and coverage; those results are not an additional supplied global correspondence premise. Record the existing M.7a registry reuse keys for the bound models, class, primitives, E, observations and carrier; reuse is valid only under the recorded qualifications. Source-property evidence remains separate, with its selected source domain, unresolved choices and monitor obligations. Checkpoint use additionally requires the existing uniformly proved Core.SourceCheckpointProfile. This row does not qualify arbitrary elaborations or enlarge EL1–EL8.

Constructive source-property owner. Trigger: a safety-property transfer consumes source-property validity on the selected source domain; correspondence alone does not trigger this property proof. Record the chosen universal proof, exhaustive model exploration, or selected-execution method from the normative source-property discharge section. Identify the actual configured executable/formal model and its history-coverage theorem over A. For each selected profile bind the named preparation, anchor/configuration, reset and R2C semantics; no generic source simulator is presumed adequate. Checkpoint monitors additionally bind SCP and equal-history representatives. Selected-run evidence records RefProjCoverOn plus clause (2)’s all-finite-ideals proof. Validity on A-realizable histories is clause

(1)'s universal-domain evidence, not a distinct covering-run discharge. This owner is independent of transformation correctness and remains universally quantified for class claims.
E5 source covering-run owner. Trigger: the selected discharge uses J.RefProjCover, J.RefProjCoverOn or the corresponding selected-source safety-transfer clause. J.RefProjCover, J.RefProjCoverOn, both selected-source safety-transfer clauses and the J.SafeTransfer-HistCov common-case discharge note consume the current fence semantics. Establish source occurrence embeddings, applicable fence incidence, common-history induced order and predecessor closure against the selected covering run, including erased paths. Supply a direct ideal-inclusion proof or source-typed reusable evidence for the exact source domain. Bind these proofs to the consumed E5/Core.SyncFenceOrder semantic revision or definition-level content hash under Core.EvidenceHdr and M.7a's reuse-key discipline, including the relevant projection and incidence definitions.
Constructive scope and integration owner. Trigger: the selected constructive profile relies on the clock, settling, synchronization, reset or configuration scope and integration conditions stated below. Record one clock and an acyclic complete settling graph; only unary waits and paired sync transactions are covered directly. Reset is edge-sampled, with asynchronous assertion requiring an independently proved reduction/conformance result. Partial reset qualifies only with an explicit protocol whose states, ordinary observable actions, cancellation/storage effects, endpoint/epoch matching and completion guard establish RW1–RW5; the framework does not supply a universal flush protocol. Configuration readers outside paired participants need the existing governing-transaction/ownership protocol and update protection. These scope facts are not supplied by the word synchronous.
J.StableDirectInputContracts / J.StableRead-Batch — stable direct-input lifting for message safety. Trigger: required when the registered message result is applied to a source body containing repeated or conditional direct-input reads. Discharge: SD0–SD4, including real matched initialization anchors, environment stability covering anchors and every permitted use, complete physical-sample binding, in-place source reduction/lifting preserving source launch legality, uniform C0–C5 and Batch evidence for the reduced body, and the whole-unit message-projection/order proof. No source initialization copy is required; do not replace mutable local reads or manufacture conditional startup reads. Bind each configuration and any E_θ to the same instance and retain all admitted peer/environment choices. Property evidence is J.SafeTransfer-HistCov's universal-original-source or selected-run/all-ideals clause. A value-stability argument alone does not supply body correctness, boundary faithfulness or signal-prefix/certificate conformance. Evidence examples: an anchored-input/environment contract plus a substitution and generator-erasure lemma, composed with the existing parameterized process and resource proofs.
J.LocalAnchorContracts / J.LG-AnchorBatch — observable wait, paired-sync and signal safety. Trigger: the direct local-to-global route retains synchronization and anchored signal groups. Discharge A0–A6 with Core.RegisteredAnchorBatch, the chosen Sys_ref and ordinary qualification or Q.SafetyQualified(E). Evidence includes RULE 1 retained membership/value determination; actual source last-write records; per-instance signal sampling/resolution and owned joint updates; typed wait/pair readiness and completion fences; next-interval capacity and Issue discipline; enlarged generator translation and complete-edge carrier binding. J.AnchorInventory derives the read-membership closure from source qualification; implementation realization still needs evidence. Use J.AnchorDirectInputLift for stable/sync-controlled late samples while retaining complete anchor groups, and do not reuse the message-only erasure result as a partial-group projection. Example J.AnchorBatchPair supplies a symbolic paper discharge, not implementation certification.
Core.AnchorConfigObservation / J.LG-ConfigBatch — configuration-transaction observation for the constructive route. Trigger: the selected observation or correspondence uses the configuration-record profile. Bind fixed per-signal designation, per-reader real initial anchors with common reset-stable values, or one record per committed update transaction; prove CVG0–CVG3 for every actual use/sample

and configuration choice. Ordinary observable signals keep RULE 1. Qualify full complete groups and their existing generator paths under the selected observation; this does not establish correspondence for a different retained-Read observation alphabet. Message-only transfer additionally requires J.ConfigWholeGroupProjection, including E5-path contraction and no partially erased mixed group. This evidence does not discharge another observation profile's request, snapshot, local source-realization or complete fact-inventory obligations. Admitted NB cases still require Core.NBDecision, complete outcome matching and the applicable local constructive evidence. Completion-facing correspondence uses J.ConfigRecordFactCorr with full CF/FB evidence and K.CT.Agreement; stable message-body substitution instead uses J.StableReadFactCorr and its distinct scope.
Core.RegisteredResetBatch / J.LG-ResetBatch — edge-sampled dynamic reset on the constructive route. Trigger: the selected constructive carrier or result uses the dynamic-reset extension. Bind ResetScopes, ResetPriority, ResetOccurrenceConvention, ResetHold, ResetReleaseInit and ResetObservation; discharge RB0–RB5 and the applicable non-reset local cases. The source model used for property verification must implement the same RW1–RW5 effects on channels, peers, control and configuration, not only restart a process thread. Bind held-reset occurrence count, canonical ResetKeys, fresh START, initialization prefixes and affected/unaffected samples. The reset-priority closed pipeline island is a symbolic sufficient instance; partial scopes or different same-edge policies need their own declared effects/proofs. No K interruption or asynchronous-reset qualification is supplied.
Core.RegisteredR2CBatch / J.LG-R2CBatch — constructive edge-observed R2C. Trigger: the selected constructive carrier or result includes the edge-observed R2C extension. Discharge RC0–RC6 on the same relation as the applicable C/A/CVG/RB contracts: whole-network settling, multi-input observation qualification, exact emission, legal source observation, consumers/configuration isolation, reset ownership/ledger and canonical translation. Bind edge sampling and complete-inventory monitoring. Use J.ConstructiveCombinedProfile and its matrix to identify all applicable cases. R2C adds combinational signal glue/observations, not process-owned combinational message passing. This is a conditional paper theorem; operational settling, carrier conformance and joint local evidence remain to be discharged for the instance. Mechanization and executable-flow qualification are not asserted.

- Core.MemSyncOrder — array/synchronization commit-order conformance.
  Trigger: Required whenever the selected design/flow instance contains array or memory accesses in a process that are source-ordered with synchronization calls selected into S, whether the storage is process-internal or externally mapped.
  Discharge: Check the main-text "Scheduling of Array Accesses" commit-time fence at the schedule/mapping-layer level: mapped storage commits for accesses before s occur no later than s's commit, and mapped storage commits for accesses after s occur strictly later. Any allowed cache/merge/split/reorder transformation shall carry mapping-layer evidence that preserves the fence. A single merged storage commit may not represent accesses lying on opposite sides of the same synchronization boundary; such a merge is forbidden. This is a scheduling-contract qualification check recorded by M.7a, not a constructive correspondence semantic premise, and it does not add memory events to Σ. For a constructive correspondence application with cross-process shared-memory value flow, additionally discharge J.Mem; when transformed physical memory activity is hidden below the selected proof boundary, additionally discharge Core.MemFaithful.
  Evidence examples: HLS schedule/control-step reports; memory mapping metadata; generated-RTL memory-enable/address/commit assertions; or a mapping-layer proof/checker.
- Core.MemFaithful — mapped-memory boundary faithfulness. Trigger: the comparison relies on a logical boundary hiding transformed physical memory activity, as required by the applicable memory clauses. Bind the untransformed logical semantics, selected channel/signal boundary and permitted transformations. Prove preservation of every selected value, occurrence,

endpoint order and required atomic/fence effect under Core.17/E1–E5, without lost or duplicate events. For completion, Core.MemProfile MP5 additionally accounts for every hidden backpressure/resource/service condition. Neither property is derived from merely selecting an observation boundary.

- J.Mem — shared-memory lowering.
  Trigger: Required whenever the compared design contains a cross-process shared-memory dependency through which one process's memory write can affect another process's control flow, payload/value expressions, or the identity or order of its later operations.
  Discharge: Show every such dependency is lowered by the array-access mapping layer onto covered proof-visible operations (explicit point-to-point E4 channels, including a qualified logical memory boundary, and/or Σ-visible anchored signal IO), or supply the memory-state invariant extension described in the informative note in Normative Formal Core. The audit is dependency-based rather than per physical RAM access, but it shall be complete: show that all dependencies satisfying the trigger are represented and that no other unlowered cross-process memory-carried path satisfying the trigger exists. If a transaction-changing transformation would alter the event stream otherwise used for this lowering, additionally use a Core.MemProfile boundary satisfying Core.MemFaithful or the separate memory-state extension.
  Evidence examples: use of a qualified shared-memory library/profile presenting the logical boundary; mapping-layer configuration review; data/control-flow analysis establishing the trigger set; and a structural completeness check that no unlowered cross-process array dependency remains.
- Core.MemProfile — qualified memory mapping profile (when used).
  Trigger: The selected safety or completion application relies on a reusable logical memory boundary above transaction-changing cache/merge/split/reorder behavior.
  Discharge: Bind MP1–MP4, MP6 and MP7, including the untransformed logical reference and complete transformation-invariant value/accounting boundary, Core.MemFaithful, J.Mem, Core.MemSyncOrder, MP6 ownership/arbitration, the invoked Q qualification and RW1–RW5. Select the actual MP5 disposition: finite-history safety under Core.MemProfile's safety clauses; completion with all hidden response/resource/service dependencies represented in K.CT.Exposure and its standard or extension proof. Preserve all other premises of every invoked proof. Bind identity, semantic version and validity through Core.EvidenceHdr/Core.EvidencePackage. Failure of one disposition makes that conclusion unavailable, not every unrelated conclusion.
  Evidence: a proved library profile, checked logical-boundary mapping, ownership/reset and applicable progress proof, and exact instance binding. A profile name or successful example does not qualify a concrete implementation.
- RW — reset well-formedness (RW1–RW5) and reset-epoch advancement.
  Trigger: Required whenever dynamic reset can occur in the compared executions; Core.ResetEpochAdvance is additionally required whenever the selected theorem instance carries reset-epoch coordinates in its constructive carrier or other retained state interface.
  Trivial no-dynamic-reset disposition: if the selected compared executions permit no dynamic RESET after the prescribed initialization/reset-established base, bind that fact once. The retained epoch-typing interface then uses the prescribed initial valuation throughout; no dynamic-reset RW1–RW5 occurrence proof is generated.
  Discharge: Show every dynamic reset occurrence satisfies RW1–RW5 and discharge the selected constructive reset extension at each RESET boundary unit. Bind the Core.17 canonical RESET occurrence convention to Core.ResetEpochKey: the logical key is based on the matched reset scope and local occurrence ordinal rather than a model-local simulator identity. For each

Core.CycleResult.resetUnits family, prove the Core.ResetEpochAdvance conditions and Core.ResetEpochAdvance-Fun: overlapping RESETs affecting the same coordinate have the required reset-boundary order; RW5-disjoint incomparable RESETs update disjoint coordinates and commute; unaffected coordinates retain their prior key. The exact resulting valuation must be the one used by every retained carrier and consumer comparing bound reset-epoch coordinates across models. Under the no-dynamic-reset disposition, every post-base cycle has resetUnits=∅, so Core.ResetEpochAdvance_I(e,∅,e) is the identity and the fixed initial epoch valuation is threaded mechanically. A checker need not enumerate reset keys, reset scopes, or recompute a changing epoch vector for that case; it need only verify the no-dynamic-reset fact, the prescribed initial epoch binding, and the identity threading required by the consumers. This is a trivial discharge of the retained epoch coordinates, not an epoch-free alternate carrier definition.

Evidence examples: reset-scope and protocol proofs, checks of request cancellation and channel reinitialization, actual reset-unit/epoch advancement checks, and—where used—a checked exclusion of dynamic reset after the initial base.

K.CT.LazySerial — independent-staging lazy completion route. Trigger: the completion certificate selects one of the lazy-family applications identified in the informative route table at the head of K.CT.LazySerial. Common evidence: bind the exact reference/implementation instance, selected channels, abstract versus physical staging boundaries, F_c, C_c and g_c, empty-start exclusive ordered storage, the independent component law and applicable representation qualification. Record LazyEligibility, the consumers' serial or qualified group-serial discipline, whole-batch eligibility, absence of attributed capture slots for plain-FIFO batch members, arrival-independent request qualification and the ungated selected-path/Push condition. Verify the per-item LazySerialWitness restriction on the entire selected trace, including extensions, and record every mandatory transfer and each Push's LazyInventory/LazySlack classification. Selection after witness generation is permitted only if the recorded trace actually satisfies the restriction; the selected channels and certificate are independent of the implementation execution being proved. Other component kinds retain their own qualified proofs. A simulator pass or capacity sum is not conformance evidence.

Restricted-route evidence. For LazySerialRestricted, record the global serial/group-serial witness discipline, complete LazyClosure/LazyDemandCertificate or LazyGroupCertificate construction, and the applicable LazyRank/LazyMixedRank hypotheses and ranks. For LazyExtendedStandard, record its restricted all-singleton construction, original LazyRank evidence, protected extension and resulting standard closure. These global inventory/rank requirements are triggered by those restricted results, not by selected-channel eligibility alone.

Mixed-composition evidence. For LazyComposition, record the local consumer discipline on selected channels and the separately qualified witness modes of other owners. Bind all selected and unselected units to one complete support graph, retain every cross-component dependency and other resource conjunct, and supply a proof or sound acyclicity check of the assembled completed-unit graph after auxiliary-demand elimination. Do not require restricted global LazyClosure or a restricted rank theorem merely because local lazy channels are selected. Graph completeness, actual W membership and exposure validity remain separate checks.

Simulator and extension evidence. For K.CT.LazySimulation, bind the source-side process-facing demand interface independently of item arrival, issue/phase legality, and the movement selector's old-state, simultaneous-update and conflict behavior. Distinguish early ingress-to-staging movement from excluded at-site split capture. Record separately any independently selected snooping/co-simulation dependency; lazy movement itself introduces no per-stimulus RTL probe. When a protected singleton extension is used, record the local occurrence-specific premises, whole-design continuation window,

actual added completions and Facts/Agreement coverage detailed in the following demand-and-extension entry. A mixed extension does not inherit the restricted global rank result.
K.CT.LazySerial demand and extension evidence. For each resource replacement record the exact target Push, matched demand/member and admitted item, all lower-ordinal and owner/older-work support, prospectively fixed CtrlSupport and its fact provenance, and the applicable DemandPlacement or resource-independent BatchPlacement proof. Record the expanded graph, auxiliary-node elimination and common acyclic completed-unit graph; no auxiliary node is a W completion. For each singleton extension record the fixed pending set, legal continuation and actual retirements, continued clocks/reference guard progress, and absence of affecting reset/cancellation/testbench termination until that set completes. Group-member demands use member-demand support, not the singleton extension theorem. Record Facts coverage for all consumed historical and added completion facts over every admitted implementation prefix and Agreement for the witness actually used, including any extension. Bind the corresponding J adapter evidence and existing component/service premises. Evidence and checker reuse keys include topology, issue/group discipline, source movement law, control inventory, instance continuation contract and fact signature; revalidate affected entries on change. Identify the acyclicity discharge: LazyRank/LazyMixedRank for their restricted domains, or a proof/sound graph check for LazyComposition. Bind the complete cross-component dependency inventory, component-to-unit exposure mapping, additional resource conjuncts and whole-unit atomic membership separately from the acyclicity result. Record source extension protection over the whole design. Reuse keys additionally include selected channels, consumer issue/placement and batch profiles, gating and the chosen graph-check evidence; a successful topological sort alone supplies no missing semantic qualification. For every Push on a selected channel record its LazySlack classification. Ordinary cases retain the standard floor release, with actual completed predecessors when k>=g_c and the initial inventory bound when k<g_c. Extra-slack cases record DemandPushResource support or the protected-extension singleton completion used by the standard proof. Record whether the claim invokes the general LazyComposition theorem with local consumer issue qualifications or the restricted LazySerialRestricted corollary with global serial/group-serial and rank hypotheses; keep LazyExtendedStandard's direct Standard route distinct. For each selected source-extension demand record its issued/unretired status, matching admitted item, retired lower-ordinal Pops and completed preparation support. In a mixed application establish these local facts directly rather than claiming restricted global LazyClosure from channel eligibility.
The ledger records the hypotheses actually consumed by the chosen constructive safety or completion result. It creates no extra semantic assumption and no automatic discharge. Preserve source legality, all triggered interface contracts and the exact theorem domain when composing or reusing evidence.

Engineering intuition	**Formal question that remains**	**Current owner**
"The back-annotated/elaborated source model matches RTL buffering."	Is the selected boundary/model actually complete and faithful, including hidden storage, credits, coupling, and projection/order effects?	E4; applicable Q.SafetyQualified/Q.1 and Q.CTQualified; B-faithfulness. Ordinary selection requires proof that its actual boundary satisfies the ordinary profile.
"Snooping/NB-CF makes hidden choices agree."	Were all witness-significant ε choices covered, were outcomes causally admissible, and is the resulting source behavior legal?	L.WC; NB-CF; Appendix L; qualified constructive choice/decision evidence.
"The source and RTL runs match side by side."	Does a legal reachable Sys_ref witness with complete provenance, settling, annotations, prerequisites,	J.LocalOriginContracts / J.LocalAnchorContracts and the

Engineering intuition	Formal question that remains	Current owner
	and required-order support actually exist?	selected constructive composition theorem.
“One matched pair is safe.”	Why does Φ hold for every reachable RTL prefix under the fixed theorem instance?	Constructive coverage of the declared RTL-prefix domain; J.SafeTransfer-HistCov or J.SafeTransfer-HistCovOn and independent source-property evidence. One matched pair alone is insufficient.
“The document is enormous, so safety must need K too.”	Which progress and service obligations are additional to finite-prefix safety?	The selected constructive theorem lists its exact operational and local-evidence premises; completion adds its separately declared service and exposure laws.
“M.7a is long, therefore the mathematics is long.”	How much is evidence ownership, reusable qualification, provenance, and rebinding rather than new semantic argument?	M.7a discharge ledger and the named constructive/completion interfaces.
“The obvious result should be one sentence.”	Which qualifications were added because shorter formulations were ambiguous or unsound under adversarial review?	The audit history records prior changes; normative owners include Core, B, G, J, K, L, N, Q and M.7a.

Appendix N – Scheduler and Environment Fairness Assumptions

This appendix states pre-grant arbitration service and conditional channel-progress premises used by K.CT where invoked. Core.TransferLaw and Core.SyncProtocol make eligible sampled handshakes mandatory. B2 concerns the independently defined pre-grant eligibility of a required arbitration action; B3 concerns primitive channel discharge under its stated support and continuing-edge conditions. Core.IssueFair governs source issue, while B.OF.Service/B.CT.Service govern preparation, movement, capture and restoration where separately needed. No one service law silently supplies another. Finite-prefix safety requires its operational contracts, not fairness or B1 by default; B1 remains a separate trace-determinism property.

- **B2 (Weak fairness of pre-grant arbitration).** Use the Core B2 definition: define ArbEligible from actual pending request, legal owner/phase and peer/resource/guard support independently of the arbiter’s own grant. A continuously eligible action receives its bound grant/service after finitely many applicable opportunities. Bind the complete joint unit and prove the grant’s effect. A request that is only recurrently eligible needs a separate recurrent-opportunity or finite-service contract if eventual service is claimed. Actual settled ready/valid overlap commits immediately by Core.TransferLaw; synchronization uses Core.SyncProtocol. Neither has a post-handshake B2 selector.

Core.IssueFair (source issue; separate from B2). A reached operation that remains legally issue-enabled at every applicable L1 opportunity must eventually issue on an execution satisfying this premise. The legality predicate includes Core.16 and bound source choices. K.CT source eventual-issue arguments

consume this premise explicitly; a finite workload witness alone does not. Boundary commitment, internal movement and joint service remain separate requirements.

- B3 Channel Progress Invariants (ready/valid form). For a primitive channel element c, blocking Push/Pop endpoint requests are non-withdrawable during the outstanding attempt within one reset epoch. If a primitive buffered Push is blocked because the represented storage is full and the complementary Pop request persists, some Pop eventually commits and discharges the full condition. If a primitive buffered Pop is blocked because the represented storage is empty and the complementary Push request persists, some Push eventually commits and discharges the empty condition. For a primitive B(c)=0 rendezvous, persistent matching endpoint requests eventually complete the atomic rendezvous. Push payload remains stable while its request is outstanding; an affecting RESET ends the dynamic instance under RW2.

  For Sys_B^elab, these primitive obligations are composed through the declared finite stateless ready/valid network. A boundary may remain ChannelDisabled while its local storage is nonfull/nonempty because another explicit handshake cause is missing. Q.1 requires that every such cause be exposed through explicit stored channel state or modeled process-facing request roots and that the next settled valuation respond deterministically when those roots change. Therefore B3 does not assert that every coupler-disabled boundary eventually enables; a closed wait cycle may persist without violating that conditional primitive obligation. K.CT instead proves acceptance of the required workload when its separate alignment, closure, exposure and service premises hold. B2 applies only at a separately declared pre-grant arbitration interface with its ArbEligible antecedent proved. Actual ChannelEnabled at an eligible settled edge commits by Core.TransferLaw, without additional selection. For an uncoupled primitive these eventual discharge statements follow at the eligible edge from the persistent complementary request and old-state E4; continuing clocks and absence of an affecting reset are explicit scope conditions. They do not supply peer requests or coupled support.
  Clarification of B3's conclusion. A complementary Pop discharges full storage and a complementary Push discharges empty storage; that is not the original endpoint's same-edge commit. At an ordinary uncoupled FIFO, its still-pending original request commits on the next eligible settled edge, assuming that edge occurs without an affecting reset. This follows from Core.TransferLaw, not B2. In Sys_B^elab, other declared coupled inputs may still withhold the handshake after occupancy changes; prove their Q-level support and any pre-grant service separately. An uncoupled B(c)=0 pair's discharge is the atomic rendezvous itself.

These B-rules are theorem-applicability assumptions, not guarantees automatically provided by the HLS tool, and their discharge is not exclusively an environment responsibility. Depending on the selected theorem instance, they may constrain source/scheduler semantics, HLS-generated or hand-written RTL arbitration and back-pressure, channel-library behavior, or external masters. In particular, arbitration introduced by HLS for resource sharing, memory-port scheduling, channel selection, or similar implementation functions must satisfy B2 when the selected claim invokes its independently bound pre-grant ArbEligible service. Grants affect the complete settled signals; they cannot select among already sampled transfers. The corresponding implementation-side evidence is recorded under the Appendix M.7a B2/B3 discharge; reusable tool or library qualification may supply that evidence when applicable.

Priority arbiter example: fair vs unfair priority

Consider a simple two-input priority arbiter inside the post-HLS RTL. Each input is driven by a message-passing channel; the arbiter issues Pop operations to select which request to serve in each cycle.

- Input 0: high-priority channel c_high
- Input 1: low-priority channel c_low

Both channels may have pending requests at the same time.

1. **Fair priority arbiter (B2 under its eligibility conditions).**

A fair implementation may prefer c_high at some decisions, but a completion claim for c_low must prove that its independently defined ArbEligible support persists and that the arbiter eventually grants it. Possible implementations, subject to actual resource/peer and clock support, include:

- A **round-robin or rotating-priority** arbiter that periodically moves the "highest" priority to the next requester, or
- A bounded-burst priority scheme in which c_high may win at most N consecutive cycles while c_low is requesting; after that, the next grant is forced to c_low. Qualification additionally proves persistent ArbEligible resource/peer support; request presence alone is insufficient.

Under these implementations:

- If Pop_low remains ArbEligible before the arbiter's own grant, B2 requires an eventual grant with its bound effect. The grant participates in the settled signal equations. When that creates the actual ready/valid overlap, Pop_low commits on that edge by Core.TransferLaw; there is no second selection.
- A full low-priority FIFO and a pending service request do not alone establish arbitration progress. Prove persistent pre-grant eligibility and eventual grant where B2 is invoked, or the named alternative service contract. Primitive P_push/P_pop apply only with their actual persistent complementary requests and primitive scope. A withheld arbiter grant is a declared coupled cause, not a failure of the uncoupled FIFO transfer law.

The arbiter may retain priority preferences while ensuring eventual grant for every continuously ArbEligible requester in its service scope. Its actual grant affects the settled handshake, which then commits mandatorily. Channel discharge still requires the relevant primitive or coupled support; arbitration fairness alone does not prove B3 applicability.

2. **Unfair priority arbiter (B2 failure under persistent eligibility).**

A more naive design might implement:

```
if (req_high) grant_high();
else if (req_low) grant_low();
```

combined with an environment in which req_high can remain asserted indefinitely. In this case, even if req_low is also continuously asserted:

- Pop_low may remain continuously ArbEligible while a fixed-priority arbiter never grants it. Its actual settled ChannelEnabled can remain false because the declared grant is absent. This is pre-grant starvation, not a legal skipped ready/valid handshake.
- Low-priority requests can starve indefinitely, even though they are continuously eligible for selection.

This violates B2 if the pre-grant ArbEligible antecedent holds throughout the admitted opportunities. If peer/resource support is only intermittent, that weak-fairness antecedent has not been shown; any stronger promised service must be checked separately. Do not infer a primitive B3 violation merely from a full FIFO behind an ungranted coupled interface.

In such a design, the safety properties E1–E5 may still hold (trace-compatibility on actions that do occur), but the required-workload completion results that consume the violated B2 service premise no longer apply. This does not by itself demonstrate a primitive B3 violation.

Intentional priority does not waive the required-workload premises. Persistent pre-grant eligibility with permanent denial violates B2 when B2 is invoked, even if source and RTL share the policy. A reached

source operation that is continuously IssueEnabled but never issued violates Core.IssueFair instead; source control that never reaches it is a distinct control-starvation case. Intermittent issue or arbitration eligibility is outside those weak-fairness antecedents unless a separately proved service contract covers it. An actual eligible sampled handshake must commit regardless of fairness: omitting it violates Core.TransferLaw directly. These distinctions preserve the conditional scope of K.CT.

Patterns that typically satisfy the fairness assumptions

The following patterns can support B2/B3 and the separately declared service contracts when their actual enabling, arbitration and environment conditions are proved:

- **Round-robin or rotating-priority arbiters.** Any requester that remains enabled will eventually be at the front of the rotation and will be granted.
- **Age-based or credit-based arbiters.** Requesters accumulate age or credits while waiting; the arbiter prefers older or more-starved requests, guaranteeing that long-pending actions eventually win.
- **Bounded-burst fixed priority.** Fixed priority is combined with a counter that limits how long a higher-priority requester can dominate while lower-priority requesters are pending. After the burst limit is reached, lower-priority requests are forced to win until the system is "caught up."
- **Back-pressure with bounded stall (additional service contract). A design or environment may separately guarantee bounded or eventual service under explicitly stated request, availability, clock/reset and scheduling conditions. Such a contract can support a progress discharge, but is stronger than the definition of B3 and is not implied by a lone asserted ready/valid endpoint. For example, subject to those conditions:**
    - Downstream consumers are required (by specification) to service queues at least once every N cycles when data is present.
    - External bus masters are configured such that they cannot indefinitely defer reading from full DUT FIFOs that are logically part of the interface contract.

These examples need evidence on the actual design: prove pre-grant ArbEligible and service where B2 is invoked, primitive complementary-request/clock support where B3 is used, and the separate internal movement/restoration laws. Actual eligible sampled transfers commit by the transfer laws. Clocks and required environment service continue until the workload completes; no global drain-to-empty or permanent-quiescence condition is imposed.

Patterns that tend to violate the fairness assumptions

The following patterns can defeat required-workload completion when they violate an applicable boundary or internal service contract:

- **Pure fixed-priority arbitration with unbounded high-priority traffic.** A static priority tree with no rotation or aging, combined with workloads in which a high-priority master can keep its request asserted indefinitely, can starve lower-priority channels forever.
- **"Best-effort" external masters with no progress guarantee.** For example:
    - A software driver that reads from a DUT output FIFO only when it happens to poll, and that may be pre-empted indefinitely by higher-priority threads. If it never asserts the complementary request, that absence alone is not a violation of the primitive B3 discharge condition. Any required eventual polling/service must come from a separately declared contract.
    - An external bus or DMA engine may violate a declared service contract by withholding grants despite its proved eligibility antecedent. B2 applies only to that bound pre-grant

interface, and primitive P_push/P_pop only under their stated complementary-request and continuing-edge conditions. A DUT request alone or a full/empty FIFO alone establishes neither violation. Ignoring an actual eligible sampled handshake violates Core.TransferLaw directly, independent of fairness.

- Circular back-pressure loops with no escape. A closed wait cycle alone need not violate B3. K.CT does not provide a general closed-wait-cycle exclusion theorem; its conclusion is acceptance of the required workload under its stated premises. B3 is violated only if a stated progress antecedent remains continuously true but its required discharge never occurs.

A violation of an applicable B2/B3 or internal-service premise makes the corresponding required-workload completion conclusion unavailable. It does not by itself invalidate independently proved constructive safety with its own complete operational and observation premises. Designers and verification engineers should therefore either:

- Establish the actual boundary and internal service contracts for the claimed workload, or
- Treat required-workload completion as unproved for the affected claim, while retaining only independently discharged safety results.

Appendix O – Support for Multiple Clock Domains (Informative Sketch)

This appendix is an informative sketch of how the single-clock formalism of Appendices G–K could be organized in a multi-clock design. The formal results of Appendices G–K are stated for a single global clock unless an explicit multi-clock semantic layer is added.

Within a single clock domain d, one may treat that domain as a synchronous island with a local cycle counter clk_d and apply the existing R- and E-rules to processes wholly inside that domain, interpreting clk(·) as clk_d(·) for local synchronization, signal I/O, and message-passing events. Inter-domain communication should be represented by explicit clock-domain-crossing components, such as CDC FIFOs, whose source-side and destination-side commit semantics are specified at their respective clock-domain boundaries.

A full formal lift requires additional definitions not supplied by Appendices G–K themselves: a global event order or partial order relating local-domain cycles, precise CDC commit semantics, occupancy sampling rules across source and destination clocks, synchronizer fairness/metastability abstraction assumptions, and progress obligations for CDC components. Once those additional definitions are supplied, the intended proof strategy is to treat each synchronous island using the existing single-domain rules and to treat each CDC component as an explicit channel/refinement boundary with its own stated E4-style legality and progress obligations. Without those additional definitions, Appendix O should not be read as claiming that the single-clock proofs apply unchanged to arbitrary multi-clock systems.

Appendix P – AI Discussion of Alternative Formal Frameworks

These external discussions and links are informative and supply no normative theorem or implementation qualification. The following links provide AI discussions comparing alternative formal frameworks with the one presented in this document:

https://drive.google.com/file/d/1ccU-eRW7H2ggh-AZyKnCOzO-KLPejob_/view?usp=sharing

https://chatgpt.com/share/69457119-ec30-8006-8684-9a2a8b19f96f

https://drive.google.com/file/d/1R3-46DtZLsBn1hdTbLEXyC0FVB50Gymz/view?usp=sharing

Appendix Q – Multiple Input Pipelines Back-Annotation

Core.1 defines the source-reference model ladder; Appendix B explains back-annotation of pipelined loops, including the single-input case.
In more complex cases such as HW pipelines with multiple message-passing inputs, the back-annotation mechanism may elaborate the Sys_B (source-level) simulation by introducing additional internal realization structure. In this document, we adopt the following simplified modeling discipline:

1. Storage/accounting. Every stateful buffering, skid/elastic stage, slot, or credit that can hold or backpressure a message is accounted for in explicit finite-capacity channel state with B(·), occ_·(t) and any required location/credit fields. A qualified channel realization may contain several internal register locations with its declared simultaneous update law; this does not silently make each register a separate E4 primitive. If the elaboration does introduce separate primitive channels, each retains its own E4 old-state acceptance law. Neither representation may hide capacity, credit or handshake state.
2. Handshake logic. Every non-storage join, mutual-stall, bundle, gate, or glue function is represented as stateless deterministic combinational ready/valid/data logic. It may connect internal elaborated boundaries or sit directly at a process-facing outer boundary.
3. Settled boundary meaning. BoundaryReq remains the source-operation-attributed own-side request. ChannelEnabled/ChannelDisabled is the resulting settled handshake at that boundary after the complete declared combinational network has evaluated. E4 governs storage/rendezvous legality and committed-transfer accounting; at a coupling-sensitive boundary the complementary ready/valid value may be stricter than the local not-full/not-empty condition.

Observation-boundary convention. Π_elab/π_Σ,E supplies the complete outer Sys_B channel history, retaining both endpoints of every accounted outer channel. The later restriction ρ_DUT selects the outer Σ_DUT history, subject to the existing atomic-group requirements. Accounting uses the complete history, not that restricted observation. Proof-internal storage channels may be projected away from Σ_DUT while still contributing explicit channel state to Appendix K. Stateless coupler nodes are not channels, do not contribute capacity. Core.TransferLaw makes each declared physical ready/valid transfer mandatory on its eligible sampled overlap. An abstract internal movement permission remains separate only with a realization proof accounting for any concrete transfers it hides; internal placement and projection do not create an exemption.

Concretely, a combinational coupler reads upstream valid/data and downstream ready values and deterministically computes downstream valid/data and upstream ready values. It contains no state and no credit/capacity. Any stateful feedback passes through an explicit finite-capacity channel. The same definition covers a coupler placed immediately at a process boundary: it remains part of the elaboration's same-cycle handshake network and does not become a source process and owns no source operation or attributed source request.

Example (two inputs with early staging, source-facing paired Pops at stage 3, II=1 under the stated sustained-flow conditions). Consider the source body Pop_c1; Pop_c2; Push_cout(pair(v1,v2)), with neither Pop descriptor depending on the other return. The bound controller has two independent input-storage paths before stage 3, a declared atomic group of the two process-facing Pops at stage 3, and the process-facing Push at stage 4. The paired value is used no earlier than the cycle after both Pop returns, under Core.RegSeq. A group-serial witness issues the two Pops as one qualified batch, retaining their legal internal issue order and all external dependencies. This is a specified sufficient organization, not an attribution of stage-1/2 physical reads to those source Pops.

- Input c1 uses a physical FIFO of F_1>0 entries followed by two one-item locations at physical stages 1 and 2. Input c2 independently uses a physical FIFO of F_2>0 entries followed by two one-item locations at stages 1 and 2. Each path satisfies Q.CT.InputStaging, preserves its own unmodified values and ordinals, and supplies a retained item to its stage-3 Pop. Physical early reads and internal forward steps are channel actions under the declared elaboration/projection, not process-facing Pop issues. No join or sibling-ready condition obstructs the isolation-scoped forwarding law.
- The process-owned stage-3 requests are attributed to Pop_c1(j) and Pop_c2(j) under Core.10/Core.IC. The controller legally prepares and issues both in one pre-activation L1 window, as required for the atomic group. The settled process-facing join commits them together exactly when the complete qualified settled handshake holds, including both terminal items, attributed requests and the joint guard. The coupler is stateless; the held input values reside in the two declared paths. This example has no B(0) leg: physical handoff wiring is not a process-facing rendezvous when accepted input storage precedes the selected Pop.
- E3 and the iteration boundary. Neither stage-3 request for iteration j+1 may issue before Push_cout(j) has issued, and Core.16 must permit the batch. In an unstalled interval that Push at stage 4 and both next-iteration Pops at stage 3 can issue in the same cycle, preserving E3 with equality across the iteration boundary. After frontier activation no new later batch launches; already-issued work may drain. Bubble movement inside either input path is not new source issue and remains subject to the fixed channel/epoch accounting. This is the two-input counterpart of B.BoundaryBinding. For sustained II=1, both F_1 and F_2 must be at least 2 under the old-state ingress rule. Both paths select simultaneous replacement and a full shift each cycle, both input streams remain supplied, the output accepts each cycle, the process stages support that retirement/advance schedule, and no credit or guard throttles it. Equality of issue times alone proves none of these rate conditions.
- Storage accounting. Without an additional credit cap, B(c1)=F_1+2 and B(c2)=F_2+2 summarize the respective input storage, with floors 1 <= g_1 <= F_1 and 1 <= g_2 <= F_2. These sums do not replace the actual elaborated handshake equations. Once the paired Pops retire, their values are process state through stage 4. There is no post-join channel in this example and no pre-retirement joined bundle occupying output storage. Push_cout uses a separately declared output channel and its own resource law.

Qualification of the example. Q.CT.InputStagingProgress proves each input's space and isolated delivery facts; independent retention makes both items coexist for Q.CT.Groups and K.CT.JointLemma. The legal source batch, boundary/issue mapping, complete fact correspondence, clock semantics, stage progress, joint guard and service remain explicit conformance obligations. A physical controller that instead makes the stage-2 c2 handoff its process-facing Pop while delaying the older Push to stage 4 may violate E3 at II=1 and is not qualified by this example. An internal joined-work channel or an earlier output boundary requires its own faithful mapping and component/extension proof. For completion, eligible independent paths may instead use K.CT.LazySimulation with whole-batch eligibility and demand-supported extra-slack releases; ordinary Push cases retain the standard floor argument. The stage-1/2 moves must be

optional abstract moves under the declared reference law, not mandatory physical ready/valid boundaries. The lazy group-serial witness is distinct from the sustained II=1 schedule above. See the atomic-batch case in K.CT.LazySimulation.Example and the route table at the head of K.CT.LazySerial.
Informative coupler illustration — storage-free join leg. The preceding example has independently staged inputs. A different and important topology has a one-item channel F1 on the first input, a zero-storage handoff leg F2 on the second input, and a two-item channel F34 for paired work after a join. For a physical pipeline drawing these may occupy stage 1, the stage-2 join, and stages 3–4 respectively. These physical labels do not identify the source-facing Pops or prove an initiation interval.
Let a mean that F1 has its old head available, b mean that the second input presents its stable offered item, and d mean that F34 has space in the old state. A stateless join has fire = a AND b AND d, complementary readiness towards F1 equal to b AND d, and complementary readiness towards F2 equal to a AND d. On fire it consumes the old F1 head and the offered F2 item together and stores their pair in F34. If a is false, F2 cannot accept even though F34 has room; if d is false, neither input is consumed. The F2 leg retains no item on its own. These equations illustrate mutual acceptance dependence, not withdrawal of a process-owned request: the actual request roots and their attribution remain governed by Core.10/Core.IC. Each sampled fire performs all declared effects mandatorily. Prove the mapping between the two input transfers and the output pair; absence of storage in the coupler alone does not identify these distinct endpoint effects.
F1 contributes one item of pre-join storage and F34 contributes two paired-item slots after the join; F2 and the combinational coupler contribute no capacity. The paired-item accounting must track both input ordinals without duplicating their inventory. Downstream paired storage cannot be assigned as independent freely usable slack to each input. Nor is F34 automatically an output channel: it holds joined input work until a separately proved boundary mapping says otherwise. This is why explicit storage and the actual coupler equations are both needed.
Qualification boundary for this illustration. This topology is not the independent-forwarding Q.CT.InputStaging component: acceptance of one leg depends on the other, and the second leg has no independently retained input. Treat it as a candidate elaboration requiring the applicable Q.SafetyQualified accounting, source-legality, request ownership, settling and projection proofs, plus Q.CT.Extension for completion unless a separately stated component theorem applies. In particular, do not identify the stage-2 handoff with Pop_c2(j+1) while asserting Push_cout(j) only at stage 4 and claim II=1 under E3. A process-facing mapping must satisfy B.BoundaryBinding, E3 and Core.16, with its own storage/resource and joint-progress proof. The drawing and equations explain coupling; they do not supply that conformance proof.

This elaboration refines the abstract bounded channels at the chosen outer observation boundary without changing the outer event labels or E4 capacity/order meaning. It may introduce proof-internal storage-channel commits, and it may place stateless coupling logic at internal or outer process-facing boundaries. Any elaboration must satisfy:
• Outer-history preservation: π_Σ,E neither creates, drops, duplicates, nor relabels outer Push/Pop events at the complete accounting boundary. The subsequent ρ_DUT restriction selects DUT observations without changing accounting.
• E4 storage legality: every explicit finite-capacity/rendezvous channel has its own B(·), occ_·(t), FIFO/rendezvous legality, and committed-transfer accounting. Outer projected commits remain E4-legal under Π_elab.
• Declared handshake completeness: every ready/valid/data value used by ChannelEnabled/ChannelDisabled is produced by the E-declared finite same-cycle network from fixed process request roots, explicit channel state, and permitted fixed inputs. A coupling-sensitive boundary may therefore be disabled despite local capacity, but never because of an undeclared predicate.

• No double counting: movement within one represented storage realization is ε unless it crosses an explicit abstract channel boundary; a committed proof-internal channel transfer may be projected away from Σ_DUT but remains part of the proof-internal history.
• Conservation/accounting: internal movements erased by π_Σ,E preserve outer occupancy/accounting. A transfer mapped to an outer commit updates that accounting even when ρ_DUT later hides the event from DUT observations. Bundle/join accounting is declared by E.
• **Multi-input pipelines / combinational couplers (mutual stall + explicit capacity):**
Any downstream storage reachable only through a join is modeled as explicit finite-capacity channel state rather than as hidden coupler capacity. The coupler may enforce mutual stall by computing ready/valid from several explicit channel/request conditions, but it remains stateless. Accordingly, the coupler does not redefine E4's storage capacity/order legality; it determines the actual settled handshake eligibility at the boundary. This same rule applies if the coupler is placed at the process-facing outer boundary.

Projection/accounting and handshake invariant for Sys_B^elab (normative implementation of Core.ElabProj). Π_elab preserves the selected outer event history and occupancy accounting. Separately, the E-declared combinational network preserves the actual handshake structure: a source-attributed BoundaryReq is never withdrawn by the coupler, every complementary ready/valid value is a deterministic function of the declared current roots, and an attributed boundary action commits exactly when its BoundaryReq and settled complementary signal are asserted at an eligible sampled edge under Core.TransferLaw. The qualified equations establish E4 legality and atomic/ownership coherence before overlap; those conditions are not later vetoes. Cross-channel/join dependence may therefore appear directly in ChannelEnabled/ChannelDisabled at any boundary designated by E, including a process-facing outer boundary; what is forbidden is hidden or stateful coupling outside E.

A stateful staging element affecting acceptance or progress must be represented in the declared storage and accounting. Stateless couplers are represented by their complete actual handshake equations. Internal transfers may be erased from the selected outer trace only under the projection/accounting laws; any fact consumed by completion must still be covered by J.InternalFactCorr or a separate state-conformance invariant.
• Combinational well-formedness: the complete coupler/ready-valid network is finite, deterministic, and acyclic after cutting at explicit registered/storage boundaries. Any feedback or remembered condition required for correctness is represented by explicit finite-capacity/registered state. This condition is exactly what permits one settled handshake valuation to be reconstructed from E plus the current state/request roots.

The proof does not depend on a particular back-annotation algorithm. The supplied Sys_B^elab(E) satisfies Q.SafetyQualified's actual storage, accounting, ownership, projection and complete handshake obligations. Q.1(i) supplies the elaboration-owned settling contribution; a consuming constructive theorem separately establishes the combined operational settling and root/equation correspondence it requires. Core.SettleRootAgreement applies only with its stated premises. Completion additionally uses Q.CTQualified and K.CT's resource, exposure and service contracts; these are not reconstructed from history alone. Aggregate accounting uses the complete outer history from π_Σ,E, before the ρ_DUT observation restriction.

Lean-oriented clarification / proof-parameter convention

The formal theorems do not require a unique algorithm that constructs Sys_B^elab from RTL_B. Instead, the chosen comparison boundary Sys_ref is part of the theorem instance. For ordinary bounded-FIFO cases, Sys_ref may be Sys_B. For cases involving proof-relevant internal staging, multi-input joins, combinational couplers, sync-boundary epoch gates, or shared-capacity effects, Sys_ref shall be supplied as Sys_B^elab(E), with E satisfying Q.SafetyQualified and, for completion, the applicable Q.CTQualified obligations. Every invoked theorem retains its own qualification requirements.

Thus elaboration is not an extra semantic freedom inside the proof. It is an explicit input to the proof instance, and all Appendices J and K definitions are interpreted over that chosen Sys_ref.

Qualification selection. Q.SafetyQualified supplies the operational, history and accounting interface of Q.1. Q.CTQualified additionally supplies the component progress laws for completion. Each invoked constructive theorem retains its local, carrier, source-legality and feature restrictions; these interfaces do not prove their own implementation realization.

Definition Q.1 (Elaboration package for Sys_B^elab).

For a chosen outer source reference boundary Sys_B, its complete outer history alphabet Σ_outer, and a selected DUT-observation alphabet Σ_DUT, an elaboration package E consists of:

1. an elaborated source reference model Sys_B^elab(E);
2. a finite set of explicit abstract channels Chan_elab(E), each with finite capacity B_E(d) ≥ 0 and nonnegative cycle-boundary occupancy occ_d(t);
3. a distinguished subset of outer-boundary channels Chan_outer and a channel projection map

   Π_elab : Chan_elab(E) → Chan_outer ∪ {⊥};

4. a complete outer-accounting trace projection π_Σ,E : Σ_elab* → Σ_outer* and a subsequent observation restriction ρ_DUT : Σ_outer* → Σ_DUT*, with π_DUT,E := ρ_DUT ∘ π_Σ,E. Here Σ_outer includes both committed Push_c and Pop_c events for every c ∈ Chan_outer, together with other outer event kinds selected by the theorem instance. Σ_DUT may omit an endpoint event, subject to the whole-atomic-group requirements below; that omission does not alter the complete accounting history. The trace notation retains the existing cycle/occurrence, reset-epoch and proof-unit structure rather than introducing a new serialization convention;
5. a state-accounting map A_E with signed integer codomain and, for each outer channel c, a finite E-declared accounting-support set AccountSupport_E(c). This set names the explicit channel/storage, token, credit and accounting fields from which A_E(c,σ) is computed. The ordinary case uses only the fiber Π_elab^{-1}(c); a qualified shared-storage representation may additionally name fields outside that fiber. Support sets are fixed by E independently of a selected history, and their fields are part of the explicit elaborated state or proved derived views of that state; and
6. a finite proof-internal elaboration-token universe ElabToken(E), a token projection

   Π_token,E : ElabToken(E) → Set(Chan_outer),

   with Π_token,E(u) finite for every token u, together with an E-declared attribution that is functional when defined: each relevant staged/bundle/join transfer occurrence is assigned to at most one token. The attribution may be undefined for an occurrence with no outer effect. Π_token,E identifies the finite set of outer channels whose π_Σ,E effects are accounted for by that token; π_Σ,E determines the

corresponding outer Push/Pop labels, payloads, and ordering.

Qualification scope. Q.SafetyQualified(E) comprises items 1–6, clauses (a)–(i), including (c-order), and the Accounting-support and Shared-storage qualifications below. A qualified or well-formed elaboration package means this interface. It retains all actual storage, handshake, ownership, payload/occurrence, reset, atomicity and required-order obligations. Completion additionally requires Q.CTQualified. These declarations do not establish source/implementation correspondence, source reachability, complete whole-system settling, progress or source-property validity: the chosen consumer separately discharges those obligations. No scheduling permission or capacity is introduced by qualification.

The following clauses specify these qualifications; their scope is the qualification rule above:

(a) Complete outer-trace preservation. At the complete outer Sys_B accounting boundary, π_Σ,E neither creates, drops, duplicates, nor relabels any Push_c(v) / Pop_c(v) event. A qualified Core.LogicalPopPayload value is part of its event definition before this projection, not a value change performed by π_Σ,E; physically sampled observations cannot acquire that exception through projection. This obligation precedes ρ_DUT; hiding an endpoint from DUT observations is permitted only at that subsequent restriction and does not waive (a)–(c).

(b) Occupancy/accounting preservation. For every outer channel c and every ε-quiescent cycle-boundary state σ at time t, let π_t be the c-projection of the complete outer history produced by π_Σ,E, before ρ_DUT and including both Push_c and Pop_c endpoints, after the most recent RESET affecting outer channel c (or its reset-empty initialization) and strictly before cycle t. Here σ is the cycle-entry boundary view before the selected cycle-t transfers; post-L3 current-state observations are not substituted for this view. The signed state-accounting value computed from the elaborated state shall agree with that independently reconstructed outer history:

$$A_E(c, \sigma) = \mathrm{push}(\pi_t) - \mathrm{pop}(\pi_t).$$

Here, because c ∈ Chan_outer, E4, occ_c, and B(c) mean the corresponding outer Sys_B channel semantics, not the B_E(d)/occ_d semantics of the explicit elaborated channels in item 2. By that outer E4 legality, this same signed balance equals the legal nonnegative outer cycle-entry occupancy occ_c(t) and satisfies $0 \le \mathrm{push}(\pi_t) - \mathrm{pop}(\pi_t) \le B(c)$. Thus, viewing the legal occupancy as an integer, $A_E(c,\sigma) = \mathrm{occ}_c(t)$ and $0 \le A_E(c,\sigma) \le B(c)$. This range is a derived consequence of the Q.1(b) agreement plus outer E4 legality, not a consequence of A_E's signed codomain. Every π_Σ,E-projected Push_c / Pop_c commit updates A_E exactly as required by outer E4 for channel c, whether or not ρ_DUT retains that event. Affecting RESET restarts its accounting at zero in accordance with RW1–RW3; RW5, together with the reset-preservation proof required by Accounting-support qualification, preserves unaffected outer-channel accounting. Shared support fields may change under a reset, but the qualification must prove that each unaffected outer balance remains invariant; it need not freeze every field read by that balance. Discarded reset contents are not fabricated outer Pop events. At other intermediate states, A_E follows the initialization and update rules whose preservation is required by Accounting-support qualification and Q.1(c); this cycle-entry equation is not a claim that post-L3 occupancy equals beginning-of-cycle occupancy.

(c) Internal refinement and multi-effect projection. An internal staged/bundle/join transfer with no complete outer-accounting effect may be projected away by π_Σ,E and may have no attributed token. Any internal staged/bundle/join transfer that contributes one or more outer effects shall have the item-

6 attribution defined and is therefore attributed by E to exactly one token u ∈ ElabToken(E); Π_token,E(u) is the uniquely determined finite outer-channel projection for that token, and π_Σ,E determines the corresponding one-or-more outer Push/Pop commit effects consistent with it. A single token may therefore account for multiple outer channels/effects. No outer effect may be invented, omitted, duplicated, or multiply charged by the token projection. Transfers erased by π_Σ,E preserve every outer signed accounting value A_E(c,σ). A transfer retained by π_Σ,E but omitted only by ρ_DUT is not an accounting-erased transfer: its outer effects still update A_E. For example, an accepted input followed by its loop-body Pop contributes one Push and one Pop to π_Σ,E and returns the input balance to zero even if the DUT-observation trace retains only the Push.

(c-order) DUT projection-group typing and required-order reflection. The accounting and observation projections remain separate: clauses (a)–(c) apply to π_Σ,E, while the selected groups below describe π_DUT,E = ρ_DUT ∘ π_Σ,E. The observation restriction may retain a complete selected atomic group or erase it as a whole; it may not expose only a proper fragment of an explicitly atomic proof unit or already contracted proof atomic group. In particular, omitting a buffered loop-body Pop that is a separate group does not authorize splitting a rendezvous pair or another indivisible group. For every complete proof-internal observable/proof-unit occurrence x whose selected π_DUT,E image is nonempty, E shall declare unique projection-group metadata ProjGroup_E(x)=⟨gid_x,Eff_x,≺_x⟩. gid_x is a canonical group occurrence identity inherited from x together with the fixed E/token projection identity; Eff_x is a finite nonempty occurrence-indexed family of constituent Σ_DUT event occurrences; and ≺_x records the E-declared internal strict order/alignment among those constituents. Flattening ProjGroup_E(x) shall reproduce exactly the complete π_DUT,E trace fragment contributed by x, with the same labels, payloads, event occurrence identities, and required internal order; a singleton image is the one-event special case. When one token or contracted proof action contributes several outer effects, all of those effects belong to this one indivisible projection group for ideal-prefix purposes: no projected history or ideal may expose only a proper subset of its constituents.

When E is bound to a concrete theorem instance and legal proof history, required outer order shall also be reflected structurally. For any two distinct retained projection groups A and B, if the selected outer semantics preserved by π_DUT,E require a constituent of A strictly before a constituent of B under E1–E5, outer E4/FIFO order, or another explicitly selected outer ordering rule, E shall supply a finite proof-unit sequence x_0,…,x_n with ProjGroup_E(x_0)=A and ProjGroup_E(x_n)=B such that every adjacent pair is justified by one already normative strict predecessor rule of that theorem instance: an ordinary Core.17/E1–E5 required-order rule or a separately declared proof-internal predecessor/successor rule. Every rule used by such a chain shall itself carry the intrinsic two-part prefix-well-formedness certificate needed downstream: on every legal carrier, every legal prefix containing its completed successor also contains its completed predecessor, allowing co-completion, and the truth of the rule between two already-existing endpoint occurrences is stable under extension/restriction of legal prefixes so later history cannot retroactively create that old-old edge. The certificate therefore does not invent a second ordering relation and does not depend definitionally on Appendix J; Definition J.OuterCanonHist later collects exactly these admissible strict rules into ≺_req, so the supplied sequence becomes the required ≺_req^+ path. An ordering that is merely an incidental serialization chosen by π_DUT,E among semantically independent selected outer effects need not receive such a cross-group certificate. Conversely, the package may not omit a required outer order merely because the responsible transfers occur on different explicit fibers in Π_elab^{-1}(c), pass through erased internal units, or arise from multi-effect token projections. Thus projection-group internal order plus the certified normative edge chains reconstruct every required ordering of the flattened outer trace. These two prefix properties are separate from joint acyclicity. The union of normative proof-unit generators and any nontrivial

projection contraction must also satisfy J.RequiredOrderWF, by operational derivation or independently checked strict-order evidence before canonical histories are compared. On covered constructive prefixes, J.SourceOrderReflection may supply retained RTL group acyclicity from the constructed source; any consumed unmapped proof-unit subgraph still requires evidence. This is a proof qualification of the existing required order, not permission to add order, merge units or change scheduling.

(d) Complete stateless handshake network. Every same-cycle join, mutual-stall, bundle, sync/epoch gate, shared-capacity handshake, and process-boundary coupler that can affect a selected boundary ready/valid/data value is included in E as deterministic combinational logic. Such logic introduces no stored state and may be attached to an internal or outer process-facing boundary.

(e) Settled boundary visibility. For every evaluated frontier boundary, E identifies the own-side source-attributed BoundaryReq root and the complementary mirror value used by Core.11. Primitive channel storage legality is E4; the actual ChannelEnabled/ChannelDisabled value is the settled handshake produced by the complete E network. No undeclared grant, throttle, backpressure, or guard may affect it.

(f) Combinational well-formedness. After cutting at explicit registered/finite-capacity state, the complete elaboration-owned ready/valid/data network is finite, deterministic, and acyclic; any feedback/storage required for correctness is represented by explicit state.
Accounting-support qualification. AccountSupport_E permits shared state to contribute to several outer balances without changing the physical capacity of any explicit channel. Each support field has one declared operational owner and exact initialization, transfer and reset update rules, or is a derived view proved to track those rules; a copied accounting view creates neither independent storage nor a new transfer. Prove that A_E depends only on its declared support, that the support is available from the qualified explicit state, and that every projected transfer, erased internal movement and affecting/unaffected reset preserves the Q.1(b)–(c) accounting invariant. State-versus-history agreement remains an independently proved invariant: A_E may not be defined by reading the outer history it is required to validate. Token effect attribution and Π_token,E do not by themselves prove this state invariant.
Shared-storage scope. Per-tag explicit queues may represent a shared pool when the declared occupancy/credit invariant and joint admission equations enforce the aggregate capacity, including simultaneous admissions. If head-of-line behavior depends on tag interleaving, retain that order as explicit state, for example in the single shared FIFO read by the relevant accounting supports, or in a separately qualified ordering-token representation with synchronized updates. Independent per-tag queues alone do not preserve that order. Any alternative boundary placement must still satisfy outer-event, value/order, handshake conformance and the selected completion obligations. These are permitted representations subject to qualification, not a qualification theorem for a particular template.
(g) Endpoint roles and process ownership. Every source-level logical message channel retains exactly one producer process and one consumer process, and every source-attributed frontier item records its owning process/operation/direction. An explicit Sys_B^elab storage boundary may have a stateless coupler on one side; the coupler is an endpoint realization component, not a modeled process. Shared source resources that truly have multiple process owners are remodeled through explicit arbitration and point-to-point process-facing channels.
Clarification (logical channel versus explicit boundary). A source logical channel retains its producer and consumer operation owners. An elaborated internal endpoint may instead connect to a stateless coupler, which is not a source process and owns no operation. MsgOf_P(x) identifies source ownership;

BoundaryOf_P(x) identifies the actual evaluated handshake. Do not replace a coupled boundary by a direct peer/full/empty test unless its complete equations justify that replacement.
(h) Frontier-item declaration and boundary attribution. E declares every proof-internal frontier item used by Sys_B^elab, including its owning process, Ω_P/Ω_msg,P membership, owning source-level message instance MsgOf_P(x), evaluated handshake boundary BoundaryOf_P(x), and ≤_P relation. BoundaryReq(x,σ) remains the persistent own-side request attributed to that operation/frontier item. A stateless coupler may transform the complementary mirror signal at the same boundary but does not acquire MsgOf ownership and does not create an independent source request. For elaborated calls with several frontier items, E states how those items and any proof-internal commits project through π_Σ,E/Π_elab/Π_token,E and A_E back to the outer operation/event/accounting view.
Typing and topology changes. Relabeling a boundary is a typing-only change only if storage, transfer semantics, handshake equations and projection/accounting are unchanged. Adding or removing a buffer, registered stage or credit changes the reference model and requires fresh qualification of every affected accounting, source realization, capacity and completion law. Renaming a boundary does not preserve evidence for a different topology.
(i) Qualified elaboration-side same-cycle settling. The elaboration package is responsible for the Core.Settle L2-frame and source-side settling facts contributed by the staged/bundle/join/coupler/epoch-gate storage and handshake structure. For that elaboration-owned cycle-local subnetwork, every applicable →ε,L2 step shall respect the L2 frame—preserving the cycle index, registered process/FSM/pipeline state, registered explicit-channel storage/occupancy/credit state, and cycle-t exogenous inputs, with no explicit-boundary Σ commit or next-cycle registered update—and, after cutting at the state-bearing boundaries, the subnetwork shall be finite, deterministic, combinationally acyclic, and complete for every ready/valid/data and boundary predicate it owns. The package shall provide or cite a compositional/template proof of these template-side facts; that proof may be qualified once for each legal elaboration template and then bound to the concrete E instance.

Consumer-triggered whole-source composition note. The template-side facts above do not by themselves imply QualifiedSynchronousModel_M(I) for M = Sys_B^elab(E), and whole-source design/template composition is not a Q.1 package-well-formedness obligation. When a selected consumer requires source-side Core.QSync, that consumer's Core.QSync discharge shall separately establish the applicable design-owned source-settling facts, including stable sequential drives under Core.RegSeq/direct-input contracts, inclusion and deterministic evaluation of R2C combinational components, the applicable non-template L2-frame facts, and completeness at non-template selected boundaries, together with a checked template/design settling-composition condition covering every cross-partition dependency and satisfying QS2–QS5, in particular QS4 for the complete combined same-cycle settling graph. A checked combined dependency rank or topological order remains a sufficient mechanically checkable discharge. The M.7a Core.QSync entry is the authoritative evidence interface for this consumer-triggered composition; together with the Q.1(i) template facts it establishes the applicable QualifiedSynchronousModel_M(I) fact used by Core.SourceL2Settle. Definition Core.Settle by itself does not assume a fixed point exists. Safety-only locality specialization. J.ElaboratedLocalityProfile and J.ElaboratedConstructiveComposition admit the stated separately owned FIFO/coupler schemas under Q.SafetyQualified(E), complete operational settling/matching and their local evidence. They do not infer whole-source settling from this template clause, admit shared pools, or imply completion qualification.

Q.CT Completion qualification of elaborated components

Q.CTQualified(E). This is Q.SafetyQualified(E), the applicable B.CT operational declarations and invariants, and the component clauses below. The chosen J.InternalFactCorr adapter retains its complete local fact, source-legality, attribution and carrier obligations. Safety-only qualification is not progress qualification. Required order, complete accounting, atomic projection, B-faithfulness and same-cycle settling remain explicit; no history equality substitutes for exposure or service.
Q.CT.Inventory. The elaboration identifies every progress-relevant state field, operation, control gate and same-cycle dependency, with initialization and exact successor effects. It assigns each stored item, reserved slot, credit and request one owner and a stable logical key; complete accounting precedes DUT observation restriction. Every channel resource is supplied by B.CT.FIFO or an explicitly proved replacement law, every guard by B.CT.Guard, and every grouped transfer by B.CT.Joint or an explicit replacement. State sharing between components requires a joint invariant and ownership proof; separate component declarations cannot each spend the same credit or claim exclusive control of the same slot. A template name or equality of nominal capacities supplies none of these facts.
Q.CT.Positive. An ordinary FIFO or FIFO-plus-reservation component satisfying B.CT.FIFO, B.CT.EndpointOrder and B.CT.Capture has guaranteed resource floor g and persistent item attribution. Proof: initialization establishes empty balance. A Push increments unmatched inventory by one and appends its unique item; capture moves an item from queue to its attributed slot without changing unmatched inventory; retirement from the attributed slot removes that item and decrements inventory. A coincident read-and-retire removes the matching old head directly from the queue and decrements unmatched inventory by one, without creating persistent slot occupancy. The combined old-state update of every permitted simultaneous action preserves A=queue+slots and FIFO matching, including simultaneous read/write: the read removes the old head and the write appends the new tail. The floor inequalities are static parameters. K.CT.ResourceLemma derives target availability from this invariant and predecessor completion. The argument supplies resource progress facts, not a legal lift of an arbitrarily retimed source execution; K.CT.Capacity retains that separate source obligation.
Q.CT.InputStaging. This sufficient component replaces the primitive at-site delivery law for an input channel with one exclusive ordered producer and one exclusive ordered process-facing consumer. It has an initially empty ingress FIFO of F>0 entries followed by a fixed finite nonempty linear chain of registered one-item locations before the selected Pop. A terminal location retains the item until that Pop retires. All locations, including any additional retained slot, are explicit channel storage with one location per item; internal transfers move, never copy, an item. Payloads are unchanged, ordinals follow the channel's global FIFO order, and later items never pass earlier ones. The last location is not revoked or overwritten while its old item remains unretired. If that item retires on an edge, the declared simultaneous-replacement option below may install its successor in the same edge update. No shared storage, side exit, logical payload transformation, affecting reset, internal epoch gate or hidden admission dependency is allowed in this component. A physical representation that does not retain the original payload requires the separate Q.CT.LogicalPayload qualification; otherwise the unchanged-payload realization remains required. An optional exclusive outstanding-item credit C>0 is permitted: A counts accepted Pushes minus retired process-facing Pops, equals FIFO plus all staged inventory, and capture/forwarding does not release credit. The producer's exact own-resource predicate is ingress occupancy <F and, if credit applies, A<C; other dependencies need a separate qualified law.
Q.CT.InputStaging edge law. Declare either old-empty-only acceptance at each internal location, or simultaneous replacement. In the latter option a location vacated by a selected forward move or process-facing retirement on an edge may receive the next item on that edge. All eligibility tests and payloads use the common old registered state and the qualified settled current-cycle signals; the complete action selection is consistent and all next-state updates are installed together. For an internal

location i, its outgoing selection requires an old resident item, and its incoming selection requires that it was empty or that its old resident is selected to leave. Its next occupancy is old occupancy minus outgoing plus incoming. An outgoing payload is always the old resident, never the newly received value: an item crosses at most one internal location boundary on an edge, and a newly filled empty terminal location cannot retire that new item on the same edge. A qualified acyclic downstream-to-upstream selection may use the selected departure to determine replacement eligibility; it does not read next-cycle state. A constructive local-realization use of the simultaneous-replacement option discharges Q.CT.InputStagingResidue (or an explicitly proved equivalent C3 residue argument), with its M.7a evidence entry; the operational edge equation alone is not that discharge.
The ingress remains an old-state non-fall-through FIFO: forwarding uses its old head and requires old occupancy>0; producer Push requires old occupancy<F and the declared old credit test. Forwarding from a full ingress does not enable a previously disabled producer Push on that edge. Internal locations using simultaneous replacement are part of the declared channel realization, not separate old-empty-only E4 primitives. If they are instead modeled as separate such primitives, only their own legal edges may be selected. At every explicit boundary, retain its E4 law and exact projection/accounting; the internal replacement option is not a fall-through or full-FIFO bypass exception at that boundary.
Q.CT.InputStagingResidue (fixed-selection residue obligation). For a declared replacement-capable component, fix the selected set T and every transferred owner/payload from the common pre-edge snapshot. For a chain of m internal locations, let in_i and out_i denote its selected incoming and outgoing descriptors, when present. Order selected moves downstream-to-upstream, with terminal retirement before a move replacing its slot. At each filled destination, the selected outgoing descriptor precedes its selected incoming descriptor; at a source, removal of its old resident precedes any selected replacement. Supply an acyclic local order L consistent with all other required dependencies. For a predecessor-closed executed subset D of T, define occ_D[i] = old_occ[i] − 1[out_i in D] + 1[in_i in D], treating absent descriptors as indicator zero. If in_i is in D, the resident is its frozen incoming owner/payload; otherwise it is the old resident when old_occ[i] = 1 and out_i is not in D, and empty otherwise. At the first location the frozen incoming value is the old ingress head, not an undefined predecessor register. For an L-minimal incoming move outside D, any selected outgoing move from its destination is already in D. The selection law old_empty(i) or outgoing_selected(i) therefore makes the current residue destination empty. Its source outgoing move is this unexecuted descriptor, so the source still holds the frozen old resident. These residue and frozen-snapshot facts discharge the corresponding C3 primitive premises when the declared source component has those transitions. Executing D never selects an additional move, changes a transferred payload, or permits multi-hop fall-through. The ingress and every separately declared E4 primitive retain their own old-state premises. For coupled, staggered or transformed representations, instantiate this argument only after proving the joint ownership/representation invariant, indivisible group transitions and combined acyclic dependency order. This is a component proof obligation under the general constructive contracts, not an extension of ordinary E4 or of ELM/ELA primitive coverage. Scope: this local-realization obligation applies whenever the selected constructive safety proof consumes these replacement transitions, independently of any K.CT completion claim. Its location in Q.CT does not impose completion or service premises on that safety use. Discharge it, or the explicitly mapped equivalent C3 argument allowed by M.7a, together with the remaining local contracts; Q.SafetyQualified alone does not supply that constructive realization proof.
Throughput interpretation (informative). The progress lemma below establishes eventual delivery, not a rate. With m internal locations, a full shift and terminal retirement on each edge can preserve a full chain while replenishing an ingress held at occupancy one, provided F>=2, producer supply and downstream acceptance are continuous, the simultaneous-replacement option is selected and exercised, and all process advance, guard and credit conditions permit that schedule. With F=1, the old-

state ingress rule prevents Push and removal from occurring together, so this ingress cannot sustain one new item per cycle; stored items can only mask the limit temporarily. Old-empty-only internal locations likewise do not support the illustrated sustained full shift. Finite-response guards and fair service alone give no throughput guarantee.
Forwarding law. When a target Pop is at its bound site, legally pending and permanently isolated under B.CT.Guard, and its matching accepted item is the oldest remaining item, each next move towards the terminal location exposes an actual legal opportunity when its destination remains empty, either immediately or through a finite-response forward-control realization satisfying B.CT.Guard's decreasing-measure and internal-service conditions; the exposed move then receives declared internal service. It cannot require the target Pop to retire first, the item already to be at the terminal location, new upstream traffic, an unfinished sibling or another input's arrival. Actual moves preserve identity/ownership, advance along the fixed chain and cannot be reversed by surrounding activity. Outside this isolation condition, a non-compressing controller may hold staging behind older incomplete process work; bubble compression is permitted, not required. At the terminal location, B.CT.Guard establishes the required readiness; a qualified physical handshake retires the operation on its eligible edge under Core.TransferLaw. A genuinely abstract retirement instead uses its separately qualified service. The item remains available until retirement, and the required clock/support premises remain in force. This is an eventuality under isolation, not a prohibition on earlier forwarding. For the standard staging completion proof, establish target placement, legal preparation and persistent operation-owned BoundaryReq assertion without assuming that target item's traversal to the terminal location or its retirement. Earlier completed control/value predecessors remain available. These are proof dependencies: earlier forwarding is permitted, but the arrival conclusion cannot be used to establish the placement/request premises of the same delivery lemma. A data-gated transfer pulse is not a substitute for the attributed pending request of B.OF.Request.
Boundary and compatibility obligations. Bind the concrete source/RTL instance under B.BoundaryBinding and Q.SafetyQualified, including B.CT.State/EndpointOrder, exact internal/outer projection, E3/Core.16, Core.IC, Core.RegSeq, the declared InputStaging edge law, old-state non-fall-through ingress and explicit-boundary equations, full consumed-fact correspondence and actual service for every forwarding step. The forward law is a local implementation requirement, not inferred from FIFO order or nominal capacity. This component supplies positive resources for blocking singleton units and pure positive same-site atomic groups. Under this independent-path law each grouped input has disjoint storage/credit and forwards and retains independently. The separate Q.CT.CoupledInputStaging law replaces independent forwarding for its equal-length pair only; Q.CT.StaggeredInputStaging supplies the fixed-offset late-read alternative. Source batch legality and B.CT.Joint remain required in either case. It does not by itself qualify mixed B(0) groups, NB retry controllers, physically transformed staging or epoch-gated/reservation-sharing variants. Q.CT.LogicalPayload supplies the physically transformed singleton case; Q.CT.LogicalPair supplies the restricted computed positive pair. Other transformed groups and combinations require their explicit Q.CT.Extension proof. Physical movement never licenses otherwise forbidden source launch or issue. This component uses the declared synchronous relation; an interleaving presentation needs its own exact edge/frame and service proof, since K.CT.ClockRefinement's at-site carrier does not automatically represent early forwarding. Apply this noncircular placement/request qualification also to the coupled and staggered alternatives. A controller whose only route to terminal placement and request assertion is travel with the target item needs the separate pre-request progress proof described in Q.CT.Extension.
Lemma Q.CT.InputStagingProgress (guaranteed space and isolated delivery). Assume the storage, edge, forwarding and retirement laws of Q.CT.InputStaging, either directly or as the operational clauses retained by Q.CT.LogicalPayload, and completed K.CT.Closure predecessors for a missing blocking unit. Choose any positive integer g<=F and, when credit is present, g<=C. For Push(k), endpoint order

excludes later Pushes. For zero-based k, if k>=g, Pop(k−g) and earlier Pops have retired, removing k−g+1 of the k earlier items and leaving at most g−1 unmatched; if k<g, at most k<=g−1 earlier items remain. Ingress occupancy is bounded by this inventory, hence is <F, and A<C when applicable. Until Push(k) retires, no later producer call takes that space and receiver forwarding/retirement cannot increase ingress occupancy or unmatched inventory. Thus its own space support persists. Its item-delivery clause assumes independent forwarding; coupled-pair delivery uses K.CT.CoupledDeliveryLemma under Q.CT.CoupledInputStaging instead. The per-channel floor arithmetic is unchanged. Fixed-offset staggered delivery instead uses K.CT.StaggeredDeliveryLemma; the independent item-delivery clause is not applied to its late-read coupling. For an eligible lazy witness, K.CT.DemandPushResource is the alternative Push-space proof. It leaves this guaranteed-floor lemma unchanged and uses the same independent isolation-scoped forwarding law.

For Pop(k), the matching Push and all earlier Pops have retired as required by Closure. Exact FIFO accounting places item k uniquely in the ingress or a staged location. There is no older item downstream, and younger items are behind it, so every location ahead of k is empty and cannot be stolen by younger traffic. PlacementLemma and lawful request assertion establish permanent isolation and a pending target unless it has already retired. Apply the forwarding law to the next required move: persistent empty-destination support gives finite forward-control progress if needed, and internal service performs the move. Its remaining distance strictly decreases. Repeating over finitely many locations brings the item to the terminal location, where it is retained. No uniform cycle bound or bubble compression before isolation is assumed. The qualified immediate or finite-response retirement guard then gives persistent support. A physical singleton or atomic handshake commits on its eligible edge under Core.TransferLaw, with the required clocks/support continuing; a genuinely abstract retirement uses its declared service. For finitely many independent grouped inputs, all such retained arrivals coexist; the joint guard and complete transfer equations yield indivisible physical commitment, or the separately qualified abstract-unit service applies. This proves the resource/delivery substitution used by K.CT.Standard, not the separate source-witness capacity lift, arbitrary early-capture correctness or the narrower canonical interleaving refinement. For simultaneous replacement, the old-state selection rule preserves the same invariant: each moving item advances one location, any replacement arrives behind it, and no newly arrived item advances again on that edge. Unique ownership and total inventory are preserved by internal moves; only the selected outer Push and Pop change unmatched inventory. Hence the isolated-delivery argument and guaranteed floor apply to either declared internal acceptance option. This resource/delivery argument uses item identity, ownership, occupancy, order and the stated operational/service laws, not physical retention of the original payload. It therefore applies equally under the retained operational clauses of Q.CT.LogicalPayload; that profile’s separate logical-payload and legal-later-use obligations remain required. The independently retained-arrival argument is not used for coupled paths. Their matched-movement and bubble/drainage laws are discharged by K.CT.CoupledDeliveryLemma, with both target admissions included in completed closure. For staggered physical reads, K.CT.StaggeredDeliveryLemma uses the early-owner and late-head matching invariant instead. None of these delivery arguments requires a separable physical representation when Q.CT.PairedRepresentation establishes the unchanged logical inventory and operational laws. The placement/request premise is established independently of the target arrival concluded here, as required by B.BoundaryBinding.

Q.CT.CoupledInputStaging (equal-length flushable positive pair). This is a separate forwarding alternative for two positive input paths whose process-facing Pops form one declared same-owner, same-site atomic pair. Each leg retains Q.CT.InputStaging's initially empty exclusive ingress, finite one-item storage, per-channel capacity/credit accounting, old-state producer acceptance, endpoint order and retention laws. Both chains have the same length m>=1 and aligned occupancy and iteration ordinals. Ingress arrivals remain independent; a hidden joint ingress move takes the two corresponding

old heads into the first paired location. It is not the process-facing Pop or a source-iteration launch. Different values may use qualified separate representations or the joined-cache law Q.CT.PairedRepresentation. Joint representation preserves the two logical owners and does not introduce a fungible capacity pool. Shared credits or fungible cross-channel storage pools, partial in-flight pairs, staggered reads, unequal coupled paths, affecting reset, epoch gates, cancellation and rendezvous legs are not admitted by this component.
Paired ownership and clock law. Every occupied location owns (j,j) for one iteration ordinal; both members move together and cannot overtake, split or be reassigned. A paired move reads the old origin representation of both owners, either separate values or their qualified combined cache, and may enter an old-empty paired destination or, under simultaneous replacement, one vacated by a selected downstream departure on that edge. Identical per-location move selections apply to both legs. Each item crosses at most one storage location boundary; neither a new ingress arrival nor a new terminal arrival is consumed on that same edge. All DUT and modeled testbench sequential processes read the common old state and qualified selected signals and install one combined next state. The replacement selection graph is downstream-to-upstream and acyclic, with no recursive process-facing retirement predicate. Producer admission still uses old ingress space and old exclusive credit, independently on each leg.
Flushable movement and service. An occupied downstream pair shall not wait for a younger iteration's read, either producer's next item, or upstream readiness. If a younger ingress read is unavailable, it leaves a bubble while older eligible work drains. A non-compressing controller may hold work behind incomplete older downstream work; it may not retain that hold after the older work has completed and exited. For a legally pending, permanently isolated target, each move into an empty destination has persistent eligibility under either a true forwarding guard or a declared completed-downstream guard that is then continuously true. The first move needs both matching old heads; later moves do not need new ingress traffic or the target's own retirement. Every continuously eligible paired move receives joint internal service, even with no subsequent read. A more general finite-response forwarding guard requires its explicit extension proof; this paired law does not inherit every independent-path guard. Unconditional global-stall behavior fails this applicability condition. The inherited noncircular placement/request qualification applies to the terminal pair. Earlier paired forwarding is permitted; this isolated-delivery proof does not derive terminal request presence from its own arrival conclusion.
Retirement and preparation. The process-facing pair predicate is the conjunction of both old terminal resources, persistent legally issued member requests and the declared joint retirement guard. Both members retire as one unit. Preparation of both requests in the same L1 window before any prohibiting Core.16 activation, no dependence on unresolved sibling returns, and the actual E3/Core.RegSeq issue/use constraints remain required. Requests and held owners persist. Neither movement into storage nor its physical private computation grants source preparation or iteration-launch permission. Group service and joint internal-move service are distinct obligations.
Reference and local realization. A selected source realization may retain the permissive independent internal move law on the two unchanged logical-value paths while retaining the same atomic process-facing pair. Its bound producer and process-facing readiness predicates must match the actual old-state ones; neither may inspect an unselected internal move's permission. Every selected paired implementation move is then a legal simultaneous selection in that source. Realize the process-facing pair as one atomic action, then the selected hidden moves downstream-to-upstream within the lawful L3 boundary segment before the L4 update. That ordering witnesses one simultaneous edge: a created vacancy cannot authorize an originally unselected move, and all operand owners remain old owners. Storage movements are lawful already-admitted in-flight drain, not invisible computation. Complete local preparation, settling, frame, projection, generator and fact-carrier evidence remains required; this clause does not expand ELM by fiat. With Q.CT.PairedRepresentation, the reference still holds

unchanged logical items separately; the physical joint cache is related to their two causal values. Its computation does not create a reference fact or alter the selected readiness equations. This permissive source choice applies only to genuinely abstract hidden-movement actions. If a path boundary is declared as an actual ready/valid transfer interface, its eligible sampled overlap must commit under Core.TransferLaw and the local realization must include it. Hiding a concrete interface requires a separate realization proof preserving all its effects and the matched readiness; it is not obtained by calling the move unselected.
Coupled invariant and consumer. Empty initialization gives equal stage-owner arrays. Independent admissions leave them unchanged; equal prior joint removals make the next two queue-head ordinals equal. Joint ingress, matched moves and joint retirement preserve equal occupancy and ordinals by induction over old-state edges, including replacement. Per-leg inventory is unchanged by movement. Internal moves add no outer completion facts or order generators; the declared complete order/atomicity inventory must still pass the two-way J translation. K.CT.CoupledDeliveryLemma proves delivery from this invariant, completed prerequisites, placement and the movement/service laws. Q.CT.InputStagingProgress remains the independent-delivery lemma and is not applied separately to these coupled legs.
Q.CT.StaggeredInputStaging (fixed-offset physical reads and terminal pair). This separate positive-input alternative retains one same-owner, same-site process-facing Pop atom while its physical captures are staggered. Fix m>=2 and 1<=s<m. A has m initially empty one-item locations numbered 0,...,m−1 after an exclusive ingress FIFO of depth F_a>0. B has an exclusive ingress of depth F_b>0 and m−s locations aligned with A positions s,...,m−1. A moves alone through positions 0,...,s−1; the move into s takes the old A occupant and old B head together. Thereafter the pair moves together to one terminal retirement. Per-channel nominal capacities are F_a+m and F_b+(m−s); choose floors 1<=g_l<=F_l. This family has no extra credit cap or shared-capacity pool. It does not qualify staggered source-facing retirements, variable offsets, rendezvous, affecting resets, epoch gates, side exits, cancellation or data-dependent routing. Q.CT.CoupledInputStaging retains its own equal-length scope.
Staggered ownership and clock law. Every A-only prefix occupant has one fixed admission ordinal; every suffix occupant owns the corresponding pair (j,j). On each channel, occupied locations from terminal to ingress followed by that ingress queue enumerate the unmatched owners in FIFO order. Let r count terminal retirements and t count suffix occupants. The suffix owners are r,...,r+t−1; an A owner at s−1 and a nonempty B head therefore both have ordinal r+t. A late read must attach exactly that head to that A occurrence. Empty initialization, separate admissions, ordered A-only moves, matched suffix moves and joint retirement preserve this invariant by induction. Replacement moves use incoming old owners, never a new same-edge admission or the departing destination's value. Each item crosses at most one location per edge. All DUT/TB updates read the common old state and install next state together; old-full ingress removal does not license a producer Push. Here r is a count from the empty-channel base and owner ordinals are zero-based as in B.CT.EndpointOrder: the first unmatched owner at r=0 is 0. The empty suffix t=0 contributes no owners. Configuration transactions do not reset r or channel ownership.
Staggered flushability and service. Missing B may hold its own A occurrence at the late-read site and younger work behind it. It shall not freeze older occupied suffix locations downstream merely because that younger read is unavailable. Missing younger A supply likewise leaves bubbles rather than imposing a global stall. Holds behind incomplete older work may persist only until that work completes and exits. With a legally pending and permanently isolated target, each next old-empty move has persistent eligibility under a true or continuously true completed-downstream forwarding guard. Early A moves receive internal service; the late A/B read and all paired suffix moves receive actual joint internal service. A missing younger read, a new producer item unrelated to the target, or the target's prior retirement cannot be required. More general forward-control guards need their explicit extension proof.

Terminal requests, reservation, preparation, continuing clocks and any required pre-grant or abstract/internal service retain the Q.CT.LogicalPair/B.CT obligations. An actual terminal joint handshake commits indivisibly under Core.TransferLaw; service is not a second choice after that overlap. The inherited noncircular placement/request qualification applies to the terminal pair; permissive early movement alone does not prove terminal request presence.
Staggered reference and safety. The selected source may use permissive independent hidden moves on logical paths of lengths m and m−s, with the same terminal Pop atom, actual old-state ingress tests and matching process-facing readiness. A physical edge selects B's move vector exactly as A's suffix vector; it is a permitted selection in that reference. Realize the selected terminal atom and hidden moves within the lawful L3 segment, downstream-to-upstream on each path before L4; intermediate vacating cannot enable an originally unselected move. Early A storage ownership is not source-call issue or source-iteration launch. The matched late read emits no additional completion fact or canonical order generator. Complete source preparation, observation, two-way order translation and exact fact-carrier evidence remain required. When physical values combine at the late read, Q.CT.PairedRepresentation supplies the joint-cache invariant without changing source readiness or logical payloads. This permissive source choice applies only to genuinely abstract hidden-movement actions. If a path boundary is declared as an actual ready/valid transfer interface, its eligible sampled overlap must commit under Core.TransferLaw and the local realization must include it. Hiding a concrete interface requires a separate realization proof preserving all its effects and the matched readiness; it is not obtained by calling the move unselected.
Staggered completion consumer. K.CT.StaggeredDeliveryLemma proves delivery from the preceding ownership, placement, preparation and service obligations. The equal-length paired-arrival invariant is not applied to the A-only prefix. A legal workload witness still uses the selected reference and declared source atom, with mandatory floor-based closure for the standard route and any separately proved source capacity lift. A missing producer admission remains a predecessor obligation; this component does not rule out cycles through producers by local topology. Representation and operational safety may be used by J without assuming completion service; no production-family conformance follows from the component name. K.CT.LazySerial does not replace these staggered floor obligations: this component is outside that route without its own witness restriction and progress theorem. Under K.CT.LazyComposition this component may remain unselected with its existing floor, closure and delivery proof while other independent channels use demand support. Its membership in an atomic demand batch does not satisfy LazyEligibility.
Q.CT.LogicalPayload (computed singleton representation and explicitly qualified pairs). This qualification adds Core.LogicalPopPayload to the ordered positive input staging laws. The base case uses blocking singleton Pops and Pushes with a source-aligned no-wait iteration/control structure. Q.CT.LogicalPair is the only named atomic extension. The restriction to item-only pure finite computation applies to the staged functions f_i in the representation law below, not to every subsequent process-local computation. Later legal source computation may read or update loop-carried process state only when the selected J local evidence separately establishes the state/operand relation, ordering, frames and actual result equality, and completion additionally discharges the resulting provenance/agreement and preparation obligations. This representation qualification alone proves neither a stateful kernel nor speculative use of loop state. The component has no NB or signal/direct-input observations, data-dependent routing, fungible shared-capacity pools, side exits, affecting reset, epoch withdrawal, cancellation or rendezvous group; an atomic group is admitted only under Q.CT.LogicalPair. Its logical raw input item is unchanged. J may use the representation and operational clauses independently of completion; invoking Q.CTQualified and standard completion additionally requires the existing forwarding, preparation, exposure and service obligations. For joined positions, Q.CT.PairedRepresentation replaces per-item physical equations by pure pair equations K_i(x,y); it adds

no pre-Pop control decision or loop-state computation. Configuration scope clarification. The exclusion of signal/direct-input observations is local to this representation component; it is not a ban on separately qualified configuration use elsewhere in the enclosing system. The singleton clause's separately proved later source computation retains its stated scope. A configuration-dependent staged map f_i^θ, joined map K_i^θ, or LogicalPair expression S_θ is not automatically qualified by J.StableReadFactCorr or J.ConfigRecordFactCorr. Such a use requires an explicit parameterized representation/composition proof within the selected J route, and Q.CT.Extension for completion where it extends these operational clauses. Bind the genuine initial root or governing record, prove the representation equations and legal source use uniformly over the admitted parameters, preserve live operands and owner-to-record attribution across physical sampling and updates, and establish shared-state/frame and control compatibility. A changing configuration must not rebind an in-flight item to a later record. The fact adapter supplies only its stated full-F correspondence; it does not discharge these representation, readiness, delivery or service cases. The named item-only and pair-only equations remain unchanged absent that additional qualification.

Representation law. Fix a finite chain of pure total functions f_0,...,f_(m−1), with declared arithmetic and no hidden state, timing or side effects, and write G_i=f_i∘...∘f_0. A physical ingress item has its actual admitted value x. If logical owner j occupies internal location i, that physical location represents G_i(x_j), where x_j was causally recorded at j's actual admission. A first-stage selected move computes f_0 from the old ingress word; a later selected move computes f_i from the old upstream representation. Holds preserve the represented owner/value pair. Simultaneous replacement uses the arriving old owner's operand, never the departing destination's value. Physical storage need not retain x_j or any invertible encoding of it. No ordinal need be physically stored when the causal ownership relation proves its attribution. This separate-value representation law applies before joining and on independent paths. At joined positions expressly using Q.CT.PairedRepresentation, its joint-cache equations replace the separate physical values while both logical items remain unchanged.

Operational separation. The occupancy, capacity/credit, request, selected-move and process-facing retirement laws remain the actually bound laws of the selected Q.CT.InputStaging, Q.CT.CoupledInputStaging or Q.CT.StaggeredInputStaging alternative, together with the applicable B.CT clauses. Each alternative retains its own topology, accounting, guard and service restrictions. All DUT and modeled testbench sequential updates read the common old state and qualified current-cycle signals and install next state together. Each item advances at most one internal location on an edge. Representation computation supplies no additional cache-valid/readiness dependency, token, storage capacity or source launch. If an implementation has a downstream Pop gate, that gate must already be present in the chosen source/RTL handshake contract; an ungated source cannot be justified by logical payload attribution. A separately required computational resource or latency guard needs its own complete qualification. For Q.CT.LogicalPair, issue reserves an empty process-owned result position U and the invariant pending pair implies U empty persists until retirement. Within this reserved-empty profile U adds no retirement-readiness conjunct; the reservation remains a genuine issue prerequisite. If a controller has a U or downstream gate not implied by that invariant, it requires the actual matching handshake law and its own qualification. The same operational separation applies when the joined storage uses Q.CT.PairedRepresentation; a computation-valid or resource guard cannot be hidden in a logical value label.

Legal later use. On actual retirement of j, the logical source Pop returns x_j while the physical terminal may hold G_(m−1)(x_j). A physical result may be copied into held process state on that edge; for Q.CT.LogicalPair, the physical retirement may privately combine two old terminal results or copy/transform the old joined cache into the reserved process result position on that edge, but its source-attributed evaluation/use and any dependent Issue occur only at a legal later Horizon under Core.RegSeq and Core.16. The conformance relation binds the result to that Pop's actual returned

location and occurrence, proves equality with the source expression there, and respects the preparation and field-sensitive footprint constraints. Private representation work before retirement leaves abstract source preparation/lifecycle unchanged; the accompanying token move is explicit channel progress with its own lawful in-flight action, not invisible computation. For the pair, equality is required against the source expression on both actual logical returns, with declared machine arithmetic. Process storage and any same-edge process advance must preserve that result and cannot authorize same-cycle dependent source Issue.
Representation proof. Empty initialization establishes the relation. Induct over every actual edge. An ingress admission records its actual word. A move into location 0 installs f_0(x_j); a move into location i installs f_i(G_(i−1)(x_j))=G_i(x_j) from the old upstream state. Holding and old-owner departure/replacement preserve unique ownership and the corresponding equation. Hence terminal retirement has the bound x_j and physical result G_(m−1)(x_j). At its legal later use, source evaluation on the recorded return gives that same result. This proof requires no inverse function and remains valid for repeated inputs and non-injective functions. It does not justify the source preparation step unless that step separately satisfies its control and activation laws. For Q.CT.LogicalPair, apply this induction separately to both paths and then the declared combination equation. Old-terminal operands give the correct private pair result; its legal later evaluation consumes the two actual return locations together. Under Q.CT.PairedRepresentation, replace the separate physical-value induction after joining by its joint K_i equation; causal values and legal later source use remain the same.
Consumer obligations. J.LogicalPayloadRefinement supplies the observer and payload cases; the same reference's complete local source, resource, settling, environment, order and carrier evidence remains required. In particular, this representation qualification does not enlarge ELM's primitive or simultaneous-replacement scope and does not itself provide a source-witness capacity lift. For completion, Q.CT.InputStagingProgress applies to the independent operational law; K.CT.CoupledDeliveryLemma applies to Q.CT.CoupledInputStaging; K.CT.StaggeredDeliveryLemma applies to Q.CT.StaggeredInputStaging. Each requires its own occupancy, order, preparation, placement, forwarding and B.CT/Q.CT service premises. The witness and its floor-g closure remain in the same logical source model. Q.CT.LogicalPair is the restricted transformed positive pair. Other transformed groups, mixed B(0), NB and excluded cases require an explicit extension; the ordinary group clause supplies none by inheritance. For the independent operational alternative only, K.CT.LazySerial may replace designated standard floor edges under its complete demand, source-legality, representation and fact qualifications. The coupled and staggered alternatives acquire no such permission by representation equivalence.
Q.CT.LogicalPair (restricted computed positive two-Pop representation). The source iteration consists of one declared atomic pair Pop_a(j), Pop_b(j), followed by a declared pure finite source expression S(x_j,y_j), including H(G_a(x_j),G_b(y_j)) as a special case, and a singleton output Push. The inputs are distinct positive channels, with separate logical item accounting and any credits allowed by the selected operational family. Both Pop descriptors are legally preparable without either unresolved return, and both requests are issued in one L1 window before activation can forbid later issue. The two process-facing members share one owner site and commit together with corresponding iteration ordinals. The family selects two independently forwarding Q.CT.InputStaging paths, which may differ in length, Q.CT.CoupledInputStaging's equal-length flushable paths, or Q.CT.StaggeredInputStaging's fixed-offset physical reads with one terminal logical pair. It has no data-dependent routing, pre-Pop control decision, loop-carried computation, NB/signal/direct-input observation, side exit, affecting reset, epoch withdrawal, fungible shared-capacity pools or mixed B(0) group. Larger or other transformed groups need an explicit extension. The configuration scope clarification in Q.CT.LogicalPayload applies here. Separately qualified configuration elsewhere in the enclosing system does not by itself violate this component's observation exclusion, but replacing the declared S(x_j,y_j) by S_θ or adding configuration-

dependent pair control needs explicit representation/composition and applicable completion-extension evidence. A root or record fact alone does not waive this profile's computation or control restrictions.
Pair representation and reserved result. Apply Core.LogicalPopPayload separately to the two admission ledgers. Physical state satisfies either the Q.CT.LogicalPayload per-item equations or, after an actual qualified join, Q.CT.PairedRepresentation. Each logical member returns its own actually admitted word in channel ordinal order; neither internal endpoint is physically sampled by the consumed observation interface. The pair is issued only when process-owned result position U is free, and pending pair implies U empty until retirement. Within this reserved-empty profile U reservation adds no conjunct to pending-pair retirement readiness; it still constrains issue timing. Older output state cannot be overwritten. Physical joined stages may already compute on both inputs under Q.CT.PairedRepresentation. Retirement copies or lawfully transforms the old terminal representation into U under the declared process advance; its result equals the later source expression S on both actual returns. No original operand need remain separately recoverable. Both physical admissions and any physically observed downstream use retain their actual sampled values. Logical attribution cannot repair an incorrect output, suppress a physical effect or change readiness.
Result-storage timing and rate scope. If a single shared U is reserved at L1 issue and held through Pair(j) retirement at edge t until Push(j) commits, or until a later-edge safe advance releases it, U is still occupied in the L1 window before that releasing edge. Pair(j+1) cannot issue in that window. In particular, release at edge t+1 cannot enable Pair(j+1) in cycle t+1's pre-settle window; this single-slot profile has sustained pair initiation interval at least two cycles, with other stalls potentially increasing it. Merely splitting storage into U3 and U4 with transfer U3 to U4 at edge t+1 does not remove this restriction. The conditional shared-window permission below does not override empty-at-issue reservation. Appendix Q's independent InputStaging II=1 example does not establish II=1 for this single-slot LogicalPair realization.
Q.CT.PairedRepresentation (joint cache after a qualified join). In the Q.CT.LogicalPair family, a joined position may represent both admitted values by one cache K_i(x_j,y_j), possibly a tuple of partial results, rather than separate G_a and G_b values. The join must be the actual paired ingress of Q.CT.CoupledInputStaging or the matched late read of Q.CT.StaggeredInputStaging; independent paths do not gain an earlier joint computation by this clause. Before a staggered join, the A-only prefix retains its per-item representation. The joined allocation owns two distinct logical items (j,j), each charged once to its own channel until terminal retirement. Sharing their physical representation is not logical merging, duplicate capacity, an interchangeable storage pool or permission to admit unrelated owners into that allocation.
Actual computation equations. Bind pure total physical transition maps and their exact arithmetic. At the join, prove T_join(x,y)=K_0(x,y) for joint ingress, or T_join(G_pre(x),y)=K_s(x,y) for staggered capture. At every later joined stage prove T_i(K_(i−1)(x,y))=K_i(x,y). At terminal retirement prove T_ret(K_last(x,y))=S(x,y), where S is the declared later source expression. A copy is an allowed T_ret. The equations hold over the entire admitted value/state domain, not only a selected workload. Every operand is read from the actual old physical state; ledger values cannot supply a missing physical operand. The maps may compute cross-input products or other pure intermediate functions without retaining either raw value or an invertible encoding. If a later source use cannot be justified from the actual physical result, this qualification fails.
Joint representation induction and boundary. Causal admissions fix x_j and y_j before joining. The join equation establishes K for their matched owners; holds preserve it, and selected old-origin moves establish the next equation. Simultaneous replacement installs the incoming owner's representation, not a transformation of the outgoing cache. Thus the terminal result satisfies S on the two actual logical returns. No cache computation changes allocation, readiness, requests, source preparation, control or observation timing beyond its separately bound operational law. The private computation may precede

the logical Pops, but source-attributed evaluation and dependent Issue wait until the legal later Horizon under Core.RegSeq/Core.16. Both logical returns remain one indivisible two-key packet; physical observations retain their actual sampled values. J.LogicalPayloadRefinement consumes this invariant together with its complete local, order and carrier cases. Completion still needs the selected delivery lemma and service; joint representation alone supplies neither.
Pair source and fact proof. Per-channel ownership fixes x_j and y_j causally. The separate or joint representation induction preserves their owner pair and proves the held result equals S(x_j,y_j). The actual group edge supplies one source atom returning both values at distinct call keys. Only at a legal later Horizon may source evaluation and dependent Issue use those returns. E3, Core.ReachPrep/Core.RegSeq and the Core.16 frontier restrictions govern all intervening preparation. A subsequent output and next pair may share a legal pre-settle issue window only when those laws permit it; an already active frontier cannot be bypassed. J.LogicalPayloadRefinement supplies these payload/observer cases together with complete remaining local, generator, CF and carrier evidence. Two member facts are emitted once in the same indivisible packet. The source and implementation group partitions agree; internal storage/representation steps create neither extra completion facts nor incidental serialization generators. Likewise, using a safety property from an ungrouped source needs an explicit observation/order/atomicity-preserving inclusion proof, not merely simultaneous placement of two Pops. J may use the representation, operational safety and local source/fact clauses independently of completion, with the complete remaining constructive evidence. The delivery, witness, closure and service obligations in the following Pair completion and witness clause apply only when completion is consumed; they are not premises of this safety-only discharge.
II=1-capable result-storage alternatives require explicit realization evidence. An exclusively reserved slot assignment may use two slots U0 and U1, with iteration j reserving U[j mod 2] only when that slot is already free in its L1 window and retaining exclusive ownership through pair retirement and safe output consumption/advance. For example, if Push(j−1) releases its slot at edge t while Pair(j) fills the other slot, Pair(j+1) can find its own slot free in the next L1 window. Prove the slot ownership, reuse, output order and backpressure invariants and their source/control and resource correspondence; two slots alone do not guarantee a rate. Alternatively, allowing issue against an occupied single U and retirement only when old_empty(U) or depart(U) holds is a different handshake profile: downstream commitment/advance is an explicit retirement-readiness conjunct. It requires the matching source law, B.CT.Guard qualification, strictly preceding downstream-completion support in K.CT.Closure, and complete local/refinement evidence under Q.CT.Extension. It cannot cite the reserved-empty invariant or its no-extra-conjunct conclusions. In either case, a sustained II=1 claim separately establishes legal preparation, supply, sink, slot release, advance and service timing; the present completion theorem guarantees no rate.
Pair completion and witness. Q.CT.InputStagingProgress supplies independent arrivals, K.CT.CoupledDeliveryLemma equal-length paired arrival, and K.CT.StaggeredDeliveryLemma fixed-offset staggered arrival. All require legal preparation, completed predecessor closure and their actual service assumptions. A source witness uses the same declared atomic pair under K.CT.GroupSerialWitness or another proved legal schedule; an individually serial Pop log is not automatically such a witness. Direct simulation of the elaborated reference remains permitted with the required floor-based W closure for K.CT.Standard. A plain-storage witness requires its separate K.CT.Capacity source lift at guaranteed floors, not nominal aggregate depth. Representation and the pair delivery proof provide neither that lift nor agreement for excluded timing-sensitive features. The independent-input alternative may instead use K.CT.LazySerial with qualified group-serial issue and member-demand support. Coupled and staggered storage remain outside that route unless separately supplied with their own restriction and progress theorem; a permissive independent reference move does not establish their joint forwarding. Under K.CT.LazyComposition this component may remain unselected with its existing floor, closure and

delivery proof while other independent channels use demand support. Its membership in an atomic demand batch does not satisfy LazyEligibility.
Q.CT.Groups. A stateless positive fork, join or same-stage atomic bundle qualifies when it has the fixed group partition and conjunctive resource law of B.CT.Joint, with distinct positive channels and exact projection of all member commits. Positive inputs may be retained in attributed slots until the whole group commits; partial groups retain each completed member separately instead. For a direct B(0) leg, close the entire component under both matching endpoints and atomic-stage membership. Reject multiple sites of the same owner, positive same-component producer/consumer dependencies, unresolved returned-data cycles and strict endpoint/control cycles. The combined combinational network must meet the actual settling qualification; logical grouping does not establish acyclicity of an RTL ready/valid network. Under these restrictions, captured positive inputs persist, no B(0) input can retire alone, and the group requires only its explicitly declared resources. K.CT.JointLemma proves joint progress from these operational facts. Pure positive groups may consume Q.CT.InputStaging paths with disjointness, independent forwarding and retained arrivals, using Q.CT.InputStagingProgress. A positive pair may instead use Q.CT.CoupledInputStaging and K.CT.CoupledDeliveryLemma, with matched ordinals, explicit bubble drainage and joint internal service. A fixed-offset physical-read pair may instead use Q.CT.StaggeredInputStaging and K.CT.StaggeredDeliveryLemma; it still has one terminal source-facing atom. These additions do not widen the mixed B(0) profile. A computed pair must additionally satisfy Q.CT.LogicalPair; other atomic groups do not inherit its physical-representation exception.
Q.CT.Epoch. Bind each gate to its controlling unary/paired synchronization and the operation occurrences whose acceptance it governs. The explicit state and projections obey B.CT.Sync/Restore. All blocking work required to reach the boundary is a predecessor of that sync; the sync is a predecessor of gated later work, and every admitted execution shall prevent a later closing boundary or equivalent availability withdrawal from preceding retirement of the protected target, including while its aligned preparation or request assertion is still pending. This is an operational conformance requirement, not a consequence of ordering nodes in W. Core.SyncFence may discharge it when the target is a required predecessor of the closing synchronization on a participating owner; another realization needs its explicit invariant. Stored items crossing a boundary retain their global matching ordinals, epoch attribution and accounted location; no reset or empty-queue inference is made at sync. The component proves its capacity/credit law in each active region, including already resident inventory. A fully drained balanced boundary is a sufficient specialization, not a requirement of this clause. If restoration or boundary reach needs data behind its own gate, the corresponding dependency must be represented and the cyclic witness is rejected. K.CT.SyncLemma proves the progress consequence; unsupported cross-boundary capacity sharing requires an additional invariant.
Q.CT.Retry. A nonblocking retry may implement one logical transfer obligation when its identity and descriptor remain fixed until success, failure makes no externally relevant update other than retry-local state, and failed attempts neither consume credit nor retire the obligation. Its own item/space support and endpoint order satisfy the corresponding B.CT laws. Its persistent logical obligation, rather than an individual unsuccessful attempt, supplies request support for any finite-response guard realization. Such a realization must retain its progress across failed attempts; a retry cannot reset or replenish the measure. Once resources and the qualified guard are persistently available, a retry attempt is continuously eligible as a local action and receives eventual service; an attempt in that state succeeds under Core.NBDecision and the declared channel equations. Its payload and control after success depend on the logical result, not elapsed time or the number of failed attempts. The safety correspondence justifies the projection of failed attempts and matches the successful logical operation; this clause does not assert agreement of timing-dependent raw NB observations. Proof: persistent own resources and the retained logical obligation support finite guard progress, if required; afterward the complete support lasts until the eventually selected eligible attempt, whose success completes exactly

the logical transfer. Thus K.CT.Exposure/Service may be discharged for that logical unit without pretending that each unsuccessful poll is a persistent blocking request. General outcome-dependent nonblocking control is outside this clause.
Q.CT.Composition. Component laws are universal over their declared interface actions. Compose only components with compatible ownership, state invariants, units, order and service contracts. Every cross-component ready/valid, storage, credit, synchronization and value dependency is included in the combined settled network and witness closure; internal erased transfers retain their progress-relevant accounting and attribution. A global unit is the full closure of declared atomicity, not a set of independently fair endpoints. Local invariant preservation under each admitted external action and the exact simultaneous update law establish the product invariant by induction on edges. Fairness is supplied for the product's actual required opportunities; composition of individually fair components is not assumed to create joint service. With the complete standard predecessor closure and its other premises, K.CT.Standard applies directly to this product. If qualified extra-slack floor releases are replaced by demand support, use K.CT.LazyComposition with one complete assembled graph and its acyclicity evidence; this does not weaken standard closure or the component invariants.
Q.CT.Extension. A controller or elaboration outside the standard clauses may be used only with a stated replacement operational law and a proof of the exact K.CT.Resources, Guard, preparation, exposure and service conclusions it consumes. The proof quantifies over the complete admitted continuation domain and preserves the same instance, accounting, boundary and source-legality contracts. It may not assume required-workload completion to establish exposure, restrict the domain to successful executions, or hide a missing condition under B-faithfulness. A new profile requires an explicit theorem and applicability statement; unproved combinations remain outside the standard discharge.
Q.CT.InputStaging, Q.CT.CoupledInputStaging and Q.CT.StaggeredInputStaging are the three named sufficient early-input alternatives, with Q.CT.InputStagingProgress, K.CT.CoupledDeliveryLemma and K.CT.StaggeredDeliveryLemma respectively. Early capture outside the applicable clauses of those three alternatives, and explicit output transport beyond another stated component, still need their own finite delivery/ownership and boundary proofs; arbitrary pipeline storage is not qualified by the example. A delayed guard satisfying B.CT.Guard is admitted for direct synchronous progress; that fact alone does not extend the canonical interleaving serialization theorem. Q.CT.LogicalPayload and Q.CT.LogicalPair are the named singleton and restricted positive-pair exceptions for physical representation of unchanged logical items. Q.CT.CoupledInputStaging is the equal-length flushable alternative to independent forwarding, with K.CT.CoupledDeliveryLemma. No exception is supplied at physically sampled observations. Q.CT.StaggeredInputStaging is the separate fixed-offset physical-read family with K.CT.StaggeredDeliveryLemma, and Q.CT.PairedRepresentation permits a combined cache after either qualified join. Staggered process-facing retirements, variable offsets, other unequal coupled organizations, other transformed groups, shared-capacity pools and pre-Pop control decisions remain separately qualified. Pre-request data-driven staging alternative. If the target reaches its selected site and asserts its request only by traveling with the stored item, provide a separate delivery/placement proof over every admitted continuation. Bind an actual pre-request forwarding/drainage law that, once its older-work and resource prerequisites hold, gives serviced progress into persistently empty downstream destinations without requiring the target’s terminal request or arrival. Prove destination ownership/reservation, no overtaking, retention, compatibility with older-work drainage, and a finite distance/rank argument, jointly for coupled or staggered paths where applicable. Then prove legal terminal placement, preparation and persistent request assertion upon or after arrival under E3, Core.RegSeq, Core.IC and Core.16, followed by resource/guard service and retirement. Hidden channel movement does not itself launch a source iteration or issue its Pop. This law is unconditional with respect to terminal request/arrival, not with respect to genuine backpressure, occupied destinations or source legality. It must account for those prerequisites without cyclic dependence on the target’s

retirement. Merely changing the forwarding trigger to stage emptiness supplies no qualification. This paragraph specifies the required extension evidence; it does not add a proved fourth standard staging profile.

Appendix R – Compact Derived Formal Core and Mechanization View

Status and authority. Appendix R is derived and non-normative. It summarizes retained dependencies and audit questions; every definition, hypothesis and theorem it names has its authoritative home in Core or the other normative sections. Removing R must leave the formal development well-defined and unchanged. Navigational pointers to this summary are permitted, but an earlier normative clause shall not consume an R-defined condition or use this appendix to discharge a premise. Such a dependency is an architectural regression and must be moved to its proper normative owner.

R.1 Retained proof structure

Operational semantics fixes legal source and implementation carriers, choices, phases, requests and primitive actions independently of correspondence. Constructive local evidence proves legal source extension at every actual edge; the named carrier lift gives selected-history coverage. Source-property transfer and full internal-fact correspondence consume that coverage with their additional premises.

R.2 Completion dependency chain

Standard branch. Core.EvidenceHdr binds the instance. J.ConstructiveFactCorr supplies J.InternalFactCorr over the complete consumed F domain; Core.FinitePrefixAgreement and J.InternalFactAgreement supply conditional identity/value agreement, including the workload witness. K.CT.Witness/Closure bind actual witness members and the finite prerequisite graph, including floor-g space edges and alignment predecessors. B.OF.Control/B.CT.Control bind source-aligned control and finite preparation. K.CT.Alignment uses the agreed facts before K.CT.Standard derives placement, resources, residual/joint enabling and gate restoration from the applicable B.OF/B.CT/Q.CT laws. K.CT.Service binds boundary and internal progress; K.CT.Completion concludes acceptance of R. No conclusion in this chain supplies its own premise. The boundary record separates physical overlap from source-facing Issue under E3/Core.16. Immediate or finite-response guard realization precedes persistent enabling; Q.CT.InputStagingProgress supplies independent early-input delivery and K.CT.CoupledDeliveryLemma supplies the Q.CT.CoupledInputStaging alternative; K.CT.StaggeredDeliveryLemma supplies the Q.CT.StaggeredInputStaging alternative, while other early-capture/output-transport extensions retain their explicit qualification. The Witness/Closure record includes K.CT.CtrlSupport, and Alignment covers its negative membership and entry decisions as well as positive target descriptors. The full-fact interface may be discharged by J.StableReadFactCorr or J.ConfigRecordFactCorr within their stated scopes; neither specialization supplies agreement or progress by itself. For physical interfaces, guards and grants feed the complete sampled equation and the transfer is mandatory. A finite eventual grant can satisfy Exposure by actual completion; no continuous handshake antecedent before that grant is needed. All service-dependent progress conclusions in this chain use K.CT.UntilRScope. Facts, conditional agreement, Alignment and service-independent resource invariants keep their stated domains.

Lazy branches. The K.CT.LazySerial family uses LazyInventory/Slack. Within that family, K.CT.LazySerialRestricted uses the applicable finite LazyClosure and LazyRank/LazyMixedRank construction; mixed applications instead supply complete assembled-graph acyclicity evidence. A restricted all-singleton protected extension returns to Standard through LazyExtendedStandard; a demand certificate uses RecordedDemand, DemandPlacement or BatchPlacement, and DemandPushResource before invoking Completion. Facts retains its full implementation-prefix domain

and Agreement covers the actual witness. Auxiliary nodes are removed before the completed-unit induction. K.CT.LazyComposition combines selected lazy resource cases with other qualified unit proofs in one complete finite graph; its checked acyclicity alternative is distinct from the restricted LazyRank/LazyMixedRank guarantee. The general mixed-component result is Theorem K.CT.LazyComposition; Corollary K.CT.LazySerialRestricted supplies its restricted rank-derived specialization. Ordinary selected-channel Pushes keep standard floor resources; only qualified extra-slack cases use demand replacement. The informative route table at the head of K.CT.LazySerial summarizes these branches. Floor-based standard closure in the opening chain is not an additional requirement on demand-replaced extra-slack edges.

R.3 Scope and observation

Canonical outer-history safety is distinct from exact internal-fact correspondence. Only an exact full-F lift discharges K.CT.Facts. Clock-edge refinement alone is a safety statement and supplies no service or eventual restoration. An admission-derived internal payload additionally uses Core.LogicalPopPayload and J.LogicalPayloadRefinement; its event value is fixed before projection and before witness agreement. A Q.CT.LogicalPair exports two keyed returns as one atomic packet; its selected source atom and exact full-F carrier must agree. Neither a finer ungrouped safety model nor independent endpoint fairness is silently substituted.

R.4 Dependency map and safety branches

Read the dependency structure by interfaces, not document page order. Core fixes operational semantics, source choices, identity and evidence binding. B and Q describe the applicable pipeline/component realization contracts. J constructs legal source histories and transfers independently established source properties. K consumes exact internal facts, conditional agreement and separately qualified operational progress. L supplies legally realized snooping/witness evidence where selected; M records each consumed obligation and its evidence owner. This is an overview of proof consumption, not a claim that B/Q safety qualification requires K or that L is mandatory for every constructive application.

Local construction branch: the selected process/primitive and anchor contracts supply C0–C5/A0–A6, including legal finite source preparation, complete settled decisions and shared-action realization. J.LocalBoundaryCompose reconciles the common boundary step; J.LocalOriginCanonExt supplies equal extended canonical histories after the required-order evidence. J.ConstructiveEdgeInduction and J.LG-Origin establish origin coverage. For the ordinary class, J.OrdinaryConstructiveProfile, LocalEvidence, Glue and SettlingEvidence are discharged first; the assembled J.OrdinaryRelation and its preservation are derived. Elaborated and combined profiles retain their named additional obligations. For simultaneous-replacement staging, bind Q.CT.InputStagingResidue or its explicitly mapped equivalent C3 proof under the dedicated M.7a trigger, including in safety-only applications.

Exact prefix branch: the qualified BM/AB clock-edge inventory and actual carrier map every admitted finite implementation prefix to the completed history of its base or last realized edge. J.BatchPrefixCoverage and the applicable anchor/reset/configuration/R2C lift export exact selected-history coverage. Origin correspondence alone covers its declared origin domain. Exact outer-history coverage does not automatically yield exact internal-fact correspondence for every finer prefix; the fact-facing consumers retain their separate full-F and carrier obligations.

Property branch: J.SafeTransfer-HistCov or J.SafeTransfer-HistCovOn consumes the proved coverage and an independently supplied source property. A checkpoint domain additionally requires Core.SourceCheckpointProfile and J.SourceCheckpointRepresentative. The selected-source-history option requires J.RefProjCover or J.RefProjCoverOn and the relevant all-finite-ideals property proof.

These source-property options do not replace RTL coverage. UCL0–UCL3 and J.ConstructiveClassSafe lift the selected bindings and proofs uniformly over the admitted class, not over successful test samples.

R.5 Mechanization work packages

A useful decomposition separates: operational source/implementation carriers and reset epochs; primitive action, frame and settling laws; local source realization and order translation; ordinary/elaborated composition; origin coverage through J.LG-Origin and exact all-finite-prefix coverage through the qualified J.BatchPrefixCoverage lift; source-property transfer; full-fact adapters and conditional agreement; and the generic workload induction with its standard and lazy exposure discharges. The lazy work package includes simulator/witness legality, inventory/slack, demand placement and resources, auxiliary-node elimination, protected singleton extension, restricted ranks and mixed composition; use the route table at the head of K.CT.LazySerial and the exact targets in S.2. Each package exports its exact hypotheses and domain. This decomposition adds no Lean module path or completed implementation claim; S gives the target ordering and the normative owners fix every interface.

R.6 Retained-route audit checklist

Check the declared source and implementation domains before constructing evidence. Are choices, environment behavior and carrier cuts fixed independently of success? Does every actual edge have a legal source realization with matching identities, NB snapshots and complete atomic membership? Does the full generator inventory have independent source strictness, projection reflection and prefix stability? Are all relevant reset and environment effects represented? Do the carrier and property proofs cover the same boundary and domain? For replacement-capable staging, check the triggered Q.CT.InputStagingResidue/C3 discharge independently of whether completion is claimed.
For completion, check that F includes required acceptance, W members and consumed predecessor facts; every W member occurs in the legal witness; agreement covers both witness and correspondence prefixes; and B.OF.Control/B.CT.Control justify actual source-aligned reach through K.CT.Alignment. Check owner-path/older-iteration closure, the resource dependencies of the selected route, persistent isolation, indivisible joint exposure, pre-pending gate protection, any actually used capacity lift and actual boundary/internal service. Standard and ordinary lazy Push cases retain their floor releases; only eligible extra-slack cases use proved demand support. For lazy use, check selected-channel and whole-batch eligibility, complete trace compliance, actual W membership, complete demand support and auxiliary-node elimination. Validate restricted rank hypotheses or assembled-graph acyclicity as applicable. A protected singleton extension additionally needs the whole-design continuation window and Facts/Agreement coverage of the witness actually used. Lazy selection at unchanged capacities requires no capacity lift merely for restricting optional moves. A local edge match, outer safety proof or passing witness does not discharge the other checks. Check the completeness of each CtrlSupport(q) inventory and its recursive fact/invariant support; absence of a site from W alone is not evidence that it is untaken.
Structural audit: resolve every live formal reference to its normative owner, distinguish shared definitions from retired-route-only material, and check the Core.ExtensionConsumerCoverage mapping for added cases. Check R against those owners after each normative revision, and verify that removing R introduces no undefined normative name or missing proof premise. Historical revision entries are records of earlier states, not current definitions.

Appendix S – Lean Proof Strategy Document

Authority. This strategy is derived from the rules document's normative definitions and theorem statements. A Lean encoding must expose discrepancies for correction at the normative owner; it shall not silently weaken a premise, narrow a domain, add a scheduling law or redefine an observation to make a proof pass. The targets below are a development order and review inventory, not implemented module names or claims of completed mechanization. Core.ExtensionConsumerCoverage and Core.EvidenceRealization own the cross-cutting extension and extraction requirements.

S.1 Mechanization targets

Formalize the operational carriers and independently defined source legality first. Prove the primitive action/frame, complete-settling and local-preparation contracts C0–C5 and, for anchors, A0–A6. Mechanize J.ConstructiveEdgeInduction and the qualified local-composition results, with the complete unit/generator inventory, prefix closure, restriction stability and independent strictness evidence. Export exact prefix coverage through the qualified BM/AB Batch lift. Mechanize J.SafeTransfer-HistCov and its domain-specific version with independent source-property evidence. These are targets, not claims of completed Lean proofs or supplied import modules.

S.2 Completion targets and remaining qualification

Mechanize J.ConstructiveFactEvidence/J.ConstructiveFactCorr and Core.RegisteredFactPrefix under CF0–CF3 and FB0–FB2 for the complete fact signature. Prove Core.FinitePrefixAgreement and J.InternalFactAgreement with witness-inclusive availability. Formalize B.OF.Control/B.CT.Control as conformance hypotheses, then K.CT.Alignment, ResourceLemma, PlacementLemma, ResidualLemma, JointLemma, SyncLemma and ClockRefinement, and assemble K.CT.Standard as a sufficient exposure discharge for the independently stated finite K.CT.Completion induction. Keep source-aligned reach, permanent isolation after older work exits, actual same-edge semantics, split/coincident retirement, floor capacity, atomic grouping, sync fences, resident storage and delayed restoration explicit. Implementation/controller conformance remains a separate universally quantified proof; finite examples do not discharge it or a stimulus-class claim. Keep Alignment's conditional preparation result independent of PlacementLemma. Add proof cases for B.CT.Guard's finite-response measure, preserved support and joint readiness; do not broaden ClockRefinement by reusing its canonical proof. Formalize Q.CT.InputStaging and Q.CT.InputStagingProgress with isolation-scoped forwarding, floor g<=min(F,C) when credit applies (otherwise g<=F), and independent retained arrivals for pure positive groups. Other early storage still needs its selected extension. Preserve B.BoundaryBinding's source-issue distinction. The input-staging delivery proof consumes PlacementLemma and lawful request assertion before deriving item arrival; PlacementLemma does not assume that arrival. Establish this component lemma before the residual/joint cases that consume it. For InputStaging, distinguish the two internal acceptance options and prove invariant preservation for the selected simultaneous update, with each payload drawn from its old location. Do not apply an internal replacement law across an independently declared primitive boundary or infer sustained II=1 from eventual service. Add Q.CT.CoupledInputStaging and K.CT.CoupledDeliveryLemma as a distinct branch: aligned stage ordinals, joint ingress, no waiting on younger reads, permanent isolation, decreasing pair distance and joint internal service. Treat producer admissions as earlier W obligations, not consequences of consumer demand. For Q.CT.LogicalPair, mechanize the two-return packet and private combination into reserved process state. Add the staggered branch separately: A-only prefix ownership, ordinal matching at the late B read, paired suffix retention and drainage past a blocked younger read. Mechanize Q.CT.PairedRepresentation using the actual join, stage and retirement maps and the source-expression equation, without assuming raw recovery or separable per-input registers. Keep two logical

ownership/accounting records per joined allocation. Mechanize CtrlSupport records separately from W, including some/none alignment, older-iteration and preceding-stage membership coverage, and iteration-entry decisions. Closure must provide strictly preceding historical producers or independent invariant discharges before Alignment is used by PlacementLemma. For replacement staging, instantiate Q.CT.InputStagingResidue with fixed T, frozen old owners/payloads and predecessor-closed residues, retaining explicit E4 ingress tests. Any reusable ELM/ELA extension for atomic process-facing paired-Pop retirements, internal joint moves consuming one old head from each of two ingress FIFOs, or simultaneous-replacement staging shifts is a separate locality theorem work item: specify each primitive transition and supply its complete local contracts, footprints/frame, projection/accounting and composition evidence. The current rules do not assert that extension. Mechanize J.StableReadFactCorr's full-F original/reduced lift and J.ConfigRecordFactCorr's CO1/CO2 delta, frame and exact-prefix cases, including zero-length intervals and governing records used by CtrlSupport. Use epoch-qualified zero-based endpoint ordinals and test the floor boundary k=g. Keep downstream drainage distinct from pre-issued younger completion and prove the stalled-work frame. Audit the placement/request-to-delivery dependency; pre-request token-driven movement remains a separate extension theorem. The general completion induction does not mandate the standard CtrlSupport record schema; any alternative exposure proof explicitly discharges its consumed control/membership dependencies. Reuse of standard Alignment/PlacementLemma retains CtrlSupport. Q.CT.InputStagingResidue's M.7a entry owns its triggered local-realization evidence. Mechanize result storage by the selected realization: single-slot reservation/release and its timing limitation; multi-slot U[j mod 2] ownership, target-slot emptiness, reuse, output-order and backpressure invariants; or the separately qualified occupied-slot/departure-dependent handshake with downstream closure and guard evidence. Keep rate claims separate from eventual completion. For K.CT.LazySerial, mechanize the component-bound witness restriction, one-staged-item invariant, old-state slack classification, finite occurrence closure and rank proof; then the protected singleton source extension and coverage of its enlarged W within the prospectively declared F and attribution. Keep the declaration distinct from the consumed fact inventory; encode genuinely changed signatures through J.InternalFactCorr's new-instance/header and revalidation procedure. Separately generalize Alignment/CtrlSupport to recorded demands, prove DemandPlacement and resource-independent BatchPlacement, and prove the three-case persistent-space lemma over all admitted continuations. Check auxiliary-node elimination and the independent-group certificate, including replacement of extra-slack edges whose groups complete later. Keep pending demands distinct from completed units, source extension protection distinct from RTL service until R, and coupled/staggered/reserved-slot exclusions explicit. The rules text supplies mathematical statements and proofs, not a claim that this mechanization has been completed. Add LazyEligibility, trace-based selection, the unified singleton/group LazyMixedRank and LazyComposition. Prove composition from per-unit Exposure and the common finite acyclic graph; keep graph completeness, exposure validity and acyclicity checking separate. Preserve local exclusions while allowing unselected components elsewhere. Complete the theorem assembly explicitly: prove LazyComposition from per-unit exposure and one complete acyclic graph; derive LazySerialRestricted using its global witness discipline and prescribed rank construction; and prove the restricted all-singleton LazyExtendedStandard return to Standard with the protected extension and original LazyRank. Mixed uses of LazySourceExtension feed LazyComposition with their own graph evidence. The informative route table at the head of K.CT.LazySerial records this hierarchy. Encode K.CT.UntilRScope explicitly in the service-dependent helper signatures and their consumers: assume that the admitted service-qualified implementation execution never accepts the bound R, or use a separately proved equivalent acceptance-or-progress interface. The ten supporting progress results enumerated by that convention must not retain unguarded eventualities under merely until-R service. Keep K.CT.Alignment semantic and conditional; audit and remove any unnecessary service premise rather than narrowing its conclusion automatically.

Keep service-independent resource lemmas free of this guard. Derive the completion conclusion for all qualified executions by the accepting/nonaccepting case split; do not filter the theorem's conclusion or the Facts domain by nonacceptance.

S.3 Proof boundary checks

Check every exported result against its exact prefix, feature, observation and environment domain. Distinguish a supplied conformance law from a theorem deriving it. Check the retained source-property premises separately from a workload witness. Do not weaken a surviving hypothesis merely because its former neighboring theorem has been removed.

S.4 Declaration and proof order

Declare Core.SourceChoiceBinding and Core.ReachPrep before Core.PreparationEvidence. The source transition relation and permitted choices are independently defined; preparation evidence proves that a selected finite segment is legal in that relation. A constructor failure cannot redefine admitted source or RTL behavior. Encode Core.RegSeq, Core.DeferredNBHole/NoDependentUse and the actual decision snapshot before permitting unresolved-result-independent preparation. The scope includes control, aliases, addresses and payloads, not merely syntactic reads of a return variable.

Bind J.GeneratorCompleteness and source-generator realization before proving source strictness and the required-order graph. Establish J.SourceOrderWF/J.RequiredOrderWF and the applicable J.SourceOrderReflection obligations before concluding canonical equality. Do not infer acyclicity from equality with the as-yet-unproved target order. Every admitted extra generator or projection case must have its typed consumers under Core.ExtensionConsumerCoverage, including erased-support paths and whole-group contraction. For every applicable generator, prove both directions of restriction stability on old occurrences, separately from predecessor closure and acyclicity. Use forward persistence as well as the no-new-old-edge direction in J.OuterCanonIdeal and its erased-only equality proof; preserve old projection/group records explicitly.

Prove primitive simultaneous-action/frame and residue laws before J.LocalBoundaryCompose; then prove J.LocalOriginCanonExt and discharge the local hypotheses used by J.ConstructiveEdgeInduction/J.LG-Origin. Only afterward apply J.BatchPrefixCoverage or the applicable named carrier lift. For ordinary composition, establish the profile, restricted local evidence, glue and settling contracts before defining and proving preservation of the assembled relation. In that theorem, derive the assembled invariant from these components rather than adding its preservation as an unexplained global axiom. General local-origin applications still prove their own stated base/successor hypotheses.

J.OrdinarySettlingEvidence offers QF and QSafe as alternatives, with QM required in either case. QF reuses qualified source settling; QSafe proves its safety-specific operational requirements. Neither separate source/RTL fixed-point uniqueness nor stable roots alone proves matching snapshots. Preserve the ordinary profile's START/FINISH exclusions and other feature restrictions. Export checkpoint representatives only after SCP1–SCP3; neither ordinary topology nor a cycle-granular monitor supplies that proof.

Lazy declaration order. Fix source movement legality and LazyEligibility/LazySerialWitness before proving LazyInventory and LazySlack. Establish the complete occurrence/support records and applicable closure/rank construction without assuming missing demand completions. Generalize Alignment to RecordedDemand, then prove DemandPlacement/BatchPlacement and DemandPushResource from the existing controller/component laws. Prove auxiliary-node elimination and assemble LazyComposition from the generic Completion theorem. Derive LazySerialRestricted only under its global rank hypotheses. The separate protected singleton extension, with its original rank evidence, Facts covering every added completion within the declared F and Agreement over the extended witness, gives

LazyExtendedStandard. This ordering adds no implementation qualification or new source scheduling permission.

S.5 Carrier, order and completion safeguards

Treat the E5/Core.SyncFenceOrder blocking-predecessor edge as a genuine strict canonical edge that may complete in the same cycle as its synchronization successor. Use J.RequiredPredPrefix for legal-prefix closure and the source-order/required-order proofs for strictness. A qualified J.StandardCycleRank can orient that same-cycle edge by its secondary rank; J.RequiredPredCycleOrder covers only generators whose own rules establish a strict clock inequality. Do not rewrite every strict ordinary edge into a strict-cycle edge.

For Batch, prove the real edge inventory and between-edge invariants on the actual carrier. Core.QSync supplies same-cycle settling under its own qualification; it does not supply real-cycle completion, arbitrary-prefix coverage, fairness or a future successor. A selected source-property checkpoint must not replace the exact source state maintained by the operational induction.

For standard completion, formalize source-aligned control as an explicit implementation-conformance hypothesis, not a consequence of completed-event matching. Alignment uses agreed predecessor facts to determine the target; control and preparation laws establish actual eligibility. Placement obtains permanent isolation by finite exit of relevant older work, not by assuming temporary isolation persists. Prove resources, residual single-member exposure, joint placement, epoch protection and noncircular restoration before the K.CT.Standard discharge. Service is an additional execution premise. The final topological induction consumes these results and does not prove them by assuming eventual acceptance. For delayed guards, establish persistent support first, then finite internal readiness progress, then the actual eligible handshake and mandatory commitment; separately prove pre-grant or abstract-unit service where needed. Next-edge behavior is one sufficient controller realization, not a consequence of determinism. Lazy safeguards: a recorded source demand asserts neither an RTL request nor a W completion. Placement/request assertion must be derived from completed support; an extra-slack release must not assume the very batch retirement it supports. Only selected eligible extra-slack cases replace floor releases. Local eligibility does not imply restricted global rank applicability. Check one complete graph after demand elimination; a topological sort supplies neither inventory completeness nor exposure validity. Source-extension protection may extend beyond R even though required RTL service is only through R.

S.6 Foundational dependency inventory

Use the retained definitions of Core.1–Core.4 for the reference model, traces and operational quiescence; G's R/E rules, including R2C; the retained Core.8–Core.17 issue/commit, frontier, request, synchronization and occurrence clauses; Core.16's admitted-work discipline; Core.SourceChoiceBinding/ReachPrep/RegSeq and NB snapshot/decision/hole rules; actual cycle, origin, epoch and Batch carriers; J.ConstructionAction and its field-sensitive footprints; and Q.1(a)–(i) for the selected elaboration. This is a semantic inventory, not permission to restore missing or retired numbered definitions. Exact consumed members are bound by Core.EvidencePackage and M.7a.

Keep source-property coverage and observation modules separate from full-F correspondence. FB0–FB2 and CF0–CF3 must cover every fact consumed by K.CT, including non-transfer outcomes and acceptance facts; outer projection may erase facts that completion still needs. Keep mandatory physical commitment, pre-grant arbitration fairness, source issue fairness and internal movement/restoration service in distinct interfaces. No proof of finite-prefix safety imports completion service by default. For logical Pop payloads, mechanize observer totality and erasure over every physical successor, unique old-owner retirement, representation induction for non-injective functions, lawful later use and exact fact emission. Preserve the restriction to non-physically-observed internal singletons or the explicit

Q.CT.LogicalPair family; mechanize the independent, equal-length coupled and fixed-offset staggered delivery contracts without merging their premises.

S.7 Reuse, extraction and extension review

Apply Core.EvidenceRealization when choosing between a parameterized existence proof and executable witness production. The mathematical family theorem need not contain an executable extractor. If a total extractor is advertised, its termination/totality and soundness must cover the declared qualified domain. Individually accepted outputs of a partial generator prove only their checked instances. Sound checker acceptance and empirical success rates are different from a coverage theorem. For executable lazy witness production/checking, bind K.CT.LazySimulation demand visibility, legal optional selection, mandatory-transfer inventory and M.7a's route-specific records. A deterministic prompt selector is generator guidance, not a schedule-dominance theorem or proof that the generator finds every qualifying witness. Preserve the distinction between restricted rank validation and mixed-graph checking.
Review every extension against Core.ExtensionConsumerCoverage. A new primitive, compound unit, reset policy, projection or carrier must update every applicable source-legality, local/frame, order, history, fact and progress consumer, or prove that consumer inapplicable. Reusable generic cases need explicit quantified proofs. Bind semantic revisions and instance validity in M.7a; new schema fields and permissive decoding cannot silently extend the theorem’s class.

Appendix T – Scheduling-Specific Obligations Beyond Standard Simulation Theory

T.1 Purpose and Nature of the Formal Result

Status. Appendix T is informative and non-normative. It highlights selected scheduling-specific library gaps but does not add, remove, redefine, or exhaustively enumerate formal obligations. Normative authority remains with the Normative Formal Core and the applicable theorem interfaces; Appendix M.7a supplies the definitive consolidated side-condition discharge record, and Appendix S lists the retained Lean mechanization targets.
Standard formal-methods libraries provide reusable foundations for labeled transition systems, weak or epsilon-abstracted transitions, forward simulation, finite and infinite executions, trace inclusion, well-founded induction, fixed-point reasoning, fairness, and related proof techniques. Those foundations can substantially reduce the generic infrastructure needed for a Lean mechanization of this document. They do not, however, establish the HLS scheduling contract or its safety and required-workload completion guarantees by themselves.
The safety proof constructs a legal source history for each covered implementation prefix and preserves complete observable units, values and the required scheduling-specific order. Independent units may commute or have different cycle groupings across source and RTL. Canonical-history projection, atomic grouping and predecessor contraction determine the exact observation relation. A generic forward-simulation theorem can supply infrastructure, but does not itself discharge these contracts or source-property coverage. No reverse realizability theorem for an arbitrary selected source execution is asserted.
The completion result is a global induction over a finite prerequisite-ordered workload. Exact internal-fact correspondence and conditional agreement connect the source witness to implementation events. Source-aligned control under B.OF.Control/B.CT.Control and K.CT.Alignment first connects agreed predecessor facts to the required operation and descriptor. Resource, preparation, ordered placement, persistent enabling, joint-exposure and restoration laws make the next required unit continuously

serviceable; physical units commit at the eligible settled handshake, while any pre-grant or genuinely abstract service uses its separately declared contract. These scheduling-specific obligations are separate from finite-history safety and from matching individual clock edges.

T.2 Scheduling-specific obligations

Generic simulation theory does not determine this document's source issue/return phases, complete edge inventory, synchronization fences, back-annotated capacities or elaborated handshake laws. The retained Core, B and Q clauses supply those semantics, and the constructive local contracts prove their implementation realization.

T.2.1 Initial states and reset epochs

Generic simulation starts from related initial states; it does not identify the correct HLS reset realization or prove that stateful channels and peer endpoints agree. RW1–RW5, the canonical reset occurrence and epoch-key rules, J.LocalResetContracts and J.LG-ResetBatch provide the selected obligations. Match request cancellation, retained or cleared storage, ending-epoch actions and new-epoch control under the actual edge priority. An implementation-local reset object or simulator timestamp is not a cross-model epoch identity. Partial reset needs its explicit communication disposition; reset of one process is not permission to discard a peer's outstanding obligation.

T.2.2 Dynamic operation identity and legal choices

Added FSM states, overlapped iterations and eager preparation separate a source call's logical identity from the physical cycle in which its request appears. Core.ImplIssueOrder and Core.IC require state invariants tying each request to its dynamic source occurrence, endpoint, interval and epoch, preserving source-order issue closure and persistence. Comparing timestamps after the fact cannot show that the correct pending operation existed before a boundary decision. Core.SourceChoiceBinding and the NB decision law also distinguish selected exogenous choices from outcomes computed by the actual source snapshot; annotations and snooping may record an outcome, but cannot override that law.

T.2.3 Old-state boundary actions and atomic units

At an ordinary positive-capacity E4 boundary, all actions selected for one edge read the same beginning-of-cycle occupancy and data. If both Push and Pop are legal from that state, their joint update must preserve the old head, appended tail, capacity and occurrence order. A serialization used in a proof may not manufacture a same-edge Pop from an empty FIFO or space in a full FIFO by using the other action's next state. A zero-depth transfer and any declared atomic multi-member transaction require a joint realization; independent endpoint steps are not a substitute. Primitive commutation/residue and frame proofs must justify composition against the actual clock-edge semantics. Core.TransferLaw includes every actual settled handshake in that joint update. E4/cardinality checks cannot be applied afterward to discard an inconvenient member.

T.2.4 Boundary and memory faithfulness

Back-annotation is more than assigning a depth. The logical boundary determines which accepted but unmatched items are channel storage, which registers are process state, and how credit, joins, gates or bypass behavior affects availability. B-faithfulness and the qualified Q elaboration connect actual state and handshake decisions to that boundary. Completion's guaranteed resource floor may be smaller than nominal reference capacity. For memories, Core.MemFaithful/Core.MemProfile must account for value flow and permitted cache/merge/split/reorder transformations at a stable logical boundary; a variable physical transaction stream cannot be equated with the original stream by renaming events. For eligible lazy cases, distinguish nominal storage, the standard guaranteed floor and demand-supported extra-

slack availability: DemandPushResource proves the last without increasing the floor or changing the bound capacities.

T.2.5 Projection and required order

Hiding an internal event must preserve any required dependency it carries between retained events. Q.1(c-order), J.OuterCanonHist and the source-generator/order-reflection proofs bind complete outer groups, contracted paths and their required order. The observable property is evaluated on that qualified history, not an incidental logger sequence. Whole atomic groups cannot be partially retained. Generic graph projection alone does not prove that the projected grouping and order match the declared outer channel/synchronization semantics.

T.3 Why local composition needs explicit evidence

Same-edge decisions read the common declared snapshot; local frames and primitive residue laws justify combining their source realizations. Returned data is not available for dependent same-cycle source work. Complete unit identity and normative generator translation establish canonical equality independently of an incidental execution ordering. Environment and testbench steps are part of the same proof domain.

Local facts do not combine into a reachable global execution by conjunction. Two otherwise plausible local witnesses may require conflicting writes, different NB snapshots, incompatible synchronization partners or different resolutions of one shared choice. The composition proof therefore uses one fixed prefix and old-state boundary, reconciles complete atomic interactions, validates footprints and frames, and restores a related global successor. Reset and environment steps belong to that same domain. A response contract cannot be replaced by the assumption that the environment will cooperate with the future witness.

Coverage is a separate proof step. One matched prefix says nothing about an unexamined reachable successor. Actual-edge induction quantifies over every admitted successor; the exact Batch lift extends origin coverage to all admitted finite prefixes by the complete-edge inventory. Origin correspondence alone covers only its declared origin domain. Neither constructor success nor a monitor’s sampled checkpoints can define away the remaining executions. Source-property checkpoint and selected-source-history options retain their own independent coverage obligations.

T.4 Workload progress

Finite-history safety does not ensure a request becomes present or remains enabled. Completion additionally needs source-aligned control under B.OF.Control/B.CT.Control, K.CT.Alignment and legal preparation, ordered movement/drainage, persistent requests and guards, sufficient guaranteed resources, joint exposure and actual service. A delayed gate needs a noncircular restoration proof. A passing reference run supplies the workload witness; implementation conformance supplies the progress laws. Physical early capture and bubble movement are not automatically source issue or iteration launch: B.BoundaryBinding must prove their ownership and Core.16/E3 interpretation. A registered guard can qualify through finite internally serviced progress to stable enabling. Neither that guard option nor a flush directive establishes early-staging or output-channel conformance. The named Q.CT.InputStaging profile supplies early-input ownership and isolated delivery under its stated laws; further storage organizations remain separately qualified. The exact cross-iteration issue inequality, rather than physical pipeline depth alone, determines whether a proposed site mapping meets E3. Core.LogicalPopPayload with Q.CT.LogicalPayload permits causal original-value attribution for a restricted internal computed input path without physical raw retention. Q.CT.LogicalPair additionally qualifies the restricted positive computed pair, with independent paths, equal-length Q.CT.CoupledInputStaging or fixed-offset Q.CT.StaggeredInputStaging. Q.CT.PairedRepresentation

permits private cross-input computation after the physical join while preserving two logical items and legal later source use. It does not alter readiness or license earlier source use. Record joint internal service and bubble drainage; unconditional global stall, staggered source-facing retirements and other transformed groups remain outside this named pair profile. A settled physical handshake is sufficient for its declared transfer on the eligible edge. All guard, grant and resource prerequisites act before that sample; service needed to reach it is a separate qualified obligation.
Lazy workload progress uses an additional scheduling-specific argument on eligible independent staging. The source restriction and serial/group-serial consumer discipline allow at most one unretired item per channel outside ingress. Old-state admission then separates ordinary floor-supported Pushes from extra-slack cases. For an extra-slack target, complete recorded-demand support derives the corresponding implementation placement/request; independent forwarding can release ingress space before the supporting Pop or batch retires. Auxiliary demands contribute support paths and are eliminated before induction over actual completed W units; they create no completion facts. The route table at the head of K.CT.LazySerial distinguishes restricted rank-derived acyclicity, mixed-graph checking and protected singleton extension. All standing fact, agreement, conformance and service premises remain; generic simulation theory or a passing lazy run supplies none of these arguments by itself.

T.5 Interpretation limits

The completion result ends at acceptance of R and provides no cycle bound or guarantee for unrelated later work. Failed serial certification is inconclusive. General NB timing dependence, obligation-cutting resets, unsupported compounds and controller classes require separate qualification. The obligations ledger distinguishes retained framework theorems from tool-family and instance evidence.

T.6 Reusing generic libraries and checking evidence

Lean and existing formal-methods libraries can supply much of the general infrastructure: transition systems, execution histories, simulation, finite orders, induction, fixed points and temporal reasoning. Reuse these foundations where their assumptions match the models in this document. The main design-specific work is connecting that infrastructure to HLS scheduling and the implemented system. The mechanization must preserve operation identity, communication and synchronization semantics, legal choices, reset behavior, source-aligned control and service laws, and the relationship between local evidence and a reachable global execution. Lazy checker work additionally validates selected-channel eligibility, source-demand/movement records, complete support, auxiliary-node elimination and the chosen rank or assembled-graph evidence under M.7a.
Keep reusable semantic proofs separate from instance data, and keep correspondence, behavior coverage and property transfer as distinct proof steps. Class-wide reuse requires a proof over the admitted class; sample executions do not establish it. Safety and required-workload completion retain different assumptions and should remain separate in the mechanization. A fact about equal outer histories cannot replace the complete internal-fact premise of completion.
Certificates and checkers are evidence interfaces, not substitutes for the mathematical obligations. A checked record must refer to the actual source/RTL artifacts, semantic revision, domain and stated premises. Proving that a checker accepts a file does not prove that the checker validates those premises, nor that an extractor handles every admitted instance. Core.EvidenceHdr, Core.EvidencePackage, Core.EvidenceRealization and M.7a make those trust and coverage boundaries explicit. This use of checked evidence does not reinstate the retired certificate correspondence route.

Appendix U – Revision Record

31 October 2026 — Fact-signature lifecycle clarification following the Lean v125 review (R125-M1). Distinguished the prospectively declared F and attribution from prefix ledgers and the finite fact inventory consumed by a workload certificate. Same-instance source extension may add actual W completions and consume further facts only within that declaration; fixing pending demands before extension does not establish pre-workload prospectivity. Specified a new theorem-instance/Core.EvidenceHdr binding with affected fact, agreement, provenance, attribution, witness and evidence-reuse obligations for a genuinely changed declaration. Aligned FB0, J.InternalFactCorr, K.CT.Facts/Agreement, LazyClosure, LazyExtendedStandard, LazyFactsAgreement, the M.7a completion-binding ledger, the responsibility table and Appendix S. Preserved full implementation-prefix coverage, protected source-continuation requirements and until-R service scope. This edition changes the rules document only; the separate Lean strategy and architecture guide are not revised here.

29 October 2026 — Rules and Lean v122 linkage review repairs. Added K.CT.UntilRScope and explicit until-R Exposure/Service forms, with a shared scope for the ten service-dependent supporting progress results, Standard and lazy completion discharges. Recast the Completion proof as an accepting/nonaccepting execution argument; retained the full Facts domain, semantic Alignment, service-independent resource invariants and the separate protected source-extension requirements. Aligned B.OF service use, M.4/M.7a, R.2 and S.2 with that scope. Strengthened J.RequiredOrderWF/J.OuterCanonHist restriction stability to equality on old occurrences, required generator evidence in both directions, and completed the induced-ideal and erased-only history-equality arguments using forward edge preservation and fixed group records; aligned the affected generator, locality and mechanization guidance. Removed the obsolete certificate-route wording and bare K reference from J.ConstructiveClassSafe. These are rules-document repairs; the separate Lean strategy and architecture guide are not revised in this edition.

28 October 2026 — Targeted rules clarification following the Lean-strategy issues review. Clarified the plain-FIFO guaranteed floor including credit, persistent enabling in Exposure, and rank-derived discharge of the restricted lazy witness acyclicity requirement. Completed the positive-channel accounting proof for coincident read-and-retire; aligned physical-commit wording with the mandatory transfer law. Added explicit M.7a applicability triggers and updated consumed-interface references while retaining current source-realization, exact-prefix, history-preservation, frame, designation and carrier-completion obligations. Removed undefined or retired references selectively, preserving current observation and progress scope limits. The existing arrival-independent placement/request requirement and safety/completion choice-binding distinction remain in force. Follow-up wording cleanup removes two further retired certificate-admission disclaimers, uses committed-transfer terminology in the ordinary FIFO residue calculation, and names auxiliary-node elimination alongside the rank lemma in the restricted lazy premise. No new component qualification, completed mechanization or implementation-conformance result is asserted.

26 October 2026 — Lazy completion integration across the document. Added one informative completion-route table at K.CT.LazySerial; split M.7a common, restricted, mixed and extension evidence so mixed applications do not inherit global rank/issue requirements; made LazyExtendedStandard's restricted all-singleton hypotheses explicit while retaining local mixed extension through LazyComposition. Added the qualified atomic-batch staging example and distinguished issue policy, lazy movement, actual capacity/boundary configuration and performance simulation. Integrated Abstract/Introduction, reading paths, B/Q examples, K resource/checklist navigation, M responsibilities and R/S/T summaries and proof/checker targets. Clarified R.5 by naming J.LG-Origin and J.BatchPrefixCoverage. Standard floor lemmas, generic Completion, full implementation-prefix Facts and standing conformance/service obligations are retained.

26 October 2026 — Named lazy simulator guidance and citations. Named the operational block K.CT.LazySimulation with identifiable demand-visibility, selection, informative-example and RTL-independence clauses. Matched Appendix K's unbolded label style and replaced positional/unnamed references in Formal Guarantees and M.7a with the stable identifier.
26 October 2026 — Preferred lazy pre-HLS workflow and operational example. Added prominent source-only simulation guidance and the preference for lazy mode where eligible, while retaining standard floor proofs and standing qualifications. Explained the process-facing demand versus early staging movement distinction, data-independent legal Pop/batch issue, unchanged producer valid behavior, and optional-movement/mandatory-handshake limits. Added prompt deterministic selection guidance without claiming schedule dominance, and a depth-one ingress/two-stage example distinguishing usable lazy inventory F+1 from physical capacity F+m. Clarified that no per-stimulus RTL execution is introduced, while independent snooping qualifications retain their dependencies. Aligned Appendix F, SerialWitness and M.7a; plain unreserved FIFOs need no new movement restriction.
25 October 2026 — Local source-extension premises and theorem naming. Replaced LazySourceExtension's restricted global LazyClosure premise by explicit local facts for a fixed finite set of eligible pending singleton demands, retaining the protected continuation and reference movement/retirement conditions. Restricted LazyClosure supplies those facts; mixed composition establishes them directly. Made K.CT.Witness compliance an explicit requirement of LazyComposition. Renamed the rank-derived corollary K.CT.LazySerialRestricted and updated its proof and precise consumers while retaining K.CT.LazySerial as the route name. Updated M.7a extension evidence.
25 October 2026 — Lazy completion theorem hierarchy and resource cases. Made LazyComposition the general theorem and placed it before the restricted LazySerial corollary. Limited the latter's global serial/group-serial premise to its rank-derived profile; the composition theorem requires the local consumer discipline only on selected channels. Kept LazyExtendedStandard's direct Standard route separate. Explicitly split ordinary selected-channel Pushes, including k<g_c, from extra-slack demand-resource cases in the theorem, proof, navigation and M.7a evidence record.
25 October 2026 — LazySerial mixed-component composition and integration. Added K.CT.LazyEligibility and K.CT.LazyComposition so eligible ungated independent input channels may use lazy demand support within mixed designs while other units retain their qualified closure/exposure/service proofs. Added the unified LazyMixedRank for singleton/group certificates and the checked assembled-graph alternative; distinguished graph acyclicity from dependency completeness and exposure validity. Defined unreserved coincident-retirement plain-FIFO group members, retained the split-capture exclusion, and required trace compliance on every selected channel including extensions. Clarified consumer issue/placement and whole-batch eligibility, protected mixed-design source extension and synchronization accounting. Corrected the LazySerial heading level, visible floor summary, M.7 table and Appendix K map; updated M.7a and proof-navigation pointers. Preserved the cached TOC and all unrelated content. No reserved-slot proof or implementation qualification is asserted.
25 October 2026 — Added K.CT.LazySerial as a restricted independent-input-staging completion alternative. Added the component-bound abstract lazy witness, inventory/slack and finite closure/rank lemmas, protected singleton source extension, recorded-demand placement and resource-independent atomic batch placement, demand-supported Push exposure, and singleton/group certificate constructions. Coordinated Facts, Agreement, Witness, CtrlSupport and Alignment target domains; retained actual W-completion membership and the full implementation-prefix fact domain. Added B/F/Q route pointers and M.7a evidence entries. Coupled, staggered and separate reserved-slot profiles remain outside without their own restriction/progress theorem. The standard floor discharge and generic Completion theorem are preserved. This is rules text and mathematical proof development, not Lean mechanization, implementation-family qualification or closure of the separate baseline audit.

25 October 2026 — Cross-audit repair follow-up. Removed the stale ramp-down requirement from J.AnchorDirectInputLift while retaining Core.SyncFence, capacity withdrawal and SP5 old-value protection. Removed snooping from the stress checklist's mechanisms for enforcing causal dependencies. Restored the informative default Matchlib In<> skidbuffer description with a version/configuration check. Aligned SyncFire wording with later-cycle L2 settling, stated B3 independently of B2, clarified mandatory next-interval capacity withdrawal under Core.16(7), and repaired the direct-input and external-link introductory wording. Named the existing provenance guidance Core.FixedSupportValidation and updated CtrlSupport and M.7a references; its content and the agreement lemma are unchanged. No new proof route, implementation qualification or baseline-audit closure is asserted.

25 October 2026 — Follow-up repairs from the independent baseline audit. Applied the accepted residual wording, scope and checklist corrections to the latest medium-and-LOW repaired document. This revision does not adopt the other audit's gate closures, change the frozen baseline, or assert implementation conformance.

Provenance (AUD-A02-PROV-01): documented a noncircular validation pattern for fixed witness-relative support under data-selected dependencies. Lower-ranked selector agreement, actual-source availability and occurrence/epoch attribution must be established over the independent domain. Core.FinitePrefixAgreement and its fixed-Pred premises are unchanged; no staged-predecessor interface or automatic qualification was introduced. Linked CtrlSupport and M.7a to that validation and retained the independent agreement alternative.

Direct-input summaries (AUD-A02-G-DISYNC-01): replaced automatic ramp-down claims by completed or retained/isolated old-value uses before the persistent sync offer. Closing next-interval capacity is not itself old-use protection. Retained the existing mandatory-handshake, physical sync and evaluated-boundary repairs.

Core and Appendix G (AUD-A02-FINISH-01, RESETEMPTY-01, BRULES-01, SYNCFIRE-01, G-OVERVIEW-01, G-ONEWAY-01 and G-CAUSAL-01): clarified actual termination events, affected-reset scope, consumer-specific B assumptions and next-sample effects of sync gate changes. Corrected the issue-versus-commit-fence implication, source compatibility, memory and observation summaries, stale implicit-sync wording, runtime-assertion scope and causal methodology language.

Completion and evidence records (AUD-A07-RESIDUAL-01, AUD-A08-OWNER-01, AUD-A10-SVCSHAPE-01, APPL-01, LEDGER-01 and AUD-A12-M7A-01): stated whole-unit residual completion duties, mixed-owner path closure and noncircular service requirements while retaining independently qualified recurrent-opportunity and bounded-response contracts. Clarified singleton logical-payload applicability and arbitration exclusions; added explicit ledger pointers and the logical-retry conformance trigger.

Supporting text (AUD-A11-SNOOPFAIR-01, EXTCLAIM-01, AUD-A12-NAV-02 and EDIT-01): restricted snooping fairness language to its actual consumers, marked external implementation/example descriptions as informative and unverified without weakening normative tool requirements, corrected the Appendix P navigation description, and removed the two stray spaces before semicolons.

Structural/package checks preserve unrelated paragraphs, formatting, objects and the existing TOC; no PDF or raster checks or TOC regeneration were performed. The independent audit's experiments were not rerun.

25 October 2026 — Boundary terminology follow-up to the LOW repairs. Corrected Appendix G's request-persistence summary to refer to the endpoint-owned BoundaryReq at the selected evaluated boundary under Core.IC(1), rather than an arbitrary physical boundary request. Aligned Core.NBPollInterpretation with the selected evaluated boundary. This preserves Core.10's GQ1 distinction between the persistent attributed request and a gated downstream physical signal, and Core.IC's ownership limits. Commit/reset limits and mandatory handshake commitment are unchanged.

25 October 2026 — Seven LOW wording and scope repairs. Applied to the supplied 23 October posted repaired document, preserving its medium repairs and other intervening edits. The entries below describe textual repairs; they do not certify an implementation, discharge outstanding theorem premises, or close the separate baseline audit.
AUD-NAME-01 — Aligned the Core E1 summary, Appendix G explanation and synchronization-pair note with strict per-process synchronization cycle separation and occurrence matching. Preserved paired cross-process simultaneity and the separate same-edge message/synchronization allowance.
AUD-DOM-01 — Defined qualified implementation execution at K.CT.Completion. Kept Facts over all operationally admitted prefixes independently of service and success, Exposure over its service-qualified domain, and Standard as a conditional exposure discharge rather than a success filter.
AUD-A02-OBS-01 — Distinguished physical request levels and auxiliary Issue evidence from committed Σ events. Made ε classification relative to the selected observation/proof alphabet and preserved failed-poll call identities, outcomes and consumed internal facts as proof evidence where required.
AUD-G-WRAPPER — Replaced automatic preservation and blanket failed-poll nonmatching claims with the source-legality, complete-choice, dynamic-call and L.WC/W1–W5/L.Dir qualifications; aligned the Appendix L introduction.
AUD-L-TB-REPLICATION — Qualified replicated and shared reactive testbench alternatives by the fixed stimulus/choice/runtime/environment binding. Equal code or seed and trace-functional reactions alone do not establish legal replay, coverage or correspondence.
AUD-A-REVERSE-TIMELINE — Marked the reverse-order minute-2/minute-3 timetable as hypothetical and unreachable, and stated the actual mutually blocked state while preserving the call-time versus formal Issue distinction.
AUD-G-TRANSFORM-SUMMARY — Qualified the scheduling and transformation summaries for actual data/control dependencies, source legality, complete observation membership, direct-input sampling/update protection, environment behavior and component refinement. Distinguished finite runtime assertion evidence from a universal no-backpressure invariant.
23 October 2026 — Consolidated medium repair record R1–R5. The separate repaired document completes the planned textual repairs for the three MEDIUM findings listed below. This entry consolidates the staged R1–R5 records retained at the end of this appendix. R6 adds this revision record only; the substantive rules remain identical to R5.
Cycle-ideal carrier gap — AUD-A02-CAR-01. Removed the optional cycle-ideal safety route and its exclusive frame/designation, partial-cycle completion and cycle-carrier machinery, rather than asserting a repaired proof of the disputed carrier claim. Pruned the dependent references and orphaned closing remark, and corrected the residual Appendix R description. Preserved the independent constructive Origin and exact Batch routes, occurrence-bound QSync settling, and the CompleteCycle/SilentCycle and shared request-persistence material belonging to Core.CycleResult. Ideal-prefix closure remains where independently required by the selected-source-history method.
Mandatory handshake semantics — AUD-HANDSHAKE-SEMANTICS. Core.TransferLaw now requires every eligible settled physical ready/valid handshake to commit, with complete transfer effects and fixed event identity. Legality, reset and atomic coherence must be resolved consistently before that overlap; no later scheduler choice may veto it. Reconciled source-selection language, local evidence and composition contracts, join/staging consumers, and K/Q service obligations. B2 addresses pre-grant arbitration; actual physical commitment does not require an additional fairness choice. Primitive progress still requires the stated clock, support and reset conditions, while coupled exposure and internal-service obligations remain. Core.10 and TL2 explain the declared-complementary-gate and gate-before-issue bindings, retaining exact transfer correspondence and blocking-request persistence. AUD-NAME-02 is incorporated into this coordinated repair.

Physical sync protocol — AUD-SYNC-PROTOCOL. Core.SyncProtocol now binds physical issue, transaction identity, persistent ready/valid offers, eligible-edge completion, reset disposition and unary scope. A pending sync request cannot be withdrawn before commit except on an affecting reset. The DUT must finish or protect old-value uses before offering sync ready, and maintain that protection through the relevant pending interval and later protected uses. The process-facing fence permits same-edge predecessors and later external delivery of previously buffered messages. The main-body testbench recipe explains the sequence informally and points to the formal Core protocol.
Integrated review and scope. R4 reviewed affected safety/completion dependencies and corrected residual completeness and service wording. R5 aligned the remaining service terms and headings, stated the two positive gated-request forms, and simplified repeated main-body explanations. Targeted structural, reproducibility and finite regression checks were performed; these do not establish implementation qualification or complete the adversarial proof audit. The frozen baseline and its issue inventory are unchanged, and G4/G5 remain open. The baseline audit will resume separately and its final findings will later be reconciled with this repair branch. LOW findings were not the target except for incidental clarification associated with the medium repairs.
14 October 2026 — Representation scope and completion-ledger integration. Clarified InputStagingResidue's safety-only trigger and added R audit pointers without making it a blanket elaboration qualification. Distinguished component-local direct-input exclusions from enclosing-system configuration use; configuration-dependent representation/computation still requires explicit composition evidence rather than automatic permission from fact adapters. Aligned O2/O6/O10/O13 with standard staging noncircularity and route-specific control support; named occupied-slot/departure-gated LogicalPair extensions. Expanded M.7a/S.2 result-storage evidence to cover selected single-slot, multi-slot and separately qualified downstream-dependent realizations, without a throughput guarantee. Added concise fact-adapter pointers in M.4/R.2. Preserved prior revision records and the posted document's cached contents.
14 October 2026 — Completion scope summaries and evidence ownership. Aligned the abstract, at-a-glance workflow, Appendix F, embedded Guide and M.4/M.7 responsibilities with standard control-support certification and arrival-independent placement/request proofs. Named arrival-dependent placement in the extension list, without excluding controllers solely by valid-bit implementation style. Scoped CtrlSupport to the standard alignment/placement discharge while retaining complete semantic control-support obligations for alternative exposure proofs. Replaced undefined PopJoin shorthand with explicit paired-retirement/internal-join descriptions; assigned InputStagingResidue a local-proof trigger and owner with edge-law/C3 pointers. Centralized the stalled-work frame in Core.16, named the registered-fact specialization corollaries, and aligned SerialWitness exactly with Core.PolicyS's same-interval blocking-call restriction.
14 October 2026 — Core glossary, candidate frontier and reference consistency. Restored the Σ glossary summary; made Core.7 the message candidate slice and restored Core.7a as a blocking-call candidate slice distinct from the operative Core.14/Core.15 pending frontier. Moved registered-anchor scope prose out of the Core.3 heading and corrected only its cached contents label, without recalculating fields or page numbers. Replaced retired external-environment and reset-domain references, corrected the Q.1 range to (a)–(i), and named J.JoinFailureFirst. Updated B.CT.Compatibility to distinguish the three delivery alternatives from their separate representation qualifications. H1–H4 and M1–M4 changes remain in force.
14 October 2026 — Configuration fact adapters, zero-based floor ordinals and staging proof dependencies. Added J.StableReadFactCorr and J.ConfigRecordFactCorr with full-signature initialization, local realization and exact-prefix obligations, retaining separate root and committed-record treatment. Standardized empty-resource endpoint ordinals and all current floor-space formulas to k>=g / k<g, consistent with staggered ownership. Distinguished downstream drainage from pre-issued younger

completion and stated the stalled-work frame. Made standard staging placement/request independence from target arrival explicit; specified the separate evidence required for a pre-request data-driven extension without claiming that extension as a proved standard profile. H1–H4 changes remain in force.
14 October 2026 — Result storage, control support and local proof scope. Clarified the single reserved-empty LogicalPair result slot's issue/rate limitation and the evidence needed for multi-slot or downstream-dependent alternatives. Added the fixed-selection InputStagingResidue obligation without changing explicit E4 boundaries. Added finite CtrlSupport inventories, negative/control-decision closure and alignment before PlacementLemma, with corresponding evidence and mechanization checks. Kept W restricted to actual witness completions and retained existing ELM/ELA primitive scope; broader locality support remains a separate proof obligation.
12 October 2026 — Input-staging scope alignment. Updated B.CT.Capture, the K.CT exclusions, Q.CT.LogicalPayload operational separation and Q.CT.Extension to name the independent, equal-length coupled and fixed-offset staggered input-staging alternatives consistently. Each retains its own topology, accounting, guard, delivery and service restrictions. No additional profile or theorem premise is introduced.
12 October 2026 — Staggered input staging and joined representations. Added the fixed-offset physical-read alternative Q.CT.StaggeredInputStaging and K.CT.StaggeredDeliveryLemma, preserving terminal joint source retirement and explicit drainage past blocked younger reads. Added Q.CT.PairedRepresentation for pure cross-input computation after a qualified join, with actual transition equations, two retained logical owners, unchanged accounting and legal later source use. Updated the J, K, Q and conformance consumers while retaining the equal-length component's exclusion of staggered topology.
12 October 2026 — Logical payload scope and safety-only use. Restored Core.LogicalPopPayload scope in terms of the representation and operational clauses for both singletons and declared positive pairs, without requiring the completed component qualification. Made the pair's safety-only use explicit and separated its completion obligations; kept ungrouped safety-property reuse subject to its own inclusion proof.
12 October 2026 — Positive computed pairs and flushable coupled delivery. Added Q.CT.LogicalPair and Q.CT.CoupledInputStaging as separate restricted qualifications, with K.CT.CoupledDeliveryLemma and its resource, joint, standard and ledger consumers. Extended causal logical-Pop attribution and J's local proof to an indivisible two-return packet and private same-edge combination with legal later source use. Preserved the independent-delivery law, explicit bubble drainage, grouped-source witness requirements and physical-observation exclusions. Aligned applicability and conformance guidance; staggered reads, unconditional global stall and other unqualified combinations remain excluded.
12 October 2026 — Logical payload integration clarifications. Made the InputStaging progress lemma explicitly applicable to the operational clauses retained by the logical-payload representation profile. Clarified B-faithfulness label interpretation, safety-only conformance ownership, the representation/operational scope of the Core contract and the separate obligations for later use of loop-carried state. Replaced an undefined source-model abbreviation and restored the Appendix B guide label formatting.
12 October 2026 — Internal logical Pop payload attribution. Added Core.LogicalPopPayload for causal admission-derived values at qualified internal logical Pops, with actual retirement, unique ownership, observer totality/erasure and unchanged physical observations. Added Q.CT.LogicalPayload for restricted computed singleton representation and legal later use, and J.LogicalPayloadRefinement for its conditional local and full-fact discharge. Updated boundary descriptions, projection discipline, completion fact interpretation and conformance ownership. No transformed-group qualification, readiness relaxation or witness-capacity lift is implied.

11 October 2026 — Coupler illustration and staging edge precision. Restored the informative F1/F2/F34 storage-free join illustration beside the independently staged-input example, with an explicit source-mapping and completion-qualification boundary. Specified old-empty-only and simultaneous-replacement internal staging options, common-old-state payload/action selection and no multi-hop same-edge transfer, while retaining primitive ingress/E4 restrictions. Qualified sustained II=1 by ingress depth, replacement/full-shift behavior, supply, acceptance and non-throttling controller conditions. Updated component evidence and mechanization guidance. Existing theorem scope and prior revision entries are preserved.
11 October 2026 — Named early-input staging component. Promoted the restricted ordered early-input organization to Q.CT.InputStaging with Q.CT.InputStagingProgress, preserving separate source/RTL conformance and capacity-lift obligations. Generalized the guaranteed floor to g<=F (and g<=C for exclusive credit), restricted mandatory forwarding to target isolation, and rewrote the two-input Q example with paired process-facing Pops at stage 3 and Push at stage 4. Updated applicability, GC1, the Guide, M and derived summaries; stated the exact cross-iteration issue bound without making physical stage count a universal criterion. B.BoundaryBinding is now a peer section. Prior revision entries are unchanged.
11 October 2026 — Pipeline boundary and finite-response guard clarification. Added B.BoundaryBinding and a restricted four-stage input-staging example separating physical capture, process-facing issue and bubble movement. The example explicitly requires early-staging extension evidence; E3, Core.16 and the at-site capture profile are unchanged. Added a finite-response guard option with persistent support, decreasing local progress and joint-readiness obligations, while retaining the narrower canonical ClockRefinement theorem. Centralized the placement proof in K, removed the Alignment placement dependency, clarified requests/internal arbitration and per-instance conformance, and aligned the main-body, F, M and R/S/T guidance. Prior revision entries are unchanged.
11 October 2026 — Retained-route R/S/T restoration. Expanded the derived dependency map, mechanization ordering, foundational inventory and explanations of HLS-specific proof obligations. Restored paper-first authority, deletion-safe R status, open-class consumer completeness and proof-family/extractor distinctions with normative owners in Core and discharge ownership in M. Clarified that checkpoint and selected-source-history options narrow source-property proof and do not supply RTL coverage. Retained theorem statements and prior history are unchanged; no retired route, import module or completed mechanization is reintroduced.
11 October 2026 — Retained-route guide and appendix alignment. Removed obsolete reverse-realizability claims and missing-remark/wait-for-graph references; aligned the Guide and R/S/T with constructive safety and standard workload completion, including source-aligned control and K.CT.Alignment. Clarified qualified polling/retry scope and boundary versus internal service. Retired unconsumed B5/B6 and permanent terminal-idle/environment-terminal extension machinery while preserving modeled termination, ordinary silent cycles, EnvStep and source-cycle closure.
11 October 2026 — B1-run retirement and completion-ledger alignment. Removed the single-RTL-run safety shortcut and its exclusive observable-prefix, maximum-run, cross-run fence, reset and projection congruence machinery; retained B1 only as a separately claimable determinism property and retained source covering-history evidence under constructive safety. Updated the abstract, guarantees, Guide, Appendix G heading and Appendix M to the surviving routes. M now explicitly binds source-aligned control, K.CT.Alignment, full fact/witness domains, pre-pending gate protection, standard-profile limits and capacity lifting. Replaced obsolete assumption labels, corrected runtime/environment/NB owners, and recorded retired M section numbers without renumbering.
11 October 2026 — Opening guarantees and relocated-definition presentation. Clarified the abstract and guarantees summary around constructive correspondence and required-workload completion, including coverage, control alignment, guaranteed-resource dependencies and the limits of a passing

witness. Distinguished the retained finite-job coverage shortcut from the retired certificate correspondence route. Removed the orphaned former preparation-lemma sentence, restored the deferred-NB definition heading, separated shared Core conventions from ReachPrep, and clarified environment and B4 references. Existing revision entries and the cached table of contents are unchanged.

11 October 2026 — Appendix I consolidation. Retained Appendix I as a reserved placeholder and preserved Appendix L. Moved shared non-blocking preparation and implementation issue-order definitions into Core, process-environment contracts into J, and WC/NB-CF into L. Retired unused replay lemmas and Core.ReplayHistory; retained J.OuterCanonHist and all constructive/completion obligations. Updated failed-poll matching to name direct constructive occurrence/snapshot evidence alongside legal witness selection, and corrected reading paths and stale preparation-proof descriptions. Existing historical entries are unchanged.

11 October 2026 — Standard completion exposure repair. Bound iteration permission, required membership and preparation to source-aligned implementation control; added K.CT.Alignment and complete owner-path/older-iteration closure. Replaced transient-isolation persistence with finite older-work exit and eventual permanent isolation. Made gate protection an execution-level conformance obligation, restricted standard partial units to singletons and excluded unqualified standalone NB units, and made fact-domain and witness availability coverage explicit. Corrected remaining closed-wait-cycle descriptions. The generic completion induction and full-fact adapter proofs are retained; the standard exposure discharge now states the additional control binding and explicit closure it consumes.

11 October 2026 — Shared non-blocking operational semantics restored. Added Core.NBDecisionSnapshot and Core.NBDecision for old-state positive FIFO, coupled and zero-depth outcomes, with silent failure and no annotation override; bound the retained preparation, decision and snooping consumers and the deferred NB record to these owners. Corrected Q's obsolete reconstruction description and remaining synchronization, ownership and observation-profile terminology. No retired certificate or strict-deadlock route is restored, and the constructive/completion theorem scope is unchanged. Existing revision history and cached table of contents are preserved.

11 October 2026 — Post-retirement consistency cleanup. Replaced stale certificate and strict-deadlock guarantee descriptions with the retained constructive-safety and required-workload completion scope. Updated the front matter, memory and pipeline guidance, Appendix F constraints and J/K/L/M/T summaries; restored the independent L.Dir snooping discharge entry, removed an orphaned caption and obsolete M.8 references. Existing revision entries and the cached table of contents are unchanged.

10 October 2026 — Coordinated certificate and strict-route retirement. Removed the old J certificate construction/coverage routes, old K strict wait-cycle/serial-reduction theorems and Appendix K-P. Retained constructive semantic safety, the full internal-fact adapter and K.CT workload completion. Separated shared action, cycle-carrier and synchronization semantics from reconstruction machinery; simplified Q to operational/accounting qualification plus its completion extension. Updated summaries, memory guidance, fairness/qualification triggers and mechanization targets. The retained completion guarantee is workload-relative and does not assert strict wait-cycle freedom after acceptance.

10 October 2026 — Constructive internal-fact correspondence and shared required order. Added Core.RegisteredFactBatch and its exact finite-prefix lemma, J.ConstructiveFactEvidence, J.ConstructiveFactCorr and the registered message/anchor specialization. These clauses derive completed internal/external fact correspondence from complete local realization and F-preserving carrier evidence, without importing certificate admission or K-facing reconstruction. Moved the unchanged ordinary generator inventory into J.RequiredOrderBasis and made the intrinsic history definition consume that shared basis. Updated the K.CT consumer, scope notes and obligations registry. Existing certificate and strict K/K-P routes remain pending coordinated retirement; no additional implementation class or completion guarantee is implied.

10 October 2026 — Shared-interface migration for workload completion. Common evidence integrity and finite-prefix history are owned by the Core; source L2 settling is separated from its KCut consequence. Safety agreement consumers use conditional finite-prefix agreement without importing serial dominance. Existing certificate elaboration qualification is retained explicitly as Q.CertificateCompatible. Memory-profile dispositions distinguish finite-history safety, required-workload progress and strict wait-cycle use. Guidance and responsibility records lead with completion. Old K/K-P and the pre-8 October revision record are retained.
10 October 2026 — Completion witness capacity clarification. B.CT.FIFO and K.CT.SerialWitness distinguish the capacity used for the legal source run from the guaranteed resource floor used by K.CT.Closure. Standard space-release edges use Pop(k−g) before Push(k), including when the source run uses the full reference capacity. Failure of the resulting closure or acyclicity check is inconclusive for completion. The strict routes are retained pending consumer migration.
10 October 2026 — Completion consolidation editorial correction. Promoted B.CT and Q.CT to independent sections and placed Q.CT after the existing Q.1/Q.2 material. Replaced K.CT.SyncRegion's duplicate obligations with an informative overview of the authoritative clauses and proof. Clarified the optional guaranteed floor and direct-reference witness alternatives, and named B.CT/Q.CT conformance in the guarantees summary. Restored the pre-8 October Audit 7 caveat verbatim and preserved all earlier revision history. Unreleased entries remain limited to document changes. Cached TOC and fields were not regenerated.
10 October 2026 — Normative completion consolidation. Added B.CT operational progress requirements and compatible profile declarations, Q.CTQualified component interfaces, and the K.CT standard placement/resource/residual/joint/synchronization and clock-refinement lemmas with proofs. The standard completion theorem is now stated and discharged wholly through definitions and proofs in this document. Distinguished generic mathematical validity, legal source evidence and implementation conformance; retained conditional extension interfaces and the existing strict K/K-P route. Removed development-package status references from the current completion text and unreleased revision entries.
10 October 2026 — Staged synchronization scope and placement clarification. Distinguished fully drained two-party region boundaries, direct joint B(0) components, delayed future-region restoration and concrete implementation conformance. Moved the shared Core evidence block after the complete PolicyS remark and engineering note.
10 October 2026 — Paired synchronization, group-serial witness and workflow corrections. Added K.CT.SyncRegion with participant reach, joint service, explicit future-epoch capacity withdrawal and noncircular restoration, plus same-edge fence constraints. Added qualified group-serial legal Policy-L preparation for same-owner atomic stages without changing ordinary Policy S. The serial workflow now explicitly requires bound or conservatively lifted capacities and states timing-dependent agreement limits. Updated Formal Guarantees at a Glance; closed occurrence-to-historical-availability wording without weakening registered-result timing; moved shared Core interfaces after PolicyS, numbered J.2c, and normalized registry style/line breaks. Strict K/K-P machinery and concrete implementation conformance obligations remain.
10 October 2026 — Completion G1 interface extraction and serial-witness default. Added route-neutral Core.EvidenceHdr, acceptance/drained diagnostics and finite-prefix provenance/agreement; added J.InternalFactCorr and its conditional value-transfer adapters; made conservative finite Core.PolicyS execution the default K.CT witness without complete-run monitors. K.CT now consumes these interfaces rather than K.P1/CV or the legacy diagnostic wrapper. Retargeted the Core.IssueFair and boundary-service registry entries explicitly to completion consumers, while distinguishing finite witness legality from eventual-issue fairness. K.CertHdr remains a legacy extension with unchanged triggered obligations. The strict K/K-P route remains intact; shared safety-package dependencies and

feature qualifications are not declared closed. The migration target is the useful Policy-S workload workflow, with the different completion guarantee explicitly retained.
10 October 2026 — Ordered automatic-flush progress and workload completion. Added B.OF as a substantive conformance profile: ordered iterations, completion/local service, drainage without mandatory bubble compression, upstream-only blocking, legal entry and request preparation; derived B.OF.Target under explicit closure and preparation hypotheses. Added K.CT with finite prerequisite-unit induction, internal correspondence and agreement, exposure/service contracts, optional capacity lift, residual/joint qualification and deployment limits. Scoped the existing K standing assumptions to the strict route and linked the 7 October end-of-test clarification to K.CT. Existing K.1 onward and K-P machinery is retained pending production binding and consumer audit.
6 October 2026 — Audit 21 architecture-summary clarification.
Repaired and rechecked F21-1–F21-3: M.1 distinguishes informative overview text from the explicitly normative M.7a–M.7c discharge/packaging clauses; M.3 includes the LE1-certified current-open-Horizon L3 suffix while retaining exact-prefix preservation; R.5 removes the superseded single-domain reset/terminal restriction and refers to the existing prospective SFControl/Handoff/Response split. Targeted checks used the controlling M.7 clauses, J.LocalGWRExtCert/J.GWR-CompExt, and K.PolicySCompareDomains/K.ResetDisposition/K.TerminalDisposition. Normative theorem premises and historical revision entries are unchanged. Final audit closure remains open: declaration-ledger reconciliation, missing Audit 5/6 review records, Audit 3 obligations, paused Audit 2/combined-TB work, and deferred Audit 18 are not discharged by these wording repairs.
5 October 2026 — Audit 19 source-facing guidance clarification.
Repaired and rechecked F19-1–F19-4: skidbuffer configuration now retains boundary/capacity/handshake qualification; the preserved-loop signal example no longer treats added states as new anchors; Appendix F includes qualified constructive NB handling; and Appendix H states the actual shared-memory dependency/value-flow requirement. Added narrow throughput and one-way-handshake qualifications. M.8 now distinguishes selected RTL B1-run safety (including modeled terminal, BRM2 and remaining correspondence/property premises), source covering-run evidence, and constructive/class coverage with independent source safety. Source determinism is not promoted to RefProjCover, nor is a passing run promoted to universal safety. Targeted review covered the cited Core/R/E, Appendix B/F/Q, ordinary/elaborated constructive NB, memory and M.7c/M.8 safety interfaces. Normative theorem premises are unchanged; concrete tool, library and implementation qualification remains open. Audit 18 and the K terminal/response interaction remain deferred.
5 October 2026 — Consolidated witness fairness and FIFO clarification.
Repaired F17-1–F17-4 by removing blanket B1/K/fairness implications from selected-run wrapper use, binding source throttling and NB-CF selection to the unchanged legal source/protocol model, requiring complete L.Dir evidence from monitor/static/model-checking alternatives, and qualifying snoop omission and stress-test failure attribution. Linked F17-4's fairness wording with F2-4: Appendix N examples now distinguish B2/B3 antecedents from stronger declared environment-service contracts; formal B2/B3 are unchanged. Clarified the residual required-order summary's FIFO consumption pairing within each reset epoch without renumbering Core.17 identities, completing the F12-1 explanatory recheck. Targeted review covered W1–W5, L.2/L.3/L.5, WC, M.7a L.Dir and M.7c witness variants; checked examples distinguish equal timing from payload agreement, legal outcomes from imposed failures, and discarded pre-reset messages from post-reset consumption. Findings are repaired and rechecked at the rules-interface level; concrete wrapper/implementation qualification remains open. The paused K investigations, general CS2 deferral and live-NB origin limitation are not discharged.
5 October 2026 — Audit 13 settling endpoint clarification.
Follow-up to F13-1: defined a settling-relation fixed point as an endpoint with no enabled step, including redundant evaluations, and distinguished it from satisfaction of the node equations. Retained QS3 finite

legal evaluation availability and made its equation-solution-to-operational-fixed-point direction explicit. Aligned QSync source/post and KObs proofs, R2C redundant-evaluation commentary and M.7a evidence records with unchanged Core.4 quiescence. Rechecked three small relations: a stalled evaluator lacks a settling path; an evaluator with a self-loop at the solution fails endpoint qualification; an evaluator permitting redundant work only before the solution admits a finite path to a quiescent endpoint. These are targeted semantic checks, not implementation-conformance evidence. No fairness, all-schedules termination or global redefinition of quiescence is introduced. F13-1's availability and endpoint clarifications are repaired and rechecked; Audit 13 remains provisionally accepted with concrete conformance and previously recorded interface obligations pending. Historical Appendix U entries and the TOC are unchanged.

5 October 2026 — Audit 13 operational settling availability clarification.

Clarified QS3 that realization includes availability of a finite legal topological evaluation in the actual settling relation under the frozen frame, not only accounting of existing steps and mathematical existence of an equation solution. Aligned Core.QSync-Ref, Core.QSync-Post, KSQ3/Core.KObsEval-Ref and their M.7a discharge entries. F13-1 is repaired and rechecked: the stalled evaluator for x = 1, y = x from (0,0) fails availability, while a legal topological evaluator reaches (1,1) from every Boolean origin. Redundant evaluations remain permitted; no fairness or all-schedules termination requirement is added. J.SourceSettleRealization already requires actual legal evaluation paths and is unchanged, as are the safety-only RTL carrier tier, cross-cycle progress obligations and historical Appendix U entries. Audit 13 remains provisionally accepted, with representative implementation/monitor conformance and the recorded interface-specific obligations still pending.

5 October 2026 — Audit 12 reset-epoch FIFO pairing clarification.

Clarified Appendix G's causal-dependency explanation that k-th Push-to-k-th Pop consumption pairing is within the channel's reset epoch. An affecting RESET may discard unmatched stored messages without fabricating Pop events. The change does not renumber Core.17 cross-model endpoint occurrences, change FIFO/reset semantics, or add canonical-order generators. F12-1 is repaired and rechecked against the no-reset case and the Push x / RESET / Push y / Pop y example; the first lifetime Pop in that example consumes the second lifetime Push. Core.17 matching, E4, RW1–RW5, the TOC and historical Appendix U entries are unchanged. Audit 12 remains provisionally accepted pending concrete carrier/generator evidence; general certificate CS2 insertion remains deferred.

4 October 2026 — Elaborated locality accounting-support clarification.

Clarified EL1 that overlap between Q.1-qualified AccountSupport_E sets does not itself constitute excluded shared capacity. Fixed multi-effect FIFO representations retain separate storage ownership, declared physical capacity, admitted primitive/template laws and full applicable safety qualification. No capacity pool, dynamic allocation, extra storage or admission permission is introduced. The distinction follows Q.1's existing accounting-support qualification; the channel map does not determine transfer erasure.

Audit 7 followup. The paired join–move–fork paper witness supplies concrete accounting, payload/order, atomic-group, local-process, settling and carrier bindings; targeted review checked its FIFO supply/capacity paths and later-Horizon preparation discipline. The authored finite register model checks are supporting evidence, not a source-phase mechanization or synthesis-flow qualification. O7-1 has feature-bearing paper evidence available for owning-audit review; O7-2 representative implementation/evidence production and O7-3 dependency reviews, including Audit 8 extension base bindings, remain open. Audit 7 is not closed.

4 October 2026 — Audit 7 constructive base and locality clarifications.

Added explicit related-base history equality to J.ConstructiveEdgeInduction and made its producer explicit in the elaborated composition proof. Added the ELM/ELA side-condition discharge entry in M.7a and corrected the M.8 comparison and elaboration guidance. C0 and the ordinary composition proof

already provide equal base histories. The terminal-base countermodel now fails the stated base premise; the checked C0, ordinary and elaborated base producers meet that premise. These conservative changes do not enlarge the supported profiles or introduce a design-specific global correspondence obligation.
Audit status and pending work. F7-1 and F7-2 are repaired with targeted base and traceability rechecks; Audit 7 remains provisionally accepted at the profile level, not closed. O7-1 (a complete feature-bearing elaboration qualification witness, especially join/fork), O7-2 (representative implementation qualification and evidence production), and O7-3 (owning-audit dependency rechecks, including A/RB/RC base bindings in Audit 8) remain open. Audit 3 O-3.4, O-3.5 and O-3.6 remain open; Audit 4 general CS2 insertion remains deferred. The paused Audit 2 and combined DUT/testbench investigation is not resumed. Historical Appendix U entries are unchanged.

2 October 2026 — Elaborated constructive locality for separately owned templates.
Added J.ElaboratedLocalityProfile and J.ElaboratedConstructiveComposition (ELM/ELA) alongside unchanged OM/OA. The restricted extension covers qualified MoveAB, request-fed JoinFire and FIFO-fed ForkABC primitives plus existing ordinary endpoints/rendezvous. Added explicit enlarged LP3/LP4 completion-packet evidence, a closed normative generator inventory, owned FIFO residues, exact process-facing attribution, ordinary frozen NB and the qualified positive direct-join NB law, gate-source/phase restrictions, and proofs reusing J.CycleRankOrderWF/J.StandardCycleRank. The local arguments derive constructive boundary realization, preparation and canonical extension; concrete primitive/process refinement, complete settling/matching, Q.SafetyQualified accounting/projection, environment/carrier and source-property evidence remain required. Updated constructive route guidance and Q.1 cross-references. No shared-capacity result, arbitrary rendezvous-compound qualification, general CS2 insertion or constructive-to-K interface is asserted.

1 October 2026 — Audit 4 certificate construction scope restriction.
Restricted the supported normal-compositional certificate route to a complete consumed proof-unit inventory with CS1 action-backed support only and no CS2 order-only units. J.LocalGWR owns the restriction; J.SupportInsertionExists now proves only the empty-CS2 case, and the construction, extension, uniform-coverage, route guidance, M.7a, cost-accounting and mechanization references are aligned. Coverage applications, including the current certificate-fed K route, require the restriction throughout their declared package domain. CS2 definitions and independently validated J.GWR-Direct placement remain available; genuine operational elaboration actions are not excluded merely because they are internal or erased from outer observations. NB remains subject to its distinct existing qualification requirements. This addresses Audit 4 F4-1 by intentionally narrowing the supported route, not by proving general CS2 insertion. Compatibility of mixed required-order/anchor paths with canonical action order, atomic contraction and exact prefix extension remains deferred.

1 October 2026 — Audit 3 in-flight negative-evidence clarifications.
Clarified the existing K.InFlightQual IS2 obligation at path level: omitted relation behavior and checker-pruned states, actions, edges or branches must not permit a WaitRoots_D change at any intermediate point, with the existing QB1 exact-occurrence or QB2 complete-fiber binding. Aligned K.InFlightNegativeEvidenceHeredity with that obligation while retaining the same relation, search omissions and start binding, and added references in the exploration-certificate, M.7a and mechanization text. Core.SyncInFlightOpportunityQual now makes explicit that omitting an unqualified edge does not preserve exact negatives without the required IQ5 evidence. Replaced the undefined optional Q.InFlightActionEffectComplete name with a reference to template-local contributions to K.InFlightRelationComplete.

23 October 2026 — Medium repair R1 replacement. This separate branch starts from the frozen audit baseline (SHA-256 b1e5121bf4a18225ae45bc3beb55115fbcaf8e8dd14395e16cbbe0d099505e24). It supersedes the earlier R1 carrier-strengthening draft: the optional cycle-ideal safety lift and its exclusive frame/designation, partial-cycle completion and cycle-carrier machinery are removed. Independent ConstructiveOriginCarrier and exact Batch routes remain; Core.QSync retains occurrence-bound finite settling. CompleteCycle, SilentCycle and shared persistence/reset text are preserved and placed with CycleResult. Ideal-prefix closure remains for its independent selected-source-history use. This removes the disputed route rather than proving its former carrier claim. Handshake-semantics and physical sync-protocol MEDIUM repairs remain pending. LOW findings are not otherwise repaired. The baseline audit and G4/G5 remain unchanged.
23 October 2026 — Medium repair R2 (sync protocol and residual pruning). Built from R1 replacement with the cycle-ideal route removed. Deleted the orphaned closing carrier remark and the residual ideal-lift sentence in Appendix R. Core.SyncProtocol now binds physical issue, transaction identity, held ready/valid offers, mandatory eligible-edge completion, reset disposition, unary scope and protection of sync-controlled pins. Direct consumers and the one-cycle TB recipe are reconciled; synchronization is excluded from discretionary L3/B2 selection. The process-facing fence permits same-edge predecessors and later buffered external delivery. This staged branch still retains the unrepaired message-transfer selection semantics: AUD-HANDSHAKE-SEMANTICS, including its message B2/B3 and arbitration reconciliation, is the next repair stage. This is a textual repair with targeted regressions, not a final proof-audit discharge. The frozen audit and G4/G5 remain unchanged.
23 October 2026 — Medium repair R3 (mandatory message handshake). Built from R2. Core.TransferLaw binds all declared physical transfer interfaces, complete settled commit sets, fixed event identity, old-state legality, reset and atomic coherence with no post-handshake veto. Core.SelectLatitude now retains legal issue and genuinely abstract-action latitude only. C1–C4 and join/staging consumers require every forced effect. B2 now concerns independently established pre-grant eligibility; uncoupled primitive B3 discharge and subsequent commitment use the transfer law, with continuing-edge and reset scope explicit. Coupled exposure and internal service remain separate. R2 sync rules and the cycle-ideal removal are preserved. Historical skipped-handshake source records require revalidation and are not revised-model evidence. This is a separate repair branch, not a discharge of the frozen audit or G4/G5. Integrated review and implementation-specific qualification remain necessary.
23 October 2026 — R4 integrated repair review. Reviewed the R1–R3 changes across the affected constructive safety and completion dependencies, separately from the paused baseline audit. Made forced-transfer completeness explicit in the ordinary/elaborated local-evidence and composition contracts, corrected remaining K service wording that suggested selection after an eligible handshake, and clarified that internal pending state cannot replace an actual gated signal without a valid transfer-equivalence mapping. The cycle-ideal route remains removed, the sync protocol and no-late-veto transfer law remain, and P1228 is preserved. Conditional proof structure is retained; concrete instance qualification and historical-evidence reuse remain separate obligations. This focused repair review does not complete the full baseline audit or close G4/G5.
23 October 2026 — R5 terminology and gated-request clarification. Built from R4. Aligned the remaining K/Q service terminology and the B2/SelectLatitude titles with mandatory physical commitment and pre-grant arbitration. Core.10 now states the declared-complementary-gate and gate-before-issue request bindings positively, retaining exact transfer equivalence, persistence and noncircular staging qualifications; TL2 points to those forms. Removed the repeated service explanation from the seven reader-facing paragraphs and shortened the main-body sync recipe, with pointers to the formal Core protocol. The medium-repair semantics and cycle-ideal removal remain; the frozen baseline audit and G4/G5 are unchanged.